# Universal Serial Bus (USB)

This chapter describes the USB of the device.

**Topic** **Page**

## Introduction

The USB controller provides a low-cost connectivity solution for numerous consumer portable devices by providing a mechanism for data transfer between USB devices with a line/bus speed up to 480 Mbps. The device USB subsystem has two independent USB 2.0 Modules built around two OTG controllers. The OTG supplement feature, the support for a dynamic role change, is also supported. Each port has the support for a dual-role feature allowing for additional versatility enabling operation capability as a host or peripheral. Both ports have identical capabilities and operate independent of each other.

Each USB controller is built around the Mentor USB OTG controller (musbmhdrc) and TI PHY. Each USB controller has user configurable 32K Bytes of Endpoint FIFO and has the support for 15 ‘Transmit’ endpoints and 15 ‘Receive’ endpoints in addition to Endpoint 0. The USB subsystem makes use of the CPPI 4.1 DMA for accelerating data movement via a dedicated DMA hardware.

The two USB modules share the CPPI DMA controller and accompanying Queue Manager, Interrupt Pacer, Power Management module, and PHY/UTMI clock.

Within the descriptions of the USB subsystem that would follow, the term USB controller or USB PHY is used to mean/refer to any of the two USB controllers or PHYs existing within the USB subsystem. The term USB module is used to mean/refer to any of the two USB modules. USB0 is used to refer to one of the USB modules and USB1 is used to refer to the other USB module.

### *16.0.4 Acronyms, Abbreviations, and Definitions*

| | |
|---|---|
| AHB | Advanced High-performance Bus |
| CBA | Common Bus Architecture |
| CDC | Change Data Capture |
| CPPI | Communications Port Programming Interface |
| CPU | Central Processing Unit |
| DFT | Design for Test |
| DMA | Direct Memory Access |
| DV | Design Verification |
| EOI | End of Interrupt |
| EOP | End of Packet |
| FIFO | First-In First-Out |
| FS | Full-Speed USB data rate |
| HNP | Host Negotiation |
| HS | High-Speed USB data rate |
| INTD | Interrupt Distributor |
| IP | Intellectual Property |
| ISO | Isochronous transfer type |
| LS | Low-Speed USB data rate |
| MHz | Megahertz |
| MOP | Middle of Packet |
| OCP | Open Core Protocol |
| OCP HD | OCP High Performance |
| OCP MMR | OCP Memory Mapped Registers |
| OTG | On-The-Go |
| PDR | Physical Design Requirements |
| PHY | Physical Layer Device |
| PPU | Packet Processing Unit |
| RAM | Random Access Memory |
| RNDIS | Remote Network Driver Interface Specification |

| | |
|---|---|
| RX | Receive |
| SCR | Switched Central Resource |
| SOC | System On a Chip |
| SOP | Start of Packet |
| SRP | Session Resume |
| TX | Transmit |
| USB | Universal Serial Bus |
| USB0 | One of the two USB 2.0 Compliant USB Module |
| USB1 | One of the two USB2.0 Compliant USB Module |
| USBSS | USB Subsystem (contains USB0 and USB1) |
| UTMI | USB 2.0 Transceiver Macrocell Interface |
| XDMA | Transfer DMA (DMA other than CPPI DMA used within the Controller) |

### USB Features

The main features of the USB subsystem are:

- Contains 2 usb20otg_f controller modules with the following features:
  - Built around the Mentor USB 2.0 OTG core (musbmhdrc)
  - Supports USB 2.0 peripheral at speeds HS (480 Mb/s) and FS (12 Mb/s)
  - Supports USB 2.0 host or OTG at speeds HS (480 Mb/s), FS (12 Mb/s), and LS (1.5 Mb/s)
  - Supports all modes of transfers (control, bulk, interrupt, and isochronous)
  - Supports high bandwidth ISO mode
  - Supports 16 Transmit (TX) and 16 Receive (RX) endpoints including endpoint 0
  - Supports USB OTG extensions for Session Resume (SRP) and Host Negotiation (HNP)
  - Includes a 32K endpoint FIFO RAM, and supports programmable FIFO sizes
  - Includes RNDIS mode for accelerating RNDIS type protocols using short packet termination over USB
  - Includes CDC Linux mode for accelerating CDC type protocols using short packet termination over USB
  - Includes an RNDIS like mode for terminating RNDIS type protocols without using short packet termination for support of MSC applications
- Includes two USB2.0 OTG PHYs
- Interfaces to the CPU via 3 OCP interfaces:
  - Master OCP HP interface for the DMA
  - Master OCP HP interface for the Queue manager
  - Slave OCP MMR interface
- Includes a CPPI 4.1 compliant DMA controller sub-module with 30 RX and 30 TX simultaneous data connections
- Includes a CPPI 4.1 DMA scheduler
- DMA supports CPPI host descriptor formats
- DMA supports stall on buffer starvation
- Supports data buffer sizes up to 4M bytes
- CPPI FIFO interface per TX/RX endpoint
- Provides a CPPI Queue Manager module with 92 queues for queuing/de-queuing packets.
- DMA pacing logic for interrupts
- Loopback MGC test using the UTMI interfaces

### 16.0.5 Unsupported USB OTG and PHY Features

USBOTG and USB charger-detect circuitry is not supported.

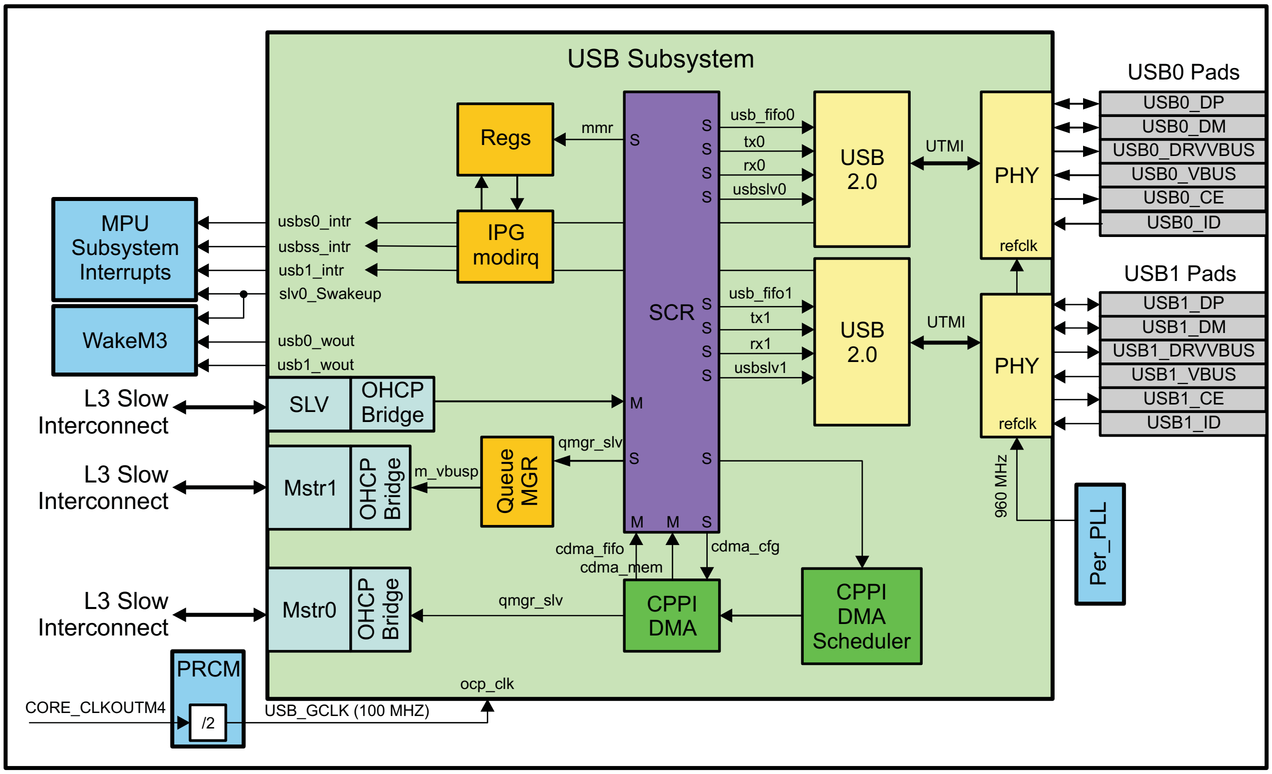
USB 2.0 ECN: Link Power Management (LPM) is not supported.

## 16.1 Integration

This device implements the USB2.0 OTG dual port module and PHY for interfacing to USB as a peripheral or host.

Figure 16-1 shows the integration of the USB module on this device.

**Figure 16-1. USB Integration**




### 16.1.1 USB Connectivity Attributes

The USB module itself has a very large number of interrupt outputs. For ease of integration, these outputs are all routed to a pair of interrupt aggregators. These aggregator blocks generate a single interrupt on the first occurrence of any interrupt from the USB or DMA logic of the module.

**Table 16-1. USB Connectivity Attributes**

| Attributes | Type |
|---|---|
| Power domain | Peripheral Domain |
| Clock domain | PD_PER_L3S_GCLK (OCP)<br>CLKDCOLDO (PHY) |
| Reset signals | DEF_DOM_RST_N<br>USB_POR_N |
| Idle/Wakeup signals | Smart idle<br>Wakeup<br>Standby |

**Table 16-1. USB Connectivity Attributes (continued)**

| Attributes | Type |
|---|---|
| Interrupt request | 4 interrupts<br>usbss (USBSSINT) to MPU subsystem<br>usb0 (USBINT0) to MPU subsystem<br>usb1 (USBINT1) to MPU subsystem<br>slv0p_Swakeup (USBWAKEUP) to MPU subsystem and WakeM3<br>2 Wakeup Events to WakeM3<br>usb0_wuout<br>usb1_wuout |
| DMA request | None |
| Physical address | L3 Slow slave port |

### 16.1.2 USB Clock and Reset Management

Each USB controller has a PHY module that generates the UTMI clock. The UTMI clock is fixed in the UTMI specification for an 8-bit interface at 60 MHz (480 Mb/s). The PHYs require a low-jitter 960-MHz source clock.

**Table 16-2. USB Clock Signals**

| Clock Signal | Max Freq | Reference / Source | Comments |
|---|---|---|---|
| ocp_clk<br>OCP / Functional clock | 100 MHz | CORE_CLKOUTM4 / 2 | pd_per_l3s_gclk<br>From PRCM |
| phy0_other_refclk960m<br>Phy reference clock | 960 MHz | CLKDCOLDO | clkdcoldo_po<br>From Per PLL |
| phy1_other_refclk960m<br>Phy reference clock | 960 MHz | CLKDCOLDO | clkdcoldo_po<br>From Per PLL |

### 16.1.3 USB Pin List

The USB external interface signals are shown in Table 16-3.

**Table 16-3. USB Pin List**

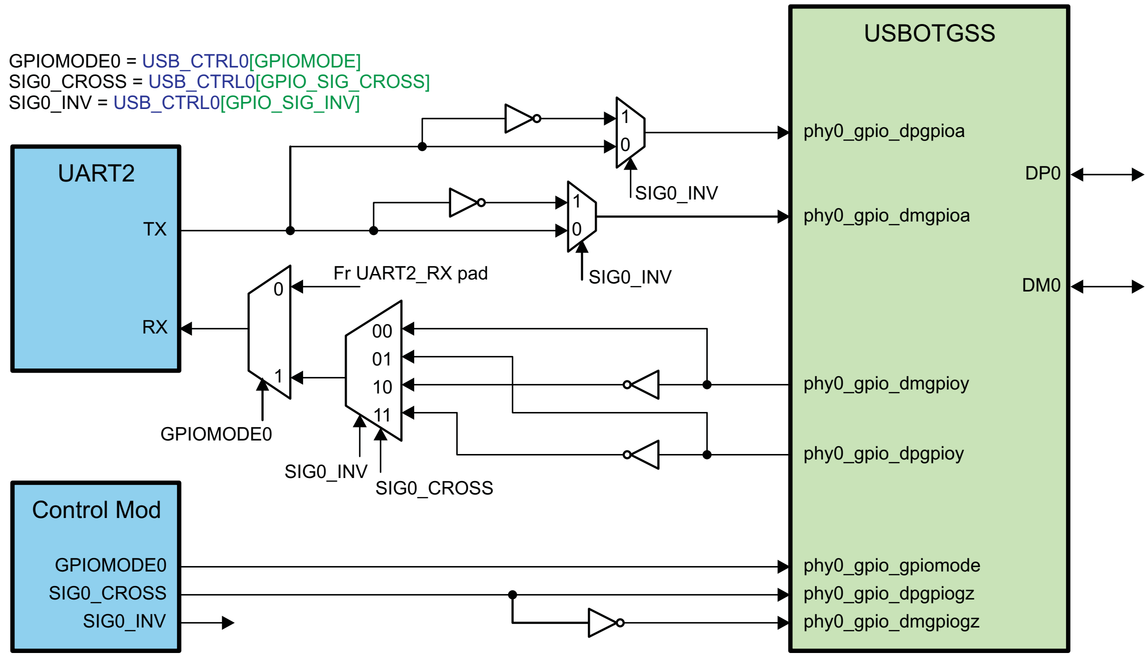
| Pin | Type | | Description |
|---|---|---|---|
| USBx_DP(1) | USB | Analog I/O | USBx data differential pair |
| USBx_DM(1) | GPIO | Digital I/O | |
| USBx_DRVVBUS | Digital output | | USBx VBUS supply control |
| USBx_VBUS | Analog input | | USBx VBUS (input only for voltage sensing) |
| USBx_ID | Analog input | | USBx OTG identification |
| USBx_CE | Digital output | | USBx Phy charge enable |

(1) Analog in USB mode; CMOS in GPIO mode.

### 16.1.4 USB GPIO Details

The USB module supports configuration of the DP and DM pins as pass-through GPIOs. In this device, the GPIO mode is used to provide UART over USB interface functionality. Chip level logic allows connection of the UART TX/RX data signals to either DP or DM in either normal or inverted state as shown in Figure 16-2. The diagram shows the UART2 / USB0 port implementation. This logic is also replicated for the UART3 / USB1 ports.

**Figure 16-2. USB GPIO Integration**




## 16.1.5 USB Unbonded PHY Pads

The USBOTGSS includes two USB PHY modules. On packages where only 1 PHY is bonded out to pins (e.g. 13x13 package), the following procedures must be followed in order to ensure that the unbonded PHY pads do not cause issues with USBOTGSS operation:

- The USB Controller corresponding to the unbonded PHY must be placed in Host Mode by setting Bit2 of the USB Core DEVCTL register.
- The unbonded PHY must be placed in the SUSPEND state by setting Bits[1:0] of the USB Core POWER register.
- The Control Module USB_CTRLx register corresponding to the unbonded PHY must have the following bits programmed as shown:
  - CHGDET_DIS = 1
  - CM_PWRDN = 1
  - GPIOMODE = 0

## 16.2 Functional Description

The USB controller within the device supports 15 Transmit endpoints and 15 Receive endpoints in addition to Endpoint 0. (The use of these endpoints for IN and OUT transactions depends on whether the USB controller is being used as a device/peripheral or as a host. When used as a peripheral, IN transactions are processed through TX endpoints and OUT transactions are processed through Rx endpoints. When used as a host, IN transactions is processed through Rx endpoints and OUT transactions is processed through TX endpoints.)

These additional endpoints can be individually configured in software to handle either Bulk transfers (which also allows them to handle Interrupt transfers), Isochronous transfers or Control transfers. Further, the endpoints can also be allocated to different target device functions on the fly – maximizing the number of devices that can be simultaneously supported.

Each endpoint requires a chunk of memory allocated within the FIFO to be associated with it. The USB module has a single block of FIFO RAM (32 Kbytes in size) to be shared by all endpoints. The FIFO for Endpoint 0 is required to be 64 bytes deep and will buffer 1 packet. The first 64 Bytes of the FIFO RAM is reserved by hardware for Endpoint 0 usage and for this reason user software does not need (nor has the capability) to allocate FIFO for Endpoint 0. The rest of the FIFO RAM is user configurable with regard to the other endpoint FIFOs, which may be from 8 to 8192 bytes in size and can buffer either 1 or 2 packets.

Separate FIFOs may be associated with each endpoint: alternatively a TX endpoint and the Rx endpoint with the same Endpoint number can be configured to use the same FIFO, for example to reduce the size of RAM block needed, provided they can never be active at the same time.

The role (host or device) that the USB controller assumes is chosen by user firmware programming the respective USB controller MODE register defined within the USB subsystem space. Note that some USB pins have not been bonded out and the functions of these pins are controlled by user software via dedicated registers.

The user has access to the controller through the OCP slave interface via the CPU. The user can process USB transactions entirely from the CPU or can also use the DMA to perform data transfer. The CPPI DMA can be used to service Endpoints 1 to 15 not Endpoint 0. CPU access method is used to service Endpoint 0 transactions.

### *16.2.1 VBUS Voltage Sourcing Control*

When any of the USB controllers assumes the role of a host, the USB is required to supply a 5V power source to an attached device through its VBUS line. In order to achieve this task, the USB controller requires the use of an external power logic (or charge pump) capable of sourcing 5V power. A USB_DRVVBUS is used as a control signal to enable/disable this external power logic to either source or disable power on the VBUS line. The control on the USB_DRVVBUS is automatic and is handled by the USB controller. The control should be transparent to the user so long as the proper hardware connection and software initialization are in place. The USB controller drives the USB_DRVVBUS signal high when it assumes the role of a host while the controller is in session. When assuming the role of a device, the controller drives the USB_DRVVBUS signal low disabling the external charge pump/power logic; hence, no power is driven on the VBUS line (in this case, power is expected to be sourced by the external host).

Note that both USBs are self-powered and the device does not rely on the voltage on the VBUS line sourced by an external host for controller operation when assuming the role of a device. The power on the VBUS is used to identify the presence of a Host. It is also used to power up the pull-up on the D+ line. The USB PHY would continually monitor the voltage on the VBUS and report the status to USB controller.

### *16.2.2 Pullup/PullDown Resistors*

Since the USB controllers are dual role controllers, capable of assuming a role of a host or device, the necessary required pull-up/pull-down resistors can not exist external to the device. These pull-up/pull-down resistors exist internal to the device, within the PHY to be more specific, and are enabled and disabled based on the role the controller assumes allowing dynamic hardware configuration.

When assuming the role of a host, the data lines are pulled low by the PHY enabling the internal 15KOhms resistors. When assuming the role of a device the required 1.5Kohm pull-up resistor on the D+ line is enabled automatically to signify the USB capability to the external host as a FS device (HS operation is negotiated during reset bus condition).

### 16.2.3 Role Assuming Method

For an OTG controller, the usual method followed by the controller to assume the role of a host or a device is governed by the state of the ID pin which in turn is controlled by the USB cable connector type. The device has bonded out these ID pins and allows the control to be handled directly from the connector, i.e. the USB controller would assume the role based on the cable end inserted to the mini or micro A/B connector. An alternate method where the ID pin signal is bypassed and the firmware controls the role the controller assumes exists via register access allowing additional control for systems that do not require the use of the USB ID pin. In this case the USBnMODE [Bit7=IDDIG_MUX] needs to be programmed with a value of 1 so that Register programming will be used.

Two registers, USB0 Mode Register at offset 10E8h and USB1 Mode Register at offset 18E8h, are used for a user to select the role the USB controller assumes. The user is required to program the corresponding register prior to the USB controller is in session.

### 16.2.4 Clock, PLL, and PHY Initialization

Prior to configuring the USB Module Registers, the USB SubSystem and PHY are required to be released from reset, enable interconnects and controllers clocks as well as configure the USB PHY with appropriate setting. Not all registers related for this task are captured within this document. Please refer to the PRCM for clocking and Control Module for PHY configuration registers definitions.

### 16.2.5 Indexed and Non-Indexed Register Spaces

The USB controller provides two mechanisms for accessing endpoint control and status registers; Indexed and Non-Indexed method.

**Indexed Endpoint Control/Status Register Space**. This register space can be thought of as a region populated with Proxy Registers. This register space is a memory-mapped region at offset 1410h to 141Fh for USB0 and 1C10h to 1C1Fh for USB1. The endpoint register space mapped at this region is selected by programming the corresponding INDEX register (@ offsets 140Eh and 1C0Eh for USB0 and USB1 respectively) of the controller.

By programming the INDEX register with the corresponding endpoint number, the control and status register corresponding to that particular endpoint is accessible from this Indexed Region. In other words, Index Register region behaves as a proxy to access a selected endpoint registers.

**Non-indexed Endpoint Control/Status Register Space**. These regions are dedicated endpoint registers memory-mapped residing at offsets 1500h to 16FFh for USB0 and 1D00h to 1EFFh for USB1. Registers at offset 1500h to 150Fh belong to Endpoint 0; registers at offset 1510h to 151Fh belong to Endpoint 1… and lastly registers at offset 16F0h to 16FFh belong to Endpoint 15 for USB0. Similarly registers at offset 1D00h to 1E0Fh belong to Endpoint 0; registers at offset 1D10h to 1D1Fh belong to Endpoint 1… and finally registers at offset 1DF0h to 1DFFh belong to Endpoint 15 for USB1.

This allows the user/firmware to access an endpoint register region directly from its unique space, as specified within a non-indexed region or from a proxy space by programming the corresponding Index register.

Note: The control and status registers/regions apply to both Tx and Rx of a given Endpoint. That is, Endpoint 1 Tx and Endpoint 1 Rx registers occupy the same register space.

For detailed information about the USB controller registers, see Section 16.4.

### 16.2.6 Dynamic FIFO Sizing

Each USB module supports a total of 32K bytes of FIFO RAM to dynamically allocate FIFO to all endpoints in use. Each endpoint requires a FIFO to be associated with it. There is one set of register per endpoint available for FIFO allocation, excluding endpoint 0. FIFO configuration registers are Indexed registers and cannot be accessed directly; configuration takes place via accessing proxy registers. The INDEX register at addresses 140Eh or 1C0Eh must be set to the appropriate endpoint. In other words, the Endpoint FIFO configuration registers do not have non-indexed regions.

The allocation of FIFO space to the different endpoints requires programming for each Tx and Rx endpoint with the following information:

- The start address of the FIFO within the RAM block
- The maximum size of packet to be supported
- Whether double-buffering is required

(These last two together define the amount of space that needs to be allocated to the FIFO.)

It is the responsibility of the firmware to ensure that all the Tx and Rx endpoints (other than endpoint 0) that are active in the current USB configuration have a block of FIFO RAM allocated for their use. The maximum FIFO size for endpoint 0 required is 64 bytes deep and which is large enough to buffer one packet. For this reason, the first 64 bytes of FIFO memory is reserved for Endpoint 0 usage and this resource allocation is implied to be available at all times and no FIFO allocation is required for Endpoint 0. For endpoints other than endpoint 0, the FIFO RAM interface is configurable and must have a minimum size of 8 bytes and should be capable of buffering either 1 or 2 packets. Separate FIFOs may be associated with each endpoint: alternatively a Tx endpoint and the Rx endpoint with the same Endpoint number can be configured to use the same FIFO, to reduce the size of RAM block needed, provided they can never be active at the same time.

**NOTE:** The option of dynamically setting FIFO sizes only applies to Endpoints 1-15. Endpoint 0 FIFO has a fixed size (64 bytes) and a fixed location which is the first 64 Bytes of FIFO (FIFO addresses 0-63).

### *16.2.7 USB Controller Host and Peripheral Modes Operation*

The two USB modules can be used in a range of different environments. They can be used as either a high-speed or a full-speed USB peripheral device attached to a conventional USB host (such as a PC) in a point-to-point type of arrangement. As a host, the USB modules can also be used with another peripheral device of any speed (high-,full-, or low-speed) in a point-to-point arrangement or can be used as the host to a range of peripheral devices in a multi-point setup; one-to-many via hub. Note that the role each USB controller is assuming is independent of each other. Two options (h/w or s/w) exist for a USB role assumption/configuration.

For a USB Peripheral configuration the user has the option of using the cable end to select the role by attaching the mini or micro b side of the cable (can consider this as a h/w option) or use the optional method where the firmware is required to program the respective USB Mode Register IDDIG bit field with a value of ‘1’ prior to the USB controller goes into session (can consider this as a s/w option).

Similarly, for a USB host configuration the user has the option of using the cable end to select the role by attaching the mini or micro a side of the cable (can consider this as a h/w option) or use the optional method where the firmware is required to program the respective USB Mode Register IDDIG bit field with a value of ‘0’ value prior to the USB controller going into session (can consider this as a s/w option).

When using the s/w option, the USB ID pin state is ignored/bypassed (this is a similar configuration where an ID pin does not exist) and the USB2.0 controller role adaptation of a host or peripheral (device) is dependent upon the state of the respective USB Mode Register IDDIG field. If the IDDIG field is programmed by the user with a value of ‘1’ prior to the USB going into session, it would assume the role of a peripheral/device. However, if the IDDIG field is programmed with the value of ‘0’, the USB controller will assume the role of a host. What this means is that the USB cable end, connector, will not be able to control the role of the OTG controller and the user needs to be aware of the firmware program setting prior to performing a USB connection.

The procedure for the USB2.0 OTG controller determining its operating modes (usb controller assuming a role of a host or a peripheral) starts when the USB 2.0 controller goes into a session. The USB 2.0 controller is in session when either it senses a voltage (>= 4.4V) on the USBx_VBUSIN pin and the controller sets its DEVCTL[SESSION] bit field or when the firmware sets the DEVCTL[SESSION] bit; assuming that it will operating as a host.

When the DEVCTL[SESSION] bit is set, the controller will start sensing the state of the Iddig signal, which in turn is controlled either by the USBx_ID pin (h/w option) or by the IDDIG bit field of the MODE Register (s/w option). If the state of the iddig signal is found to be low (IDDIG is programmed with ‘0’) then the USB2.0 controller will assume the role of a host and when it finds the iddig signal to be high (IDDIG is programmed with ‘1’) then it will assume the role of a device. Note that iddig is an internal signal that could be driven to high or low via firmware from dedicated registers.

Upon the USB controller determining its role as a host, it will drive the USBx_DRVVBUS pin high to enable the external power logic so that it start sourcing the required 5V power (must be ≥ 4.4V but to account for the voltage drop on the cable it is suggested to be in the neighborhood of 4.75V). The USB2.0 controller will then wait to for the voltage of the USBx_VBUSIN to go high. After 100 ms, if it does not see the voltage on the USBX_VBUSIN pin to be within a Vbus Valid range (>= 4.4V), it will generate a Vbus error interrupt. Assuming that the voltage level of the USBX_VBUSIN is found to within the Vbus Valid range, the USB 2.0 host controller will wait for a device to connect; that is for it to see one of its data lines USB0/1_DP/DM be pulled high.

When assuming the role of a peripheral, assuming that the firmware has set the POWER[SOFTCONN] bit, and the USBx_ID pin is floating (h/w option) or USBx Mode Register IDDIG bit field is programmed with '1' (s/w option) and an external host is sourcing power on the USBx_VBUSIN line then the USB2.0 controller will set the DEVCTL[SESSION] bit instructing the controller to go into session. Not that the voltage on the USBx_VBUSIN pin must be within Vbus Valid range (i.e. greater or equal to 4.4V). When the controller goes into session, it will force the USB2.0 Controller to sense the state of the iddig signal. Once it senses iddig signal to be high, it will enable its 1.5Kohm pull-up resistor to signify the external host that it is a Full-Speed device. Note that even when operating as a High-Speed peripheral; the USB controller has to first come up as Full-Speed and then later transition to High-Speed. The USB2.0 controller will then waits for a reset signal from the external host. Then after, if High-Speed option has been selected it will negotiate for a High-Speed operation and if its request has been accepted by the host, it will enable its precision 45ohm termination resistors (to stop signal reflection) on its data lines and disable the 1.5Kohm resistor.

### 16.2.8 Protocol Description(s)

This section describes the implementation of the USB protocol(s) by the USB modules.

#### 16.2.8.1 USB Controller Peripheral Mode Operation

The USB controller assumes the role of a peripheral when the USBx_ID pin is floating or USB Mode Register[iddig=bit8] is set to 1 (provided that iddig_mux, which is bit7 of USBnMODE is also set to 1) by the user application prior to the controller goes into session. When the USB controller go into session it will assume the role of a device.

Soft connect –After a POR or USB Module soft reset, the SOFTCONN bit of POWER register (bit 6) is cleared to 0. The controller will therefore appear disconnected until the software has set the SOFTCONN bit to 1. The application software can then choose when to set the PHY into its normal mode. Systems with a lengthy initialization procedure may use this to ensure that initialization is complete and the system is ready to perform enumeration before connecting to the USB. Once the SOFTCONN bit has been set, the software can also simulate a disconnect by clearing this bit to 0.

Entry into suspend mode –When operating as a peripheral device, the controller monitors activity on the bus and when no activity has occurred for 3 ms, it goes into Suspend mode. If the Suspend interrupt has been enabled, an interrupt will be generated at this time.

At this point, the controller can then be left active (and hence able to detect when Resume signaling occurs on the USB), or the application may arrange to disable the controller by stopping its clock. However, the controller will not then be able to detect Resume signaling on the USB if clocks are not running. If such is the case, external hardware will be needed to detect Resume signaling (by monitoring the DM and DP signals), so that the clock to the controller can be restarted.

Resume Signaling –When resume signaling occurs on the bus, first the clock to the controller must be restarted if necessary. Then the controller will automatically exit Suspend mode. If the Resume interrupt is enabled, an interrupt will be generated.

Initiating a remote wakeup –If the software wants to initiate a remote wakeup while the controller is in Suspend mode, it should set the POWER[RESUME] bit to 1. The software should leave then this bit set for approximately 10 ms (minimum of 2 ms, a maximum of 15 ms) before resetting it to 0.

**NOTE:** No resume interrupt will be generated when the software initiates a remote wakeup.

Reset Signaling –When reset signaling or bus condition occurs on the bus, the controller performs the following actions:

- Clears FADDR register to 0
- Clears INDEX register to 0
- Flushes all endpoint FIFOs
- Clears all controller control/status registers
- Generates a reset interrupt
- Enables all interrupts at the core level.

If the HSENA bit within the POWER register (bit 5) is set, the controller also tries to negotiate for high-speed operation. Whether high-speed operation is selected is indicated by HSMODE bit of POWER register (bit 4). When the application software receives a reset interrupt, it should close any open pipes and wait for bus enumeration to begin.

##### 16.2.8.1.1 Control Transactions:Peripheral Mode

Endpoint 0 is the main control endpoint of the core. The software is required to handle all the standard device requests that may be sent or received via endpoint 0. These are described in Universal Serial Bus Specification, Revision 2.0, Chapter 9. The protocol for these device requests involves different numbers and types of transactions per request/command. To accommodate this, the software needs to take a state machine approach to command decoding and handling.

The Standard Device Requests received by a USB peripheral device can be divided into three categories: Zero Data Requests (in which all the information is included in the command; no additional data is required), Write Requests (in which the command will be followed by additional data), and Read Requests (in which the device is required to send data back to the host).

This section looks at the sequence of actions that the software must perform to process these different types of device request.

**NOTE:** The Setup packet associated with any standard device request should include an 8-byte command. Any setup packet containing a command field other than 8 bytes will be automatically rejected by the controller.

##### *16.2.8.1.1.1 Zero Data Requests: Peripheral Mode*

Zero data requests have all their information included in the 8-byte command and require no additional data to be transferred. Examples of Zero Data standard device requests are:

- SET_FEATURE
- CLEAR_FEATURE
- SET_ADDRESS
- SET_CONFIGURATION
- SET_INTERFACE

The sequence of events will begin, as with all requests, when the software receives an endpoint 0 interrupt. The RXPKTRDY bit of PERI_CSR0 (bit 0) will also have been set. The 8-byte command should then be read from the endpoint 0 FIFO, decoded and the appropriate action taken.

For example, if the command is SET_ADDRESS, the 7-bit address value contained in the command should be written to the FADDR register at the completion of the command. The PERI_CSR0 register should be written by setting the SERV_RXPKTRDY bit (bit 6) (indicating that the command has been read from the FIFO) and also setting the DATAEND bit (bit 3) (indicating that no further data is expected for this request). The interval between setting SERV_RXPKTRDY bit and DATAEND bit should be very small to avoid getting a SETUPEND error condition. It is highly recommended to set both bits at the same time.

When the host moves to the status stage of the request, a second endpoint 0 interrupt will be generated to indicate that the request has completed. No further action is required from the software. The second interrupt is just a confirmation that the request completed successfully. For SET_ADDRESS command, the address should be written to FADDR register at the completion of the command, i.e. when the status stage interrupt is received.

If the command is an unrecognized command, or for some other reason cannot be executed, then when it has been decoded, the PERI_CSR0 register should be written to set the SERV_RXPKTRDY bit (bit 6) and to set the SENDSTALL bit (bit 5). When the host moves to the status stage of the request, the controller will send a STALL packet telling the host that the request was not executed. A second endpoint 0 interrupt will be generated and the SENTSTALL bit (bit 2 of PERI_CSR0) will be set.

If the host sends more data after the DATAEND bit has been set, then the controller will send a STALL packet automatically. An endpoint 0 interrupt will be generated and the SENTSTALL bit (bit 2 of PERI_CSR0) will be set.

**NOTE:** DMA is not supported for endpoint 0; endpoint 0 is always serviced via CPU.

##### *16.2.8.1.1.2 Write Requests: Peripheral Mode*

Write requests involve an additional packet (or packets) of data being sent from the host after the 8-byte command. An example of a Write standard device request is: SET_DESCRIPTOR.

The sequence of events will begin, as with all requests, when the software receives an endpoint 0 interrupt. The RXPKTRDY bit of PERI_CSR0 will also have been set. The 8-byte command should then be read from the Endpoint 0 FIFO and decoded.

As with a zero data request, the PERI_CSR0 register should then be written to set the SERV_RXPKTRDY bit (bit 6) (indicating that the command has been read from the FIFO) but in this case the DATAEND bit (bit 3) should not be set (indicating that more data is expected).

When a second endpoint 0 interrupt is received, the PERI_CSR0 register should be read to check the endpoint status. The RXPKTRDY bit of PERI_CSR0 should be set to indicate that a data packet has been received. The COUNT0 register should then be read to determine the size of this data packet. The data packet can then be read from the endpoint 0 FIFO.

If the length of the data associated with the request (indicated by the wLength field in the command) is greater than the maximum packet size for endpoint 0, further data packets will be sent. In this case, PERI_CSR0 should be written to set the SERV_RXPKTRDY bit, but the DATAEND bit should not be set.

When all the expected data packets have been received, the PERI_CSR0 register should be written to set the SERV_RXPKTRDY bit and to set the DATAEND bit (indicating that no more data is expected).

When the host moves to the status stage of the request, another endpoint 0 interrupt will be generated to indicate that the request has completed. No further action is required from the software, the interrupt is just a confirmation that the request completed successfully.

If the command is an unrecognized command, or for some other reason cannot be executed, then when it has been decoded, the PERI_CSR0 register should be written to set the SERV_RXPKTRDY bit (bit 6) and to set the SENDSTALL bit (bit 5). When the host sends more data, the controller will send a STALL to tell the host that the request was not executed. An endpoint 0 interrupt will be generated and the SENTSTALL bit of PERI_CSR0 (bit 2) will be set.

If the host sends more data after the DATAEND has been set, then the controller will send a STALL. An endpoint 0 interrupt will be generated and the SENTSTALL bit of PERI_CSR0 (bit 2) will be set.

###### *16.2.8.1.1.3 Read Requests: Peripheral Mode*

Read requests have a packet (or packets) of data sent from the function to the host after the 8-byte command. Examples of Read Standard Device Requests are:

- GET_CONFIGURATION
- GET_INTERFACE
- GET_DESCRIPTOR
- GET_STATUS
- SYNCH_FRAME

The sequence of events will begin, as with all requests, when the software receives an endpoint 0 interrupt. The RXPKTRDY bit of PERI_CSR0 (bit 0) will also have been set. The 8-byte command should then be read from the endpoint 0 FIFO and decoded. The PERI_CSR0 register should then be written to set the SERV_RXPKTRDY bit (bit 6) (indicating that the command has read from the FIFO).

The data to be sent to the host should then be written to the endpoint 0 FIFO. If the data to be sent is greater than the maximum packet size for endpoint 0, only the maximum packet size should be written to the FIFO. The PERI_CSR0 register should then be written to set the TXPKTRDY bit (bit 1) (indicating that there is a packet in the FIFO to be sent). When the packet has been sent to the host, another endpoint 0 interrupt will be generated and the next data packet can be written to the FIFO.

When the last data packet has been written to the FIFO, the PERI_CSR0 register should be written to set the TXPKTRDY bit and to set the DATAEND bit (bit 3) (indicating that there is no more data after this packet).

When the host moves to the status stage of the request, another endpoint 0 interrupt will be generated to indicate that the request has completed. No further action is required from the software: the interrupt is just a confirmation that the request completed successfully.

If the command is an unrecognized command, or for some other reason cannot be executed, then when it has been decoded, the PERI_CSR0 register should be written to set the SERV_RXPKTRDY bit (bit 6) and to set the SENDSTALL bit (bit 5). When the host requests data, the controller will send a STALL to tell the host that the request was not executed. An endpoint 0 interrupt will be generated and the SENTSTALL bit of PERI_CSR0 (bit 2) will be set.

If the host requests more data after DATAEND (bit 3) has been set, then the controller will send a STALL. An endpoint 0 interrupt will be generated and the SENTSTALL bit of PERI_CSR0 (bit 2) will be set.

#### 16.2.8.1.1.4 Endpoint 0 States: Peripheral Mode

When the USB controller is operating as a peripheral device, the endpoint 0 control needs three modes – IDLE, TX and RX – corresponding to the different phases of the control transfer and the states endpoint 0 enters for the different phases of the transfer (described in later sections).

The default mode on power-up or reset should be IDLE. RXPKTRDY bit of PERI_CSR0 (bit 0) becoming set when endpoint 0 is in IDLE state indicates a new device request. Once the device request is unloaded from the FIFO, the controller decodes the descriptor to find whether there is a data phase and, if so, the direction of the data phase of the control transfer (in order to set the FIFO direction). See Figure 16-3.

Depending on the direction of the data phase, endpoint 0 goes into either TX state or RX state. If there is no Data phase, endpoint 0 remains in IDLE state to accept the next device request

The actions that the CPU needs to take at the different phases of the possible transfers (for example, loading the FIFO, setting TXPKTRDY) are indicated in Figure 16-4.

**Figure 16-3. CPU Actions at Transfer Phases**

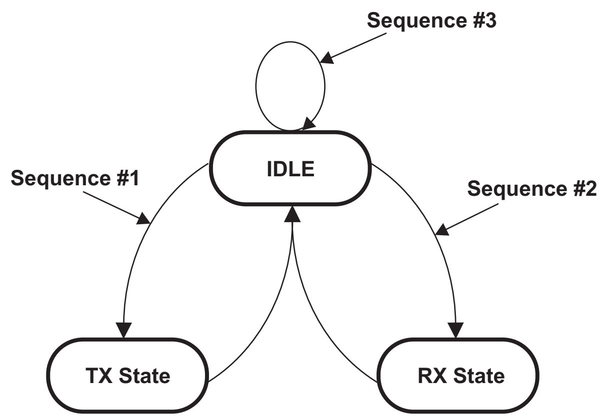
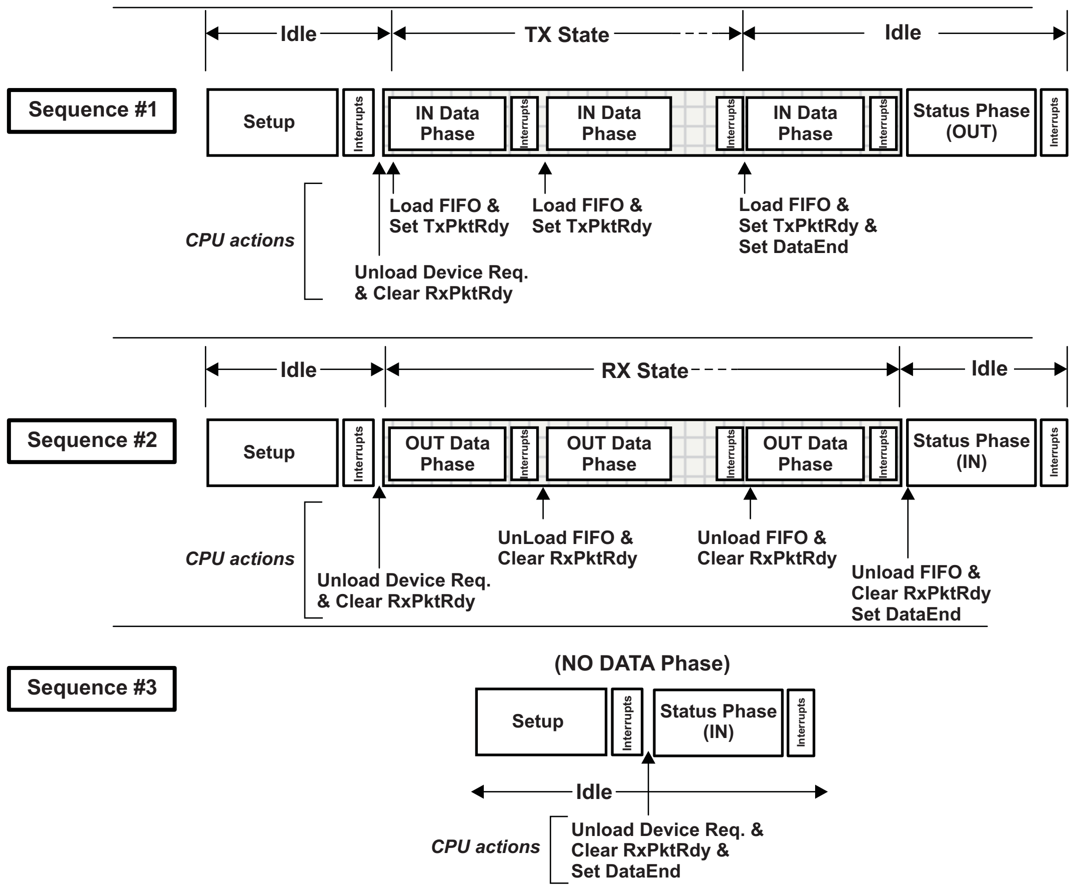


**Figure 16-4. Sequence of Transfer**



#### 16.2.8.1.1.4.1 Endpoint 0 Interrupt Generating Events: Peripheral Mode

An Endpoint 0 interrupt is generated when any of the below events occur and Interrupt Service handler should determine the right event and launch the proper handler

The following events will cause Endpoint 0 Interrupt to be generated while the controller is operating assuming the role of a device/peripheral:

- The controller sets the RXPKTRDY bit of PERI_CSR0 (bit 0) after a valid token has been received and data has been written to the FIFO (legal status condition).
- The controller clears the TXPKTRDY bit of PERI_CSR0 (bit 1) after the packet of data in the FIFO has been successfully transmitted to the host (legal status condition).
- The controller sets the SENTSTALL bit of PERI_CSR0 (bit 2) after a control transaction is ended due to a protocol violation (illegal status condition).
- The controller sets the SETUPEND bit of PERI_CSR0 (bit 4) because a control transfer has ended before DATAEND (bit 3 of PERI_CSR0) is set (illegal status condition).

In order to determine the cause of the interrupt, the detail below recommends the order of cause determination and execution.

Whenever the endpoint 0 service routine is entered, the software must first check to see if the current control transfer has been ended due to either a STALL condition or a premature end of control transfer. If the control transfer ends due to a STALL condition, the SENTSTALL bit would be set. If the control transfer ends due to a premature end of control transfer, the SETUPEND bit would be set. In either case, the software should abort processing the current control transfer and set the state to IDLE.

Once the software has determined that the interrupt was not generated by an illegal bus state, the next action taken depends on the endpoint state.

If endpoint 0 is in IDLE state, the only valid reason an interrupt can be generated is as a result of the controller receiving data from the bus. The service routine must check for this by testing the RXPKTRDY bit of PERI_CSR0 (bit 0). If this bit is set, then the controller has received a SETUP packet. This must be unloaded from the FIFO and decoded to determine the action the controller must take. Depending on the command contained within the SETUP packet, endpoint 0 will enter one of three states:

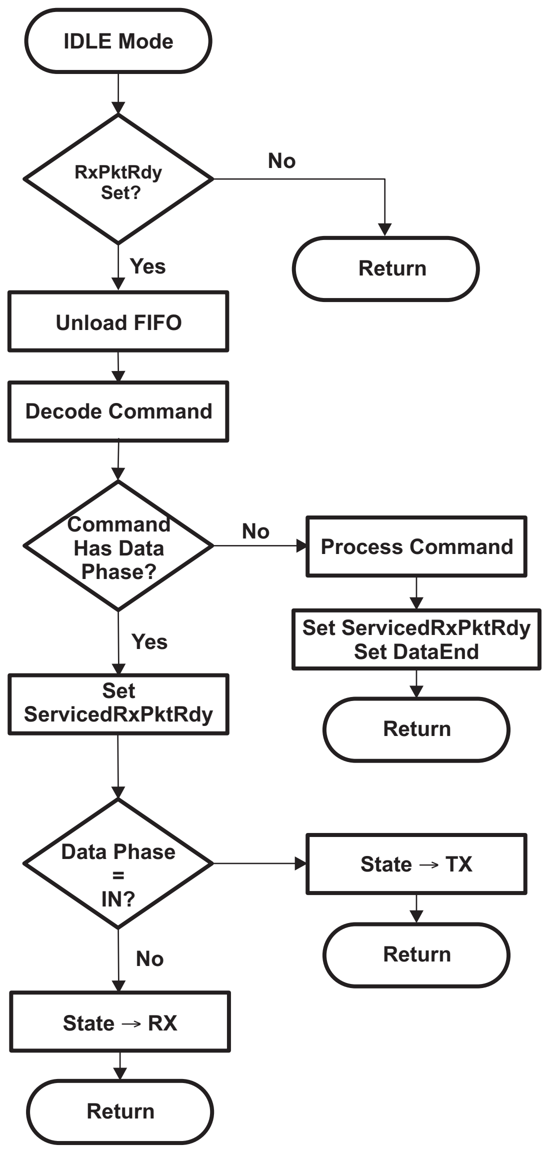
- If the command is a single packet transaction (SET_ADDRESS, SET_INTERFACE etc.) without any data phase, the endpoint will remain in IDLE state.
- If the command has an OUT data phase (SET_DESCRIPTOR etc.), the endpoint will enter RX state.
- If the command has an IN data phase (GET_DESCRIPTOR etc.), the endpoint will enter TX state.
- If the endpoint 0 is in TX state, the interrupt indicates that the core has received an IN token and data from the FIFO has been sent. The software must respond to this either by placing more data in the FIFO if the host is still expecting more data or by setting the DATAEND bit to indicate that the data phase is complete. Once the data phase of the transaction has been completed, endpoint 0 should be returned to IDLE state to await the next control transaction (status stage).
- If the endpoint is in RX state, the interrupt indicates that a data packet has been received. The software must respond by unloading the received data from the FIFO. The software must then determine whether it has received all of the expected data. If it has, the software should set the DATAEND bit and return endpoint 0 to IDLE state. If more data is expected, the firmware should set the SERV_RXPKTRDY bit of PERI_CSR0 (bit 6) to indicate that it has read the data in the FIFO and leave the endpoint in RX state.

#### 16.2.8.1.1.4.2 Idle Mode: Control Transfer of Peripheral Mode

IDLE mode is the mode the Endpoint 0 control needs to select at power-on or reset and is the mode to which the Endpoint 0 control should return when the RX and TX modes are terminated.

It is also the mode in which the SETUP phase of control transfer is handled (as outlined in Figure 16-5).

**Figure 16-5. Flow Chart of Setup Stage of a Control Transfer in Peripheral Mode**



#### 16.2.8.1.1.4.3 TX Mode: Control Transfer of Peripheral Mode

When the endpoint is in TX state, all arriving IN tokens need to be treated as part of a data phase until the required amount of data has been sent to the host. If either a SETUP or an OUT token is received while the endpoint is in the TX state, this will cause a SETUPEND condition to occur as the core expects only IN tokens.

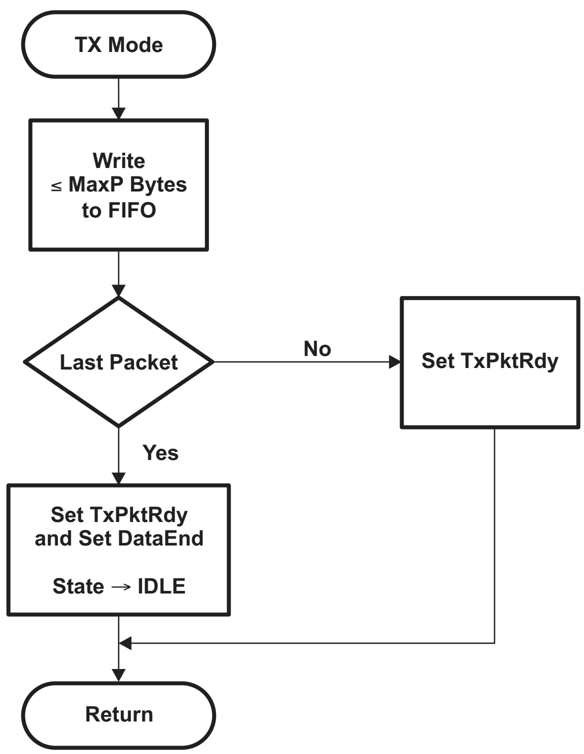
Three events can cause TX mode to be terminated before the expected amount of data has been sent as shown in Figure 16-6:

1. The host sends an invalid token causing a SETUPEND condition (bit 4 of PERI_CSR0 is set).
2. The firmware sends a packet containing less than the maximum packet size for Endpoint 0.
3. The firmware sends an empty data packet.

Until the transaction is terminated, the firmware simply needs to load the FIFO when it receives an interrupt which indicates that a packet has been sent from the FIFO. An interrupt is generated when TXPKTRDY is cleared.

When the firmware forces the termination of a transfer (by sending a short or empty data packet), it should set the DATAEND bit of PERI_CSR0 (bit 3) to indicate to the core that the data phase is complete and that the core should next receive an acknowledge packet.

**Figure 16-6. Flow Chart of Transmit Data Stage of a Control Transfer in Peripheral Mode**



#### 16.2.8.1.1.4.4 Rx Mode: Control Transfer of Peripheral Mode

In RX mode, all arriving data should be treated as part of a data phase until the expected amount of data has been received. If either a SETUP or an IN token is received while the endpoint is in RX state, a SETUPEND condition will occur as the controller expects only OUT tokens.

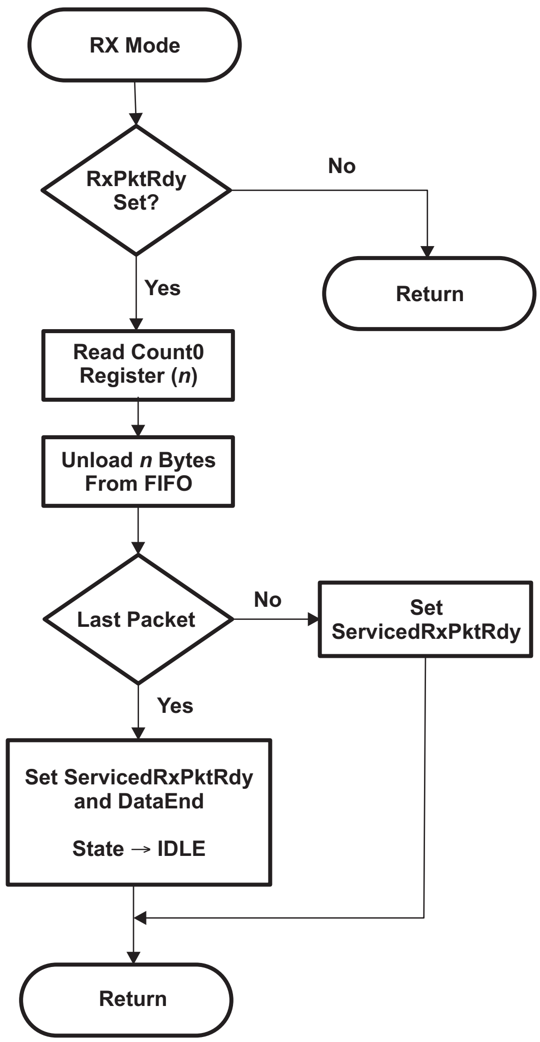
Three events can cause RX mode to be terminated before the expected amount of data has been received as shown in Figure 16-7:

1. The host sends an invalid token causing a SETUPEND condition (setting bit 4 of PERI_CSR0).
2. The host sends a packet which contains less than the maximum packet size for endpoint 0.
3. The host sends an empty data packet.

Until the transaction is terminated, the software unloads the FIFO when it receives an interrupt that indicates new data has arrived (setting RXPKTRDY bit of PERI_CSR0) and to clear RXPKTRDY by setting the SERV_RXPKTRDY bit of PERI_CSR0 (bit 6).

When the software detects the termination of a transfer (by receiving either the expected amount of data or an empty data packet), it should set the DATAEND bit (bit 3 of PERI_CSR0) to indicate to the controller that the data phase is complete and that the core should receive an acknowledge packet next.

**Figure 16-7. Flow Chart of Receive Data Stage of a Control Transfer in Peripheral Mode**



#### 16.2.8.1.1.4.5 Error Handling: Control Transfer of Peripheral Mode

A control transfer may be aborted due to a protocol error on the USB, the host prematurely ending the transfer, or if the software wishes to abort the transfer (for example, because it cannot process the command).

The controller automatically detects protocol errors and sends a STALL packet to the host under the following conditions:

- The host sends more data during the OUT Data phase of a write request than was specified in the command. This condition is detected when the host sends an OUT token after the DATAEND bit (bit 3 of PERI_CSR0) has been set.
- The host requests more data during the IN Data phase of a read request than was specified in the command. This condition is detected when the host sends an IN token after the DATAEND bit in the PERI_CSR0 register has been set.
- The host sends more than Max Packet Size data bytes in an OUT data packet.
- The host sends a non-zero length DATA1 packet during the STATUS phase of a read request.

When the controller has sent the STALL packet, it sets the SENTSTALL bit (bit 2 of PERI_CSR0) and generates an interrupt. When the software receives an endpoint 0 interrupt with the SENTSTALL bit set, it should abort the current transfer, clear the SENTSTALL bit, and return to the IDLE state.

If the host prematurely ends a transfer by entering the STATUS phase before all the data for the request has been transferred, or by sending a new SETUP packet before completing the current transfer, then the SETUPEND bit (bit 4 of PERI_CSR0) will be set and an endpoint 0 interrupt generated. When the software receives an endpoint 0 interrupt with the SETUPEND bit set, it should abort the current transfer, set the SERV_SETUPEND bit (bit 7 of PERI_CSR0), and return to the IDLE state. If the RXPKTRDY bit (bit 0 of PERI_CSR0) is set this indicates that the host has sent another SETUP packet and the software should then process this command.

If the software wants to abort the current transfer, because it cannot process the command or has some other internal error, then it should set the SENDSTALL bit (bit 5 of PERI_CSR0). The controller will then send a STALL packet to the host, set the SENTSTALL bit (bit 2 of PERI_CSR0) and generate an endpoint 0 interrupt.

#### 16.2.8.1.1.5 Additional Conditions: Control Transfer of Peripheral Mode

When working as a peripheral device, the controller automatically responds to certain conditions on the USB bus or actions by the host. The details are:

- Stall Issued to Control Transfers
  1. The host sends more data during an OUT Data phase of a Control transfer than was specified in the device request during the SETUP phase. This condition is detected by the controller when the host sends an OUT token (instead of an IN token) after the software has unloaded the last OUT packet and set DATAEND.
  2. The host requests more data during an IN data phase of a Control transfer than was specified in the device request during the SETUP phase. This condition is detected by the controller when the host sends an IN token (instead of an OUT token) after the software has cleared TXPKTRDY and set DATAEND in response to the ACK issued by the host to what should have been the last packet.
  3. The host sends more than MaxPktSize data with an OUT data token.
  4. The host sends the wrong PID for the OUT Status phase of a Control transfer.
  5. The host sends more than a zero length data packet for the OUT Status phase.
- Zero Length Out Data Packets In Control Transfer
  A zero length OUT data packet is used to indicate the end of a Control transfer. In normal operation, such packets should only be received after the entire length of the device request has been transferred (i.e., after the software has set DataEnd). If, however, the host sends a zero length OUT data packet before the entire length of device request has been transferred, this signals the premature end of the transfer. In this case, the controller will automatically flush any IN token loaded by software ready for the Data phase from the FIFO and set SETUPEND bit (bit 4 of PERI_CSR0).

#### 16.2.8.1.2 Bulk Transfer: Peripheral Mode

Bulk transactions are handled by endpoints other than endpoint 0. It is used to handle non-periodic, large bursty communication typically used for a transfer that use any available bandwidth and can also be delayed until bandwidth is available.

##### 16.2.8.1.2.1 Bulk IN Transactions: Peripheral Mode

A Bulk IN transaction is used to transfer non-periodic data from the USB peripheral device to the host.

The following optional features are available for use with a Tx endpoint used in peripheral mode for Bulk IN transactions:

- Double packet buffering: When enabled, up to two packets can be stored in the FIFO awaiting transmission to the host. Double packet buffering is enabled by setting the DPB bit of TXFIFOSZ register (bit 4).
- DMA: If DMA is enabled for the endpoint, a DMA request will be generated whenever the endpoint is able to accept another packet in its FIFO. This feature allows the DMA controller to load packets into the FIFO without processor intervention

###### 16.2.8.1.2.1.1 Bulk IN Transaction Setup: Peripheral Mode

In configuring a TX endpoint for bulk transactions, the TXMAXP register must be written with the maximum packet size (in bytes) for the endpoint. This value should be the same as the wMaxPacketSize field of the Standard Endpoint Descriptor for the endpoint and the PERI_TXCSR register (DMAEN and DMAMODE bit fields should be set when using DMA. Table 16-4 displays the PERI_TXCSR setting when used for Bulk transfer.

**Table 16-4. PERI_TXCSR Register Bit Configuration for Bulk IN Transactions**

| Bit Field | Bit Name | Description |
|---|---|---|
| Bit 15 | AUTOSET | Cleared to 0 if using DMA. For CPU Mode use, if AUTOSET bit is set, the TXPKTRDY bit will be automatically set when data of the maximum packet size is loaded into the FIFO. |
| Bit 14 | ISO | Cleared to 0 for bulk mode operation. |
| Bit 13 | MODE | Set to 1 to make sure the FIFO is enabled (only necessary if the FIFO is shared with an RX endpoint) |
| Bit 12 | DMAEN | Set to 1 to enable DMA usage; not needed if CPU is being used to service the Tx Endpoint |
| Bit 11 | FRCDATATOG | Cleared to 0 to allow normal data toggle operations. |
| Bit 10 | DMAMODE | Set to 1 when DMA is used to service Tx FIFO. |

When the endpoint is first configured (following a SET_CONFIGURATION or SET_INTERFACE command on Endpoint 0), the lower byte of PERI_TXCSR should be written to set the CLRDATATOG bit (bit 6). This will ensure that the data toggle (which is handled automatically by the controller) starts in the correct state.

Also if there are any data packets in the FIFO, indicated by the FIFONOTEMPTY bit (bit 1 of PERI_TXCSR) being set, they should be flushed by setting the FLUSHFIFO bit (bit 3 of PERI_TXCSR).

**NOTE:** It may be necessary to set this bit twice in succession if double buffering is enabled.

#### 16.2.8.1.2.1.2 Bulk IN Operation: Peripheral Mode

When data is to be transferred over a Bulk IN pipe, a data packet needs to be loaded into the FIFO and the PERI_TXCSR register written to set the TXPKTRDY bit (bit 0). When the packet has been sent, the TXPKTRDY bit will be cleared by the USB controller and an interrupt generated to signify to the user/application that the next packet can be loaded into the FIFO. If double packet buffering is enabled, then after the first packet has been loaded and the TXPKTRDY bit set, the TXPKTRDY bit will immediately be cleared (will not wait for the loaded first packet to be driven out) by the USB controller and an interrupt generated so that a second packet can be loaded into the FIFO. The software should operate in the same way, loading a packet when it receives an interrupt, regardless of whether double packet buffering is enabled or not.

In the general case, the packet size must not exceed the size specified by the lower 11 bits of the TXMAXP register. This part of the register defines the payload (packet size) for transfers over the USB and is required by the USB Specification to be either 8, 16, 32, 64 (Full-Speed or High-Speed) or 512 bytes (High-Speed only).

The host may determine that all the data for a transfer has been sent by knowing the total amount of data that is expected. Alternatively it may infer that all the data has been sent when it receives a packet which is smaller than the maximum packet size configuration for that particular endpoint (TXMAXP[10-0]). In the latter case, if the total size of the data block is a multiple of this payload, it will be necessary for the function to send a null packet after all the data has been sent. This is done by setting TXPKTRDY when the next interrupt is received, without loading any data into the FIFO.

If large blocks of data are being transferred, then the overhead of calling an interrupt service routine to load each packet can be avoided by using the CPPI DMA. A separate section detailing the use of the DMA is discussed within a latter section. Suffix is to say that the PERI_TXCSR (DMAEN and DMAMODE) bit fields need to be set and the PERI_TXCSR[AUTOSET] bit cleared at the core level when using the DMA with an endpoint configured for any IN transaction/transfer.

#### 16.2.8.1.2.1.3 Error Handling of Bulk IN Transfer: Peripheral Mode

If the software wants to shut down the Bulk IN pipe, it should set the SENDSTALL bit (bit 4 of PERI_TXCSR). When the controller receives the next IN token, it will send a STALL to the host, set the SENTSTALL bit (bit 5 of PERI_TXCSR) and generate an interrupt.

When the software receives an interrupt with the SENTSTALL bit (bit 5 of PERI_TXCSR) set, it should clear the SENTSTALL bit. It should however leave the SENDSTALL bit set until it is ready to re-enable the Bulk IN pipe.

**NOTE:** If the host failed to receive the STALL packet for some reason, it will send another IN token, so it is advisable to leave the SENDSTALL bit set until the software is ready to re-enable the Bulk IN pipe. When a pipe is re-enabled, the data toggle sequence should be restarted by setting the CLRDATATOG bit in the PERI_TXCSR register (bit 6).

#### 16.2.8.1.2.2 Bulk OUT Transfer: Peripheral Mode

A Bulk OUT transaction is used to transfer non-periodic data from the host to the function controller.

The following optional features are available for use with an Rx endpoint used in peripheral mode for Bulk OUT transactions:

- Double packet buffering: When enabled, up to two packets can be stored in the FIFO on reception from the host. Double packet buffering is enabled by setting the DPB bit of the RXFIFOSZ register (bit 4).
- DMA: If DMA is enabled for the endpoint, a DMA request will be generated whenever the endpoint has a packet in its FIFO. This feature can be used to allow the DMA controller to unload packets from the FIFO without processor intervention.

**NOTE:** When DMA is enabled, endpoint interrupt will not be generated for completion of packet reception. Endpoint interrupt will be generated only in the error conditions.

#### 16.2.8.1.2.2.1 Bulk OUT Transfer Setup: Peripheral Mode

In configuring an Rx endpoint for Bulk OUT transactions, the RXMAXP register must be written with the maximum packet size (in bytes) for the endpoint. This value should be the same as the wMaxPacketSize field of the Standard Endpoint Descriptor for the endpoint. When using DMA for Rx Endpoints, the PERI_RXCSR needs to have its DMAEN bit set and have its AUTOCLEAR and DMAMODE bits cleared. In addition, the relevant interrupt enable bit in the INTRRXE register should be set (if an interrupt is required for this endpoint) and the PERI_RXCSR register should be set as shown in Table 16-5.

**Table 16-5. PERI_RXCSR Register Bit Configuration for Bulk OUT Transactions**

| Bit Field | Bit Name | Description |
|---|---|---|
| Bit 15 | AUTOCLEAR | Cleared to 0 if using DMA. In CPU Mode of usage, if the CPU sets AUTOCLEAR bit, the RXPKTRDY bit will be automatically cleared when a packet of RXMAXP bytes has been unloaded from the Receive FIFO. |
| Bit 14 | ISO | Cleared to 0 to enable Bulk protocol. |
| Bit 13 | DMAEN | Set to 1 if a DMA request is required for this Rx endpoint. |
| Bit 12 | DISNYET | Cleared to 0 to allow normal PING flow control. This will affect only high speed transactions. |
| Bit 11 | DMAMODE | Always clear this bit to 0. |

When the Rx endpoint is first configured (following a SET_CONFIGURATION or SET_INTERFACE command on Endpoint 0), the lower byte of PERI_RXCSR should be written to set the CLRDATATOG bit (bit 7). This will ensure that the data toggle (which is handled automatically by the USB controller) starts in the correct state.

Also if there are any data packets in the FIFO (indicated by the RXPKTRDY bit (bit 0 of PERI_RXCSR) being set), they should be flushed by setting the FLUSHFIFO bit (bit 4 of PERI_RXCSR).

**NOTE:** It may be necessary to set this bit twice in succession if double buffering is enabled.

#### 16.2.8.1.2.2.2 Bulk OUT Operation: Peripheral Mode

When a data packet is received by a Bulk Rx endpoint, the RXPKTRDY bit (bit 0 of PERI_RXCSR) is set and an interrupt is generated. The software should read the RXCOUNT register for the endpoint to determine the size of the data packet. The data packet should be read from the FIFO, and then the RXPKTRDY bit should be cleared.

The packets received should not exceed the size specified in the RXMAXP register (as this should be the value set in the wMaxPacketSize field of the endpoint descriptor sent to the host). When a block of data larger than wMaxPacketSize needs to be sent to the function, it will be sent as multiple packets. All the packets will be wMaxPacketSize in size, except the last packet which will contain the residue. The software may use an application specific method of determining the total size of the block and hence when the last packet has been received. Alternatively it may infer that the entire block has been received when it receives a packet which is less than wMaxPacketSize in size. (If the total size of the data block is a multiple of wMaxPacketSize, a null data packet will be sent after the data to signify that the transfer is complete.)

In the general case, the application software will need to read each packet from the FIFO individually. If large blocks of data are being transferred, the overhead of calling an interrupt service routine to unload each packet can be avoided by using DMA.

##### 16.2.8.1.2.2.3 Bulk OUT Error Handling: Peripheral Mode

If the software wants to shut down the Bulk OUT pipe, it should set the SENDSTALL bit (bit 5 of PERI_RXCSR). When the controller receives the next packet it will send a STALL to the host, set the SENTSTALL bit (bit 6 of PERI_RXCSR) and generate an interrupt.

When the software receives an interrupt with the SENTSTALL bit (bit 6 of PERI_RXCSR) set, it should clear this bit. It should however leave the SENDSTALL bit set until it is ready to re-enable the Bulk OUT pipe.

**NOTE:** If the host failed to receive the STALL packet for some reason, it will send another packet, so it is advisable to leave the SENDSTALL bit set until the software is ready to re-enable the Bulk OUT pipe. When a Bulk OUT pipe is re-enabled, the data toggle sequence should be restarted by setting the CLRDATATOG bit (bit 7) in the PERI_RXCSR register.

#### 16.2.8.1.3 Interrupt Transfer: Peripheral Mode

An Interrupt IN transaction uses the same protocol as a Bulk IN transaction and the configuration and operation mentioned on prior sections apply to Interrupt transfer too with minor changes. Similarly, an Interrupt OUT transaction uses almost the same protocol as a Bulk OUT transaction and can be used the same way.

Tx endpoints in the USB controller have one feature for Interrupt IN transactions that they do not support in Bulk IN transactions. In Interrupt IN transactions, the endpoints support continuous toggle of the data toggle bit.

This feature is enabled by setting the FRCDATATOG bit in the PERI_TXCSR register (bit 11). When this bit is set, the controller will consider the packet as having been successfully sent and toggle the data bit for the endpoint, regardless of whether an ACK was received from the host.

Another difference is that interrupt endpoints do not support PING flow control. This means that the controller should never respond with a NYET handshake, only ACK/NAK/STALL. To ensure this, the DISNYET bit in the PERI_RXCSR register (bit 12) should be set to disable the transmission of NYET handshakes in high-speed mode.

Though DMA can be used with an interrupt OUT endpoint, it generally offers little benefit as interrupt endpoints are usually expected to transfer all their data in a single packet.

#### 16.2.8.1.4 Isochronous Transfer: Peripheral Mode

Isochronous transfers are used when working with isochronous data. Isochronous transfers provide periodic, continuous communication between host and device.

##### 16.2.8.1.4.1 Isochronous IN Transactions: Peripheral Mode

An Isochronous IN transaction is used to transfer periodic data from the function controller to the host.

The following optional features are available for use with a Tx endpoint used in Peripheral mode for Isochronous IN transactions:

- Double packet buffering: When enabled, up to two packets can be stored in the FIFO awaiting transmission to the host. Double packet buffering is enabled by setting the DPB bit of TXFIFOSZ register (bit 4).

  **NOTE:** Double packet buffering is generally advisable for isochronous transactions in order to avoid underrun errors as described in later section.
- DMA: If DMA is enabled for the endpoint, a DMA request will be generated whenever the endpoint is able to accept another packet in its FIFO. This feature allows the DMA controller to load packets into the FIFO without processor intervention.
  However, this feature is not particularly useful with Isochronous endpoints because the packets transferred are often not maximum packet size and the PERI_TXCSR register needs to be accessed following every packet to check for Underrun errors.
  When DMA is enabled and DMAMODE bit of PERI_TXCSR is set, endpoint interrupt will not be generated for completion of packet transfer. Endpoint interrupt will be generated only in the error conditions.

#### 16.2.8.1.4.1.1 Isochronous IN Transfer Setup: Peripheral Mode

In configuring a Tx endpoint for Isochronous IN transactions, the TXMAXP register must be written with the maximum packet size (in bytes) for the endpoint. This value should be the same as the wMaxPacketSize field of the Standard Endpoint Descriptor for the endpoint. In addition, the relevant interrupt enable bit in the INTRTXE register should be set (if an interrupt is required for this endpoint) and the PERI_TXCSR register should be set as shown in Table 16-6.

**Table 16-6. PERI_TXCSR Register Bit Configuration for Isochronous IN Transactions**

| Bit Field | Bit Name | Description |
|---|---|---|
| Bit 14 | ISO | Set to 1 to enable Isochronous transfer protocol. |
| Bit 13 | MODE | Set to 1 to ensure the FIFO is enabled (only necessary if the FIFO is shared with an Rx endpoint). |
| Bit 12 | DMAEN | Set to 1 if DMA Requests have to be enabled. |
| Bit 11 | FRCDATATOG | Ignored in Isochronous mode. |
| Bit 10 | DMAMODE | Set to 1 when DMA is enabled. |

#### 16.2.8.1.4.1.2 Isochronous IN Transfer Operation: Peripheral Mode

An isochronous endpoint does not support data retries, so if data underrun is to be avoided, the data to be sent to the host must be loaded into the FIFO before the IN token is received. The host will send one IN token per frame (or microframe in High-speed mode), however the timing within the frame (or microframe) can vary. If an IN token is received near the end of one frame and then at the start of the next frame, there will be little time to reload the FIFO. For this reason, double buffering of the endpoint is usually necessary.

An interrupt is generated whenever a packet is sent to the host and the software may use this interrupt to load the next packet into the FIFO and set the TXPKTRDY bit in the PERI_TXCSR register (bit 0) in the same way as for a Bulk Tx endpoint. As the interrupt could occur almost any time within a frame(/microframe), depending on when the host has scheduled the transaction, this may result in irregular timing of FIFO load requests. If the data source for the endpoint is coming from some external hardware, it may be more convenient to wait until the end of each frame(/microframe) before loading the FIFO as this will minimize the requirement for additional buffering. This can be done by using either the SOF interrupt or the external SOF_PULSE signal from the controller to trigger the loading of the next data packet. The SOF_PULSE is generated once per frame(/microframe) when a SOF packet is received. (The controller also maintains an external frame(/microframe) counter so it can still generate a SOF_PULSE when the SOF packet has been lost.) The interrupts may still be used to set the TXPKTRDY bit in PERI_TXCSR (bit 0) and to check for data overruns/underruns.

Starting up a double-buffered Isochronous IN pipe can be a source of problems. Double buffering requires that a data packet is not transmitted until the frame(/microframe) after it is loaded. There is no problem if the function loads the first data packet at least a frame(/microframe) before the host sets up the pipe (and therefore starts sending IN tokens). But if the host has already started sending IN tokens by the time the first packet is loaded, the packet may be transmitted in the same frame(/microframe) as it is loaded, depending on whether it is loaded before, or after, the IN token is received. This potential problem can be avoided by setting the ISOUPDATE bit in the POWER register (bit 7). When this bit is set, any data packet loaded into an Isochronous Tx endpoint FIFO will not be transmitted until after the next SOF packet has been received, thereby ensuring that the data packet is not sent too early.

#### 16.2.8.1.4.1.3 Isochronous IN Error Handling: Peripheral Mode

If the endpoint has no data in its FIFO when an IN token is received, it will send a null data packet to the host and set the UNDERRUN bit in the PERI_TXCSR register (bit 2). This is an indication that the software is not supplying data fast enough for the host. It is up to the application to determine how this error condition is handled.

If the software is loading one packet per frame(/microframe) and it finds that the TXPKTRDY bit in the PERI_TXCSR register (bit 0) is set when it wants to load the next packet, this indicates that a data packet has not been sent (perhaps because an IN token from the host was corrupted). It is up to the application how it handles this condition: it may choose to flush the unsent packet by setting the FLUSHFIFO bit in the PERI_TXCSR register (bit 3), or it may choose to skip the current packet.

#### *16.2.8.1.4.2 Isochronous OUT Transactions: Peripheral Mode*

An isochronous OUT transaction is used to transfer periodic data from the host to the function controller.

Following optional features are available for use with an Rx endpoint used in Peripheral mode for Isochronous OUT transactions:

- Double packet buffering: When enabled, up to two packets can be stored in the FIFO on reception from the host. Double packet buffering is enabled by setting the DPB bit of RXFIFOSZ register (bit 4). **NOTE:** Double packet buffering is generally advisable for Isochronous transactions in order to avoid overrun errors.
- DMA: If DMA is enabled for the endpoint, a DMA request will be generated whenever the endpoint has a packet in its FIFO. This feature can be used to allow the DMA controller to unload packets from the FIFO without processor intervention.
  However, this feature is not particularly useful with Isochronous endpoints because the packets transferred are often not maximum packet size and the PERI_RXCSR register needs to be accessed following every packet to check for Overrun or CRC errors.
  When DMA is enabled, endpoint interrupt will not be generated for completion of packet reception. Endpoint interrupt will be generated only in the error conditions.

#### *16.2.8.1.4.2.1 Isochronous OUT Setup: Peripheral Mode*

In configuring an Rx endpoint for Isochronous OUT transactions, the RXMAXP register must be written with the maximum packet size (in bytes) for the endpoint. This value should be the same as the wMaxPacketSize field of the Standard Endpoint Descriptor for the endpoint. In addition, the relevant interrupt enable bit in the INTRRXE register should be set (if an interrupt is required for this endpoint) and the PERI_RXCSR register should be set as shown in Table 16-7.

**Table 16-7. PERI_RXCSR Register Bit Configuration for Isochronous OUT Transactions**

| Bit Field | Bit Name | Description |
|---|---|---|
| Bit 15 | AUTOCLEAR | Cleared to 0 if using DMA. In CPU Mode of usage, if the CPU sets AUTOCLEAR bit, the RXPKTRDY bit will be automatically cleared when a packet of RXMAXP bytes has been unloaded from the Receive FIFO. |
| Bit 14 | ISO | Set to 1 to configure endpoint usage for Isochronous transfer. |
| Bit 13 | DMAEN | Set to 1 if a DMA request is required for this Rx endpoint. |
| Bit 12 | DISNYET | Ignored in Isochronous Mode. |
| Bit 11 | DMAMODE | Always clear this bit to 0. |

#### 16.2.8.1.4.2.2 Isochronous OUT Operation: Peripheral Mode

An Isochronous endpoint does not support data retries so, if a data overrun is to be avoided, there must be space in the FIFO to accept a packet when it is received. The host will send one packet per frame (or microframe in High-speed mode); however, the time within the frame can vary. If a packet is received near the end of one frame(/microframe) and another arrives at the start of the next frame, there will be little time to unload the FIFO. For this reason, double buffering of the endpoint is usually necessary.

An interrupt is generated whenever a packet is received from the host and the software may use this interrupt to unload the packet from the FIFO and clear the RXPKTRDY bit in the PERI_RXCSR register (bit 0) in the same way as for a Bulk Rx endpoint. As the interrupt could occur almost any time within a frame(/microframe), depending on when the host has scheduled the transaction, the timing of FIFO unload requests will probably be irregular. If the data sink for the endpoint is going to some external hardware, it may be better to minimize the requirement for additional buffering by waiting until the end of each frame(/microframe) before unloading the FIFO. This can be done by using either the SOF interrupt or the external SOF_PULSE signal from the controller to trigger the unloading of the data packet. The SOF_PULSE is generated once per frame(/microframe) when a SOF packet is received. (The controller also maintains an external frame(/microframe) counter so it can still generate a SOF_PULSE when the SOF packet has been lost.) The interrupts may still be used to clear the RXPKTRDY bit in PERI_RXCSR and to check for data overruns/underruns.

#### 16.2.8.1.4.2.3 Isochronous OUT Error Handling: Peripheral Mode

If there is no space in the FIFO to store a packet when it is received from the host, the OVERRUN bit in the PERI_RXCSR register (bit 2) will be set. This is an indication that the software is not unloading data fast enough for the host. It is up to the application to determine how this error condition is handled

If the controller finds that a received packet has a CRC error, it will still store the packet in the FIFO and set the RXPKTRDY bit (bit 0 of PERI_RXCSR) and the DATAERROR bit (bit 3 of PERI_RXCSR). It is left up to the application to determine how this error condition is handled.

The number of USB packets sent in any microframe will depend on the amount of data to be transferred, and is indicated through the PIDs used for the individual packets. If the indicated number of packets have not been received by the end of a microframe, the INCOMPRX bit (bit 8) bit in the PERI_RXCSR register will be set to indicate that the data in the FIFO is incomplete. Equally, if a packet of the wrong data type is received, then the PID Error bit is the PERI_RXCSR register will be set. In each case, an interrupt will, however, still be generated to allow the data that has been received to be read from the FIFO.

**Note:** The circumstances in which a PID Error or INCOMPRX is reported depends on the precise sequence of packets received.

When the core is operating in peripheral mode, the details are in Table 16-8.

**Table 16-8. Isochronous OUT Error Handling: Peripheral Mode**

| No. Packet(s) Expected | Data Packet(s) Received | Response |
|---|---|---|
| 1 | DATA0 | OK |
| | DATA1 | PID Error set |
| | DATA2 | PID Error set |
| | MDATA | PID Error set |
| 2 | DATA0 | OK |
| | DATA1 | INCOMPRX Set |
| | DATA2 | INCOMPRX Set + PID Error set |
| | MDATA | INCOMPRX Set |
| | MDATA DATA0 | PID Error Set |
| | MDATA DATA1 | OK |
| | MDATA DATA2 | PID Error Set |
| | MDATA MDATA | PID Error Set |
| 3 | DATA0 | OK |
| | DATA1 | INCOMPRX Set |

**Table 16-8. Isochronous OUT Error Handling: Peripheral Mode (continued)**

| No. Packet(s) Expected | Data Packet(s) Received | Response |
|---|---|---|
| | DATA2 | INCOMPRX Set |
| | MDATA | INCOMPRX Set |
| | MDATA DATA0 | PID Error set |
| | MDATA DATA1 | OK |
| | MDATA DATA2 | INCOMPRX Set |
| | MDATA MDATA | INCOMPRX Set |
| | MDATA MDATA DATA0 | PID Error set |
| | MDATA MDATA DATA1 | PID Error set |
| | MDATA MDATA DATA2 | OK |
| | MDATA MDATA MDATA | PID Error set |

#### 16.2.8.2 USB Controller Host Mode Operation

The USB controller assumes the role of a host when the USBx_ID pin state is grounded or USB Mode Register[iddig=bit8] is cleared to 0 (provided that iddig_mux, which is bit7 of USBnMODE is also set to 1) by the user application prior to the controller goes into session. When the USB controller go into session, application/firmware sets the DEVCTL[SESSION] bit to 1, it will assume the role of a host.

- Entry into Suspend mode. When operating as a host, the USB controller can be prompted to go into Suspend mode by setting the SUSPENDM bit in the POWER register. When this bit is set, the controller will complete the current transaction then stop the transaction scheduler and frame counter. No further transactions will be started and no SOF packets will be generated.
  If the ENSUSPM bit (bit 0 of POWER register) is set, the UTMI+ PHY will go into low-power mode when the controller goes into suspend mode.
- Sending Resume Signaling. When the application requires the controller to leave suspend mode, it needs to clear the SUSPENDM bit in the POWER register, set the RESUME bit and leave it set for 20ms. While the RESUME bit is high, the controller will generate Resume signaling on the bus. After 20 ms, the CPU should clear the RESUME bit, at which point the frame counter and transaction scheduler will be started.
- Responding to Remote Wake-up. If Resume signaling is detected from the target while the controller is in suspend mode, the UTMI+ PHY will be brought out of low-power mode and UTMI clock is restarted. The controller will then exit suspend mode and automatically set the RESUME bit in the POWER register (bit 2) to ‘1’ to take over generating the Resume signaling from the target. If the resume interrupt is enabled, an interrupt will be generated.
- Reset Signaling. If the RESET bit in the POWER register (bit 3) is set while the controller is in Host mode, it will generate Reset signaling on the bus. If the HSENAB bit in the POWER register (bit 5) was set, it will also try to negotiate for high-speed operation. The software should keep the RESET bit set for at least 20 ms to ensure correct resetting of the target device. After the software has cleared the bit, the controller will start its frame counter and transaction scheduler. Whether high-speed operation is selected will be indicated by HSMODE bit of POWER register (bit 4).

#### 16.2.8.2.1 Control Transactions: Host Mode

Host control transactions are conducted through Endpoint 0 and the software is required to handle all the Standard Device Requests that may be sent or received via Endpoint 0 (as described in Universal Serial Bus Specification, Revision 2.0). Endpoint 0 can only be serviced via CPU and DMA mode can not be used.

There are three categories of Standard Device Requests to be handled: Zero Data Requests (in which all the information is included in the command); Write Requests (in which the command will be followed by additional data); and Read Requests (in which the device is required to send data back to the host).

1. Zero Data Requests comprise a SETUP command followed by an IN Status Phase.
2. Write Requests comprise a SETUP command, followed by an OUT Data Phase which is in turn followed by an IN Status Phase.
3. Read Requests comprise a SETUP command, followed by an IN Data Phase which is in turn followed by an OUT Status Phase.

A timeout may be set to limit the length of time for which the host controller will retry a transaction which is continually NAKed by the target. This limit can be between 2 and 2 15 frames/microframes and is set through the NAKLIMIT0 register.

The following sections describe the actions that the CPU needs to take in issuing these different types of request through looking at the steps to take in the different phases of a control transaction.

Note: Before initiating any transactions as a Host, the FADDR register needs to be set to address the peripheral device. When the device is first connected, FADDR should be set to zero. After a SET_ADDRESS command is issued, FADDR should be set the target's new address.

##### 16.2.8.2.1.1 Setup Phase of Control Transaction: Host Mode

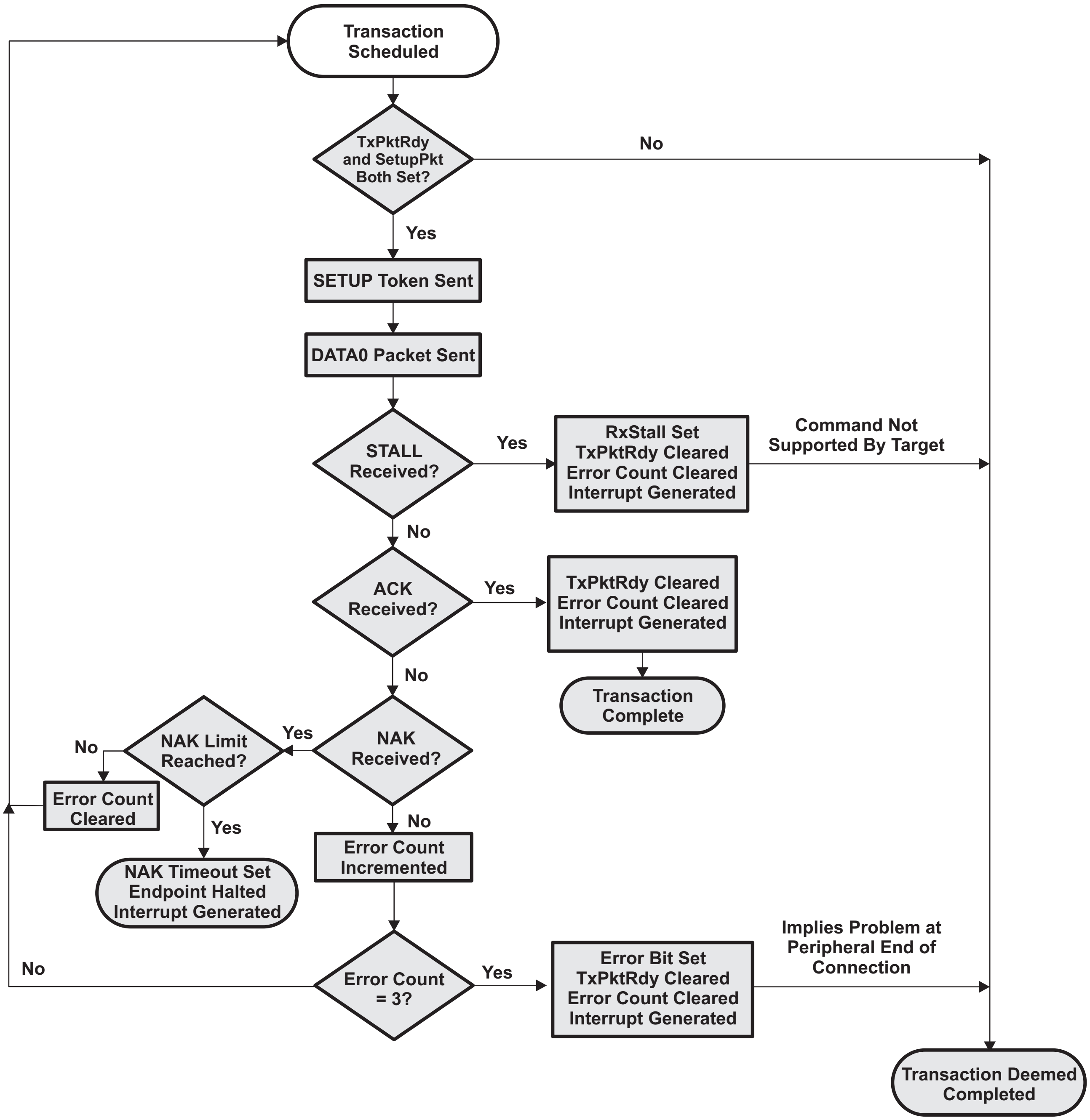
For the SETUP Phase of a control transaction (Figure 16-8), the software driving the USB host device needs to:

1. Load the 8 bytes of the required Device request command into the Endpoint 0 FIFO
2. Set SETUPPKT and TXPKTRDY (bits 3 and 1 of HOST_CSR0, respectively).
   **NOTE:** These bits must be set together.

   The controller then proceeds to send a SETUP token followed by the 8-byte command/request to Endpoint 0 of the addressed device, retrying as necessary. *Note:* On errors, the controller retries the transaction three times.
3. At the end of the attempt to send the 8-byte request data, the controller will generate an Endpoint 0 interrupt. The software should then read HOST_CSR0 to establish whether the RXSTALL bit (bit 2), the ERROR bit (bit 4) or the NAK_TIMEOUT bit (bit 7) has been set.

   If RXSTALL is set, it indicates that the target did not accept the command (for example, because it is not supported by the target device) and so has issued a STALL response. If ERROR is set, it means that the controller has tried to send the SETUP Packet and the following data packet three times without getting any response.
   If NAK_TIMEOUT is set, it means that the controller has received a NAK response to each attempt to send the SETUP packet, for longer than the time set in HOST_NAKLIMIT0. The controller can then be directed either to continue trying this transaction (until it times out again) by clearing the NAK_TIMEOUT bit or to abort the transaction by flushing the FIFO before clearing the NAK_TIMEOUT bit.
4. If none of RXSTALL, ERROR or NAK_TIMEOUT is set, the SETUP Phase has been correctly ACKed and the software should proceed to the following IN Data Phase, OUT Data Phase or IN Status Phase specified for the particular Standard Device Request.

**Figure 16-8. Flow Chart of Setup Stage of a Control Transfer in Host Mode**




#### *16.2.8.2.1.2 Data Phase (IN Data Phase) of a Control Transaction: Host Mode*

For the IN Data Phase of a control transaction (Figure 16-9), the software driving the USB host device needs to

1. Set REQPKT bit of HOST_CSR0 (bit 5)
2. Wait while the controller sends the IN token and receives the required data back.
3. When the controller generates the Endpoint 0 interrupt, read HOST_CSR0 to establish whether the RXSTALL bit (bit 2), the ERROR bit (bit 4), the NAK_TIMEOUT bit (bit 7) or RXPKTRDY bit (bit 0) has been set.

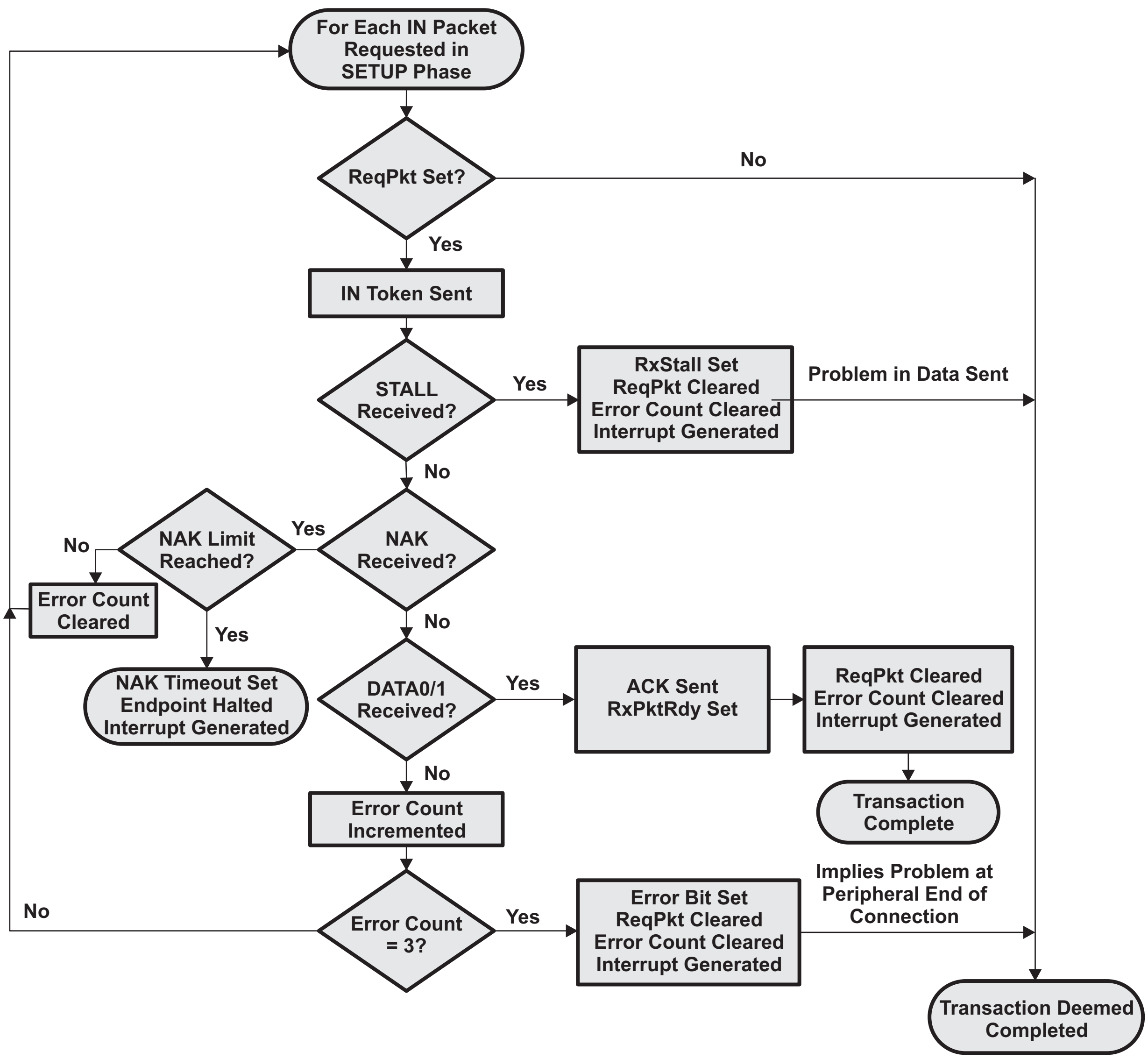
   If RXSTALL is set, it indicates that the target has issued a STALL response.

If ERROR is set, it means that the controller has tried to send the required IN token three times without getting any response. If NAK_TIMEOUT bit is set, it means that the controller has received a NAK response to each attempt to send the IN token, for longer than the time set in HOST_NAKLIMIT0. The controller can then be directed either to continue trying this transaction (until it times out again) by clearing the NAK_TIMEOUT bit or to abort the transaction by clearing REQPKT before clearing the NAK_TIMEOUT bit

4. If RXPKTRDY has been set, the software should read the data from the Endpoint 0 FIFO, then clear RXPKTRDY.
5. If further data is expected, the software should repeat Steps 1-4.

When all the data has been successfully received, the CPU should proceed to the OUT Status Phase of the Control Transaction.

**Figure 16-9. Flow Chart of Data Stage (IN Data Phase) of a Control Transfer in Host Mode**




#### *16.2.8.2.1.3 Data Phase (OUT Data Phase) of a Control Transaction: Host Mode*

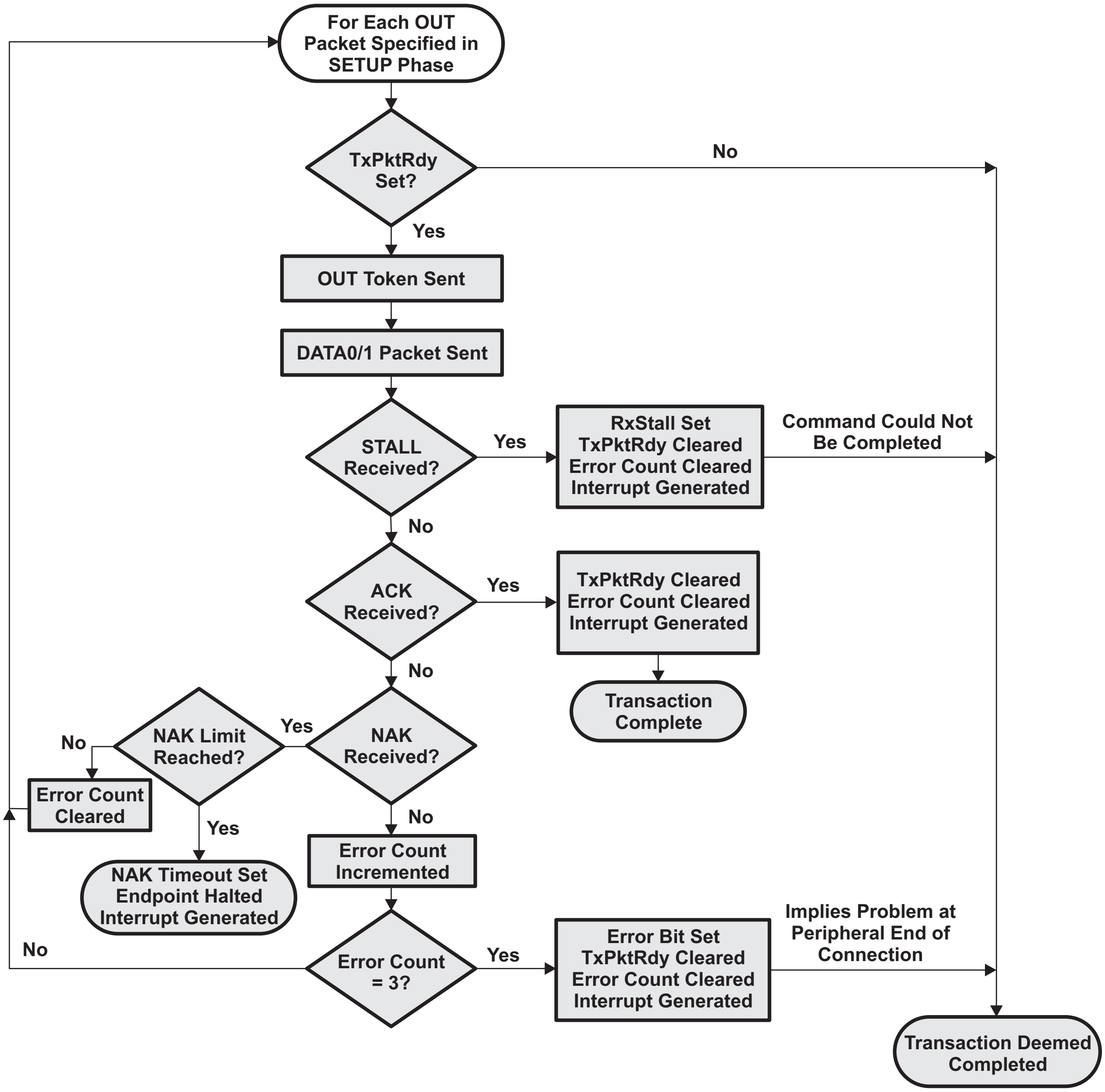
For the OUT Data Phase of a control transaction (Figure 16-10), the software driving the USB host device needs to:

1. Load the data to be sent into the endpoint 0 FIFO.
2. Set the TXPKTRDY bit of HOST_CSR0 (bit 1). The controller then proceeds to send an OUT token followed by the data from the FIFO to Endpoint 0 of the addressed device, retrying as necessary.
3. At the end of the attempt to send the data, the controller will generate an Endpoint 0 interrupt. The software should then read HOST_CSR0 to establish whether the RXSTALL bit (bit 2), the ERROR bit (bit 4) or the NAK_TIMEOUT bit (bit 7) has been set.

   If RXSTALL bit is set, it indicates that the target has issued a STALL response.
   If ERROR bit is set, it means that the controller has tried to send the OUT token and the following data packet three times without getting any response. If NAK_TIMEOUT is set, it means that the controller has received a NAK response to each attempt to send the OUT token, for longer than the time set in the HOST_NAKLIMIT0 register. The controller can then be directed either to continue trying this transaction (until it times out again) by clearing the NAK_TIMEOUT bit or to abort the transaction by flushing the FIFO before clearing the NAK_TIMEOUT bit.
   If none of RXSTALL, ERROR or NAKLIMIT is set, the OUT data has been correctly ACKed.
4. If further data needs to be sent, the software should repeat Steps 1-3.

   When all the data has been successfully sent, the software should proceed to the IN Status Phase of the Control Transaction.

**Figure 16-10. Flow Chart of Data Stage (OUT Data Phase) of a Control Transfer in Host Mode**




#### 16.2.8.2.1.4 Status Phase (Status for OUT Data Phase or Setup Phase) of a Control Transaction: Host Mode

IN Status Phase of a control transfer exists for a Zero Data Request or for a Write Request of a control transfer. The IN Status Phase follows the Setup Stage, if no Data Stage of a control transfer exists, or OUT Data Phase of a Data Stage of a control transfer.

For the IN Status Phase of a control transaction (Figure 16-11), the software driving the USB Host device needs to:

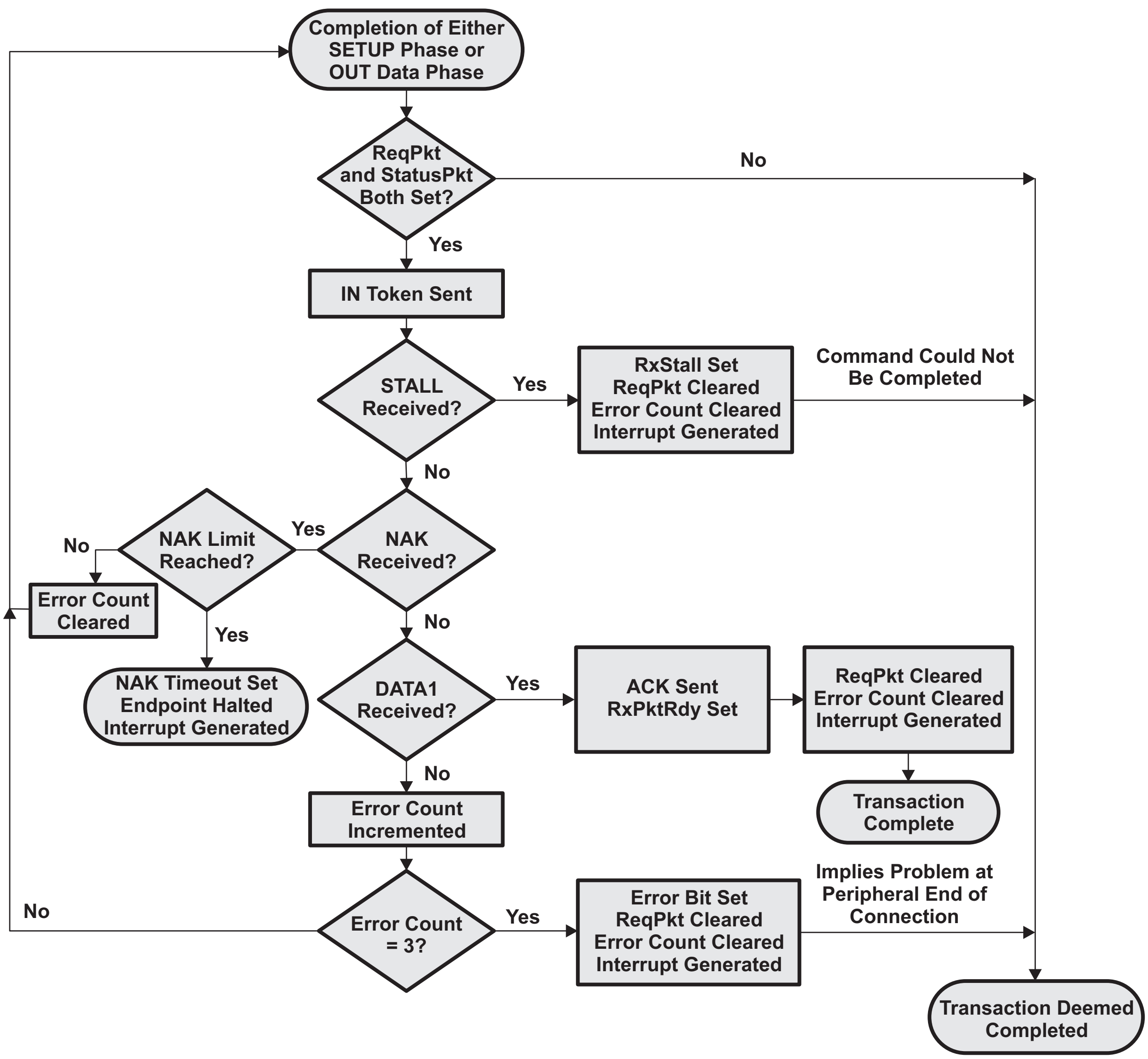
1. Set the STATUSPKT and REQPKT bits of HOST_CSR0 (bit 6 and bit 5, respectively).
2. Wait while the controller sends an IN token and receives a response from the USB peripheral device.
3. When the controller generates the Endpoint 0 interrupt, read HOST_CSR0 to establish whether the RXSTALL bit (bit 2), the ERROR bit (bit 4), the NAK_TIMEOUT bit (bit 7) or RXPKTRDY bit (bit 0) has been set.

   If RXSTALL bit is set, it indicates that the target could not complete the command and so has issued a

STALL response.
If ERROR bit is set, it means that the controller has tried to send the required IN token three times without getting any response.
If NAK_TIMEOUT bit is set, it means that the controller has received a NAK response to each attempt to send the IN token, for longer than the time set in the HOST_NAKLIMIT0 register. The controller can then be directed either to continue trying this transaction (until it times out again) by clearing the NAK_TIMEOUT bit or to abort the transaction by clearing REQPKT bit and STATUSPKT bit before clearing the NAK_TIMEOUT bit.

4. The CPU should clear the STATUSPPKT bit of HOST_CSR0 together with (i.e., in the same write operation as) RxPktRdy if this has been set.

**Figure 16-11. Flow Chart of Status Stage of Zero Data Request or Write Request of a Control Transfer in Host Mode**




#### *16.2.8.2.1.5 Status Phase for a Read Request of a Control Transaction: Host Mode*

OUT Status Phase of a control transfer exist for a Read Request or a control transfer where data was received by the host controller. The OUT Status phase follows the IN Data Stage of a control transfer.

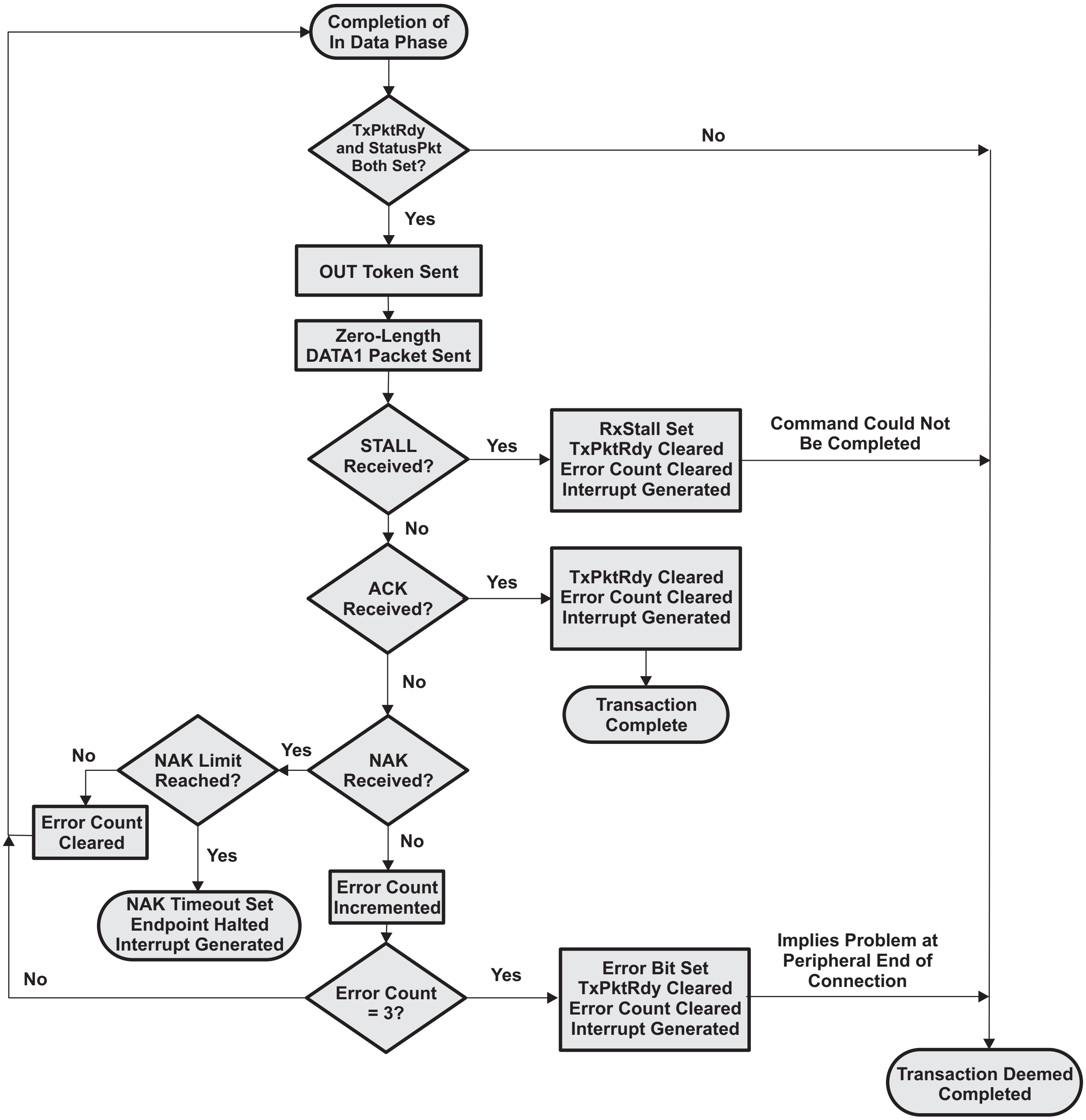
For the OUT Status Phase of a control transaction (Figure 16-12), the CPU driving the host device needs to:

1. Set STATUSPKT and TXPKTRDY bits of HOST_CSR0 (bit 6 and bit 1, respectively).

   NOTE: These bits need to be set together
2. Wait while the controller sends the OUT token and a zero-length DATA1 packet.
3. At the end of the attempt to send the data, the controller will generate an Endpoint 0 interrupt. The software should then read HOST_CSR0 to establish whether the RXSTALL bit (bit 2), the ERROR bit (bit 4) or the NAK_TIMEOUT bit (bit 7) has been set.

   If RXSTALL bit is set, it indicates that the target could not complete the command and so has issued a STALL response.
   If ERROR bit is set, it means that the controller has tried to send the STATUS Packet and the following data packet three times without getting any response.
   If NAK_TIMEOUT bit is set, it means that the controller has received a NAK response to each attempt to send the IN token, for longer than the time set in the HOST_NAKLIMIT0 register. The controller can then be directed either to continue trying this transaction (until it times out again) by clearing the NAK_TIMEOUT bit or to abort the transaction by flushing the FIFO before clearing the NAK_TIMEOUT bit.
4. If none of RXSTALL, ERROR or NAK_TIMEOUT bits is set, the STATUS Phase has been correctly ACKed.

**Figure 16-12. Chart of Status Stage of a Read Request of a Control Transfer in Host Mode**




#### *16.2.8.2.2 Bulk Transfer: Host Mode*

Bulk transactions are handled by endpoints other than endpoint 0. It is used to handle non-periodic, large bursty communication typically used for a transfer that use any available bandwidth and can also be delayed until bandwidth is available.

##### *16.2.8.2.2.1 Bulk IN Transactions: Host Mode*

A Bulk IN transaction may be used to transfer non-periodic data from the external USB peripheral to the host.

The following optional features are available for use with an Rx endpoint used in host mode to receive the data:

- Double packet buffering: When enabled, up to two packets can be stored in the FIFO on reception from the host. This allows that one packet can be received while another is being read. Double packet buffering is enabled by setting the DPB bit of RXFIFOSZ register (bit 4).
- DMA: If DMA is enabled for the endpoint, a DMA request will be generated whenever the endpoint has a packet in its FIFO. This feature can be used to allow the DMA controller to unload packets from the FIFO without processor intervention.
  When DMA is enabled, endpoint interrupt will not be generated for completion of packet reception. Endpoint interrupt will be generated only in the error conditions. For more information see section on CPPI DMA

#### *16.2.8.2.2.1.1 Bulk IN Setup: Host Mode*

Before initiating any Bulk IN Transactions in Host mode:

- The HOST_RXTYPE register for the endpoint that is to be used needs to be programmed as:
  - Operating speed in the SPEED bit field (bits 7 and 6).
  - Set 10 (binary value) in the PROT field for bulk transfer.
  - Endpoint Number of the target device in RENDPN field. This is the endpoint number contained in the Rx endpoint descriptor returned by the target device during enumeration.
- The RXMAXP register for the controller endpoint must be written with the maximum packet size (in bytes) for the transfer. This value should be the same as the wMaxPacketSize field of the Standard Endpoint Descriptor for the target endpoint.
- The HOST_RXINTERVAL register needs to be written with the required value for the NAK limit (2-215 frames/microframes), or set to zero if the NAK timeout feature is not required.
- The relevant interrupt enable bit in the INTRRXE register should be set (if an interrupt is required for this endpoint).

  **Note:** Whether interrupt is handled from PDR level or Core level, interrupt is required to be enabled at the core level. See details on Core Interrupt registers, CTRLR register, and sections on Interrupt Handling
- The AUTOREQ bit field should be used only when servicing transfers using CPU mode and must be cleared when using DMA Mode. For DMA Mode, dedicated registers USB0/1 Req Registers exist and the HOST_RXCSR[AUTOREQ] should be cleared.
  When DMA is enabled, the following bits of HOST_RXCSR register should be set as:
  - Clear AUTOCLEAR.
  - Set DMAEN (bit 13) to 1 if a DMA request is required for this endpoint.
  - Clear DSINYET (bit 12) to 0 to allow normal PING flow control. This will affect only High Speed transactions.
  - Clear DMAMODE (bit 11) to 0.

    **Note:** If DMA is enabled, the USB0/1 Auto Req register can be set for generating IN tokens automatically after receiving the data. Set the bit field RXn_AUTOREQ (where n is the endpoint number) with binary value 01 or 11.

When the endpoint is first configured, the endpoint data toggle should be cleared to 0 either by using the DATATOGWREN and DATATOG bits of HOST_RXCSR (bit 10 and bit 9) to toggle the current setting or by setting the CLRDATATOG bit of HOST_RXCSR (bit 7). This will ensure that the data toggle (which is handled automatically by the controller) starts in the correct state. Also if there are any data packets in the FIFO (indicated by the RXPKTRDY bit (bit 0 of HOST_RXCSR) being set), they should be flushed by setting the FLUSHFIFO bit of HOST_RXCSR (bit 4).

**NOTE:** It may be necessary to set this bit twice in succession if double buffering is enabled.

##### *16.2.8.2.2.1.2 Bulk IN Operation: Host Mode*

When Bulk data is required from the USB peripheral device, the software should set the REQPKT bit in the corresponding HOST_RXCSR register (bit 5). The controller will then send an IN token to the selected peripheral endpoint and waits for data to be returned.

If data is correctly received, RXPKTRDY bit of HOST_RXCSR (bit 0) is set. If the USB peripheral device responds with a STALL, RXSTALL bit (bit 6 of HOST_RXCSR) is set. If a NAK is received, the controller tries again and continues to try until either the transaction is successful or the POLINTVL_NAKLIMIT set in the HOST_RXINTERVAL register is reached. If no response at all is received, two further attempts are made before the controller reports an error by setting the ERROR bit of HOST_RXCSR (bit 2).

The controller then generates the appropriate endpoint interrupt, whereupon the software should read the corresponding HOST_RXCSR register to determine whether the RXPKTRDY, RXSTALL, ERROR or DATAERR_NAKTIMEOUT bit is set and act accordingly. If the DATAERR_NAKTIMEOUT bit is set, the controller can be directed either to continue trying this transaction (until it times out again) by clearing the DATAERR_NAKTIMEOUT bit or to abort the transaction by clearing REQPKT bit before clearing the DATAERR_NAKTIMEOUT bit.

The packets received should not exceed the size specified in the RXMAXP register (as this should be the value set in the wMaxPacketSize field of the endpoint descriptor sent to the host).

In the general case, the application software (if CPU is servicing the endpoint) will need to read each packet from the FIFO individually. If large blocks of data are being transferred, the overhead of calling an interrupt service routine to unload each packet can be avoided by using DMA.

**Note:** When using DMA, see Section 16.2.9, *Communications Port Programming Interface (CPPI) 4.1 DMA*, for the proper configuration of the core register, HOST_RXCSR.

##### *16.2.8.2.2.1.3 Bulk IN Error Handling: Host Mode*

If the target wants to shut down the Bulk IN pipe, it will send a STALL response to the IN token. This will result in the RXSTALL bit of HOST_RXCSR (bit 6) being set.

#### *16.2.8.2.2.2 Bulk OUT Transactions: Host Mode*

A Bulk OUT transaction may be used to transfer non-periodic data from the host to the USB peripheral.

Following optional features are available for use with a Tx endpoint used in Host mode to transmit this data:

- Double packet buffering: When enabled, up to two packets can be stored in the FIFO awaiting transmission to the peripheral device. Double packet buffering is enabled by setting the DPB bit of TXFIFOSZ register (bit 4).
- DMA: If DMA is enabled for the endpoint, a DMA request will be generated whenever the endpoint is able to accept another packet in its FIFO. This feature can be used to allow the DMA controller to load packets into the FIFO without processor intervention. For more information on using DMA, consult the section discussing CPPI DMA.
  When DMA is enabled and DMAMODE bit in HOST_TXCSR register is set, an endpoint interrupt will not be generated for completion of packet reception. An endpoint interrupt will be generated only in the error conditions.

#### 16.2.8.2.2.2.1 Bulk OUT Setup: Host Mode

Before initiating any bulk OUT transactions:

- The target function address needs to be set in the TXFUNCADDR register for the selected controller endpoint. (TXFUNCADDR register is available for all endpoints from EP0 to EP15.)
- The HOST_TXTYPE register for the endpoint that is to be used needs to be programmed as:
  - Operating speed in the SPEED bit field (bits 7 and 6).
  - Set PROT field to 10b for bulk transfer.
  - Enter Endpoint Number of the target device in TENDPN field. This is the endpoint number contained in the OUT(Tx) endpoint descriptor returned by the target device during enumeration.
- The TXMAXP register for the controller endpoint must be written with the maximum packet size (in bytes) for the transfer. This value should be the same as the wMaxPacketSize field of the Standard Endpoint Descriptor for the target endpoint.
- The HOST_TXINTERVAL register needs to be written with the required value for the NAK limit (2-215 frames/microframes), or set to zero if the NAK timeout feature is not required.
- The relevant interrupt enable bit in the INTRTXE register should be set (if an interrupt is required for this endpoint).
- The following bits of HOST_TXCSR register should be set as:
  - Set the MODE bit (bit 13) to 1 to ensure the FIFO is enabled (only necessary if the FIFO is shared with an Rx endpoint).
  - Clear the FRCDATATOG bit (bit 11) to 0 to allow normal data toggle operations.
  - Clear/Set AUTOSET bit (bit 15). The setting of this bit depends on the desire of the user/application in automatically setting the TXPKTRDY bit when servicing transactions using CPU.

  **Note:** If DMA is needs to be used in place of the CPU, the following table displays the setting of the core register HOST_TXCSR register in Host mode. For the CPPI DMA registers settings consult the section on CPPI DMA within this document.

| Bit Field | Bit Name | Description |
|---|---|---|
| Bit 15 | AUTOSET | Cleared to 0 if using DMA. For CPU Mode use, if AUTOSET bit is set, the TXPKTRDY bit will be automatically set when data of the maximum packet size is loaded into the FIFO. |
| Bit 14 | ISO | Cleared to 0 for bulk mode operation. |
| Bit 13 | MODE | Set to 1 to make sure the FIFO is enabled (only necessary if the FIFO is shared with an RX endpoint) |
| Bit 12 | DMAEN | Set to 1 to enable DMA usage; not needed if CPU is being used to service the Tx Endpoint |
| Bit 11 | FRCDATATOG | Cleared to 0 to allow normal data toggle operations. |
| Bit 10 | DMAMODE | Set to 1 when DMA is used to service Tx FIFO. |

When the endpoint is first configured, the endpoint data toggle should be cleared to 0 either by using the DATATOGWREN bit and DATATOG bit of HOST_TXCSR (bit 9 and bit 8) to toggle the current setting or by setting the CLRDATATOG bit of HOST_TXCSR (bit 6). This will ensure that the data toggle (which is handled automatically by the controller) starts in the correct state. Also, if there are any data packets in the FIFO (indicated by the FIFONOTEMPTY bit of HOST_TXCSR register (bit 1) being set), they should be flushed by setting the FLUSHFIFO bit (bit 3 of HOST_TXCSR).

**NOTE:** It may be necessary to set this bit twice in succession if double buffering is enabled.

##### 16.2.8.2.2.2.2 Bulk OUT Operation: Host Mode

When Bulk data is required to be sent to the USB peripheral device, the software should write the first packet of the data to the FIFO (or two packets if double-buffered) and set the TXPKTRDY bit in the corresponding HOST_TXCSR register (bit 0). The controller will then send an OUT token to the selected peripheral endpoint, followed by the first data packet from the FIFO.

If data is correctly received by the peripheral device, an ACK should be received whereupon the controller will clear TXPKTRDY bit of HOST_TXCSR (bit 0). If the USB peripheral device responds with a STALL, the RXSTALL bit (bit 5) of HOST_TXCSR is set. If a NAK is received, the controller tries again and continues to try until either the transaction is successful or the NAK limit set in the HOST_TXINTERVAL register is reached. If no response at all is received, two further attempts are made before the controller reports an error by setting ERROR bit in HOST_TXCSR (bit 2).

The controller then generates the appropriate endpoint interrupt, whereupon the software should read the corresponding HOST_TXCSR register to determine whether the RXSTALL (bit 5), ERROR (bit 2) or NAK_TIMEOUT (bit 7) bit is set and act accordingly. If the NAK_TIMEOUT bit is set, the controller can be directed either to continue trying this transaction (until it times out again) by clearing the NAK_TIMEOUT bit or to abort the transaction by flushing the FIFO before clearing the NAK_TIMEOUT bit.

If large blocks of data are being transferred, then the overhead of calling an interrupt service routine to load each packet can be avoided by using DMA.

##### 16.2.8.2.2.2.3 Bulk OUT Error Handling: Host Mode

If the target wants to shut down the Bulk OUT pipe, it will send a STALL response. This is indicated by the RXSTALL bit of HOST_TXCSR register (bit 5) being set.

##### 16.2.8.2.3 Interrupt Transfer: Host Mode

When the controller is operating as the host, interactions with an Interrupt endpoint on the USB peripheral device are handled in very much the same way as the equivalent Bulk transactions (described in previous sections).

The principal difference as far as operational steps are concerned is that the PROT field of HOST_RXTYPE and HOST_TXTYPE (bits 5-4) need to be set (binary value) to represent an Interrupt transaction. The required polling interval also needs to be set in the HOST_RXINTERVAL and HOST_TXINTERVAL registers.

##### 16.2.8.2.4 Isochronous Transfer: Host Mode

Isochronous transfers are used when working with isochronous data. Isochronous transfers provide periodic, continuous communication between host and device.

An Isochronous IN transaction is used to transfer periodic data from the USB peripheral to the host.

##### 16.2.8.2.4.1 Isochronous IN Transactions: Host Mode

The following optional features are available for use with an Rx endpoint used in Host mode to receive this data:

- Double packet buffering: When enabled, up to two packets can be stored in the FIFO on reception from the host. This allows that one packet can be received while another is being read. Double packet buffering is enabled by setting the DPB bit of RXFIFOSZ register (bit 4).
- AutoRequest: When the AutoRequest feature is enabled, the REQPKT bit of HOST_RXCSR (bit 5) will be automatically set when the RXPKTRDY bit is cleared. This only applies when the CPU is being used to service the endpoint. When using DMA, this bit field needs to be cleared to Zero. CPPI DMA has its own configuration registers that renders a similar task, USB0/1 Auto Req registers, that needs to be used to have a similar effect. For more information, see Section 16.2.9, *Communications Port Programming Interface (CPPI) 4.1 DMA*.
- DMA: If DMA is enabled for the endpoint, a DMA request will be generated whenever the endpoint has a packet in its FIFO. This feature can be used to allow the DMA controller to unload packets from the FIFO without processor intervention. However, this feature is not particularly useful with isochronous

endpoints because the packets transferred are often not maximum packet size.

- When DMA is enabled, endpoint interrupt will not be generated for completion of packet reception. Endpoint interrupt will be generated only in the error conditions.

##### 16.2.8.2.4.1.1 Isochronous IN Transfer Setup: Host

Before initiating an Isochronous IN Transactions in Host mode:

- The target function address needs to be set in the RXFUNCADDR register for the selected controller endpoint (RXFUNCADDR register is available for all endpoints from EP0 to EP4).
- The HOST_RXTYPE register for the endpoint that is to be used needs to be programmed as:
  – Operating speed in the SPEED bit field (bits 7 and 6).
  – Set 01 (binary value) in the PROT field for isochronous transfer.
  – Endpoint Number of the target device in RENDPN field. This is the endpoint number contained in the Rx endpoint descriptor returned by the target device during enumeration.
- The RXMAXP register for the controller endpoint must be written with the maximum packet size (in bytes) for the transfer. This value should be the same as the wMaxPacketSize field of the Standard Endpoint Descriptor for the target endpoint.
- The HOST_RXINTERVAL register needs to be written with the required transaction interval (usually one transaction per frame/microframe).
- The relevant interrupt enable bit in the INTRRXE register should be set (if an interrupt is required for this endpoint).
- The following bits of HOST_RXCSR register should be set as:
  – Clear AUTOCLEAR
  – Set the DMAEN bit (bit 13) to 1 if a DMA request is required for this endpoint.
  – Clear the DISNYET it (bit 12) to 0 to allow normal PING flow control. This will only affect High Speed transactions.
  – Clear DMAMODE bit (bit 11) to 0.
- If DMA is enabled, AUTOREQ register can be set for generating IN tokens automatically after receiving the data. Set the bit field RXn_AUTOREQ (where n is the endpoint number) with binary value 01 or 11. For detailed information on using CPPI DMA, consult related section within this document.

##### 16.2.8.2.4.1.2 Isochronous IN Operation: Host Mode

The operation starts with the software setting REQPKT bit of HOST_RXCSR (bit 5). This causes the controller to send an IN token to the target.

When a packet is received, an interrupt is generated which the software may use to unload the packet from the FIFO and clear the RXPKTRDY bit in the HOST_RXCSR register (bit 0) in the same way as for a Bulk Rx endpoint. As the interrupt could occur almost any time within a frame(/microframe), the timing of FIFO unload requests will probably be irregular. If the data sink for the endpoint is going to some external hardware, it may be better to minimize the requirement for additional buffering by waiting until the end of each frame before unloading the FIFO. This can be done by using the SOF_PULSE signal from the controller to trigger the unloading of the data packet. The SOF_PULSE is generated once per frame(/microframe). The interrupts may still be used to clear the RXPKTRDY bit in HOST_RXCSR.

##### 16.2.8.2.4.1.3 Isochronous IN Error Handling: Host Mode

If a CRC or bit-stuff error occurs during the reception of a packet, the packet will still be stored in the FIFO but the DATAERR_NAKTIMEOUT bit of HOST_RXCSR (bit 3) is set to indicate that the data may be corrupt.

**Note:** The number of USB packets sent in any microframe will depend on the amount of data to be transferred, and is indicated through the PIDs used for the individual packets. If the indicated numbers of packets have not been received by the end of a microframe, the INCOMPRX bit in the HOST_RXCSR register will be set to indicate that the data in the FIFO is incomplete. Equally, if a packet of the wrong data type is received, then the PID Error bit is the HOST_RXCSR register will be set. In each case, an interrupt will, however, still be generated to allow the data that has been received to be read from the FIFO.

#### *16.2.8.2.4.2 Isochronous OUT Transactions: Host Mode*

An Isochronous OUT transaction may be used to transfer periodic data from the host to the USB peripheral.

Following optional features are available for use with a Tx endpoint used in Host mode to transmit this data:

- Double packet buffering: When enabled, up to two packets can be stored in the FIFO awaiting transmission to the peripheral device. Double packet buffering is enabled by setting the DPB bit of TXFIFOSZ register (bit 4).

DMA: If DMA is enabled for the endpoint, a DMA request will be generated whenever the endpoint is able to accept another packet in its FIFO. This feature can be used to allow the DMA controller to load packets into the FIFO without processor intervention.

However, this feature is not particularly useful with isochronous endpoints because the packets transferred are often not maximum packet size.

When DMA is enabled and endpoint interrupt will not be generated for completion of packet reception. Endpoint interrupt will be generated only in the error conditions.

##### *16.2.8.2.4.2.1 Isochronous OUT Transfer Setup: Host Mode*

Before initiating any Isochronous OUT transactions:

- The target function address needs to be set in the TXFUNCADDR register for the selected controller endpoint (TXFUNCADDR register is available for all endpoints from EP0 to EP4).
- The HOST_TXTYPE register for the endpoint that is to be used needs to be programmed as:
  – Operating speed in the SPEED bit field (bits 7 and 6).
  – Set 01 (binary value) in the PROT field for isochronous transfer.
  – Endpoint Number of the target device in TENDPN field. This is the endpoint number contained in the OUT(Tx) endpoint descriptor returned by the target device during enumeration.
- The TXMAXP register for the controller endpoint must be written with the maximum packet size (in bytes) for the transfer. This value should be the same as the wMaxPacketSize field of the Standard Endpoint Descriptor for the target endpoint.
- The HOST_TXINTERVAL register needs to be written with the required transaction interval (usually one transaction per frame/microframe).
- The relevant interrupt enable bit in the INTRTXE register should be set (if an interrupt is required for this endpoint).
- The following bits of HOST_TXCSR register should be set as:
  – Set the MODE bit (bit 13) to 1 to ensure the FIFO is enabled (only necessary if the FIFO is shared with an Rx endpoint).
  – Set the DMAEN bit (bit 12) to 1 if a DMA request is required for this endpoint
  – The FRCDATATOG bit (bit 11) is ignored for isochronous transactions.
  – Set the DMAMODE bit (bit 10) to 1 when DMA is enabled.
    For more details in using DMA, consult CPPI DMA section within this document.

##### *16.2.8.2.4.2.2 Isochronous OUT Transfer Operation: Host Mode*

The operation starts when the software writes to the FIFO and sets TXPKTRDY bit of HOST_TXCSR (bit 0). This triggers the controller to send an OUT token followed by the first data packet from the FIFO.

An interrupt is generated whenever a packet is sent and the software may use this interrupt to load the next packet into the FIFO and set the TXPKTRDY bit in the HOST_TXCSR register (bit 0) in the same way as for a Bulk Tx endpoint. As the interrupt could occur almost any time within a frame, depending on when the host has scheduled the transaction, this may result in irregular timing of FIFO load requests. If the data source for the endpoint is coming from some external hardware, it may be more convenient to wait until the end of each frame before loading the FIFO as this will minimize the requirement for additional buffering. This can be done by using the SOF_PULSE signal from the controller to trigger the loading of the next data packet. The SOF_PULSE is generated once per frame(/microframe). The interrupts may still be used to set the TXPKTRDY bit in HOST_TXCSR.

###### 16.2.8.2.4.3 High Bandwidth Isochronous Transactions: Host Mode

**Place Holder: Details to be populated.**

### 16.2.9 Communications Port Programming Interface (CPPI) 4.1 DMA

The CPPI DMA module supports the transmission and reception of USB packets. The CPPI DMA is designed to facilitate the segmentation and reassembly of CPPI compliant packets to/from smaller data blocks that are natively compatible with the specific requirements of each networking port. Multiple Tx and Rx channels are provided for all endpoints (excluding endpoint 0) within the DMA allowing multiple segmentation or reassembly operations to be effectively performed in parallel (but not actually simultaneously). The DMA controller maintains state information for each of the ports/channels which allows packet segmentation and reassembly operations to be time division multiplexed between channels in order to share the underlying DMA hardware. A DMA scheduler is used to control the ordering and rate at which this multiplexing occurs.

The CPPI (version 4.1) DMA controller sub-module is a common 15 port DMA controller. It supports 15 Tx and 15 Rx channels and each port attaches to the associated endpoint in the controller. Port 1 maps to endpoint 1 and Port 2 maps to endpoint 2 and so on with Port 15 mapped to endpoint 15; endpoint 0 can not utilize the DMA and the firmware is responsible to load or offload the endpoint 0 FIFO via CPU.

#### 16.2.9.1 CPPI Terminology

**Host** — The host is an intelligent system resource that configures and manages each communications control module. The host is responsible for allocating memory, initializing all data structures, and responding to port interrupts.

**Main Memory** — The area of data storage managed by the CPU. The CPPI DMA (CDMA) reads and writes CPPI packets from and to main memory. This memory can exist internal or external from the device.

**Queue Manager (QM)** — The QM is responsible for accelerating management of a variety of Packet Queues and Free Descriptor / Buffer Queues. It provides status indications to the CDMA Scheduler when queues are empty or full.

**CPPI DMA (CDMA)** — The CDMA is responsible for transferring data between the CPPI FIFO and Main Memory. It acquires free Buffer Descriptor from the QM (Receive Submit Queue) for storage of received data, posts received packets pointers to the Receive Completion Queue, transmits packets stored on the Transmit Submit Queue (Transmit Queue) , and posts completed transmit packets to the Transmit Completion Queue.

**CDMA Scheduler (CDMAS)** — The CDMAS is responsible for scheduling CDMA transmit and receive operations. It uses Queue Indicators from the QM and the CDMA to determine the types of operations to schedule.

**CPPI FIFO** — The CPPI FIFO provides FIFO interfaces (for each of the 15 transmit and 15 receive endpoints).

**Transfer DMA (XDMA)** — The XDMA receives DMA requests from the Mentor USB 2.0 Core and initiates DMAs to the CPPI FIFO.

**Endpoint FIFOs** — The Endpoint FIFOs are the USB packet storage elements used by the Mentor USB 2.0 Core for packet transmission or reception. The XDMA transfers data between the CPPI FIFO and the Endpoint FIFOs for transmit operations and between the Endpoint FIFOs and the CPPI FIFO for receive operations.

**Port** — A port is the communications module (peripheral hardware) that contains the control logic for Direct Memory Access for a single transmit/receive interface or set of interfaces. Each port may have multiple communication channels that transfer data using homogenous or heterogeneous protocols. A port is usually subdivided into transmit and receive pairs which are independent of each other. Each endpoint, excluding endpoint 0, has its own dedicated port.

**Channel** — A channel refers to the sub-division of information (flows) that is transported across ports. Each channel has associated state information. Channels are used to segregate information flows based on the protocol used, scheduling requirements (example: CBR, VBR, ABR), or concurrency requirements (that is, blocking avoidance). All fifteen ports per USB Module have dedicated single channels, channel 0, associated for their use in a USB application.

**Data Buffer** — A data buffer is a single data structure that contains payload information for transmission to or reception from a port. A data buffer is a byte aligned contiguous block of memory used to store packet payload data. A data buffer may hold any portion of a packet and may be linked together (via descriptors) with other buffers to form packets. Data buffers may be allocated anywhere within the 32-bit memory space. The Buffer Length field of the packet descriptor indicates the number of valid data bytes in the buffer. There may be from 1 to 4M-1 valid data bytes in each buffer.

**Host Buffer Descriptor (Buffer Descriptor)** — A buffer descriptor is a single data structure that contains information about one or more data buffers. This type of descriptor is required when more than one descriptor is needed to define an entire packet, i.e., it either defines the middle of a packet or end of a packet.

**Start of Packet (SOP)** — Packet along side Buffer Descriptors are used to hold information about Packets. When multiple Descriptors are used to hold a single packet information or a single Descriptor is used to hold a single packet information, the first descriptor is referred to as a Packet Descriptor which is also Start-of-Packet Descriptor.

**Host Packet Descriptor (Packet Descriptor)** — A packet descriptor is another name for the first buffer descriptor within a packet. Some fields within a data buffer descriptor are only valid when it is a packet descriptor including the tags, packet length, packet type, and flags. This type of descriptor is always used to define a packet since it provides packet level information that is useful to both the ports and the Host in order to properly process the packet. It is the only descriptor used when single descriptor solely defines a packet. When multiple descriptors are needed to define a packet, the packet descriptor is the first descriptor used to define a packet.

**Free Descriptor/Buffer Queue** — A free descriptor/buffer queue is a hardware managed list of available descriptors with pre-linked empty buffers that are to be used by the receive ports for host type descriptors. Free Descriptor/Buffer Queues are implemented by the Queue Manager.

**Teardown Descriptor** — Teardown Descriptor is a special structure which is not used to describe either a packet or a buffer but is instead used to describe the completion of a channel halt and teardown event. Channel teardown is an important function because it ensures that when a connection is no longer needed that the hardware can be reliably halted and any remaining packets which had not yet been transmitted can be reclaimed by the Host without the possibility of losing buffer or descriptor references (which results in a memory leak).

**Packet Queue** — A packet queue is hardware managed list of valid (i.e. populated) packet descriptors that is used for forwarding a packet from one entity to another for any number of purposes.
**NOTE:** All descriptors (regardless of type) must be allocated at addresses that are naturally aligned to the smallest power of 2 that is equal to or greater than the descriptor size.

### 16.2.9.2 Data Structures

Two data structures are mainly used to identify data buffers used by packet and buffer descriptors. A third Descriptor, Teardown Descriptor, exists. The purpose of this Descriptor is a channel halt and teardown event. Each of these Descriptor layouts as well as bit fields are shown below.

#### *16.2.9.2.1 Host Packet Descriptor/ Packet Descriptor (SOP Descriptor)*

Descriptors are designed to be used when USB like application requires support for true, unlimited fragment count scatter/gather type operations. The Packet Descriptor is the first descriptor on multiple descriptors setup or the only descriptor in a single descriptors setup. The Packet Descriptor contains the following information:

- Indicator which identifies the descriptor as a packet descriptor (always 10h
- Source and destination tags (reserved)
- Packet type
- Packet length
- Protocol specific region size
- Protocol specific control/statusbits
- Pointer to the first valid byte in the SOP data buffer

- Length of the SOP data buffer
- Pointer to the next buffer descriptor in the

Packet descriptors can vary in size by design of their defined fields from 32 bytes up to 104 bytes. Within this range, packet descriptors always contain 32 bytes of required information and may also contain 8 bytes of software specific tagging information and up to 64 bytes (indicated in 4 byte increments) of protocol specific information. How much protocol specific information (and therefore the allocated size of the descriptors) is application dependent. Port will make use of the first 32 bytes only.

From a general USB use perspective, a 32-byte descriptor size is suffix and the use of this size is expected for a normal USB usage.

The packet descriptor layout is shown in Figure 16-13 and described within Table 16-9 through Table 16-16.

**Figure 16-13. Packet Descriptor Layout**

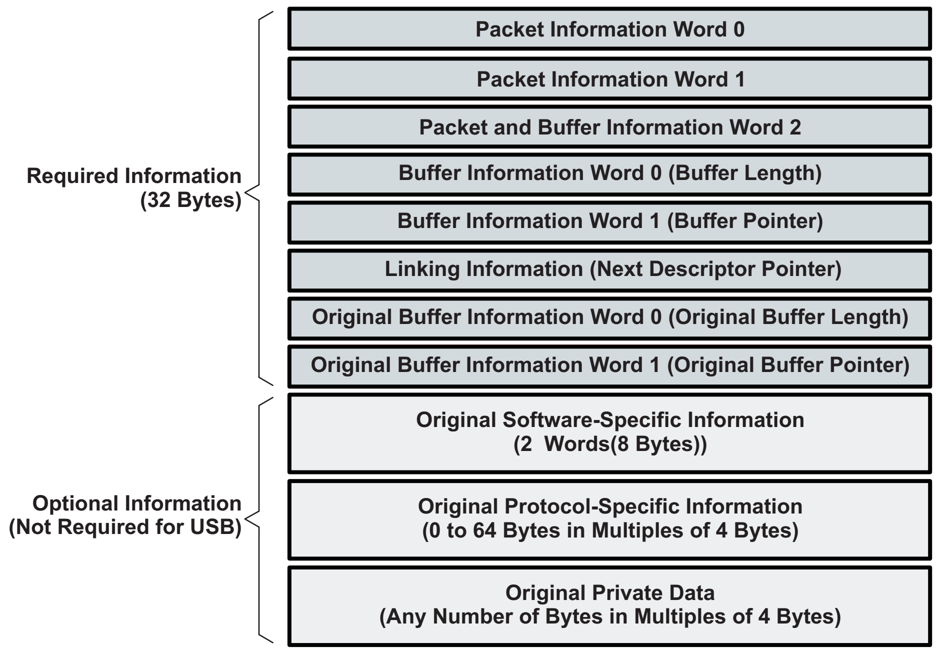



**Table 16-9. Packet Descriptor Word 0 (PD0) Bit Field Descriptions**

| Bits | Field Name | Description |
|---|---|---|
| 31-27 | Descriptor type | The host packet descriptor type is 16 decimal (10h). The CPU initializes this field. |
| 26-22 | Protocol-specific valid word count | This field indicates the valid number of 32-bit words in the protocol-specific region. The CPU initializes this field. This is encoded in increments of 4 bytes as:<br>0 = 0 byte<br>1 = 4 bytes<br>...<br>16 = 64 bytes<br>17-31 = Reserved |
| 21-0 | Packet length | The length of the packet in bytes. If the packet length is less than the sum of the buffer lengths, then the packet data will be truncated. A packet length greater than the sum of the buffers is an error. The valid range for the packet length is 0 to (4M - 1) bytes. The CPU initializes this field for transmitted packets; the DMA overwrites this field on packet reception. |

### Table 16-10. Packet Descriptor Word 1 (PD1) Bit Field Descriptions

| Bits | Field Name | Description |
|---|---|---|
| 31-27 | Source Tag; Port # | This field indicates the port number (0-31) from which the packet originated. The DMA overwrites this field on packet reception. This is the RX Endpoint number from which the packet originated. |
| 26-21 | Source Tag; Channel # | This field indicates the channel number within the port from which the packet originated. The DMA overwrites this field on packet reception. This field is always 0-67. |
| 20-16 | Source Tag: Sub-channel # | This field indicates the sub-channel number (0-31) within the channel from which the packet originated. The DMA overwrites this field on packet reception. This field is always 0. |
| 15-0 | Destination Tag | This field is application-specific. This field is always 0. |

### Table 16-11. Packet Descriptor Word 2 (PD2) Bit Field Descriptions

| Bits | Field Name | Description |
|---|---|---|
| 31 | Packet error | This bit indicates if an error occurred during reception of this packet (0 = no error occurred; 1 = error occurred). The DMA overwrites t his field on packet reception. Additional information about different errors may be encoded in the protocol specific fields in the descriptor. |
| 30-26 | | This field indicates the type of this packet. The CPU initializes this field for transmitted packets; the DMA overwrites this field on packet reception. This field is encoded as:<br>5 = USB<br>8-31 = Reserved |
| 25-20 | Reserved | Reserved |
| 19 | Zero-length packet indicator | If a zero-length USB packet is received, the XDMA will send the CDMA a data block with a byte count of 0 and this bit is set. The CDMA will then perform normal EOP termination of the packet without transferring data. For transmit, if a packet has this bit set, the XDMA will ignore the CPPI packet size and send a zero-length packet to the USB controller. |
| 18-16 | Protocol-specific | This field contains protocol-specific flags/information that can be assigned based on the packet type. Not used for USB. |
| 15 | Return policy | This field indicates the return policy for this packet. The CPU initializes this field.<br>0 = Entire packet (still linked together) should be returned to the queue specified in bits 11-0.<br>1 = Each buffer should be returned to the queue specified in bits 11-0 of Word 2 in their respective descriptors. The Tx DMA will return each buffer in sequence. |
| 14 | On-chip | This field indicates whether or not this descriptor is in a region which is in on-chip memory space (1) or in external memory (0). |
| 13-12 | Packet return queue mgr # | This field indicates which queue manager in the system the descriptor is to be returned to after transmission is complete. This field is not altered by the DMA during transmission or reception and is initialized by the CPU. There is only one queue manager in the USB HS/FS device controller. This field must always be 0. |
| 11-0 | Packet return queue # | This field indicates the queue number within the selected queue manager that the descriptor is to be returned to after transmission is complete. This field is not altered by the DMA during transmission or reception and is initialized by the CPU. |

### Table 16-12. Packet Descriptor Word 3 (PD3) Bit Field Descriptions

| Bits | Field Name | Description |
|---|---|---|
| 31-22 | Reserved | Reserved |
| 21-0 | Buffer 0 length | The buffer length field indicates how many valid data bytes are in the buffer. The CPU initializes this field for transmitted packets; the DMA overwrites this field on packet reception. |

**Table 16-13. Packet Descriptor Word 4 (PD4) Bit Field Descriptions**

| Bits | Field Name | Description |
|---|---|---|
| 31-0 | Buffer 0 pointer | The buffer pointer is the byte-aligned memory address of the buffer associated with the buffer descriptor. The CPU initializes this field for transmitted packets; the DMA overwrites this field on packet reception. |

**Table 16-14. Packet Descriptor Word 5 (PD5) Bit Field Descriptions**

| Bits | Field Name | Description |
|---|---|---|
| 31-0 | Next descriptor pointer | The 32-bit word aligned memory address of the next buffer descriptor in the packet. If the value of this pointer is zero, then the current buffer is the last buffer in the packet. The CPU initializes this field for transmitted packets; the DMA overwrites this field on packet reception. |

**Table 16-15. Packet Descriptor Word 6 (PD6) Bit Field Descriptions**

| Bits | Field Name | Description |
|---|---|---|
| 31-22 | Reserved | Reserved |
| 21-0 | Original buffer 0 length | The buffer length field indicates the original size of the buffer in bytes. This value is not overwritten during reception. This value is read by the Rx DMA to determine the actual buffer size as allocated by the CPU at initialization. Since the buffer length in Word 3 is overwritten by the Rx port during reception, this field is necessary to permanently store the buffer size information. |

**Table 16-16. Packet Descriptor Word 7 (PD7) Bit Field Descriptions**

| Bits | Field Name | Description |
|---|---|---|
| 31-22 | Reserved | Reserved |
| 21-0 | Original buffer 0 pointer | The buffer pointer is the byte-aligned memory address of the buffer associated with the buffer descriptor. This value is not overwritten during reception. This value is read by the Rx DMA to determine the actual buffer location as allocated by the CPU at initialization. Since the buffer pointer in Word 4 is overwritten by the Rx port during reception, this field is necessary to permanently store the buffer pointer information. |

#### *16.2.9.2.2 Host Buffer Descriptor/Buffer Descriptor (BD)*

The buffer descriptor is identical in size and organization to a Packet Descriptor but does not include valid information in the packet level fields and does not include a populated region for protocol specific information. The packet level fields are not needed since the SOP descriptor contains this information and additional copy of this data is not needed/necessary.

Host buffer descriptors are designed to be linked onto a host packet descriptor or another host buffer descriptor to provide support for unlimited scatter / gather type operations. Host buffer descriptors provide information about a single corresponding data buffer. Every host buffer descriptor stores the following information:

- Pointer to the first valid byte in the data
- Length of the data buffer
- Pointer to the next buffer descriptor in the packet

Buffer descriptors always contain 32 bytes of required information. Since it is a requirement that it is possible to convert a descriptor between a Buffer Descriptor and a Packet Descriptor (by filling in the appropriate fields) in practice, Buffer Descriptors will be allocated using the same sizes as Packet Descriptors. In addition, since the 5 LSBs of the Descriptor Pointers are used in CPPI 4.1 for the purpose of indicating the length of the descriptor, the minimum size of a descriptor is always 32 bytes

The buffer descriptor layout is shown in Figure 16-14 and described within Table 16-17 through Table 16-24 .

**Figure 16-14. Buffer Descriptor (BD) Layout**

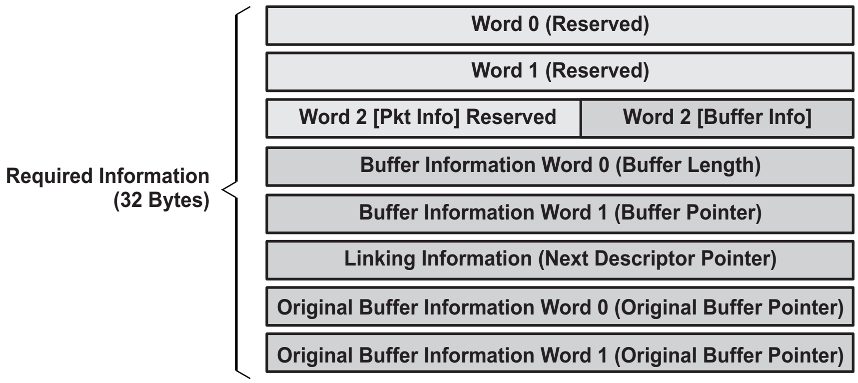



**Table 16-17. Buffer Descriptor Word 0 (BD0) Bit Field Descriptions**

| Bits | Field Name | Description |
|---|---|---|
| 31-0 | Reserved | Reserved |

**Table 16-18. Buffer Descriptor Word 1 (BD1) Bit Field Descriptions**

| Bits | Field Name | Description |
|---|---|---|
| 31-0 | Reserved | Reserved |

**Table 16-19. Buffer Descriptor Word 2 (BD2) Bit Field Descriptions**

| Bits | Field Name | Description |
|---|---|---|
| 31-15 | Reserved | Reserved |
| 14 | On-chip | This field indicates whether or not this descriptor is in a region which is on-chip memory space (1) or in external memory (0). |
| 13-12 | Packet return queue mgr # | This field indicates which queue manager in the system the descriptor is to be returned to after transmission is complete. This field is not altered by the DMA during transmission or reception and is initialized by the CPU. There is only 1 queue manager in the USB HS/FS device controller. This field must always be 0. |
| 11-0 | Packet return queue # | This field indicates the queue number within the selected queue manager that the descriptor is to be returned to after transmission is complete. This field is not altered by the DMA during transmission or reception and is initialized by the CPU. |

**Table 16-20. Buffer Descriptor Word 3 (BD3) Bit Field Descriptions**

| Bits | Field Name | Description |
|---|---|---|
| 31-22 | Reserved | Reserved |
| 21-0 | Buffer 0 length | The buffer length field indicates how many valid data bytes are in the buffer. The CPU initializes this field for transmitted packets. The DMA overwrites this field upon packet reception. |

**Table 16-21. Buffer Descriptor Word 4 (BD4) Bit Field Descriptions**

| Bits | Field Name | Description |
|---|---|---|
| 31-0 | Buffer 0 pointer | The buffer pointer is the byte-aligned memory address of the buffer associated with the buffer descriptor. The CPU initializes this field for transmitted packets. The DMA overwrites this field upon packet reception. |

**Table 16-22. Buffer Descriptor Word 5 (BD5) Bit Field Descriptions**

| Bits | Field Name | Description |
|---|---|---|
| 31-0 | Next description pointer | The 32-bit word aligned memory address of the next buffer descriptor in the packet. If the value of this pointer is zero, then the current descriptor is the last descriptor in the packet. The CPU initializes this field for transmitted packets. The DMA overwrites this field upon packet reception. |

**Table 16-23. Buffer Descriptor Word 6 (BD6) Bit Field Descriptions**

| Bits | Field Name | Description |
|---|---|---|
| 31-22 | Reserved | Reserved |
| 21-0 | Original buffer 0 length | The buffer length field indicates the original size of the buffer in bytes. This value is not overwritten during reception. This value is read by the Rx DMA to determine the actual buffer size as allocated by the CPU at initialization. Since the buffer length in Word 3 is overwritten by the Rx port during reception, this field is necessary to permanently store the buffer size information. |

**Table 16-24. Buffer Descriptor Word 7 (BD7) Bit Field Descriptions**

| Bits | Field Name | Description |
|---|---|---|
| 31-0 | Original buffer 0 pointer | The buffer pointer is the byte-aligned memory address of the buffer associated with the buffer descriptor. This value is not overwritten during reception. This value is read by the Rx DMA to determine the actual buffer location as allocated by the CPU at initialization. Since the buffer pointer in Word 4 is overwritten by the Rx port during reception, this field is necessary to permanently store the buffer pointer information. |

#### *16.2.9.2.3 Teardown Descriptor*

The Teardown Descriptor is not like the packet or buffer descriptors since it is not used to describe either a packet or a buffer. The teardown descriptor is always 32 bytes long and is comprised of 4 bytes of actual teardown information and 28 bytes of pad. The teardown descriptor layout and associated bit field descriptions are shown in the Figure below and Tables that follows. Since the 5 LSBs of the descriptor pointers are used in CPPI 4.1 for the purpose of indicating the length of the descriptor, the minimum size of a descriptor is 32 bytes.

The teardown descriptor is used to describe a channel halt and teardown event. Channel teardown ensures that when a connection is no longer needed that the hardware can be reliably halted and any remaining packets which had not yet been transmitted can be reclaimed by the host without the possibility of losing buffer or descriptor references (which results in a memory leak).

The teardown descriptor contains the following information:

- Indicator which identifies the descriptor as a teardown packet descriptor
- DMA controller number where teardown
- Channel number within DMA where teardown occurred
- Indicator of whether this teardown was for the Tx or Rx channel

**Figure 16-15. Teardown Descriptor Layout**

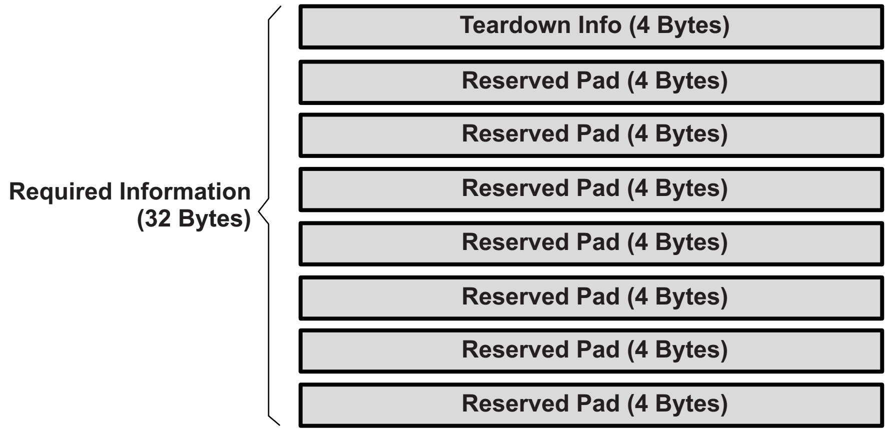



**Table 16-25. Teardown Descriptor Word 0 Bit Field Descriptions**

| Bits | Field Name | Description |
|---|---|---|
| 31-27 | Descriptor type | The teardown descriptor type is 19 decimal (13h) |
| 26-17 | Reserved | Reserved |
| 16 | TX_RX | Indicates whether teardown is a TX (0) or RX (1). |
| 15-10 | DMA number | Indicates the DMA number for this teardown. |
| 9-6 | Reserved | Reserved |
| 5-0 | Channel number | Indicates the channel number within the DMA that was torn down. |

**Table 16-26. Teardown Descriptor Words 1 to 7 Bit Field Descriptions**

| Bits | Field Name | Description |
|---|---|---|
| 31-0 | Reserved | Reserved |

Teardown operation of an endpoint requires three operations. The teardown register in the CPPI DMA must be written, the corresponding endpoint bit in TEARDOWN of the USB module must be set, and the FlushFIFO bit in the Mentor USB controller Tx/RxCSR register must be set.

The following is the Transmit teardown procedure highlighting the steps required to be followed:

1. Set the TX_TEARDOWN bit in the CPPI DMA TX channel n global configuration register (TXGCRn).
2. Set the appropriate TX_TDOWN bit in the USBOTG controller's USB teardown register (TEARDOWN). Write Tx Endpoint Number to teardown to TEARDOWN[TX_TDOWN] field.
3. Check if the teardown descriptor has been received on the teardown queue: The completion queue (see Queue Assignment table) is usually used as the Teardown Completion queue when the Teardown descriptor has been received, the descriptor address will be loaded onto CTRLD[Completion Queue #] register:
   a. If not, go to step 2
   b. If so, go to step 4
4. Set the appropriate TX_TDOWN bit in the USBOTG controller's USB teardown register (TEARDOWN). Set the bit corresponding to the Channel Number within TEARDOWN[TX_TDOWN] field.
5. Flush the TX FIFO in the Mentor OTG core: Set PERI_TXCSR[FLUSHFIFO] for the corresponding Endpoint.
6. Re-enable the Tx DMA channel.
   a. Clear TXGCRn[TX_TEARDOWN and TX_ENABLE] bit.
   b. Set TXGCRn[TX_ENABLE] bit.

### 16.2.9.3 Queue Manager

The queue manager (QM) is a hardware module that is responsible for accelerating management of the queues, i.e. queues are maintained within a queue manager module. Packets are added to a queue by writing the 32-bit descriptor address to a particular memory mapped location in the queue manager module. Packets are de-queued by reading the same location for that particular queue. A single queue manager exists within the device and handles all available 156 queues within the USB subsystem

Descriptors are queued onto a logical queue (pushed) by writing the descriptor pointer to the Queue N Register D. When the Queue N Register D is written, this kicks off the queue manager causing it to add the descriptor to the tail of queue.

The QM keeps track of the order of the descriptors pushed within a queue or the linking status of the descriptors. To accomplish this linking status, QM will first resolve the 32-bit descriptor pointer into a 16-bit index which is used for linking a queue pointer purposes. Once the address to index computation is performed, i.e., the physical index information is determined, the QM will link that descriptor onto the descriptor chain that is maintained for that logical queue by writing the linking information out to a linking RAM which is external to the queue manager. More on linking RAM discussion would follow in latter sections. The QM will also update the queue tail pointers appropriately. Since logical queues within the queue manager are maintained using linked lists, queues cannot become full and no check for fullness is required before queuing a packet descriptor.

#### *16.2.9.3.1 Queue Types*

Several types of queues exist (a total of 156 queues) within the CPPI 4.1 DMA. Regardless of the type of queue, queues are used to hold pointers to Packet or Buffer Descriptors while they are being passed between the Host and / or any of the ports in the system. All queues are maintained within the single Queue Manager module.

The following types of Queues exist:

- Free descriptor queue (unassigned to specific endpoint but assigned to specific endpoint type; receive endpoints – can be used as a receive submit queue)
- Transmit submit queue
- Transmit completion (return) queue
- Receive completion (return) queue
- Teardown queue

Dedicated queues exist where one or more queues are assigned for the use of a single endpoint and non-dedicated queues exist where no specific queue assignment to an endpoint is required (but pertains to receive endpoints only). Transmit endpoints are not allowed to use free descriptor queues.

Dedicated queues exist for each specific endpoint use and non-dedicated queues exist that can be used by any/all receive endpoints. Three queues are reserved for each endpoint/port for transmit actions with two of the three queues being transmit submit queues while the remaining queue is used as a completion/return queue. For receive actions, the only dedicated queues are completion/return queues; one queue is assigned/reserved for each receive endpoint. 32 free descriptor queues that do not have dedicated endpoint-queue assignment exist and these queues are used to service any receive endpoint as receive submit queues.

Table 16-27 displays queue-endpoint assignments.

**Table 16-27. Queue-Endpoint Assignments**

| Queue Start Number | Queue Count | Queue-Endpoint Association |
|---|---|---|
| 0 | 32 | Free Descriptor Queues/Rx Submit Queues (USB0/1 Rx Endpoints) |
| 32 | 2 | USB0 Tx Endpoint 1 |
| 34 | 2 | USB0 Tx Endpoint 2 |
| 36 | 2 | USB0 Tx Endpoint 3 |
| 38 | 2 | USB0 Tx Endpoint 4 |

### Table 16-27. Queue-Endpoint Assignments (continued)

| Queue Start Number | Queue Count | Queue-Endpoint Association |
|---|---|---|
| 40 | 2 | USB0 Tx Endpoint 5 |
| 42 | 2 | USB0 Tx Endpoint 6 |
| 44 | 2 | USB0 Tx Endpoint 7 |
| 46 | 2 | USB0 Tx Endpoint 8 |
| 48 | 2 | USB0 Tx Endpoint 9 |
| 50 | 2 | USB0 Tx Endpoint 10 |
| 52 | 2 | USB0 Tx Endpoint 11 |
| 54 | 2 | USB0 Tx Endpoint 12 |
| 56 | 2 | USB0 Tx Endpoint 13 |
| 58 | 2 | USB0 Tx Endpoint 14 |
| 60 | 2 | USB0 Tx Endpoint 15 |
| 62 | 2 | USB1 Tx Endpoint 1 |
| 64 | 2 | USB1 Tx Endpoint 2 |
| 66 | 2 | USB1 Tx Endpoint 3 |
| 68 | 2 | USB1 Tx Endpoint 4 |
| 70 | 2 | USB1 Tx Endpoint 5 |
| 72 | 2 | USB1 Tx Endpoint 6 |
| 74 | 2 | USB1 Tx Endpoint 7 |
| 76 | 2 | USB1 Tx Endpoint 8 |
| 78 | 2 | USB1 Tx Endpoint 9 |
| 80 | 2 | USB1 Tx Endpoint 10 |
| 82 | 2 | USB1 Tx Endpoint 11 |
| 84 | 2 | USB1 Tx Endpoint 12 |
| 86 | 2 | USB1 Tx Endpoint 13 |
| 88 | 2 | USB1 Tx Endpoint 14 |
| 90 | 2 | USB1 Tx Endpoint 15 |
| 92 | 1 | Reserved |
| 93 | 1 | USB0 Tx Endpoint 1 completion queue |
| 94 | 1 | USB0 Tx Endpoint 2 completion queue |
| 95 | 1 | USB0 Tx Endpoint 3 completion queue |
| 96 | 1 | USB0 Tx Endpoint 4 completion queue |
| 97 | 1 | USB0 Tx Endpoint 5 completion queue |
| 98 | 1 | USB0 Tx Endpoint 6 completion queue |
| 99 | 1 | USB0 Tx Endpoint 7 completion queue |
| 100 | 1 | USB0 Tx Endpoint 8 completion queue |
| 101 | 1 | USB0 Tx Endpoint 9 completion queue |
| 102 | 1 | USB0 Tx Endpoint 10 completion queue |
| 103 | 1 | USB0 Tx Endpoint 11 completion queue |
| 104 | 1 | USB0 Tx Endpoint 12 completion queue |
| 104 | 1 | USB0 Tx Endpoint 13 completion queue |
| 106 | 1 | USB0 Tx Endpoint 14 completion queue |
| 107 | 1 | USB0 Tx Endpoint 15 completion queue |
| 108 | 1 | Reserved |
| 109 | 1 | USB0 Rx Endpoint 1 completion queue |
| 110 | 1 | USB0 Rx Endpoint 2 completion queue |
| 111 | 1 | USB0 Rx Endpoint 3 completion queue |
| 112 | 1 | USB0 Rx Endpoint 4 completion queue |

### Table 16-27. Queue-Endpoint Assignments (continued)

| Queue Start Number | Queue Count | Queue-Endpoint Association |
|---|---|---|
| 113 | 1 | USB0 Rx Endpoint 5 completion queue |
| 114 | 1 | USB0 Rx Endpoint 6 completion queue |
| 115 | 1 | USB0 Rx Endpoint 7 completion queue |
| 116 | 1 | USB0 Rx Endpoint 8 completion queue |
| 117 | 1 | USB0 Rx Endpoint 9 completion queue |
| 118 | 1 | USB0 Rx Endpoint 10 completion queue |
| 119 | 1 | USB0 Rx Endpoint 11 completion queue |
| 120 | 1 | USB0 Rx Endpoint 12 completion queue |
| 121 | 1 | USB0 Rx Endpoint 13 completion queue |
| 122 | 1 | USB0 Rx Endpoint 14 completion queue |
| 123 | 1 | USB0 Rx Endpoint 15 completion queue |
| 124 | 1 | Reserved |
| 125 | 1 | USB1 Tx Endpoint 1 completion queue |
| 126 | 1 | USB1 Tx Endpoint 2 completion queue |
| 127 | 1 | USB1 Tx Endpoint 3 completion queue |
| 128 | 1 | USB1 Tx Endpoint 4 completion queue |
| 129 | 1 | USB1 Tx Endpoint 5 completion queue |
| 130 | 1 | USB1 Tx Endpoint 6 completion queue |
| 131 | 1 | USB1 Tx Endpoint 7 completion queue |
| 132 | 1 | USB1 Tx Endpoint 8 completion queue |
| 133 | 1 | USB1 Tx Endpoint 9 completion queue |
| 134 | 1 | USB1 Tx Endpoint 10 completion queue |
| 135 | 1 | USB1 Tx Endpoint 11 completion queue |
| 136 | 1 | USB1 Tx Endpoint 12 completion queue |
| 137 | 1 | USB1 Tx Endpoint 13 completion queue |
| 138 | 1 | USB1 Tx Endpoint 14 completion queue |
| 139 | 1 | USB1 Tx Endpoint 15 completion queue |
| 140 | 1 | Reserved |
| 141 | 1 | USB1 Rx Endpoint 1 completion queue |
| 142 | 1 | USB1 Rx Endpoint 2 completion queue |
| 143 | 1 | USB1 Rx Endpoint 3 completion queue |
| 144 | 1 | USB1 Rx Endpoint 4 completion queue |
| 145 | 1 | USB1 Rx Endpoint 5 completion queue |
| 146 | 1 | USB1 Rx Endpoint 6 completion queue |
| 147 | 1 | USB1 Rx Endpoint 7 completion queue |
| 148 | 1 | USB1 Rx Endpoint 8 completion queue |
| 149 | 1 | USB1 Rx Endpoint 9 completion queue |
| 150 | 1 | USB1 Rx Endpoint 10 completion queue |
| 151 | 1 | USB1 Rx Endpoint 11 completion queue |
| 152 | 1 | USB1 Rx Endpoint 12 completion queue |
| 153 | 1 | USB1 Rx Endpoint 13 completion queue |
| 154 | 1 | USB1 Rx Endpoint 14 completion queue |
| 155 | 1 | USB1 Rx Endpoint 15 completion queue |

#### 16.2.9.3.2 Free Descriptor Queue (Receive Submit Queue)

Receive endpoints/channels use queues, referred to as free descriptor queues, or receive submit queues, to forward completed receive packets to the host. The entries on the free descriptor queues have pre-attached empty buffers whose size and location are described in the original buffer information fields in the descriptor. The host is required to allocate both the descriptor and buffer and pre-link them prior to adding (submitting) a descriptor to one of the available free descriptor queue. The first 32 queues (queue 0 up to queue 31) are reserved for all 30 USB0/1 (endpoints 1 to 15 of each USB module) receive endpoints/channels to handle incoming packets to the device.

#### 16.2.9.3.3 Transmit Submit Queue

Transmit ports use packet queues, referred to as transmit submit queues, to store the packets that are waiting to be transmitted. Each transmit endpoint has dedicated queues (2 queues per port) that are reserved exclusively for a use by a single endpoint. Multiple queues per port/channel are allocated to facilitate quality of service (QoS) for applications that require QoS. The first 30 queues, queue 32 up to queue 61, are allocated for USB0 transmit endpoints 1 to 15 with two queues per endpoint.queues 62 up to queue 91, are allocated for transmit USB1 endpoints 1 to 15.

#### 16.2.9.3.4 Transmit Completion (Return) Queue

Transmit ports use packet queues, referred to as "transmit completion queues, to return packet descriptors to the host after packets have been transmitted. A singe queue is reserved to be used by a single transmit endpoint. Application s/w needs to insure that the right set of queues is used to return the Descriptors after the completion of a packet transmission based on the endpoint number used. For USB0 transmit endpoints 1 to 15, queues 93–107 are reserved and assigned for use as a completion queue, respectively. Similarly for USB1 transmit endpoints 1 to 15, queues 125–139 are used as completion queues, respectively.

Transmit Completion Queues are also used to return packet Descriptors when performing a Transmit channel teardown operation.

#### 16.2.9.3.5 Receive Completion (Return) Queue

Receive ports use packet queues, referred to as receive completion queues, to return packet descriptors to the port after packets have been received. A singe queue is reserved to be used by a single receive endpoint. Application s/w needs to insure that the right set of queues is used to return the descriptors after the completion of a packet reception based on the endpoint number used. For USB0 receive endpoints 1 to 15 queues 122–136 are reserved/assigned for use as a receive completion queue respectively. Similarly for USB1 receive endpoints 1 to 15, queues 137–151 is used as completion queue respectively.

Receive channel teardown is not necessary for receive transactions and no channel teardown information nor resource is available

#### 16.2.9.3.6 Diverting Queue Packets from one Queue to Another

The host can move the entire contents of one queue to another queue by writing the source queue number and the destination queue number to the Queue Diversion register. When diverting packets, the descriptors are pushed onto the tail of the destination queue.

### 16.2.9.4 Memory Regions and Linking RAM

In addition to allocating buffer for raw data and memory for Descriptors, the host is responsible for allocating additional memory for exclusive use of the CPPI DMA Queue Manager to be used as a scratch PAD RAM. The queue manager uses this memory to manage states of descriptors submitted within the submit queues. In other words, this memory needs not to be managed by the user software and firmware responsibility lies only for allocation/reserving a chunk of memory for the use of the queue manager. The allocated memory can be a single block of memory that is contiguous or two blocks of memory that are not contiguous. These two blocks of memory are referred to as a Linking RAM Regions and should not be confused with memory regions that are used to store descriptors. That is, the use of the term *region* should be used in the context of its use.

To accomplish the linking of submitted descriptors, the queue manager will first resolve the 32-bit descriptor pointer into a 16-bit index which is used for linking and queue pointer purposes. Once the physical index information is determined, the queue manager will link that descriptor onto the descriptor chain that is maintained for that logical queue by writing the linking information out to a linking RAM which is external to the queue manager. The queue manager will also update the queue tail pointer appropriately. Since logical queues within the queue manager are maintained using linked lists, queues cannot become full and no check for fullness is required before queuing a packet descriptor.

The actual physical size of the Linking RAM region(s) to be allocated depends on the total number of descriptors defined within all memory regions. A minimum of four bytes of memory needs to be allocated for each Descriptor defined within all 16 memory regions.

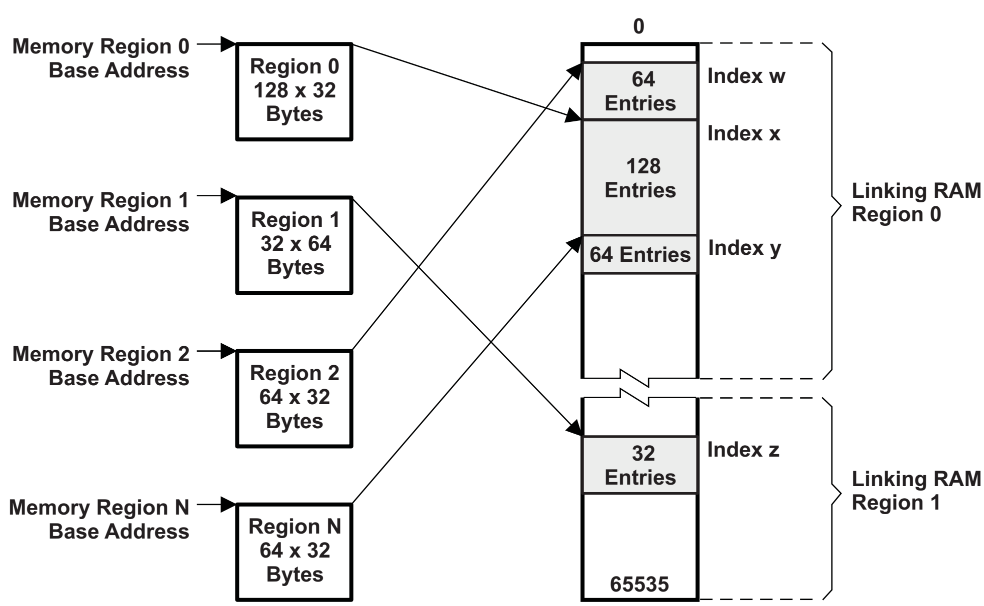
The queue manager has the capability of managing up to 16 memory regions. These memory regions are used to store descriptors of variable sizes. The total number of descriptors that can be managed by the queue manager should not exceed 64K. Each memory region has descriptors of one configurable size, that is, descriptors with different sizes cannot be housed within a single memory region. These 64K descriptors are referenced internally in the queue manager by a 16-bit quantity index.

The information about the Linking RAM regions and the size that are allocated is communicated to the CPPI DMA via three registers dedicated for this purpose. Two of the three registers are used to store the 32-bit aligned start addresses of the Linking RAM regions. The remaining one register is used to store the size of the first Linking RAM. The linking RAM size value stored here is the number of descriptors that is to be managed by the queue manager within that region not the physical size of the buffer, which is four times the number of descriptors.

Note that application is not required to use both linking RAM regions, if the size of the Linking RAM for the first region is large enough to accommodate all descriptors defined. No linking RAM size register for linking RAM region 2 exists. The size of the second linking RAM, when used, is indirectly computed from the total number of descriptors defined less the number of descriptors managed by the first linking RAM

Figure 16-16 displays the relationship between several memory regions and linking RAM.

**Figure 16-16. Relationship Between Memory Regions and Linking RAM**




### 16.2.9.5 Zero Length Packets

A special case is the handling of null packets with the CPPI 4.1 compliant DMA controller. Upon receiving a zero length USB packet, the XFER DMA will send a data block to the DMA controller with byte count of zero and the zero byte packet bit of INFO Word 2 set. The DMA controller will then perform normal End of Packet termination of the packet, without transferring data.

If a zero-length USB packet is received, the XDMA will send the CDMA a data block with a byte count of 0 and this bit set. The CDMA will then perform normal EOP termination of the packet without transferring data. For transmit, if a packet has this bit set, the XDMA will ignore the CPPI packet size and send a zero-length packet to the USB controller.

#### 16.2.9.6 CPPI DMA Scheduler

The CPPI DMA scheduler is responsible for controlling the rate and order between the different Tx and Rx threads/transfers that are provided in the CPPI DMA controller. The scheduler table RAM exists within the scheduler.

The DMA controller maintains state information for each of the channels which allows packet segmentation and reassembly operations to be time division multiplexed between channels in order to share the underlying DMA hardware. A DMA scheduler is used to control the ordering and rate at which this multiplexing occurs.

##### *16.2.9.6.1 CPPI DMA Scheduler Operation*

Once the scheduler is enabled it will begin processing the entries in the table. When appropriate the scheduler will pass credits to the DMA controller to perform a Tx or Rx operation. The operation of the DMA controller is as follows:

1. After the DMA scheduler is enabled it begins with the table index set to 0.
2. The scheduler reads the entry pointed to by the index and checks to see if the channel in question is currently in a state where a DMA operation can be accepted.
   a. The DMA channel must be enabled AND
   b. The CPPI FIFO that the channel talks to has free space on TX (FIFO full signal is not asserted) or a valid block on Rx (FIFO empty signal is not asserted)
3. If the DMA channel is capable of processing a credit to transfer a block, the DMA scheduler will issue that credit via the DMA scheduling interface which is a point to point connection between the DMA Scheduler and the DMA Controller.
   a. The DMA controller may not be ready to accept the credit immediately and is provided a sched_ready signal which is used to stall the scheduler until it can accept the credit. The DMA controller only asserts the sched_ready signal when it is in the IDLE state.
   b. Once a credit has been accepted (indicated by sched_req and sched_ready both asserted), the scheduler will increment the index to the next entry and will start again at step 2.
4. If the channel in question is not currently capable of processing a credit, the scheduler will increment the index in the scheduler table to the next entry and will start at step 2.
5. Note that when the scheduler attempts to increment its index, to the value programmed in the table size register, the index will be reset to 0.

###### *16.2.9.6.1.1 CPPI DMA Scheduler Initialization*

Before the scheduler can be used, the host is required to initialize and enable the block. This initialization is performed as follows:

1. The Host initializes entries within an internal memory array in the scheduler. This array contains up to 256 entries (4 entries per table word n where n=0-63) and each entry consists of a DMA channel number and a bit indicating if this is a Tx or Rx opportunity. These entries represent both the order and frequency that various Tx and Rx channels will be processed. A table size of 256 entries allows channel bandwidth to be allocated with a maximum precision of 1/256th of the total DMA bandwidth. The more entries that are present for a given channel, the bigger the slice of the bandwidth that channel will be given. Larger tables can be accommodated to allow for more precision. This array can only be written by the Host, it cannot be read.
2. If the application does not need to use the entire 256 entries, firmware can initialize the portion of the 256 entries and indicate the size of the entries used by writing onto an internal register in the scheduler which sets the actual size of the array (it can be less than 256 entries).
3. The host writes an internal register bit to enable the scheduler. The scheduler is not required to be disabled in order to change the scheduler array contents.

#### *16.2.9.6.1.2 CPPI DMA Scheduler Programming Examples*

Consider a three endpoints use on a system with the following configurations: EP1-Tx, EP2-Rx, and EP2-Tx. Two assumptions are considered:

**Case 1:** Assume that you would like to service each enabled endpoints (EP1-Tx, EP2-Rx, and EP2-Tx) with equal priority.

The scheduler handles the rate at which an endpoint is serviced by the number of credits programmed (entries) for that particular endpoint within the scheduler Table Words. The scheduler has up to 256 credits that it can grant and for this example the user can configure the number of entries/credits to be anywhere from 3 to 256 with a set of 3 entries.

However, the optimum and direct programming for this scenario would be programming only the first three entries of the scheduler via scheduler Table WORD[0]. Since this case expects the Scheduler to use only the first three entries, you communicate that by programming DMA_SCHED_CTRL.LAST_ENTRY with 2 (that is, 3 -1). The Enabled Endpoint numbers and the data transfer direction is then communicated by programming the first three entries of WORD[0] (ENTRY0_CHANNEL = 1: ENTRY0_RXTX = 0; ENTRY1_CHANNEL = 2: ENTRY1_RXTX = 1;ENTRY2_CHANNEL = 2: ENTRY2_RXTX = 0). With this programming, the Scheduler will only service the first three entries in a round-robin fashion, checking each credited endpoint for transfer one after the other, and servicing the endpoint that has data to transfer.

**Case 2:** Enabled endpoint EP1-Tx is serviced at twice the rate as the other enabled endpoints (EP2-Rx and EP2-Tx).

The number of entries/credit that has to be awarded to EP1-Tx has to be twice as much of the others. Four entries/credits would suffix to satisfy our requirement with two credits for EP1-Tx, one credit for EP2-Rx, and one credit for EP2-Tx. This requirement is satisfied by allocating any two of the four entries to EP1-Tx endpoint. Again for this example, scheduler Table WORD[0] would suffice since it can handle the first 4 entries. Even though several scenarios exist to programming the order of service for this case, one scenario would be to allow servicing EP1-Tx to back-to-back followed by the other enabled endpoints. Program DMA_SCHED_CTRL.LAST_ENTRY with 3 (that is, 4 -1). Program WORD[0] (ENTRY0_CHANNEL = 1: ENTRY0_RXTX = 0; ENTRY1_CHANNEL = 1: ENTRY1_RXTX = 0; ENTRY2_CHANNEL = 2: ENTRY2_RXTX = 1; ENTRY3_CHANNEL = 2: ENTRY3_RXTX = 0).

### 16.2.9.7 CPPI DMA State Registers

The port must store and maintain state information for each transmit and receive port/channel. The state information is referred to as the Tx DMA State and Rx DMA State.

#### *16.2.9.7.1 Transmit DMA State Registers*

The Tx DMA State is a combination of control fields and protocol specific port scratchpad space used to manipulate data structures and transmit packets. Each transmit channel has two queues. Each queue has a one head descriptor pointer and one completion pointer. There are thirty Tx DMA State registers; one for each port/channel.

The following information is stored in the Tx DMA State:

- Tx Queue Head Descriptor Pointer(s)
- Tx Completion Pointer(s)
- Protocol specific control/status (port scratchpad)

#### *16.2.9.7.2 Receive DMA State Registers*

The Rx DMA State is a combination of control fields and protocol specific port scratchpad space used to manipulate data structures in order to receive packets. Each receive channel has only one queue. Each channel queue has one head descriptor pointer and one completion pointer. There are thirty Rx DMA State registers; one for each port/channel.

The following information is stored in the Rx DMA State:

- Rx Queue Head Descriptor Pointer
- Rx Queue Completion Pointer

- Rx Buffer Offset

### 16.2.9.8 CPPI DMA Protocols Supported

Four different type of DMA transfers are supported by the CPPI 4.1 DMA; Transparent, RNDIS, Generic RNDIS, and Linux CDC. The following sections will outline the details on these DMA transfer types.

#### *16.2.9.8.1 Transparent DMA Transfer*

Transparent Mode DMA operation is the default DMA mode where DMA interrupt is generated whenever a DMA packet is transferred. In the transparent mode, DMA packet size cannot be greater than USB MaxPktSize for the endpoint. This transfer type is ideal for transfer (not packet) sizes that are less than a max packet size.

##### *16.2.9.8.1.1 Transparent DMA Transfer Setup*

The following will configure all 30 ports/channels for Transparent DMA Transfer type.

Make sure that RNDIS Mode is disabled globally. This allows the application to configure the CPPI DMA protocol in use to be configured per endpoint need. To disable RNDIS operation, clear RNDIS bit in the USB Control Registers corresponding to the USB Module (default setting), i.e., CTRL0[RNDIS]=0 and CTRL1[RNDIS]=0.

Configure the USB0/1 Tx/Rx DMA Mode Registers (USB0/1 TX(RX)MODE0/1) for the Endpoint field in use is programmed for Transparent Mode, i.e., TXMODE0/1[TXn_MODE] = 00b and RXMODE0/1[RXn_MODE] = 00b.

#### *16.2.9.8.2 RNDIS DMA Transfer*

RNDIS mode DMA is used for large transfers (i.e., total data size to be transferred is greater than USB MaxPktSize where the MzxPktSize is a multiple of 64 bytes) that requires multiple USB packets. This is accomplished by breaking the larger packet into smaller packets, where each packet size being USB MaxPktSize except the last packet where its size is less than USB MaxPktSize, including zero bytes. This implies that multiple USB packets of MaxPktSize will be received and transferred together as a single large DMA transfer and the DMA interrupt is generated only at the end of the complete reception of DMA transfer. The protocol defines the end of the complete transfer by receiving a short USB packet (smaller than USB MaxPktSize as mentioned in USB specification 2.0). If the DMA packet size is an exact multiple of USB MaxPktSize, the DMA controller waits for a zero byte packet at the end of complete transfer to signify the completion of the transfer.

**NOTE:** RNDIS Mode DMA is supported only when USB MaxPktSize is an integral multiple of 64 bytes.

##### *16.2.9.8.2.1 RNDIS DMA Transfer Setup*

If all endpoints in use are desired to operate in RNDIS mode, it is only suffix to configure RNDIS DMA operation in RNDIS mode at the global level and application can ignore individual endpoint DMA mode configuration. This is achieved by programming CTRLR0/1[RNDIS] with a ‘1.’

However if DMA mode is required to be configured at the endpoint level, it is required to disable the use of RNDIS at the global level, this is achieved by clearing RNDIS bit field (CTRLR0/1[RNDIS]=0), since global configuration over-rides endpoint configuration.

To configure RNDIS DMA mode use, configure the field that correspond to the USB module endpoint using the corresponding USB0/1 TX(RX) Mode Register, i.e., TXMODE0/1[TXn_MODE] = 01b and RXMODE0/1[RXn_MODE] = 01b

#### *16.2.9.8.3 Generic RNDIS DMA Transfer*

Generic RNDIS DMA transfer mode is identical to the normal RNDIS mode in nearly all respects, except for the exception case where the last packet of the transfer can either be a short packet or the MaxPktSize. When the last packet size is equal to the MaxPktSize then no additional zero-byte packet is sent when using Generic RNDIS transfer. Generic RNDIS transfer makes use of a USB0/1 GENERIC RNDIS EPn Size register (there exists a register for each endpoint) that must be programmed with the transfer size (in bytes) of the transfer for the USB Module (USB0 or USB1) prior to posting a transfer transaction. If the transfer size is an integer multiple of USB MaxPktSize then no additional zero-byte packet is sent when using Generic RNDIS transfer. However, if the a short packet has been sent prior to programmed size count, the transfer would end in a similar fashion an RDIS transfer would behave. For example, it the USB MaxPktSize (Tx/RxMaxP) is programmed with a value of 64, the Generic RNDIS EP Size register for that endpoint must be programmed with a value that is an integer multiple of 64 (for example, 64, 128, 192, 256, etc.) for it behave differently than RNDIS transfer. In other words, when using Generic RNDIS mode and the DMA is tasked to transfer data transfer size that is less or equal the size value programmed within the USB0/1 GENERIC RNDIS EPn Size register.

This means that Generic RNDIS mode will perform data transfer in the same manner as RNDIS mode, closing the CPPI packet if a USB packet with a size less than the USB MaxPktSize size value is received. Otherwise, the packet will be closed when the value in the Generic RNDIS EP Size register is reached.

Using USB0/1 GENERIC RNDIS EPn Size register, a packet of up to 64K bytes (65536 bytes) can be transferred. This is to allow the host software to program the USB module to transfer data that is an exact multiple of the USB MaxPktSize (Tx/RxMaxP programmed value) without having to send an additional short packet to terminate.

**NOTE:** As in RNDIS mode, the USB max packet size (Tx/RxMaxp programmed value) of any Generic RNDIS mode enabled endpoints must be a multiple of 64 bytes. Generic RNDIS acceleration should not be enabled for endpoints where the max packet size is not a multiple of 64 bytes. Only transparent mode should be used for such endpoints.

###### *16.2.9.8.3.1 Generic RNDIS DMA Transfer Setup*

Disable the use of RNDIS at the global level, this is achieved by clearing RNDIS bit field (CTRLR0/1[RNDIS]=0), since global configuration over-rides endpoint configuration.

Configure the field that correspond to the USB module endpoint using the corresponding USB0/1 TX(RX) Mode Register, i.e., TXMODE0/1[TXn_MODE] = 11b and RXMODE0/1[RXn_MODE] = 11b.

##### *16.2.9.8.4 Linux CDC DMA Transfer*

Linux CDC DMA transfer mode acts in the same manner as RNDIS packets, except for the case where the last data matches the max USB packet size requiring additional zero-byte packet transfer in RNDIS mode and this is not the case for Linux CDC. If the last data packet of a transfer is a short packet where the data size is greater than zero and less the USB MaxPktSize, then the behavior of the Linux CDC DMA transfer type is identical with the RNDIS DMA transfer type. The only exception is when the short packet length terminating the transfer is a Null Packet. In this case, instead of transferring the Null Packet, it will transfer a data packet of size 1 byte with the data value of 00h.

In transmit operation, if an endpoint is configured or CDC Linux mode, upon receiving a Null Packet from the CPPI DMA, the XFER DMA will then generate a packet containing 1 byte of data, whose value is 00h, indicating the end of the transfer. During receive operation, the XFER DMA will recognize the one byte zero packet as a termination of the data transfer, and sends a block of data with the EOP indicator set and a byte count of one to the CPPI DMA controller. The CPPI DMA realizing the end of the transfer termination will not update/increase the packet size count of the Host Packet Descriptor.

###### *16.2.9.8.4.1 Linux CDC DMA Transfer Setup*

Disable the use of RNDIS at the global level, this is achieved by clearing RNDIS bit field (CTRLR0/1[RNDIS]=0), since global configuration over-rides endpoint configuration.

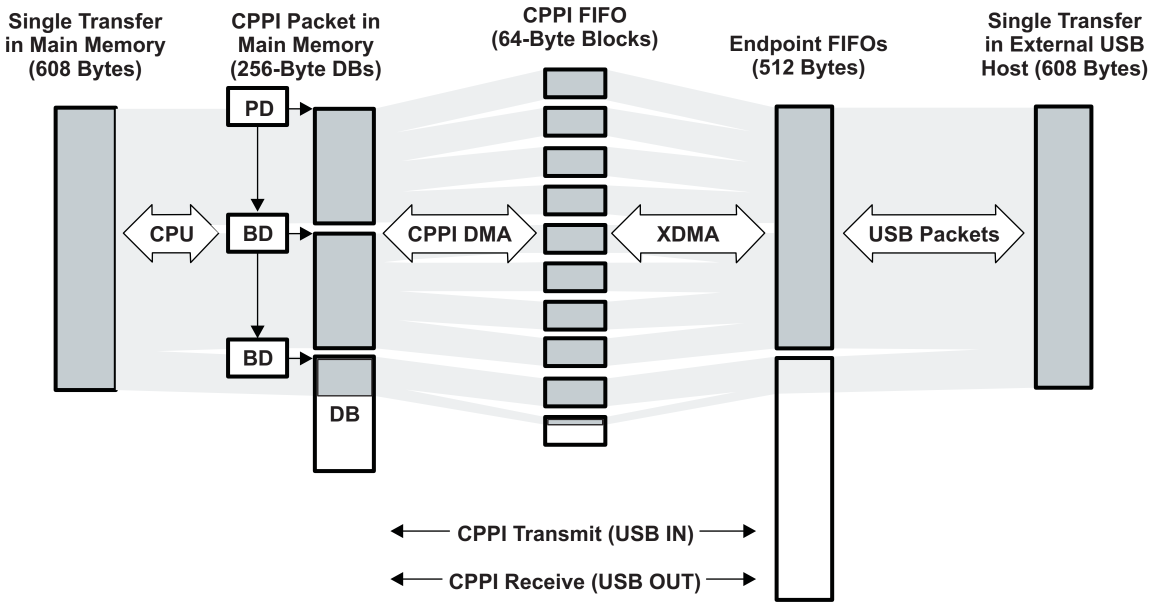
Configure the field that correspond to the USB module endpoint using the corresponding USB0/1 TX(RX) Mode Register, i.e., TXMODE0/1[TXn_MODE] = 10b and RXMODE0/1[RXn_MODE] = 10b.

#### 16.2.9.9 USB Data Flow Using DMA

The necessary steps required to perform a USB data transfer using the CPPI 4.1 DMA is expressed using an example for both transmit and receive cases. Assume USB0 is ready to perform a USB data transfer of size 608 bytes (see Figure 16-17).

The 608 bytes data to be transferred is described using three descriptors. Since each data buffer defined within a single descriptor has a size of 256 bytes, a 608 bytes data buffer would require three descriptors.

**Figure 16-17. High-level Transmit and Receive Data Transfer Example**




**Example assumptions:**

- The CPPI data buffers are 256 bytes in length.
- USB0 module is to be used to perform this transfer. Note that the steps required is similar for USB1 use.
- The USB0 endpoint 1 Tx and Rx endpoint 1 size are programmed to handle max USB packet size of 512 bytes.
- A single transfer length is 608 bytes.
- The SOP offset is 0.

The above translates to the following multi-descriptor setup:

### Transmit Case

Transmit setup is as follows:

- One packet descriptor with packet length (this is NOT data buffer length, the term packet used here is to mean a transfer length not USB packet) field of 608 bytes and a Data Buffer of size 256 Bytes linked to the 1st host buffer descriptor.
- Two buffer descriptors with first buffer descriptor (this is the one linked to the packet descriptor) defining the second data buffer size of 256 Bytes which in turn is linked to the next (second) buffer descriptor.
- Second buffer fescriptor with a fata buffer size of 96 bytes (can be greater, the packet descriptor contain the size of the packet) linked to no other descriptor (NULL).

### Receive Case

For this example since each data buffer size is 256 bytes, we would require a minimum of three descriptors that would define data buffer size of 608 bytes. The receive setup is as follows:

- Two buffer descriptors with 256 bytes of data buffer size
- One buffer descriptor with 96 bytes (can be greater) of data buffer size

Within the rest of this section, the following nomenclature is used:

**BD** — Buffer Descriptor or Host Buffer Descriptor

**DB** — Data Buffer Size of 256 Bytes

**PBD** — Pointer to Host Buffer Descriptor

**PD** — Host Packet Descriptor

**PPD** — Pointer to Host Packet Descriptor

**TXSQ** — Transmit Queue or Transmit Submit Queue (for USB0 EP1, use Queues 32 or 33, for USB1 EP1 use Queues 62 or 63)

**TXCQ** — Transmit Completion Queue or Transmit Return Queue (for USB0 Tx EP1, use Queue 93, and for USB1 Tx EP1 use Queue 125)

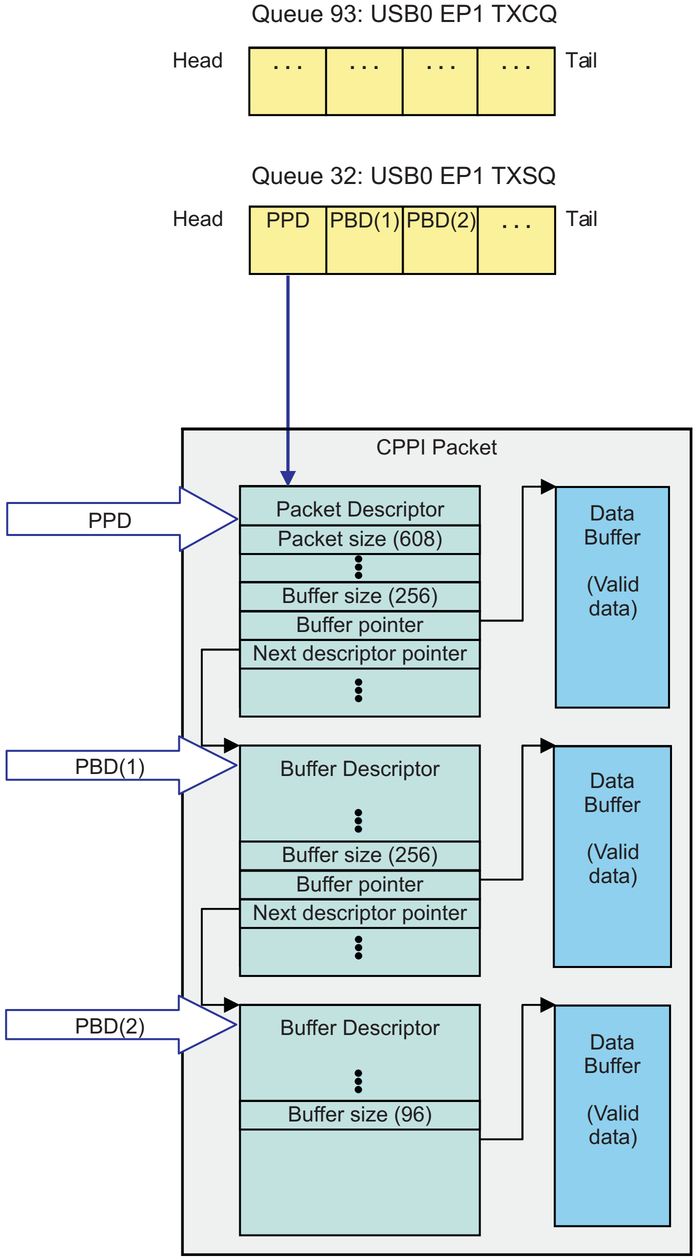
**RXCQ** — Receive Completion Queue or Receive Return Queue (for USB0 Rx EP1, use Queue 109, for USB1 Rx EP1 use Queue 141)

**RXSQ** — Receive Free/Buffer Descriptor Queue or Receive Submit Queue. (For USB0 Rx EP1 Queue 0 is used and for USB1 Rx EP1 Queue 16 should be used

#### *16.2.9.9.1 Transmit USB Data Flow Using DMA*

The transmit descriptors and queue status configuration prior to the transfer taking place is shown in Figure 16-18.

**Figure 16-18. Transmit Descriptors and Queue Status Configuration**



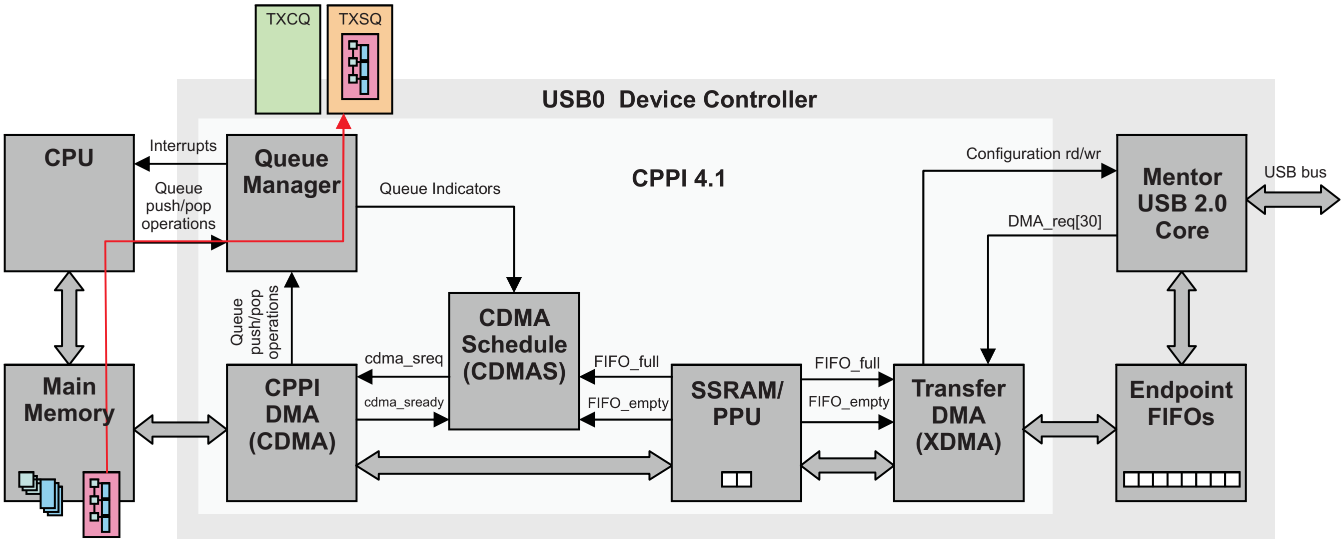

#### 16.2.9.9.1.1 Transmit Initialization (Step 1)

The CPU performs the following steps for transmit initialization:

1. Initializes Memory Region 0 base address and Memory Region 0 size, Link RAM0 Base address, Link RAM0 data size, and Link RAM1 Base address.
2. Creates PD, BDs, and DBs in main memory and link as indicated in Figure 16-19.
3. Initializes and configures the Queue Manager, Channel Setup, DMA Scheduler, and Mentor USB 2.0 Core.
4. Adds (pushes) the PPD and the two PBDs to the TXSQ by writing the Packet Descriptor address to the TXSQ CTRL D Register.

Figure 16-19 captures the BD/DB pair in main memory and later submitted within the TXSQ.

**Figure 16-19. Transmit USB Data Flow Example (Initialization)**




### 16.2.9.9.1.2 CDMA and XDMA Transfer Packet Data Into Endpoint FIFO (Step 2)

The steps for CDMA and XDMA to transfer packet data into endpoint FIFO are as follows:

1. The Queue Manager informs the CDMAS that the TXSQ is not empty.
2. CDMAS checks that the CPPI FIFO FIFO_full is not asserted, then issues a credit to the CDMA.
3. CDMA reads the Packet Descriptor pointer (PPD) and descriptor size hint from the queue manager
4. For each 64-byte block of data in the packet data payload (note that packet refers here to CPPI packet which is not the same as USB packet and it means to refer to data transfer size):
   a. The CDMA transfers a max burst of 64-byte block (OCP burst) from the data to be transferred in main memory to the CPPI FIFO
   b. The XDMA sees FIFO_empty not asserted and transfers 64-byte block from CPPI FIFO to Endpoint FIFO.
   c. The CDMA performs the above two steps (‘a’ and ‘b’) three more times since the data size of the HPD is 256 bytes.
5. The CDMA reads the first buffer descriptor pointer (PBD).
6. For each 64-byte block of data in the packet data payload:
   a. The CDMA transfers a max burst of 64-byte block from the data to be transferred in main memory to the CPPI FIFO.
   b. The XDMA sees FIFO_empty not asserted and transfers 64-byte block from CPPI FIFO to Endpoint FIFO.
   c. The CDMA performs the above two steps (‘a’ and ‘b’) two more times since data size of the HBD is 256 bytes.
7. The CDMA reads the second Buffer Descriptor pointer (PBD
8. For each 64-byte block of data in the packet data payload:
   a. The CDMA transfers a max burst of 64-byte block from the data to be transferred in main memory to the CPPI FIFO.
   b. The XDMA sees FIFO_empty not asserted and transfers 64-byte block from CPPI FIFO to Endpoint FIFO.
   c. The CDMA transfers the last remaining 32-byte from the data to be transferred in main memory to the CPPI FIFO.
   d. The XDMA sees FIFO_empty not asserted and transfers 32-byte block from CPPI FIFO to Endpoint FIFO.

#### 16.2.9.9.1.3 USB 2.0 Core Transmits USB Packets for Tx (Step 3)

1. Once the XDMA has transferred enough 64-byte blocks of burst of data from the CPPI FIFO to fill the Endpoint FIFO (accumulated 512 bytes), it signals the Mentor USB 2.0 Core that a TX packet is ready (sets the endpoint's TxPktRdy bit).
2. The USB 2.0 Core will transmit the packet from the Endpoint FIFO out on the USB BUS when it receives a corresponding IN request from the attached USB Host.
3. After the USB packet is transferred, the USB 2.0 Core issues a TX DMA_req to the XDMA.
4. This process is repeated until (the above steps '1', '2', and '3') the entire packet has been transmitted. The XDMA will also generate the required termination packet depending on the termination mode configured for the endpoint.

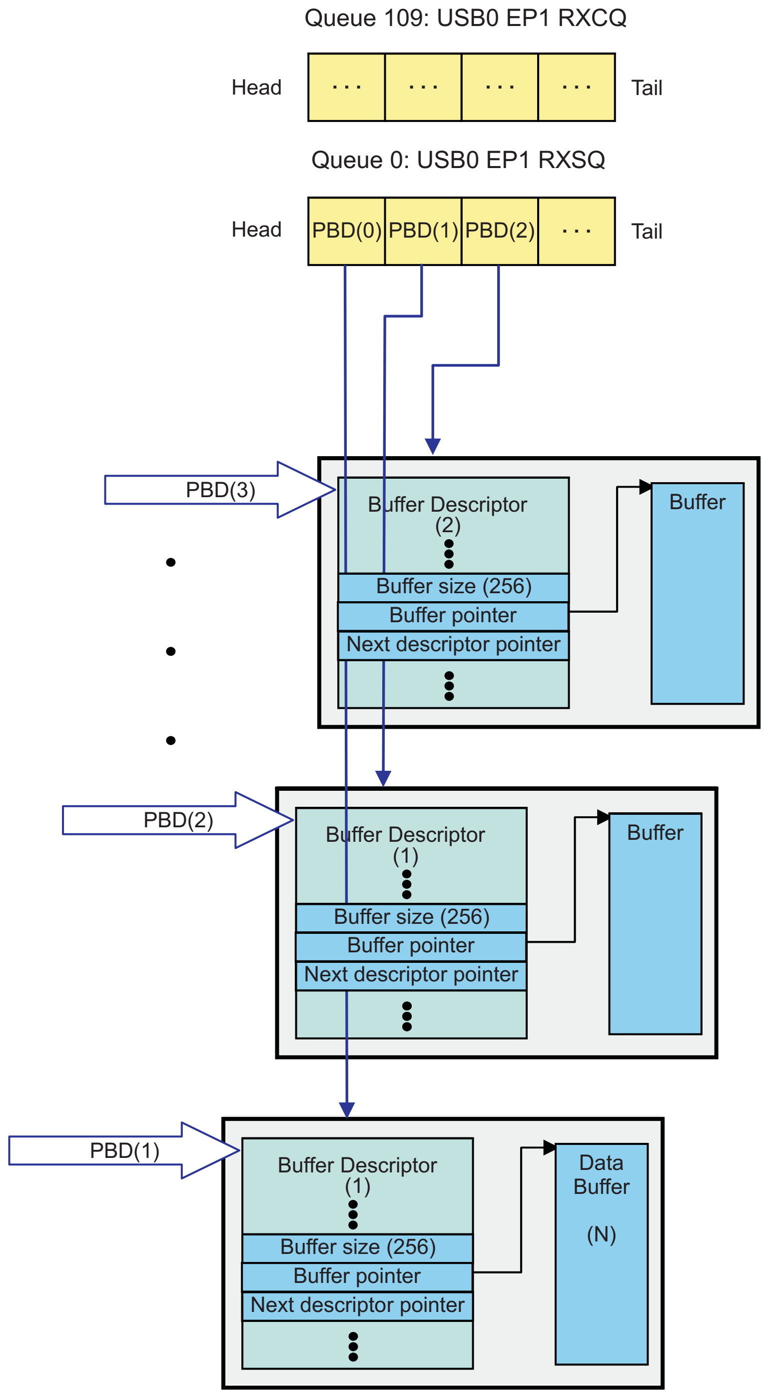
#### 16.2.9.9.1.4 Return Packet to Completion Queue and Interrupt CPU for Tx (Step 4)

1. After all data for the packet has been transmitted (as specified by the packet size field), the CDMA will write the pointer of the Packet Descriptor only to the TX Completion Queue specified in the return queue manager / queue number fields of the packet descriptor.
2. The Queue Manager then indicates the status of the TXSQ (empty) to the CDMAS and the TXCQ to the CPU via an interrupt.

#### 16.2.9.9.2 Receive USB Data Flow Using DMA

The receive buffer descriptors and queue status configuration prior to the transfer taking place is shown in Figure 16-20.

**Figure 16-20. Receive Buffer Descriptors and Queue Status Configuration**



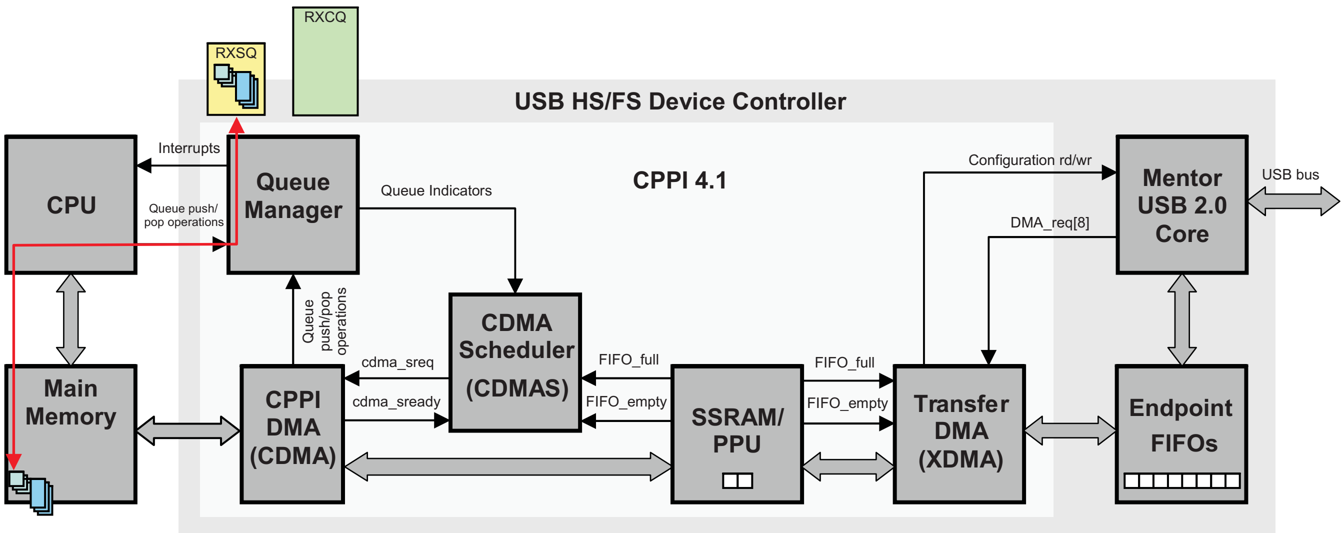

#### 16.2.9.9.2.1 Receive Initialization (Step 1)

1. The CPU initializes Queue Manager with the Memory Region 0 base address and Memory Region 0 size, Link RAM0 Base address, Link RAM0 data size, and Link RAM1 Base address.
2. The CPU creates BDs, and DBs in main memory and link them creating a BD and DB pairs.
3. CPU then initializes the RXCQ queue and configures the Queue Manager, Channel Setup, DMA Scheduler, and USB 2.0 Core.
4. It then adds (pushes) the address of the addresses/pointers of three Buffer Descriptors (PBDs) into the

RXSQ by writing onto CTRL D register.

Figure 16-21 displays receive operation Initialization.

**Figure 16-21. Receive USB Data Flow Example (Initialization)**




### 16.2.9.9.2.2 USB 2.0 Core Receives a Packet, XDMA Starts Data Transfer for Receive (Step 2)

1. The USB 2.0 Core receives a USB packet from the USB Host and stores it in the Endpoint FIFO.
2. It then asserts a DMA_req to the XDMA informing it that data is available in the Endpoint FIFO.
3. The XDMA verifies the corresponding CPPI FIFO is not full via the FIFO_full signal, then starts transferring 64-byte data blocks (burst) from the Endpoint FIFO into the CPPI FIFO.

### 16.2.9.9.2.3 CDMA Transfers Data from SSRAM / PPU to Main Memory for Receive (Step 3)

1. The CDMAS see FIFO_empty de-asserted (there is RX data in the FIFO) and issues a transaction credit to the CDMA.
2. The CDMA begins packet reception by fetching the first PBD from the Queue Manager using the Free Descriptor / Buffer Queue 0 (Rx Submit Queue) index that was initialized in the RX port DMA state for that channel.
3. The CDMA will then begin writing the 64-byte block of packet data into this DB.
4. The CDMA will continue filling the buffer with additional 64-byte blocks of data from the CPPI FIFO and will fetch additional PBD as needed using the Free Descriptor / Buffer Queue 1, 2, and 3 indexes for the 2nd, 3rd, and remaining buffers in the packet. After each buffer is filled, the CDMA writes the buffer descriptor to main memory.

### 16.2.9.9.2.4 CDMA Completes the Packet Transfer for Receive (Step 4)

1. After the entire packet has been received, the CDMA writes the packet descriptor to main memory.
2. The CDMA then writes the packet descriptor to the RXCQ specified in the Queue Manager / Queue Number fields in the RX Global Configuration Register.
3. The Queue Manager then indicates the status of the RXCQ to the CPU via an interrupt.
4. The CPU can then process the received packet by popping the received packet information from the RXCQ and accessing the packet's data from main memory.

### 16.2.10 USB 2.0 Test Modes

The USB2.0 controller supports the four USB 2.0 test modes (Test_SE0_NAK, Test_J, Test_K, and Test_Packet) defined for high-speed functions. It also supports an additional “FIFO access” test mode that can be used to test the operation of the CPU interface, the DMA controller and the RAM block.

The test modes are entered by writing to the TESTMODE register. A test mode is usually requested by the host sending a SET_FEATURE request to Endpoint 0. When the software receives the request, it should wait until the Endpoint 0 transfer has completed (when it receives the Endpoint 0 interrupt indicating the status phase has completed) then write to the TESTMODE register.

**Note:** These test modes have no purpose in normal operation.

#### 16.2.10.1 TEST_SE0_NAK

To enter the Test_SE0_NAK test mode, the software should set the TEST_SE0_NAK bit in the TESTMODE register to 1. The USB controller will then go into a mode in which it responds to any valid IN token with a NAK.

#### 16.2.10.2 TEST_J

To enter the Test_J test mode, the software should set the TEST_J bit in the TESTMODE register to 1. The USB controller will then go into a mode in which it transmits a continuous J on the bus.

#### 16.2.10.3 TEST_K

To enter the Test_K test mode, the software should set the TEST_K bit in the TESTMODE register to 1. The USB controller will then go into a mode in which it transmits a continuous K on the bus.

#### 16.2.10.4 TEST_PACKET

To execute the Test_Packet, the software should:

1. Start a session (if the core is being used in Host mode.
2. Write the standard test packet (shown below) to the Endpoint 0 FIFO.
3. Write 8h to the TESTMODE register (TEST_PACKET = 1) to enter Test_Packet test mode.
4. Set the TxPktRdy bit in the HOST/PERI_CSR0 register (bit 1).

The 53 byte test packet to load is as follows (all bytes in hex). The test packet only has to be loaded once; the USB controller will keep re-sending the test packet without any further intervention from the software.

Table 16-28 displays the 53 Bytes test packet content.

**Table 16-28. 53 Bytes Test Packet Content**

| | | | | | | | |
|---|---|---|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| 0 | AA | AA | AA | AA | AA | AA | AA |
| AA | EE | EE | EE | EE | EE | EE | EE |
| EE | FE | FF | FF | FF | FF | FF | FF |
| FF | FF | FF | FF | FF | 7F | BF | DF |
| EF | F7 | FB | FD | FC | 7E | BF | DF |
| EF | F7 | FB | FD | 7E | | | |

This data sequence is defined in Universal Serial Bus Specification Revision 2.0, Section 7.1.20. The USB controller will add the DATAA0 PID to the head of the data sequence and the CRC to the end.

#### 16.2.10.5 FIFO_ACCESS

The FIFO Access test mode allows you to test the operation of CPU Interface, the DMA controller (if configured), and the RAM block by loading a packet of up to 64 bytes into the Endpoint 0 FIFO and then reading it back out again. Endpoint 0 is used because it is a bidirectional endpoint that uses the same area of RAM for its Tx and Rx FIFOs.

**NOTE:** The core does not need to be connected to the USB bus to run this test. If it is connected, then no session should be in progress when the test is run.

The test procedure is as follows:

1. Load a packet of up to 64 bytes into the Endpoint 0 Tx FIFO.
2. Set HOST/PERI_CSR0[TXPKTRDY].
3. Write 40h to the TESTMODE register (FIFO_ACCESS = 1).
4. Unload the packet from the Endpoint Rx FIFO, again.
5. Set HOST/PERI_CSR0[SERVICEDRXPKTRDY].

Writing 40h to the TESTMODE register causes the following sequence of events:

The Endpoint 0 CPU pointer (that records the number of bytes to be transmitted) is copied to the Endpoint 0 USB pointer (that records the number of bytes received).

1. The Endpoint 0 CPU pointer is reset.
2. HOST/PERI_CSR0[TXPKTRDY] is cleared.
3. HOST/PERI_CSR0[RXPKTRDY] is set.
4. An Endpoint 0 interrupt is generated (if enabled).

The effect of these steps is to make the Endpoint 0 controller act as if the packet loaded into the Tx FIFO has flushed and the same packet received over the USB. The data that was loaded in the Tx FIFO can now be read out of the Rx FIFO.

#### 16.2.10.6 FORCE_HOST

The Force Host test mode enables you to instruct the core to operate in Host mode, regardless of whether it is actually connected to any peripheral; that is, the state of the CID input and the LINESTATE and HOSTDISCON signals are ignored. (While in this mode, the state of the HOSTDISCON signal can be read from the BDEVICE bit in the device control register (DEVCTL)) .

This mode, which is selected by writing 80h to the TESTMODE register (FORCE_HOST = 1), allows implementation of the USB Test_Force_Enable (USB 2.0 Specification Section 7.1.20). It can also be used for debugging PHY problems in hardware.

While the FORCE_HOST bit remains set, the core enters the Host mode when the SESSION bit in DEVCTL is set to 1 and remains in the Host mode until the SESSION bit is cleared to 0 even if a connected device is disconnected during the session. The operating speed while in this mode is determined by the FORCE_HS and FORCE_FS bits in the TESTMODE register.

#### 16.2.10.7 USB Behavior During Emulation

To configure the USB subsystem to stop during emulation suspend events (for example, debugger breakpoints), set up the USB Subsystem and the Debug Subsystem:

1. Set SYSCONFIG.FREEEMU=0. This will allow the Suspend_Control signal from the Debug Subsystem (Chapter 27) to stop and start the USBSS. Note that if FREEEMU=1, the Suspend_Control signal is ignored and the USBSS is free running regardless of any debug suspend event. This FREEEMU bit gives local control from a module perspective to gate the suspend signal coming from the Debug Subsystem.
2. Set the appropriate xxx_Suspend_Control register = 0x9, as described in Section 27.1.1.1, *Debug Suspend Support for Peripherals*. Choose the register appropriate to the peripheral you want to suspend during a suspend event.

## 16.3 Supported Use Cases

The USB Subsystem supports two independent USB Modules where each USB module can operate as a host or peripheral. When operating as a host, it is capable of interfacing to a single target/peripheral directly or to multiple targets via hub at low-, full-, or high-speed. When operating as a peripheral, it is capable of interfacing to a host at a full- or high-speed.

## 16.4 USB Registers

### 16.4.1 USBSS Registers

Table 16-29 lists the memory-mapped registers for the USBSS. All register offset addresses not listed in Table 16-29 should be considered as reserved locations and the register contents should not be modified.

**Table 16-29. USBSS Registers**

| Offset | Acronym | Register Name | Section |
|---|---|---|---|
| 0h | REVREG | | Section 16.4.1.1 |
| 10h | SYSCONFIG | | Section 16.4.1.2 |
| 24h | IRQSTATRAW | | Section 16.4.1.3 |
| 28h | IRQSTAT | | Section 16.4.1.4 |
| 2Ch | IRQENABLER | | Section 16.4.1.5 |
| 30h | IRQCLEARR | | Section 16.4.1.6 |
| 100h | IRQDMATHOLDTX00 | | Section 16.4.1.7 |
| 104h | IRQDMATHOLDTX01 | | Section 16.4.1.8 |
| 108h | IRQDMATHOLDTX02 | | Section 16.4.1.9 |
| 10Ch | IRQDMATHOLDTX03 | | Section 16.4.1.10 |
| 110h | IRQDMATHOLDRX00 | | Section 16.4.1.11 |
| 114h | IRQDMATHOLDRX01 | | Section 16.4.1.12 |
| 118h | IRQDMATHOLDRX02 | | Section 16.4.1.13 |
| 11Ch | IRQDMATHOLDRX03 | | Section 16.4.1.14 |
| 120h | IRQDMATHOLDTX10 | | Section 16.4.1.15 |
| 124h | IRQDMATHOLDTX11 | | Section 16.4.1.16 |
| 128h | IRQDMATHOLDTX12 | | Section 16.4.1.17 |
| 12Ch | IRQDMATHOLDTX13 | | Section 16.4.1.18 |
| 130h | IRQDMATHOLDRX10 | | Section 16.4.1.19 |
| 134h | IRQDMATHOLDRX11 | | Section 16.4.1.20 |
| 138h | IRQDMATHOLDRX12 | | Section 16.4.1.21 |
| 13Ch | IRQDMATHOLDRX13 | | Section 16.4.1.22 |
| 140h | IRQDMAENABLE0 | | Section 16.4.1.23 |
| 144h | IRQDMAENABLE1 | | Section 16.4.1.24 |
| 200h | IRQFRAMETHOLDTX00 | | Section 16.4.1.25 |
| 204h | IRQFRAMETHOLDTX01 | | Section 16.4.1.26 |
| 208h | IRQFRAMETHOLDTX02 | | Section 16.4.1.27 |
| 20Ch | IRQFRAMETHOLDTX03 | | Section 16.4.1.28 |
| 210h | IRQFRAMETHOLDRX00 | | Section 16.4.1.29 |
| 214h | IRQFRAMETHOLDRX01 | | Section 16.4.1.30 |
| 218h | IRQFRAMETHOLDRX02 | | Section 16.4.1.31 |
| 21Ch | IRQFRAMETHOLDRX03 | | Section 16.4.1.32 |
| 220h | IRQFRAMETHOLDTX10 | | Section 16.4.1.33 |
| 224h | IRQFRAMETHOLDTX11 | | Section 16.4.1.34 |
| 228h | IRQFRAMETHOLDTX12 | | Section 16.4.1.35 |
| 22Ch | IRQFRAMETHOLDTX13 | | Section 16.4.1.36 |
| 230h | IRQFRAMETHOLDRX10 | | Section 16.4.1.37 |
| 234h | IRQFRAMETHOLDRX11 | | Section 16.4.1.38 |
| 238h | IRQFRAMETHOLDRX12 | | Section 16.4.1.39 |
| 23Ch | IRQFRAMETHOLDRX13 | | Section 16.4.1.40 |
| 240h | IRQFRAMEENABLE0 | | Section 16.4.1.41 |
| 244h | IRQFRAMEENABLE1 | | Section 16.4.1.42 |

### 16.4.1.1 REVREG Register (offset = 0h) [reset = 4EA20800h]

REVREG is shown in Figure 16-22 and described in Table 16-30.

**Figure 16-22. REVREG Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 | 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| SCHEME | | RESERVED | | FUNC | | | | | | | | | | | |
| R-1h | | R- | | R-EA2h | | | | | | | | | | | |
| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 | 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
| R_RTL | | | | | X_MAJOR | | | CUSTOM | | Y_MINOR | | | | | |
| R-1h | | | | | R-0h | | | R-0h | | R-0h | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-30. REVREG Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-30 | SCHEME | R | 1h | Used to distinguish between legacy interface scheme and current.<br>0 = Legacy<br>1 = Current |
| 29-28 | RESERVED | R | | |
| 27-16 | FUNC | R | EA2h | Function indicates a software compatible module family. |
| 15-11 | R_RTL | R | 1h | RTL revision.<br>Will vary depending on release. |
| 10-8 | X_MAJOR | R | 0h | Major revision. |
| 7-6 | CUSTOM | R | 0h | Custom revision |
| 5-0 | Y_MINOR | R | 0h | Minor revision.<br>USBSS Revision Register |

#### 16.4.1.2 SYSCONFIG Register (offset = 10h) [reset = 28h]

SYSCONFIG is shown in Figure 16-23 and described in Table 16-31.

**Figure 16-23. SYSCONFIG Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| RESERVED | | | | | | | |
| R/W-0h | | | | | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| RESERVED | | | | | | | |
| R/W-0h | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| RESERVED | | | | USB0_OCP_EN_N | PHY0_UTMI_EN_N | USB1_OCP_EN_N | PHY1_UTMI_EN_N |
| R/W-0h | | | | R/W-0h | R/W-0h | R/W-0h | R/W-0h |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| RESERVED | | STANDBY_MODE | | IDLEMODE | | FREEEMU | SOFT_RESET |
| R/W-0h | | R/W-2h | | R/W-2h | | R/W-0h | R/W-0h |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-31. SYSCONFIG Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-12 | RESERVED | R/W | 0h | |
| 11 | USB0_OCP_EN_N | R/W | 0h | Active low clock enable for usb0_ocp_clk<br>0 = enable<br>1 = disable |
| 10 | PHY0_UTMI_EN_N | R/W | 0h | Active low clock enable for phy0_utmi_clk<br>0 = enable<br>1 = disable |
| 9 | USB1_OCP_EN_N | R/W | 0h | Active low clock enable for usb1_ocp_clk<br>0 = enable<br>1 = disable |
| 8 | PHY1_UTMI_EN_N | R/W | 0h | Active low clock enable for phy1_utmi_clk<br>0 = enable<br>1 = disable |
| 7-6 | RESERVED | R/W | 0h | |
| 5-4 | STANDBY_MODE | R/W | 2h | Configuration of the local initiator state management mode.<br>0 = force-standby mode.<br>1 = no-standby mode.<br>2 = smart-standby mode.<br>3 = Reserved. |
| 3-2 | IDLEMODE | R/W | 2h | Configuration of the local target state management mode.<br>0 = force-idle mode.<br>1 = no-idle mode.<br>2 = smart-idle mode.<br>3 = smart-idle with wakeup. |
| 1 | FREEEMU | R/W | 0h | Sensitivity to emulation (debug) suspend input signal.<br>0 = sensitive to emulation suspend event from Debug Subsystem.<br>1 = NOT sensitive to emulation suspend from Debug Subsystem. |
| 0 | SOFT_RESET | R/W | 0h | Software reset of USBSS, USB0, and USB1 modules<br>Write 0 = No action.<br>Write 1 = Initiate software reset.<br>Read 0 = Reset done, no action.<br>Read 1 = Reset ongoing. |

### 16.4.1.3 IRQSTATRAW Register (offset = 24h) [reset = 0h]

IRQSTATRAW is shown in Figure 16-24 and described in Table 16-32.

**Figure 16-24. IRQSTATRAW Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| RESERVED | | | | | | | |
| R/W-0h | | | | | | | |
| **23** | **22** | **21** | **20** | **19** | **18** | **17** | **16** |
| RESERVED | | | | | | | |
| R/W-0h | | | | | | | |
| **15** | **14** | **13** | **12** | **11** | **10** | **9** | **8** |
| RESERVED | | | | RX_PKT_CMP_1 | TX_PKT_CMP_1 | RX_PKT_CMP_0 | TX_PKT_CMP_0 |
| R/W-0h | | | | R/W-0h | R/W-0h | R/W-0h | R/W-0h |
| **7** | **6** | **5** | **4** | **3** | **2** | **1** | **0** |
| RESERVED | | | | | PD_CMP_FLAG | RX_MOP_STARVATION | RX_SOP_STARVATION |
| R/W-0h | | | | | R/W-0h | R/W-0h | R/W-0h |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-32. IRQSTATRAW Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-12 | RESERVED | R/W | 0h | |
| 11 | RX_PKT_CMP_1 | R/W | 0h | Interrupt status for USB1 Rx CPPI DMA packet completion status |
| 10 | TX_PKT_CMP_1 | R/W | 0h | Interrupt status for USB1 Tx CPPI DMA packet completion status |
| 9 | RX_PKT_CMP_0 | R/W | 0h | Interrupt status for USB0 Rx CPPI DMA packet completion status |
| 8 | TX_PKT_CMP_0 | R/W | 0h | Interrupt status for USB0 Tx CPPI DMA packet completion status |
| 7-3 | RESERVED | R/W | 0h | |
| 2 | PD_CMP_FLAG | R/W | 0h | Interrupt status when the packet is completed, the differ bits is set, and the Packet Descriptor is pushed into the queue manager |
| 1 | RX_MOP_STARVATION | R/W | 0h | Interrupt status when queue manager cannot allocate an Rx buffer in the middle of a packet. |
| 0 | RX_SOP_STARVATION | R/W | 0h | Interrupt status when queue manager cannot allocate an Rx buffer at the start of a packet.<br>USBSS IRQ_STATUS_RAW Register |

#### 16.4.1.4 IRQSTAT Register (offset = 28h) [reset = 0h]

IRQSTAT is shown in Figure 16-25 and described in Table 16-33.

**Figure 16-25. IRQSTAT Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| RESERVED | | | | | | | |
| R/W-0h | | | | | | | |
| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
| RESERVED | | | | | | | |
| R/W-0h | | | | | | | |
| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
| RESERVED | | | | RX_PKT_CMP_1 | TX_PKT_CMP_1 | RX_PKT_CMP_0 | TX_PKT_CMP_0 |
| R/W-0h | | | | R/W-0h | R/W-0h | R/W-0h | R/W-0h |
| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
| RESERVED | | | | | PD_CMP_FLAG | RX_MOP_STARVATION | RX_SOP_STARVATION |
| R/W-0h | | | | | R/W-0h | R/W-0h | R/W-0h |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-33. IRQSTAT Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-12 | RESERVED | R/W | 0h | |
| 11 | RX_PKT_CMP_1 | R/W | 0h | Interrupt status for USB1 Rx CPPI DMA packet completion status |
| 10 | TX_PKT_CMP_1 | R/W | 0h | Interrupt status for USB1 Tx CPPI DMA packet completion status |
| 9 | RX_PKT_CMP_0 | R/W | 0h | Interrupt status for USB0 Rx CPPI DMA packet completion status |
| 8 | TX_PKT_CMP_0 | R/W | 0h | Interrupt status for USB0 Tx CPPI DMA packet completion status |
| 7-3 | RESERVED | R/W | 0h | |
| 2 | PD_CMP_FLAG | R/W | 0h | Interrupt status when the packet is completed, the differ bits is set, and the Packet Descriptor is pushed into the queue manager |
| 1 | RX_MOP_STARVATION | R/W | 0h | Interrupt status when queue manager cannot allocate an Rx buffer in the middle of a packet. |
| 0 | RX_SOP_STARVATION | R/W | 0h | Interrupt status when queue manager cannot allocate an Rx buffer at the start of a packet.<br>USBSS IRQ_STATUS Register |

### 16.4.1.5 IRQENABLER Register (offset = 2Ch) [reset = 0h]

IRQENABLER is shown in Figure 16-26 and described in Table 16-34.

**Figure 16-26. IRQENABLER Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| RESERVED | | | | | | | |
| R/W-0h | | | | | | | |
| **23** | **22** | **21** | **20** | **19** | **18** | **17** | **16** |
| RESERVED | | | | | | | |
| R/W-0h | | | | | | | |
| **15** | **14** | **13** | **12** | **11** | **10** | **9** | **8** |
| RESERVED | | | | RX_PKT_CMP_1 | TX_PKT_CMP_1 | RX_PKT_CMP_0 | TX_PKT_CMP_0 |
| R/W-0h | | | | R/W-0h | R/W-0h | R/W-0h | R/W-0h |
| **7** | **6** | **5** | **4** | **3** | **2** | **1** | **0** |
| RESERVED | | | | | PD_CMP_FLAG | RX_MOP_STARVATION | RX_SOP_STARVATION |
| R/W-0h | | | | | R/W-0h | R/W-0h | R/W-0h |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-34. IRQENABLER Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-12 | RESERVED | R/W | 0h | |
| 11 | RX_PKT_CMP_1 | R/W | 0h | Interrupt enable for USB1 Rx CPPI DMA packet completion status |
| 10 | TX_PKT_CMP_1 | R/W | 0h | Interrupt enable for USB1 Tx CPPI DMA packet completion status |
| 9 | RX_PKT_CMP_0 | R/W | 0h | Interrupt enable for USB0 Rx CPPI DMA packet completion status |
| 8 | TX_PKT_CMP_0 | R/W | 0h | Interrupt enable for USB0 Tx CPPI DMA packet completion status |
| 7-3 | RESERVED | R/W | 0h | |
| 2 | PD_CMP_FLAG | R/W | 0h | Interrupt enable when the packet is completed, the differ bits is set, and the Packet Descriptor is pushed into the queue manager |
| 1 | RX_MOP_STARVATION | R/W | 0h | Interrupt enable when queue manager cannot allocate an Rx buffer in the middle of a packet. |
| 0 | RX_SOP_STARVATION | R/W | 0h | Interrupt enable when queue manager cannot allocate an Rx buffer at the start of a packet.<br>USBSS IRQ_ENABLE_SET Register |

### 16.4.1.6 IRQCLEARR Register (offset = 30h) [reset = 0h]

IRQCLEARR is shown in Figure 16-27 and described in Table 16-35.

**Figure 16-27. IRQCLEARR Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| RESERVED | | | | | | | |
| R/W-0h | | | | | | | |
| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
| RESERVED | | | | | | | |
| R/W-0h | | | | | | | |
| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
| RESERVED | | | | RX_PKT_CMP_1 | TX_PKT_CMP_1 | RX_PKT_CMP_0 | TX_PKT_CMP_0 |
| R/W-0h | | | | R/W-0h | R/W-0h | R/W-0h | R/W-0h |
| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
| RESERVED | | | | | PD_CMP_FLAG | RX_MOP_STARVATION | RX_SOP_STARVATION |
| R/W-0h | | | | | R/W-0h | R/W-0h | R/W-0h |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-35. IRQCLEARR Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-12 | RESERVED | R/W | 0h | |
| 11 | RX_PKT_CMP_1 | R/W | 0h | Interrupt enable for USB1 Rx CPPI DMA packet completion status |
| 10 | TX_PKT_CMP_1 | R/W | 0h | Interrupt enable for USB1 Tx CPPI DMA packet completion status |
| 9 | RX_PKT_CMP_0 | R/W | 0h | Interrupt enable for USB0 Rx CPPI DMA packet completion status |
| 8 | TX_PKT_CMP_0 | R/W | 0h | Interrupt enable for USB0 Tx CPPI DMA packet completion status |
| 7-3 | RESERVED | R/W | 0h | |
| 2 | PD_CMP_FLAG | R/W | 0h | Interrupt enable when the packet is completed, the differ bits is set, and the Packet Descriptor is pushed into the queue manager |
| 1 | RX_MOP_STARVATION | R/W | 0h | Interrupt enable when queue manager cannot allocate an Rx buffer in the middle of a packet. |
| 0 | RX_SOP_STARVATION | R/W | 0h | Interrupt enable when queue manager cannot allocate an Rx buffer at the start of a packet.<br>USBSS IRQ_ENABLE_CLR Register |

#### 16.4.1.7 IRQDMATHOLDTX00 Register (offset = 100h) [reset = 0h]

IRQDMATHOLDTX00 is shown in Figure 16-28 and described in Table 16-36.

**Figure 16-28. IRQDMATHOLDTX00 Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 | 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| DMA_THRES_TX0_3 | | | | | | | | DMA_THRES_TX0_2 | | | | | | | |
| R/W-0h | | | | | | | | R/W-0h | | | | | | | |
| **15** | **14** | **13** | **12** | **11** | **10** | **9** | **8** | **7** | **6** | **5** | **4** | **3** | **2** | **1** | **0** |
| DMA_THRES_TX0_1 | | | | | | | | RESERVED | | | | | | | |
| R/W-0h | | | | | | | | R/W-0h | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-36. IRQDMATHOLDTX00 Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-24 | DMA_THRES_TX0_3 | R/W | 0h | DMA threshold value for tx_pkt_cmp_0 for USB0 Endpoint 3. |
| 23-16 | DMA_THRES_TX0_2 | R/W | 0h | DMA threshold value for tx_pkt_cmp_0 for USB0 Endpoint 2. |
| 15-8 | DMA_THRES_TX0_1 | R/W | 0h | DMA threshold value for tx_pkt_cmp_0 for USB0 Endpoint 1. |
| 7-0 | RESERVED | R/W | 0h | |

#### 16.4.1.8 IRQDMATHOLDTX01 Register (offset = 104h) [reset = 0h]

IRQDMATHOLDTX01 is shown in Figure 16-29 and described in Table 16-37.

**Figure 16-29. IRQDMATHOLDTX01 Register**

| 31-24 | 23-16 |
|---|---|
| DMA_THRES_TX0_7 | DMA_THRES_TX0_6 |
| R/W-0h | R/W-0h |

| 15-8 | 7-0 |
|---|---|
| DMA_THRES_TX0_5 | DMA_THRES_TX0_4 |
| R/W-0h | R/W-0h |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-37. IRQDMATHOLDTX01 Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-24 | DMA_THRES_TX0_7 | R/W | 0h | DMA threshold value for tx_pkt_cmp_0 for USB0 Endpoint 7. |
| 23-16 | DMA_THRES_TX0_6 | R/W | 0h | DMA threshold value for tx_pkt_cmp_0 for USB0 Endpoint 6. |
| 15-8 | DMA_THRES_TX0_5 | R/W | 0h | DMA threshold value for tx_pkt_cmp_0 for USB0 Endpoint 5. |
| 7-0 | DMA_THRES_TX0_4 | R/W | 0h | DMA threshold value for tx_pkt_cmp_0 for USB0 Endpoint 4.<br>USBSS IRQ_DMA_THRSHOLD_TX0_1 Register |

### 16.4.1.9 IRQDMATHOLDTX02 Register (offset = 108h) [reset = 0h]

IRQDMATHOLDTX02 is shown in Figure 16-30 and described in Table 16-38.

**Figure 16-30. IRQDMATHOLDTX02 Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 | 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| DMA_THRES_TX0_11 | | | | | | | | DMA_THRES_TX0_10 | | | | | | | |
| R/W-0h | | | | | | | | R/W-0h | | | | | | | |
| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 | 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
| DMA_THRES_TX0_9 | | | | | | | | DMA_THRES_TX0_8 | | | | | | | |
| R/W-0h | | | | | | | | R/W-0h | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-38. IRQDMATHOLDTX02 Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-24 | DMA_THRES_TX0_11 | R/W | 0h | DMA threshold value for tx_pkt_cmp_0 for USB0 Endpoint 11. |
| 23-16 | DMA_THRES_TX0_10 | R/W | 0h | DMA threshold value for tx_pkt_cmp_0 for USB0 Endpoint 10. |
| 15-8 | DMA_THRES_TX0_9 | R/W | 0h | DMA threshold value for tx_pkt_cmp_0 for USB0 Endpoint 9. |
| 7-0 | DMA_THRES_TX0_8 | R/W | 0h | DMA threshold value for tx_pkt_cmp_0 for USB0 Endpoint 8.<br>USBSS IRQ_DMA_THRSHOLD_TX0_2 Register |

### 16.4.1.10 IRQDMATHOLDTX03 Register (offset = 10Ch) [reset = 0h]

IRQDMATHOLDTX03 is shown in Figure 16-31 and described in Table 16-39.

**Figure 16-31. IRQDMATHOLDTX03 Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 | 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| DMA_THRES_TX0_15 | | | | | | | | DMA_THRES_TX0_14 | | | | | | | |
| R/W-0h | | | | | | | | R/W-0h | | | | | | | |
| **15** | **14** | **13** | **12** | **11** | **10** | **9** | **8** | **7** | **6** | **5** | **4** | **3** | **2** | **1** | **0** |
| DMA_THRES_TX0_13 | | | | | | | | DMA_THRES_TX0_12 | | | | | | | |
| R/W-0h | | | | | | | | R/W-0h | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-39. IRQDMATHOLDTX03 Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-24 | DMA_THRES_TX0_15 | R/W | 0h | DMA threshold value for tx_pkt_cmp_0 for USB0 Endpoint 15. |
| 23-16 | DMA_THRES_TX0_14 | R/W | 0h | DMA threshold value for tx_pkt_cmp_0 for USB0 Endpoint 14. |
| 15-8 | DMA_THRES_TX0_13 | R/W | 0h | DMA threshold value for tx_pkt_cmp_0 for USB0 Endpoint 13. |
| 7-0 | DMA_THRES_TX0_12 | R/W | 0h | DMA threshold value for tx_pkt_cmp_0 for USB0 Endpoint 12.<br>USBSS IRQ_DMA_THRSHOLD_TX0_3 Register |

### 16.4.1.11 IRQDMATHOLDRX00 Register (offset = 110h) [reset = 0h]

IRQDMATHOLDRX00 is shown in Figure 16-32 and described in Table 16-40.

**Figure 16-32. IRQDMATHOLDRX00 Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 | 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| DMA_THRES_RX0_3 | | | | | | | | DMA_THRES_RX0_2 | | | | | | | |
| R/W-0h | | | | | | | | R/W-0h | | | | | | | |
| **15** | **14** | **13** | **12** | **11** | **10** | **9** | **8** | **7** | **6** | **5** | **4** | **3** | **2** | **1** | **0** |
| DMA_THRES_RX0_1 | | | | | | | | RESERVED | | | | | | | |
| R/W-0h | | | | | | | | R/W-0h | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-40. IRQDMATHOLDRX00 Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-24 | DMA_THRES_RX0_3 | R/W | 0h | DMA threshold value for rx_pkt_cmp_0 for USB0 Endpoint 3. |
| 23-16 | DMA_THRES_RX0_2 | R/W | 0h | DMA threshold value for rx_pkt_cmp_0 for USB0 Endpoint 2. |
| 15-8 | DMA_THRES_RX0_1 | R/W | 0h | DMA threshold value for rx_pkt_cmp_0 for USB0 Endpoint 1. |
| 7-0 | RESERVED | R/W | 0h | |

#### 16.4.1.12 IRQDMATHOLDRX01 Register (offset = 114h) [reset = 0h]

IRQDMATHOLDRX01 is shown in Figure 16-33 and described in Table 16-41.

**Figure 16-33. IRQDMATHOLDRX01 Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 | 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| DMA_THRES_RX0_7 | | | | | | | | DMA_THRES_RX0_6 | | | | | | | |
| R/W-0h | | | | | | | | R/W-0h | | | | | | | |
| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 | 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
| DMA_THRES_RX0_5 | | | | | | | | DMA_THRES_RX0_4 | | | | | | | |
| R/W-0h | | | | | | | | R/W-0h | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-41. IRQDMATHOLDRX01 Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-24 | DMA_THRES_RX0_7 | R/W | 0h | DMA threshold value for rx_pkt_cmp_0 for USB0 Endpoint 7. |
| 23-16 | DMA_THRES_RX0_6 | R/W | 0h | DMA threshold value for rx_pkt_cmp_0 for USB0 Endpoint 6. |
| 15-8 | DMA_THRES_RX0_5 | R/W | 0h | DMA threshold value for rx_pkt_cmp_0 for USB0 Endpoint 5. |
| 7-0 | DMA_THRES_RX0_4 | R/W | 0h | DMA threshold value for rx_pkt_cmp_0 for USB0 Endpoint 4.<br>USBSS IRQ_DMA_THRSHOLD_RX0_1 Register |

### 16.4.1.13 IRQDMATHOLDRX02 Register (offset = 118h) [reset = 0h]

IRQDMATHOLDRX02 is shown in Figure 16-34 and described in Table 16-42.

**Figure 16-34. IRQDMATHOLDRX02 Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 | 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| DMA_THRES_RX0_11 | | | | | | | | DMA_THRES_RX0_10 | | | | | | | |
| R/W-0h | | | | | | | | R/W-0h | | | | | | | |
| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 | 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
| DMA_THRES_RX0_9 | | | | | | | | DMA_THRES_RX0_8 | | | | | | | |
| R/W-0h | | | | | | | | R/W-0h | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-42. IRQDMATHOLDRX02 Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-24 | DMA_THRES_RX0_11 | R/W | 0h | DMA threshold value for rx_pkt_cmp_0 for USB0 Endpoint 11. |
| 23-16 | DMA_THRES_RX0_10 | R/W | 0h | DMA threshold value for rx_pkt_cmp_0 for USB0 Endpoint 10. |
| 15-8 | DMA_THRES_RX0_9 | R/W | 0h | DMA threshold value for rx_pkt_cmp_0 for USB0 Endpoint 9. |
| 7-0 | DMA_THRES_RX0_8 | R/W | 0h | DMA threshold value for rx_pkt_cmp_0 for USB0 Endpoint 8.<br>USBSS IRQ_DMA_THRSHOLD_RX0_2 Register |

#### 16.4.1.14 IRQDMATHOLDRX03 Register (offset = 11Ch) [reset = 0h]

IRQDMATHOLDRX03 is shown in Figure 16-35 and described in Table 16-43.

**Figure 16-35. IRQDMATHOLDRX03 Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 | 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| DMA_THRES_RX0_15 | | | | | | | | DMA_THRES_RX0_14 | | | | | | | |
| R/W-0h | | | | | | | | R/W-0h | | | | | | | |
| **15** | **14** | **13** | **12** | **11** | **10** | **9** | **8** | **7** | **6** | **5** | **4** | **3** | **2** | **1** | **0** |
| DMA_THRES_RX0_13 | | | | | | | | DMA_THRES_RX0_12 | | | | | | | |
| R/W-0h | | | | | | | | R/W-0h | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-43. IRQDMATHOLDRX03 Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-24 | DMA_THRES_RX0_15 | R/W | 0h | DMA threshold value for rx_pkt_cmp_0 for USB0 Endpoint 15. |
| 23-16 | DMA_THRES_RX0_14 | R/W | 0h | DMA threshold value for rx_pkt_cmp_0 for USB0 Endpoint 14. |
| 15-8 | DMA_THRES_RX0_13 | R/W | 0h | DMA threshold value for rx_pkt_cmp_0 for USB0 Endpoint 13. |
| 7-0 | DMA_THRES_RX0_12 | R/W | 0h | DMA threshold value for rx_pkt_cmp_0 for USB0 Endpoint 12.<br>USBSS IRQ_DMA_THRSHOLD_RX0_3 Register |

### 16.4.1.15 IRQDMATHOLDTX10 Register (offset = 120h) [reset = 0h]

IRQDMATHOLDTX10 is shown in Figure 16-36 and described in Table 16-44.

**Figure 16-36. IRQDMATHOLDTX10 Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 | 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| DMA_THRES_TX1_3 | | | | | | | | DMA_THRES_TX1_2 | | | | | | | |
| R/W-0h | | | | | | | | R/W-0h | | | | | | | |
| **15** | **14** | **13** | **12** | **11** | **10** | **9** | **8** | **7** | **6** | **5** | **4** | **3** | **2** | **1** | **0** |
| DMA_THRES_TX1_1 | | | | | | | | RESERVED | | | | | | | |
| R/W-0h | | | | | | | | R/W-0h | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-44. IRQDMATHOLDTX10 Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-24 | DMA_THRES_TX1_3 | R/W | 0h | DMA threshold value for tx_pkt_cmp_0 for USB1 Endpoint 3. |
| 23-16 | DMA_THRES_TX1_2 | R/W | 0h | DMA threshold value for tx_pkt_cmp_0 for USB1 Endpoint 2. |
| 15-8 | DMA_THRES_TX1_1 | R/W | 0h | DMA threshold value for tx_pkt_cmp_0 for USB1 Endpoint 1. |
| 7-0 | RESERVED | R/W | 0h | |

### 16.4.1.16 IRQDMATHOLDTX11 Register (offset = 124h) [reset = 0h]

IRQDMATHOLDTX11 is shown in Figure 16-37 and described in Table 16-45.

**Figure 16-37. IRQDMATHOLDTX11 Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 | 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| DMA_THRES_TX1_7 | | | | | | | | DMA_THRES_TX1_6 | | | | | | | |
| R/W-0h | | | | | | | | R/W-0h | | | | | | | |
| **15** | **14** | **13** | **12** | **11** | **10** | **9** | **8** | **7** | **6** | **5** | **4** | **3** | **2** | **1** | **0** |
| DMA_THRES_TX1_5 | | | | | | | | DMA_THRES_TX1_4 | | | | | | | |
| R/W-0h | | | | | | | | R/W-0h | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-45. IRQDMATHOLDTX11 Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-24 | DMA_THRES_TX1_7 | R/W | 0h | DMA threshold value for tx_pkt_cmp_0 for USB1 Endpoint 7. |
| 23-16 | DMA_THRES_TX1_6 | R/W | 0h | DMA threshold value for tx_pkt_cmp_0 for USB1 Endpoint 6. |
| 15-8 | DMA_THRES_TX1_5 | R/W | 0h | DMA threshold value for tx_pkt_cmp_0 for USB1 Endpoint 5. |
| 7-0 | DMA_THRES_TX1_4 | R/W | 0h | DMA threshold value for tx_pkt_cmp_0 for USB1 Endpoint 4. USBSS IRQ_DMA_THRSHOLD_TX1_1 Register |

### 16.4.1.17 IRQDMATHOLDTX12 Register (offset = 128h) [reset = 0h]

IRQDMATHOLDTX12 is shown in Figure 16-38 and described in Table 16-46.

**Figure 16-38. IRQDMATHOLDTX12 Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 | 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| DMA_THRES_TX1_11 | | | | | | | | DMA_THRES_TX1_10 | | | | | | | |
| R/W-0h | | | | | | | | R/W-0h | | | | | | | |
| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 | 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
| DMA_THRES_TX1_9 | | | | | | | | DMA_THRES_TX1_8 | | | | | | | |
| R/W-0h | | | | | | | | R/W-0h | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-46. IRQDMATHOLDTX12 Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-24 | DMA_THRES_TX1_11 | R/W | 0h | DMA threshold value for tx_pkt_cmp_0 for USB1 Endpoint 11. |
| 23-16 | DMA_THRES_TX1_10 | R/W | 0h | DMA threshold value for tx_pkt_cmp_0 for USB1 Endpoint 10. |
| 15-8 | DMA_THRES_TX1_9 | R/W | 0h | DMA threshold value for tx_pkt_cmp_0 for USB1 Endpoint 9. |
| 7-0 | DMA_THRES_TX1_8 | R/W | 0h | DMA threshold value for tx_pkt_cmp_0 for USB1 Endpoint 8.<br>USBSS IRQ_DMA_THRSHOLD_TX1_2 Register |

### 16.4.1.18 IRQDMATHOLDTX13 Register (offset = 12Ch) [reset = 0h]

IRQDMATHOLDTX13 is shown in Figure 16-39 and described in Table 16-47.

**Figure 16-39. IRQDMATHOLDTX13 Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 | 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| DMA_THRES_TX1_15 | | | | | | | | DMA_THRES_TX1_14 | | | | | | | |
| R/W-0h | | | | | | | | R/W-0h | | | | | | | |
| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 | 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
| DMA_THRES_TX1_13 | | | | | | | | DMA_THRES_TX1_12 | | | | | | | |
| R/W-0h | | | | | | | | R/W-0h | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-47. IRQDMATHOLDTX13 Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-24 | DMA_THRES_TX1_15 | R/W | 0h | DMA threshold value for tx_pkt_cmp_0 for USB1 Endpoint 15. |
| 23-16 | DMA_THRES_TX1_14 | R/W | 0h | DMA threshold value for tx_pkt_cmp_0 for USB1 Endpoint 14. |
| 15-8 | DMA_THRES_TX1_13 | R/W | 0h | DMA threshold value for tx_pkt_cmp_0 for USB1 Endpoint 13. |
| 7-0 | DMA_THRES_TX1_12 | R/W | 0h | DMA threshold value for tx_pkt_cmp_0 for USB1 Endpoint 12. USBSS IRQ_DMA_THRSHOLD_TX1_3 Register |

### 16.4.1.19 IRQDMATHOLDRX10 Register (offset = 130h) [reset = 0h]

IRQDMATHOLDRX10 is shown in Figure 16-40 and described in Table 16-48.

**Figure 16-40. IRQDMATHOLDRX10 Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 | 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| DMA_THRES_RX1_3 | | | | | | | | DMA_THRES_RX1_2 | | | | | | | |
| R/W-0h | | | | | | | | R/W-0h | | | | | | | |
| **15** | **14** | **13** | **12** | **11** | **10** | **9** | **8** | **7** | **6** | **5** | **4** | **3** | **2** | **1** | **0** |
| DMA_THRES_RX1_1 | | | | | | | | RESERVED | | | | | | | |
| R/W-0h | | | | | | | | R/W-0h | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-48. IRQDMATHOLDRX10 Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-24 | DMA_THRES_RX1_3 | R/W | 0h | DMA threshold value for rx_pkt_cmp_0 for USB1 Endpoint 3. |
| 23-16 | DMA_THRES_RX1_2 | R/W | 0h | DMA threshold value for rx_pkt_cmp_0 for USB1 Endpoint 2. |
| 15-8 | DMA_THRES_RX1_1 | R/W | 0h | DMA threshold value for rx_pkt_cmp_0 for USB1 Endpoint 1. |
| 7-0 | RESERVED | R/W | 0h | |

### 16.4.1.20 IRQDMATHOLDRX11 Register (offset = 134h) [reset = 0h]

IRQDMATHOLDRX11 is shown in Figure 16-41 and described in Table 16-49.

**Figure 16-41. IRQDMATHOLDRX11 Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 | 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| DMA_THRES_RX1_7 | | | | | | | | DMA_THRES_RX1_6 | | | | | | | |
| R/W-0h | | | | | | | | R/W-0h | | | | | | | |
| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 | 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
| DMA_THRES_RX1_5 | | | | | | | | DMA_THRES_RX1_4 | | | | | | | |
| R/W-0h | | | | | | | | R/W-0h | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-49. IRQDMATHOLDRX11 Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-24 | DMA_THRES_RX1_7 | R/W | 0h | DMA threshold value for rx_pkt_cmp_0 for USB1 Endpoint 7. |
| 23-16 | DMA_THRES_RX1_6 | R/W | 0h | DMA threshold value for rx_pkt_cmp_0 for USB1 Endpoint 6. |
| 15-8 | DMA_THRES_RX1_5 | R/W | 0h | DMA threshold value for rx_pkt_cmp_0 for USB1 Endpoint 5. |
| 7-0 | DMA_THRES_RX1_4 | R/W | 0h | DMA threshold value for rx_pkt_cmp_0 for USB1 Endpoint 4.<br>USBSS IRQ_DMA_THRSHOLD_RX1_1 Register |

### 16.4.1.21 IRQDMATHOLDRX12 Register (offset = 138h) [reset = 0h]

IRQDMATHOLDRX12 is shown in Figure 16-42 and described in Table 16-50.

**Figure 16-42. IRQDMATHOLDRX12 Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 | 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| DMA_THRES_RX1_11 | | | | | | | | DMA_THRES_RX1_10 | | | | | | | |
| R/W-0h | | | | | | | | R/W-0h | | | | | | | |
| **15** | **14** | **13** | **12** | **11** | **10** | **9** | **8** | **7** | **6** | **5** | **4** | **3** | **2** | **1** | **0** |
| DMA_THRES_RX1_9 | | | | | | | | DMA_THRES_RX1_8 | | | | | | | |
| R/W-0h | | | | | | | | R/W-0h | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-50. IRQDMATHOLDRX12 Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-24 | DMA_THRES_RX1_11 | R/W | 0h | DMA threshold value for rx_pkt_cmp_0 for USB1 Endpoint 11. |
| 23-16 | DMA_THRES_RX1_10 | R/W | 0h | DMA threshold value for rx_pkt_cmp_0 for USB1 Endpoint 10. |
| 15-8 | DMA_THRES_RX1_9 | R/W | 0h | DMA threshold value for rx_pkt_cmp_0 for USB1 Endpoint 9. |
| 7-0 | DMA_THRES_RX1_8 | R/W | 0h | DMA threshold value for rx_pkt_cmp_0 for USB1 Endpoint 8.<br>USBSS IRQ_DMA_THRSHOLD_RX1_2 Register |

#### 16.4.1.22 IRQDMATHOLDRX13 Register (offset = 13Ch) [reset = 0h]

IRQDMATHOLDRX13 is shown in Figure 16-43 and described in Table 16-51.

**Figure 16-43. IRQDMATHOLDRX13 Register**

| 31-24 | 23-16 |
|---|---|
| DMA_THRES_RX1_15 | DMA_THRES_RX1_14 |
| R/W-0h | R/W-0h |

| 15-8 | 7-0 |
|---|---|
| DMA_THRES_RX1_13 | DMA_THRES_RX1_12 |
| R/W-0h | R/W-0h |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-51. IRQDMATHOLDRX13 Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-24 | DMA_THRES_RX1_15 | R/W | 0h | DMA threshold value for rx_pkt_cmp_0 for USB1 Endpoint 15. |
| 23-16 | DMA_THRES_RX1_14 | R/W | 0h | DMA threshold value for rx_pkt_cmp_0 for USB1 Endpoint 14. |
| 15-8 | DMA_THRES_RX1_13 | R/W | 0h | DMA threshold value for rx_pkt_cmp_0 for USB1 Endpoint 13. |
| 7-0 | DMA_THRES_RX1_12 | R/W | 0h | DMA threshold value for rx_pkt_cmp_0 for USB1 Endpoint 12. USBSS IRQ_DMA_THRSHOLD_RX1_3 Register |

### 16.4.1.23 IRQDMAENABLE0 Register (offset = 140h) [reset = 0h]

IRQDMAENABLE0 is shown in Figure 16-44 and described in Table 16-52.

**Figure 16-44. IRQDMAENABLE0 Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| DMA_EN_RX0_15 | DMA_EN_RX0_14 | DMA_EN_RX0_13 | DMA_EN_RX0_12 | DMA_EN_RX0_11 | DMA_EN_RX0_10 | DMA_EN_RX0_9 | DMA_EN_RX0_8 |
| R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h |
| **23** | **22** | **21** | **20** | **19** | **18** | **17** | **16** |
| DMA_EN_RX0_7 | DMA_EN_RX0_6 | DMA_EN_RX0_5 | DMA_EN_RX0_4 | DMA_EN_RX0_3 | DMA_EN_RX0_2 | DMA_EN_RX0_1 | RESERVED |
| R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h |
| **15** | **14** | **13** | **12** | **11** | **10** | **9** | **8** |
| DMA_EN_TX0_15 | DMA_EN_TX0_14 | DMA_EN_TX0_13 | DMA_EN_TX0_12 | DMA_EN_TX0_11 | DMA_EN_TX0_10 | DMA_EN_TX0_9 | DMA_EN_TX0_8 |
| R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h |
| **7** | **6** | **5** | **4** | **3** | **2** | **1** | **0** |
| DMA_EN_TX0_7 | DMA_EN_TX0_6 | DMA_EN_TX0_5 | DMA_EN_TX0_4 | DMA_EN_TX0_3 | DMA_EN_TX0_2 | DMA_EN_TX0_1 | RESERVED |
| R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-52. IRQDMAENABLE0 Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31 | DMA_EN_RX0_15 | R/W | 0h | DMA threshold enable value for rx_pkt_cmp_0 for USB0 Endpoint 15. |
| 30 | DMA_EN_RX0_14 | R/W | 0h | DMA threshold enable value for rx_pkt_cmp_0 for USB0 Endpoint 14. |
| 29 | DMA_EN_RX0_13 | R/W | 0h | DMA threshold enable value for rx_pkt_cmp_0 for USB0 Endpoint 13. |
| 28 | DMA_EN_RX0_12 | R/W | 0h | DMA threshold enable value for rx_pkt_cmp_0 for USB0 Endpoint 12. |
| 27 | DMA_EN_RX0_11 | R/W | 0h | DMA threshold enable value for rx_pkt_cmp_0 for USB0 Endpoint 11. |
| 26 | DMA_EN_RX0_10 | R/W | 0h | DMA threshold enable value for rx_pkt_cmp_0 for USB0 Endpoint 10. |
| 25 | DMA_EN_RX0_9 | R/W | 0h | DMA threshold enable value for rx_pkt_cmp_0 for USB0 Endpoint 9. |
| 24 | DMA_EN_RX0_8 | R/W | 0h | DMA threshold enable value for rx_pkt_cmp_0 for USB0 Endpoint 8. |
| 23 | DMA_EN_RX0_7 | R/W | 0h | DMA threshold enable value for rx_pkt_cmp_0 for USB0 Endpoint 7. |
| 22 | DMA_EN_RX0_6 | R/W | 0h | DMA threshold enable value for rx_pkt_cmp_0 for USB0 Endpoint 6. |
| 21 | DMA_EN_RX0_5 | R/W | 0h | DMA threshold enable value for rx_pkt_cmp_0 for USB0 Endpoint 5. |
| 20 | DMA_EN_RX0_4 | R/W | 0h | DMA threshold enable value for rx_pkt_cmp_0 for USB0 Endpoint 4. |
| 19 | DMA_EN_RX0_3 | R/W | 0h | DMA threshold enable value for rx_pkt_cmp_0 for USB0 Endpoint 3. |
| 18 | DMA_EN_RX0_2 | R/W | 0h | DMA threshold enable value for rx_pkt_cmp_0 for USB0 Endpoint 2. |
| 17 | DMA_EN_RX0_1 | R/W | 0h | DMA threshold enable value for rx_pkt_cmp_0 for USB0 Endpoint 1. |
| 16 | RESERVED | R/W | 0h | |
| 15 | DMA_EN_TX0_15 | R/W | 0h | DMA threshold enable value for tx_pkt_cmp_0 for USB0 Endpoint 15. |
| 14 | DMA_EN_TX0_14 | R/W | 0h | DMA threshold enable value for tx_pkt_cmp_0 for USB0 Endpoint 14. |

**Table 16-52. IRQDMAENABLE0 Register Field Descriptions (continued)**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13 | DMA_EN_TX0_13 | R/W | 0h | DMA threshold enable value for tx_pkt_cmp_0 for USB0 Endpoint 13. |
| 12 | DMA_EN_TX0_12 | R/W | 0h | DMA threshold enable value for tx_pkt_cmp_0 for USB0 Endpoint 12. |
| 11 | DMA_EN_TX0_11 | R/W | 0h | DMA threshold enable value for tx_pkt_cmp_0 for USB0 Endpoint 11. |
| 10 | DMA_EN_TX0_10 | R/W | 0h | DMA threshold enable value for tx_pkt_cmp_0 for USB0 Endpoint 10. |
| 9 | DMA_EN_TX0_9 | R/W | 0h | DMA threshold enable value for tx_pkt_cmp_0 for USB0 Endpoint 9. |
| 8 | DMA_EN_TX0_8 | R/W | 0h | DMA threshold enable value for tx_pkt_cmp_0 for USB0 Endpoint 8. |
| 7 | DMA_EN_TX0_7 | R/W | 0h | DMA threshold enable value for tx_pkt_cmp_0 for USB0 Endpoint 7. |
| 6 | DMA_EN_TX0_6 | R/W | 0h | DMA threshold enable value for tx_pkt_cmp_0 for USB0 Endpoint 6. |
| 5 | DMA_EN_TX0_5 | R/W | 0h | DMA threshold enable value for tx_pkt_cmp_0 for USB0 Endpoint 5. |
| 4 | DMA_EN_TX0_4 | R/W | 0h | DMA threshold enable value for tx_pkt_cmp_0 for USB0 Endpoint 4. |
| 3 | DMA_EN_TX0_3 | R/W | 0h | DMA threshold enable value for tx_pkt_cmp_0 for USB0 Endpoint 3. |
| 2 | DMA_EN_TX0_2 | R/W | 0h | DMA threshold enable value for tx_pkt_cmp_0 for USB0 Endpoint 2. |
| 1 | DMA_EN_TX0_1 | R/W | 0h | DMA threshold enable value for tx_pkt_cmp_0 for USB0 Endpoint 1. |
| 0 | RESERVED | R/W | 0h | |

### 16.4.1.24 IRQDMAENABLE1 Register (offset = 144h) [reset = 0h]

IRQDMAENABLE1 is shown in Figure 16-45 and described in Table 16-53.

**Figure 16-45. IRQDMAENABLE1 Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| DMA_EN_RX1_15 | DMA_EN_RX1_14 | DMA_EN_RX1_13 | DMA_EN_RX1_12 | DMA_EN_RX1_11 | DMA_EN_RX1_10 | DMA_EN_RX1_9 | DMA_EN_RX1_8 |
| R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h |
| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
| DMA_EN_RX1_7 | DMA_EN_RX1_6 | DMA_EN_RX1_5 | DMA_EN_RX1_4 | DMA_EN_RX1_3 | DMA_EN_RX1_2 | DMA_EN_RX1_1 | RESERVED |
| R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h |
| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
| DMA_EN_TX1_15 | DMA_EN_TX1_14 | DMA_EN_TX1_13 | DMA_EN_TX1_12 | DMA_EN_TX1_11 | DMA_EN_TX1_10 | DMA_EN_TX1_9 | DMA_EN_TX1_8 |
| R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h |
| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
| DMA_EN_TX1_7 | DMA_EN_TX1_6 | DMA_EN_TX1_5 | DMA_EN_TX1_4 | DMA_EN_TX1_3 | DMA_EN_TX1_2 | DMA_EN_TX1_1 | RESERVED |
| R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-53. IRQDMAENABLE1 Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31 | DMA_EN_RX1_15 | R/W | 0h | DMA threshold enable value for rx_pkt_cmp_0 for USB0 Endpoint 15. |
| 30 | DMA_EN_RX1_14 | R/W | 0h | DMA threshold enable value for rx_pkt_cmp_0 for USB0 Endpoint 14. |
| 29 | DMA_EN_RX1_13 | R/W | 0h | DMA threshold enable value for rx_pkt_cmp_0 for USB0 Endpoint 13. |
| 28 | DMA_EN_RX1_12 | R/W | 0h | DMA threshold enable value for rx_pkt_cmp_0 for USB0 Endpoint 12. |
| 27 | DMA_EN_RX1_11 | R/W | 0h | DMA threshold enable value for rx_pkt_cmp_0 for USB0 Endpoint 11. |
| 26 | DMA_EN_RX1_10 | R/W | 0h | DMA threshold enable value for rx_pkt_cmp_0 for USB0 Endpoint 10. |
| 25 | DMA_EN_RX1_9 | R/W | 0h | DMA threshold enable value for rx_pkt_cmp_0 for USB0 Endpoint 9. |
| 24 | DMA_EN_RX1_8 | R/W | 0h | DMA threshold enable value for rx_pkt_cmp_0 for USB0 Endpoint 8. |
| 23 | DMA_EN_RX1_7 | R/W | 0h | DMA threshold enable value for rx_pkt_cmp_0 for USB0 Endpoint 7. |
| 22 | DMA_EN_RX1_6 | R/W | 0h | DMA threshold enable value for rx_pkt_cmp_0 for USB0 Endpoint 6. |
| 21 | DMA_EN_RX1_5 | R/W | 0h | DMA threshold enable value for rx_pkt_cmp_0 for USB0 Endpoint 5. |
| 20 | DMA_EN_RX1_4 | R/W | 0h | DMA threshold enable value for rx_pkt_cmp_0 for USB0 Endpoint 4. |
| 19 | DMA_EN_RX1_3 | R/W | 0h | DMA threshold enable value for rx_pkt_cmp_0 for USB0 Endpoint 3. |
| 18 | DMA_EN_RX1_2 | R/W | 0h | DMA threshold enable value for rx_pkt_cmp_0 for USB0 Endpoint 2. |
| 17 | DMA_EN_RX1_1 | R/W | 0h | DMA threshold enable value for rx_pkt_cmp_0 for USB0 Endpoint 1. |
| 16 | RESERVED | R/W | 0h | |
| 15 | DMA_EN_TX1_15 | R/W | 0h | DMA threshold enable value for tx_pkt_cmp_0 for USB0 Endpoint 15. |
| 14 | DMA_EN_TX1_14 | R/W | 0h | DMA threshold enable value for tx_pkt_cmp_0 for USB0 Endpoint 14. |

**Table 16-53. IRQDMAENABLE1 Register Field Descriptions (continued)**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13 | DMA_EN_TX1_13 | R/W | 0h | DMA threshold enable value for tx_pkt_cmp_0 for USB0 Endpoint 13. |
| 12 | DMA_EN_TX1_12 | R/W | 0h | DMA threshold enable value for tx_pkt_cmp_0 for USB0 Endpoint 12. |
| 11 | DMA_EN_TX1_11 | R/W | 0h | DMA threshold enable value for tx_pkt_cmp_0 for USB0 Endpoint 11. |
| 10 | DMA_EN_TX1_10 | R/W | 0h | DMA threshold enable value for tx_pkt_cmp_0 for USB0 Endpoint 10. |
| 9 | DMA_EN_TX1_9 | R/W | 0h | DMA threshold enable value for tx_pkt_cmp_0 for USB0 Endpoint 9. |
| 8 | DMA_EN_TX1_8 | R/W | 0h | DMA threshold enable value for tx_pkt_cmp_0 for USB0 Endpoint 8. |
| 7 | DMA_EN_TX1_7 | R/W | 0h | DMA threshold enable value for tx_pkt_cmp_0 for USB0 Endpoint 7. |
| 6 | DMA_EN_TX1_6 | R/W | 0h | DMA threshold enable value for tx_pkt_cmp_0 for USB0 Endpoint 6. |
| 5 | DMA_EN_TX1_5 | R/W | 0h | DMA threshold enable value for tx_pkt_cmp_0 for USB0 Endpoint 5. |
| 4 | DMA_EN_TX1_4 | R/W | 0h | DMA threshold enable value for tx_pkt_cmp_0 for USB0 Endpoint 4. |
| 3 | DMA_EN_TX1_3 | R/W | 0h | DMA threshold enable value for tx_pkt_cmp_0 for USB0 Endpoint 3. |
| 2 | DMA_EN_TX1_2 | R/W | 0h | DMA threshold enable value for tx_pkt_cmp_0 for USB0 Endpoint 2. |
| 1 | DMA_EN_TX1_1 | R/W | 0h | DMA threshold enable value for tx_pkt_cmp_0 for USB0 Endpoint 1. |
| 0 | RESERVED | R/W | 0h | |

### 16.4.1.25 IRQFRAMETHOLDTX00 Register (offset = 200h) [reset = 0h]

IRQFRAMETHOLDTX00 is shown in Figure 16-46 and described in Table 16-54.

**Figure 16-46. IRQFRAMETHOLDTX00 Register**

| 31-24 | 23-16 |
|---|---|
| FRAME_THRES_TX1_3 | FRAME_THRES_TX1_2 |
| R/W-0h | R/W-0h |

| 15-8 | 7-0 |
|---|---|
| FRAME_THRES_TX1_1 | RESERVED |
| R/W-0h | R/W-0h |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-54. IRQFRAMETHOLDTX00 Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-24 | FRAME_THRES_TX1_3 | R/W | 0h | FRAME threshold value for tx_pkt_cmp_0 for USB0 Endpoint 3. |
| 23-16 | FRAME_THRES_TX1_2 | R/W | 0h | FRAME threshold value for tx_pkt_cmp_0 for USB0 Endpoint 2. |
| 15-8 | FRAME_THRES_TX1_1 | R/W | 0h | FRAME threshold value for tx_pkt_cmp_0 for USB0 Endpoint 1. |
| 7-0 | RESERVED | R/W | 0h | |

### 16.4.1.26 IRQFRAMETHOLDTX01 Register (offset = 204h) [reset = 0h]

IRQFRAMETHOLDTX01 is shown in Figure 16-47 and described in Table 16-55.

**Figure 16-47. IRQFRAMETHOLDTX01 Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 | 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| FRAME_THRES_TX1_7 | | | | | | | | FRAME_THRES_TX1_6 | | | | | | | |
| R/W-0h | | | | | | | | R/W-0h | | | | | | | |
| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 | 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
| FRAME_THRES_TX1_5 | | | | | | | | FRAME_THRES_TX1_4 | | | | | | | |
| R/W-0h | | | | | | | | R/W-0h | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-55. IRQFRAMETHOLDTX01 Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-24 | FRAME_THRES_TX1_7 | R/W | 0h | FRAME threshold value for tx_pkt_cmp_0 for USB0 Endpoint 7. |
| 23-16 | FRAME_THRES_TX1_6 | R/W | 0h | FRAME threshold value for tx_pkt_cmp_0 for USB0 Endpoint 6. |
| 15-8 | FRAME_THRES_TX1_5 | R/W | 0h | FRAME threshold value for tx_pkt_cmp_0 for USB0 Endpoint 5. |
| 7-0 | FRAME_THRES_TX1_4 | R/W | 0h | FRAME threshold value for tx_pkt_cmp_0 for USB0 Endpoint 4.<br>USBSS IRQ_FRAME_THRSHOLD_TX0_1 Register |

#### 16.4.1.27 IRQFRAMETHOLDTX02 Register (offset = 208h) [reset = 0h]

IRQFRAMETHOLDTX02 is shown in Figure 16-48 and described in Table 16-56.

**Figure 16-48. IRQFRAMETHOLDTX02 Register**

| 31-24 | 23-16 |
|---|---|
| FRAME_THRES_TX1_11 | FRAME_THRES_TX1_10 |
| R/W-0h | R/W-0h |

| 15-8 | 7-0 |
|---|---|
| FRAME_THRES_TX1_9 | FRAME_THRES_TX1_8 |
| R/W-0h | R/W-0h |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-56. IRQFRAMETHOLDTX02 Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-24 | FRAME_THRES_TX1_11 | R/W | 0h | FRAME threshold value for tx_pkt_cmp_0 for USB0 Endpoint 11. |
| 23-16 | FRAME_THRES_TX1_10 | R/W | 0h | FRAME threshold value for tx_pkt_cmp_0 for USB0 Endpoint 10. |
| 15-8 | FRAME_THRES_TX1_9 | R/W | 0h | FRAME threshold value for tx_pkt_cmp_0 for USB0 Endpoint 9. |
| 7-0 | FRAME_THRES_TX1_8 | R/W | 0h | FRAME threshold value for tx_pkt_cmp_0 for USB0 Endpoint 8.<br>USBSS IRQ_FRAME_THRSHOLD_TX0_2 Register |

### 16.4.1.28 IRQFRAMETHOLDTX03 Register (offset = 20Ch) [reset = 0h]

IRQFRAMETHOLDTX03 is shown in Figure 16-49 and described in Table 16-57.

**Figure 16-49. IRQFRAMETHOLDTX03 Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 | 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| FRAME_THRES_TX1_15 | | | | | | | | FRAME_THRES_TX1_14 | | | | | | | |
| R/W-0h | | | | | | | | R/W-0h | | | | | | | |
| **15** | **14** | **13** | **12** | **11** | **10** | **9** | **8** | **7** | **6** | **5** | **4** | **3** | **2** | **1** | **0** |
| FRAME_THRES_TX1_13 | | | | | | | | FRAME_THRES_TX1_12 | | | | | | | |
| R/W-0h | | | | | | | | R/W-0h | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-57. IRQFRAMETHOLDTX03 Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-24 | FRAME_THRES_TX1_15 | R/W | 0h | FRAME threshold value for tx_pkt_cmp_0 for USB0 Endpoint 15. |
| 23-16 | FRAME_THRES_TX1_14 | R/W | 0h | FRAME threshold value for tx_pkt_cmp_0 for USB0 Endpoint 14. |
| 15-8 | FRAME_THRES_TX1_13 | R/W | 0h | FRAME threshold value for tx_pkt_cmp_0 for USB0 Endpoint 13. |
| 7-0 | FRAME_THRES_TX1_12 | R/W | 0h | FRAME threshold value for tx_pkt_cmp_0 for USB0 Endpoint 12.<br>USBSS IRQ_FRAME_THRSHOLD_TX0_3 Register |

### 16.4.1.29 IRQFRAMETHOLDRX00 Register (offset = 210h) [reset = 0h]

IRQFRAMETHOLDRX00 is shown in Figure 16-50 and described in Table 16-58.

**Figure 16-50. IRQFRAMETHOLDRX00 Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 | 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| FRAME_THRES_RX1_3 | | | | | | | | FRAME_THRES_RX1_2 | | | | | | | |
| R/W-0h | | | | | | | | R/W-0h | | | | | | | |
| **15** | **14** | **13** | **12** | **11** | **10** | **9** | **8** | **7** | **6** | **5** | **4** | **3** | **2** | **1** | **0** |
| FRAME_THRES_RX1_1 | | | | | | | | RESERVED | | | | | | | |
| R/W-0h | | | | | | | | R/W-0h | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-58. IRQFRAMETHOLDRX00 Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-24 | FRAME_THRES_RX1_3 | R/W | 0h | FRAME threshold value for rx_pkt_cmp_0 for USB0 Endpoint 3. |
| 23-16 | FRAME_THRES_RX1_2 | R/W | 0h | FRAME threshold value for rx_pkt_cmp_0 for USB0 Endpoint 2. |
| 15-8 | FRAME_THRES_RX1_1 | R/W | 0h | FRAME threshold value for rx_pkt_cmp_0 for USB0 Endpoint 1. |
| 7-0 | RESERVED | R/W | 0h | |

### 16.4.1.30 IRQFRAMETHOLDRX01 Register (offset = 214h) [reset = 0h]

IRQFRAMETHOLDRX01 is shown in Figure 16-51 and described in Table 16-59.

**Figure 16-51. IRQFRAMETHOLDRX01 Register**

| 31-24 | 23-16 |
|---|---|
| FRAME_THRES_RX1_7 | FRAME_THRES_RX1_6 |
| R/W-0h | R/W-0h |
| **15-8** | **7-0** |
| FRAME_THRES_RX1_5 | FRAME_THRES_RX1_4 |
| R/W-0h | R/W-0h |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-59. IRQFRAMETHOLDRX01 Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-24 | FRAME_THRES_RX1_7 | R/W | 0h | FRAME threshold value for rx_pkt_cmp_0 for USB0 Endpoint 7. |
| 23-16 | FRAME_THRES_RX1_6 | R/W | 0h | FRAME threshold value for rx_pkt_cmp_0 for USB0 Endpoint 6. |
| 15-8 | FRAME_THRES_RX1_5 | R/W | 0h | FRAME threshold value for rx_pkt_cmp_0 for USB0 Endpoint 5. |
| 7-0 | FRAME_THRES_RX1_4 | R/W | 0h | FRAME threshold value for rx_pkt_cmp_0 for USB0 Endpoint 4. USBSS IRQ_FRAME_THRSHOLD_RX0_1 Register |

### 16.4.1.31 IRQFRAMETHOLDRX02 Register (offset = 218h) [reset = 0h]

IRQFRAMETHOLDRX02 is shown in Figure 16-52 and described in Table 16-60.

**Figure 16-52. IRQFRAMETHOLDRX02 Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 | 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| FRAME_THRES_RX1_11 | | | | | | | | FRAME_THRES_RX1_10 | | | | | | | |
| R/W-0h | | | | | | | | R/W-0h | | | | | | | |
| **15** | **14** | **13** | **12** | **11** | **10** | **9** | **8** | **7** | **6** | **5** | **4** | **3** | **2** | **1** | **0** |
| FRAME_THRES_RX1_9 | | | | | | | | FRAME_THRES_RX1_8 | | | | | | | |
| R/W-0h | | | | | | | | R/W-0h | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-60. IRQFRAMETHOLDRX02 Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-24 | FRAME_THRES_RX1_11 | R/W | 0h | FRAME threshold value for rx_pkt_cmp_0 for USB0 Endpoint 11. |
| 23-16 | FRAME_THRES_RX1_10 | R/W | 0h | FRAME threshold value for rx_pkt_cmp_0 for USB0 Endpoint 10. |
| 15-8 | FRAME_THRES_RX1_9 | R/W | 0h | FRAME threshold value for rx_pkt_cmp_0 for USB0 Endpoint 9. |
| 7-0 | FRAME_THRES_RX1_8 | R/W | 0h | FRAME threshold value for rx_pkt_cmp_0 for USB0 Endpoint 8.<br>USBSS IRQ_FRAME_THRSHOLD_RX0_2 Register |

### 16.4.1.32 IRQFRAMETHOLDRX03 Register (offset = 21Ch) [reset = 0h]

IRQFRAMETHOLDRX03 is shown in Figure 16-53 and described in Table 16-61.

**Figure 16-53. IRQFRAMETHOLDRX03 Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 | 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| FRAME_THRES_RX1_15 | | | | | | | | FRAME_THRES_RX1_14 | | | | | | | |
| R/W-0h | | | | | | | | R/W-0h | | | | | | | |
| **15** | **14** | **13** | **12** | **11** | **10** | **9** | **8** | **7** | **6** | **5** | **4** | **3** | **2** | **1** | **0** |
| FRAME_THRES_RX1_13 | | | | | | | | FRAME_THRES_RX1_12 | | | | | | | |
| R/W-0h | | | | | | | | R/W-0h | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-61. IRQFRAMETHOLDRX03 Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-24 | FRAME_THRES_RX1_15 | R/W | 0h | FRAME threshold value for rx_pkt_cmp_0 for USB0 Endpoint 15. |
| 23-16 | FRAME_THRES_RX1_14 | R/W | 0h | FRAME threshold value for rx_pkt_cmp_0 for USB0 Endpoint 14. |
| 15-8 | FRAME_THRES_RX1_13 | R/W | 0h | FRAME threshold value for rx_pkt_cmp_0 for USB0 Endpoint 13. |
| 7-0 | FRAME_THRES_RX1_12 | R/W | 0h | FRAME threshold value for rx_pkt_cmp_0 for USB0 Endpoint 12.<br>USBSS IRQ_FRAME_THRSHOLD_RX0_3 Register |

### 16.4.1.33 IRQFRAMETHOLDTX10 Register (offset = 220h) [reset = 0h]

IRQFRAMETHOLDTX10 is shown in Figure 16-54 and described in Table 16-62.

**Figure 16-54. IRQFRAMETHOLDTX10 Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 | 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| FRAME_THRES_TX1_3 | | | | | | | | FRAME_THRES_TX1_2 | | | | | | | |
| R/W-0h | | | | | | | | R/W-0h | | | | | | | |
| **15** | **14** | **13** | **12** | **11** | **10** | **9** | **8** | **7** | **6** | **5** | **4** | **3** | **2** | **1** | **0** |
| FRAME_THRES_TX1_1 | | | | | | | | RESERVED | | | | | | | |
| R/W-0h | | | | | | | | R/W-0h | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-62. IRQFRAMETHOLDTX10 Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-24 | FRAME_THRES_TX1_3 | R/W | 0h | FRAME threshold value for tx_pkt_cmp_0 for USB1 Endpoint 3. |
| 23-16 | FRAME_THRES_TX1_2 | R/W | 0h | FRAME threshold value for tx_pkt_cmp_0 for USB1 Endpoint 2. |
| 15-8 | FRAME_THRES_TX1_1 | R/W | 0h | FRAME threshold value for tx_pkt_cmp_0 for USB1 Endpoint 1. |
| 7-0 | RESERVED | R/W | 0h | |

### 16.4.1.34 IRQFRAMETHOLDTX11 Register (offset = 224h) [reset = 0h]

IRQFRAMETHOLDTX11 is shown in Figure 16-55 and described in Table 16-63.

**Figure 16-55. IRQFRAMETHOLDTX11 Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 | 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| FRAME_THRES_TX1_7 | | | | | | | | FRAME_THRES_TX1_6 | | | | | | | |
| R/W-0h | | | | | | | | R/W-0h | | | | | | | |
| **15** | **14** | **13** | **12** | **11** | **10** | **9** | **8** | **7** | **6** | **5** | **4** | **3** | **2** | **1** | **0** |
| FRAME_THRES_TX1_5 | | | | | | | | FRAME_THRES_TX1_4 | | | | | | | |
| R/W-0h | | | | | | | | R/W-0h | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-63. IRQFRAMETHOLDTX11 Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-24 | FRAME_THRES_TX1_7 | R/W | 0h | FRAME threshold value for tx_pkt_cmp_0 for USB1 Endpoint 7. |
| 23-16 | FRAME_THRES_TX1_6 | R/W | 0h | FRAME threshold value for tx_pkt_cmp_0 for USB1 Endpoint 6. |
| 15-8 | FRAME_THRES_TX1_5 | R/W | 0h | FRAME threshold value for tx_pkt_cmp_0 for USB1 Endpoint 5. |
| 7-0 | FRAME_THRES_TX1_4 | R/W | 0h | FRAME threshold value for tx_pkt_cmp_0 for USB1 Endpoint 4.<br>USBSS IRQ_FRAME_THRSHOLD_TX1_1 Register |

### 16.4.1.35 IRQFRAMETHOLDTX12 Register (offset = 228h) [reset = 0h]

IRQFRAMETHOLDTX12 is shown in Figure 16-56 and described in Table 16-64.

**Figure 16-56. IRQFRAMETHOLDTX12 Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 | 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| FRAME_THRES_TX1_11 | | | | | | | | FRAME_THRES_TX1_10 | | | | | | | |
| R/W-0h | | | | | | | | R/W-0h | | | | | | | |
| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 | 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
| FRAME_THRES_TX1_9 | | | | | | | | FRAME_THRES_TX1_8 | | | | | | | |
| R/W-0h | | | | | | | | R/W-0h | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-64. IRQFRAMETHOLDTX12 Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-24 | FRAME_THRES_TX1_11 | R/W | 0h | FRAME threshold value for tx_pkt_cmp_0 for USB1 Endpoint 11. |
| 23-16 | FRAME_THRES_TX1_10 | R/W | 0h | FRAME threshold value for tx_pkt_cmp_0 for USB1 Endpoint 10. |
| 15-8 | FRAME_THRES_TX1_9 | R/W | 0h | FRAME threshold value for tx_pkt_cmp_0 for USB1 Endpoint 9. |
| 7-0 | FRAME_THRES_TX1_8 | R/W | 0h | FRAME threshold value for tx_pkt_cmp_0 for USB1 Endpoint 8.<br>USBSS IRQ_FRAME_THRSHOLD_TX1_2 Register |

### 16.4.1.36 IRQFRAMETHOLDTX13 Register (offset = 22Ch) [reset = 0h]

IRQFRAMETHOLDTX13 is shown in Figure 16-57 and described in Table 16-65.

**Figure 16-57. IRQFRAMETHOLDTX13 Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 | 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| FRAME_THRES_TX1_15 | | | | | | | | FRAME_THRES_TX1_14 | | | | | | | |
| R/W-0h | | | | | | | | R/W-0h | | | | | | | |
| **15** | **14** | **13** | **12** | **11** | **10** | **9** | **8** | **7** | **6** | **5** | **4** | **3** | **2** | **1** | **0** |
| FRAME_THRES_TX1_13 | | | | | | | | FRAME_THRES_TX1_12 | | | | | | | |
| R/W-0h | | | | | | | | R/W-0h | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-65. IRQFRAMETHOLDTX13 Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-24 | FRAME_THRES_TX1_15 | R/W | 0h | FRAME threshold value for tx_pkt_cmp_0 for USB1 Endpoint 15. |
| 23-16 | FRAME_THRES_TX1_14 | R/W | 0h | FRAME threshold value for tx_pkt_cmp_0 for USB1 Endpoint 14. |
| 15-8 | FRAME_THRES_TX1_13 | R/W | 0h | FRAME threshold value for tx_pkt_cmp_0 for USB1 Endpoint 13. |
| 7-0 | FRAME_THRES_TX1_12 | R/W | 0h | FRAME threshold value for tx_pkt_cmp_0 for USB1 Endpoint 12. USBSS IRQ_FRAME_THRSHOLD_TX1_3 Register |

### 16.4.1.37 IRQFRAMETHOLDRX10 Register (offset = 230h) [reset = 0h]

IRQFRAMETHOLDRX10 is shown in Figure 16-58 and described in Table 16-66.

**Figure 16-58. IRQFRAMETHOLDRX10 Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 | 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| FRAME_THRES_RX1_3 | | | | | | | | FRAME_THRES_RX1_2 | | | | | | | |
| R/W-0h | | | | | | | | R/W-0h | | | | | | | |
| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 | 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
| FRAME_THRES_RX1_1 | | | | | | | | RESERVED | | | | | | | |
| R/W-0h | | | | | | | | R/W-0h | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-66. IRQFRAMETHOLDRX10 Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-24 | FRAME_THRES_RX1_3 | R/W | 0h | FRAME threshold value for rx_pkt_cmp_0 for USB1 Endpoint 3. |
| 23-16 | FRAME_THRES_RX1_2 | R/W | 0h | FRAME threshold value for rx_pkt_cmp_0 for USB1 Endpoint 2. |
| 15-8 | FRAME_THRES_RX1_1 | R/W | 0h | FRAME threshold value for rx_pkt_cmp_0 for USB1 Endpoint 1. |
| 7-0 | RESERVED | R/W | 0h | |

### 16.4.1.38 IRQFRAMETHOLDRX11 Register (offset = 234h) [reset = 0h]

IRQFRAMETHOLDRX11 is shown in Figure 16-59 and described in Table 16-67.

**Figure 16-59. IRQFRAMETHOLDRX11 Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 | 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| FRAME_THRES_RX1_7 | | | | | | | | FRAME_THRES_RX1_6 | | | | | | | |
| R/W-0h | | | | | | | | R/W-0h | | | | | | | |
| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 | 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
| FRAME_THRES_RX1_5 | | | | | | | | FRAME_THRES_RX1_4 | | | | | | | |
| R/W-0h | | | | | | | | R/W-0h | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-67. IRQFRAMETHOLDRX11 Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-24 | FRAME_THRES_RX1_7 | R/W | 0h | FRAME threshold value for rx_pkt_cmp_0 for USB1 Endpoint 7. |
| 23-16 | FRAME_THRES_RX1_6 | R/W | 0h | FRAME threshold value for rx_pkt_cmp_0 for USB1 Endpoint 6. |
| 15-8 | FRAME_THRES_RX1_5 | R/W | 0h | FRAME threshold value for rx_pkt_cmp_0 for USB1 Endpoint 5. |
| 7-0 | FRAME_THRES_RX1_4 | R/W | 0h | FRAME threshold value for rx_pkt_cmp_0 for USB1 Endpoint 4. USBSS IRQ_FRAME_THRSHOLD_RX1_1 Register |

### 16.4.1.39 IRQFRAMETHOLDRX12 Register (offset = 238h) [reset = 0h]

IRQFRAMETHOLDRX12 is shown in Figure 16-60 and described in Table 16-68.

**Figure 16-60. IRQFRAMETHOLDRX12 Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 | 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| FRAME_THRES_RX1_11 | | | | | | | | FRAME_THRES_RX1_10 | | | | | | | |
| R/W-0h | | | | | | | | R/W-0h | | | | | | | |
| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 | 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
| FRAME_THRES_RX1_9 | | | | | | | | FRAME_THRES_RX1_8 | | | | | | | |
| R/W-0h | | | | | | | | R/W-0h | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-68. IRQFRAMETHOLDRX12 Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-24 | FRAME_THRES_RX1_11 | R/W | 0h | FRAME threshold value for rx_pkt_cmp_0 for USB1 Endpoint 11. |
| 23-16 | FRAME_THRES_RX1_10 | R/W | 0h | FRAME threshold value for rx_pkt_cmp_0 for USB1 Endpoint 10. |
| 15-8 | FRAME_THRES_RX1_9 | R/W | 0h | FRAME threshold value for rx_pkt_cmp_0 for USB1 Endpoint 9. |
| 7-0 | FRAME_THRES_RX1_8 | R/W | 0h | FRAME threshold value for rx_pkt_cmp_0 for USB1 Endpoint 8.<br>USBSS IRQ_FRAME_THRSHOLD_RX1_2 Register |

### 16.4.1.40 IRQFRAMETHOLDRX13 Register (offset = 23Ch) [reset = 0h]

IRQFRAMETHOLDRX13 is shown in Figure 16-61 and described in Table 16-69.

**Figure 16-61. IRQFRAMETHOLDRX13 Register**

| 31-24 | 23-16 |
|---|---|
| FRAME_THRES_RX1_15 | FRAME_THRES_RX1_14 |
| R/W-0h | R/W-0h |
| **15-8** | **7-0** |
| FRAME_THRES_RX1_13 | FRAME_THRES_RX1_12 |
| R/W-0h | R/W-0h |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-69. IRQFRAMETHOLDRX13 Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-24 | FRAME_THRES_RX1_15 | R/W | 0h | FRAME threshold value for rx_pkt_cmp_0 for USB1 Endpoint 15. |
| 23-16 | FRAME_THRES_RX1_14 | R/W | 0h | FRAME threshold value for rx_pkt_cmp_0 for USB1 Endpoint 14. |
| 15-8 | FRAME_THRES_RX1_13 | R/W | 0h | FRAME threshold value for rx_pkt_cmp_0 for USB1 Endpoint 13. |
| 7-0 | FRAME_THRES_RX1_12 | R/W | 0h | FRAME threshold value for rx_pkt_cmp_0 for USB1 Endpoint 12.<br>USBSS IRQ_FRAME_THRSHOLD_RX1_3 Register |

### 16.4.1.41 IRQFRAMEENABLE0 Register (offset = 240h) [reset = 0h]

IRQFRAMEENABLE0 is shown in Figure 16-62 and described in Table 16-70.

**Figure 16-62. IRQFRAMEENABLE0 Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| FRAME_EN_RX0_15 | RESERVED | | | | | | |
| R/W-0h | R/W-0h | | | | | | |
| **23** | **22** | **21** | **20** | **19** | **18** | **17** | **16** |
| RESERVED | | | | | | FRAME_EN_RX0_1 | RESERVED |
| R/W-0h | | | | | | R/W-0h | R/W-0h |
| **15** | **14** | **13** | **12** | **11** | **10** | **9** | **8** |
| FRAME_EN_TX0_15 | RESERVED | | | | | | |
| R/W-0h | R/W-0h | | | | | | |
| **7** | **6** | **5** | **4** | **3** | **2** | **1** | **0** |
| RESERVED | | | | | | FRAME_EN_TX0_1 | RESERVED |
| R/W-0h | | | | | | R/W-0h | R/W-0h |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-70. IRQFRAMEENABLE0 Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31 | FRAME_EN_RX0_15 | R/W | 0h | FRAME threshold enable value for rx_pkt_cmp_0 for USB0 Endpoint 15.<br>...<br>...<br>...<br>...<br>... |
| 30-18 | RESERVED | R/W | 0h | |
| 17 | FRAME_EN_RX0_1 | R/W | 0h | FRAME threshold enable value for rx_pkt_cmp_0 for USB0 Endpoint 1. |
| 16 | RESERVED | R/W | 0h | |
| 15 | FRAME_EN_TX0_15 | R/W | 0h | FRAME threshold enable value for tx_pkt_cmp_0 for USB0 Endpoint 15.<br>...<br>...<br>...<br>...<br>... |
| 14-2 | RESERVED | R/W | 0h | |
| 1 | FRAME_EN_TX0_1 | R/W | 0h | FRAME threshold enable value for tx_pkt_cmp_0 for USB0 Endpoint 1. |
| 0 | RESERVED | R/W | 0h | |

#### 16.4.1.42 IRQFRAMEENABLE1 Register (offset = 244h) [reset = 0h]

IRQFRAMEENABLE1 is shown in Figure 16-63 and described in Table 16-71.

**Figure 16-63. IRQFRAMEENABLE1 Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| FRAME_EN_RX1_15 | RESERVED | | | | | | |
| R/W-0h | R/W-0h | | | | | | |
| **23** | **22** | **21** | **20** | **19** | **18** | **17** | **16** |
| RESERVED | | | | | | FRAME_EN_RX1_1 | RESERVED |
| R/W-0h | | | | | | R/W-0h | R/W-0h |
| **15** | **14** | **13** | **12** | **11** | **10** | **9** | **8** |
| FRAME_EN_TX1_15 | RESERVED | | | | | | |
| R/W-0h | R/W-0h | | | | | | |
| **7** | **6** | **5** | **4** | **3** | **2** | **1** | **0** |
| RESERVED | | | | | | FRAME_EN_TX1_1 | RESERVED |
| R/W-0h | | | | | | R/W-0h | R/W-0h |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-71. IRQFRAMEENABLE1 Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31 | FRAME_EN_RX1_15 | R/W | 0h | FRAME threshold enable value for rx_pkt_cmp_1 for USB1 Endpoint 15.<br>...<br>...<br>...<br>...<br>... |
| 30-18 | RESERVED | R/W | 0h | |
| 17 | FRAME_EN_RX1_1 | R/W | 0h | FRAME threshold enable value for rx_pkt_cmp_1 for USB1 Endpoint 1. |
| 16 | RESERVED | R/W | 0h | |
| 15 | FRAME_EN_TX1_15 | R/W | 0h | FRAME threshold enable value for tx_pkt_cmp_1 for USB1 Endpoint 15.<br>...<br>...<br>...<br>...<br>... |
| 14-2 | RESERVED | R/W | 0h | |
| 1 | FRAME_EN_TX1_1 | R/W | 0h | FRAME threshold enable value for tx_pkt_cmp_1 for USB1 Endpoint 1. |
| 0 | RESERVED | R/W | 0h | |

### *16.4.2 USB0_CTRL Registers*

Table 16-72 lists the memory-mapped registers for the USB0_CTRL. All register offset addresses not listed in Table 16-72 should be considered as reserved locations and the register contents should not be modified.

**Table 16-72. USB0_CTRL Registers**

| Offset | Acronym | Register Name | Section |
|---|---|---|---|
| 1000h | USB0REV | | Section 16.4.2.1 |
| 1014h | USB0CTRL | | Section 16.4.2.2 |
| 1018h | USB0STAT | | Section 16.4.2.3 |
| 1020h | USB0IRQMSTAT | | Section 16.4.2.4 |
| 1028h | USB0IRQSTATRAW0 | | Section 16.4.2.5 |
| 102Ch | USB0IRQSTATRAW1 | | Section 16.4.2.6 |
| 1030h | USB0IRQSTAT0 | | Section 16.4.2.7 |
| 1034h | USB0IRQSTAT1 | | Section 16.4.2.8 |
| 1038h | USB0IRQENABLESET0 | | Section 16.4.2.9 |
| 103Ch | USB0IRQENABLESET1 | | Section 16.4.2.10 |
| 1040h | USB0IRQENABLECLR0 | | Section 16.4.2.11 |
| 1044h | USB0IRQENABLECLR1 | | Section 16.4.2.12 |
| 1070h | USB0TXMODE | | Section 16.4.2.13 |
| 1074h | USB0RXMODE | | Section 16.4.2.14 |
| 1080h | USB0GENRNDISEP1 | | Section 16.4.2.15 |
| 1084h | USB0GENRNDISEP2 | | Section 16.4.2.16 |
| 1088h | USB0GENRNDISEP3 | | Section 16.4.2.17 |
| 108Ch | USB0GENRNDISEP4 | | Section 16.4.2.18 |
| 1090h | USB0GENRNDISEP5 | | Section 16.4.2.19 |
| 1094h | USB0GENRNDISEP6 | | Section 16.4.2.20 |
| 1098h | USB0GENRNDISEP7 | | Section 16.4.2.21 |
| 109Ch | USB0GENRNDISEP8 | | Section 16.4.2.22 |
| 10A0h | USB0GENRNDISEP9 | | Section 16.4.2.23 |
| 10A4h | USB0GENRNDISEP10 | | Section 16.4.2.24 |
| 10A8h | USB0GENRNDISEP11 | | Section 16.4.2.25 |
| 10ACh | USB0GENRNDISEP12 | | Section 16.4.2.26 |
| 10B0h | USB0GENRNDISEP13 | | Section 16.4.2.27 |
| 10B4h | USB0GENRNDISEP14 | | Section 16.4.2.28 |
| 10B8h | USB0GENRNDISEP15 | | Section 16.4.2.29 |
| 10D0h | USB0AUTOREQ | | Section 16.4.2.30 |
| 10D4h | USB0SRPFIXTIME | | Section 16.4.2.31 |
| 10D8h | USB0_TDOWN | | Section 16.4.2.32 |
| 10E0h | USB0UTMI | | Section 16.4.2.33 |
| 10E4h | USB0MGCUTMILB | | Section 16.4.2.34 |
| 10E8h | USB0MODE | | Section 16.4.2.35 |

#### 16.4.2.1 USB0REV Register (offset = 1000h) [reset = 4EA20800h]

USB0REV is shown in Figure 16-64 and described in Table 16-73.

**Figure 16-64. USB0REV Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 | 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| SCHEME | | RESERVED | | FUNC | | | | | | | | | | | |
| R-1h | | R-0h | | R-EA2h | | | | | | | | | | | |
| **15** | **14** | **13** | **12** | **11** | **10** | **9** | **8** | **7** | **6** | **5** | **4** | **3** | **2** | **1** | **0** |
| R_RTL | | | | | X_MAJOR | | | CUSTOM | | Y_MINOR | | | | | |
| R-1h | | | | | R-0h | | | R-0h | | R-0h | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-73. USB0REV Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-30 | SCHEME | R | 1h | Used to distinguish between legacy interface scheme and current.<br>0 = Legacy<br>1 = Current |
| 29-28 | RESERVED | R | 0h | |
| 27-16 | FUNC | R | EA2h | Function indicates a software compatible module family. |
| 15-11 | R_RTL | R | 1h | RTL revision.<br>Will vary depending on release. |
| 10-8 | X_MAJOR | R | 0h | Major revision. |
| 7-6 | CUSTOM | R | 0h | Custom revision |
| 5-0 | Y_MINOR | R | 0h | Minor revision.<br>USB0 Revision Register |

### 16.4.2.2 USB0CTRL Register (offset = 1014h) [reset = 0h]

USB0CTRL is shown in Figure 16-65 and described in Table 16-74.

**Figure 16-65. USB0CTRL Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| DIS_DEB | DIS_SRP | RESERVED | | | | | |
| R/W-0h | R/W-0h | R/W-0h | | | | | |
| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
| RESERVED | | | | | | | |
| R/W-0h | | | | | | | |
| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
| RESERVED | | | | | | | |
| R/W-0h | | | | | | | |
| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
| RESERVED | | SOFT_RESET_ISOLATION | RNDIS | UINT | RESERVED | CLKFACK | SOFT_RESET |
| R/W-0h | | R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-74. USB0CTRL Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31 | DIS_DEB | R/W | 0h | Disable the VBUS debouncer circuit fix |
| 30 | DIS_SRP | R/W | 0h | Disable the OTG Session Request Protocol (SRP) AVALID circuit fix.<br>When enabled (=0) this allows additional time for the VBUS signal to be measured against the VBUS thresholds.<br>The time is specified in the USB0 SRP Fix Time Register. |
| 29-6 | RESERVED | R/W | 0h | |
| 5 | SOFT_RESET_ISOLATION | R/W | 0h | Soft reset isolation.<br>When high this bit forces all USB0 signals that connect to the USBSS to known values during a soft reset via bit 0 of this register.<br>This bit should be set high prior to setting bit 0 and cleared after bit 0 is cleared. |
| 4 | RNDIS | R/W | 0h | Global RNDIS mode enable for all endpoints. |
| 3 | UINT | R/W | 0h | USB interrupt enable<br>1 = Legacy<br>0 = Current (recommended setting)<br>If uint is set high, then the mentor controller generic interrupt for USB[9] will be generated (if enabled).<br>This requires S/W to read the mentor controller's registers to determine which interrupt from USB[0] to USB[7] that occurred.<br>If uint is set low, then the usb20otg_f module will automatically read the mentor controller's registers and set the appropriate interrupt from USB[0] to USB[7] (if enabled).<br>The generic interrupt for USB[9] will not be generated. |
| 2 | RESERVED | R/W | 0h | |
| 1 | CLKFACK | R/W | 0h | Clock stop fast ack enable. |

**Table 16-74. USB0CTRL Register Field Descriptions (continued)**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 0 | SOFT_RESET | R/W | 0h | Software reset of USB0.<br>Write 0 = no action<br>Write 1 = Initiate software reset<br>Read 0 = Reset done, no action<br>Read 1 = Reset ongoing<br>Both the soft_reset and soft_reset_isolation bits should be asserted simultaneously.<br>This will cause the following sequence of actions to occur over multiple cycles:<br>- All USB0 output signals will go to a known constant value via multiplexers.<br>This removes the possibility of timing errors due to the asynchronous resets.<br>- All USB0 registers will be reset.<br>- The USB0 resets will be de-asserted.<br>- The reset isolation multiplexer inputs will be de-selected.<br>- Both the soft_reset and soft_reset_isolation bits will be automatically cleared.<br>Setting only the soft_reset_isolation bit will cause all USB0 output signals to go to a known constant value via multiplexers.<br>This will prevent future access to USB0.<br>To clear this condition the USBSS must be reset via a hard or soft reset. |

### 16.4.2.3 USB0STAT Register (offset = 1018h) [reset = 0h]

USB0STAT is shown in Figure 16-66 and described in Table 16-75.

**Figure 16-66. USB0STAT Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| RESERVED | | | | | | | |
| R-0h | | | | | | | |
| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
| RESERVED | | | | | | | |
| R-0h | | | | | | | |
| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
| RESERVED | | | | | | | |
| R-0h | | | | | | | |
| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
| RESERVED | | | | | | | DRVVBUS |
| R-0h | | | | | | | R-0h |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-75. USB0STAT Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-1 | RESERVED | R | 0h | |
| 0 | DRVVBUS | R | 0h | Current DRVVBUS value USB0 Status Register |

### 16.4.2.4 USB0IRQMSTAT Register (offset = 1020h) [reset = 0h]

USB0IRQMSTAT is shown in Figure 16-67 and described in Table 16-76.

**Figure 16-67. USB0IRQMSTAT Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| RESERVED | | | | | | | |
| R-0h | | | | | | | |
| **23** | **22** | **21** | **20** | **19** | **18** | **17** | **16** |
| RESERVED | | | | | | | |
| R-0h | | | | | | | |
| **15** | **14** | **13** | **12** | **11** | **10** | **9** | **8** |
| RESERVED | | | | | | | |
| R-0h | | | | | | | |
| **7** | **6** | **5** | **4** | **3** | **2** | **1** | **0** |
| RESERVED | | | | | | BANK1 | BANK0 |
| R-0h | | | | | | R-0h | R-0h |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-76. USB0IRQMSTAT Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-2 | RESERVED | R | 0h | |
| 1 | BANK1 | R | 0h | 0: No events pending from IRQ_STATUS_1<br>1: At least one event is pending from IRQ_STATUS_1 |
| 0 | BANK0 | R | 0h | 0: No events pending from IRQ_STATUS_0<br>1: At least one event is pending from IRQ_STATUS_0 USB0 IRQ_MERGED_STATUS Register |

### 16.4.2.5 USB0IRQSTATRAW0 Register (offset = 1028h) [reset = 0h]

USB0IRQSTATRAW0 is shown in Figure 16-68 and described in Table 16-77.

**Figure 16-68. USB0IRQSTATRAW0 Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| RX_EP_15 | RX_EP_14 | RX_EP_13 | RX_EP_12 | RX_EP_11 | RX_EP_10 | RX_EP_9 | RX_EP_8 |
| R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h |
| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
| RX_EP_7 | RX_EP_6 | RX_EP_5 | RX_EP_4 | RX_EP_3 | RX_EP_2 | RX_EP_1 | RESERVED |
| R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h |
| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
| TX_EP_15 | TX_EP_14 | TX_EP_13 | TX_EP_12 | TX_EP_11 | TX_EP_10 | TX_EP_9 | TX_EP_8 |
| R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h |
| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
| TX_EP_7 | TX_EP_6 | TX_EP_5 | TX_EP_4 | TX_EP_3 | TX_EP_2 | TX_EP_1 | TX_EP_0 |
| R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-77. USB0IRQSTATRAW0 Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31 | RX_EP_15 | R/W | 0h | Interrupt status for RX endpoint 15 |
| 30 | RX_EP_14 | R/W | 0h | Interrupt status for RX endpoint 14 |
| 29 | RX_EP_13 | R/W | 0h | Interrupt status for RX endpoint 13 |
| 28 | RX_EP_12 | R/W | 0h | Interrupt status for RX endpoint 12 |
| 27 | RX_EP_11 | R/W | 0h | Interrupt status for RX endpoint 11 |
| 26 | RX_EP_10 | R/W | 0h | Interrupt status for RX endpoint 10 |
| 25 | RX_EP_9 | R/W | 0h | Interrupt status for RX endpoint 9 |
| 24 | RX_EP_8 | R/W | 0h | Interrupt status for RX endpoint 8 |
| 23 | RX_EP_7 | R/W | 0h | Interrupt status for RX endpoint 7 |
| 22 | RX_EP_6 | R/W | 0h | Interrupt status for RX endpoint 6 |
| 21 | RX_EP_5 | R/W | 0h | Interrupt status for RX endpoint 5 |
| 20 | RX_EP_4 | R/W | 0h | Interrupt status for RX endpoint 4 |
| 19 | RX_EP_3 | R/W | 0h | Interrupt status for RX endpoint 3 |
| 18 | RX_EP_2 | R/W | 0h | Interrupt status for RX endpoint 2 |
| 17 | RX_EP_1 | R/W | 0h | Interrupt status for RX endpoint 1 |
| 16 | RESERVED | R/W | 0h | |
| 15 | TX_EP_15 | R/W | 0h | Interrupt status for TX endpoint 15 |
| 14 | TX_EP_14 | R/W | 0h | Interrupt status for TX endpoint 14 |
| 13 | TX_EP_13 | R/W | 0h | Interrupt status for TX endpoint 13 |
| 12 | TX_EP_12 | R/W | 0h | Interrupt status for TX endpoint 12 |
| 11 | TX_EP_11 | R/W | 0h | Interrupt status for TX endpoint 11 |
| 10 | TX_EP_10 | R/W | 0h | Interrupt status for TX endpoint 10 |
| 9 | TX_EP_9 | R/W | 0h | Interrupt status for TX endpoint 9 |
| 8 | TX_EP_8 | R/W | 0h | Interrupt status for TX endpoint 8 |
| 7 | TX_EP_7 | R/W | 0h | Interrupt status for TX endpoint 7 |
| 6 | TX_EP_6 | R/W | 0h | Interrupt status for TX endpoint 6 |

**Table 16-77. USB0IRQSTATRAW0 Register Field Descriptions (continued)**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 5 | TX_EP_5 | R/W | 0h | Interrupt status for TX endpoint 5 |
| 4 | TX_EP_4 | R/W | 0h | Interrupt status for TX endpoint 4 |
| 3 | TX_EP_3 | R/W | 0h | Interrupt status for TX endpoint 3 |
| 2 | TX_EP_2 | R/W | 0h | Interrupt status for TX endpoint 2 |
| 1 | TX_EP_1 | R/W | 0h | Interrupt status for TX endpoint 1 |
| 0 | TX_EP_0 | R/W | 0h | Interrupt status for TX endpoint 0 USB0 IRQ_STATUS_RAW_0 Register |

### 16.4.2.6 USB0IRQSTATRAW1 Register (offset = 102Ch) [reset = 0h]

USB0IRQSTATRAW1 is shown in Figure 16-69 and described in Table 16-78.

**Figure 16-69. USB0IRQSTATRAW1 Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| TX_FIFO_15 | TX_FIFO_14 | TX_FIFO_13 | TX_FIFO_12 | TX_FIFO_11 | TX_FIFO_10 | TX_FIFO_9 | TX_FIFO_8 |
| R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h |
| **23** | **22** | **21** | **20** | **19** | **18** | **17** | **16** |
| TX_FIFO_7 | TX_FIFO_6 | TX_FIFO_5 | TX_FIFO_4 | TX_FIFO_3 | TX_FIFO_2 | TX_FIFO_1 | TX_FIFO_0 |
| R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h |
| **15** | **14** | **13** | **12** | **11** | **10** | **9** | **8** |
| RESERVED | | | | | | USB_9 | USB_8 |
| R/W-0h | | | | | | R/W-0h | R/W-0h |
| **7** | **6** | **5** | **4** | **3** | **2** | **1** | **0** |
| USB_7 | USB_6 | USB_5 | USB_4 | USB_3 | USB_2 | USB_1 | USB_0 |
| R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-78. USB0IRQSTATRAW1 Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31 | TX_FIFO_15 | R/W | 0h | Interrupt status for TX FIFO endpoint 15 |
| 30 | TX_FIFO_14 | R/W | 0h | Interrupt status for TX FIFO endpoint 14 |
| 29 | TX_FIFO_13 | R/W | 0h | Interrupt status for TX FIFO endpoint 13 |
| 28 | TX_FIFO_12 | R/W | 0h | Interrupt status for TX FIFO endpoint 12 |
| 27 | TX_FIFO_11 | R/W | 0h | Interrupt status for TX FIFO endpoint 11 |
| 26 | TX_FIFO_10 | R/W | 0h | Interrupt status for TX FIFO endpoint 10 |
| 25 | TX_FIFO_9 | R/W | 0h | Interrupt status for TX FIFO endpoint 9 |
| 24 | TX_FIFO_8 | R/W | 0h | Interrupt status for TX FIFO endpoint 8 |
| 23 | TX_FIFO_7 | R/W | 0h | Interrupt status for TX FIFO endpoint 7 |
| 22 | TX_FIFO_6 | R/W | 0h | Interrupt status for TX FIFO endpoint 6 |
| 21 | TX_FIFO_5 | R/W | 0h | Interrupt status for TX FIFO endpoint 5 |
| 20 | TX_FIFO_4 | R/W | 0h | Interrupt status for TX FIFO endpoint 4 |
| 19 | TX_FIFO_3 | R/W | 0h | Interrupt status for TX FIFO endpoint 3 |
| 18 | TX_FIFO_2 | R/W | 0h | Interrupt status for TX FIFO endpoint 2 |
| 17 | TX_FIFO_1 | R/W | 0h | Interrupt status for TX FIFO endpoint 1 |
| 16 | TX_FIFO_0 | R/W | 0h | Interrupt status for TX FIFO endpoint 0 |
| 15-10 | RESERVED | R/W | 0h | |
| 9 | USB_9 | R/W | 0h | Interrupt status for Mentor controller USB_INT generic interrupt |
| 8 | USB_8 | R/W | 0h | Interrupt status for DRVVBUS level change |
| 7 | USB_7 | R/W | 0h | Interrupt status for VBUS < VBUS valid threshold |
| 6 | USB_6 | R/W | 0h | Interrupt status for SRP detected |
| 5 | USB_5 | R/W | 0h | Interrupt status for device disconnected (host mode) |
| 4 | USB_4 | R/W | 0h | Interrupt status for device connected (host mode) |
| 3 | USB_3 | R/W | 0h | Interrupt status for SOF started |
| 2 | USB_2 | R/W | 0h | Interrupt status for Reset signaling detected (peripheral mode)<br>Babble detected (host mode) |

**Table 16-78. USB0IRQSTATRAW1 Register Field Descriptions (continued)**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 1 | USB_1 | R/W | 0h | Interrupt status for Resume signaling detected |
| 0 | USB_0 | R/W | 0h | Interrupt status for Suspend signaling detected USB0 IRQ_STATUS_RAW_1 Register |

### 16.4.2.7 USB0IRQSTAT0 Register (offset = 1030h) [reset = 0h]

USB0IRQSTAT0 is shown in Figure 16-70 and described in Table 16-79.

**Figure 16-70. USB0IRQSTAT0 Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| RX_EP_15 | RX_EP_14 | RX_EP_13 | RX_EP_12 | RX_EP_11 | RX_EP_10 | RX_EP_9 | RX_EP_8 |
| R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h |
| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
| RX_EP_7 | RX_EP_6 | RX_EP_5 | RX_EP_4 | RX_EP_3 | RX_EP_2 | RX_EP_1 | RESERVED |
| R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h |
| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
| TX_EP_15 | TX_EP_14 | TX_EP_13 | TX_EP_12 | TX_EP_11 | TX_EP_10 | TX_EP_9 | TX_EP_8 |
| R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h |
| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
| TX_EP_7 | TX_EP_6 | TX_EP_5 | TX_EP_4 | TX_EP_3 | TX_EP_2 | TX_EP_1 | TX_EP_0 |
| R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-79. USB0IRQSTAT0 Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31 | RX_EP_15 | R/W | 0h | Interrupt status for RX endpoint 15 |
| 30 | RX_EP_14 | R/W | 0h | Interrupt status for RX endpoint 14 |
| 29 | RX_EP_13 | R/W | 0h | Interrupt status for RX endpoint 13 |
| 28 | RX_EP_12 | R/W | 0h | Interrupt status for RX endpoint 12 |
| 27 | RX_EP_11 | R/W | 0h | Interrupt status for RX endpoint 11 |
| 26 | RX_EP_10 | R/W | 0h | Interrupt status for RX endpoint 10 |
| 25 | RX_EP_9 | R/W | 0h | Interrupt status for RX endpoint 9 |
| 24 | RX_EP_8 | R/W | 0h | Interrupt status for RX endpoint 8 |
| 23 | RX_EP_7 | R/W | 0h | Interrupt status for RX endpoint 7 |
| 22 | RX_EP_6 | R/W | 0h | Interrupt status for RX endpoint 6 |
| 21 | RX_EP_5 | R/W | 0h | Interrupt status for RX endpoint 5 |
| 20 | RX_EP_4 | R/W | 0h | Interrupt status for RX endpoint 4 |
| 19 | RX_EP_3 | R/W | 0h | Interrupt status for RX endpoint 3 |
| 18 | RX_EP_2 | R/W | 0h | Interrupt status for RX endpoint 2 |
| 17 | RX_EP_1 | R/W | 0h | Interrupt status for RX endpoint 1 |
| 16 | RESERVED | R/W | 0h | |
| 15 | TX_EP_15 | R/W | 0h | Interrupt status for TX endpoint 15 |
| 14 | TX_EP_14 | R/W | 0h | Interrupt status for TX endpoint 14 |
| 13 | TX_EP_13 | R/W | 0h | Interrupt status for TX endpoint 13 |
| 12 | TX_EP_12 | R/W | 0h | Interrupt status for TX endpoint 12 |
| 11 | TX_EP_11 | R/W | 0h | Interrupt status for TX endpoint 11 |
| 10 | TX_EP_10 | R/W | 0h | Interrupt status for TX endpoint 10 |
| 9 | TX_EP_9 | R/W | 0h | Interrupt status for TX endpoint 9 |
| 8 | TX_EP_8 | R/W | 0h | Interrupt status for TX endpoint 8 |
| 7 | TX_EP_7 | R/W | 0h | Interrupt status for TX endpoint 7 |
| 6 | TX_EP_6 | R/W | 0h | Interrupt status for TX endpoint 6 |

**Table 16-79. USB0IRQSTAT0 Register Field Descriptions (continued)**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 5 | TX_EP_5 | R/W | 0h | Interrupt status for TX endpoint 5 |
| 4 | TX_EP_4 | R/W | 0h | Interrupt status for TX endpoint 4 |
| 3 | TX_EP_3 | R/W | 0h | Interrupt status for TX endpoint 3 |
| 2 | TX_EP_2 | R/W | 0h | Interrupt status for TX endpoint 2 |
| 1 | TX_EP_1 | R/W | 0h | Interrupt status for TX endpoint 1 |
| 0 | TX_EP_0 | R/W | 0h | Interrupt status for TX endpoint 0 USB0 IRQ_STATUS_0 Register |

### 16.4.2.8 USB0IRQSTAT1 Register (offset = 1034h) [reset = 0h]

USB0IRQSTAT1 is shown in Figure 16-71 and described in Table 16-80.

**Figure 16-71. USB0IRQSTAT1 Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| TX_FIFO_15 | TX_FIFO_14 | TX_FIFO_13 | TX_FIFO_12 | TX_FIFO_11 | TX_FIFO_10 | TX_FIFO_9 | TX_FIFO_8 |
| R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h |
| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
| TX_FIFO_7 | TX_FIFO_6 | TX_FIFO_5 | TX_FIFO_4 | TX_FIFO_3 | TX_FIFO_2 | TX_FIFO_1 | TX_FIFO_0 |
| R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h |
| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
| RESERVED | | | | | | USB_9 | USB_8 |
| R/W-0h | | | | | | R/W-0h | R/W-0h |
| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
| USB_7 | USB_6 | USB_5 | USB_4 | USB_3 | USB_2 | USB_1 | USB_0 |
| R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-80. USB0IRQSTAT1 Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31 | TX_FIFO_15 | R/W | 0h | Interrupt status for TX FIFO endpoint 15 |
| 30 | TX_FIFO_14 | R/W | 0h | Interrupt status for TX FIFO endpoint 14 |
| 29 | TX_FIFO_13 | R/W | 0h | Interrupt status for TX FIFO endpoint 13 |
| 28 | TX_FIFO_12 | R/W | 0h | Interrupt status for TX FIFO endpoint 12 |
| 27 | TX_FIFO_11 | R/W | 0h | Interrupt status for TX FIFO endpoint 11 |
| 26 | TX_FIFO_10 | R/W | 0h | Interrupt status for TX FIFO endpoint 10 |
| 25 | TX_FIFO_9 | R/W | 0h | Interrupt status for TX FIFO endpoint 9 |
| 24 | TX_FIFO_8 | R/W | 0h | Interrupt status for TX FIFO endpoint 8 |
| 23 | TX_FIFO_7 | R/W | 0h | Interrupt status for TX FIFO endpoint 7 |
| 22 | TX_FIFO_6 | R/W | 0h | Interrupt status for TX FIFO endpoint 6 |
| 21 | TX_FIFO_5 | R/W | 0h | Interrupt status for TX FIFO endpoint 5 |
| 20 | TX_FIFO_4 | R/W | 0h | Interrupt status for TX FIFO endpoint 4 |
| 19 | TX_FIFO_3 | R/W | 0h | Interrupt status for TX FIFO endpoint 3 |
| 18 | TX_FIFO_2 | R/W | 0h | Interrupt status for TX FIFO endpoint 2 |
| 17 | TX_FIFO_1 | R/W | 0h | Interrupt status for TX FIFO endpoint 1 |
| 16 | TX_FIFO_0 | R/W | 0h | Interrupt status for TX FIFO endpoint 0 |
| 15-10 | RESERVED | R/W | 0h | |
| 9 | USB_9 | R/W | 0h | Interrupt status for Mentor controller USB_INT generic interrupt |
| 8 | USB_8 | R/W | 0h | Interrupt status for DRVVBUS level change |
| 7 | USB_7 | R/W | 0h | Interrupt status for VBUS < VBUS valid threshold |
| 6 | USB_6 | R/W | 0h | Interrupt status for SRP detected |
| 5 | USB_5 | R/W | 0h | Interrupt status for device disconnected (host mode) |
| 4 | USB_4 | R/W | 0h | Interrupt status for device connected (host mode) |
| 3 | USB_3 | R/W | 0h | Interrupt status for SOF started |
| 2 | USB_2 | R/W | 0h | Interrupt status for Reset signaling detected (peripheral mode)<br>Babble detected (host mode) |

**Table 16-80. USB0IRQSTAT1 Register Field Descriptions (continued)**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 1 | USB_1 | R/W | 0h | Interrupt status for Resume signaling detected |
| 0 | USB_0 | R/W | 0h | Interrupt status for Suspend signaling detected USB0 IRQ_STATUS_1 Register |

### 16.4.2.9 USB0IRQENABLESET0 Register (offset = 1038h) [reset = 0h]

USB0IRQENABLESET0 is shown in Figure 16-72 and described in Table 16-81.

**Figure 16-72. USB0IRQENABLESET0 Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| RX_EP_15 | RX_EP_14 | RX_EP_13 | RX_EP_12 | RX_EP_11 | RX_EP_10 | RX_EP_9 | RX_EP_8 |
| R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h |
| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
| RX_EP_7 | RX_EP_6 | RX_EP_5 | RX_EP_4 | RX_EP_3 | RX_EP_2 | RX_EP_1 | RESERVED |
| R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h |
| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
| TX_EP_15 | TX_EP_14 | TX_EP_13 | TX_EP_12 | TX_EP_11 | TX_EP_10 | TX_EP_9 | TX_EP_8 |
| R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h |
| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
| TX_EP_7 | TX_EP_6 | TX_EP_5 | TX_EP_4 | TX_EP_3 | TX_EP_2 | TX_EP_1 | TX_EP_0 |
| R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-81. USB0IRQENABLESET0 Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31 | RX_EP_15 | R/W | 0h | Interrupt enable for RX endpoint 15 |
| 30 | RX_EP_14 | R/W | 0h | Interrupt enable for RX endpoint 14 |
| 29 | RX_EP_13 | R/W | 0h | Interrupt enable for RX endpoint 13 |
| 28 | RX_EP_12 | R/W | 0h | Interrupt enable for RX endpoint 12 |
| 27 | RX_EP_11 | R/W | 0h | Interrupt enable for RX endpoint 11 |
| 26 | RX_EP_10 | R/W | 0h | Interrupt enable for RX endpoint 10 |
| 25 | RX_EP_9 | R/W | 0h | Interrupt enable for RX endpoint 9 |
| 24 | RX_EP_8 | R/W | 0h | Interrupt enable for RX endpoint 8 |
| 23 | RX_EP_7 | R/W | 0h | Interrupt enable for RX endpoint 7 |
| 22 | RX_EP_6 | R/W | 0h | Interrupt enable for RX endpoint 6 |
| 21 | RX_EP_5 | R/W | 0h | Interrupt enable for RX endpoint 5 |
| 20 | RX_EP_4 | R/W | 0h | Interrupt enable for RX endpoint 4 |
| 19 | RX_EP_3 | R/W | 0h | Interrupt enable for RX endpoint 3 |
| 18 | RX_EP_2 | R/W | 0h | Interrupt enable for RX endpoint 2 |
| 17 | RX_EP_1 | R/W | 0h | Interrupt enable for RX endpoint 1 |
| 16 | RESERVED | R/W | 0h | |
| 15 | TX_EP_15 | R/W | 0h | Interrupt enable for TX endpoint 15 |
| 14 | TX_EP_14 | R/W | 0h | Interrupt enable for TX endpoint 14 |
| 13 | TX_EP_13 | R/W | 0h | Interrupt enable for TX endpoint 13 |
| 12 | TX_EP_12 | R/W | 0h | Interrupt enable for TX endpoint 12 |
| 11 | TX_EP_11 | R/W | 0h | Interrupt enable for TX endpoint 11 |
| 10 | TX_EP_10 | R/W | 0h | Interrupt enable for TX endpoint 10 |
| 9 | TX_EP_9 | R/W | 0h | Interrupt enable for TX endpoint 9 |
| 8 | TX_EP_8 | R/W | 0h | Interrupt enable for TX endpoint 8 |
| 7 | TX_EP_7 | R/W | 0h | Interrupt enable for TX endpoint 7 |
| 6 | TX_EP_6 | R/W | 0h | Interrupt enable for TX endpoint 6 |

**Table 16-81. USB0IRQENABLESET0 Register Field Descriptions (continued)**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 5 | TX_EP_5 | R/W | 0h | Interrupt enable for TX endpoint 5 |
| 4 | TX_EP_4 | R/W | 0h | Interrupt enable for TX endpoint 4 |
| 3 | TX_EP_3 | R/W | 0h | Interrupt enable for TX endpoint 3 |
| 2 | TX_EP_2 | R/W | 0h | Interrupt enable for TX endpoint 2 |
| 1 | TX_EP_1 | R/W | 0h | Interrupt enable for TX endpoint 1 |
| 0 | TX_EP_0 | R/W | 0h | Interrupt enable for TX endpoint 0 USB0 IRQ_ENABLE_SET_0 Register |

### 16.4.2.10 USB0IRQENABLESET1 Register (offset = 103Ch) [reset = 0h]

USB0IRQENABLESET1 is shown in Figure 16-73 and described in Table 16-82.

**Figure 16-73. USB0IRQENABLESET1 Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| TX_FIFO_15 | TX_FIFO_14 | TX_FIFO_13 | TX_FIFO_12 | TX_FIFO_11 | TX_FIFO_10 | TX_FIFO_9 | TX_FIFO_8 |
| R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h |
| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
| TX_FIFO_7 | TX_FIFO_6 | TX_FIFO_5 | TX_FIFO_4 | TX_FIFO_3 | TX_FIFO_2 | TX_FIFO_1 | TX_FIFO_0 |
| R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h |
| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
| RESERVED | | | | | | USB_9 | USB_8 |
| R/W-0h | | | | | | R/W-0h | R/W-0h |
| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
| USB_7 | USB_6 | USB_5 | USB_4 | USB_3 | USB_2 | USB_1 | USB_0 |
| R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-82. USB0IRQENABLESET1 Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31 | TX_FIFO_15 | R/W | 0h | Interrupt enable for TX FIFO endpoint 15 |
| 30 | TX_FIFO_14 | R/W | 0h | Interrupt enable for TX FIFO endpoint 14 |
| 29 | TX_FIFO_13 | R/W | 0h | Interrupt enable for TX FIFO endpoint 13 |
| 28 | TX_FIFO_12 | R/W | 0h | Interrupt enable for TX FIFO endpoint 12 |
| 27 | TX_FIFO_11 | R/W | 0h | Interrupt enable for TX FIFO endpoint 11 |
| 26 | TX_FIFO_10 | R/W | 0h | Interrupt enable for TX FIFO endpoint 10 |
| 25 | TX_FIFO_9 | R/W | 0h | Interrupt enable for TX FIFO endpoint 9 |
| 24 | TX_FIFO_8 | R/W | 0h | Interrupt enable for TX FIFO endpoint 8 |
| 23 | TX_FIFO_7 | R/W | 0h | Interrupt enable for TX FIFO endpoint 7 |
| 22 | TX_FIFO_6 | R/W | 0h | Interrupt enable for TX FIFO endpoint 6 |
| 21 | TX_FIFO_5 | R/W | 0h | Interrupt enable for TX FIFO endpoint 5 |
| 20 | TX_FIFO_4 | R/W | 0h | Interrupt enable for TX FIFO endpoint 4 |
| 19 | TX_FIFO_3 | R/W | 0h | Interrupt enable for TX FIFO endpoint 3 |
| 18 | TX_FIFO_2 | R/W | 0h | Interrupt enable for TX FIFO endpoint 2 |
| 17 | TX_FIFO_1 | R/W | 0h | Interrupt enable for TX FIFO endpoint 1 |
| 16 | TX_FIFO_0 | R/W | 0h | Interrupt enable for TX FIFO endpoint 0 |
| 15-10 | RESERVED | R/W | 0h | |
| 9 | USB_9 | R/W | 0h | Interrupt enable for Mentor controller USB_INT generic interrupt |
| 8 | USB_8 | R/W | 0h | Interrupt enable for DRVVBUS level change |
| 7 | USB_7 | R/W | 0h | Interrupt enable for VBUS < VBUS valid threshold |
| 6 | USB_6 | R/W | 0h | Interrupt enable for SRP detected |
| 5 | USB_5 | R/W | 0h | Interrupt enable for device disconnected (host mode) |
| 4 | USB_4 | R/W | 0h | Interrupt enable for device connected (host mode) |
| 3 | USB_3 | R/W | 0h | Interrupt enable for SOF started |
| 2 | USB_2 | R/W | 0h | Interrupt enable for Reset signaling detected (peripheral mode) Babble detected (host mode) |

**Table 16-82. USB0IRQENABLESET1 Register Field Descriptions (continued)**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 1 | USB_1 | R/W | 0h | Interrupt enable for Resume signaling detected |
| 0 | USB_0 | R/W | 0h | Interrupt enable for Suspend signaling detected USB0 IRQ_ENABLE_SET_1 Register |

### 16.4.2.11 USB0IRQENABLECLR0 Register (offset = 1040h) [reset = 0h]

USB0IRQENABLECLR0 is shown in Figure 16-74 and described in Table 16-83.

**Figure 16-74. USB0IRQENABLECLR0 Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| RX_EP_15 | RX_EP_14 | RX_EP_13 | RX_EP_12 | RX_EP_11 | RX_EP_10 | RX_EP_9 | RX_EP_8 |
| R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h |
| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
| RX_EP_7 | RX_EP_6 | RX_EP_5 | RX_EP_4 | RX_EP_3 | RX_EP_2 | RX_EP_1 | RESERVED |
| R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h |
| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
| TX_EP_15 | TX_EP_14 | TX_EP_13 | TX_EP_12 | TX_EP_11 | TX_EP_10 | TX_EP_9 | TX_EP_8 |
| R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h |
| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
| TX_EP_7 | TX_EP_6 | TX_EP_5 | TX_EP_4 | TX_EP_3 | TX_EP_2 | TX_EP_1 | TX_EP_0 |
| R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-83. USB0IRQENABLECLR0 Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31 | RX_EP_15 | R/W | 0h | Interrupt enable for RX endpoint 15 |
| 30 | RX_EP_14 | R/W | 0h | Interrupt enable for RX endpoint 14 |
| 29 | RX_EP_13 | R/W | 0h | Interrupt enable for RX endpoint 13 |
| 28 | RX_EP_12 | R/W | 0h | Interrupt enable for RX endpoint 12 |
| 27 | RX_EP_11 | R/W | 0h | Interrupt enable for RX endpoint 11 |
| 26 | RX_EP_10 | R/W | 0h | Interrupt enable for RX endpoint 10 |
| 25 | RX_EP_9 | R/W | 0h | Interrupt enable for RX endpoint 9 |
| 24 | RX_EP_8 | R/W | 0h | Interrupt enable for RX endpoint 8 |
| 23 | RX_EP_7 | R/W | 0h | Interrupt enable for RX endpoint 7 |
| 22 | RX_EP_6 | R/W | 0h | Interrupt enable for RX endpoint 6 |
| 21 | RX_EP_5 | R/W | 0h | Interrupt enable for RX endpoint 5 |
| 20 | RX_EP_4 | R/W | 0h | Interrupt enable for RX endpoint 4 |
| 19 | RX_EP_3 | R/W | 0h | Interrupt enable for RX endpoint 3 |
| 18 | RX_EP_2 | R/W | 0h | Interrupt enable for RX endpoint 2 |
| 17 | RX_EP_1 | R/W | 0h | Interrupt enable for RX endpoint 1 |
| 16 | RESERVED | R/W | 0h | |
| 15 | TX_EP_15 | R/W | 0h | Interrupt enable for TX endpoint 15 |
| 14 | TX_EP_14 | R/W | 0h | Interrupt enable for TX endpoint 14 |
| 13 | TX_EP_13 | R/W | 0h | Interrupt enable for TX endpoint 13 |
| 12 | TX_EP_12 | R/W | 0h | Interrupt enable for TX endpoint 12 |
| 11 | TX_EP_11 | R/W | 0h | Interrupt enable for TX endpoint 11 |
| 10 | TX_EP_10 | R/W | 0h | Interrupt enable for TX endpoint 10 |
| 9 | TX_EP_9 | R/W | 0h | Interrupt enable for TX endpoint 9 |
| 8 | TX_EP_8 | R/W | 0h | Interrupt enable for TX endpoint 8 |
| 7 | TX_EP_7 | R/W | 0h | Interrupt enable for TX endpoint 7 |
| 6 | TX_EP_6 | R/W | 0h | Interrupt enable for TX endpoint 6 |

**Table 16-83. USB0IRQENABLECLR0 Register Field Descriptions (continued)**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 5 | TX_EP_5 | R/W | 0h | Interrupt enable for TX endpoint 5 |
| 4 | TX_EP_4 | R/W | 0h | Interrupt enable for TX endpoint 4 |
| 3 | TX_EP_3 | R/W | 0h | Interrupt enable for TX endpoint 3 |
| 2 | TX_EP_2 | R/W | 0h | Interrupt enable for TX endpoint 2 |
| 1 | TX_EP_1 | R/W | 0h | Interrupt enable for TX endpoint 1 |
| 0 | TX_EP_0 | R/W | 0h | Interrupt enable for TX endpoint 0 USB0 IRQ_ENABLE_CLR_0 Register |

### 16.4.2.12 USB0IRQENABLECLR1 Register (offset = 1044h) [reset = 0h]

USB0IRQENABLECLR1 is shown in Figure 16-75 and described in Table 16-84.

**Figure 16-75. USB0IRQENABLECLR1 Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| TX_FIFO_15 | TX_FIFO_14 | TX_FIFO_13 | TX_FIFO_12 | TX_FIFO_11 | TX_FIFO_10 | TX_FIFO_9 | TX_FIFO_8 |
| R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h |
| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
| TX_FIFO_7 | TX_FIFO_6 | TX_FIFO_5 | TX_FIFO_4 | TX_FIFO_3 | TX_FIFO_2 | TX_FIFO_1 | TX_FIFO_0 |
| R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h |
| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
| RESERVED | | | | | | USB_9 | USB_8 |
| R/W-0h | | | | | | R/W-0h | R/W-0h |
| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
| USB_7 | USB_6 | USB_5 | USB_4 | USB_3 | USB_2 | USB_1 | USB_0 |
| R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-84. USB0IRQENABLECLR1 Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31 | TX_FIFO_15 | R/W | 0h | Interrupt enable for TX FIFO endpoint 15 |
| 30 | TX_FIFO_14 | R/W | 0h | Interrupt enable for TX FIFO endpoint 14 |
| 29 | TX_FIFO_13 | R/W | 0h | Interrupt enable for TX FIFO endpoint 13 |
| 28 | TX_FIFO_12 | R/W | 0h | Interrupt enable for TX FIFO endpoint 12 |
| 27 | TX_FIFO_11 | R/W | 0h | Interrupt enable for TX FIFO endpoint 11 |
| 26 | TX_FIFO_10 | R/W | 0h | Interrupt enable for TX FIFO endpoint 10 |
| 25 | TX_FIFO_9 | R/W | 0h | Interrupt enable for TX FIFO endpoint 9 |
| 24 | TX_FIFO_8 | R/W | 0h | Interrupt enable for TX FIFO endpoint 8 |
| 23 | TX_FIFO_7 | R/W | 0h | Interrupt enable for TX FIFO endpoint 7 |
| 22 | TX_FIFO_6 | R/W | 0h | Interrupt enable for TX FIFO endpoint 6 |
| 21 | TX_FIFO_5 | R/W | 0h | Interrupt enable for TX FIFO endpoint 5 |
| 20 | TX_FIFO_4 | R/W | 0h | Interrupt enable for TX FIFO endpoint 4 |
| 19 | TX_FIFO_3 | R/W | 0h | Interrupt enable for TX FIFO endpoint 3 |
| 18 | TX_FIFO_2 | R/W | 0h | Interrupt enable for TX FIFO endpoint 2 |
| 17 | TX_FIFO_1 | R/W | 0h | Interrupt enable for TX FIFO endpoint 1 |
| 16 | TX_FIFO_0 | R/W | 0h | Interrupt enable for TX FIFO endpoint 0 |
| 15-10 | RESERVED | R/W | 0h | |
| 9 | USB_9 | R/W | 0h | Interrupt enable for Mentor controller USB_INT generic interrupt |
| 8 | USB_8 | R/W | 0h | Interrupt enable for DRVVBUS level change |
| 7 | USB_7 | R/W | 0h | Interrupt enable for VBUS < VBUS valid threshold |
| 6 | USB_6 | R/W | 0h | Interrupt enable for SRP detected |
| 5 | USB_5 | R/W | 0h | Interrupt enable for device disconnected (host mode) |
| 4 | USB_4 | R/W | 0h | Interrupt enable for device connected (host mode) |
| 3 | USB_3 | R/W | 0h | Interrupt enable for SOF started |
| 2 | USB_2 | R/W | 0h | Interrupt enable for Reset signaling detected (peripheral mode)<br>Babble detected (host mode) |

**Table 16-84. USB0IRQENABLECLR1 Register Field Descriptions (continued)**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 1 | USB_1 | R/W | 0h | Interrupt enable for Resume signaling detected |
| 0 | USB_0 | R/W | 0h | Interrupt enable for Suspend signaling detected USB0 IRQ_ENABLE_CLR_1 Register |

### 16.4.2.13 USB0TXMODE Register (offset = 1070h) [reset = 0h]

USB0TXMODE is shown in Figure 16-76 and described in Table 16-85.

**Figure 16-76. USB0TXMODE Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| RESERVED | | TX15_MODE | | TX14_MODE | | TX13_MODE | |
| R/W-0h | | R/W-0h | | R/W-0h | | R/W-0h | |
| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
| TX12_MODE | | TX11_MODE | | TX10_MODE | | TX9_MODE | |
| R/W-0h | | R/W-0h | | R/W-0h | | R/W-0h | |
| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
| TX8_MODE | | TX7_MODE | | TX6_MODE | | TX5_MODE | |
| R/W-0h | | R/W-0h | | R/W-0h | | R/W-0h | |
| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
| TX4_MODE | | TX3_MODE | | TX2_MODE | | TX1_MODE | |
| R/W-0h | | R/W-0h | | R/W-0h | | R/W-0h | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-85. USB0TXMODE Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-30 | RESERVED | R/W | 0h | |
| 29-28 | TX15_MODE | R/W | 0h | 00: Transparent Mode on TX endpoint 14<br>01: RNDIS Mode on TX endpoint 14<br>10: CDC Mode on TX endpoint 14<br>11: Generic RNDIS Mode on TX endpoint 14 |
| 27-26 | TX14_MODE | R/W | 0h | 00: Transparent Mode on TX endpoint 13<br>01: RNDIS Mode on TX endpoint 13<br>10: CDC Mode on TX endpoint 13<br>11: Generic RNDIS Mode on TX endpoint 13 |
| 25-24 | TX13_MODE | R/W | 0h | 00: Transparent Mode on TX endpoint 12<br>01: RNDIS Mode on TX endpoint 12<br>10: CDC Mode on TX endpoint 12<br>11: Generic RNDIS Mode on TX endpoint 12 |
| 23-22 | TX12_MODE | R/W | 0h | 00: Transparent Mode on TX endpoint 11<br>01: RNDIS Mode on TX endpoint 11<br>10: CDC Mode on TX endpoint 11<br>11: Generic RNDIS Mode on TX endpoint 11 |
| 21-20 | TX11_MODE | R/W | 0h | 00: Transparent Mode on TX endpoint 10<br>01: RNDIS Mode on TX endpoint 10<br>10: CDC Mode on TX endpoint 10<br>11: Generic RNDIS Mode on TX endpoint 10 |
| 19-18 | TX10_MODE | R/W | 0h | 00: Transparent Mode on TX endpoint 9<br>01: RNDIS Mode on TX endpoint 9<br>10: CDC Mode on TX endpoint 9<br>11: Generic RNDIS Mode on TX endpoint 9 |
| 17-16 | TX9_MODE | R/W | 0h | 00: Transparent Mode on TX endpoint 8<br>01: RNDIS Mode on TX endpoint 8<br>10: CDC Mode on TX endpoint 8<br>11: Generic RNDIS Mode on TX endpoint 8 |
| 15-14 | TX8_MODE | R/W | 0h | 00: Transparent Mode on TX endpoint 7<br>01: RNDIS Mode on TX endpoint 7<br>10: CDC Mode on TX endpoint 7<br>11: Generic RNDIS Mode on TX endpoint 7 |
| 13-12 | TX7_MODE | R/W | 0h | 00: Transparent Mode on TX endpoint 6<br>01: RNDIS Mode on TX endpoint 6<br>10: CDC Mode on TX endpoint 6<br>11: Generic RNDIS Mode on TX endpoint 6 |

**Table 16-85. USB0TXMODE Register Field Descriptions (continued)**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 11-10 | TX6_MODE | R/W | 0h | 00: Transparent Mode on TX endpoint 5<br>01: RNDIS Mode on TX endpoint 5<br>10: CDC Mode on TX endpoint 5<br>11: Generic RNDIS Mode on TX endpoint 5 |
| 9-8 | TX5_MODE | R/W | 0h | 00: Transparent Mode on TX endpoint 4<br>01: RNDIS Mode on TX endpoint 4<br>10: CDC Mode on TX endpoint 4<br>11: Generic RNDIS Mode on TX endpoint 4 |
| 7-6 | TX4_MODE | R/W | 0h | 00: Transparent Mode on TX endpoint 3<br>01: RNDIS Mode on TX endpoint 3<br>10: CDC Mode on TX endpoint 3<br>11: Generic RNDIS Mode on TX endpoint 3 |
| 5-4 | TX3_MODE | R/W | 0h | 00: Transparent Mode on TX endpoint 2<br>01: RNDIS Mode on TX endpoint 2<br>10: CDC Mode on TX endpoint 2<br>11: Generic RNDIS Mode on TX endpoint 2 |
| 3-2 | TX2_MODE | R/W | 0h | 00: Transparent Mode on TX endpoint 1<br>01: RNDIS Mode on TX endpoint 1<br>10: CDC Mode on TX endpoint 1<br>11: Generic RNDIS Mode on TX endpoint 1 |
| 1-0 | TX1_MODE | R/W | 0h | 00: Transparent Mode on TX endpoint 0<br>01: RNDIS Mode on TX endpoint 0<br>10: CDC Mode on TX endpoint 0<br>11: Generic RNDIS Mode on TX endpoint 0 USB0 Tx Mode Registers |

### 16.4.2.14 USB0RXMODE Register (offset = 1074h) [reset = 0h]

USB0RXMODE is shown in Figure 16-77 and described in Table 16-86.

**Figure 16-77. USB0RXMODE Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| RESERVED | | RX15_MODE | | RX14_MODE | | RX13_MODE | |
| R/W-0h | | R/W-0h | | R/W-0h | | R/W-0h | |
| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
| RX12_MODE | | RX11_MODE | | RX10_MODE | | RX9_MODE | |
| R/W-0h | | R/W-0h | | R/W-0h | | R/W-0h | |
| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
| RX8_MODE | | RX7_MODE | | RX6_MODE | | RX5_MODE | |
| R/W-0h | | R/W-0h | | R/W-0h | | R/W-0h | |
| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
| RX4_MODE | | RX3_MODE | | RX2_MODE | | RX1_MODE | |
| R/W-0h | | R/W-0h | | R/W-0h | | R/W-0h | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-86. USB0RXMODE Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-30 | RESERVED | R/W | 0h | |
| 29-28 | RX15_MODE | R/W | 0h | 00: Transparent Mode on RX endpoint 14<br>01: RNDIS Mode on RX endpoint 14<br>10: CDC Mode on RX endpoint 14<br>11: Generic RNDIS or Infinite Mode on RX endpoint 14 |
| 27-26 | RX14_MODE | R/W | 0h | 00: Transparent Mode on RX endpoint 13<br>01: RNDIS Mode on RX endpoint 13<br>10: CDC Mode on RX endpoint 13<br>11: Generic RNDIS or Infinite Mode on RX endpoint 13 |
| 25-24 | RX13_MODE | R/W | 0h | 00: Transparent Mode on RX endpoint 12<br>01: RNDIS Mode on RX endpoint 12<br>10: CDC Mode on RX endpoint 12<br>11: Generic RNDIS or Infinite Mode on RX endpoint 12 |
| 23-22 | RX12_MODE | R/W | 0h | 00: Transparent Mode on RX endpoint 11<br>01: RNDIS Mode on RX endpoint 11<br>10: CDC Mode on RX endpoint 11<br>11: Generic RNDIS or Infinite Mode on RX endpoint 11 |
| 21-20 | RX11_MODE | R/W | 0h | 00: Transparent Mode on RX endpoint 10<br>01: RNDIS Mode on RX endpoint 10<br>10: CDC Mode on RX endpoint 10<br>11: Generic RNDIS or Infinite Mode on RX endpoint 10 |
| 19-18 | RX10_MODE | R/W | 0h | 00: Transparent Mode on RX endpoint 9<br>01: RNDIS Mode on RX endpoint 9<br>10: CDC Mode on RX endpoint 9<br>11: Generic RNDIS or Infinite Mode on RX endpoint 9 |
| 17-16 | RX9_MODE | R/W | 0h | 00: Transparent Mode on RX endpoint 8<br>01: RNDIS Mode on RX endpoint 8<br>10: CDC Mode on RX endpoint 8<br>11: Generic RNDIS Mode on RX endpoint 8 |
| 15-14 | RX8_MODE | R/W | 0h | 00: Transparent Mode on RX endpoint 7<br>01: RNDIS Mode on RX endpoint 7<br>10: CDC Mode on RX endpoint 7<br>11: Generic RNDIS or Infinite Mode on RX endpoint 7 |
| 13-12 | RX7_MODE | R/W | 0h | 00: Transparent Mode on RX endpoint 6<br>01: RNDIS Mode on RX endpoint 6<br>10: CDC Mode on RX endpoint 6<br>11: Generic RNDIS or Infinite Mode on RX endpoint 6 |

**Table 16-86. USB0RXMODE Register Field Descriptions (continued)**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 11-10 | RX6_MODE | R/W | 0h | 00: Transparent Mode on RX endpoint 5<br>01: RNDIS Mode on RX endpoint 5<br>10: CDC Mode on RX endpoint 5<br>11: Generic RNDIS or Infinite Mode on RX endpoint 5 |
| 9-8 | RX5_MODE | R/W | 0h | 00: Transparent Mode on RX endpoint 4<br>01: RNDIS Mode on RX endpoint 4<br>10: CDC Mode on RX endpoint 4<br>11: Generic RNDIS or Infinite Mode on RX endpoint 4 |
| 7-6 | RX4_MODE | R/W | 0h | 00: Transparent Mode on RX endpoint 3<br>01: RNDIS Mode on RX endpoint 3<br>10: CDC Mode on RX endpoint 3<br>11: Generic RNDIS or Infinite Mode on RX endpoint 3 |
| 5-4 | RX3_MODE | R/W | 0h | 00: Transparent Mode on RX endpoint 2<br>01: RNDIS Mode on RX endpoint 2<br>10: CDC Mode on RX endpoint 2<br>11: Generic RNDIS or Infinite Mode on RX endpoint 2 |
| 3-2 | RX2_MODE | R/W | 0h | 00: Transparent Mode on RX endpoint 1<br>01: RNDIS Mode on RX endpoint 1<br>10: CDC Mode on RX endpoint 1<br>11: Generic RNDIS or Infinite Mode on RX endpoint 1 |

**Table 16-86. USB0RXMODE Register Field Descriptions (continued)**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 1-0 | RX1_MODE | R/W | 0h | 00: Transparent Mode on RX endpoint 0<br>01: RNDIS Mode on RX endpoint 0<br>10: CDC Mode on RX endpoint 0<br>11: Generic RNDIS or Infinite Mode on RX endpoint 0 Each RX endpoint can be configured into 1 of 5 packet termination modes. Transparent mode (b00) * Supports USB endpoint sizes of o FS : 8, 16, 32, 64, and 1023 o HS : 64, 128, 512, and 1024 * Mentor Controller's RXMAXP/TXMAXP must be a valid USB endpoint size.<br>* With autoreq=3 XDMA will always generate a ReqPkt's at the end of the packet.<br>* With autoreq=1 XDMA will never generatate a ReqPkt at the end of the packet.<br>* Each USB packet is equivalent to a single CPPI DMA packet.<br>* Each received CPPI packet will be no larger than the USB max packet size (1023 bytes for FS, 1024 bytes for HS) and will be both a SOP and EOP.<br>* Transmitted CPPI packets can be no larger than the USB max packet size.<br>* Primarily used for interrupt or isochronous endpoints, as CPU will receive an interrupt for every USB packet (if enabled).<br>RNDIS mode (b01) * Supports USB endpoint sizes of o FS : 64 o HS : 64, 128, 512, and 1024 * Mentor controller's RXMAXP/TXMAXP must be an integer multiple of the USB endpoint size.<br>* With autoreq=3 XDMA will always generate a ReqPkt's at the end of the packet.<br>* With autoreq=1 XDMA will continue the generation of ReqPkt's to the Mentor controller until a short packet is received.<br>* Supports transmission of CPPI packets larger than the USB max packet size.<br>* The end of a CPPI packet is defined by a USB short packet (a USB packet less than the USB max packet size).<br>* In the case where the CPPI packet is an exact multiple of the USB max packet size, a zero length terminating packet is used (XDMA recognizes this terminating packet on reception, and generates it on transmission).<br>* Designed for use in an RNDIS compliant networking type of USB device and bulk endpoints Linux CDC mode (b10) * Same as RNDIS mode, except terminating packet has<br>1-byte of 0 data.<br>* Designed for use with a Linux OS and USB driver stack and bulk endpoints.<br>Generic RNDIS mode (b11 and Generic RNDIS EP N Size register is greater than 0) * Same as RNDIS mode, except the end of a CPPI packet is determined by a USB short packet or when the value in the GENERIC RNDIS EPn SIZE Register is reached (Rx mode only)(i.e. Tx mode does not use this register).<br>The CPU configures this register with a value that is a multiple of the corresponding endpoint size.<br>* No terminating packet is used.<br>* Designed for general bulk endpoint use where specific packet terminations are not required.<br>* Transmit mode transfers only support FS or HS EP sizes through the XDMA and Mentor controller.<br>The CPPI DMA size can be of any legal size as defined by the CPPI 4.1 specification.<br>Infinite mode (b11 and Generic RNDIS EP N Size register equal to 0) * Same as RNDIS mode, except the end of a CPPI packet is determined by a USB short packet or when the CPPI DMA closes up the packet after a defined number of buffers have been filled.<br>This is defined in Rx Channel N Global Configuration Register bits 28:26 rx_max_buffer_cnt.<br>* The CPPI DMA will ignore SOPs generated by the XDMA.<br>* The XDMA packet is assumed to be infinite.<br>* The XDMA starts the packet with a SOP (ignored by the CPPI DMA).<br>* The XDMA ends the packet when a short packet occurs.<br>* Short packets are defined when RXMAXP (0x14) is not equal to RXCOUNT (0x18). |

**Table 16-86. USB0RXMODE Register Field Descriptions (continued)**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| | | | | RXMAXP and RXCOUNT are both registers within the Mentor controller.<br>* The XDMA will continue to request RX packets from the Mentor controller until a short packet is received.<br>* When the CPPI is configured for chaining mode, the Buffer Descriptor sizes must be integer multiples of the RXMAXP size.<br>* Transmit mode is not supported.<br>* USB0 or USB1 Auto Req register should be set to AUTO REQ ALWAYS. |

### 16.4.2.15 USB0GENRNDISEP1 Register (offset = 1080h) [reset = 0h]

USB0GENRNDISEP1 is shown in Figure 16-78 and described in Table 16-87.

**Figure 16-78. USB0GENRNDISEP1 Register**

| 31–17 | 16–0 |
|---|---|
| RESERVED | EP1_SIZE |
| R/W-0h | R/W-0h |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-87. USB0GENRNDISEP1 Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-17 | RESERVED | R/W | 0h | |
| 16-0 | EP1_SIZE | R/W | 0h | Generic RNDIS packet size.<br>USB0 Generic RNDIS EP N Size Register |

### 16.4.2.16 USB0GENRNDISEP2 Register (offset = 1084h) [reset = 0h]

USB0GENRNDISEP2 is shown in Figure 16-79 and described in Table 16-88.

**Figure 16-79. USB0GENRNDISEP2 Register**

| 31-17 | 16-0 |
|---|---|
| RESERVED | EP2_SIZE |
| R/W-0h | R/W-0h |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-88. USB0GENRNDISEP2 Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-17 | RESERVED | R/W | 0h | |
| 16-0 | EP2_SIZE | R/W | 0h | Generic RNDIS packet size.<br>USB0 Generic RNDIS EP N Size Register |

### 16.4.2.17 USB0GENRNDISEP3 Register (offset = 1088h) [reset = 0h]

USB0GENRNDISEP3 is shown in Figure 16-80 and described in Table 16-89.

**Figure 16-80. USB0GENRNDISEP3 Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 | 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 | 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 | 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| RESERVED | | | | | | | | | | | | | | | EP3_SIZE | | | | | | | | | | | | | | | | |
| R/W-0h | | | | | | | | | | | | | | | R/W-0h | | | | | | | | | | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-89. USB0GENRNDISEP3 Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-17 | RESERVED | R/W | 0h | |
| 16-0 | EP3_SIZE | R/W | 0h | Generic RNDIS packet size.<br>USB0 Generic RNDIS EP N Size Register |

### 16.4.2.18 USB0GENRNDISEP4 Register (offset = 108Ch) [reset = 0h]

USB0GENRNDISEP4 is shown in Figure 16-81 and described in Table 16-90.

**Figure 16-81. USB0GENRNDISEP4 Register**

| 31–17 | 16–0 |
|---|---|
| RESERVED | EP4_SIZE |
| R/W-0h | R/W-0h |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-90. USB0GENRNDISEP4 Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-17 | RESERVED | R/W | 0h | |
| 16-0 | EP4_SIZE | R/W | 0h | Generic RNDIS packet size.<br>USB0 Generic RNDIS EP N Size Register |

### 16.4.2.19 USB0GENRNDISEP5 Register (offset = 1090h) [reset = 0h]

USB0GENRNDISEP5 is shown in Figure 16-82 and described in Table 16-91.

**Figure 16-82. USB0GENRNDISEP5 Register**

| 31-17 | 16-0 |
|---|---|
| RESERVED | EP5_SIZE |
| R/W-0h | R/W-0h |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-91. USB0GENRNDISEP5 Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-17 | RESERVED | R/W | 0h | |
| 16-0 | EP5_SIZE | R/W | 0h | Generic RNDIS packet size.<br>USB0 Generic RNDIS EP N Size Register |

### 16.4.2.20 USB0GENRNDISEP6 Register (offset = 1094h) [reset = 0h]

USB0GENRNDISEP6 is shown in Figure 16-83 and described in Table 16-92.

**Figure 16-83. USB0GENRNDISEP6 Register**

| 31–17 | 16–0 |
|---|---|
| RESERVED | EP6_SIZE |
| R/W-0h | R/W-0h |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-92. USB0GENRNDISEP6 Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-17 | RESERVED | R/W | 0h | |
| 16-0 | EP6_SIZE | R/W | 0h | Generic RNDIS packet size.<br>USB0 Generic RNDIS EP N Size Register |

### 16.4.2.21 USB0GENRNDISEP7 Register (offset = 1098h) [reset = 0h]

USB0GENRNDISEP7 is shown in Figure 16-84 and described in Table 16-93.

**Figure 16-84. USB0GENRNDISEP7 Register**

| 31–17 | 16–0 |
|---|---|
| RESERVED | EP7_SIZE |
| R/W-0h | R/W-0h |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-93. USB0GENRNDISEP7 Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-17 | RESERVED | R/W | 0h | |
| 16-0 | EP7_SIZE | R/W | 0h | Generic RNDIS packet size.<br>USB0 Generic RNDIS EP N Size Register |

### 16.4.2.22 USB0GENRNDISEP8 Register (offset = 109Ch) [reset = 0h]

USB0GENRNDISEP8 is shown in Figure 16-85 and described in Table 16-94.

**Figure 16-85. USB0GENRNDISEP8 Register**

| 31–17 | 16–0 |
|---|---|
| RESERVED | EP8_SIZE |
| R/W-0h | R/W-0h |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-94. USB0GENRNDISEP8 Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-17 | RESERVED | R/W | 0h | |
| 16-0 | EP8_SIZE | R/W | 0h | Generic RNDIS packet size.<br>USB0 Generic RNDIS EP N Size Register |

### 16.4.2.23 USB0GENRNDISEP9 Register (offset = 10A0h) [reset = 0h]

USB0GENRNDISEP9 is shown in Figure 16-86 and described in Table 16-95.

**Figure 16-86. USB0GENRNDISEP9 Register**

| 31–17 | 16–0 |
|---|---|
| RESERVED | EP9_SIZE |
| R/W-0h | R/W-0h |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-95. USB0GENRNDISEP9 Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-17 | RESERVED | R/W | 0h | |
| 16-0 | EP9_SIZE | R/W | 0h | Generic RNDIS packet size.<br>USB0 Generic RNDIS EP N Size Register |

### 16.4.2.24 USB0GENRNDISEP10 Register (offset = 10A4h) [reset = 0h]

USB0GENRNDISEP10 is shown in Figure 16-87 and described in Table 16-96.

**Figure 16-87. USB0GENRNDISEP10 Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 | 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 | 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 | 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| RESERVED | | | | | | | | | | | | | | | EP10_SIZE | | | | | | | | | | | | | | | | |
| R/W-0h | | | | | | | | | | | | | | | R/W-0h | | | | | | | | | | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-96. USB0GENRNDISEP10 Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-17 | RESERVED | R/W | 0h | |
| 16-0 | EP10_SIZE | R/W | 0h | Generic RNDIS packet size.<br>USB0 Generic RNDIS EP N Size Register |

### 16.4.2.25 USB0GENRNDISEP11 Register (offset = 10A8h) [reset = 0h]

USB0GENRNDISEP11 is shown in Figure 16-88 and described in Table 16-97.

**Figure 16-88. USB0GENRNDISEP11 Register**

| 31–17 | 16–0 |
|---|---|
| RESERVED | EP11_SIZE |
| R/W-0h | R/W-0h |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-97. USB0GENRNDISEP11 Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-17 | RESERVED | R/W | 0h | |
| 16-0 | EP11_SIZE | R/W | 0h | Generic RNDIS packet size.<br>USB0 Generic RNDIS EP N Size Register |

### 16.4.2.26 USB0GENRNDISEP12 Register (offset = 10ACh) [reset = 0h]

USB0GENRNDISEP12 is shown in Figure 16-89 and described in Table 16-98.

**Figure 16-89. USB0GENRNDISEP12 Register**

| 31–17 | 16–0 |
|---|---|
| RESERVED | EP12_SIZE |
| R/W-0h | R/W-0h |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-98. USB0GENRNDISEP12 Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-17 | RESERVED | R/W | 0h | |
| 16-0 | EP12_SIZE | R/W | 0h | Generic RNDIS packet size.<br>USB0 Generic RNDIS EP N Size Register |

### 16.4.2.27 USB0GENRNDISEP13 Register (offset = 10B0h) [reset = 0h]

USB0GENRNDISEP13 is shown in Figure 16-90 and described in Table 16-99.

**Figure 16-90. USB0GENRNDISEP13 Register**

| 31-17 | 16-0 |
|---|---|
| RESERVED | EP13_SIZE |
| R/W-0h | R/W-0h |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-99. USB0GENRNDISEP13 Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-17 | RESERVED | R/W | 0h | |
| 16-0 | EP13_SIZE | R/W | 0h | Generic RNDIS packet size.<br>USB0 Generic RNDIS EP N Size Register |

### 16.4.2.28 USB0GENRNDISEP14 Register (offset = 10B4h) [reset = 0h]

USB0GENRNDISEP14 is shown in Figure 16-91 and described in Table 16-100.

**Figure 16-91. USB0GENRNDISEP14 Register**

| 31-17 | 16-0 |
|---|---|
| RESERVED | EP14_SIZE |
| R/W-0h | R/W-0h |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-100. USB0GENRNDISEP14 Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-17 | RESERVED | R/W | 0h | |
| 16-0 | EP14_SIZE | R/W | 0h | Generic RNDIS packet size.<br>USB0 Generic RNDIS EP N Size Register |

### 16.4.2.29 USB0GENRNDISEP15 Register (offset = 10B8h) [reset = 0h]

USB0GENRNDISEP15 is shown in Figure 16-92 and described in Table 16-101.

**Figure 16-92. USB0GENRNDISEP15 Register**

| 31-17 | 16-0 |
|---|---|
| RESERVED | EP15_SIZE |
| R/W-0h | R/W-0h |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-101. USB0GENRNDISEP15 Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-17 | RESERVED | R/W | 0h | |
| 16-0 | EP15_SIZE | R/W | 0h | Generic RNDIS packet size.<br>USB0 Generic RNDIS EP N Size Register |

### 16.4.2.30 USB0AUTOREQ Register (offset = 10D0h) [reset = 0h]

USB0AUTOREQ is shown in Figure 16-93 and described in Table 16-102.

**Figure 16-93. USB0AUTOREQ Register**

| 31-30 | 29-28 | 27-26 | 25-24 |
|---|---|---|---|
| RESERVED | RX15_AUTOREQ | RX14_AUTOREQ | RX13_AUTOREQ |
| R/W-0h | R/W-0h | R/W-0h | R/W-0h |
| **23-22** | **21-20** | **19-18** | **17-16** |
| RX12_AUTOREQ | RX11_AUTOREQ | RX10_AUTOREQ | RX9_AUTOREQ |
| R/W-0h | R/W-0h | R/W-0h | R/W-0h |
| **15-14** | **13-12** | **11-10** | **9-8** |
| RX8_AUTOREQ | RX7_AUTOREQ | RX6_AUTOREQ | RX5_AUTOREQ |
| R/W-0h | R/W-0h | R/W-0h | R/W-0h |
| **7-6** | **5-4** | **3-2** | **1-0** |
| RX4_AUTOREQ | RX3_AUTOREQ | RX2_AUTOREQ | RX1_AUTOREQ |
| R/W-0h | R/W-0h | R/W-0h | R/W-0h |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-102. USB0AUTOREQ Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-30 | RESERVED | R/W | 0h | |
| 29-28 | RX15_AUTOREQ | R/W | 0h | RX endpoint 14 Auto Req enable<br>00 = no auto req<br>01 = auto req on all but EOP<br>10 = reserved<br>11 = auto req always |
| 27-26 | RX14_AUTOREQ | R/W | 0h | RX endpoint 13 Auto Req enable<br>00 = no auto req<br>01 = auto req on all but EOP<br>10 = reserved<br>11 = auto req always |
| 25-24 | RX13_AUTOREQ | R/W | 0h | RX endpoint 12 Auto Req enable<br>00 = no auto req<br>01 = auto req on all but EOP<br>10 = reserved<br>11 = auto req always |
| 23-22 | RX12_AUTOREQ | R/W | 0h | RX endpoint 11 Auto Req enable<br>00 = no auto req<br>01 = auto req on all but EOP<br>10 = reserved<br>11 = auto req always |
| 21-20 | RX11_AUTOREQ | R/W | 0h | RX endpoint 10 Auto Req enable<br>00 = no auto req<br>01 = auto req on all but EOP<br>10 = reserved<br>11 = auto req always |
| 19-18 | RX10_AUTOREQ | R/W | 0h | RX endpoint 9 Auto Req enable<br>00 = no auto req<br>01 = auto req on all but EOP<br>10 = reserved<br>11 = auto req always |
| 17-16 | RX9_AUTOREQ | R/W | 0h | RX endpoint 8 Auto Req enable<br>00 = no auto req<br>01 = auto req on all but EOP<br>10 = reserved<br>11 = auto req always |

**Table 16-102. USB0AUTOREQ Register Field Descriptions (continued)**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 15-14 | RX8_AUTOREQ | R/W | 0h | RX endpoint 7 Auto Req enable<br>00 = no auto req<br>01 = auto req on all but EOP<br>10 = reserved<br>11 = auto req always |
| 13-12 | RX7_AUTOREQ | R/W | 0h | RX endpoint 6 Auto Req enable<br>00 = no auto req<br>01 = auto req on all but EOP<br>10 = reserved<br>11 = auto req always |
| 11-10 | RX6_AUTOREQ | R/W | 0h | RX endpoint 5 Auto Req enable<br>00 = no auto req<br>01 = auto req on all but EOP<br>10 = reserved<br>11 = auto req always |
| 9-8 | RX5_AUTOREQ | R/W | 0h | RX endpoint 4 Auto Req enable<br>00 = no auto req<br>01 = auto req on all but EOP<br>10 = reserved<br>11 = auto req always |
| 7-6 | RX4_AUTOREQ | R/W | 0h | RX endpoint 3 Auto Req enable<br>00 = no auto req<br>01 = auto req on all but EOP<br>10 = reserved<br>11 = auto req always |
| 5-4 | RX3_AUTOREQ | R/W | 0h | RX endpoint 2 Auto Req enable<br>00 = no auto req<br>01 = auto req on all but EOP<br>10 = reserved<br>11 = auto req always |
| 3-2 | RX2_AUTOREQ | R/W | 0h | RX endpoint 1 Auto Req enable<br>00 = no auto req<br>01 = auto req on all but EOP<br>10 = reserved<br>11 = auto req always |
| 1-0 | RX1_AUTOREQ | R/W | 0h | RX endpoint 0 Auto Req enable<br>00 = no auto req<br>01 = auto req on all but EOP<br>10 = reserved<br>11 = auto req always USB0 Auto Req Registers |

#### 16.4.2.31 USB0SRPFIXTIME Register (offset = 10D4h) [reset = 280DE80h]

USB0SRPFIXTIME is shown in Figure 16-94 and described in Table 16-103.

**Figure 16-94. USB0SRPFIXTIME Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 | 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 | 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 | 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| SRPFIXTIME | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | |
| R/W-280DE80h | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-103. USB0SRPFIXTIME Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-0 | SRPFIXTIME | R/W | 280DE80h | SRP Fix maximum time in 60 MHz cycles.<br>Default is 700 ms.<br>USB0 SRP Fix Time Register |

### 16.4.2.32 USB0_TDOWN Register (offset = 10D8h) [reset = 0h]

USB0_TDOWN is shown in Figure 16-95 and described in Table 16-104.

**Figure 16-95. USB0_TDOWN Register**

| 31–17 | 16 | 15–1 | 0 |
|---|---|---|---|
| TX_TDOWN | RESERVED | RX_TDOWN | RESERVED |
| R/W-0h | R/W-0h | R/W-0h | R/W-0h |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-104. USB0_TDOWN Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-17 | TX_TDOWN | R/W | 0h | Tx Endpoint Teardown.<br>Write '1' to corresponding bit to set.<br>Read as '0'.<br>Bit<br>31 = Endpoint 15 ...<br>Bit<br>17 = Endpoint 1 |
| 16 | RESERVED | R/W | 0h | |
| 15-1 | RX_TDOWN | R/W | 0h | RX Endpoint Teardown.<br>Write '1' to corresponding bit to set.<br>Read as '0'.<br>Bit<br>15 = Endpoint 15 ...<br>Bit<br>1 = Endpoint 1 |
| 0 | RESERVED | R/W | 0h | |

### 16.4.2.33 USB0UTMI Register (offset = 10E0h) [reset = 200002h]

USB0UTMI is shown in Figure 16-96 and described in Table 16-105.

**Figure 16-96. USB0UTMI Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| RESERVED | | | | | | | |
| R/W-0h | | | | | | | |
| **23** | **22** | **21** | **20** | **19** | **18** | **17** | **16** |
| TXBITSTUFFEN | TXBITSTUFFENH | OTGDISABLE | VBUSVLDEXTSEL | VBUSVLDEXT | TXENABLEN | FSXCVROWNER | TXVALIDH |
| R/W-0h | R/W-0h | R/W-1h | R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h |
| **15** | **14** | **13** | **12** | **11** | **10** | **9** | **8** |
| DATAINH | | | | | | | |
| R/W-0h | | | | | | | |
| **7** | **6** | **5** | **4** | **3** | **2** | **1** | **0** |
| RESERVED | | | | | WORDINTERFACE | FSDATAEXT | FSSE0EXT |
| R/W-0h | | | | | R/W-0h | R/W-1h | R/W-0h |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-105. USB0UTMI Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-24 | RESERVED | R/W | 0h | |
| 23 | TXBITSTUFFEN | R/W | 0h | PHY UTMI input for signal txbitstuffen |
| 22 | TXBITSTUFFENH | R/W | 0h | PHY UTMI input for signal txbitstuffenh |
| 21 | OTGDISABLE | R/W | 1h | PHY UTMI input for signal otgdisable |
| 20 | VBUSVLDEXTSEL | R/W | 0h | PHY UTMI input for signal vbusvldextsel |
| 19 | VBUSVLDEXT | R/W | 0h | PHY UTMI input for signal vbusvldext |
| 18 | TXENABLEN | R/W | 0h | PHY UTMI input for signal txenablen |
| 17 | FSXCVROWNER | R/W | 0h | PHY UTMI input for signal fsxcvrowner |
| 16 | TXVALIDH | R/W | 0h | PHY UTMI input for signal txvalidh |
| 15-8 | DATAINH | R/W | 0h | PHY UTMI input for signal datainh |
| 7-3 | RESERVED | R/W | 0h | |
| 2 | WORDINTERFACE | R/W | 0h | PHY UTMI input for signal wordinterface |
| 1 | FSDATAEXT | R/W | 1h | PHY UTMI input for signal fsdataext |
| 0 | FSSE0EXT | R/W | 0h | PHY UTMI input for signal fsse0ext USB0 PHY UTMI Register |

### 16.4.2.34 USB0MGCUTMILB Register (offset = 10E4h) [reset = 82h]

USB0MGCUTMILB is shown in Figure 16-97 and described in Table 16-106.

**Figure 16-97. USB0MGCUTMILB Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| RESERVED | | | SUSPENDM | OPMODE | | TXVALID | XCVRSEL |
| R/W-0h | | | R-0h | R-0h | | R-0h | R-0h |
| **23** | **22** | **21** | **20** | **19** | **18** | **17** | **16** |
| XCVRSEL | TERMSEL | DRVVBUS | CHRGVBUS | DISCHRGVBUS | DPPULLDOWN | DMPULLDOWN | IDPULLUP |
| R-0h | R-0h | R-0h | R-0h | R-0h | R-0h | R-0h | R-0h |
| **15** | **14** | **13** | **12** | **11** | **10** | **9** | **8** |
| RESERVED | | | | IDDIG | HOSTDISCON | SESSEND | AVALID |
| R/W-0h | | | | R/W-0h | R/W-0h | R/W-0h | R/W-0h |
| **7** | **6** | **5** | **4** | **3** | **2** | **1** | **0** |
| VBUSVALID | RXERROR | RESERVED | | LINESTATE | | RESERVED | |
| R/W-1h | R/W-0h | R/W-0h | | R/W-0h | | R/W-2h | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-106. USB0MGCUTMILB Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-29 | RESERVED | R/W | 0h | |
| 28 | SUSPENDM | R | 0h | loopback test observed value for suspendm |
| 27-26 | OPMODE | R | 0h | loopback test observed value for opmode |
| 25 | TXVALID | R | 0h | loopback test observed value for txvalid |
| 24-23 | XCVRSEL | R | 0h | loopback test observed value for xcvrsel |
| 22 | TERMSEL | R | 0h | loopback test observed value for termsel |
| 21 | DRVVBUS | R | 0h | loopback test observed value for drvvbus |
| 20 | CHRGVBUS | R | 0h | loopback test observed value for chrgvbus |
| 19 | DISCHRGVBUS | R | 0h | loopback test observed value for dischrgvbus |
| 18 | DPPULLDOWN | R | 0h | loopback test observed value for dppulldown |
| 17 | DMPULLDOWN | R | 0h | loopback test observed value for dmpulldown |
| 16 | IDPULLUP | R | 0h | loopback test observed value for idpullup |
| 15-12 | RESERVED | R/W | 0h | |
| 11 | IDDIG | R/W | 0h | loopback test value for iddig |
| 10 | HOSTDISCON | R/W | 0h | loopback test value for hostdiscon |
| 9 | SESSEND | R/W | 0h | loopback test value for sessend |
| 8 | AVALID | R/W | 0h | loopback test value for avalid |
| 7 | VBUSVALID | R/W | 1h | loopback test value for vbusvalid |
| 6 | RXERROR | R/W | 0h | loopback test value for rxerror |
| 5-4 | RESERVED | R/W | 0h | |
| 3-2 | LINESTATE | R/W | 0h | loopback test value for linestate |
| 1-0 | RESERVED | R/W | 2h | |

### 16.4.2.35 USB0MODE Register (offset = 10E8h) [reset = 100h]

USB0MODE is shown in Figure 16-98 and described in Table 16-107.

**Figure 16-98. USB0MODE Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| RESERVED | | | | | | | |
| R/W-0h | | | | | | | |
| **23** | **22** | **21** | **20** | **19** | **18** | **17** | **16** |
| RESERVED | | | | | | | |
| R/W-0h | | | | | | | |
| **15** | **14** | **13** | **12** | **11** | **10** | **9** | **8** |
| RESERVED | | | | | | | IDDIG |
| R/W-0h | | | | | | | R/W-1h |
| **7** | **6** | **5** | **4** | **3** | **2** | **1** | **0** |
| IDDIG_MUX | RESERVED | | | | | PHY_TEST | LOOPBACK |
| R/W-0h | R/W-0h | | | | | R/W-0h | R/W-0h |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-107. USB0MODE Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-9 | RESERVED | R/W | 0h | |
| 8 | IDDIG | R/W | 1h | MGC input value for iddig<br>0=A type<br>1=B type |
| 7 | IDDIG_MUX | R/W | 0h | Multiplexer control for IDDIG signal going to the controller.<br>0 = IDDIG is from PHY0.<br>1 = IDDIG is from bit 8 (IDDIG) of this USB0MODE register. |
| 6-2 | RESERVED | R/W | 0h | |
| 1 | PHY_TEST | R/W | 0h | PHY test<br>0 = Normal mode<br>1 = PHY test mode |
| 0 | LOOPBACK | R/W | 0h | Loopback test mode<br>0 = Normal mode<br>1 = Loopback test mode USB0 Mode Register |

## *16.4.3 USB1_CTRL Registers*

Table 16-108 lists the memory-mapped registers for the USB1_CTRL. All register offset addresses not listed in Table 16-108 should be considered as reserved locations and the register contents should not be modified.

**Table 16-108. USB1_CTRL Registers**

| Offset | Acronym | Register Name | Section |
|---|---|---|---|
| 1800h | USB1REV | | Section 16.4.3.1 |
| 1814h | USB1CTRL | | Section 16.4.3.2 |
| 1818h | USB1STAT | | Section 16.4.3.3 |
| 1820h | USB1IRQMSTAT | | Section 16.4.3.4 |
| 1828h | USB1IRQSTATRAW0 | | Section 16.4.3.5 |
| 182Ch | USB1IRQSTATRAW1 | | Section 16.4.3.6 |
| 1830h | USB1IRQSTAT0 | | Section 16.4.3.7 |
| 1834h | USB1IRQSTAT1 | | Section 16.4.3.8 |
| 1838h | USB1IRQENABLESET0 | | Section 16.4.3.9 |
| 183Ch | USB1IRQENABLESET1 | | Section 16.4.3.10 |

**Table 16-108. USB1_CTRL Registers (continued)**

| Offset | Acronym | Register Name | Section |
|---|---|---|---|
| 1840h | USB1IRQENABLECLR0 | | Section 16.4.3.11 |
| 1844h | USB1IRQENABLECLR1 | | Section 16.4.3.12 |
| 1870h | USB1TXMODE | | Section 16.4.3.13 |
| 1874h | USB1RXMODE | | Section 16.4.3.14 |
| 1880h | USB1GENRNDISEP1 | | Section 16.4.3.15 |
| 1884h | USB1GENRNDISEP2 | | Section 16.4.3.16 |
| 1888h | USB1GENRNDISEP3 | | Section 16.4.3.17 |
| 188Ch | USB1GENRNDISEP4 | | Section 16.4.3.18 |
| 1890h | USB1GENRNDISEP5 | | Section 16.4.3.19 |
| 1894h | USB1GENRNDISEP6 | | Section 16.4.3.20 |
| 1898h | USB1GENRNDISEP7 | | Section 16.4.3.21 |
| 189Ch | USB1GENRNDISEP8 | | Section 16.4.3.22 |
| 18A0h | USB1GENRNDISEP9 | | Section 16.4.3.23 |
| 18A4h | USB1GENRNDISEP10 | | Section 16.4.3.24 |
| 18A8h | USB1GENRNDISEP11 | | Section 16.4.3.25 |
| 18ACh | USB1GENRNDISEP12 | | Section 16.4.3.26 |
| 18B0h | USB1GENRNDISEP13 | | Section 16.4.3.27 |
| 18B4h | USB1GENRNDISEP14 | | Section 16.4.3.28 |
| 18B8h | USB1GENRNDISEP15 | | Section 16.4.3.29 |
| 18D0h | USB1AUTOREQ | | Section 16.4.3.30 |
| 18D4h | USB1SRPFIXTIME | | Section 16.4.3.31 |
| 18D8h | USB1TDOWN | | Section 16.4.3.32 |
| 18E0h | USB1UTMI | | Section 16.4.3.33 |
| 18E4h | USB1UTMILB | | Section 16.4.3.34 |
| 18E8h | USB1MODE | | Section 16.4.3.35 |

### 16.4.3.1 USB1REV Register (offset = 1800h) [reset = 4EA20800h]

USB1REV is shown in Figure 16-99 and described in Table 16-109.

**Figure 16-99. USB1REV Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 | 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| SCHEME | | RESERVED | | FUNC | | | | | | | | | | | |
| R-1h | | R-0h | | R-EA2h | | | | | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 | 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| R_RTL | | | | | X_MAJOR | | | CUSTOM | | Y_MINOR | | | | | |
| R-1h | | | | | R-0h | | | R-0h | | R-0h | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-109. USB1REV Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-30 | SCHEME | R | 1h | Used to distinguish between legacy interface scheme and current.<br>0 = Legacy<br>1 = Current |
| 29-28 | RESERVED | R | 0h | |
| 27-16 | FUNC | R | EA2h | Function indicates a software compatible module family. |
| 15-11 | R_RTL | R | 1h | RTL revision.<br>Will vary depending on release. |
| 10-8 | X_MAJOR | R | 0h | Major revision. |
| 7-6 | CUSTOM | R | 0h | Custom revision |
| 5-0 | Y_MINOR | R | 0h | Minor revision.<br>USB1 Revision Register |

### 16.4.3.2 USB1CTRL Register (offset = 1814h) [reset = 0h]

USB1CTRL is shown in Figure 16-100 and described in Table 16-110.

**Figure 16-100. USB1CTRL Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| DIS_DEB | DIS_SRP | RESERVED | | | | | |
| R/W-0h | R/W-0h | R/W-0h | | | | | |
| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
| RESERVED | | | | | | | |
| R/W-0h | | | | | | | |
| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
| RESERVED | | | | | | | |
| R/W-0h | | | | | | | |
| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
| RESERVED | | SOFT_RESET_ISOLATION | RNDIS | UINT | RESERVED | CLKFACK | SOFT_RESET |
| R/W-0h | | R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-110. USB1CTRL Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31 | DIS_DEB | R/W | 0h | Disable the VBUS debouncer circuit fix |
| 30 | DIS_SRP | R/W | 0h | Disable the OTG Session Request Protocol (SRP) AVALID circuit fix.<br>When enabled (=0) this allows additional time for the VBUS signal to be measured against the VBUS thresholds.<br>The time is specified in the USB0 SRP Fix Time Register. |
| 29-6 | RESERVED | R/W | 0h | |
| 5 | SOFT_RESET_ISOLATION | R/W | 0h | Soft reset isolation.<br>When high this bit forces all USB1 signals that connect to the USBSS to known values during a soft reset via bit 0 of this register.<br>This bit should be set high prior to setting bit 0 and cleared after bit 0 is cleared. |
| 4 | RNDIS | R/W | 0h | Global RNDIS mode enable for all endpoints. |
| 3 | UINT | R/W | 0h | USB interrupt enable<br>1 = Legacy<br>0 = Current (recommended setting)<br>If uint is set high, then the mentor controller generic interrupt for USB[9] will be generated (if enabled).<br>This requires S/W to read the mentor controller's registers to determine which interrupt from USB[0] to USB[7] that occurred.<br>If uint is set low, then the usb20otg_f module will automatically read the mentor controller's registers and set the appropriate interrupt from USB[0] to USB[7] (if enabled).<br>The generic interrupt for USB[9] will not be generated. |
| 2 | RESERVED | R/W | 0h | |
| 1 | CLKFACK | R/W | 0h | Clock stop fast ack enable. |

**Table 16-110. USB1CTRL Register Field Descriptions (continued)**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 0 | SOFT_RESET | R/W | 0h | Software reset of USB1.<br>Write 0 = no action<br>Write 1 = Initiate software reset<br>Read 0 = Reset done, no action<br>Read 1 = Reset ongoing<br>Both the soft_reset and soft_reset_isolation bits should be asserted simultaneously.<br>This will cause the following sequence of actions to occur over multiple cycles:<br>- All USB0 output signals will go to a known constant value via multiplexers.<br>This removes the possibility of timing errors due to the asynchronous resets.<br>- All USB0 registers will be reset.<br>- The USB0 resets will be de-asserted.<br>- The reset isolation multiplexer inputs will be de-selected.<br>- Both the soft_reset and soft_reset_isolation bits will be automatically cleared.<br>Setting only the soft_reset_isolation bit will cause all USB0 output signals to go to a known constant value via multiplexers.<br>This will prevent future access to USB0.<br>To clear this condition the USBSS must be reset via a hard or soft reset. |

### 16.4.3.3 USB1STAT Register (offset = 1818h) [reset = 0h]

USB1STAT is shown in Figure 16-101 and described in Table 16-111.

**Figure 16-101. USB1STAT Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| RESERVED | | | | | | | |
| R-0h | | | | | | | |
| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
| RESERVED | | | | | | | |
| R-0h | | | | | | | |
| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
| RESERVED | | | | | | | |
| R-0h | | | | | | | |
| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
| RESERVED | | | | | | | DRVVBUS |
| R-0h | | | | | | | R-0h |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-111. USB1STAT Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-1 | RESERVED | R | 0h | |
| 0 | DRVVBUS | R | 0h | Current DRVVBUS value USB1 Status Register |

#### 16.4.3.4 USB1IRQMSTAT Register (offset = 1820h) [reset = 0h]

USB1IRQMSTAT is shown in Figure 16-102 and described in Table 16-112.

**Figure 16-102. USB1IRQMSTAT Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| RESERVED | | | | | | | |
| R-0h | | | | | | | |
| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
| RESERVED | | | | | | | |
| R-0h | | | | | | | |
| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
| RESERVED | | | | | | | |
| R-0h | | | | | | | |
| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
| RESERVED | | | | | | BANK1 | BANK0 |
| R-0h | | | | | | R-0h | R-0h |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-112. USB1IRQMSTAT Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-2 | RESERVED | R | 0h | |
| 1 | BANK1 | R | 0h | 0: No events pending from IRQ_STATUS_1<br>1: At least one event is pending from IRQ_STATUS_1 |
| 0 | BANK0 | R | 0h | 0: No events pending from IRQ_STATUS_0<br>1: At least one event is pending from IRQ_STATUS_0 USB1 IRQ_MERGED_STATUS Register |

### 16.4.3.5 USB1IRQSTATRAW0 Register (offset = 1828h) [reset = 0h]

USB1IRQSTATRAW0 is shown in Figure 16-103 and described in Table 16-113.

**Figure 16-103. USB1IRQSTATRAW0 Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| RX_EP_15 | RX_EP_14 | RX_EP_13 | RX_EP_12 | RX_EP_11 | RX_EP_10 | RX_EP_9 | RX_EP_8 |
| R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h |
| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
| RX_EP_7 | RX_EP_6 | RX_EP_5 | RX_EP_4 | RX_EP_3 | RX_EP_2 | RX_EP_1 | RESERVED |
| R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h |
| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
| TX_EP_15 | TX_EP_14 | TX_EP_13 | TX_EP_12 | TX_EP_11 | TX_EP_10 | TX_EP_9 | TX_EP_8 |
| R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h |
| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
| TX_EP_7 | TX_EP_6 | TX_EP_5 | TX_EP_4 | TX_EP_3 | TX_EP_2 | TX_EP_1 | TX_EP_0 |
| R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-113. USB1IRQSTATRAW0 Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31 | RX_EP_15 | R/W | 0h | Interrupt status for RX endpoint 15 |
| 30 | RX_EP_14 | R/W | 0h | Interrupt status for RX endpoint 14 |
| 29 | RX_EP_13 | R/W | 0h | Interrupt status for RX endpoint 13 |
| 28 | RX_EP_12 | R/W | 0h | Interrupt status for RX endpoint 12 |
| 27 | RX_EP_11 | R/W | 0h | Interrupt status for RX endpoint 11 |
| 26 | RX_EP_10 | R/W | 0h | Interrupt status for RX endpoint 10 |
| 25 | RX_EP_9 | R/W | 0h | Interrupt status for RX endpoint 9 |
| 24 | RX_EP_8 | R/W | 0h | Interrupt status for RX endpoint 8 |
| 23 | RX_EP_7 | R/W | 0h | Interrupt status for RX endpoint 7 |
| 22 | RX_EP_6 | R/W | 0h | Interrupt status for RX endpoint 6 |
| 21 | RX_EP_5 | R/W | 0h | Interrupt status for RX endpoint 5 |
| 20 | RX_EP_4 | R/W | 0h | Interrupt status for RX endpoint 4 |
| 19 | RX_EP_3 | R/W | 0h | Interrupt status for RX endpoint 3 |
| 18 | RX_EP_2 | R/W | 0h | Interrupt status for RX endpoint 2 |
| 17 | RX_EP_1 | R/W | 0h | Interrupt status for RX endpoint 1 |
| 16 | RESERVED | R/W | 0h | |
| 15 | TX_EP_15 | R/W | 0h | Interrupt status for TX endpoint 15 |
| 14 | TX_EP_14 | R/W | 0h | Interrupt status for TX endpoint 14 |
| 13 | TX_EP_13 | R/W | 0h | Interrupt status for TX endpoint 13 |
| 12 | TX_EP_12 | R/W | 0h | Interrupt status for TX endpoint 12 |
| 11 | TX_EP_11 | R/W | 0h | Interrupt status for TX endpoint 11 |
| 10 | TX_EP_10 | R/W | 0h | Interrupt status for TX endpoint 10 |
| 9 | TX_EP_9 | R/W | 0h | Interrupt status for TX endpoint 9 |
| 8 | TX_EP_8 | R/W | 0h | Interrupt status for TX endpoint 8 |
| 7 | TX_EP_7 | R/W | 0h | Interrupt status for TX endpoint 7 |
| 6 | TX_EP_6 | R/W | 0h | Interrupt status for TX endpoint 6 |

**Table 16-113. USB1IRQSTATRAW0 Register Field Descriptions (continued)**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 5 | TX_EP_5 | R/W | 0h | Interrupt status for TX endpoint 5 |
| 4 | TX_EP_4 | R/W | 0h | Interrupt status for TX endpoint 4 |
| 3 | TX_EP_3 | R/W | 0h | Interrupt status for TX endpoint 3 |
| 2 | TX_EP_2 | R/W | 0h | Interrupt status for TX endpoint 2 |
| 1 | TX_EP_1 | R/W | 0h | Interrupt status for TX endpoint 1 |
| 0 | TX_EP_0 | R/W | 0h | Interrupt status for TX endpoint 0 USB1 IRQ_STATUS_RAW_0 Register |

### 16.4.3.6 USB1IRQSTATRAW1 Register (offset = 182Ch) [reset = 0h]

USB1IRQSTATRAW1 is shown in Figure 16-104 and described in Table 16-114.

**Figure 16-104. USB1IRQSTATRAW1 Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| TX_FIFO_15 | TX_FIFO_14 | TX_FIFO_13 | TX_FIFO_12 | TX_FIFO_11 | TX_FIFO_10 | TX_FIFO_9 | TX_FIFO_8 |
| R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h |
| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
| TX_FIFO_7 | TX_FIFO_6 | TX_FIFO_5 | TX_FIFO_4 | TX_FIFO_3 | TX_FIFO_2 | TX_FIFO_1 | RESERVED |
| R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h |
| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
| RESERVED | | | | | | USB_9 | USB_8 |
| R/W-0h | | | | | | R/W-0h | R/W-0h |
| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
| USB_7 | USB_6 | USB_5 | USB_4 | USB_3 | USB_2 | USB_1 | USB_0 |
| R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-114. USB1IRQSTATRAW1 Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31 | TX_FIFO_15 | R/W | 0h | Interrupt status for TX FIFO endpoint 15 |
| 30 | TX_FIFO_14 | R/W | 0h | Interrupt status for TX FIFO endpoint 14 |
| 29 | TX_FIFO_13 | R/W | 0h | Interrupt status for TX FIFO endpoint 13 |
| 28 | TX_FIFO_12 | R/W | 0h | Interrupt status for TX FIFO endpoint 12 |
| 27 | TX_FIFO_11 | R/W | 0h | Interrupt status for TX FIFO endpoint 11 |
| 26 | TX_FIFO_10 | R/W | 0h | Interrupt status for TX FIFO endpoint 10 |
| 25 | TX_FIFO_9 | R/W | 0h | Interrupt status for TX FIFO endpoint 9 |
| 24 | TX_FIFO_8 | R/W | 0h | Interrupt status for TX FIFO endpoint 8 |
| 23 | TX_FIFO_7 | R/W | 0h | Interrupt status for TX FIFO endpoint 7 |
| 22 | TX_FIFO_6 | R/W | 0h | Interrupt status for TX FIFO endpoint 6 |
| 21 | TX_FIFO_5 | R/W | 0h | Interrupt status for TX FIFO endpoint 5 |
| 20 | TX_FIFO_4 | R/W | 0h | Interrupt status for TX FIFO endpoint 4 |
| 19 | TX_FIFO_3 | R/W | 0h | Interrupt status for TX FIFO endpoint 3 |
| 18 | TX_FIFO_2 | R/W | 0h | Interrupt status for TX FIFO endpoint 2 |
| 17 | TX_FIFO_1 | R/W | 0h | Interrupt status for TX FIFO endpoint 1 |
| 16-10 | RESERVED | R/W | 0h | |
| 9 | USB_9 | R/W | 0h | Interrupt status for Mentor controller USB_INT generic interrupt |
| 8 | USB_8 | R/W | 0h | Interrupt status for DRVVBUS level change |
| 7 | USB_7 | R/W | 0h | Interrupt status for VBUS < VBUS valid threshold |
| 6 | USB_6 | R/W | 0h | Interrupt status for SRP detected |
| 5 | USB_5 | R/W | 0h | Interrupt status for device disconnected (host mode) |
| 4 | USB_4 | R/W | 0h | Interrupt status for device connected (host mode) |
| 3 | USB_3 | R/W | 0h | Interrupt status for SOF started |
| 2 | USB_2 | R/W | 0h | Interrupt status for Reset signaling detected (peripheral mode)<br>Babble detected (host mode) |
| 1 | USB_1 | R/W | 0h | Interrupt status for Resume signaling detected |

**Table 16-114. USB1IRQSTATRAW1 Register Field Descriptions (continued)**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 0 | USB_0 | R/W | 0h | Interrupt status for Suspend signaling detected USB1 IRQ_STATUS_RAW_1 Register |

#### 16.4.3.7 USB1IRQSTAT0 Register (offset = 1830h) [reset = 0h]

USB1IRQSTAT0 is shown in Figure 16-105 and described in Table 16-115.

**Figure 16-105. USB1IRQSTAT0 Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| RX_EP_15 | RX_EP_14 | RX_EP_13 | RX_EP_12 | RX_EP_11 | RX_EP_10 | RX_EP_9 | RX_EP_8 |
| R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h |
| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
| RX_EP_7 | RX_EP_6 | RX_EP_5 | RX_EP_4 | RX_EP_3 | RX_EP_2 | RX_EP_1 | RESERVED |
| R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h |
| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
| TX_EP_15 | TX_EP_14 | TX_EP_13 | TX_EP_12 | TX_EP_11 | TX_EP_10 | TX_EP_9 | TX_EP_8 |
| R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h |
| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
| TX_EP_7 | TX_EP_6 | TX_EP_5 | TX_EP_4 | TX_EP_3 | TX_EP_2 | TX_EP_1 | TX_EP_0 |
| R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-115. USB1IRQSTAT0 Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31 | RX_EP_15 | R/W | 0h | Interrupt status for RX endpoint 15 |
| 30 | RX_EP_14 | R/W | 0h | Interrupt status for RX endpoint 14 |
| 29 | RX_EP_13 | R/W | 0h | Interrupt status for RX endpoint 13 |
| 28 | RX_EP_12 | R/W | 0h | Interrupt status for RX endpoint 12 |
| 27 | RX_EP_11 | R/W | 0h | Interrupt status for RX endpoint 11 |
| 26 | RX_EP_10 | R/W | 0h | Interrupt status for RX endpoint 10 |
| 25 | RX_EP_9 | R/W | 0h | Interrupt status for RX endpoint 9 |
| 24 | RX_EP_8 | R/W | 0h | Interrupt status for RX endpoint 8 |
| 23 | RX_EP_7 | R/W | 0h | Interrupt status for RX endpoint 7 |
| 22 | RX_EP_6 | R/W | 0h | Interrupt status for RX endpoint 6 |
| 21 | RX_EP_5 | R/W | 0h | Interrupt status for RX endpoint 5 |
| 20 | RX_EP_4 | R/W | 0h | Interrupt status for RX endpoint 4 |
| 19 | RX_EP_3 | R/W | 0h | Interrupt status for RX endpoint 3 |
| 18 | RX_EP_2 | R/W | 0h | Interrupt status for RX endpoint 2 |
| 17 | RX_EP_1 | R/W | 0h | Interrupt status for RX endpoint 1 |
| 16 | RESERVED | R/W | 0h | |
| 15 | TX_EP_15 | R/W | 0h | Interrupt status for TX endpoint 15 |
| 14 | TX_EP_14 | R/W | 0h | Interrupt status for TX endpoint 14 |
| 13 | TX_EP_13 | R/W | 0h | Interrupt status for TX endpoint 13 |
| 12 | TX_EP_12 | R/W | 0h | Interrupt status for TX endpoint 12 |
| 11 | TX_EP_11 | R/W | 0h | Interrupt status for TX endpoint 11 |
| 10 | TX_EP_10 | R/W | 0h | Interrupt status for TX endpoint 10 |
| 9 | TX_EP_9 | R/W | 0h | Interrupt status for TX endpoint 9 |
| 8 | TX_EP_8 | R/W | 0h | Interrupt status for TX endpoint 8 |
| 7 | TX_EP_7 | R/W | 0h | Interrupt status for TX endpoint 7 |
| 6 | TX_EP_6 | R/W | 0h | Interrupt status for TX endpoint 6 |

**Table 16-115. USB1IRQSTAT0 Register Field Descriptions (continued)**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 5 | TX_EP_5 | R/W | 0h | Interrupt status for TX endpoint 5 |
| 4 | TX_EP_4 | R/W | 0h | Interrupt status for TX endpoint 4 |
| 3 | TX_EP_3 | R/W | 0h | Interrupt status for TX endpoint 3 |
| 2 | TX_EP_2 | R/W | 0h | Interrupt status for TX endpoint 2 |
| 1 | TX_EP_1 | R/W | 0h | Interrupt status for TX endpoint 1 |
| 0 | TX_EP_0 | R/W | 0h | Interrupt status for TX endpoint 0 USB1 IRQ_STATUS_0 Register |

### 16.4.3.8 USB1IRQSTAT1 Register (offset = 1834h) [reset = 0h]

USB1IRQSTAT1 is shown in Figure 16-106 and described in Table 16-116.

**Figure 16-106. USB1IRQSTAT1 Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| TX_FIFO_15 | TX_FIFO_14 | TX_FIFO_13 | TX_FIFO_12 | TX_FIFO_11 | TX_FIFO_10 | TX_FIFO_9 | TX_FIFO_8 |
| R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h |
| **23** | **22** | **21** | **20** | **19** | **18** | **17** | **16** |
| TX_FIFO_7 | TX_FIFO_6 | TX_FIFO_5 | TX_FIFO_4 | TX_FIFO_3 | TX_FIFO_2 | TX_FIFO_1 | RESERVED |
| R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h |
| **15** | **14** | **13** | **12** | **11** | **10** | **9** | **8** |
| RESERVED | | | | | | USB_9 | USB_8 |
| R/W-0h | | | | | | R/W-0h | R/W-0h |
| **7** | **6** | **5** | **4** | **3** | **2** | **1** | **0** |
| USB_7 | USB_6 | USB_5 | USB_4 | USB_3 | USB_2 | USB_1 | USB_0 |
| R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-116. USB1IRQSTAT1 Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31 | TX_FIFO_15 | R/W | 0h | Interrupt status for TX FIFO endpoint 15 |
| 30 | TX_FIFO_14 | R/W | 0h | Interrupt status for TX FIFO endpoint 14 |
| 29 | TX_FIFO_13 | R/W | 0h | Interrupt status for TX FIFO endpoint 13 |
| 28 | TX_FIFO_12 | R/W | 0h | Interrupt status for TX FIFO endpoint 12 |
| 27 | TX_FIFO_11 | R/W | 0h | Interrupt status for TX FIFO endpoint 11 |
| 26 | TX_FIFO_10 | R/W | 0h | Interrupt status for TX FIFO endpoint 10 |
| 25 | TX_FIFO_9 | R/W | 0h | Interrupt status for TX FIFO endpoint 9 |
| 24 | TX_FIFO_8 | R/W | 0h | Interrupt status for TX FIFO endpoint 8 |
| 23 | TX_FIFO_7 | R/W | 0h | Interrupt status for TX FIFO endpoint 7 |
| 22 | TX_FIFO_6 | R/W | 0h | Interrupt status for TX FIFO endpoint 6 |
| 21 | TX_FIFO_5 | R/W | 0h | Interrupt status for TX FIFO endpoint 5 |
| 20 | TX_FIFO_4 | R/W | 0h | Interrupt status for TX FIFO endpoint 4 |
| 19 | TX_FIFO_3 | R/W | 0h | Interrupt status for TX FIFO endpoint 3 |
| 18 | TX_FIFO_2 | R/W | 0h | Interrupt status for TX FIFO endpoint 2 |
| 17 | TX_FIFO_1 | R/W | 0h | Interrupt status for TX FIFO endpoint 1 |
| 16-10 | RESERVED | R/W | 0h | |
| 9 | USB_9 | R/W | 0h | Interrupt status for Mentor controller USB_INT generic interrupt |
| 8 | USB_8 | R/W | 0h | Interrupt status for DRVVBUS level change |
| 7 | USB_7 | R/W | 0h | Interrupt status for VBUS < VBUS valid threshold |
| 6 | USB_6 | R/W | 0h | Interrupt status for SRP detected |
| 5 | USB_5 | R/W | 0h | Interrupt status for device disconnected (host mode) |
| 4 | USB_4 | R/W | 0h | Interrupt status for device connected (host mode) |
| 3 | USB_3 | R/W | 0h | Interrupt status for SOF started |
| 2 | USB_2 | R/W | 0h | Interrupt status for Reset signaling detected (peripheral mode)<br>Babble detected (host mode) |
| 1 | USB_1 | R/W | 0h | Interrupt status for Resume signaling detected |

**Table 16-116. USB1IRQSTAT1 Register Field Descriptions (continued)**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 0 | USB_0 | R/W | 0h | Interrupt status for Suspend signaling detected USB1 IRQ_STATUS_1 Register |

### 16.4.3.9 USB1IRQENABLESET0 Register (offset = 1838h) [reset = 0h]

USB1IRQENABLESET0 is shown in Figure 16-107 and described in Table 16-117.

**Figure 16-107. USB1IRQENABLESET0 Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| RX_EP_15 | RX_EP_14 | RX_EP_13 | RX_EP_12 | RX_EP_11 | RX_EP_10 | RX_EP_9 | RX_EP_8 |
| R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h |
| **23** | **22** | **21** | **20** | **19** | **18** | **17** | **16** |
| RX_EP_7 | RX_EP_6 | RX_EP_5 | RX_EP_4 | RX_EP_3 | RX_EP_2 | RX_EP_1 | RESERVED |
| R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h |
| **15** | **14** | **13** | **12** | **11** | **10** | **9** | **8** |
| TX_EP_15 | TX_EP_14 | TX_EP_13 | TX_EP_12 | TX_EP_11 | TX_EP_10 | TX_EP_9 | TX_EP_8 |
| R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h |
| **7** | **6** | **5** | **4** | **3** | **2** | **1** | **0** |
| TX_EP_7 | TX_EP_6 | TX_EP_5 | TX_EP_4 | TX_EP_3 | TX_EP_2 | TX_EP_1 | TX_EP_0 |
| R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-117. USB1IRQENABLESET0 Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31 | RX_EP_15 | R/W | 0h | Interrupt enable for RX endpoint 15 |
| 30 | RX_EP_14 | R/W | 0h | Interrupt enable for RX endpoint 14 |
| 29 | RX_EP_13 | R/W | 0h | Interrupt enable for RX endpoint 13 |
| 28 | RX_EP_12 | R/W | 0h | Interrupt enable for RX endpoint 12 |
| 27 | RX_EP_11 | R/W | 0h | Interrupt enable for RX endpoint 11 |
| 26 | RX_EP_10 | R/W | 0h | Interrupt enable for RX endpoint 10 |
| 25 | RX_EP_9 | R/W | 0h | Interrupt enable for RX endpoint 9 |
| 24 | RX_EP_8 | R/W | 0h | Interrupt enable for RX endpoint 8 |
| 23 | RX_EP_7 | R/W | 0h | Interrupt enable for RX endpoint 7 |
| 22 | RX_EP_6 | R/W | 0h | Interrupt enable for RX endpoint 6 |
| 21 | RX_EP_5 | R/W | 0h | Interrupt enable for RX endpoint 5 |
| 20 | RX_EP_4 | R/W | 0h | Interrupt enable for RX endpoint 4 |
| 19 | RX_EP_3 | R/W | 0h | Interrupt enable for RX endpoint 3 |
| 18 | RX_EP_2 | R/W | 0h | Interrupt enable for RX endpoint 2 |
| 17 | RX_EP_1 | R/W | 0h | Interrupt enable for RX endpoint 1 |
| 16 | RESERVED | R/W | 0h | |
| 15 | TX_EP_15 | R/W | 0h | Interrupt enable for TX endpoint 15 |
| 14 | TX_EP_14 | R/W | 0h | Interrupt enable for TX endpoint 14 |
| 13 | TX_EP_13 | R/W | 0h | Interrupt enable for TX endpoint 13 |
| 12 | TX_EP_12 | R/W | 0h | Interrupt enable for TX endpoint 12 |
| 11 | TX_EP_11 | R/W | 0h | Interrupt enable for TX endpoint 11 |
| 10 | TX_EP_10 | R/W | 0h | Interrupt enable for TX endpoint 10 |
| 9 | TX_EP_9 | R/W | 0h | Interrupt enable for TX endpoint 9 |
| 8 | TX_EP_8 | R/W | 0h | Interrupt enable for TX endpoint 8 |
| 7 | TX_EP_7 | R/W | 0h | Interrupt enable for TX endpoint 7 |
| 6 | TX_EP_6 | R/W | 0h | Interrupt enable for TX endpoint 6 |

**Table 16-117. USB1IRQENABLESET0 Register Field Descriptions (continued)**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 5 | TX_EP_5 | R/W | 0h | Interrupt enable for TX endpoint 5 |
| 4 | TX_EP_4 | R/W | 0h | Interrupt enable for TX endpoint 4 |
| 3 | TX_EP_3 | R/W | 0h | Interrupt enable for TX endpoint 3 |
| 2 | TX_EP_2 | R/W | 0h | Interrupt enable for TX endpoint 2 |
| 1 | TX_EP_1 | R/W | 0h | Interrupt enable for TX endpoint 1 |
| 0 | TX_EP_0 | R/W | 0h | Interrupt enable for TX endpoint 0 USB1 IRQ_ENABLE_SET_0 Register |

### 16.4.3.10 USB1IRQENABLESET1 Register (offset = 183Ch) [reset = 0h]

USB1IRQENABLESET1 is shown in Figure 16-108 and described in Table 16-118.

**Figure 16-108. USB1IRQENABLESET1 Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| TX_FIFO_15 | TX_FIFO_14 | TX_FIFO_13 | TX_FIFO_12 | TX_FIFO_11 | TX_FIFO_10 | TX_FIFO_9 | TX_FIFO_8 |
| R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h |
| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
| TX_FIFO_7 | TX_FIFO_6 | TX_FIFO_5 | TX_FIFO_4 | TX_FIFO_3 | TX_FIFO_2 | TX_FIFO_1 | RESERVED |
| R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h |
| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
| RESERVED | | | | | | USB_9 | USB_8 |
| R/W-0h | | | | | | R/W-0h | R/W-0h |
| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
| USB_7 | USB_6 | USB_5 | USB_4 | USB_3 | USB_2 | USB_1 | USB_0 |
| R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-118. USB1IRQENABLESET1 Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31 | TX_FIFO_15 | R/W | 0h | Interrupt enable for TX FIFO endpoint 15 |
| 30 | TX_FIFO_14 | R/W | 0h | Interrupt enable for TX FIFO endpoint 14 |
| 29 | TX_FIFO_13 | R/W | 0h | Interrupt enable for TX FIFO endpoint 13 |
| 28 | TX_FIFO_12 | R/W | 0h | Interrupt enable for TX FIFO endpoint 12 |
| 27 | TX_FIFO_11 | R/W | 0h | Interrupt enable for TX FIFO endpoint 11 |
| 26 | TX_FIFO_10 | R/W | 0h | Interrupt enable for TX FIFO endpoint 10 |
| 25 | TX_FIFO_9 | R/W | 0h | Interrupt enable for TX FIFO endpoint 9 |
| 24 | TX_FIFO_8 | R/W | 0h | Interrupt enable for TX FIFO endpoint 8 |
| 23 | TX_FIFO_7 | R/W | 0h | Interrupt enable for TX FIFO endpoint 7 |
| 22 | TX_FIFO_6 | R/W | 0h | Interrupt enable for TX FIFO endpoint 6 |
| 21 | TX_FIFO_5 | R/W | 0h | Interrupt enable for TX FIFO endpoint 5 |
| 20 | TX_FIFO_4 | R/W | 0h | Interrupt enable for TX FIFO endpoint 4 |
| 19 | TX_FIFO_3 | R/W | 0h | Interrupt enable for TX FIFO endpoint 3 |
| 18 | TX_FIFO_2 | R/W | 0h | Interrupt enable for TX FIFO endpoint 2 |
| 17 | TX_FIFO_1 | R/W | 0h | Interrupt enable for TX FIFO endpoint 1 |
| 16-10 | RESERVED | R/W | 0h | |
| 9 | USB_9 | R/W | 0h | Interrupt enable for Mentor controller USB_INT generic interrupt |
| 8 | USB_8 | R/W | 0h | Interrupt enable for DRVVBUS level change |
| 7 | USB_7 | R/W | 0h | Interrupt enable for VBUS < VBUS valid threshold |
| 6 | USB_6 | R/W | 0h | Interrupt enable for SRP detected |
| 5 | USB_5 | R/W | 0h | Interrupt enable for device disconnected (host mode) |
| 4 | USB_4 | R/W | 0h | Interrupt enable for device connected (host mode) |
| 3 | USB_3 | R/W | 0h | Interrupt enable for SOF started |
| 2 | USB_2 | R/W | 0h | Interrupt enable for Reset signaling detected (peripheral mode) Babble detected (host mode) |
| 1 | USB_1 | R/W | 0h | Interrupt enable for Resume signaling detected |

**Table 16-118. USB1IRQENABLESET1 Register Field Descriptions (continued)**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 0 | USB_0 | R/W | 0h | Interrupt enable for Suspend signaling detected USB1 IRQ_ENABLE_SET_1 Register |

### 16.4.3.11 USB1IRQENABLECLR0 Register (offset = 1840h) [reset = 0h]

USB1IRQENABLECLR0 is shown in Figure 16-109 and described in Table 16-119.

**Figure 16-109. USB1IRQENABLECLR0 Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| RX_EP_15 | RX_EP_14 | RX_EP_13 | RX_EP_12 | RX_EP_11 | RX_EP_10 | RX_EP_9 | RX_EP_8 |
| R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h |
| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
| RX_EP_7 | RX_EP_6 | RX_EP_5 | RX_EP_4 | RX_EP_3 | RX_EP_2 | RX_EP_1 | RESERVED |
| R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h |
| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
| TX_EP_15 | TX_EP_14 | TX_EP_13 | TX_EP_12 | TX_EP_11 | TX_EP_10 | TX_EP_9 | TX_EP_8 |
| R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h |
| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
| TX_EP_7 | TX_EP_6 | TX_EP_5 | TX_EP_4 | TX_EP_3 | TX_EP_2 | TX_EP_1 | TX_EP_0 |
| R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-119. USB1IRQENABLECLR0 Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31 | RX_EP_15 | R/W | 0h | Interrupt enable for RX endpoint 15 |
| 30 | RX_EP_14 | R/W | 0h | Interrupt enable for RX endpoint 14 |
| 29 | RX_EP_13 | R/W | 0h | Interrupt enable for RX endpoint 13 |
| 28 | RX_EP_12 | R/W | 0h | Interrupt enable for RX endpoint 12 |
| 27 | RX_EP_11 | R/W | 0h | Interrupt enable for RX endpoint 11 |
| 26 | RX_EP_10 | R/W | 0h | Interrupt enable for RX endpoint 10 |
| 25 | RX_EP_9 | R/W | 0h | Interrupt enable for RX endpoint 9 |
| 24 | RX_EP_8 | R/W | 0h | Interrupt enable for RX endpoint 8 |
| 23 | RX_EP_7 | R/W | 0h | Interrupt enable for RX endpoint 7 |
| 22 | RX_EP_6 | R/W | 0h | Interrupt enable for RX endpoint 6 |
| 21 | RX_EP_5 | R/W | 0h | Interrupt enable for RX endpoint 5 |
| 20 | RX_EP_4 | R/W | 0h | Interrupt enable for RX endpoint 4 |
| 19 | RX_EP_3 | R/W | 0h | Interrupt enable for RX endpoint 3 |
| 18 | RX_EP_2 | R/W | 0h | Interrupt enable for RX endpoint 2 |
| 17 | RX_EP_1 | R/W | 0h | Interrupt enable for RX endpoint 1 |
| 16 | RESERVED | R/W | 0h | |
| 15 | TX_EP_15 | R/W | 0h | Interrupt enable for TX endpoint 15 |
| 14 | TX_EP_14 | R/W | 0h | Interrupt enable for TX endpoint 14 |
| 13 | TX_EP_13 | R/W | 0h | Interrupt enable for TX endpoint 13 |
| 12 | TX_EP_12 | R/W | 0h | Interrupt enable for TX endpoint 12 |
| 11 | TX_EP_11 | R/W | 0h | Interrupt enable for TX endpoint 11 |
| 10 | TX_EP_10 | R/W | 0h | Interrupt enable for TX endpoint 10 |
| 9 | TX_EP_9 | R/W | 0h | Interrupt enable for TX endpoint 9 |
| 8 | TX_EP_8 | R/W | 0h | Interrupt enable for TX endpoint 8 |
| 7 | TX_EP_7 | R/W | 0h | Interrupt enable for TX endpoint 7 |
| 6 | TX_EP_6 | R/W | 0h | Interrupt enable for TX endpoint 6 |

**Table 16-119. USB1IRQENABLECLR0 Register Field Descriptions (continued)**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 5 | TX_EP_5 | R/W | 0h | Interrupt enable for TX endpoint 5 |
| 4 | TX_EP_4 | R/W | 0h | Interrupt enable for TX endpoint 4 |
| 3 | TX_EP_3 | R/W | 0h | Interrupt enable for TX endpoint 3 |
| 2 | TX_EP_2 | R/W | 0h | Interrupt enable for TX endpoint 2 |
| 1 | TX_EP_1 | R/W | 0h | Interrupt enable for TX endpoint 1 |
| 0 | TX_EP_0 | R/W | 0h | Interrupt enable for TX endpoint 0 USB1 IRQ_ENABLE_CLR_0 Register |

### 16.4.3.12 USB1IRQENABLECLR1 Register (offset = 1844h) [reset = 0h]

USB1IRQENABLECLR1 is shown in Figure 16-110 and described in Table 16-120.

**Figure 16-110. USB1IRQENABLECLR1 Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| TX_FIFO_15 | TX_FIFO_14 | TX_FIFO_13 | TX_FIFO_12 | TX_FIFO_11 | TX_FIFO_10 | TX_FIFO_9 | TX_FIFO_8 |
| R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h |
| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
| TX_FIFO_7 | TX_FIFO_6 | TX_FIFO_5 | TX_FIFO_4 | TX_FIFO_3 | TX_FIFO_2 | TX_FIFO_1 | RESERVED |
| R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h |
| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
| RESERVED | | | | | | USB_9 | USB_8 |
| R/W-0h | | | | | | R/W-0h | R/W-0h |
| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
| USB_7 | USB_6 | USB_5 | USB_4 | USB_3 | USB_2 | USB_1 | USB_0 |
| R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-120. USB1IRQENABLECLR1 Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31 | TX_FIFO_15 | R/W | 0h | Interrupt enable for TX FIFO endpoint 15 |
| 30 | TX_FIFO_14 | R/W | 0h | Interrupt enable for TX FIFO endpoint 14 |
| 29 | TX_FIFO_13 | R/W | 0h | Interrupt enable for TX FIFO endpoint 13 |
| 28 | TX_FIFO_12 | R/W | 0h | Interrupt enable for TX FIFO endpoint 12 |
| 27 | TX_FIFO_11 | R/W | 0h | Interrupt enable for TX FIFO endpoint 11 |
| 26 | TX_FIFO_10 | R/W | 0h | Interrupt enable for TX FIFO endpoint 10 |
| 25 | TX_FIFO_9 | R/W | 0h | Interrupt enable for TX FIFO endpoint 9 |
| 24 | TX_FIFO_8 | R/W | 0h | Interrupt enable for TX FIFO endpoint 8 |
| 23 | TX_FIFO_7 | R/W | 0h | Interrupt enable for TX FIFO endpoint 7 |
| 22 | TX_FIFO_6 | R/W | 0h | Interrupt enable for TX FIFO endpoint 6 |
| 21 | TX_FIFO_5 | R/W | 0h | Interrupt enable for TX FIFO endpoint 5 |
| 20 | TX_FIFO_4 | R/W | 0h | Interrupt enable for TX FIFO endpoint 4 |
| 19 | TX_FIFO_3 | R/W | 0h | Interrupt enable for TX FIFO endpoint 3 |
| 18 | TX_FIFO_2 | R/W | 0h | Interrupt enable for TX FIFO endpoint 2 |
| 17 | TX_FIFO_1 | R/W | 0h | Interrupt enable for TX FIFO endpoint 1 |
| 16-10 | RESERVED | R/W | 0h | |
| 9 | USB_9 | R/W | 0h | Interrupt enable for Mentor controller USB_INT generic interrupt |
| 8 | USB_8 | R/W | 0h | Interrupt enable for DRVVBUS level change |
| 7 | USB_7 | R/W | 0h | Interrupt enable for VBUS < VBUS valid threshold |
| 6 | USB_6 | R/W | 0h | Interrupt enable for SRP detected |
| 5 | USB_5 | R/W | 0h | Interrupt enable for device disconnected (host mode) |
| 4 | USB_4 | R/W | 0h | Interrupt enable for device connected (host mode) |
| 3 | USB_3 | R/W | 0h | Interrupt enable for SOF started |
| 2 | USB_2 | R/W | 0h | Interrupt enable for Reset signaling detected (peripheral mode) Babble detected (host mode) |
| 1 | USB_1 | R/W | 0h | Interrupt enable for Resume signaling detected |

**Table 16-120. USB1IRQENABLECLR1 Register Field Descriptions (continued)**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 0 | USB_0 | R/W | 0h | Interrupt enable for Suspend signaling detected USB1 IRQ_ENABLE_CLR_1 Register |

### 16.4.3.13 USB1TXMODE Register (offset = 1870h) [reset = 0h]

USB1TXMODE is shown in Figure 16-111 and described in Table 16-121.

**Figure 16-111. USB1TXMODE Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| RESERVED | | TX15_MODE | | TX14_MODE | | TX13_MODE | |
| R/W-0h | | R/W-0h | | R/W-0h | | R/W-0h | |
| **23** | **22** | **21** | **20** | **19** | **18** | **17** | **16** |
| TX12_MODE | | TX11_MODE | | TX10_MODE | | TX9_MODE | |
| R/W-0h | | R/W-0h | | R/W-0h | | R/W-0h | |
| **15** | **14** | **13** | **12** | **11** | **10** | **9** | **8** |
| TX8_MODE | | TX7_MODE | | TX6_MODE | | TX5_MODE | |
| R/W-0h | | R/W-0h | | R/W-0h | | R/W-0h | |
| **7** | **6** | **5** | **4** | **3** | **2** | **1** | **0** |
| TX4_MODE | | TX3_MODE | | TX2_MODE | | TX1_MODE | |
| R/W-0h | | R/W-0h | | R/W-0h | | R/W-0h | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-121. USB1TXMODE Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-30 | RESERVED | R/W | 0h | |
| 29-28 | TX15_MODE | R/W | 0h | 00: Transparent Mode on TX endpoint 14<br>01: RNDIS Mode on TX endpoint 14<br>10: CDC Mode on TX endpoint 14<br>11: Generic RNDIS Mode on TX endpoint 14 |
| 27-26 | TX14_MODE | R/W | 0h | 00: Transparent Mode on TX endpoint 13<br>01: RNDIS Mode on TX endpoint 13<br>10: CDC Mode on TX endpoint 13<br>11: Generic RNDIS Mode on TX endpoint 13 |
| 25-24 | TX13_MODE | R/W | 0h | 00: Transparent Mode on TX endpoint 12<br>01: RNDIS Mode on TX endpoint 12<br>10: CDC Mode on TX endpoint 12<br>11: Generic RNDIS Mode on TX endpoint 12 |
| 23-22 | TX12_MODE | R/W | 0h | 00: Transparent Mode on TX endpoint 11<br>01: RNDIS Mode on TX endpoint 11<br>10: CDC Mode on TX endpoint 11<br>11: Generic RNDIS Mode on TX endpoint 11 |
| 21-20 | TX11_MODE | R/W | 0h | 00: Transparent Mode on TX endpoint 10<br>01: RNDIS Mode on TX endpoint 10<br>10: CDC Mode on TX endpoint 10<br>11: Generic RNDIS Mode on TX endpoint 10 |
| 19-18 | TX10_MODE | R/W | 0h | 00: Transparent Mode on TX endpoint 9<br>01: RNDIS Mode on TX endpoint 9<br>10: CDC Mode on TX endpoint 9<br>11: Generic RNDIS Mode on TX endpoint 9 |
| 17-16 | TX9_MODE | R/W | 0h | 00: Transparent Mode on TX endpoint 8<br>01: RNDIS Mode on TX endpoint 8<br>10: CDC Mode on TX endpoint 8<br>11: Generic RNDIS Mode on TX endpoint 8 |
| 15-14 | TX8_MODE | R/W | 0h | 00: Transparent Mode on TX endpoint 7<br>01: RNDIS Mode on TX endpoint 7<br>10: CDC Mode on TX endpoint 7<br>11: Generic RNDIS Mode on TX endpoint 7 |
| 13-12 | TX7_MODE | R/W | 0h | 00: Transparent Mode on TX endpoint 6<br>01: RNDIS Mode on TX endpoint 6<br>10: CDC Mode on TX endpoint 6<br>11: Generic RNDIS Mode on TX endpoint 6 |

**Table 16-121. USB1TXMODE Register Field Descriptions (continued)**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 11-10 | TX6_MODE | R/W | 0h | 00: Transparent Mode on TX endpoint 5<br>01: RNDIS Mode on TX endpoint 5<br>10: CDC Mode on TX endpoint 5<br>11: Generic RNDIS Mode on TX endpoint 5 |
| 9-8 | TX5_MODE | R/W | 0h | 00: Transparent Mode on TX endpoint 4<br>01: RNDIS Mode on TX endpoint 4<br>10: CDC Mode on TX endpoint 4<br>11: Generic RNDIS Mode on TX endpoint 4 |
| 7-6 | TX4_MODE | R/W | 0h | 00: Transparent Mode on TX endpoint 3<br>01: RNDIS Mode on TX endpoint 3<br>10: CDC Mode on TX endpoint 3<br>11: Generic RNDIS Mode on TX endpoint 3 |
| 5-4 | TX3_MODE | R/W | 0h | 00: Transparent Mode on TX endpoint 2<br>01: RNDIS Mode on TX endpoint 2<br>10: CDC Mode on TX endpoint 2<br>11: Generic RNDIS Mode on TX endpoint 2 |
| 3-2 | TX2_MODE | R/W | 0h | 00: Transparent Mode on TX endpoint 1<br>01: RNDIS Mode on TX endpoint 1<br>10: CDC Mode on TX endpoint 1<br>11: Generic RNDIS Mode on TX endpoint 1 |
| 1-0 | TX1_MODE | R/W | 0h | 00: Transparent Mode on TX endpoint 0<br>01: RNDIS Mode on TX endpoint 0<br>10: CDC Mode on TX endpoint 0<br>11: Generic RNDIS Mode on TX endpoint 0 USB1 Tx Mode Registers |

### 16.4.3.14 USB1RXMODE Register (offset = 1874h) [reset = 0h]

USB1RXMODE is shown in Figure 16-112 and described in Table 16-122.

**Figure 16-112. USB1RXMODE Register**

| 31-30 | 29-28 | 27-26 | 25-24 |
|---|---|---|---|
| RESERVED | RX15_MODE | RX14_MODE | RX13_MODE |
| R/W-0h | R/W-0h | R/W-0h | R/W-0h |
| **23-22** | **21-20** | **19-18** | **17-16** |
| RX12_MODE | RX11_MODE | RX10_MODE | RX9_MODE |
| R/W-0h | R/W-0h | R/W-0h | R/W-0h |
| **15-14** | **13-12** | **11-10** | **9-8** |
| RX8_MODE | RX7_MODE | RX6_MODE | RX5_MODE |
| R/W-0h | R/W-0h | R/W-0h | R/W-0h |
| **7-6** | **5-4** | **3-2** | **1-0** |
| RX4_MODE | RX3_MODE | RX2_MODE | RX1_MODE |
| R/W-0h | R/W-0h | R/W-0h | R/W-0h |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-122. USB1RXMODE Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-30 | RESERVED | R/W | 0h | |
| 29-28 | RX15_MODE | R/W | 0h | 00: Transparent Mode on RX endpoint 14<br>01: RNDIS Mode on RX endpoint 14<br>10: CDC Mode on RX endpoint 14<br>11: Generic RNDIS or Infinite Mode on RX endpoint 14 |
| 27-26 | RX14_MODE | R/W | 0h | 00: Transparent Mode on RX endpoint 13<br>01: RNDIS Mode on RX endpoint 13<br>10: CDC Mode on RX endpoint 13<br>11: Generic RNDIS or Infinite Mode on RX endpoint 13 |
| 25-24 | RX13_MODE | R/W | 0h | 00: Transparent Mode on RX endpoint 12<br>01: RNDIS Mode on RX endpoint 12<br>10: CDC Mode on RX endpoint 12<br>11: Generic RNDIS or Infinite Mode on RX endpoint 12 |
| 23-22 | RX12_MODE | R/W | 0h | 00: Transparent Mode on RX endpoint 11<br>01: RNDIS Mode on RX endpoint 11<br>10: CDC Mode on RX endpoint 11<br>11: Generic RNDIS or Infinite Mode on RX endpoint 11 |
| 21-20 | RX11_MODE | R/W | 0h | 00: Transparent Mode on RX endpoint 10<br>01: RNDIS Mode on RX endpoint 10<br>10: CDC Mode on RX endpoint 10<br>11: Generic RNDIS or Infinite Mode on RX endpoint 10 |
| 19-18 | RX10_MODE | R/W | 0h | 00: Transparent Mode on RX endpoint 9<br>01: RNDIS Mode on RX endpoint 9<br>10: CDC Mode on RX endpoint 9<br>11: Generic RNDIS or Infinite Mode on RX endpoint 9 |
| 17-16 | RX9_MODE | R/W | 0h | 00: Transparent Mode on RX endpoint 8<br>01: RNDIS Mode on RX endpoint 8<br>10: CDC Mode on RX endpoint 8<br>11: Generic RNDIS or Infinite Mode on RX endpoint 8 |
| 15-14 | RX8_MODE | R/W | 0h | 00: Transparent Mode on RX endpoint 7<br>01: RNDIS Mode on RX endpoint 7<br>10: CDC Mode on RX endpoint 7<br>11: Generic RNDIS or Infinite Mode on RX endpoint 7 |
| 13-12 | RX7_MODE | R/W | 0h | 00: Transparent Mode on RX endpoint 6<br>01: RNDIS Mode on RX endpoint 6<br>10: CDC Mode on RX endpoint 6<br>11: Generic RNDIS or Infinite Mode on RX endpoint 6 |

**Table 16-122. USB1RXMODE Register Field Descriptions (continued)**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 11-10 | RX6_MODE | R/W | 0h | 00: Transparent Mode on RX endpoint 5<br>01: RNDIS Mode on RX endpoint 5<br>10: CDC Mode on RX endpoint 5<br>11: Generic RNDIS or Infinite Mode on RX endpoint 5 |
| 9-8 | RX5_MODE | R/W | 0h | 00: Transparent Mode on RX endpoint 4<br>01: RNDIS Mode on RX endpoint 4<br>10: CDC Mode on RX endpoint 4<br>11: Generic RNDIS or Infinite Mode on RX endpoint 4 |
| 7-6 | RX4_MODE | R/W | 0h | 00: Transparent Mode on RX endpoint 3<br>01: RNDIS Mode on RX endpoint 3<br>10: CDC Mode on RX endpoint 3<br>11: Generic RNDIS or Infinite Mode on RX endpoint 3 |
| 5-4 | RX3_MODE | R/W | 0h | 00: Transparent Mode on RX endpoint 2<br>01: RNDIS Mode on RX endpoint 2<br>10: CDC Mode on RX endpoint 2<br>11: Generic RNDIS or Infinite Mode on RX endpoint 2 |
| 3-2 | RX2_MODE | R/W | 0h | 00: Transparent Mode on RX endpoint 1<br>01: RNDIS Mode on RX endpoint 1<br>10: CDC Mode on RX endpoint 1<br>11: Generic RNDIS or Infinite Mode on RX endpoint 1 |
| 1-0 | RX1_MODE | R/W | 0h | 00: Transparent Mode on RX endpoint 0<br>01: RNDIS Mode on RX endpoint 0<br>10: CDC Mode on RX endpoint 0<br>11: Generic RNDIS or Infinite Mode on RX endpoint 0 USB1 Rx Mode Registers |

### 16.4.3.15 USB1GENRNDISEP1 Register (offset = 1880h) [reset = 0h]

USB1GENRNDISEP1 is shown in Figure 16-113 and described in Table 16-123.

**Figure 16-113. USB1GENRNDISEP1 Register**

| 31-17 | 16-0 |
|---|---|
| RESERVED | EP1_SIZE |
| R/W-0h | R/W-0h |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-123. USB1GENRNDISEP1 Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-17 | RESERVED | R/W | 0h | |
| 16-0 | EP1_SIZE | R/W | 0h | Generic RNDIS packet size.<br>USB1 Generic RNDIS EP N Size Register |

### 16.4.3.16 USB1GENRNDISEP2 Register (offset = 1884h) [reset = 0h]

USB1GENRNDISEP2 is shown in Figure 16-114 and described in Table 16-124.

**Figure 16-114. USB1GENRNDISEP2 Register**

| 31-17 | 16-0 |
|---|---|
| RESERVED | EP2_SIZE |
| R/W-0h | R/W-0h |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-124. USB1GENRNDISEP2 Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-17 | RESERVED | R/W | 0h | |
| 16-0 | EP2_SIZE | R/W | 0h | Generic RNDIS packet size.<br>USB1 Generic RNDIS EP N Size Register |

### 16.4.3.17 USB1GENRNDISEP3 Register (offset = 1888h) [reset = 0h]

USB1GENRNDISEP3 is shown in Figure 16-115 and described in Table 16-125.

**Figure 16-115. USB1GENRNDISEP3 Register**

| 31–17 | 16–0 |
|---|---|
| RESERVED | EP3_SIZE |
| R/W-0h | R/W-0h |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-125. USB1GENRNDISEP3 Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-17 | RESERVED | R/W | 0h | |
| 16-0 | EP3_SIZE | R/W | 0h | Generic RNDIS packet size.<br>USB1 Generic RNDIS EP N Size Register |

### 16.4.3.18 USB1GENRNDISEP4 Register (offset = 188Ch) [reset = 0h]

USB1GENRNDISEP4 is shown in Figure 16-116 and described in Table 16-126.

**Figure 16-116. USB1GENRNDISEP4 Register**

| 31-17 | 16-0 |
|---|---|
| RESERVED | EP4_SIZE |
| R/W-0h | R/W-0h |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-126. USB1GENRNDISEP4 Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-17 | RESERVED | R/W | 0h | |
| 16-0 | EP4_SIZE | R/W | 0h | Generic RNDIS packet size.<br>USB1 Generic RNDIS EP N Size Register |

### 16.4.3.19 USB1GENRNDISEP5 Register (offset = 1890h) [reset = 0h]

USB1GENRNDISEP5 is shown in Figure 16-117 and described in Table 16-127.

**Figure 16-117. USB1GENRNDISEP5 Register**

| 31-17 | 16-0 |
|---|---|
| RESERVED | EP5_SIZE |
| R/W-0h | R/W-0h |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-127. USB1GENRNDISEP5 Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-17 | RESERVED | R/W | 0h | |
| 16-0 | EP5_SIZE | R/W | 0h | Generic RNDIS packet size.<br>USB1 Generic RNDIS EP N Size Register |

### 16.4.3.20 USB1GENRNDISEP6 Register (offset = 1894h) [reset = 0h]

USB1GENRNDISEP6 is shown in Figure 16-118 and described in Table 16-128.

**Figure 16-118. USB1GENRNDISEP6 Register**

| 31-17 | 16-0 |
|---|---|
| RESERVED | EP6_SIZE |
| R/W-0h | R/W-0h |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-128. USB1GENRNDISEP6 Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-17 | RESERVED | R/W | 0h | |
| 16-0 | EP6_SIZE | R/W | 0h | Generic RNDIS packet size.<br>USB1 Generic RNDIS EP N Size Register |

### 16.4.3.21 USB1GENRNDISEP7 Register (offset = 1898h) [reset = 0h]

USB1GENRNDISEP7 is shown in Figure 16-119 and described in Table 16-129.

**Figure 16-119. USB1GENRNDISEP7 Register**

| 31-17 | 16-0 |
|---|---|
| RESERVED | EP7_SIZE |
| R/W-0h | R/W-0h |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-129. USB1GENRNDISEP7 Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-17 | RESERVED | R/W | 0h | |
| 16-0 | EP7_SIZE | R/W | 0h | Generic RNDIS packet size.<br>USB1 Generic RNDIS EP N Size Register |

### 16.4.3.22 USB1GENRNDISEP8 Register (offset = 189Ch) [reset = 0h]

USB1GENRNDISEP8 is shown in Figure 16-120 and described in Table 16-130.

**Figure 16-120. USB1GENRNDISEP8 Register**

| 31-17 | 16-0 |
|---|---|
| RESERVED | EP8_SIZE |
| R/W-0h | R/W-0h |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-130. USB1GENRNDISEP8 Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-17 | RESERVED | R/W | 0h | |
| 16-0 | EP8_SIZE | R/W | 0h | Generic RNDIS packet size.<br>USB1 Generic RNDIS EP N Size Register |

### 16.4.3.23 USB1GENRNDISEP9 Register (offset = 18A0h) [reset = 0h]

USB1GENRNDISEP9 is shown in Figure 16-121 and described in Table 16-131.

**Figure 16-121. USB1GENRNDISEP9 Register**

| 31-17 | 16-0 |
|---|---|
| RESERVED | EP9_SIZE |
| R/W-0h | R/W-0h |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-131. USB1GENRNDISEP9 Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-17 | RESERVED | R/W | 0h | |
| 16-0 | EP9_SIZE | R/W | 0h | Generic RNDIS packet size.<br>USB1 Generic RNDIS EP N Size Register |

### 16.4.3.24 USB1GENRNDISEP10 Register (offset = 18A4h) [reset = 0h]

USB1GENRNDISEP10 is shown in Figure 16-122 and described in Table 16-132.

**Figure 16-122. USB1GENRNDISEP10 Register**

| 31-17 | 16-0 |
|---|---|
| RESERVED | EP10_SIZE |
| R/W-0h | R/W-0h |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-132. USB1GENRNDISEP10 Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-17 | RESERVED | R/W | 0h | |
| 16-0 | EP10_SIZE | R/W | 0h | Generic RNDIS packet size.<br>USB1 Generic RNDIS EP N Size Register |

### 16.4.3.25 USB1GENRNDISEP11 Register (offset = 18A8h) [reset = 0h]

USB1GENRNDISEP11 is shown in Figure 16-123 and described in Table 16-133.

**Figure 16-123. USB1GENRNDISEP11 Register**

| 31-17 | 16-0 |
|---|---|
| RESERVED | EP11_SIZE |
| R/W-0h | R/W-0h |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-133. USB1GENRNDISEP11 Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-17 | RESERVED | R/W | 0h | |
| 16-0 | EP11_SIZE | R/W | 0h | Generic RNDIS packet size.<br>USB1 Generic RNDIS EP N Size Register |

### 16.4.3.26 USB1GENRNDISEP12 Register (offset = 18ACh) [reset = 0h]

USB1GENRNDISEP12 is shown in Figure 16-124 and described in Table 16-134.

**Figure 16-124. USB1GENRNDISEP12 Register**

| 31-17 | 16-0 |
|---|---|
| RESERVED | EP12_SIZE |
| R/W-0h | R/W-0h |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-134. USB1GENRNDISEP12 Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-17 | RESERVED | R/W | 0h | |
| 16-0 | EP12_SIZE | R/W | 0h | Generic RNDIS packet size.<br>USB1 Generic RNDIS EP N Size Register |

### 16.4.3.27 USB1GENRNDISEP13 Register (offset = 18B0h) [reset = 0h]

USB1GENRNDISEP13 is shown in Figure 16-125 and described in Table 16-135.

**Figure 16-125. USB1GENRNDISEP13 Register**

| 31-17 | 16-0 |
|---|---|
| RESERVED | EP13_SIZE |
| R/W-0h | R/W-0h |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-135. USB1GENRNDISEP13 Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-17 | RESERVED | R/W | 0h | |
| 16-0 | EP13_SIZE | R/W | 0h | Generic RNDIS packet size.<br>USB1 Generic RNDIS EP N Size Register |

### 16.4.3.28 USB1GENRNDISEP14 Register (offset = 18B4h) [reset = 0h]

USB1GENRNDISEP14 is shown in Figure 16-126 and described in Table 16-136.

**Figure 16-126. USB1GENRNDISEP14 Register**

| 31-17 | 16-0 |
|---|---|
| RESERVED | EP14_SIZE |
| R/W-0h | R/W-0h |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-136. USB1GENRNDISEP14 Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-17 | RESERVED | R/W | 0h | |
| 16-0 | EP14_SIZE | R/W | 0h | Generic RNDIS packet size.<br>USB1 Generic RNDIS EP N Size Register |

### 16.4.3.29 USB1GENRNDISEP15 Register (offset = 18B8h) [reset = 0h]

USB1GENRNDISEP15 is shown in Figure 16-127 and described in Table 16-137.

**Figure 16-127. USB1GENRNDISEP15 Register**

| 31–17 | 16–0 |
|---|---|
| RESERVED | EP15_SIZE |
| R/W-0h | R/W-0h |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-137. USB1GENRNDISEP15 Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-17 | RESERVED | R/W | 0h | |
| 16-0 | EP15_SIZE | R/W | 0h | Generic RNDIS packet size.<br>USB1 Generic RNDIS EP N Size Register |

### 16.4.3.30 USB1AUTOREQ Register (offset = 18D0h) [reset = 0h]

USB1AUTOREQ is shown in Figure 16-128 and described in Table 16-138.

**Figure 16-128. USB1AUTOREQ Register**

| 31-30 | 29-28 | 27-26 | 25-24 |
|---|---|---|---|
| RESERVED | RX15_AUTOREQ | RX14_AUTOREQ | RX13_AUTOREQ |
| R/W-0h | R/W-0h | R/W-0h | R/W-0h |
| **23-22** | **21-20** | **19-18** | **17-16** |
| RX12_AUTOREQ | RX11_AUTOREQ | RX10_AUTOREQ | RX9_AUTOREQ |
| R/W-0h | R/W-0h | R/W-0h | R/W-0h |
| **15-14** | **13-12** | **11-10** | **9-8** |
| RX8_AUTOREQ | RX7_AUTOREQ | RX6_AUTOREQ | RX5_AUTOREQ |
| R/W-0h | R/W-0h | R/W-0h | R/W-0h |
| **7-6** | **5-4** | **3-2** | **1-0** |
| RX4_AUTOREQ | RX3_AUTOREQ | RX2_AUTOREQ | RX1_AUTOREQ |
| R/W-0h | R/W-0h | R/W-0h | R/W-0h |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-138. USB1AUTOREQ Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-30 | RESERVED | R/W | 0h | |
| 29-28 | RX15_AUTOREQ | R/W | 0h | RX endpoint 14 Auto Req enable<br>00 = no auto req<br>01 = auto req on all but EOP<br>10 = reserved<br>11 = auto req always |
| 27-26 | RX14_AUTOREQ | R/W | 0h | RX endpoint 13 Auto Req enable<br>00 = no auto req<br>01 = auto req on all but EOP<br>10 = reserved<br>11 = auto req always |
| 25-24 | RX13_AUTOREQ | R/W | 0h | RX endpoint 12 Auto Req enable<br>00 = no auto req<br>01 = auto req on all but EOP<br>10 = reserved<br>11 = auto req always |
| 23-22 | RX12_AUTOREQ | R/W | 0h | RX endpoint 11 Auto Req enable<br>00 = no auto req<br>01 = auto req on all but EOP<br>10 = reserved<br>11 = auto req always |
| 21-20 | RX11_AUTOREQ | R/W | 0h | RX endpoint 10 Auto Req enable<br>00 = no auto req<br>01 = auto req on all but EOP<br>10 = reserved<br>11 = auto req always |
| 19-18 | RX10_AUTOREQ | R/W | 0h | RX endpoint 9 Auto Req enable<br>00 = no auto req<br>01 = auto req on all but EOP<br>10 = reserved<br>11 = auto req always |
| 17-16 | RX9_AUTOREQ | R/W | 0h | RX endpoint 8 Auto Req enable<br>00 = no auto req<br>01 = auto req on all but EOP<br>10 = reserved<br>11 = auto req always |

**Table 16-138. USB1AUTOREQ Register Field Descriptions (continued)**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 15-14 | RX8_AUTOREQ | R/W | 0h | RX endpoint 7 Auto Req enable<br>00 = no auto req<br>01 = auto req on all but EOP<br>10 = reserved<br>11 = auto req always |
| 13-12 | RX7_AUTOREQ | R/W | 0h | RX endpoint 6 Auto Req enable<br>00 = no auto req<br>01 = auto req on all but EOP<br>10 = reserved<br>11 = auto req always |
| 11-10 | RX6_AUTOREQ | R/W | 0h | RX endpoint 5 Auto Req enable<br>00 = no auto req<br>01 = auto req on all but EOP<br>10 = reserved<br>11 = auto req always |
| 9-8 | RX5_AUTOREQ | R/W | 0h | RX endpoint 4 Auto Req enable<br>00 = no auto req<br>01 = auto req on all but EOP<br>10 = reserved<br>11 = auto req always |
| 7-6 | RX4_AUTOREQ | R/W | 0h | RX endpoint 3 Auto Req enable<br>00 = no auto req<br>01 = auto req on all but EOP<br>10 = reserved<br>11 = auto req always |
| 5-4 | RX3_AUTOREQ | R/W | 0h | RX endpoint 2 Auto Req enable<br>00 = no auto req<br>01 = auto req on all but EOP<br>10 = reserved<br>11 = auto req always |
| 3-2 | RX2_AUTOREQ | R/W | 0h | RX endpoint 1 Auto Req enable<br>00 = no auto req<br>01 = auto req on all but EOP<br>10 = reserved<br>11 = auto req always |
| 1-0 | RX1_AUTOREQ | R/W | 0h | RX endpoint 0 Auto Req enable<br>00 = no auto req<br>01 = auto req on all but EOP<br>10 = reserved<br>11 = auto req always USB1 Auto Req Registers |

### 16.4.3.31 USB1SRPFIXTIME Register (offset = 18D4h) [reset = 280DE80h]

USB1SRPFIXTIME is shown in Figure 16-129 and described in Table 16-139.

**Figure 16-129. USB1SRPFIXTIME Register**

| 31–0 |
|---|
| SRPFIXTIME |
| R/W-280DE80h |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-139. USB1SRPFIXTIME Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-0 | SRPFIXTIME | R/W | 280DE80h | SRP Fix maximum time in 60 MHz cycles.<br>Default is 700 ms.<br>USB1 SRP Fix Time Register |

### 16.4.3.32 USB1TDOWN Register (offset = 18D8h) [reset = 0h]

USB1TDOWN is shown in Figure 16-130 and described in Table 16-140.

**Figure 16-130. USB1TDOWN Register**

| 31-17 | 16 | 15-1 | 0 |
|---|---|---|---|
| TX_TDOWN | RESERVED | RX_TDOWN | RESERVED |
| R/W-0h | R/W-0h | R/W-0h | R/W-0h |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-140. USB1TDOWN Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-17 | TX_TDOWN | R/W | 0h | Tx Endpoint Teardown.<br>Write '1' to corresponding bit to set.<br>Read as '0'.<br>Bit<br>31 = Endpoint 15 ...<br>Bit<br>17 = Endpoint 1 |
| 16 | RESERVED | R/W | 0h | |
| 15-1 | RX_TDOWN | R/W | 0h | RX Endpoint Teardown.<br>Write '1' to corresponding bit to set.<br>Read as '0'.<br>Bit<br>15 = Endpoint 15 ...<br>Bit<br>1 = Endpoint 1 |
| 0 | RESERVED | R/W | 0h | |

### 16.4.3.33 USB1UTMI Register (offset = 18E0h) [reset = 200002h]

USB1UTMI is shown in Figure 16-131 and described in Table 16-141.

**Figure 16-131. USB1UTMI Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| RESERVED | | | | | | | |
| R/W-0h | | | | | | | |
| **23** | **22** | **21** | **20** | **19** | **18** | **17** | **16** |
| TXBITSTUFFEN | TXBITSTUFFENH | OTGDISABLE | VBUSVLDEXTSEL | VBUSVLDEXT | TXENABLEN | FSXCVROWNER | TXVALIDH |
| R/W-0h | R/W-0h | R/W-1h | R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h |
| **15** | **14** | **13** | **12** | **11** | **10** | **9** | **8** |
| DATAINH | | | | | | | |
| R/W-0h | | | | | | | |
| **7** | **6** | **5** | **4** | **3** | **2** | **1** | **0** |
| RESERVED | | | | | WORDINTERFACE | FSDATAEXT | FSSE0EXT |
| R/W-0h | | | | | R/W-0h | R/W-1h | R/W-0h |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-141. USB1UTMI Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-24 | RESERVED | R/W | 0h | |
| 23 | TXBITSTUFFEN | R/W | 0h | PHY UTMI input for signal txbitstuffen |
| 22 | TXBITSTUFFENH | R/W | 0h | PHY UTMI input for signal txbitstuffenh |
| 21 | OTGDISABLE | R/W | 1h | PHY UTMI input for signal otgdisable |
| 20 | VBUSVLDEXTSEL | R/W | 0h | PHY UTMI input for signal vbusvldextsel |
| 19 | VBUSVLDEXT | R/W | 0h | PHY UTMI input for signal vbusvldext |
| 18 | TXENABLEN | R/W | 0h | PHY UTMI input for signal txenablen |
| 17 | FSXCVROWNER | R/W | 0h | PHY UTMI input for signal fsxcvrowner |
| 16 | TXVALIDH | R/W | 0h | PHY UTMI input for signal txvalidh |
| 15-8 | DATAINH | R/W | 0h | PHY UTMI input for signal datainh |
| 7-3 | RESERVED | R/W | 0h | |
| 2 | WORDINTERFACE | R/W | 0h | PHY UTMI input for signal wordinterface |
| 1 | FSDATAEXT | R/W | 1h | PHY UTMI input for signal fsdataext |
| 0 | FSSE0EXT | R/W | 0h | PHY UTMI input for signal fsse0ext USB1 PHY UTMI Register |

### 16.4.3.34 USB1UTMILB Register (offset = 18E4h) [reset = 82h]

USB1UTMILB is shown in Figure 16-132 and described in Table 16-142.

**Figure 16-132. USB1UTMILB Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| RESERVED | | | SUSPENDM | OPMODE | | TXVALID | XCVRSEL |
| R/W-0h | | | R-0h | R-0h | | R-0h | R-0h |
| **23** | **22** | **21** | **20** | **19** | **18** | **17** | **16** |
| XCVRSEL | TERMSEL | DRVVBUS | CHRGVBUS | DISCHRGVBUS | DPPULLDOWN | DMPULLDOWN | IDPULLUP |
| R-0h | R-0h | R-0h | R-0h | R-0h | R-0h | R-0h | R-0h |
| **15** | **14** | **13** | **12** | **11** | **10** | **9** | **8** |
| RESERVED | | | | IDDIG | HOSTDISCON | SESSEND | AVALID |
| R/W-0h | | | | R/W-0h | R/W-0h | R/W-0h | R/W-0h |
| **7** | **6** | **5** | **4** | **3** | **2** | **1** | **0** |
| VBUSVALID | RXERROR | RESERVED | | LINESTATE | | RESERVED | |
| R/W-1h | R/W-0h | R/W-0h | | R/W-0h | | R/W-2h | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-142. USB1UTMILB Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-29 | RESERVED | R/W | 0h | |
| 28 | SUSPENDM | R | 0h | loopback test observed value for suspendm |
| 27-26 | OPMODE | R | 0h | loopback test observed value for opmode |
| 25 | TXVALID | R | 0h | loopback test observed value for txvalid |
| 24-23 | XCVRSEL | R | 0h | loopback test observed value for xcvrsel |
| 22 | TERMSEL | R | 0h | loopback test observed value for termsel |
| 21 | DRVVBUS | R | 0h | loopback test observed value for drvvbus |
| 20 | CHRGVBUS | R | 0h | loopback test observed value for chrgvbus |
| 19 | DISCHRGVBUS | R | 0h | loopback test observed value for dischrgvbus |
| 18 | DPPULLDOWN | R | 0h | loopback test observed value for dppulldown |
| 17 | DMPULLDOWN | R | 0h | loopback test observed value for dmpulldown |
| 16 | IDPULLUP | R | 0h | loopback test observed value for idpullup |
| 15-12 | RESERVED | R/W | 0h | |
| 11 | IDDIG | R/W | 0h | loopback test value for iddig |
| 10 | HOSTDISCON | R/W | 0h | loopback test value for hostdiscon |
| 9 | SESSEND | R/W | 0h | loopback test value for sessend |
| 8 | AVALID | R/W | 0h | loopback test value for avalid |
| 7 | VBUSVALID | R/W | 1h | loopback test value for vbusvalid |
| 6 | RXERROR | R/W | 0h | loopback test value for rxerror |
| 5-4 | RESERVED | R/W | 0h | |
| 3-2 | LINESTATE | R/W | 0h | loopback test value for linestate |
| 1-0 | RESERVED | R/W | 2h | |

#### 16.4.3.35 USB1MODE Register (offset = 18E8h) [reset = 100h]

USB1MODE is shown in Figure 16-133 and described in Table 16-143.

**Figure 16-133. USB1MODE Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| RESERVED | | | | | | | |
| R/W-0h | | | | | | | |
| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
| RESERVED | | | | | | | |
| R/W-0h | | | | | | | |
| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
| RESERVED | | | | | | | IDDIG |
| R/W-0h | | | | | | | R/W-1h |
| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
| IDDIG_MUX | RESERVED | | | | | PHY_TEST | LOOPBACK |
| R/W-0h | R/W-0h | | | | | R/W-0h | R/W-0h |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-143. USB1MODE Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-9 | RESERVED | R/W | 0h | |
| 8 | IDDIG | R/W | 1h | MGC input value for iddig<br>0=A type<br>1=B type |
| 7 | IDDIG_MUX | R/W | 0h | Multiplexer control for IDDIG signal going to the controller.<br>0 = IDDIG is from PHY1.<br>1 = IDDIG is from bit8 (IDDIG) of this USB1MODE register. |
| 6-2 | RESERVED | R/W | 0h | |
| 1 | PHY_TEST | R/W | 0h | PHY test<br>0 = Normal mode<br>1 = PHY test mode |
| 0 | LOOPBACK | R/W | 0h | Loopback test mode<br>0 = Normal mode<br>1 = Loopback test mode USB1 Mode Register |

### *16.4.4 USB2PHY Registers*

Table 16-144 lists the memory-mapped registers for the USB2PHY. All register offset addresses not listed in Table 16-144 should be considered as reserved locations and the register contents should not be modified.

**Table 16-144. USB2PHY Registers**

| Offset | Acronym | Register Name | Section |
|---|---|---|---|
| 0h | Termination_control | | Section 16.4.4.1 |
| 4h | RX_CALIB | | Section 16.4.4.2 |
| 8h | DLLHS_2 | | Section 16.4.4.3 |
| Ch | RX_TEST_2 | | Section 16.4.4.4 |
| 14h | CHRG_DET | | Section 16.4.4.5 |
| 18h | PWR_CNTL | | Section 16.4.4.6 |
| 1Ch | UTMI_INTERFACE_CNTL_1 | | Section 16.4.4.7 |
| 20h | UTMI_INTERFACE_CNTL_2 | | Section 16.4.4.8 |
| 24h | BIST | | Section 16.4.4.9 |
| 28h | BIST_CRC | | Section 16.4.4.10 |

**Table 16-144. USB2PHY Registers (continued)**

| Offset | Acronym | Register Name | Section |
|---|---|---|---|
| 2Ch | CDR_BIST2 | | Section 16.4.4.11 |
| 30h | GPIO | | Section 16.4.4.12 |
| 34h | DLLHS | | Section 16.4.4.13 |
| 3Ch | USB2PHYCM_CONFIG | | Section 16.4.4.14 |
| 44h | AD_INTERFACE_REG1 | | Section 16.4.4.15 |
| 48h | AD_INTERFACE_REG2 | | Section 16.4.4.16 |
| 4Ch | AD_INTERFACE_REG3 | | Section 16.4.4.17 |
| 54h | ANA_CONFIG2 | | Section 16.4.4.18 |

#### 16.4.4.1 Termination_control Register (offset = 0h) [reset = 1000800h]

Register mask: FFFFFFFFh

Termination_control is shown in Figure 16-134 and described in Table 16-145.

contains bits related to control of terminations in USB2PHY

**Figure 16-134. Termination_control Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| RESERVED | | ALWAYS_UPDATE | RTERM_CAL_DONE | FS_CODE_SEL | | | |
| R/W-0h | | R/W-0h | R-0h | R/W-1h | | | |
| **23** | **22** | **21** | **20** | **19** | **18** | **17** | **16** |
| RESERVED | | USE_RTERM_RMX_REG | RTERM_RMX | | | | |
| R/W-0h | | R/W-0h | R/W-0h | | | | |
| **15** | **14** | **13** | **12** | **11** | **10** | **9** | **8** |
| RTERM_RMX | | HS_CODE_SEL | | | RTERM_COMP_OUT | RESTART_RTERM_CAL | DISABLE_TEMP_TRACK |
| R/W-0h | | R/W-1h | | | R-0h | R/W-0h | R/W-0h |
| **7** | **6** | **5** | **4** | **3** | **2** | **1** | **0** |
| USE_RTERM_CAL_REG | RTERM_CAL | | | | | | |
| R/W-0h | R/W-0h | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-145. Termination_control Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-30 | RESERVED | R/W | 0h | |
| 29 | ALWAYS_UPDATE | R/W | 0h | When set to 1 , the calibration code is updated immediately after a code computation without waiting for idle periods. |
| 28 | RTERM_CAL_DONE | R | 0h | Rterm calibration is done.<br>1st time cal is done this bit gets set and gets reset at a restart cal.Read value is valid only if VDDLDO is on. |
| 27-24 | FS_CODE_SEL | R/W | 1h | FS Code selection control |
| 23-22 | RESERVED | R/W | 0h | |
| 21 | USE_RTERM_RMX_REG | R/W | 0h | Override termination resistor trim code with RTERM_RMX from this register |
| 20-14 | RTERM_RMX | R/W | 0h | When read, this field returns the current Termination resistor trim code.Read value is valid only if VDDLDO is on.<br>The value written to this field is used as Termination resistor trim code if bit 21 is set to 1 |
| 13-11 | HS_CODE_SEL | R/W | 1h | HS Code selection control.<br>Each increment increases the termination impedance by ~1.5%.<br>Valid values are 000b 011b. |
| 10 | RTERM_COMP_OUT | R | 0h | Master loop comparator output.<br>Read value is valid only if VDDLDO is on. |
| 9 | RESTART_RTERM_CAL | R/W | 0h | Restart the rterm calibration.<br>The calibration restarts on any toggle 0 to 1 or 1 to 0 on this bit. |
| 8 | DISABLE_TEMP_TRACK | R/W | 0h | Disables the temperature tracking function of the termination calibration |
| 7 | USE_RTERM_CAL_REG | R/W | 0h | when '1' the rterm cal code is overridden by values in RTERM_CAL |
| 6-0 | RTERM_CAL | R/W | 0h | When read this field returns the current rterm calibration code.<br>Read value is valid only if VDDLDO is on.<br>The value written to this field is used as rterm calibration code if the bit USE_RTERM_CAL_REG is 1. |

### 16.4.4.2 RX_CALIB Register (offset = 4h) [reset = 0h]

Register mask: FFFFFFFFh

RX_CALIB is shown in Figure 16-135 and described in Table 16-146.

Contains bits related to RX calibration

**Figure 16-135. RX_CALIB Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| RESTART_HSRX_CAL | USE_HS_OFF_REG | HS_OFF_CODE | | | | | |
| R/W-0h | R/W-0h | R/W-0h | | | | | |
| **23** | **22** | **21** | **20** | **19** | **18** | **17** | **16** |
| HSRX_COMP_OUT | HSRX_CAL_DONE | USE_SQ_OFF_DAC1 | SQ_OFF_CODE_DAC1 | | | | |
| R-0h | R-0h | R/W-0h | R/W-0h | | | | |
| **15** | **14** | **13** | **12** | **11** | **10** | **9** | **8** |
| SQ_OFF_CODE_DAC1 | USE_SQ_OFF_DAC2 | SQ_OFF_CODE_DAC2 | | | | | USE_SQ_OFF_DAC3 |
| R/W-0h | R/W-0h | R/W-0h | | | | | R/W-0h |
| **7** | **6** | **5** | **4** | **3** | **2** | **1** | **0** |
| SQ_OFF_CODE_DAC3 | | | | | SQ_COMP_OUT | SQ_CAL_DONE | RESTART_SQ_CAL |
| R/W-0h | | | | | R-0h | R-0h | R/W-0h |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-146. RX_CALIB Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31 | RESTART_HSRX_CAL | R/W | 0h | Restart the HSRX calibration state machine when this bit goes from 0 to 1. |
| 30 | USE_HS_OFF_REG | R/W | 0h | Override HS offset correction with HS_OFF_CODE when set to '1' |
| 29-24 | HS_OFF_CODE | R/W | 0h | HS offset code, this code is forced when bit 30 is 1 .<br>Code is updated from calibration logic when bit<br>30 = 0. |
| 23 | HSRX_COMP_OUT | R | 0h | The output of the HSRX comparator.Read value is valid only if VDDLDO is on. |
| 22 | HSRX_CAL_DONE | R | 0h | Signal that indicates that the HSRX calibration is done.<br>This gets reset at every restart.Read value is valid only if VDDLDO is on. |
| 21 | USE_SQ_OFF_DAC1 | R/W | 0h | Override Squelch offset DAC1 code when '1' |
| 20-15 | SQ_OFF_CODE_DAC1 | R/W | 0h | When read returns current Sq offset code for DAC1, if VDDLDO is on.<br>When written this is used as Sq offset code for DAC1 when USE_SQ_OFF_DAC<br>1 = '1' |
| 14 | USE_SQ_OFF_DAC2 | R/W | 0h | Override Squelch offset DAC2 code when '1' |
| 13-9 | SQ_OFF_CODE_DAC2 | R/W | 0h | When read returns current Sq offset code for DAC2, if VDDLDO is on.<br>When written this is used as Sq offset code for DAC2 when USE_SQ_OFF_DAC<br>2 = '1' |
| 8 | USE_SQ_OFF_DAC3 | R/W | 0h | Override Squelch offset DAC3 code when '1' |
| 7-3 | SQ_OFF_CODE_DAC3 | R/W | 0h | When read returns current Sq offset code for DAC3, if VDDLDO is on.<br>When written this is used as Sq offset code for DAC3 when USE_SQ_OFF_DAC<br>3 = '1' |

**Table 16-146. RX_CALIB Register Field Descriptions (continued)**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 2 | SQ_COMP_OUT | R | 0h | Sq comp output.Read value is valid only if VDDLDO is on. |
| 1 | SQ_CAL_DONE | R | 0h | Sq calibration is done when this bit = 1.<br>see RESTART_SQ_CAL for more description.Read value is valid only if VDDLDO is on. |
| 0 | RESTART_SQ_CAL | R/W | 0h | the squelch calibration continuously goes through restart cycles when this bit is 1 .<br>i.e.<br>restarts waits for done then restarts again etc. |

### 16.4.4.3 DLLHS_2 Register (offset = 8h) [reset = 1Fh]

Register mask: FFFFFFFFh

DLLHS_2 is shown in Figure 16-136 and described in Table 16-147.

the 2nd DLLHS control register. bits 4:0 are unrelated to the DLLHS and are linestate filter settings

**Figure 16-136. DLLHS_2 Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| DLLHS_CNTRL_LDO | | | | | | | |
| R/W-0h | | | | | | | |
| **23** | **22** | **21** | **20** | **19** | **18** | **17** | **16** |
| DLLHS_STATUS_LDO | | | | | | | |
| R-0h | | | | | | | |
| **15** | **14** | **13** | **12** | **11** | **10** | **9** | **8** |
| RESERVED | | | | | | | |
| R/W-0h | | | | | | | |
| **7** | **6** | **5** | **4** | **3** | **2** | **1** | **0** |
| RESERVED | | | LINESTATE_DEBOUNCE_EN | LINESTATE_DEBOUNCE_CNTL | | | |
| R/W-0h | | | R/W-1h | R/W-Fh | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-147. DLLHS_2 Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-24 | DLLHS_CNTRL_LDO | R/W | 0h | See DFT spec for details |
| 23-16 | DLLHS_STATUS_LDO | R | 0h | See DFT spec for details |
| 15-5 | RESERVED | R/W | 0h | |
| 4 | LINESTATE_DEBOUNCE_EN | R/W | 1h | Enables the linestate debounce filter |
| 3-0 | LINESTATE_DEBOUNCE_CNTL | R/W | Fh | Used for control of the linestate debounce filter when going from syncronous to async linestate. |

#### 16.4.4.4 RX_TEST_2 Register (offset = Ch) [reset = 0h]

Register mask: FFFFFFFFh

RX_TEST_2 is shown in Figure 16-137 and described in Table 16-148.

the 2nd receiver test register

**Figure 16-137. RX_TEST_2 Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| HSOSREVERSAL | HSOSBITINVERSION | PHYCLKOUTINVERSION | RXPIDERR | USEINTDATAOUT | INTDATAOUTREG | | |
| R/W-0h | R/W-0h | R/W-0h | R-0h | R/W-0h | R/W-0h | | |
| **23** | **22** | **21** | **20** | **19** | **18** | **17** | **16** |
| INTDATAOUTREG | | | | | | | |
| R/W-0h | | | | | | | |
| **15** | **14** | **13** | **12** | **11** | **10** | **9** | **8** |
| INTDATAOUTREG | | | | | RESERVED | | |
| R/W-0h | | | | | R/W-0h | | |
| **7** | **6** | **5** | **4** | **3** | **2** | **1** | **0** |
| CDR_TESTOUT | | | | | | | |
| R-0h | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-148. RX_TEST_2 Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31 | HSOSREVERSAL | R/W | 0h | Swaps the dataout from HSOS |
| 30 | HSOSBITINVERSION | R/W | 0h | Inverts the HSOS bits |
| 29 | PHYCLKOUTINVERSION | R/W | 0h | This inverts the phase for the PHYCLKOUT |
| 28 | RXPIDERR | R | 0h | Flags if the RX data packet has PID error.<br>NOT IMPLEMENTED YET |
| 27 | USEINTDATAOUT | R/W | 0h | This will bypass the analog and will send data packet to controller incase of receiver (Faking the receive data).<br>data used will be INTDATAOUTREG |
| 26-11 | INTDATAOUTREG | R/W | 0h | This register will be loaded through OCP and this data will be given to the controller if USEINTDATAOUT is set to 1 |
| 10-8 | RESERVED | R/W | 0h | |
| 7-0 | CDR_TESTOUT | R | 0h | CDR debug bits.<br>Read value is valid only if VDDLDO is on.<br>see DFT spec for details |

### 16.4.4.5 CHRG_DET Register (offset = 14h) [reset = 0h]

Register mask: FFFFFFFFh

CHRG_DET is shown in Figure 16-138 and described in Table 16-149.

this is the charger detect register. this register is not used in the dead battery case.

**Figure 16-138. CHRG_DET Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| RESERVED | | USE_CHG_DET_REG | DIS_CHG_DET | SRC_ON_DM | SINK_ON_DP | CHG_DET_EXT_CTL | RESTART_CHG_DET |
| R/W-0h | | R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h |
| **23** | **22** | **21** | **20** | **19** | **18** | **17** | **16** |
| CHG_DET_DONE | CHG_DETECTED | DATA_DET | RESERVED | | CHG_ISINK_EN | CHG_VSRC_EN | COMP_DP |
| R-0h | R-0h | R-0h | R/W-0h | | R/W-0h | R/W-0h | R-0h |
| **15** | **14** | **13** | **12** | **11** | **10** | **9** | **8** |
| COMP_DM | CHG_DET_OSC_CNTRL | | CHG_DET_TIMER | | | | |
| R-0h | R/W-0h | | R/W-0h | | | | |
| **7** | **6** | **5** | **4** | **3** | **2** | **1** | **0** |
| CHG_DET_TIMER | RESERVED | | CHG_DET_ICTRL | | CHG_DET_VCTRL | | FOR_CE |
| R/W-0h | R/W-0h | | R/W-0h | | R/W-0h | | R/W-0h |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-149. CHRG_DET Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-30 | RESERVED | R/W | 0h | |
| 29 | USE_CHG_DET_REG | R/W | 0h | Use bits<br>28-24 and<br>18-17 from this register |
| 28 | DIS_CHG_DET | R/W | 0h | When read, returns current value of charger detect input.<br>When USE_CHG_DET_REG = 1 , the value written to this field overrides the corresponding charger detect input. |
| 27 | SRC_ON_DM | R/W | 0h | When read, returns current value of charger detect input.<br>When USE_CHG_DET_REG = 1 , the value written to this field overrides the corresponding charger detect input. |
| 26 | SINK_ON_DP | R/W | 0h | When read, returns current value of charger detect input.<br>When USE_CHG_DET_REG = 1 , the value written to this field overrides the corresponding charger detect input. |
| 25 | CHG_DET_EXT_CTL | R/W | 0h | When read, returns current value of charger detect input.<br>When USE_CHG_DET_REG = 1 , the value written to this field overrides the corresponding charger detect input. |
| 24 | RESTART_CHG_DET | R/W | 0h | Restart the charger detection protocol when this goes from 0 to 1 |
| 23 | CHG_DET_DONE | R | 0h | Charger detect protocol has completed |
| 22 | CHG_DETECTED | R | 0h | Same signal as CE |
| 21 | DATA_DET | R | 0h | Output of the data det comparator |
| 20-19 | RESERVED | R/W | 0h | |
| 18 | CHG_ISINK_EN | R/W | 0h | When read, returns current value of charger detect input.<br>When USE_CHG_DET_REG = 1 , the value written to this field overrides the corresponding charger detect input. |
| 17 | CHG_VSRC_EN | R/W | 0h | When read, returns current value of charger detect input.<br>When USE_CHG_DET_REG = 1 , the value written to this field overrides the corresponding charger detect input. |
| 16 | COMP_DP | R | 0h | Comparator on the DP line value |
| 15 | COMP_DM | R | 0h | Comparator on the DM line value |

**Table 16-149. CHRG_DET Register Field Descriptions (continued)**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 14-13 | CHG_DET_OSC_CNTRL | R/W | 0h | Charger detect osc control |
| 12-7 | CHG_DET_TIMER | R/W | 0h | Charger detect timer control.<br>See charger detect section for details |
| 6-5 | RESERVED | R/W | 0h | |
| 4-3 | CHG_DET_ICTRL | R/W | 0h | Charger detect current control |
| 2-1 | CHG_DET_VCTRL | R/W | 0h | Charger detect voltage buffer control |
| 0 | FOR_CE | R/W | 0h | Force CE = 1 when this bit is set |

### 16.4.4.6 PWR_CNTL Register (offset = 18h) [reset = 400000h]

Register mask: FFFFFFFFh

PWR_CNTL is shown in Figure 16-139 and described in Table 16-150.

has all the power control bits

**Figure 16-139. PWR_CNTL Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| RESETDONETCLK | RESET_DONE_VMAIN | VMAIN_GLOBAL_RESET_DONE | RESETDONEMCLK | RESETDONE_CHGDET | LDOPWRCOUNTER | | |
| R-0h | R-0h | R-0h | R-0h | R-0h | R/W-400h | | |
| **23** | **22** | **21** | **20** | **19** | **18** | **17** | **16** |
| LDOPWRCOUNTER | | | | | | | |
| R/W-400h | | | | | | | |
| **15** | **14** | **13** | **12** | **11** | **10** | **9** | **8** |
| LDOPWRCOUNTER | | | | FORCEPLLSLOWCLK | FORCELDOON | FORCEPLLON | RESERVED |
| R/W-400h | | | | R/W-0h | R/W-0h | R/W-0h | R/W-0h |
| **7** | **6** | **5** | **4** | **3** | **2** | **1** | **0** |
| RESERVED | PLLLOCK | USEPLLLOCK | USE_DATAPOLARITYN_REG | DATAPOLARITYN | USE_PD_REG | PD | RESERVED |
| R/W-0h | R-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-150. PWR_CNTL Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31 | RESETDONETCLK | R | 0h | Goes high when the RESET is synchronized to TCLK |
| 30 | RESET_DONE_VMAIN | R | 0h | Goes high when LDO domain is up and PLL LOCK is available and utmi_reset is de-asserted. |
| 29 | VMAIN_GLOBAL_RESET_DONE | R | 0h | Goes high when LDO domain is up and PLL LOCK is available. |
| 28 | RESETDONEMCLK | R | 0h | Goes high when the RESET is synchronized to MCLK |
| 27 | RESETDONE_CHGDET | R | 0h | Goes high when the RESET is synchronized to charger detect oscillator clock domain |
| 26-12 | LDOPWRCOUNTER | R/W | 400h | This is the value of the counter used for LDO power up.<br>RESET to default. |
| 11 | FORCEPLLSLOWCLK | R/W | 0h | Forces the PLL to the slow clk mode |
| 10 | FORCELDOON | R/W | 0h | Forces the LDO to be ON. |
| 9 | FORCEPLLON | R/W | 0h | Forces the PLL to be ON. |
| 8-7 | RESERVED | R/W | 0h | |
| 6 | PLLLOCK | R | 0h | Lock signal from the PLL |
| 5 | USEPLLLOCK | R/W | 0h | This signal is used to indicate to the Phy, not to do any clock related activity until PLLLOCK = 1 .This is not the default option.0 do not use PLLLOCK.<br>1 use PLLLOCK as a clock gate. |
| 4 | USE_DATAPOLARITYN_REG | R/W | 0h | 1 -> use bit 3 as override for the DATAPOLARITYN signal. |
| 3 | DATAPOLARITYN | R/W | 0h | Override value of datapolarityn |
| 2 | USE_PD_REG | R/W | 0h | Use bit 1 from this register as PD override when set to '1' |
| 1 | PD | R/W | 0h | Override value for PD |
| 0 | RESERVED | R/W | 0h | |

#### 16.4.4.7 UTMI_INTERFACE_CNTL_1 Register (offset = 1Ch) [reset = 0h]

Register mask: FFFFFFFFh

UTMI_INTERFACE_CNTL_1 is shown in Figure 16-140 and described in Table 16-151.

register to override UTMI interface control pins.

**Figure 16-140. UTMI_INTERFACE_CNTL_1 Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| USEUTMIDATAREG | UTMIDATAIN | | | | | | |
| R/W-0h | R/W-0h | | | | | | |
| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
| UTMIDATAIN | | | | | | | |
| R/W-0h | | | | | | | |
| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
| UTMIDATAIN | RESERVED | USEDATABUSREG | DATABUS16OR8 | USEOPMODEREG | OPMODE | | OVERRIDESUSRESET |
| R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h | | R/W-0h |
| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
| SUSPENDM | UTMIRESET | OVERRIDEXCVRSEL | XCVRSEL | | USETXVALIDREG | TXVALID | TXVALIDH |
| R/W-0h | R/W-0h | R/W-0h | R/W-0h | | R/W-0h | R/W-0h | R/W-0h |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-151. UTMI_INTERFACE_CNTL_1 Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31 | USEUTMIDATAREG | R/W | 0h | Use datain from UTMI interface register |
| 30-15 | UTMIDATAIN | R/W | 0h | Override value for the UTMIDATAIN |
| 14 | RESERVED | R/W | 0h | |
| 13 | USEDATABUSREG | R/W | 0h | When set to 1 use bit 12 from register instead of interface |
| 12 | DATABUS16OR8 | R/W | 0h | Override value for UTMI signal DATABUS16OR8 |
| 11 | USEOPMODEREG | R/W | 0h | When set to 1 use bit 10-9 from register instead of interface |
| 10-9 | OPMODE | R/W | 0h | Override value for UTMI signal OPMODE [1:0] |
| 8 | OVERRIDESUSRESET | R/W | 0h | Override the suspend and reset values. Use bits 6 and 7 |
| 7 | SUSPENDM | R/W | 0h | Override value for UTMI signal SUSPENDM |
| 6 | UTMIRESET | R/W | 0h | Override value for UTMI signal UTMIRESET |
| 5 | OVERRIDEXCVRSEL | R/W | 0h | When set to 1 use bit 4-3 from register instead of interface |
| 4-3 | XCVRSEL | R/W | 0h | Override value for UTMI signal XCVRSEL [1:0] |
| 2 | USETXVALIDREG | R/W | 0h | When set to 1 use bit 1-0 from register instead of interface |
| 1 | TXVALID | R/W | 0h | Override value for UTMI signal TXVALID |
| 0 | TXVALIDH | R/W | 0h | Override value for UTMI signal TXVALIDH |

### 16.4.4.8 UTMI_INTERFACE_CNTL_2 Register (offset = 20h) [reset = 0h]

Register mask: FFFFFFFFh

UTMI_INTERFACE_CNTL_2 is shown in Figure 16-141 and described in Table 16-152.

UTMI interface override and observe register 2

**Figure 16-141. UTMI_INTERFACE_CNTL_2 Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| RXRCV | RXDP | RXDM | HOSTDISCONNECT | LINESTATE | | RXVALID | RXVALIDH |
| R-0h | R-0h | R-0h | R-0h | R-0h | | R-0h | R-0h |
| **23** | **22** | **21** | **20** | **19** | **18** | **17** | **16** |
| RXACTIVE | RXERROR | TXREADY | UTMIRESETDONE | USEBITSTUFFREG | TXBITSTUFFENABLE | TXBITSTUFFENABLEH | USETERMCONTROLREG |
| R-0h | R-0h | R-0h | R-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h |
| **15** | **14** | **13** | **12** | **11** | **10** | **9** | **8** |
| TERMSEL | DPPULLDOWN | DMPULLDOWN | RESERVED | | | USEREGSERIALMODE | TXSE0 |
| R/W-0h | R/W-0h | R/W-0h | R/W-0h | | | R/W-0h | R/W-0h |
| **7** | **6** | **5** | **4** | **3** | **2** | **1** | **0** |
| TXDAT | FSLSSERIALMODE | TXENABLEN | RESERVED | | | | sig_bypass_suspendmpulse_incr |
| R/W-0h | R/W-0h | R/W-0h | R/W-0h | | | | R/W-0h |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-152. UTMI_INTERFACE_CNTL_2 Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31 | RXRCV | R | 0h | Read for UTMI signal.Read value is valid only if VDDLDO is on.Read value is valid only if VDDLDO is on. |
| 30 | RXDP | R | 0h | Read for UTMI signal.Read value is valid only if VDDLDO is on. |
| 29 | RXDM | R | 0h | Read for UTMI signal.Read value is valid only if VDDLDO is on. |
| 28 | HOSTDISCONNECT | R | 0h | Read for UTMI signal.Read value is valid only if VDDLDO is on. |
| 27-26 | LINESTATE | R | 0h | Read for UTMI signal.Read value is valid only if VDDLDO is on. |
| 25 | RXVALID | R | 0h | Read for UTMI signal.Read value is valid only if VDDLDO is on. |
| 24 | RXVALIDH | R | 0h | Read for UTMI signal.Read value is valid only if VDDLDO is on. |
| 23 | RXACTIVE | R | 0h | Read for UTMI signal.Read value is valid only if VDDLDO is on. |
| 22 | RXERROR | R | 0h | Read for UTMI signal.Read value is valid only if VDDLDO is on. |
| 21 | TXREADY | R | 0h | Read for UTMI signal.Read value is valid only if VDDLDO is on. |
| 20 | UTMIRESETDONE | R | 0h | Read for UTMIRESETDONE signal |
| 19 | USEBITSTUFFREG | R/W | 0h | When set to 1 use bits 18-17 from register instead of interface |
| 18 | TXBITSTUFFENABLE | R/W | 0h | Override value for signal TXBITSTUFFENABLE |
| 17 | TXBITSTUFFENABLEH | R/W | 0h | Override value for pin TXBITSTUFFENABLE |
| 16 | USETERMCONTROLREG | R/W | 0h | -When set to 1 use bits 15-13 from register instead of interface |
| 15 | TERMSEL | R/W | 0h | Override value for signal TERMSEL |
| 14 | DPPULLDOWN | R/W | 0h | Override value for signal DPPULLDOWN |
| 13 | DMPULLDOWN | R/W | 0h | Override value for signal DMPULLDOWN |
| 12-10 | RESERVED | R/W | 0h | |

**Table 16-152. UTMI_INTERFACE_CNTL_2 Register Field Descriptions (continued)**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 9 | USEREGSERIALMODE | R/W | 0h | When set to 1 use bits<br>8-5 from register instead of interface |
| 8 | TXSE0 | R/W | 0h | Override value for signal TXSE0 |
| 7 | TXDAT | R/W | 0h | Override value for signal TXDAT |
| 6 | FSLSSERIALMODE | R/W | 0h | Override value for signal FSLSSERIALMODE |
| 5 | TXENABLEN | R/W | 0h | Override value for signal TXENABLEN |
| 4-1 | RESERVED | R/W | 0h | |
| 0 | sig_bypass_suspendmpulse_incr | R/W | 0h | If the suspend signal is asserted for very short-time, it is pulse extended so that all the sampling logic samples it reliably.<br>This pulse extention can be bypassed by writin a '1' to this bit ( so that IP's behaviour is similar to previous versions) |

### 16.4.4.9 BIST Register (offset = 24h) [reset = 0h]

Register mask: FFFFFFFFh

BIST is shown in Figure 16-142 and described in Table 16-153.

COntains bits related to the built in self test of the phy

**Figure 16-142. BIST Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| BIST_START | REDUCED_SWING | BIST_CRC_CALC_EN | BIST_PKT_LENGTH | | | | |
| R/W-0h | R/W-0h | R/W-0h | R/W-0h | | | | |
| **23** | **22** | **21** | **20** | **19** | **18** | **17** | **16** |
| BIST_PKT_LENGTH | | | | LOOPBACK_EN | BIST_OP_PHASE_SEL | | |
| R/W-0h | | | | R/W-0h | R/W-0h | | |
| **15** | **14** | **13** | **12** | **11** | **10** | **9** | **8** |
| SWEEP_EN | SWEEP_MODE | | | BIST_PASS | BIST_BUSY | RESERVED | |
| R/W-0h | R/W-0h | | | R-0h | R-0h | R/W-0h | |
| **7** | **6** | **5** | **4** | **3** | **2** | **1** | **0** |
| RESERVED | OP_CODE | | RX_TEST_MODE | RESERVED | INTER_PKT_DELAY_TEST | HS_ALL_ONES_TEST | USE_BIST_TX_PHASES |
| R/W-0h | R/W-0h | | R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-153. BIST Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31 | BIST_START | R/W | 0h | When set to 1 the BIST mode is started. |
| 30 | REDUCED_SWING | R/W | 0h | When 1 the TX swing is reduced in BIST mode |
| 29 | BIST_CRC_CALC_EN | R/W | 0h | Enables CRC calculation during BIST when set to 1 |
| 28-20 | BIST_PKT_LENGTH | R/W | 0h | Address for which BIST to select |
| 19 | LOOPBACK_EN | R/W | 0h | Enables the loopback mode |
| 18-16 | BIST_OP_PHASE_SEL | R/W | 0h | Selects which phase to use for data transmission during BIST |
| 15 | SWEEP_EN | R/W | 0h | Enables freq sweep on CDR |
| 14-12 | SWEEP_MODE | R/W | 0h | Selects the freq sweep mode.<br>Details in DFT spec. |
| 11 | BIST_PASS | R | 0h | Indicates that the BIST has passed.<br>Read value is valid only if VDDLDO is on. |
| 10 | BIST_BUSY | R | 0h | Indicates that BIST is running.<br>Read value is valid only if VDDLDO is on. |
| 9-7 | RESERVED | R/W | 0h | |
| 6-5 | OP_CODE | R/W | 0h | Defined in DFT spec |
| 4 | RX_TEST_MODE | R/W | 0h | Defined in DFT spec |
| 3 | RESERVED | R/W | 0h | |
| 2 | INTER_PKT_DELAY_TEST | R/W | 0h | Defined in DFT spec |
| 1 | HS_ALL_ONES_TEST | R/W | 0h | Defined in DFT spec |
| 0 | USE_BIST_TX_PHASES | R/W | 0h | When set to 1 bits<br>18-16 are activated for choosing the transmitting phase. |

#### 16.4.4.10 BIST_CRC Register (offset = 28h) [reset = 0h]

Register mask: FFFFFFFFh

BIST_CRC is shown in Figure 16-143 and described in Table 16-154.

the CRC code for BIST test

**Figure 16-143. BIST_CRC Register**

| 31–0 |
|---|
| BIST_CRC |
| R/W-0h |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-154. BIST_CRC Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-0 | BIST_CRC | R/W | 0h | The CRC value from the BIST. |

### 16.4.4.11 CDR_BIST2 Register (offset = 2Ch) [reset = 0h]

Register mask: FFFFFFFFh

CDR_BIST2 is shown in Figure 16-144 and described in Table 16-155.

clock data recovery register and BIST register 2

**Figure 16-144. CDR_BIST2 Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| CDR_EXE_EN | CDR_EXE_MODE | | | NUM_DECISIONS | | | CDR_CHOSEN_PHASE |
| R/W-0h | R/W-0h | | | R/W-0h | | | R-0h |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| CDR_CHOSEN_PHASE | | FORCE_CDR_PHASE | | | DISABLE_CDR_FREQ_TRACK | CDR_CONFIGURE | |
| R-0h | | R-0h | | | R-0h | R-0h | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| CDR_CONFIGURE | | | FORCE_CDR_PHASE_EN | Bist_start_addr | | | |
| R-0h | | | R-0h | R/W-0h | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| Bist_start_addr | | Bist_end_addr | | | | | |
| R/W-0h | | R/W-0h | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-155. CDR_BIST2 Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31 | CDR_EXE_EN | R/W | 0h | CDR debug bits |
| 30-28 | CDR_EXE_MODE | R/W | 0h | CDR debug bits |
| 27-25 | NUM_DECISIONS | R/W | 0h | CDR debug bits |
| 24-22 | CDR_CHOSEN_PHASE | R | 0h | |
| 21-19 | FORCE_CDR_PHASE | R | 0h | |
| 18 | DISABLE_CDR_FREQ_TRACK | R | 0h | |
| 17-13 | CDR_CONFIGURE | R | 0h | |
| 12 | FORCE_CDR_PHASE_EN | R | 0h | Use bits<br>21-19 as the phase to be forced on the CDR. |
| 11-6 | Bist_start_addr | R/W | 0h | See DFT spec for details |
| 5-0 | Bist_end_addr | R/W | 0h | See DFT spec for details |

#### 16.4.4.12 GPIO Register (offset = 30h) [reset = 0h]

Register mask: FFFFFFFFh

GPIO is shown in Figure 16-145 and described in Table 16-156.

GPIO mode configurations and reads

**Figure 16-145. GPIO Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| USEGPIOMODEREG | GPIOMODE | DPGPIOGZ | DMGPIOGZ | DPGPIOA | DMGPIOA | DPGPIOY | DMGPIOY |
| R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h | R-0h | R-0h |
| **23** | **22** | **21** | **20** | **19** | **18** | **17** | **16** |
| GPIO1P8VCONFIG | GPIOCONFIG | | | DMGPIOPIPD | DPGPIOPIPD | RESERVED | |
| R/W-0h | R/W-0h | | | R/W-0h | R/W-0h | R/W-0h | |
| **15** | **14** | **13** | **12** | **11** | **10** | **9** | **8** |
| RESERVED | | | | | | | |
| R/W-0h | | | | | | | |
| **7** | **6** | **5** | **4** | **3** | **2** | **1** | **0** |
| RESERVED | | | | | | | |
| R/W-0h | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-156. GPIO Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31 | USEGPIOMODEREG | R/W | 0h | When set to 1 use bits 31 24 from this register instead of primary inputs |
| 30 | GPIOMODE | R/W | 0h | Overrides the corresponding primary input |
| 29 | DPGPIOGZ | R/W | 0h | Overrides the corresponding primary input |
| 28 | DMGPIOGZ | R/W | 0h | Overrides the corresponding primary input |
| 27 | DPGPIOA | R/W | 0h | Overrides the corresponding primary input |
| 26 | DMGPIOA | R/W | 0h | Overrides the corresponding primary input |
| 25 | DPGPIOY | R | 0h | The GPIO Y output is stored here |
| 24 | DMGPIOY | R | 0h | The GPIO Y output is stored here |
| 23 | GPIO1P8VCONFIG | R/W | 0h | Overrides the corresponding primary input |
| 22-20 | GPIOCONFIG | R/W | 0h | Used for configuring the GPIOs.<br>Details to be updated. |
| 19 | DMGPIOPIPD | R/W | 0h | GPIO mode DM pull down enabled.<br>Overrides the corresponding primary input |
| 18 | DPGPIOPIPD | R/W | 0h | GPIO mode DP pull-down enabled.<br>Overrides the corresponding primary input. |
| 17-0 | RESERVED | R/W | 0h | |

### 16.4.4.13 DLLHS Register (offset = 34h) [reset = 8000h]

Register mask: FFFFFFFFh

DLLHS is shown in Figure 16-146 and described in Table 16-157.

bits for control and debug of the DLL inside the phy

**Figure 16-146. DLLHS Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| RESERVED | | | DLLHS_LOCK | DLLHS_GENERATED_CODE | | | |
| R/W-0h | | | R-0h | R-0h | | | |
| **23** | **22** | **21** | **20** | **19** | **18** | **17** | **16** |
| DLLHS_GENERATED_CODE | | DLL_SEL_CODE_PHS | DLL_LOCKCHK | | DLL_SEL_COD | | |
| R-0h | | R/W-0h | R/W-0h | | R/W-0h | | |
| **15** | **14** | **13** | **12** | **11** | **10** | **9** | **8** |
| DLL_PHS0_8 | DLL_FORCED_CODE | | | | | | FORCE_DLL_CODE |
| R/W-1h | R/W-0h | | | | | | R/W-0h |
| **7** | **6** | **5** | **4** | **3** | **2** | **1** | **0** |
| DLL_RATE | | DLL_FILT | | DLL_CDR_MODE | DLL_IDLE | DLL_FREEZE | RESERVED |
| R/W-0h | | R/W-0h | | R/W-0h | R/W-0h | R/W-0h | R/W-0h |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-157. DLLHS Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-29 | RESERVED | R/W | 0h | |
| 28 | DLLHS_LOCK | R | 0h | Read the AFE output by this name |
| 27-22 | DLLHS_GENERATED_CODE | R | 0h | Read the AFE output by this name.Read value is valid only if VDDLDO is on. |
| 21 | DLL_SEL_CODE_PHS | R/W | 0h | Connect to DLLHS_TEST_LDO[0] on AFE interface.<br>see DFT spec for details. |
| 20-19 | DLL_LOCKCHK | R/W | 0h | Connect to DLLHS_TEST_LDO<br>[2:1] on AFE interface.<br>see DFT spec for details. |
| 18-16 | DLL_SEL_COD | R/W | 0h | Connect to DLLHS_TEST_LDO<br>[5:3] on AFE interface.<br>see DFT spec for details. |
| 15 | DLL_PHS0_8 | R/W | 1h | Connect to DLLHS_TEST_LDO[6] on AFE interface.<br>see DFT spec for details. |
| 14-9 | DLL_FORCED_CODE | R/W | 0h | Connect to the pin of this name on AFE interface.<br>see DFT spec for details. |
| 8 | FORCE_DLL_CODE | R/W | 0h | Connect to DLLHS_TEST_LDO[11] on AFE interface.<br>see DFT spec for details. |
| 7-6 | DLL_RATE | R/W | 0h | Connect to DLLHS_TEST_LDO<br>[8:7] on AFE interface.<br>see DFT spec for details. |
| 5-4 | DLL_FILT | R/W | 0h | Connect to DLLHS_TEST_LDO<br>[10:9] on AFE interface.<br>see DFT spec for details. |
| 3 | DLL_CDR_MODE | R/W | 0h | Connect to the pin of this name on AFE interface.<br>see DFT spec for details. |
| 2 | DLL_IDLE | R/W | 0h | Connect to DLLHS_TEST_LDO[12] on AFE interface.<br>see DFT spec for details. |
| 1 | DLL_FREEZE | R/W | 0h | Connect to DLLHS_TEST_LDO[13] on AFE interface.<br>see DFT spec for details. |

**Table 16-157. DLLHS Register Field Descriptions (continued)**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 0 | RESERVED | R/W | 0h | |

### 16.4.4.14 USB2PHYCM_CONFIG Register (offset = 3Ch) [reset = 0h]

Register mask: FFFFFFFFh

USB2PHYCM_CONFIG is shown in Figure 16-147 and described in Table 16-158.

Config and status register for the USB2PHYCM and LDO

**Figure 16-147. USB2PHYCM_CONFIG Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| CONFIGURECM | | | | | | | |
| R/W-0h | | | | | | | |
| **23** | **22** | **21** | **20** | **19** | **18** | **17** | **16** |
| CMSTATUS | | | | | | LDOCONFIG | |
| R-0h | | | | | | R/W-0h | |
| **15** | **14** | **13** | **12** | **11** | **10** | **9** | **8** |
| LDOCONFIG | | | | | | | |
| R/W-0h | | | | | | | |
| **7** | **6** | **5** | **4** | **3** | **2** | **1** | **0** |
| LDOCONFIG | | | | | | LDOSTATUS | |
| R/W-0h | | | | | | R-0h | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-158. USB2PHYCM_CONFIG Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-24 | CONFIGURECM | R/W | 0h | Connects to the CONFIGURECM pins.<br>see DFT spec for details. |
| 23-18 | CMSTATUS | R | 0h | Reads the CMSTATUS bits.<br>see DFT spec for details. |
| 17-2 | LDOCONFIG | R/W | 0h | The LDOCONFIG bit settings.<br>See DFT spec for details. |
| 1-0 | LDOSTATUS | R | 0h | Reads the LDOSTATUS bits.<br>see DFT spec for details. |

### 16.4.4.15 AD_INTERFACE_REG1 Register (offset = 44h) [reset = 0h]

Register mask: FFFFFFFFh

AD_INTERFACE_REG1 is shown in Figure 16-148 and described in Table 16-159.

All bits (unless defined) are bypass bits for internal analog to digital interface pins with the same name. All the bits of this register, except the over-ride bits return a '0' on read, if VDDLDO is off.

**Figure 16-148. AD_INTERFACE_REG1 Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| USE_AD_DATA_REG | HS_TX_DATA | FS_TX_DATA | TEST_PRE_EN_CNTRL | SQ_PRE_EN | HS_TX_PRE_EN | HS_RX_PRE_EN | TEST_EN_CNTRL |
| R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h |
| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
| HS_TX_EN | FS_RX_EN | RESERVED | SQ_EN | HS_RX_EN | TEST_HS_MODE | HS_HV_SW | HS_CHIRP |
| R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h |
| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
| TEST_FS_MODE | FSTX_GZ | FSTX_PRE_EN | RESERVED | TEST_SQ_CAL_CONTROL | SQ_CAL_EN3 | SQ_CAL_EN1 | SQ_CAL_EN2 |
| R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h |
| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
| TEST_RTERM_CAL_CONTROL | RTERM_CAL_EN | DLL_RX_DATA | DISCON_DETECT | USE_LSHOST_REG | LSHOSTMODE | LSFS_RX_DATA | SQUELCH |
| R/W-0h | R/W-0h | R-0h | R-0h | R/W-0h | R/W-0h | R-0h | R-0h |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-159. AD_INTERFACE_REG1 Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31 | USE_AD_DATA_REG | R/W | 0h | Override for bits 30-29 |
| 30 | HS_TX_DATA | R/W | 0h | |
| 29 | FS_TX_DATA | R/W | 0h | |
| 28 | TEST_PRE_EN_CNTRL | R/W | 0h | Override for bits 27-25 |
| 27 | SQ_PRE_EN | R/W | 0h | |
| 26 | HS_TX_PRE_EN | R/W | 0h | |
| 25 | HS_RX_PRE_EN | R/W | 0h | |
| 24 | TEST_EN_CNTRL | R/W | 0h | Override for bits 23-19 |
| 23 | HS_TX_EN | R/W | 0h | |
| 22 | FS_RX_EN | R/W | 0h | |
| 21 | RESERVED | R/W | 0h | |
| 20 | SQ_EN | R/W | 0h | |
| 19 | HS_RX_EN | R/W | 0h | |
| 18 | TEST_HS_MODE | R/W | 0h | Override for bits 17-16 |
| 17 | HS_HV_SW | R/W | 0h | |
| 16 | HS_CHIRP | R/W | 0h | |
| 15 | TEST_FS_MODE | R/W | 0h | Override for bits 14 12 |
| 14 | FSTX_GZ | R/W | 0h | |
| 13 | FSTX_PRE_EN | R/W | 0h | |
| 12 | RESERVED | R/W | 0h | |

**Table 16-159. AD_INTERFACE_REG1 Register Field Descriptions (continued)**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 11 | TEST_SQ_CAL_CONTROL | R/W | 0h | Override for bits 10 8 |
| 10 | SQ_CAL_EN3 | R/W | 0h | |
| 9 | SQ_CAL_EN1 | R/W | 0h | |
| 8 | SQ_CAL_EN2 | R/W | 0h | |
| 7 | TEST_RTERM_CAL_CONTROL | R/W | 0h | Override for bits 6 |
| 6 | RTERM_CAL_EN | R/W | 0h | |
| 5 | DLL_RX_DATA | R | 0h | |
| 4 | DISCON_DETECT | R | 0h | |
| 3 | USE_LSHOST_REG | R/W | 0h | Use bit 2 for this reg |
| 2 | LSHOSTMODE | R/W | 0h | |
| 1 | LSFS_RX_DATA | R | 0h | |
| 0 | SQUELCH | R | 0h | |

### 16.4.4.16 AD_INTERFACE_REG2 Register (offset = 48h) [reset = 0h]

Register mask: FFFFFFFFh

AD_INTERFACE_REG2 is shown in Figure 16-149 and described in Table 16-160.

All bits (unless defined) are bypass bits for internal analog to digital interface pins with the same name. All the bits of this register, except the over-ride bits return a '0' on read, if VDDLDO is off.

**Figure 16-149. AD_INTERFACE_REG2 Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| USE_SUSP_DRV_REG | SUS_DRV_DP_DATA | SUS_DRV_DP_EN | SUS_DRV_DM_DATA | SUS_DRV_DM_EN | USE_DISCON_REG | DISCON_EN | DISCON_PRE_EN |
| R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| RESERVED | SPARE_OUT_CORE | | | | | SERX_DP_CORE | SERX_DM_CORE |
| R/W-0h | R-0h | | | | | R-0h | R-0h |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| USE_HSRX_CAL_EN_REG | HSRX_CAL_EN | USE_RPU_RPD_REG | RPU_DP_SW1_EN_CORE | RPU_DP_SW2_EN_CORE | RPU_DM_SW1_EN_CORE | RPU_DM_SW2_EN_CORE | DP_PULLDOWN_EN_CORE |
| R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| DM_PULLDOWN_EN_CORE | DP_DM_5V_SHORT | SPARE_IN_CORE | | | | | PORZ |
| R/W-0h | R-0h | R/W-0h | | | | | R-0h |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-160. AD_INTERFACE_REG2 Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31 | USE_SUSP_DRV_REG | R/W | 0h | Use bits 27-30 from this register as overrides |
| 30 | SUS_DRV_DP_DATA | R/W | 0h | |
| 29 | SUS_DRV_DP_EN | R/W | 0h | |
| 28 | SUS_DRV_DM_DATA | R/W | 0h | |
| 27 | SUS_DRV_DM_EN | R/W | 0h | |
| 26 | USE_DISCON_REG | R/W | 0h | Use bits 24-25 from this register as override |
| 25 | DISCON_EN | R/W | 0h | |
| 24 | DISCON_PRE_EN | R/W | 0h | |
| 23 | RESERVED | R/W | 0h | |
| 22-18 | SPARE_OUT_CORE | R | 0h | |
| 17 | SERX_DP_CORE | R | 0h | |
| 16 | SERX_DM_CORE | R | 0h | |
| 15 | USE_HSRX_CAL_EN_REG | R/W | 0h | Use bit 14 from this register as override |
| 14 | HSRX_CAL_EN | R/W | 0h | |
| 13 | USE_RPU_RPD_REG | R/W | 0h | Use override from bits 7-12 |
| 12 | RPU_DP_SW1_EN_CORE | R/W | 0h | |
| 11 | RPU_DP_SW2_EN_CORE | R/W | 0h | |
| 10 | RPU_DM_SW1_EN_CORE | R/W | 0h | |

**Table 16-160. AD_INTERFACE_REG2 Register Field Descriptions (continued)**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 9 | RPU_DM_SW2_EN_CORE | R/W | 0h | |
| 8 | DP_PULLDOWN_EN_CORE | R/W | 0h | |
| 7 | DM_PULLDOWN_EN_CORE | R/W | 0h | |
| 6 | DP_DM_5V_SHORT | R | 0h | |
| 5-1 | SPARE_IN_CORE | R/W | 0h | |
| 0 | PORZ | R | 0h | Read only bit -> the PORZ generated from the digital registered on the A-D interface. |

### 16.4.4.17 AD_INTERFACE_REG3 Register (offset = 4Ch) [reset = 0h]

Register mask: FFFFFFFFh

AD_INTERFACE_REG3 is shown in Figure 16-150 and described in Table 16-161.

All bits (unless defined) are bypass bits for internal analog to digital interface pins with the same name. All the bits of this register, except the over-ride bits return a '0' on read, if VDDLDO is off.

**Figure 16-150. AD_INTERFACE_REG3 Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| USE_HSOS_DATA_REG | HSOS_DATA | | | | | | |
| R/W-0h | R/W-0h | | | | | | |
| **23** | **22** | **21** | **20** | **19** | **18** | **17** | **16** |
| HSOS_DATA | USE_FS_REG3 | FSTX_MODE | FSTX_SE0 | USE_HS_TERM_RES_REG | HS_TERM_RES | SPARE_IN_LDO | |
| R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h | R/W-0h | |
| **15** | **14** | **13** | **12** | **11** | **10** | **9** | **8** |
| SPARE_IN_LDO | | | | | | SPARE_OUT_LDO | |
| R/W-0h | | | | | | R-0h | |
| **7** | **6** | **5** | **4** | **3** | **2** | **1** | **0** |
| SPARE_OUT_LDO | | | | | | USE_FARCORE_REG | FARCORE |
| R-0h | | | | | | R/W-0h | R/W-0h |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-161. AD_INTERFACE_REG3 Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31 | USE_HSOS_DATA_REG | R/W | 0h | Use bits<br>30-23 in this register as bypass bits |
| 30-23 | HSOS_DATA | R/W | 0h | |
| 22 | USE_FS_REG3 | R/W | 0h | Use bits<br>20-21 as bypass bits |
| 21 | FSTX_MODE | R/W | 0h | |
| 20 | FSTX_SE0 | R/W | 0h | |
| 19 | USE_HS_TERM_RES_REG | R/W | 0h | Use bit 18 as override bit |
| 18 | HS_TERM_RES | R/W | 0h | |
| 17-10 | SPARE_IN_LDO | R/W | 0h | |
| 9-2 | SPARE_OUT_LDO | R | 0h | |
| 1 | USE_FARCORE_REG | R/W | 0h | Use bit 0 from this register as bypass |
| 0 | FARCORE | R/W | 0h | |

#### 16.4.4.18 ANA_CONFIG2 Register (offset = 54h) [reset = 0h]

Register mask: FFFFFFFFh

ANA_CONFIG2 is shown in Figure 16-151 and described in Table 16-162.

Used to configure and debug the analog blocks.

**Figure 16-151. ANA_CONFIG2 Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| RESERVED | | | | | REF_GEN_TEST | | |
| R/W-0h | | | | | R/W-0h | | |
| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
| REF_GEN_TEST | | | | RESERVED | | RTERM_TEST | |
| R/W-0h | | | | R/W-0h | | R/W-0h | |
| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
| RTERM_TEST | RESERVED | | | | | | |
| R/W-0h | R/W-0h | | | | | | |
| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
| RESERVED | | | | | | | |
| R/W-0h | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-162. ANA_CONFIG2 Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-27 | RESERVED | R/W | 0h | Reserved. |
| 26-20 | REF_GEN_TEST | R/W | 0h | 0000000b - Produces the typical vertical eye diagram amplitude (default).<br>1100000b - Increases vertical eye diagram amplitude by 15 mV.<br>1010000b - Decreases vertical eye diagram amplitude by 15 mV.<br>All other values are reserved. |
| 19-18 | RESERVED | R/W | 0h | Reserved. |
| 17-15 | RTERM_TEST | R/W | 0h | 000b - Typical termination impedance (default).<br>011b - Decreases the termination impedance by 2 to 3% and can be used to increase the vertical eye diagram amplitude by 1 to 1.5%.<br>All other values reserved. |
| 14-0 | RESERVED | R/W | 0h | Reserved. |

### *16.4.5 CPPI_DMA Registers*

Table 16-163 lists the memory-mapped registers for the CPPI_DMA. All register offset addresses not listed in Table 16-163 should be considered as reserved locations and the register contents should not be modified.

**Table 16-163. CPPI_DMA REGISTERS**

| Offset | Acronym | Register Name | Section |
|---|---|---|---|
| 0h | DMAREVID | | Section 16.4.5.1 |
| 4h | TDFDQ | | Section 16.4.5.2 |
| 8h | DMAEMU | | Section 16.4.5.3 |
| 800h | TXGCR0 | | Section 16.4.5.4 |
| 808h | RXGCR0 | | Section 16.4.5.5 |
| 80Ch | RXHPCRA0 | | Section 16.4.5.6 |
| 810h | RXHPCRB0 | | Section 16.4.5.7 |
| 820h | TXGCR1 | | Section 16.4.5.8 |
| 828h | RXGCR1 | | Section 16.4.5.9 |

**Table 16-163. CPPI_DMA REGISTERS (continued)**

| Offset | Acronym | Register Name | Section |
|---|---|---|---|
| 82Ch | RXHPCRA1 | | Section 16.4.5.10 |
| 830h | RXHPCRB1 | | Section 16.4.5.11 |
| 840h | TXGCR2 | | Section 16.4.5.12 |
| 848h | RXGCR2 | | Section 16.4.5.13 |
| 84Ch | RXHPCRA2 | | Section 16.4.5.14 |
| 850h | RXHPCRB2 | | Section 16.4.5.15 |
| 860h | TXGCR3 | | Section 16.4.5.16 |
| 868h | RXGCR3 | | Section 16.4.5.17 |
| 86Ch | RXHPCRA3 | | Section 16.4.5.18 |
| 870h | RXHPCRB3 | | Section 16.4.5.19 |
| 880h | TXGCR4 | | Section 16.4.5.20 |
| 888h | RXGCR4 | | Section 16.4.5.21 |
| 88Ch | RXHPCRA4 | | Section 16.4.5.22 |
| 890h | RXHPCRB4 | | Section 16.4.5.23 |
| 8A0h | TXGCR5 | | Section 16.4.5.24 |
| 8A8h | RXGCR5 | | Section 16.4.5.25 |
| 8ACh | RXHPCRA5 | | Section 16.4.5.26 |
| 8B0h | RXHPCRB5 | | Section 16.4.5.27 |
| 8C0h | TXGCR6 | | Section 16.4.5.28 |
| 8C8h | RXGCR6 | | Section 16.4.5.29 |
| 8CCh | RXHPCRA6 | | Section 16.4.5.30 |
| 8D0h | RXHPCRB6 | | Section 16.4.5.31 |
| 8E0h | TXGCR7 | | Section 16.4.5.32 |
| 8E8h | RXGCR7 | | Section 16.4.5.33 |
| 8ECh | RXHPCRA7 | | Section 16.4.5.34 |
| 8F0h | RXHPCRB7 | | Section 16.4.5.35 |
| 900h | TXGCR8 | | Section 16.4.5.36 |
| 908h | RXGCR8 | | Section 16.4.5.37 |
| 90Ch | RXHPCRA8 | | Section 16.4.5.38 |
| 910h | RXHPCRB8 | | Section 16.4.5.39 |
| 920h | TXGCR9 | | Section 16.4.5.40 |
| 928h | RXGCR9 | | Section 16.4.5.41 |
| 92Ch | RXHPCRA9 | | Section 16.4.5.42 |
| 930h | RXHPCRB9 | | Section 16.4.5.43 |
| 940h | TXGCR10 | | Section 16.4.5.44 |
| 948h | RXGCR10 | | Section 16.4.5.45 |
| 94Ch | RXHPCRA10 | | Section 16.4.5.46 |
| 950h | RXHPCRB10 | | Section 16.4.5.47 |
| 960h | TXGCR11 | | Section 16.4.5.48 |
| 968h | RXGCR11 | | Section 16.4.5.49 |
| 96Ch | RXHPCRA11 | | Section 16.4.5.50 |
| 970h | RXHPCRB11 | | Section 16.4.5.51 |
| 980h | TXGCR12 | | Section 16.4.5.52 |
| 988h | RXGCR12 | | Section 16.4.5.53 |
| 98Ch | RXHPCRA12 | | Section 16.4.5.54 |
| 990h | RXHPCRB12 | | Section 16.4.5.55 |
| 9A0h | TXGCR13 | | Section 16.4.5.56 |

**Table 16-163. CPPI_DMA REGISTERS (continued)**

| Offset | Acronym | Register Name | Section |
|---|---|---|---|
| 9A8h | RXGCR13 | | Section 16.4.5.57 |
| 9ACh | RXHPCRA13 | | Section 16.4.5.58 |
| 9B0h | RXHPCRB13 | | Section 16.4.5.59 |
| 9C0h | TXGCR14 | | Section 16.4.5.60 |
| 9C8h | RXGCR14 | | Section 16.4.5.61 |
| 9CCh | RXHPCRA14 | | Section 16.4.5.62 |
| 9D0h | RXHPCRB14 | | Section 16.4.5.63 |
| 9E0h | TXGCR15 | | Section 16.4.5.64 |
| 9E8h | RXGCR15 | | Section 16.4.5.65 |
| 9ECh | RXHPCRA15 | | Section 16.4.5.66 |
| 9F0h | RXHPCRB15 | | Section 16.4.5.67 |
| A00h | TXGCR16 | | Section 16.4.5.68 |
| A08h | RXGCR16 | | Section 16.4.5.69 |
| A0Ch | RXHPCRA16 | | Section 16.4.5.70 |
| A10h | RXHPCRB16 | | Section 16.4.5.71 |
| A20h | TXGCR17 | | Section 16.4.5.72 |
| A28h | RXGCR17 | | Section 16.4.5.73 |
| A2Ch | RXHPCRA17 | | Section 16.4.5.74 |
| A30h | RXHPCRB17 | | Section 16.4.5.75 |
| A40h | TXGCR18 | | Section 16.4.5.76 |
| A48h | RXGCR18 | | Section 16.4.5.77 |
| A4Ch | RXHPCRA18 | | Section 16.4.5.78 |
| A50h | RXHPCRB18 | | Section 16.4.5.79 |
| A60h | TXGCR19 | | Section 16.4.5.80 |
| A68h | RXGCR19 | | Section 16.4.5.81 |
| A6Ch | RXHPCRA19 | | Section 16.4.5.82 |
| A70h | RXHPCRB19 | | Section 16.4.5.83 |
| A80h | TXGCR20 | | Section 16.4.5.84 |
| A88h | RXGCR20 | | Section 16.4.5.85 |
| A8Ch | RXHPCRA20 | | Section 16.4.5.86 |
| A90h | RXHPCRB20 | | Section 16.4.5.87 |
| AA0h | TXGCR21 | | Section 16.4.5.88 |
| AA8h | RXGCR21 | | Section 16.4.5.89 |
| AACh | RXHPCRA21 | | Section 16.4.5.90 |
| AB0h | RXHPCRB21 | | Section 16.4.5.91 |
| AC0h | TXGCR22 | | Section 16.4.5.92 |
| AC8h | RXGCR22 | | Section 16.4.5.93 |
| ACCh | RXHPCRA22 | | Section 16.4.5.94 |
| AD0h | RXHPCRB22 | | Section 16.4.5.95 |
| AE0h | TXGCR23 | | Section 16.4.5.96 |
| AE8h | RXGCR23 | | Section 16.4.5.97 |
| AECh | RXHPCRA23 | | Section 16.4.5.98 |
| AF0h | RXHPCRB23 | | Section 16.4.5.99 |
| B00h | TXGCR24 | | Section 16.4.5.100 |
| B08h | RXGCR24 | | Section 16.4.5.101 |
| B0Ch | RXHPCRA24 | | Section 16.4.5.102 |
| B10h | RXHPCRB24 | | Section 16.4.5.103 |

**Table 16-163. CPPI_DMA REGISTERS (continued)**

| Offset | Acronym | Register Name | Section |
|---|---|---|---|
| B20h | TXGCR25 | | Section 16.4.5.104 |
| B28h | RXGCR25 | | Section 16.4.5.105 |
| B2Ch | RXHPCRA25 | | Section 16.4.5.106 |
| B30h | RXHPCRB25 | | Section 16.4.5.107 |
| B40h | TXGCR26 | | Section 16.4.5.108 |
| B48h | RXGCR26 | | Section 16.4.5.109 |
| B4Ch | RXHPCRA26 | | Section 16.4.5.110 |
| B50h | RXHPCRB26 | | Section 16.4.5.111 |
| B60h | TXGCR27 | | Section 16.4.5.112 |
| B68h | RXGCR27 | | Section 16.4.5.113 |
| B6Ch | RXHPCRA27 | | Section 16.4.5.114 |
| B70h | RXHPCRB27 | | Section 16.4.5.115 |
| B80h | TXGCR28 | | Section 16.4.5.116 |
| B88h | RXGCR28 | | Section 16.4.5.117 |
| B8Ch | RXHPCRA28 | | Section 16.4.5.118 |
| B90h | RXHPCRB28 | | Section 16.4.5.119 |
| BA0h | TXGCR29 | | Section 16.4.5.120 |
| BA8h | RXGCR29 | | Section 16.4.5.121 |
| BACh | RXHPCRA29 | | Section 16.4.5.122 |
| BB0h | RXHPCRB29 | | Section 16.4.5.123 |

### 16.4.5.1 DMAREVID Register (offset = 0h) [reset = 530901h]

DMAREVID is shown in Figure 16-152 and described in Table 16-164.

**Figure 16-152. DMAREVID Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| Reserved | | MODID | | | | | |
| | | R-0 | | | | | |
| **23** | **22** | **21** | **20** | **19** | **18** | **17** | **16** |
| MODID | | | | | | | |
| R-0 | | | | | | | |
| **15** | **14** | **13** | **12** | **11** | **10** | **9** | **8** |
| REVRTL | | | | | REVMAJ | | |
| R-0 | | | | | R-0 | | |
| **7** | **6** | **5** | **4** | **3** | **2** | **1** | **0** |
| REVMIN | | | | | | | |
| R-0 | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-164. DMAREVID Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 29-16 | MODID | R-0 | 0 | Module ID field |
| 15-11 | REVRTL | R-0 | 0 | RTL revision.<br>Will vary depending on release. |
| 10-8 | REVMAJ | R-0 | 0 | Major revision. |
| 7-0 | REVMIN | R-0 | 0 | Minor revision.<br>CPPI DMA Revision Register |

### 16.4.5.2 TDFDQ Register (offset = 4h) [reset = 0h]

TDFDQ is shown in Figure 16-153 and described in Table 16-165.

**Figure 16-153. TDFDQ Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| Reserved | | | | | | | |
| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
| Reserved | | | | | | | |
| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
| Reserved | | TD_DESC_QMGR | | TD_DESC_QNUM | | | |
| | | R/W-0h | | R/W-0h | | | |
| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
| TD_DESC_QNUM | | | | | | | |
| R/W-0h | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-165. TDFDQ Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-12 | TD_DESC_QMGR | R/W | 0h | This field controls which of the 4 Queue Managers the DMA will access in order to allocate a channel teardown descriptor from the teardown descriptor queue. |
| 11-0 | TD_DESC_QNUM | R/W | 0h | This field controls which of the 2K queues in the indicated queue manager should be read in order to allocate channel teardown descriptors.<br>CPPI DMA Teardown Free Descriptor Queue Control Register |

### 16.4.5.3 DMAEMU Register (offset = 8h) [reset = 0h]

DMAEMU is shown in Figure 16-154 and described in Table 16-166.

**Figure 16-154. DMAEMU Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| Reserved | | | | | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| Reserved | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| Reserved | | | | | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| Reserved | | | | | | SOFT | FREE |
| | | | | | | R/W-0h | R/W-0h |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-166. DMAEMU Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 1 | SOFT | R/W | 0h | Control for emulation pause request<br>1 = Does not force emu_pause_req low<br>0 = Forces emu_pause_req low |
| 0 | FREE | R/W | 0h | Enable for emulation suspend CPPI DMA Emulation Control Register |

### 16.4.5.4 TXGCR0 Register (offset = 800h) [reset = 0h]

TXGCR0 is shown in Figure 16-155 and described in Table 16-167.

**Figure 16-155. TXGCR0 Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| TX_ENABLE | TX_TEARDOWN | Reserved | | | | | |
| R/W-0h | R/W-0h | | | | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| Reserved | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| Reserved | | TX_DEFAULT_QMGR | | TX_DEFAULT_QNUM | | | |
| | | W-0h | | W-0h | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| TX_DEFAULT_QNUM | | | | | | | |
| W-0h | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-167. TXGCR0 Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31 | TX_ENABLE | R/W | 0h | This field enables or disables the channel<br>0 = channel is disabled<br>1 = channel is enabled This field will be cleared after a channel teardown is complete. |
| 30 | TX_TEARDOWN | R/W | 0h | Setting this bit will request the channel to be torn down.<br>This field will remain set after a channel teardown is complete. |
| 13-12 | TX_DEFAULT_QMGR | W | 0h | This field controls the default queue manager number that will be used to queue teardown descriptors back to the host. |
| 11-0 | TX_DEFAULT_QNUM | W | 0h | This field controls the default queue number within the selected queue manager onto which teardown descriptors will be queued back to the host.<br>Table<br>98 -Tx Channel N Global Configuration Registers |

### 16.4.5.5 RXGCR0 Register (offset = 808h) [reset = 0h]

RXGCR0 is shown in Figure 16-156 and described in Table 16-168.

**Figure 16-156. RXGCR0 Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| RX_ENABLE | RX_TEARDOWN | RX_PAUSE | Reserved | | | | RX_ERROR_HANDLING |
| R/W-0h | R/W-0h | R/W-0h | | | | | W-0h |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| RX_SOP_OFFSET | | | | | | | |
| W-0h | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| RX_DEFAULT_DESC_TYPE | | RX_DEFAULT_RQ_QMGR | | RX_DEFAULT_RQ_QNUM | | | |
| W-0h | | W-0h | | W-0h | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| RX_DEFAULT_RQ_QNUM | | | | | | | |
| W-0h | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-168. RXGCR0 Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31 | RX_ENABLE | R/W | 0h | This field enables or disables the channel<br>0 = channel is disabled<br>1 = channel is enabled This field will be cleared after a channel teardown is complete. |
| 30 | RX_TEARDOWN | R/W | 0h | This field indicates whether or not an Rx teardown operation is complete.<br>This field should be cleared when a channel is initialized.<br>This field will be set after a channel teardown is complete. |
| 29 | RX_PAUSE | R/W | 0h | Setting this bit causes the CPPI DMA to be suspended for rx channels.<br>If a pause is being requested and the channel is not in a packet then drop the credit. |
| 24 | RX_ERROR_HANDLING | W | 0h | This bit controls the error handling mode for the channel and is only used when channel errors (i.e.<br>descriptor or buffer starvation occurs):<br>0 = Starvation errors result in dropping packet and reclaiming any used descriptor or buffer resources back to the original queues/pools they were allocated to<br>1 = Starvation errors result in subsequent re-try of the descriptor allocation operation.<br>In this mode, the DMA will return to the IDLE state without saving it's internal operational state back to the internal state RAM and without issuing an advance operation on the FIFO interface.<br>This results in the DMA re-initiating the FIFO block transfer at a later time with the intention that additional free buffers and/or descriptors will have been added.<br>Regardless of the value of this bit, the DMA will assert the cdma_rx_sof_overrun (for SOP) or cdma_rx_mof_overrun (for non-SOP) when |
| 23-16 | RX_SOP_OFFSET | W | 0h | This field specifies the number of bytes that are to be skipped in the SOP buffer before beginning to write the payload.<br>This value must be less than the minimum size of a buffer in the system.<br>Valid values are<br>0 - 255 bytes. |

**Table 16-168. RXGCR0 Register Field Descriptions (continued)**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 15-14 | RX_DEFAULT_DESC_TYPE | W | 0h | This field indicates the default descriptor type to use:<br>0 = Reserved<br>1 = Host<br>2 = Reserved<br>3 = Reserved The actual descriptor type that will be used for reception can be overridden by information provided in the CPPI FIFO data block. |
| 13-12 | RX_DEFAULT_RQ_QMGR | W | 0h | This field indicates the default receive queue manager that this channel should use.<br>The actual receive queue manager index can be overridden by information provided in the CPPI FIFO data block. |
| 11-0 | RX_DEFAULT_RQ_QNUM | W | 0h | This field indicates the default receive queue that this channel should use.<br>The actual receive queue that will be used for reception can be overridden by information provided in the CPPI FIFO data block. Table<br>99 -Rx Channel N Global Configuration Registers |

### 16.4.5.6 RXHPCRA0 Register (offset = 80Ch) [reset = 0h]

RXHPCRA0 is shown in Figure 16-157 and described in Table 16-169.

**Figure 16-157. RXHPCRA0 Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| Reserved | | RX_HOST_FDQ1_QMGR | | RX_HOST_FDQ1_QNUM | | | |
| | | W-0h | | W-0h | | | |
| **23** | **22** | **21** | **20** | **19** | **18** | **17** | **16** |
| RX_HOST_FDQ1_QNUM | | | | | | | |
| W-0h | | | | | | | |
| **15** | **14** | **13** | **12** | **11** | **10** | **9** | **8** |
| Reserved | | RX_HOST_FDQ0_QMGR | | RX_HOST_FDQ0_QNUM | | | |
| | | W-0h | | W-0h | | | |
| **7** | **6** | **5** | **4** | **3** | **2** | **1** | **0** |
| RX_HOST_FDQ0_QNUM | | | | | | | |
| W-0h | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-169. RXHPCRA0 Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 29-28 | RX_HOST_FDQ1_QMGR | W | 0h | This field specifies which Buffer Manager should be used for the second Rx buffer in a host type packet. |
| 27-16 | RX_HOST_FDQ1_QNUM | W | 0h | This field specifies which Free Descriptor / Buffer Pool should be used for the 2nd Rx buffer in a host type packet |
| 13-12 | RX_HOST_FDQ0_QMGR | W | 0h | This field specifies which Buffer Manager should be used for the second Rx buffer in a host type packet. |
| 11-0 | RX_HOST_FDQ0_QNUM | W | 0h | This field specifies which Free Descriptor / Buffer Pool should be used for the 1st Rx buffer in a host type packet Table 100 -Rx Channel N Host Packet Configuration Registers A |

### 16.4.5.7 RXHPCRB0 Register (offset = 810h) [reset = 0h]

RXHPCRB0 is shown in Figure 16-158 and described in Table 16-170.

**Figure 16-158. RXHPCRB0 Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| Reserved | | RX_HOST_FDQ3_QMGR | | RX_HOST_FDQ3_QNUM | | | |
| | | W-0h | | W-0h | | | |
| **23** | **22** | **21** | **20** | **19** | **18** | **17** | **16** |
| RX_HOST_FDQ3_QNUM | | | | | | | |
| W-0h | | | | | | | |
| **15** | **14** | **13** | **12** | **11** | **10** | **9** | **8** |
| Reserved | | RX_HOST_FDQ2_QMGR | | RX_HOST_FDQ2_QNUM | | | |
| | | W-0h | | W-0h | | | |
| **7** | **6** | **5** | **4** | **3** | **2** | **1** | **0** |
| RX_HOST_FDQ2_QNUM | | | | | | | |
| W-0h | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-170. RXHPCRB0 Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 29-28 | RX_HOST_FDQ3_QMGR | W | 0h | This field specifies which Manager should be used for the 4th or later Rx buffers in a host type packet |
| 27-16 | RX_HOST_FDQ3_QNUM | W | 0h | This field specifies which Free Descriptor Queue should be used for the 4th or later Rx buffers in a host type packet |
| 13-12 | RX_HOST_FDQ2_QMGR | W | 0h | This field specifies which Buffer Manager should be used for the 3rd Rx buffer in a host type packet |
| 11-0 | RX_HOST_FDQ2_QNUM | W | 0h | This field specifies which Free Descriptor / Buffer Pool should be used for the 3rd Rx buffer in a host type packet Table 101 -Rx Channel N Host Packet Configuration Registers B |

### 16.4.5.8 TXGCR1 Register (offset = 820h) [reset = 0h]

TXGCR1 is shown in Figure 16-159 and described in Table 16-171.

**Figure 16-159. TXGCR1 Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| TX_ENABLE | TX_TEARDOWN | Reserved | | | | | |
| R/W-0h | R/W-0h | | | | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| Reserved | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| Reserved | | TX_DEFAULT_QMGR | | TX_DEFAULT_QNUM | | | |
| | | W-0h | | W-0h | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| TX_DEFAULT_QNUM | | | | | | | |
| W-0h | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-171. TXGCR1 Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31 | TX_ENABLE | R/W | 0h | This field enables or disables the channel<br>0 = channel is disabled<br>1 = channel is enabled This field will be cleared after a channel teardown is complete. |
| 30 | TX_TEARDOWN | R/W | 0h | Setting this bit will request the channel to be torn down.<br>This field will remain set after a channel teardown is complete. |
| 13-12 | TX_DEFAULT_QMGR | W | 0h | This field controls the default queue manager number that will be used to queue teardown descriptors back to the host. |
| 11-0 | TX_DEFAULT_QNUM | W | 0h | This field controls the default queue number within the selected queue manager onto which teardown descriptors will be queued back to the host.<br>Table<br>98 -Tx Channel N Global Configuration Registers |

#### 16.4.5.9 RXGCR1 Register (offset = 828h) [reset = 0h]

RXGCR1 is shown in Figure 16-160 and described in Table 16-172.

**Figure 16-160. RXGCR1 Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| RX_ENABLE | RX_TEARDOWN | RX_PAUSE | Reserved | | | | RX_ERROR_HANDLING |
| R/W-0h | R/W-0h | R/W-0h | | | | | W-0h |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| RX_SOP_OFFSET | | | | | | | |
| W-0h | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| RX_DEFAULT_DESC_TYPE | | RX_DEFAULT_RQ_QMGR | | RX_DEFAULT_RQ_QNUM | | | |
| W-0h | | W-0h | | W-0h | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| RX_DEFAULT_RQ_QNUM | | | | | | | |
| W-0h | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-172. RXGCR1 Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31 | RX_ENABLE | R/W | 0h | This field enables or disables the channel<br>0 = channel is disabled<br>1 = channel is enabled This field will be cleared after a channel teardown is complete. |
| 30 | RX_TEARDOWN | R/W | 0h | This field indicates whether or not an Rx teardown operation is complete.<br>This field should be cleared when a channel is initialized.<br>This field will be set after a channel teardown is complete. |
| 29 | RX_PAUSE | R/W | 0h | Setting this bit causes the CPPI DMA to be suspended for rx channels.<br>If a pause is being requested and the channel is not in a packet then drop the credit. |
| 24 | RX_ERROR_HANDLING | W | 0h | This bit controls the error handling mode for the channel and is only used when channel errors (i.e.<br>descriptor or buffer starvation occurs):<br>0 = Starvation errors result in dropping packet and reclaiming any used descriptor or buffer resources back to the original queues/pools they were allocated to<br>1 = Starvation errors result in subsequent re-try of the descriptor allocation operation.<br>In this mode, the DMA will return to the IDLE state without saving it's internal operational state back to the internal state RAM and without issuing an advance operation on the FIFO interface.<br>This results in the DMA re-initiating the FIFO block transfer at a later time with the intention that additional free buffers and/or descriptors will have been added.<br>Regardless of the value of this bit, the DMA will assert the cdma_rx_sof_overrun (for SOP) or cdma_rx_mof_overrun (for non-SOP) when |
| 23-16 | RX_SOP_OFFSET | W | 0h | This field specifies the number of bytes that are to be skipped in the SOP buffer before beginning to write the payload.<br>This value must be less than the minimum size of a buffer in the system.<br>Valid values are<br>0 - 255 bytes. |

### Table 16-172. RXGCR1 Register Field Descriptions (continued)

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 15-14 | RX_DEFAULT_DESC_TYPE | W | 0h | This field indicates the default descriptor type to use:<br>0 = Reserved<br>1 = Host<br>2 = Reserved<br>3 = Reserved The actual descriptor type that will be used for reception can be overridden by information provided in the CPPI FIFO data block. |
| 13-12 | RX_DEFAULT_RQ_QMGR | W | 0h | This field indicates the default receive queue manager that this channel should use.<br>The actual receive queue manager index can be overridden by information provided in the CPPI FIFO data block. |
| 11-0 | RX_DEFAULT_RQ_QNUM | W | 0h | This field indicates the default receive queue that this channel should use.<br>The actual receive queue that will be used for reception can be overridden by information provided in the CPPI FIFO data block. Table<br>99 -Rx Channel N Global Configuration Registers |

### 16.4.5.10 RXHPCRA1 Register (offset = 82Ch) [reset = 0h]

RXHPCRA1 is shown in Figure 16-161 and described in Table 16-173.

**Figure 16-161. RXHPCRA1 Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| Reserved | | RX_HOST_FDQ1_QMGR | | RX_HOST_FDQ1_QNUM | | | |
| | | W-0h | | W-0h | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| RX_HOST_FDQ1_QNUM | | | | | | | |
| W-0h | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| Reserved | | RX_HOST_FDQ0_QMGR | | RX_HOST_FDQ0_QNUM | | | |
| | | W-0h | | W-0h | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| RX_HOST_FDQ0_QNUM | | | | | | | |
| W-0h | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-173. RXHPCRA1 Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 29-28 | RX_HOST_FDQ1_QMGR | W | 0h | This field specifies which Buffer Manager should be used for the second Rx buffer in a host type packet. |
| 27-16 | RX_HOST_FDQ1_QNUM | W | 0h | This field specifies which Free Descriptor / Buffer Pool should be used for the 2nd Rx buffer in a host type packet |
| 13-12 | RX_HOST_FDQ0_QMGR | W | 0h | This field specifies which Buffer Manager should be used for the second Rx buffer in a host type packet. |
| 11-0 | RX_HOST_FDQ0_QNUM | W | 0h | This field specifies which Free Descriptor / Buffer Pool should be used for the 1st Rx buffer in a host type packet Table 100 -Rx Channel N Host Packet Configuration Registers A |

### 16.4.5.11 RXHPCRB1 Register (offset = 830h) [reset = 0h]

RXHPCRB1 is shown in Figure 16-162 and described in Table 16-174.

**Figure 16-162. RXHPCRB1 Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| Reserved | | RX_HOST_FDQ3_QMGR | | RX_HOST_FDQ3_QNUM | | | |
| | | W-0h | | W-0h | | | |
| **23** | **22** | **21** | **20** | **19** | **18** | **17** | **16** |
| RX_HOST_FDQ3_QNUM | | | | | | | |
| W-0h | | | | | | | |
| **15** | **14** | **13** | **12** | **11** | **10** | **9** | **8** |
| Reserved | | RX_HOST_FDQ2_QMGR | | RX_HOST_FDQ2_QNUM | | | |
| | | W-0h | | W-0h | | | |
| **7** | **6** | **5** | **4** | **3** | **2** | **1** | **0** |
| RX_HOST_FDQ2_QNUM | | | | | | | |
| W-0h | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-174. RXHPCRB1 Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 29-28 | RX_HOST_FDQ3_QMGR | W | 0h | This field specifies which Manager should be used for the 4th or later Rx buffers in a host type packet |
| 27-16 | RX_HOST_FDQ3_QNUM | W | 0h | This field specifies which Free Descriptor Queue should be used for the 4th or later Rx buffers in a host type packet |
| 13-12 | RX_HOST_FDQ2_QMGR | W | 0h | This field specifies which Buffer Manager should be used for the 3rd Rx buffer in a host type packet |
| 11-0 | RX_HOST_FDQ2_QNUM | W | 0h | This field specifies which Free Descriptor / Buffer Pool should be used for the 3rd Rx buffer in a host type packet Table 101 -Rx Channel N Host Packet Configuration Registers B |

### 16.4.5.12 TXGCR2 Register (offset = 840h) [reset = 0h]

TXGCR2 is shown in Figure 16-163 and described in Table 16-175.

**Figure 16-163. TXGCR2 Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| TX_ENABLE | TX_TEARDOWN | Reserved | | | | | |
| R/W-0h | R/W-0h | | | | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| Reserved | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| Reserved | | TX_DEFAULT_QMGR | | TX_DEFAULT_QNUM | | | |
| | | W-0h | | W-0h | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| TX_DEFAULT_QNUM | | | | | | | |
| W-0h | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-175. TXGCR2 Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31 | TX_ENABLE | R/W | 0h | This field enables or disables the channel<br>0 = channel is disabled<br>1 = channel is enabled This field will be cleared after a channel teardown is complete. |
| 30 | TX_TEARDOWN | R/W | 0h | Setting this bit will request the channel to be torn down.<br>This field will remain set after a channel teardown is complete. |
| 13-12 | TX_DEFAULT_QMGR | W | 0h | This field controls the default queue manager number that will be used to queue teardown descriptors back to the host. |
| 11-0 | TX_DEFAULT_QNUM | W | 0h | This field controls the default queue number within the selected queue manager onto which teardown descriptors will be queued back to the host.<br>Table<br>98 -Tx Channel N Global Configuration Registers |

### 16.4.5.13 RXGCR2 Register (offset = 848h) [reset = 0h]

RXGCR2 is shown in Figure 16-164 and described in Table 16-176.

**Figure 16-164. RXGCR2 Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| RX_ENABLE | RX_TEARDOWN | RX_PAUSE | Reserved | | | | RX_ERROR_HANDLING |
| R/W-0h | R/W-0h | R/W-0h | | | | | W-0h |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| RX_SOP_OFFSET | | | | | | | |
| W-0h | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| RX_DEFAULT_DESC_TYPE | | RX_DEFAULT_RQ_QMGR | | RX_DEFAULT_RQ_QNUM | | | |
| W-0h | | W-0h | | W-0h | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| RX_DEFAULT_RQ_QNUM | | | | | | | |
| W-0h | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-176. RXGCR2 Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31 | RX_ENABLE | R/W | 0h | This field enables or disables the channel<br>0 = channel is disabled<br>1 = channel is enabled This field will be cleared after a channel teardown is complete. |
| 30 | RX_TEARDOWN | R/W | 0h | This field indicates whether or not an Rx teardown operation is complete.<br>This field should be cleared when a channel is initialized.<br>This field will be set after a channel teardown is complete. |
| 29 | RX_PAUSE | R/W | 0h | Setting this bit causes the CPPI DMA to be suspended for rx channels.<br>If a pause is being requested and the channel is not in a packet then drop the credit. |
| 24 | RX_ERROR_HANDLING | W | 0h | This bit controls the error handling mode for the channel and is only used when channel errors (i.e.<br>descriptor or buffer starvation occurs):<br>0 = Starvation errors result in dropping packet and reclaiming any used descriptor or buffer resources back to the original queues/pools they were allocated to<br>1 = Starvation errors result in subsequent re-try of the descriptor allocation operation.<br>In this mode, the DMA will return to the IDLE state without saving it's internal operational state back to the internal state RAM and without issuing an advance operation on the FIFO interface.<br>This results in the DMA re-initiating the FIFO block transfer at a later time with the intention that additional free buffers and/or descriptors will have been added.<br>Regardless of the value of this bit, the DMA will assert the cdma_rx_sof_overrun (for SOP) or cdma_rx_mof_overrun (for non-SOP) when |
| 23-16 | RX_SOP_OFFSET | W | 0h | This field specifies the number of bytes that are to be skipped in the SOP buffer before beginning to write the payload.<br>This value must be less than the minimum size of a buffer in the system.<br>Valid values are<br>0 - 255 bytes. |

**Table 16-176. RXGCR2 Register Field Descriptions (continued)**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 15-14 | RX_DEFAULT_DESC_TYPE | W | 0h | This field indicates the default descriptor type to use:<br>0 = Reserved<br>1 = Host<br>2 = Reserved<br>3 = Reserved The actual descriptor type that will be used for reception can be overridden by information provided in the CPPI FIFO data block. |
| 13-12 | RX_DEFAULT_RQ_QMGR | W | 0h | This field indicates the default receive queue manager that this channel should use.<br>The actual receive queue manager index can be overridden by information provided in the CPPI FIFO data block. |
| 11-0 | RX_DEFAULT_RQ_QNUM | W | 0h | This field indicates the default receive queue that this channel should use.<br>The actual receive queue that will be used for reception can be overridden by information provided in the CPPI FIFO data block. Table<br>99 -Rx Channel N Global Configuration Registers |

### 16.4.5.14 RXHPCRA2 Register (offset = 84Ch) [reset = 0h]

RXHPCRA2 is shown in Figure 16-165 and described in Table 16-177.

**Figure 16-165. RXHPCRA2 Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| Reserved | | RX_HOST_FDQ1_QMGR | | RX_HOST_FDQ1_QNUM | | | |
| | | W-0h | | W-0h | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| RX_HOST_FDQ1_QNUM | | | | | | | |
| W-0h | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| Reserved | | RX_HOST_FDQ0_QMGR | | RX_HOST_FDQ0_QNUM | | | |
| | | W-0h | | W-0h | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| RX_HOST_FDQ0_QNUM | | | | | | | |
| W-0h | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-177. RXHPCRA2 Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 29-28 | RX_HOST_FDQ1_QMGR | W | 0h | This field specifies which Buffer Manager should be used for the second Rx buffer in a host type packet. |
| 27-16 | RX_HOST_FDQ1_QNUM | W | 0h | This field specifies which Free Descriptor / Buffer Pool should be used for the 2nd Rx buffer in a host type packet |
| 13-12 | RX_HOST_FDQ0_QMGR | W | 0h | This field specifies which Buffer Manager should be used for the second Rx buffer in a host type packet. |
| 11-0 | RX_HOST_FDQ0_QNUM | W | 0h | This field specifies which Free Descriptor / Buffer Pool should be used for the 1st Rx buffer in a host type packet Table 100 -Rx Channel N Host Packet Configuration Registers A |

### 16.4.5.15 RXHPCRB2 Register (offset = 850h) [reset = 0h]

RXHPCRB2 is shown in Figure 16-166 and described in Table 16-178.

**Figure 16-166. RXHPCRB2 Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| Reserved | | RX_HOST_FDQ3_QMGR | | RX_HOST_FDQ3_QNUM | | | |
| | | W-0h | | W-0h | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| RX_HOST_FDQ3_QNUM | | | | | | | |
| W-0h | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| Reserved | | RX_HOST_FDQ2_QMGR | | RX_HOST_FDQ2_QNUM | | | |
| | | W-0h | | W-0h | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| RX_HOST_FDQ2_QNUM | | | | | | | |
| W-0h | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-178. RXHPCRB2 Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 29-28 | RX_HOST_FDQ3_QMGR | W | 0h | This field specifies which Manager should be used for the 4th or later Rx buffers in a host type packet |
| 27-16 | RX_HOST_FDQ3_QNUM | W | 0h | This field specifies which Free Descriptor Queue should be used for the 4th or later Rx buffers in a host type packet |
| 13-12 | RX_HOST_FDQ2_QMGR | W | 0h | This field specifies which Buffer Manager should be used for the 3rd Rx buffer in a host type packet |
| 11-0 | RX_HOST_FDQ2_QNUM | W | 0h | This field specifies which Free Descriptor / Buffer Pool should be used for the 3rd Rx buffer in a host type packet Table 101 -Rx Channel N Host Packet Configuration Registers B |

### 16.4.5.16 TXGCR3 Register (offset = 860h) [reset = 0h]

TXGCR3 is shown in Figure 16-167 and described in Table 16-179.

**Figure 16-167. TXGCR3 Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| TX_ENABLE | TX_TEARDOWN | Reserved | | | | | |
| R/W-0h | R/W-0h | | | | | | |
| **23** | **22** | **21** | **20** | **19** | **18** | **17** | **16** |
| Reserved | | | | | | | |
| **15** | **14** | **13** | **12** | **11** | **10** | **9** | **8** |
| Reserved | | TX_DEFAULT_QMGR | | TX_DEFAULT_QNUM | | | |
| | | W-0h | | W-0h | | | |
| **7** | **6** | **5** | **4** | **3** | **2** | **1** | **0** |
| TX_DEFAULT_QNUM | | | | | | | |
| W-0h | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-179. TXGCR3 Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31 | TX_ENABLE | R/W | 0h | This field enables or disables the channel<br>0 = channel is disabled<br>1 = channel is enabled This field will be cleared after a channel teardown is complete. |
| 30 | TX_TEARDOWN | R/W | 0h | Setting this bit will request the channel to be torn down.<br>This field will remain set after a channel teardown is complete. |
| 13-12 | TX_DEFAULT_QMGR | W | 0h | This field controls the default queue manager number that will be used to queue teardown descriptors back to the host. |
| 11-0 | TX_DEFAULT_QNUM | W | 0h | This field controls the default queue number within the selected queue manager onto which teardown descriptors will be queued back to the host.<br>Table<br>98 -Tx Channel N Global Configuration Registers |

### 16.4.5.17 RXGCR3 Register (offset = 868h) [reset = 0h]

RXGCR3 is shown in Figure 16-168 and described in Table 16-180.

**Figure 16-168. RXGCR3 Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| RX_ENABLE | RX_TEARDOWN | RX_PAUSE | Reserved | | | | RX_ERROR_HANDLING |
| R/W-0h | R/W-0h | R/W-0h | | | | | W-0h |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| RX_SOP_OFFSET | | | | | | | |
| W-0h | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| RX_DEFAULT_DESC_TYPE | | RX_DEFAULT_RQ_QMGR | | RX_DEFAULT_RQ_QNUM | | | |
| W-0h | | W-0h | | W-0h | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| RX_DEFAULT_RQ_QNUM | | | | | | | |
| W-0h | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-180. RXGCR3 Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31 | RX_ENABLE | R/W | 0h | This field enables or disables the channel<br>0 = channel is disabled<br>1 = channel is enabled This field will be cleared after a channel teardown is complete. |
| 30 | RX_TEARDOWN | R/W | 0h | This field indicates whether or not an Rx teardown operation is complete.<br>This field should be cleared when a channel is initialized.<br>This field will be set after a channel teardown is complete. |
| 29 | RX_PAUSE | R/W | 0h | Setting this bit causes the CPPI DMA to be suspended for rx channels.<br>If a pause is being requested and the channel is not in a packet then drop the credit. |
| 24 | RX_ERROR_HANDLING | W | 0h | This bit controls the error handling mode for the channel and is only used when channel errors (i.e.<br>descriptor or buffer starvation occurs):<br>0 = Starvation errors result in dropping packet and reclaiming any used descriptor or buffer resources back to the original queues/pools they were allocated to<br>1 = Starvation errors result in subsequent re-try of the descriptor allocation operation.<br>In this mode, the DMA will return to the IDLE state without saving it's internal operational state back to the internal state RAM and without issuing an advance operation on the FIFO interface.<br>This results in the DMA re-initiating the FIFO block transfer at a later time with the intention that additional free buffers and/or descriptors will have been added.<br>Regardless of the value of this bit, the DMA will assert the cdma_rx_sof_overrun (for SOP) or cdma_rx_mof_overrun (for non-SOP) when |
| 23-16 | RX_SOP_OFFSET | W | 0h | This field specifies the number of bytes that are to be skipped in the SOP buffer before beginning to write the payload.<br>This value must be less than the minimum size of a buffer in the system.<br>Valid values are<br>0 - 255 bytes. |

**Table 16-180. RXGCR3 Register Field Descriptions (continued)**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 15-14 | RX_DEFAULT_DESC_TYPE | W | 0h | This field indicates the default descriptor type to use:<br>0 = Reserved<br>1 = Host<br>2 = Reserved<br>3 = Reserved The actual descriptor type that will be used for reception can be overridden by information provided in the CPPI FIFO data block. |
| 13-12 | RX_DEFAULT_RQ_QMGR | W | 0h | This field indicates the default receive queue manager that this channel should use.<br>The actual receive queue manager index can be overridden by information provided in the CPPI FIFO data block. |
| 11-0 | RX_DEFAULT_RQ_QNUM | W | 0h | This field indicates the default receive queue that this channel should use.<br>The actual receive queue that will be used for reception can be overridden by information provided in the CPPI FIFO data block. Table<br>99 -Rx Channel N Global Configuration Registers |

### 16.4.5.18 RXHPCRA3 Register (offset = 86Ch) [reset = 0h]

RXHPCRA3 is shown in Figure 16-169 and described in Table 16-181.

**Figure 16-169. RXHPCRA3 Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| Reserved | | RX_HOST_FDQ1_QMGR | | RX_HOST_FDQ1_QNUM | | | |
| | | W-0h | | W-0h | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| RX_HOST_FDQ1_QNUM | | | | | | | |
| W-0h | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| Reserved | | RX_HOST_FDQ0_QMGR | | RX_HOST_FDQ0_QNUM | | | |
| | | W-0h | | W-0h | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| RX_HOST_FDQ0_QNUM | | | | | | | |
| W-0h | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-181. RXHPCRA3 Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 29-28 | RX_HOST_FDQ1_QMGR | W | 0h | This field specifies which Buffer Manager should be used for the second Rx buffer in a host type packet. |
| 27-16 | RX_HOST_FDQ1_QNUM | W | 0h | This field specifies which Free Descriptor / Buffer Pool should be used for the 2nd Rx buffer in a host type packet |
| 13-12 | RX_HOST_FDQ0_QMGR | W | 0h | This field specifies which Buffer Manager should be used for the second Rx buffer in a host type packet. |
| 11-0 | RX_HOST_FDQ0_QNUM | W | 0h | This field specifies which Free Descriptor / Buffer Pool should be used for the 1st Rx buffer in a host type packet Table 100 -Rx Channel N Host Packet Configuration Registers A |

### 16.4.5.19 RXHPCRB3 Register (offset = 870h) [reset = 0h]

RXHPCRB3 is shown in Figure 16-170 and described in Table 16-182.

**Figure 16-170. RXHPCRB3 Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| Reserved | | RX_HOST_FDQ3_QMGR | | RX_HOST_FDQ3_QNUM | | | |
| | | W-0h | | W-0h | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| RX_HOST_FDQ3_QNUM | | | | | | | |
| W-0h | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| Reserved | | RX_HOST_FDQ2_QMGR | | RX_HOST_FDQ2_QNUM | | | |
| | | W-0h | | W-0h | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| RX_HOST_FDQ2_QNUM | | | | | | | |
| W-0h | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-182. RXHPCRB3 Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 29-28 | RX_HOST_FDQ3_QMGR | W | 0h | This field specifies which Manager should be used for the 4th or later Rx buffers in a host type packet |
| 27-16 | RX_HOST_FDQ3_QNUM | W | 0h | This field specifies which Free Descriptor Queue should be used for the 4th or later Rx buffers in a host type packet |
| 13-12 | RX_HOST_FDQ2_QMGR | W | 0h | This field specifies which Buffer Manager should be used for the 3rd Rx buffer in a host type packet |
| 11-0 | RX_HOST_FDQ2_QNUM | W | 0h | This field specifies which Free Descriptor / Buffer Pool should be used for the 3rd Rx buffer in a host type packet Table 101 -Rx Channel N Host Packet Configuration Registers B |

### 16.4.5.20 TXGCR4 Register (offset = 880h) [reset = 0h]

TXGCR4 is shown in Figure 16-171 and described in Table 16-183.

**Figure 16-171. TXGCR4 Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| TX_ENABLE | TX_TEARDOWN | Reserved | | | | | |
| R/W-0h | R/W-0h | | | | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| Reserved | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| Reserved | | TX_DEFAULT_QMGR | | TX_DEFAULT_QNUM | | | |
| | | W-0h | | W-0h | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| TX_DEFAULT_QNUM | | | | | | | |
| W-0h | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-183. TXGCR4 Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31 | TX_ENABLE | R/W | 0h | This field enables or disables the channel<br>0 = channel is disabled<br>1 = channel is enabled This field will be cleared after a channel teardown is complete. |
| 30 | TX_TEARDOWN | R/W | 0h | Setting this bit will request the channel to be torn down.<br>This field will remain set after a channel teardown is complete. |
| 13-12 | TX_DEFAULT_QMGR | W | 0h | This field controls the default queue manager number that will be used to queue teardown descriptors back to the host. |
| 11-0 | TX_DEFAULT_QNUM | W | 0h | This field controls the default queue number within the selected queue manager onto which teardown descriptors will be queued back to the host.<br>Table<br>98 -Tx Channel N Global Configuration Registers |

### 16.4.5.21 RXGCR4 Register (offset = 888h) [reset = 0h]

RXGCR4 is shown in Figure 16-172 and described in Table 16-184.

**Figure 16-172. RXGCR4 Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| RX_ENABLE | RX_TEARDOWN | RX_PAUSE | Reserved | | | | RX_ERROR_HANDLING |
| R/W-0h | R/W-0h | R/W-0h | | | | | W-0h |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| RX_SOP_OFFSET | | | | | | | |
| W-0h | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| RX_DEFAULT_DESC_TYPE | | RX_DEFAULT_RQ_QMGR | | RX_DEFAULT_RQ_QNUM | | | |
| W-0h | | W-0h | | W-0h | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| RX_DEFAULT_RQ_QNUM | | | | | | | |
| W-0h | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-184. RXGCR4 Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31 | RX_ENABLE | R/W | 0h | This field enables or disables the channel<br>0 = channel is disabled<br>1 = channel is enabled This field will be cleared after a channel teardown is complete. |
| 30 | RX_TEARDOWN | R/W | 0h | This field indicates whether or not an Rx teardown operation is complete.<br>This field should be cleared when a channel is initialized.<br>This field will be set after a channel teardown is complete. |
| 29 | RX_PAUSE | R/W | 0h | Setting this bit causes the CPPI DMA to be suspended for rx channels.<br>If a pause is being requested and the channel is not in a packet then drop the credit. |
| 24 | RX_ERROR_HANDLING | W | 0h | This bit controls the error handling mode for the channel and is only used when channel errors (i.e.<br>descriptor or buffer starvation occurs):<br>0 = Starvation errors result in dropping packet and reclaiming any used descriptor or buffer resources back to the original queues/pools they were allocated to<br>1 = Starvation errors result in subsequent re-try of the descriptor allocation operation.<br>In this mode, the DMA will return to the IDLE state without saving it's internal operational state back to the internal state RAM and without issuing an advance operation on the FIFO interface.<br>This results in the DMA re-initiating the FIFO block transfer at a later time with the intention that additional free buffers and/or descriptors will have been added.<br>Regardless of the value of this bit, the DMA will assert the cdma_rx_sof_overrun (for SOP) or cdma_rx_mof_overrun (for non-SOP) when |
| 23-16 | RX_SOP_OFFSET | W | 0h | This field specifies the number of bytes that are to be skipped in the SOP buffer before beginning to write the payload.<br>This value must be less than the minimum size of a buffer in the system.<br>Valid values are<br>0 - 255 bytes. |

**Table 16-184. RXGCR4 Register Field Descriptions (continued)**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 15-14 | RX_DEFAULT_DESC_TYPE | W | 0h | This field indicates the default descriptor type to use:<br>0 = Reserved<br>1 = Host<br>2 = Reserved<br>3 = Reserved The actual descriptor type that will be used for reception can be overridden by information provided in the CPPI FIFO data block. |
| 13-12 | RX_DEFAULT_RQ_QMGR | W | 0h | This field indicates the default receive queue manager that this channel should use.<br>The actual receive queue manager index can be overridden by information provided in the CPPI FIFO data block. |
| 11-0 | RX_DEFAULT_RQ_QNUM | W | 0h | This field indicates the default receive queue that this channel should use.<br>The actual receive queue that will be used for reception can be overridden by information provided in the CPPI FIFO data block. Table<br>99 -Rx Channel N Global Configuration Registers |

### 16.4.5.22 RXHPCRA4 Register (offset = 88Ch) [reset = 0h]

RXHPCRA4 is shown in Figure 16-173 and described in Table 16-185.

**Figure 16-173. RXHPCRA4 Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| Reserved | | RX_HOST_FDQ1_QMGR | | RX_HOST_FDQ1_QNUM | | | |
| | | W-0h | | W-0h | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| RX_HOST_FDQ1_QNUM | | | | | | | |
| W-0h | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| Reserved | | RX_HOST_FDQ0_QMGR | | RX_HOST_FDQ0_QNUM | | | |
| | | W-0h | | W-0h | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| RX_HOST_FDQ0_QNUM | | | | | | | |
| W-0h | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-185. RXHPCRA4 Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 29-28 | RX_HOST_FDQ1_QMGR | W | 0h | This field specifies which Buffer Manager should be used for the second Rx buffer in a host type packet. |
| 27-16 | RX_HOST_FDQ1_QNUM | W | 0h | This field specifies which Free Descriptor / Buffer Pool should be used for the 2nd Rx buffer in a host type packet |
| 13-12 | RX_HOST_FDQ0_QMGR | W | 0h | This field specifies which Buffer Manager should be used for the second Rx buffer in a host type packet. |
| 11-0 | RX_HOST_FDQ0_QNUM | W | 0h | This field specifies which Free Descriptor / Buffer Pool should be used for the 1st Rx buffer in a host type packet Table 100 -Rx Channel N Host Packet Configuration Registers A |

### 16.4.5.23 RXHPCRB4 Register (offset = 890h) [reset = 0h]

RXHPCRB4 is shown in Figure 16-174 and described in Table 16-186.

**Figure 16-174. RXHPCRB4 Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| Reserved | | RX_HOST_FDQ3_QMGR | | RX_HOST_FDQ3_QNUM | | | |
| | | W-0h | | W-0h | | | |
| **23** | **22** | **21** | **20** | **19** | **18** | **17** | **16** |
| RX_HOST_FDQ3_QNUM | | | | | | | |
| W-0h | | | | | | | |
| **15** | **14** | **13** | **12** | **11** | **10** | **9** | **8** |
| Reserved | | RX_HOST_FDQ2_QMGR | | RX_HOST_FDQ2_QNUM | | | |
| | | W-0h | | W-0h | | | |
| **7** | **6** | **5** | **4** | **3** | **2** | **1** | **0** |
| RX_HOST_FDQ2_QNUM | | | | | | | |
| W-0h | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-186. RXHPCRB4 Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 29-28 | RX_HOST_FDQ3_QMGR | W | 0h | This field specifies which Manager should be used for the 4th or later Rx buffers in a host type packet |
| 27-16 | RX_HOST_FDQ3_QNUM | W | 0h | This field specifies which Free Descriptor Queue should be used for the 4th or later Rx buffers in a host type packet |
| 13-12 | RX_HOST_FDQ2_QMGR | W | 0h | This field specifies which Buffer Manager should be used for the 3rd Rx buffer in a host type packet |
| 11-0 | RX_HOST_FDQ2_QNUM | W | 0h | This field specifies which Free Descriptor / Buffer Pool should be used for the 3rd Rx buffer in a host type packet Table 101 -Rx Channel N Host Packet Configuration Registers B |

### 16.4.5.24 TXGCR5 Register (offset = 8A0h) [reset = 0h]

TXGCR5 is shown in Figure 16-175 and described in Table 16-187.

**Figure 16-175. TXGCR5 Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| TX_ENABLE | TX_TEARDOWN | Reserved | | | | | |
| R/W-0h | R/W-0h | | | | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| Reserved | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| Reserved | | TX_DEFAULT_QMGR | | TX_DEFAULT_QNUM | | | |
| | | W-0h | | W-0h | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| TX_DEFAULT_QNUM | | | | | | | |
| W-0h | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-187. TXGCR5 Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31 | TX_ENABLE | R/W | 0h | This field enables or disables the channel<br>0 = channel is disabled<br>1 = channel is enabled This field will be cleared after a channel teardown is complete. |
| 30 | TX_TEARDOWN | R/W | 0h | Setting this bit will request the channel to be torn down.<br>This field will remain set after a channel teardown is complete. |
| 13-12 | TX_DEFAULT_QMGR | W | 0h | This field controls the default queue manager number that will be used to queue teardown descriptors back to the host. |
| 11-0 | TX_DEFAULT_QNUM | W | 0h | This field controls the default queue number within the selected queue manager onto which teardown descriptors will be queued back to the host.<br>Table<br>98 -Tx Channel N Global Configuration Registers |

### 16.4.5.25 RXGCR5 Register (offset = 8A8h) [reset = 0h]

RXGCR5 is shown in Figure 16-176 and described in Table 16-188.

**Figure 16-176. RXGCR5 Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| RX_ENABLE | RX_TEARDOWN | RX_PAUSE | Reserved | | | | RX_ERROR_HANDLING |
| R/W-0h | R/W-0h | R/W-0h | | | | | W-0h |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| RX_SOP_OFFSET | | | | | | | |
| W-0h | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| RX_DEFAULT_DESC_TYPE | | RX_DEFAULT_RQ_QMGR | | RX_DEFAULT_RQ_QNUM | | | |
| W-0h | | W-0h | | W-0h | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| RX_DEFAULT_RQ_QNUM | | | | | | | |
| W-0h | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-188. RXGCR5 Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31 | RX_ENABLE | R/W | 0h | This field enables or disables the channel<br>0 = channel is disabled<br>1 = channel is enabled This field will be cleared after a channel teardown is complete. |
| 30 | RX_TEARDOWN | R/W | 0h | This field indicates whether or not an Rx teardown operation is complete.<br>This field should be cleared when a channel is initialized.<br>This field will be set after a channel teardown is complete. |
| 29 | RX_PAUSE | R/W | 0h | Setting this bit causes the CPPI DMA to be suspended for rx channels.<br>If a pause is being requested and the channel is not in a packet then drop the credit. |
| 24 | RX_ERROR_HANDLING | W | 0h | This bit controls the error handling mode for the channel and is only used when channel errors (i.e.<br>descriptor or buffer starvation occurs):<br>0 = Starvation errors result in dropping packet and reclaiming any used descriptor or buffer resources back to the original queues/pools they were allocated to<br>1 = Starvation errors result in subsequent re-try of the descriptor allocation operation.<br>In this mode, the DMA will return to the IDLE state without saving it's internal operational state back to the internal state RAM and without issuing an advance operation on the FIFO interface.<br>This results in the DMA re-initiating the FIFO block transfer at a later time with the intention that additional free buffers and/or descriptors will have been added.<br>Regardless of the value of this bit, the DMA will assert the cdma_rx_sof_overrun (for SOP) or cdma_rx_mof_overrun (for non-SOP) when |
| 23-16 | RX_SOP_OFFSET | W | 0h | This field specifies the number of bytes that are to be skipped in the SOP buffer before beginning to write the payload.<br>This value must be less than the minimum size of a buffer in the system.<br>Valid values are<br>0 - 255 bytes. |

### Table 16-188. RXGCR5 Register Field Descriptions (continued)

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 15-14 | RX_DEFAULT_DESC_TYPE | W | 0h | This field indicates the default descriptor type to use:<br>0 = Reserved<br>1 = Host<br>2 = Reserved<br>3 = Reserved The actual descriptor type that will be used for reception can be overridden by information provided in the CPPI FIFO data block. |
| 13-12 | RX_DEFAULT_RQ_QMGR | W | 0h | This field indicates the default receive queue manager that this channel should use.<br>The actual receive queue manager index can be overridden by information provided in the CPPI FIFO data block. |
| 11-0 | RX_DEFAULT_RQ_QNUM | W | 0h | This field indicates the default receive queue that this channel should use.<br>The actual receive queue that will be used for reception can be overridden by information provided in the CPPI FIFO data block. Table<br>99 -Rx Channel N Global Configuration Registers |

### 16.4.5.26 RXHPCRA5 Register (offset = 8ACh) [reset = 0h]

RXHPCRA5 is shown in Figure 16-177 and described in Table 16-189.

**Figure 16-177. RXHPCRA5 Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| Reserved | | RX_HOST_FDQ1_QMGR | | RX_HOST_FDQ1_QNUM | | | |
| | | W-0h | | W-0h | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| RX_HOST_FDQ1_QNUM | | | | | | | |
| W-0h | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| Reserved | | RX_HOST_FDQ0_QMGR | | RX_HOST_FDQ0_QNUM | | | |
| | | W-0h | | W-0h | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| RX_HOST_FDQ0_QNUM | | | | | | | |
| W-0h | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-189. RXHPCRA5 Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 29-28 | RX_HOST_FDQ1_QMGR | W | 0h | This field specifies which Buffer Manager should be used for the second Rx buffer in a host type packet. |
| 27-16 | RX_HOST_FDQ1_QNUM | W | 0h | This field specifies which Free Descriptor / Buffer Pool should be used for the 2nd Rx buffer in a host type packet |
| 13-12 | RX_HOST_FDQ0_QMGR | W | 0h | This field specifies which Buffer Manager should be used for the second Rx buffer in a host type packet. |
| 11-0 | RX_HOST_FDQ0_QNUM | W | 0h | This field specifies which Free Descriptor / Buffer Pool should be used for the 1st Rx buffer in a host type packet Table 100 -Rx Channel N Host Packet Configuration Registers A |

### 16.4.5.27 RXHPCRB5 Register (offset = 8B0h) [reset = 0h]

RXHPCRB5 is shown in Figure 16-178 and described in Table 16-190.

**Figure 16-178. RXHPCRB5 Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| Reserved | | RX_HOST_FDQ3_QMGR | | RX_HOST_FDQ3_QNUM | | | |
| | | W-0h | | W-0h | | | |
| **23** | **22** | **21** | **20** | **19** | **18** | **17** | **16** |
| RX_HOST_FDQ3_QNUM | | | | | | | |
| W-0h | | | | | | | |
| **15** | **14** | **13** | **12** | **11** | **10** | **9** | **8** |
| Reserved | | RX_HOST_FDQ2_QMGR | | RX_HOST_FDQ2_QNUM | | | |
| | | W-0h | | W-0h | | | |
| **7** | **6** | **5** | **4** | **3** | **2** | **1** | **0** |
| RX_HOST_FDQ2_QNUM | | | | | | | |
| W-0h | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-190. RXHPCRB5 Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 29-28 | RX_HOST_FDQ3_QMGR | W | 0h | This field specifies which Manager should be used for the 4th or later Rx buffers in a host type packet |
| 27-16 | RX_HOST_FDQ3_QNUM | W | 0h | This field specifies which Free Descriptor Queue should be used for the 4th or later Rx buffers in a host type packet |
| 13-12 | RX_HOST_FDQ2_QMGR | W | 0h | This field specifies which Buffer Manager should be used for the 3rd Rx buffer in a host type packet |
| 11-0 | RX_HOST_FDQ2_QNUM | W | 0h | This field specifies which Free Descriptor / Buffer Pool should be used for the 3rd Rx buffer in a host type packet Table 101 -Rx Channel N Host Packet Configuration Registers B |

### 16.4.5.28 TXGCR6 Register (offset = 8C0h) [reset = 0h]

TXGCR6 is shown in Figure 16-179 and described in Table 16-191.

**Figure 16-179. TXGCR6 Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| TX_ENABLE | TX_TEARDOWN | Reserved | | | | | |
| R/W-0h | R/W-0h | | | | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| Reserved | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| Reserved | | TX_DEFAULT_QMGR | | TX_DEFAULT_QNUM | | | |
| | | W-0h | | W-0h | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| TX_DEFAULT_QNUM | | | | | | | |
| W-0h | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-191. TXGCR6 Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31 | TX_ENABLE | R/W | 0h | This field enables or disables the channel<br>0 = channel is disabled<br>1 = channel is enabled This field will be cleared after a channel teardown is complete. |
| 30 | TX_TEARDOWN | R/W | 0h | Setting this bit will request the channel to be torn down.<br>This field will remain set after a channel teardown is complete. |
| 13-12 | TX_DEFAULT_QMGR | W | 0h | This field controls the default queue manager number that will be used to queue teardown descriptors back to the host. |
| 11-0 | TX_DEFAULT_QNUM | W | 0h | This field controls the default queue number within the selected queue manager onto which teardown descriptors will be queued back to the host.<br>Table<br>98 -Tx Channel N Global Configuration Registers |

### 16.4.5.29 RXGCR6 Register (offset = 8C8h) [reset = 0h]

RXGCR6 is shown in Figure 16-180 and described in Table 16-192.

**Figure 16-180. RXGCR6 Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| RX_ENABLE | RX_TEARDOWN | RX_PAUSE | Reserved | | | | RX_ERROR_HANDLING |
| R/W-0h | R/W-0h | R/W-0h | | | | | W-0h |
| **23** | **22** | **21** | **20** | **19** | **18** | **17** | **16** |
| RX_SOP_OFFSET | | | | | | | |
| W-0h | | | | | | | |
| **15** | **14** | **13** | **12** | **11** | **10** | **9** | **8** |
| RX_DEFAULT_DESC_TYPE | | RX_DEFAULT_RQ_QMGR | | RX_DEFAULT_RQ_QNUM | | | |
| W-0h | | W-0h | | W-0h | | | |
| **7** | **6** | **5** | **4** | **3** | **2** | **1** | **0** |
| RX_DEFAULT_RQ_QNUM | | | | | | | |
| W-0h | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-192. RXGCR6 Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31 | RX_ENABLE | R/W | 0h | This field enables or disables the channel<br>0 = channel is disabled<br>1 = channel is enabled This field will be cleared after a channel teardown is complete. |
| 30 | RX_TEARDOWN | R/W | 0h | This field indicates whether or not an Rx teardown operation is complete.<br>This field should be cleared when a channel is initialized.<br>This field will be set after a channel teardown is complete. |
| 29 | RX_PAUSE | R/W | 0h | Setting this bit causes the CPPI DMA to be suspended for rx channels.<br>If a pause is being requested and the channel is not in a packet then drop the credit. |
| 24 | RX_ERROR_HANDLING | W | 0h | This bit controls the error handling mode for the channel and is only used when channel errors (i.e.<br>descriptor or buffer starvation occurs):<br>0 = Starvation errors result in dropping packet and reclaiming any used descriptor or buffer resources back to the original queues/pools they were allocated to<br>1 = Starvation errors result in subsequent re-try of the descriptor allocation operation.<br>In this mode, the DMA will return to the IDLE state without saving it's internal operational state back to the internal state RAM and without issuing an advance operation on the FIFO interface.<br>This results in the DMA re-initiating the FIFO block transfer at a later time with the intention that additional free buffers and/or descriptors will have been added.<br>Regardless of the value of this bit, the DMA will assert the cdma_rx_sof_overrun (for SOP) or cdma_rx_mof_overrun (for non-SOP) when |
| 23-16 | RX_SOP_OFFSET | W | 0h | This field specifies the number of bytes that are to be skipped in the SOP buffer before beginning to write the payload.<br>This value must be less than the minimum size of a buffer in the system.<br>Valid values are<br>0 - 255 bytes. |

### Table 16-192. RXGCR6 Register Field Descriptions (continued)

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 15-14 | RX_DEFAULT_DESC_TYPE | W | 0h | This field indicates the default descriptor type to use:<br>0 = Reserved<br>1 = Host<br>2 = Reserved<br>3 = Reserved The actual descriptor type that will be used for reception can be overridden by information provided in the CPPI FIFO data block. |
| 13-12 | RX_DEFAULT_RQ_QMGR | W | 0h | This field indicates the default receive queue manager that this channel should use.<br>The actual receive queue manager index can be overridden by information provided in the CPPI FIFO data block. |
| 11-0 | RX_DEFAULT_RQ_QNUM | W | 0h | This field indicates the default receive queue that this channel should use.<br>The actual receive queue that will be used for reception can be overridden by information provided in the CPPI FIFO data block. Table<br>99 -Rx Channel N Global Configuration Registers |

### 16.4.5.30 RXHPCRA6 Register (offset = 8CCh) [reset = 0h]

RXHPCRA6 is shown in Figure 16-181 and described in Table 16-193.

**Figure 16-181. RXHPCRA6 Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| Reserved | | RX_HOST_FDQ1_QMGR | | RX_HOST_FDQ1_QNUM | | | |
| | | W-0h | | W-0h | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| RX_HOST_FDQ1_QNUM | | | | | | | |
| W-0h | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| Reserved | | RX_HOST_FDQ0_QMGR | | RX_HOST_FDQ0_QNUM | | | |
| | | W-0h | | W-0h | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| RX_HOST_FDQ0_QNUM | | | | | | | |
| W-0h | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-193. RXHPCRA6 Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 29-28 | RX_HOST_FDQ1_QMGR | W | 0h | This field specifies which Buffer Manager should be used for the second Rx buffer in a host type packet. |
| 27-16 | RX_HOST_FDQ1_QNUM | W | 0h | This field specifies which Free Descriptor / Buffer Pool should be used for the 2nd Rx buffer in a host type packet |
| 13-12 | RX_HOST_FDQ0_QMGR | W | 0h | This field specifies which Buffer Manager should be used for the second Rx buffer in a host type packet. |
| 11-0 | RX_HOST_FDQ0_QNUM | W | 0h | This field specifies which Free Descriptor / Buffer Pool should be used for the 1st Rx buffer in a host type packet Table 100 -Rx Channel N Host Packet Configuration Registers A |

### 16.4.5.31 RXHPCRB6 Register (offset = 8D0h) [reset = 0h]

RXHPCRB6 is shown in Figure 16-182 and described in Table 16-194.

**Figure 16-182. RXHPCRB6 Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| Reserved | | RX_HOST_FDQ3_QMGR | | RX_HOST_FDQ3_QNUM | | | |
| | | W-0h | | W-0h | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| RX_HOST_FDQ3_QNUM | | | | | | | |
| W-0h | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| Reserved | | RX_HOST_FDQ2_QMGR | | RX_HOST_FDQ2_QNUM | | | |
| | | W-0h | | W-0h | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| RX_HOST_FDQ2_QNUM | | | | | | | |
| W-0h | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-194. RXHPCRB6 Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 29-28 | RX_HOST_FDQ3_QMGR | W | 0h | This field specifies which Manager should be used for the 4th or later Rx buffers in a host type packet |
| 27-16 | RX_HOST_FDQ3_QNUM | W | 0h | This field specifies which Free Descriptor Queue should be used for the 4th or later Rx buffers in a host type packet |
| 13-12 | RX_HOST_FDQ2_QMGR | W | 0h | This field specifies which Buffer Manager should be used for the 3rd Rx buffer in a host type packet |
| 11-0 | RX_HOST_FDQ2_QNUM | W | 0h | This field specifies which Free Descriptor / Buffer Pool should be used for the 3rd Rx buffer in a host type packet Table 101 -Rx Channel N Host Packet Configuration Registers B |

### 16.4.5.32 TXGCR7 Register (offset = 8E0h) [reset = 0h]

TXGCR7 is shown in Figure 16-183 and described in Table 16-195.

**Figure 16-183. TXGCR7 Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| TX_ENABLE | TX_TEARDOWN | Reserved | | | | | |
| R/W-0h | R/W-0h | | | | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| Reserved | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| Reserved | | TX_DEFAULT_QMGR | | TX_DEFAULT_QNUM | | | |
| | | W-0h | | W-0h | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| TX_DEFAULT_QNUM | | | | | | | |
| W-0h | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-195. TXGCR7 Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31 | TX_ENABLE | R/W | 0h | This field enables or disables the channel<br>0 = channel is disabled<br>1 = channel is enabled This field will be cleared after a channel teardown is complete. |
| 30 | TX_TEARDOWN | R/W | 0h | Setting this bit will request the channel to be torn down.<br>This field will remain set after a channel teardown is complete. |
| 13-12 | TX_DEFAULT_QMGR | W | 0h | This field controls the default queue manager number that will be used to queue teardown descriptors back to the host. |
| 11-0 | TX_DEFAULT_QNUM | W | 0h | This field controls the default queue number within the selected queue manager onto which teardown descriptors will be queued back to the host.<br>Table<br>98 -Tx Channel N Global Configuration Registers |

### 16.4.5.33 RXGCR7 Register (offset = 8E8h) [reset = 0h]

RXGCR7 is shown in Figure 16-184 and described in Table 16-196.

**Figure 16-184. RXGCR7 Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| RX_ENABLE | RX_TEARDOWN | RX_PAUSE | Reserved | | | | RX_ERROR_HANDLING |
| R/W-0h | R/W-0h | R/W-0h | | | | | W-0h |
| **23** | **22** | **21** | **20** | **19** | **18** | **17** | **16** |
| RX_SOP_OFFSET | | | | | | | |
| W-0h | | | | | | | |
| **15** | **14** | **13** | **12** | **11** | **10** | **9** | **8** |
| RX_DEFAULT_DESC_TYPE | | RX_DEFAULT_RQ_QMGR | | RX_DEFAULT_RQ_QNUM | | | |
| W-0h | | W-0h | | W-0h | | | |
| **7** | **6** | **5** | **4** | **3** | **2** | **1** | **0** |
| RX_DEFAULT_RQ_QNUM | | | | | | | |
| W-0h | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-196. RXGCR7 Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31 | RX_ENABLE | R/W | 0h | This field enables or disables the channel<br>0 = channel is disabled<br>1 = channel is enabled This field will be cleared after a channel teardown is complete. |
| 30 | RX_TEARDOWN | R/W | 0h | This field indicates whether or not an Rx teardown operation is complete.<br>This field should be cleared when a channel is initialized.<br>This field will be set after a channel teardown is complete. |
| 29 | RX_PAUSE | R/W | 0h | Setting this bit causes the CPPI DMA to be suspended for rx channels.<br>If a pause is being requested and the channel is not in a packet then drop the credit. |
| 24 | RX_ERROR_HANDLING | W | 0h | This bit controls the error handling mode for the channel and is only used when channel errors (i.e.<br>descriptor or buffer starvation occurs):<br>0 = Starvation errors result in dropping packet and reclaiming any used descriptor or buffer resources back to the original queues/pools they were allocated to<br>1 = Starvation errors result in subsequent re-try of the descriptor allocation operation.<br>In this mode, the DMA will return to the IDLE state without saving it's internal operational state back to the internal state RAM and without issuing an advance operation on the FIFO interface.<br>This results in the DMA re-initiating the FIFO block transfer at a later time with the intention that additional free buffers and/or descriptors will have been added.<br>Regardless of the value of this bit, the DMA will assert the cdma_rx_sof_overrun (for SOP) or cdma_rx_mof_overrun (for non-SOP) when |
| 23-16 | RX_SOP_OFFSET | W | 0h | This field specifies the number of bytes that are to be skipped in the SOP buffer before beginning to write the payload.<br>This value must be less than the minimum size of a buffer in the system.<br>Valid values are<br>0 - 255 bytes. |

**Table 16-196. RXGCR7 Register Field Descriptions (continued)**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 15-14 | RX_DEFAULT_DESC_TYPE | W | 0h | This field indicates the default descriptor type to use:<br>0 = Reserved<br>1 = Host<br>2 = Reserved<br>3 = Reserved The actual descriptor type that will be used for reception can be overridden by information provided in the CPPI FIFO data block. |
| 13-12 | RX_DEFAULT_RQ_QMGR | W | 0h | This field indicates the default receive queue manager that this channel should use.<br>The actual receive queue manager index can be overridden by information provided in the CPPI FIFO data block. |
| 11-0 | RX_DEFAULT_RQ_QNUM | W | 0h | This field indicates the default receive queue that this channel should use.<br>The actual receive queue that will be used for reception can be overridden by information provided in the CPPI FIFO data block. Table<br>99 -Rx Channel N Global Configuration Registers |

### 16.4.5.34 RXHPCRA7 Register (offset = 8ECh) [reset = 0h]

RXHPCRA7 is shown in Figure 16-185 and described in Table 16-197.

**Figure 16-185. RXHPCRA7 Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| Reserved | | RX_HOST_FDQ1_QMGR | | RX_HOST_FDQ1_QNUM | | | |
| | | W-0h | | W-0h | | | |
| **23** | **22** | **21** | **20** | **19** | **18** | **17** | **16** |
| RX_HOST_FDQ1_QNUM | | | | | | | |
| W-0h | | | | | | | |
| **15** | **14** | **13** | **12** | **11** | **10** | **9** | **8** |
| Reserved | | RX_HOST_FDQ0_QMGR | | RX_HOST_FDQ0_QNUM | | | |
| | | W-0h | | W-0h | | | |
| **7** | **6** | **5** | **4** | **3** | **2** | **1** | **0** |
| RX_HOST_FDQ0_QNUM | | | | | | | |
| W-0h | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-197. RXHPCRA7 Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 29-28 | RX_HOST_FDQ1_QMGR | W | 0h | This field specifies which Buffer Manager should be used for the second Rx buffer in a host type packet. |
| 27-16 | RX_HOST_FDQ1_QNUM | W | 0h | This field specifies which Free Descriptor / Buffer Pool should be used for the 2nd Rx buffer in a host type packet |
| 13-12 | RX_HOST_FDQ0_QMGR | W | 0h | This field specifies which Buffer Manager should be used for the second Rx buffer in a host type packet. |
| 11-0 | RX_HOST_FDQ0_QNUM | W | 0h | This field specifies which Free Descriptor / Buffer Pool should be used for the 1st Rx buffer in a host type packet Table 100 -Rx Channel N Host Packet Configuration Registers A |

### 16.4.5.35 RXHPCRB7 Register (offset = 8F0h) [reset = 0h]

RXHPCRB7 is shown in Figure 16-186 and described in Table 16-198.

**Figure 16-186. RXHPCRB7 Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| Reserved | | RX_HOST_FDQ3_QMGR | | RX_HOST_FDQ3_QNUM | | | |
| | | W-0h | | W-0h | | | |
| **23** | **22** | **21** | **20** | **19** | **18** | **17** | **16** |
| RX_HOST_FDQ3_QNUM | | | | | | | |
| W-0h | | | | | | | |
| **15** | **14** | **13** | **12** | **11** | **10** | **9** | **8** |
| Reserved | | RX_HOST_FDQ2_QMGR | | RX_HOST_FDQ2_QNUM | | | |
| | | W-0h | | W-0h | | | |
| **7** | **6** | **5** | **4** | **3** | **2** | **1** | **0** |
| RX_HOST_FDQ2_QNUM | | | | | | | |
| W-0h | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-198. RXHPCRB7 Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 29-28 | RX_HOST_FDQ3_QMGR | W | 0h | This field specifies which Manager should be used for the 4th or later Rx buffers in a host type packet |
| 27-16 | RX_HOST_FDQ3_QNUM | W | 0h | This field specifies which Free Descriptor Queue should be used for the 4th or later Rx buffers in a host type packet |
| 13-12 | RX_HOST_FDQ2_QMGR | W | 0h | This field specifies which Buffer Manager should be used for the 3rd Rx buffer in a host type packet |
| 11-0 | RX_HOST_FDQ2_QNUM | W | 0h | This field specifies which Free Descriptor / Buffer Pool should be used for the 3rd Rx buffer in a host type packet Table 101 -Rx Channel N Host Packet Configuration Registers B |

### 16.4.5.36 TXGCR8 Register (offset = 900h) [reset = 0h]

TXGCR8 is shown in Figure 16-187 and described in Table 16-199.

**Figure 16-187. TXGCR8 Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| TX_ENABLE | TX_TEARDOWN | Reserved | | | | | |
| R/W-0h | R/W-0h | | | | | | |
| **23** | **22** | **21** | **20** | **19** | **18** | **17** | **16** |
| Reserved | | | | | | | |
| **15** | **14** | **13** | **12** | **11** | **10** | **9** | **8** |
| Reserved | | TX_DEFAULT_QMGR | | TX_DEFAULT_QNUM | | | |
| | | W-0h | | W-0h | | | |
| **7** | **6** | **5** | **4** | **3** | **2** | **1** | **0** |
| TX_DEFAULT_QNUM | | | | | | | |
| W-0h | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-199. TXGCR8 Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31 | TX_ENABLE | R/W | 0h | This field enables or disables the channel<br>0 = channel is disabled<br>1 = channel is enabled This field will be cleared after a channel teardown is complete. |
| 30 | TX_TEARDOWN | R/W | 0h | Setting this bit will request the channel to be torn down.<br>This field will remain set after a channel teardown is complete. |
| 13-12 | TX_DEFAULT_QMGR | W | 0h | This field controls the default queue manager number that will be used to queue teardown descriptors back to the host. |
| 11-0 | TX_DEFAULT_QNUM | W | 0h | This field controls the default queue number within the selected queue manager onto which teardown descriptors will be queued back to the host.<br>Table<br>98 -Tx Channel N Global Configuration Registers |

### 16.4.5.37 RXGCR8 Register (offset = 908h) [reset = 0h]

RXGCR8 is shown in Figure 16-188 and described in Table 16-200.

**Figure 16-188. RXGCR8 Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| RX_ENABLE | RX_TEARDOWN | RX_PAUSE | Reserved | | | | RX_ERROR_HANDLING |
| R/W-0h | R/W-0h | R/W-0h | | | | | W-0h |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| RX_SOP_OFFSET | | | | | | | |
| W-0h | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| RX_DEFAULT_DESC_TYPE | | RX_DEFAULT_RQ_QMGR | | RX_DEFAULT_RQ_QNUM | | | |
| W-0h | | W-0h | | W-0h | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| RX_DEFAULT_RQ_QNUM | | | | | | | |
| W-0h | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-200. RXGCR8 Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31 | RX_ENABLE | R/W | 0h | This field enables or disables the channel<br>0 = channel is disabled<br>1 = channel is enabled This field will be cleared after a channel teardown is complete. |
| 30 | RX_TEARDOWN | R/W | 0h | This field indicates whether or not an Rx teardown operation is complete.<br>This field should be cleared when a channel is initialized.<br>This field will be set after a channel teardown is complete. |
| 29 | RX_PAUSE | R/W | 0h | Setting this bit causes the CPPI DMA to be suspended for rx channels.<br>If a pause is being requested and the channel is not in a packet then drop the credit. |
| 24 | RX_ERROR_HANDLING | W | 0h | This bit controls the error handling mode for the channel and is only used when channel errors (i.e.<br>descriptor or buffer starvation occurs):<br>0 = Starvation errors result in dropping packet and reclaiming any used descriptor or buffer resources back to the original queues/pools they were allocated to<br>1 = Starvation errors result in subsequent re-try of the descriptor allocation operation.<br>In this mode, the DMA will return to the IDLE state without saving it's internal operational state back to the internal state RAM and without issuing an advance operation on the FIFO interface.<br>This results in the DMA re-initiating the FIFO block transfer at a later time with the intention that additional free buffers and/or descriptors will have been added.<br>Regardless of the value of this bit, the DMA will assert the cdma_rx_sof_overrun (for SOP) or cdma_rx_mof_overrun (for non-SOP) when |
| 23-16 | RX_SOP_OFFSET | W | 0h | This field specifies the number of bytes that are to be skipped in the SOP buffer before beginning to write the payload.<br>This value must be less than the minimum size of a buffer in the system.<br>Valid values are<br>0 - 255 bytes. |

**Table 16-200. RXGCR8 Register Field Descriptions (continued)**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 15-14 | RX_DEFAULT_DESC_TYPE | W | 0h | This field indicates the default descriptor type to use:<br>0 = Reserved<br>1 = Host<br>2 = Reserved<br>3 = Reserved The actual descriptor type that will be used for reception can be overridden by information provided in the CPPI FIFO data block. |
| 13-12 | RX_DEFAULT_RQ_QMGR | W | 0h | This field indicates the default receive queue manager that this channel should use.<br>The actual receive queue manager index can be overridden by information provided in the CPPI FIFO data block. |
| 11-0 | RX_DEFAULT_RQ_QNUM | W | 0h | This field indicates the default receive queue that this channel should use.<br>The actual receive queue that will be used for reception can be overridden by information provided in the CPPI FIFO data block. Table<br>99 -Rx Channel N Global Configuration Registers |

### 16.4.5.38 RXHPCRA8 Register (offset = 90Ch) [reset = 0h]

RXHPCRA8 is shown in Figure 16-189 and described in Table 16-201.

**Figure 16-189. RXHPCRA8 Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| Reserved | | RX_HOST_FDQ1_QMGR | | RX_HOST_FDQ1_QNUM | | | |
| | | W-0h | | W-0h | | | |
| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
| RX_HOST_FDQ1_QNUM | | | | | | | |
| W-0h | | | | | | | |
| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
| Reserved | | RX_HOST_FDQ0_QMGR | | RX_HOST_FDQ0_QNUM | | | |
| | | W-0h | | W-0h | | | |
| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
| RX_HOST_FDQ0_QNUM | | | | | | | |
| W-0h | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-201. RXHPCRA8 Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 29-28 | RX_HOST_FDQ1_QMGR | W | 0h | This field specifies which Buffer Manager should be used for the second Rx buffer in a host type packet. |
| 27-16 | RX_HOST_FDQ1_QNUM | W | 0h | This field specifies which Free Descriptor / Buffer Pool should be used for the 2nd Rx buffer in a host type packet |
| 13-12 | RX_HOST_FDQ0_QMGR | W | 0h | This field specifies which Buffer Manager should be used for the second Rx buffer in a host type packet. |
| 11-0 | RX_HOST_FDQ0_QNUM | W | 0h | This field specifies which Free Descriptor / Buffer Pool should be used for the 1st Rx buffer in a host type packet Table 100 -Rx Channel N Host Packet Configuration Registers A |

### 16.4.5.39 RXHPCRB8 Register (offset = 910h) [reset = 0h]

RXHPCRB8 is shown in Figure 16-190 and described in Table 16-202.

**Figure 16-190. RXHPCRB8 Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| Reserved | | RX_HOST_FDQ3_QMGR | | RX_HOST_FDQ3_QNUM | | | |
| | | W-0h | | W-0h | | | |
| **23** | **22** | **21** | **20** | **19** | **18** | **17** | **16** |
| RX_HOST_FDQ3_QNUM | | | | | | | |
| W-0h | | | | | | | |
| **15** | **14** | **13** | **12** | **11** | **10** | **9** | **8** |
| Reserved | | RX_HOST_FDQ2_QMGR | | RX_HOST_FDQ2_QNUM | | | |
| | | W-0h | | W-0h | | | |
| **7** | **6** | **5** | **4** | **3** | **2** | **1** | **0** |
| RX_HOST_FDQ2_QNUM | | | | | | | |
| W-0h | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-202. RXHPCRB8 Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 29-28 | RX_HOST_FDQ3_QMGR | W | 0h | This field specifies which Manager should be used for the 4th or later Rx buffers in a host type packet |
| 27-16 | RX_HOST_FDQ3_QNUM | W | 0h | This field specifies which Free Descriptor Queue should be used for the 4th or later Rx buffers in a host type packet |
| 13-12 | RX_HOST_FDQ2_QMGR | W | 0h | This field specifies which Buffer Manager should be used for the 3rd Rx buffer in a host type packet |
| 11-0 | RX_HOST_FDQ2_QNUM | W | 0h | This field specifies which Free Descriptor / Buffer Pool should be used for the 3rd Rx buffer in a host type packet Table 101 -Rx Channel N Host Packet Configuration Registers B |

### 16.4.5.40 TXGCR9 Register (offset = 920h) [reset = 0h]

TXGCR9 is shown in Figure 16-191 and described in Table 16-203.

**Figure 16-191. TXGCR9 Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| TX_ENABLE | TX_TEARDOWN | Reserved | | | | | |
| R/W-0h | R/W-0h | | | | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| Reserved | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| Reserved | | TX_DEFAULT_QMGR | | TX_DEFAULT_QNUM | | | |
| | | W-0h | | W-0h | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| TX_DEFAULT_QNUM | | | | | | | |
| W-0h | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-203. TXGCR9 Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31 | TX_ENABLE | R/W | 0h | This field enables or disables the channel<br>0 = channel is disabled<br>1 = channel is enabled This field will be cleared after a channel teardown is complete. |
| 30 | TX_TEARDOWN | R/W | 0h | Setting this bit will request the channel to be torn down.<br>This field will remain set after a channel teardown is complete. |
| 13-12 | TX_DEFAULT_QMGR | W | 0h | This field controls the default queue manager number that will be used to queue teardown descriptors back to the host. |
| 11-0 | TX_DEFAULT_QNUM | W | 0h | This field controls the default queue number within the selected queue manager onto which teardown descriptors will be queued back to the host.<br>Table<br>98 -Tx Channel N Global Configuration Registers |

### 16.4.5.41 RXGCR9 Register (offset = 928h) [reset = 0h]

RXGCR9 is shown in Figure 16-192 and described in Table 16-204.

**Figure 16-192. RXGCR9 Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| RX_ENABLE | RX_TEARDOWN | RX_PAUSE | Reserved | | | | RX_ERROR_HANDLING |
| R/W-0h | R/W-0h | R/W-0h | | | | | W-0h |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| RX_SOP_OFFSET | | | | | | | |
| W-0h | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| RX_DEFAULT_DESC_TYPE | | RX_DEFAULT_RQ_QMGR | | RX_DEFAULT_RQ_QNUM | | | |
| W-0h | | W-0h | | W-0h | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| RX_DEFAULT_RQ_QNUM | | | | | | | |
| W-0h | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-204. RXGCR9 Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31 | RX_ENABLE | R/W | 0h | This field enables or disables the channel<br>0 = channel is disabled<br>1 = channel is enabled This field will be cleared after a channel teardown is complete. |
| 30 | RX_TEARDOWN | R/W | 0h | This field indicates whether or not an Rx teardown operation is complete.<br>This field should be cleared when a channel is initialized.<br>This field will be set after a channel teardown is complete. |
| 29 | RX_PAUSE | R/W | 0h | Setting this bit causes the CPPI DMA to be suspended for rx channels.<br>If a pause is being requested and the channel is not in a packet then drop the credit. |
| 24 | RX_ERROR_HANDLING | W | 0h | This bit controls the error handling mode for the channel and is only used when channel errors (i.e.<br>descriptor or buffer starvation occurs):<br>0 = Starvation errors result in dropping packet and reclaiming any used descriptor or buffer resources back to the original queues/pools they were allocated to<br>1 = Starvation errors result in subsequent re-try of the descriptor allocation operation.<br>In this mode, the DMA will return to the IDLE state without saving it's internal operational state back to the internal state RAM and without issuing an advance operation on the FIFO interface.<br>This results in the DMA re-initiating the FIFO block transfer at a later time with the intention that additional free buffers and/or descriptors will have been added.<br>Regardless of the value of this bit, the DMA will assert the cdma_rx_sof_overrun (for SOP) or cdma_rx_mof_overrun (for non-SOP) when |
| 23-16 | RX_SOP_OFFSET | W | 0h | This field specifies the number of bytes that are to be skipped in the SOP buffer before beginning to write the payload.<br>This value must be less than the minimum size of a buffer in the system.<br>Valid values are<br>0 - 255 bytes. |

**Table 16-204. RXGCR9 Register Field Descriptions (continued)**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 15-14 | RX_DEFAULT_DESC_TYPE | W | 0h | This field indicates the default descriptor type to use:<br>0 = Reserved<br>1 = Host<br>2 = Reserved<br>3 = Reserved The actual descriptor type that will be used for reception can be overridden by information provided in the CPPI FIFO data block. |
| 13-12 | RX_DEFAULT_RQ_QMGR | W | 0h | This field indicates the default receive queue manager that this channel should use.<br>The actual receive queue manager index can be overridden by information provided in the CPPI FIFO data block. |
| 11-0 | RX_DEFAULT_RQ_QNUM | W | 0h | This field indicates the default receive queue that this channel should use.<br>The actual receive queue that will be used for reception can be overridden by information provided in the CPPI FIFO data block. Table<br>99 -Rx Channel N Global Configuration Registers |

### 16.4.5.42 RXHPCRA9 Register (offset = 92Ch) [reset = 0h]

RXHPCRA9 is shown in Figure 16-193 and described in Table 16-205.

**Figure 16-193. RXHPCRA9 Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| Reserved | | RX_HOST_FDQ1_QMGR | | RX_HOST_FDQ1_QNUM | | | |
| | | W-0h | | W-0h | | | |
| **23** | **22** | **21** | **20** | **19** | **18** | **17** | **16** |
| RX_HOST_FDQ1_QNUM | | | | | | | |
| W-0h | | | | | | | |
| **15** | **14** | **13** | **12** | **11** | **10** | **9** | **8** |
| Reserved | | RX_HOST_FDQ0_QMGR | | RX_HOST_FDQ0_QNUM | | | |
| | | W-0h | | W-0h | | | |
| **7** | **6** | **5** | **4** | **3** | **2** | **1** | **0** |
| RX_HOST_FDQ0_QNUM | | | | | | | |
| W-0h | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-205. RXHPCRA9 Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 29-28 | RX_HOST_FDQ1_QMGR | W | 0h | This field specifies which Buffer Manager should be used for the second Rx buffer in a host type packet. |
| 27-16 | RX_HOST_FDQ1_QNUM | W | 0h | This field specifies which Free Descriptor / Buffer Pool should be used for the 2nd Rx buffer in a host type packet |
| 13-12 | RX_HOST_FDQ0_QMGR | W | 0h | This field specifies which Buffer Manager should be used for the second Rx buffer in a host type packet. |
| 11-0 | RX_HOST_FDQ0_QNUM | W | 0h | This field specifies which Free Descriptor / Buffer Pool should be used for the 1st Rx buffer in a host type packet Table 100 -Rx Channel N Host Packet Configuration Registers A |

### 16.4.5.43 RXHPCRB9 Register (offset = 930h) [reset = 0h]

RXHPCRB9 is shown in Figure 16-194 and described in Table 16-206.

**Figure 16-194. RXHPCRB9 Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| Reserved | | RX_HOST_FDQ3_QMGR | | RX_HOST_FDQ3_QNUM | | | |
| | | W-0h | | W-0h | | | |
| **23** | **22** | **21** | **20** | **19** | **18** | **17** | **16** |
| RX_HOST_FDQ3_QNUM | | | | | | | |
| W-0h | | | | | | | |
| **15** | **14** | **13** | **12** | **11** | **10** | **9** | **8** |
| Reserved | | RX_HOST_FDQ2_QMGR | | RX_HOST_FDQ2_QNUM | | | |
| | | W-0h | | W-0h | | | |
| **7** | **6** | **5** | **4** | **3** | **2** | **1** | **0** |
| RX_HOST_FDQ2_QNUM | | | | | | | |
| W-0h | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-206. RXHPCRB9 Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 29-28 | RX_HOST_FDQ3_QMGR | W | 0h | This field specifies which Manager should be used for the 4th or later Rx buffers in a host type packet |
| 27-16 | RX_HOST_FDQ3_QNUM | W | 0h | This field specifies which Free Descriptor Queue should be used for the 4th or later Rx buffers in a host type packet |
| 13-12 | RX_HOST_FDQ2_QMGR | W | 0h | This field specifies which Buffer Manager should be used for the 3rd Rx buffer in a host type packet |
| 11-0 | RX_HOST_FDQ2_QNUM | W | 0h | This field specifies which Free Descriptor / Buffer Pool should be used for the 3rd Rx buffer in a host type packet Table 101 -Rx Channel N Host Packet Configuration Registers B |

### 16.4.5.44 TXGCR10 Register (offset = 940h) [reset = 0h]

TXGCR10 is shown in Figure 16-195 and described in Table 16-207.

**Figure 16-195. TXGCR10 Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| TX_ENABLE | TX_TEARDOWN | Reserved | | | | | |
| R/W-0h | R/W-0h | | | | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| Reserved | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| Reserved | | TX_DEFAULT_QMGR | | TX_DEFAULT_QNUM | | | |
| | | W-0h | | W-0h | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| TX_DEFAULT_QNUM | | | | | | | |
| W-0h | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-207. TXGCR10 Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31 | TX_ENABLE | R/W | 0h | This field enables or disables the channel<br>0 = channel is disabled<br>1 = channel is enabled This field will be cleared after a channel teardown is complete. |
| 30 | TX_TEARDOWN | R/W | 0h | Setting this bit will request the channel to be torn down.<br>This field will remain set after a channel teardown is complete. |
| 13-12 | TX_DEFAULT_QMGR | W | 0h | This field controls the default queue manager number that will be used to queue teardown descriptors back to the host. |
| 11-0 | TX_DEFAULT_QNUM | W | 0h | This field controls the default queue number within the selected queue manager onto which teardown descriptors will be queued back to the host.<br>Table<br>98 -Tx Channel N Global Configuration Registers |

### 16.4.5.45 RXGCR10 Register (offset = 948h) [reset = 0h]

RXGCR10 is shown in Figure 16-196 and described in Table 16-208.

**Figure 16-196. RXGCR10 Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| RX_ENABLE | RX_TEARDOWN | RX_PAUSE | Reserved | | | | RX_ERROR_HANDLING |
| R/W-0h | R/W-0h | R/W-0h | | | | | W-0h |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| RX_SOP_OFFSET | | | | | | | |
| W-0h | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| RX_DEFAULT_DESC_TYPE | | RX_DEFAULT_RQ_QMGR | | RX_DEFAULT_RQ_QNUM | | | |
| W-0h | | W-0h | | W-0h | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| RX_DEFAULT_RQ_QNUM | | | | | | | |
| W-0h | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-208. RXGCR10 Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31 | RX_ENABLE | R/W | 0h | This field enables or disables the channel<br>0 = channel is disabled<br>1 = channel is enabled This field will be cleared after a channel teardown is complete. |
| 30 | RX_TEARDOWN | R/W | 0h | This field indicates whether or not an Rx teardown operation is complete.<br>This field should be cleared when a channel is initialized.<br>This field will be set after a channel teardown is complete. |
| 29 | RX_PAUSE | R/W | 0h | Setting this bit causes the CPPI DMA to be suspended for rx channels.<br>If a pause is being requested and the channel is not in a packet then drop the credit. |
| 24 | RX_ERROR_HANDLING | W | 0h | This bit controls the error handling mode for the channel and is only used when channel errors (i.e.<br>descriptor or buffer starvation occurs):<br>0 = Starvation errors result in dropping packet and reclaiming any used descriptor or buffer resources back to the original queues/pools they were allocated to<br>1 = Starvation errors result in subsequent re-try of the descriptor allocation operation.<br>In this mode, the DMA will return to the IDLE state without saving it's internal operational state back to the internal state RAM and without issuing an advance operation on the FIFO interface.<br>This results in the DMA re-initiating the FIFO block transfer at a later time with the intention that additional free buffers and/or descriptors will have been added.<br>Regardless of the value of this bit, the DMA will assert the cdma_rx_sof_overrun (for SOP) or cdma_rx_mof_overrun (for non-SOP) when |
| 23-16 | RX_SOP_OFFSET | W | 0h | This field specifies the number of bytes that are to be skipped in the SOP buffer before beginning to write the payload.<br>This value must be less than the minimum size of a buffer in the system.<br>Valid values are<br>0 - 255 bytes. |

**Table 16-208. RXGCR10 Register Field Descriptions (continued)**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 15-14 | RX_DEFAULT_DESC_TYPE | W | 0h | This field indicates the default descriptor type to use:<br>0 = Reserved<br>1 = Host<br>2 = Reserved<br>3 = Reserved The actual descriptor type that will be used for reception can be overridden by information provided in the CPPI FIFO data block. |
| 13-12 | RX_DEFAULT_RQ_QMGR | W | 0h | This field indicates the default receive queue manager that this channel should use.<br>The actual receive queue manager index can be overridden by information provided in the CPPI FIFO data block. |
| 11-0 | RX_DEFAULT_RQ_QNUM | W | 0h | This field indicates the default receive queue that this channel should use.<br>The actual receive queue that will be used for reception can be overridden by information provided in the CPPI FIFO data block.<br>Table<br>99 -Rx Channel N Global Configuration Registers |

### 16.4.5.46 RXHPCRA10 Register (offset = 94Ch) [reset = 0h]

RXHPCRA10 is shown in Figure 16-197 and described in Table 16-209.

**Figure 16-197. RXHPCRA10 Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| Reserved | | RX_HOST_FDQ1_QMGR | | RX_HOST_FDQ1_QNUM | | | |
| | | W-0h | | W-0h | | | |
| **23** | **22** | **21** | **20** | **19** | **18** | **17** | **16** |
| RX_HOST_FDQ1_QNUM | | | | | | | |
| W-0h | | | | | | | |
| **15** | **14** | **13** | **12** | **11** | **10** | **9** | **8** |
| Reserved | | RX_HOST_FDQ0_QMGR | | RX_HOST_FDQ0_QNUM | | | |
| | | W-0h | | W-0h | | | |
| **7** | **6** | **5** | **4** | **3** | **2** | **1** | **0** |
| RX_HOST_FDQ0_QNUM | | | | | | | |
| W-0h | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-209. RXHPCRA10 Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 29-28 | RX_HOST_FDQ1_QMGR | W | 0h | This field specifies which Buffer Manager should be used for the second Rx buffer in a host type packet. |
| 27-16 | RX_HOST_FDQ1_QNUM | W | 0h | This field specifies which Free Descriptor / Buffer Pool should be used for the 2nd Rx buffer in a host type packet |
| 13-12 | RX_HOST_FDQ0_QMGR | W | 0h | This field specifies which Buffer Manager should be used for the second Rx buffer in a host type packet. |
| 11-0 | RX_HOST_FDQ0_QNUM | W | 0h | This field specifies which Free Descriptor / Buffer Pool should be used for the 1st Rx buffer in a host type packet Table 100 -Rx Channel N Host Packet Configuration Registers A |

### 16.4.5.47 RXHPCRB10 Register (offset = 950h) [reset = 0h]

RXHPCRB10 is shown in Figure 16-198 and described in Table 16-210.

**Figure 16-198. RXHPCRB10 Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| Reserved | | RX_HOST_FDQ3_QMGR | | RX_HOST_FDQ3_QNUM | | | |
| | | W-0h | | W-0h | | | |
| **23** | **22** | **21** | **20** | **19** | **18** | **17** | **16** |
| RX_HOST_FDQ3_QNUM | | | | | | | |
| W-0h | | | | | | | |
| **15** | **14** | **13** | **12** | **11** | **10** | **9** | **8** |
| Reserved | | RX_HOST_FDQ2_QMGR | | RX_HOST_FDQ2_QNUM | | | |
| | | W-0h | | W-0h | | | |
| **7** | **6** | **5** | **4** | **3** | **2** | **1** | **0** |
| RX_HOST_FDQ2_QNUM | | | | | | | |
| W-0h | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-210. RXHPCRB10 Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 29-28 | RX_HOST_FDQ3_QMGR | W | 0h | This field specifies which Manager should be used for the 4th or later Rx buffers in a host type packet |
| 27-16 | RX_HOST_FDQ3_QNUM | W | 0h | This field specifies which Free Descriptor Queue should be used for the 4th or later Rx buffers in a host type packet |
| 13-12 | RX_HOST_FDQ2_QMGR | W | 0h | This field specifies which Buffer Manager should be used for the 3rd Rx buffer in a host type packet |
| 11-0 | RX_HOST_FDQ2_QNUM | W | 0h | This field specifies which Free Descriptor / Buffer Pool should be used for the 3rd Rx buffer in a host type packet Table 101 -Rx Channel N Host Packet Configuration Registers B |

### 16.4.5.48 TXGCR11 Register (offset = 960h) [reset = 0h]

TXGCR11 is shown in Figure 16-199 and described in Table 16-211.

**Figure 16-199. TXGCR11 Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| TX_ENABLE | TX_TEARDOWN | Reserved | | | | | |
| R/W-0h | R/W-0h | | | | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| Reserved | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| Reserved | | TX_DEFAULT_QMGR | | TX_DEFAULT_QNUM | | | |
| | | W-0h | | W-0h | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| TX_DEFAULT_QNUM | | | | | | | |
| W-0h | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-211. TXGCR11 Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31 | TX_ENABLE | R/W | 0h | This field enables or disables the channel<br>0 = channel is disabled<br>1 = channel is enabled This field will be cleared after a channel teardown is complete. |
| 30 | TX_TEARDOWN | R/W | 0h | Setting this bit will request the channel to be torn down.<br>This field will remain set after a channel teardown is complete. |
| 13-12 | TX_DEFAULT_QMGR | W | 0h | This field controls the default queue manager number that will be used to queue teardown descriptors back to the host. |
| 11-0 | TX_DEFAULT_QNUM | W | 0h | This field controls the default queue number within the selected queue manager onto which teardown descriptors will be queued back to the host.<br>Table<br>98 -Tx Channel N Global Configuration Registers |

### 16.4.5.49 RXGCR11 Register (offset = 968h) [reset = 0h]

RXGCR11 is shown in Figure 16-200 and described in Table 16-212.

**Figure 16-200. RXGCR11 Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| RX_ENABLE | RX_TEARDOWN | RX_PAUSE | Reserved | | | | RX_ERROR_HANDLING |
| R/W-0h | R/W-0h | R/W-0h | | | | | W-0h |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| RX_SOP_OFFSET | | | | | | | |
| W-0h | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| RX_DEFAULT_DESC_TYPE | | RX_DEFAULT_RQ_QMGR | | RX_DEFAULT_RQ_QNUM | | | |
| W-0h | | W-0h | | W-0h | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| RX_DEFAULT_RQ_QNUM | | | | | | | |
| W-0h | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-212. RXGCR11 Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31 | RX_ENABLE | R/W | 0h | This field enables or disables the channel<br>0 = channel is disabled<br>1 = channel is enabled This field will be cleared after a channel teardown is complete. |
| 30 | RX_TEARDOWN | R/W | 0h | This field indicates whether or not an Rx teardown operation is complete.<br>This field should be cleared when a channel is initialized.<br>This field will be set after a channel teardown is complete. |
| 29 | RX_PAUSE | R/W | 0h | Setting this bit causes the CPPI DMA to be suspended for rx channels.<br>If a pause is being requested and the channel is not in a packet then drop the credit. |
| 24 | RX_ERROR_HANDLING | W | 0h | This bit controls the error handling mode for the channel and is only used when channel errors (i.e.<br>descriptor or buffer starvation occurs):<br>0 = Starvation errors result in dropping packet and reclaiming any used descriptor or buffer resources back to the original queues/pools they were allocated to<br>1 = Starvation errors result in subsequent re-try of the descriptor allocation operation.<br>In this mode, the DMA will return to the IDLE state without saving it's internal operational state back to the internal state RAM and without issuing an advance operation on the FIFO interface.<br>This results in the DMA re-initiating the FIFO block transfer at a later time with the intention that additional free buffers and/or descriptors will have been added.<br>Regardless of the value of this bit, the DMA will assert the cdma_rx_sof_overrun (for SOP) or cdma_rx_mof_overrun (for non-SOP) when |
| 23-16 | RX_SOP_OFFSET | W | 0h | This field specifies the number of bytes that are to be skipped in the SOP buffer before beginning to write the payload.<br>This value must be less than the minimum size of a buffer in the system.<br>Valid values are<br>0 - 255 bytes. |

**Table 16-212. RXGCR11 Register Field Descriptions (continued)**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 15-14 | RX_DEFAULT_DESC_TYPE | W | 0h | This field indicates the default descriptor type to use:<br>0 = Reserved<br>1 = Host<br>2 = Reserved<br>3 = Reserved The actual descriptor type that will be used for reception can be overridden by information provided in the CPPI FIFO data block. |
| 13-12 | RX_DEFAULT_RQ_QMGR | W | 0h | This field indicates the default receive queue manager that this channel should use.<br>The actual receive queue manager index can be overridden by information provided in the CPPI FIFO data block. |
| 11-0 | RX_DEFAULT_RQ_QNUM | W | 0h | This field indicates the default receive queue that this channel should use.<br>The actual receive queue that will be used for reception can be overridden by information provided in the CPPI FIFO data block.<br>Table<br>99 -Rx Channel N Global Configuration Registers |

### 16.4.5.50 RXHPCRA11 Register (offset = 96Ch) [reset = 0h]

RXHPCRA11 is shown in Figure 16-201 and described in Table 16-213.

**Figure 16-201. RXHPCRA11 Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| Reserved | | RX_HOST_FDQ1_QMGR | | RX_HOST_FDQ1_QNUM | | | |
| | | W-0h | | W-0h | | | |
| **23** | **22** | **21** | **20** | **19** | **18** | **17** | **16** |
| RX_HOST_FDQ1_QNUM | | | | | | | |
| W-0h | | | | | | | |
| **15** | **14** | **13** | **12** | **11** | **10** | **9** | **8** |
| Reserved | | RX_HOST_FDQ0_QMGR | | RX_HOST_FDQ0_QNUM | | | |
| | | W-0h | | W-0h | | | |
| **7** | **6** | **5** | **4** | **3** | **2** | **1** | **0** |
| RX_HOST_FDQ0_QNUM | | | | | | | |
| W-0h | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-213. RXHPCRA11 Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 29-28 | RX_HOST_FDQ1_QMGR | W | 0h | This field specifies which Buffer Manager should be used for the second Rx buffer in a host type packet. |
| 27-16 | RX_HOST_FDQ1_QNUM | W | 0h | This field specifies which Free Descriptor / Buffer Pool should be used for the 2nd Rx buffer in a host type packet |
| 13-12 | RX_HOST_FDQ0_QMGR | W | 0h | This field specifies which Buffer Manager should be used for the second Rx buffer in a host type packet. |
| 11-0 | RX_HOST_FDQ0_QNUM | W | 0h | This field specifies which Free Descriptor / Buffer Pool should be used for the 1st Rx buffer in a host type packet Table 100 -Rx Channel N Host Packet Configuration Registers A |

### 16.4.5.51 RXHPCRB11 Register (offset = 970h) [reset = 0h]

RXHPCRB11 is shown in Figure 16-202 and described in Table 16-214.

**Figure 16-202. RXHPCRB11 Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| Reserved | | RX_HOST_FDQ3_QMGR | | RX_HOST_FDQ3_QNUM | | | |
| | | W-0h | | W-0h | | | |
| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
| RX_HOST_FDQ3_QNUM | | | | | | | |
| W-0h | | | | | | | |
| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
| Reserved | | RX_HOST_FDQ2_QMGR | | RX_HOST_FDQ2_QNUM | | | |
| | | W-0h | | W-0h | | | |
| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
| RX_HOST_FDQ2_QNUM | | | | | | | |
| W-0h | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-214. RXHPCRB11 Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 29-28 | RX_HOST_FDQ3_QMGR | W | 0h | This field specifies which Manager should be used for the 4th or later Rx buffers in a host type packet |
| 27-16 | RX_HOST_FDQ3_QNUM | W | 0h | This field specifies which Free Descriptor Queue should be used for the 4th or later Rx buffers in a host type packet |
| 13-12 | RX_HOST_FDQ2_QMGR | W | 0h | This field specifies which Buffer Manager should be used for the 3rd Rx buffer in a host type packet |
| 11-0 | RX_HOST_FDQ2_QNUM | W | 0h | This field specifies which Free Descriptor / Buffer Pool should be used for the 3rd Rx buffer in a host type packet Table 101 -Rx Channel N Host Packet Configuration Registers B |

### 16.4.5.52 TXGCR12 Register (offset = 980h) [reset = 0h]

TXGCR12 is shown in Figure 16-203 and described in Table 16-215.

**Figure 16-203. TXGCR12 Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| TX_ENABLE | TX_TEARDOWN | Reserved | | | | | |
| R/W-0h | R/W-0h | | | | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| Reserved | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| Reserved | | TX_DEFAULT_QMGR | | TX_DEFAULT_QNUM | | | |
| | | W-0h | | W-0h | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| TX_DEFAULT_QNUM | | | | | | | |
| W-0h | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-215. TXGCR12 Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31 | TX_ENABLE | R/W | 0h | This field enables or disables the channel<br>0 = channel is disabled<br>1 = channel is enabled This field will be cleared after a channel teardown is complete. |
| 30 | TX_TEARDOWN | R/W | 0h | Setting this bit will request the channel to be torn down.<br>This field will remain set after a channel teardown is complete. |
| 13-12 | TX_DEFAULT_QMGR | W | 0h | This field controls the default queue manager number that will be used to queue teardown descriptors back to the host. |
| 11-0 | TX_DEFAULT_QNUM | W | 0h | This field controls the default queue number within the selected queue manager onto which teardown descriptors will be queued back to the host.<br>Table<br>98 -Tx Channel N Global Configuration Registers |

### 16.4.5.53 RXGCR12 Register (offset = 988h) [reset = 0h]

RXGCR12 is shown in Figure 16-204 and described in Table 16-216.

**Figure 16-204. RXGCR12 Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| RX_ENABLE | RX_TEARDOWN | RX_PAUSE | Reserved | | | | RX_ERROR_HANDLING |
| R/W-0h | R/W-0h | R/W-0h | | | | | W-0h |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| RX_SOP_OFFSET | | | | | | | |
| W-0h | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| RX_DEFAULT_DESC_TYPE | | RX_DEFAULT_RQ_QMGR | | RX_DEFAULT_RQ_QNUM | | | |
| W-0h | | W-0h | | W-0h | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| RX_DEFAULT_RQ_QNUM | | | | | | | |
| W-0h | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-216. RXGCR12 Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31 | RX_ENABLE | R/W | 0h | This field enables or disables the channel<br>0 = channel is disabled<br>1 = channel is enabled This field will be cleared after a channel teardown is complete. |
| 30 | RX_TEARDOWN | R/W | 0h | This field indicates whether or not an Rx teardown operation is complete.<br>This field should be cleared when a channel is initialized.<br>This field will be set after a channel teardown is complete. |
| 29 | RX_PAUSE | R/W | 0h | Setting this bit causes the CPPI DMA to be suspended for rx channels.<br>If a pause is being requested and the channel is not in a packet then drop the credit. |
| 24 | RX_ERROR_HANDLING | W | 0h | This bit controls the error handling mode for the channel and is only used when channel errors (i.e.<br>descriptor or buffer starvation occurs):<br>0 = Starvation errors result in dropping packet and reclaiming any used descriptor or buffer resources back to the original queues/pools they were allocated to<br>1 = Starvation errors result in subsequent re-try of the descriptor allocation operation.<br>In this mode, the DMA will return to the IDLE state without saving it's internal operational state back to the internal state RAM and without issuing an advance operation on the FIFO interface.<br>This results in the DMA re-initiating the FIFO block transfer at a later time with the intention that additional free buffers and/or descriptors will have been added.<br>Regardless of the value of this bit, the DMA will assert the cdma_rx_sof_overrun (for SOP) or cdma_rx_mof_overrun (for non-SOP) when |
| 23-16 | RX_SOP_OFFSET | W | 0h | This field specifies the number of bytes that are to be skipped in the SOP buffer before beginning to write the payload.<br>This value must be less than the minimum size of a buffer in the system.<br>Valid values are<br>0 - 255 bytes. |

**Table 16-216. RXGCR12 Register Field Descriptions (continued)**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 15-14 | RX_DEFAULT_DESC_TYPE | W | 0h | This field indicates the default descriptor type to use:<br>0 = Reserved<br>1 = Host<br>2 = Reserved<br>3 = Reserved The actual descriptor type that will be used for reception can be overridden by information provided in the CPPI FIFO data block. |
| 13-12 | RX_DEFAULT_RQ_QMGR | W | 0h | This field indicates the default receive queue manager that this channel should use.<br>The actual receive queue manager index can be overridden by information provided in the CPPI FIFO data block. |
| 11-0 | RX_DEFAULT_RQ_QNUM | W | 0h | This field indicates the default receive queue that this channel should use.<br>The actual receive queue that will be used for reception can be overridden by information provided in the CPPI FIFO data block. Table<br>99 -Rx Channel N Global Configuration Registers |

### 16.4.5.54 RXHPCRA12 Register (offset = 98Ch) [reset = 0h]

RXHPCRA12 is shown in Figure 16-205 and described in Table 16-217.

**Figure 16-205. RXHPCRA12 Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| Reserved | | RX_HOST_FDQ1_QMGR | | RX_HOST_FDQ1_QNUM | | | |
| | | W-0h | | W-0h | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| RX_HOST_FDQ1_QNUM | | | | | | | |
| W-0h | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| Reserved | | RX_HOST_FDQ0_QMGR | | RX_HOST_FDQ0_QNUM | | | |
| | | W-0h | | W-0h | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| RX_HOST_FDQ0_QNUM | | | | | | | |
| W-0h | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-217. RXHPCRA12 Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 29-28 | RX_HOST_FDQ1_QMGR | W | 0h | This field specifies which Buffer Manager should be used for the second Rx buffer in a host type packet. |
| 27-16 | RX_HOST_FDQ1_QNUM | W | 0h | This field specifies which Free Descriptor / Buffer Pool should be used for the 2nd Rx buffer in a host type packet |
| 13-12 | RX_HOST_FDQ0_QMGR | W | 0h | This field specifies which Buffer Manager should be used for the second Rx buffer in a host type packet. |
| 11-0 | RX_HOST_FDQ0_QNUM | W | 0h | This field specifies which Free Descriptor / Buffer Pool should be used for the 1st Rx buffer in a host type packet Table 100 -Rx Channel N Host Packet Configuration Registers A |

### 16.4.5.55 RXHPCRB12 Register (offset = 990h) [reset = 0h]

RXHPCRB12 is shown in Figure 16-206 and described in Table 16-218.

**Figure 16-206. RXHPCRB12 Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| Reserved | | RX_HOST_FDQ3_QMGR | | RX_HOST_FDQ3_QNUM | | | |
| | | W-0h | | W-0h | | | |
| **23** | **22** | **21** | **20** | **19** | **18** | **17** | **16** |
| RX_HOST_FDQ3_QNUM | | | | | | | |
| W-0h | | | | | | | |
| **15** | **14** | **13** | **12** | **11** | **10** | **9** | **8** |
| Reserved | | RX_HOST_FDQ2_QMGR | | RX_HOST_FDQ2_QNUM | | | |
| | | W-0h | | W-0h | | | |
| **7** | **6** | **5** | **4** | **3** | **2** | **1** | **0** |
| RX_HOST_FDQ2_QNUM | | | | | | | |
| W-0h | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-218. RXHPCRB12 Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 29-28 | RX_HOST_FDQ3_QMGR | W | 0h | This field specifies which Manager should be used for the 4th or later Rx buffers in a host type packet |
| 27-16 | RX_HOST_FDQ3_QNUM | W | 0h | This field specifies which Free Descriptor Queue should be used for the 4th or later Rx buffers in a host type packet |
| 13-12 | RX_HOST_FDQ2_QMGR | W | 0h | This field specifies which Buffer Manager should be used for the 3rd Rx buffer in a host type packet |
| 11-0 | RX_HOST_FDQ2_QNUM | W | 0h | This field specifies which Free Descriptor / Buffer Pool should be used for the 3rd Rx buffer in a host type packet Table 101 -Rx Channel N Host Packet Configuration Registers B |

### 16.4.5.56 TXGCR13 Register (offset = 9A0h) [reset = 0h]

TXGCR13 is shown in Figure 16-207 and described in Table 16-219.

**Figure 16-207. TXGCR13 Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| TX_ENABLE | TX_TEARDOWN | Reserved | | | | | |
| R/W-0h | R/W-0h | | | | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| Reserved | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| Reserved | | TX_DEFAULT_QMGR | | TX_DEFAULT_QNUM | | | |
| | | W-0h | | W-0h | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| TX_DEFAULT_QNUM | | | | | | | |
| W-0h | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-219. TXGCR13 Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31 | TX_ENABLE | R/W | 0h | This field enables or disables the channel<br>0 = channel is disabled<br>1 = channel is enabled This field will be cleared after a channel teardown is complete. |
| 30 | TX_TEARDOWN | R/W | 0h | Setting this bit will request the channel to be torn down.<br>This field will remain set after a channel teardown is complete. |
| 13-12 | TX_DEFAULT_QMGR | W | 0h | This field controls the default queue manager number that will be used to queue teardown descriptors back to the host. |
| 11-0 | TX_DEFAULT_QNUM | W | 0h | This field controls the default queue number within the selected queue manager onto which teardown descriptors will be queued back to the host.<br>Table<br>98 -Tx Channel N Global Configuration Registers |

### 16.4.5.57 RXGCR13 Register (offset = 9A8h) [reset = 0h]

RXGCR13 is shown in Figure 16-208 and described in Table 16-220.

**Figure 16-208. RXGCR13 Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| RX_ENABLE | RX_TEARDOWN | RX_PAUSE | Reserved | | | | RX_ERROR_HANDLING |
| R/W-0h | R/W-0h | R/W-0h | | | | | W-0h |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| RX_SOP_OFFSET | | | | | | | |
| W-0h | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| RX_DEFAULT_DESC_TYPE | | RX_DEFAULT_RQ_QMGR | | RX_DEFAULT_RQ_QNUM | | | |
| W-0h | | W-0h | | W-0h | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| RX_DEFAULT_RQ_QNUM | | | | | | | |
| W-0h | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-220. RXGCR13 Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31 | RX_ENABLE | R/W | 0h | This field enables or disables the channel<br>0 = channel is disabled<br>1 = channel is enabled This field will be cleared after a channel teardown is complete. |
| 30 | RX_TEARDOWN | R/W | 0h | This field indicates whether or not an Rx teardown operation is complete.<br>This field should be cleared when a channel is initialized.<br>This field will be set after a channel teardown is complete. |
| 29 | RX_PAUSE | R/W | 0h | Setting this bit causes the CPPI DMA to be suspended for rx channels.<br>If a pause is being requested and the channel is not in a packet then drop the credit. |
| 24 | RX_ERROR_HANDLING | W | 0h | This bit controls the error handling mode for the channel and is only used when channel errors (i.e.<br>descriptor or buffer starvation occurs):<br>0 = Starvation errors result in dropping packet and reclaiming any used descriptor or buffer resources back to the original queues/pools they were allocated to<br>1 = Starvation errors result in subsequent re-try of the descriptor allocation operation.<br>In this mode, the DMA will return to the IDLE state without saving it's internal operational state back to the internal state RAM and without issuing an advance operation on the FIFO interface.<br>This results in the DMA re-initiating the FIFO block transfer at a later time with the intention that additional free buffers and/or descriptors will have been added.<br>Regardless of the value of this bit, the DMA will assert the cdma_rx_sof_overrun (for SOP) or cdma_rx_mof_overrun (for non-SOP) when |
| 23-16 | RX_SOP_OFFSET | W | 0h | This field specifies the number of bytes that are to be skipped in the SOP buffer before beginning to write the payload.<br>This value must be less than the minimum size of a buffer in the system.<br>Valid values are<br>0 - 255 bytes. |

**Table 16-220. RXGCR13 Register Field Descriptions (continued)**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 15-14 | RX_DEFAULT_DESC_TYPE | W | 0h | This field indicates the default descriptor type to use:<br>0 = Reserved<br>1 = Host<br>2 = Reserved<br>3 = Reserved The actual descriptor type that will be used for reception can be overridden by information provided in the CPPI FIFO data block. |
| 13-12 | RX_DEFAULT_RQ_QMGR | W | 0h | This field indicates the default receive queue manager that this channel should use.<br>The actual receive queue manager index can be overridden by information provided in the CPPI FIFO data block. |
| 11-0 | RX_DEFAULT_RQ_QNUM | W | 0h | This field indicates the default receive queue that this channel should use.<br>The actual receive queue that will be used for reception can be overridden by information provided in the CPPI FIFO data block. Table<br>99 -Rx Channel N Global Configuration Registers |

### 16.4.5.58 RXHPCRA13 Register (offset = 9ACh) [reset = 0h]

RXHPCRA13 is shown in Figure 16-209 and described in Table 16-221.

**Figure 16-209. RXHPCRA13 Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| Reserved | | RX_HOST_FDQ1_QMGR | | RX_HOST_FDQ1_QNUM | | | |
| | | W-0h | | W-0h | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| RX_HOST_FDQ1_QNUM | | | | | | | |
| W-0h | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| Reserved | | RX_HOST_FDQ0_QMGR | | RX_HOST_FDQ0_QNUM | | | |
| | | W-0h | | W-0h | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| RX_HOST_FDQ0_QNUM | | | | | | | |
| W-0h | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-221. RXHPCRA13 Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 29-28 | RX_HOST_FDQ1_QMGR | W | 0h | This field specifies which Buffer Manager should be used for the second Rx buffer in a host type packet. |
| 27-16 | RX_HOST_FDQ1_QNUM | W | 0h | This field specifies which Free Descriptor / Buffer Pool should be used for the 2nd Rx buffer in a host type packet |
| 13-12 | RX_HOST_FDQ0_QMGR | W | 0h | This field specifies which Buffer Manager should be used for the second Rx buffer in a host type packet. |
| 11-0 | RX_HOST_FDQ0_QNUM | W | 0h | This field specifies which Free Descriptor / Buffer Pool should be used for the 1st Rx buffer in a host type packet Table 100 -Rx Channel N Host Packet Configuration Registers A |

### 16.4.5.59 RXHPCRB13 Register (offset = 9B0h) [reset = 0h]

RXHPCRB13 is shown in Figure 16-210 and described in Table 16-222.

**Figure 16-210. RXHPCRB13 Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| Reserved | | RX_HOST_FDQ3_QMGR | | RX_HOST_FDQ3_QNUM | | | |
| | | W-0h | | W-0h | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| RX_HOST_FDQ3_QNUM | | | | | | | |
| W-0h | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| Reserved | | RX_HOST_FDQ2_QMGR | | RX_HOST_FDQ2_QNUM | | | |
| | | W-0h | | W-0h | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| RX_HOST_FDQ2_QNUM | | | | | | | |
| W-0h | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-222. RXHPCRB13 Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 29-28 | RX_HOST_FDQ3_QMGR | W | 0h | This field specifies which Manager should be used for the 4th or later Rx buffers in a host type packet |
| 27-16 | RX_HOST_FDQ3_QNUM | W | 0h | This field specifies which Free Descriptor Queue should be used for the 4th or later Rx buffers in a host type packet |
| 13-12 | RX_HOST_FDQ2_QMGR | W | 0h | This field specifies which Buffer Manager should be used for the 3rd Rx buffer in a host type packet |
| 11-0 | RX_HOST_FDQ2_QNUM | W | 0h | This field specifies which Free Descriptor / Buffer Pool should be used for the 3rd Rx buffer in a host type packet Table 101 -Rx Channel N Host Packet Configuration Registers B |

### 16.4.5.60 TXGCR14 Register (offset = 9C0h) [reset = 0h]

TXGCR14 is shown in Figure 16-211 and described in Table 16-223.

**Figure 16-211. TXGCR14 Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| TX_ENABLE | TX_TEARDOWN | Reserved | | | | | |
| R/W-0h | R/W-0h | | | | | | |
| **23** | **22** | **21** | **20** | **19** | **18** | **17** | **16** |
| Reserved | | | | | | | |
| **15** | **14** | **13** | **12** | **11** | **10** | **9** | **8** |
| Reserved | | TX_DEFAULT_QMGR | | TX_DEFAULT_QNUM | | | |
| | | W-0h | | W-0h | | | |
| **7** | **6** | **5** | **4** | **3** | **2** | **1** | **0** |
| TX_DEFAULT_QNUM | | | | | | | |
| W-0h | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-223. TXGCR14 Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31 | TX_ENABLE | R/W | 0h | This field enables or disables the channel<br>0 = channel is disabled<br>1 = channel is enabled This field will be cleared after a channel teardown is complete. |
| 30 | TX_TEARDOWN | R/W | 0h | Setting this bit will request the channel to be torn down.<br>This field will remain set after a channel teardown is complete. |
| 13-12 | TX_DEFAULT_QMGR | W | 0h | This field controls the default queue manager number that will be used to queue teardown descriptors back to the host. |
| 11-0 | TX_DEFAULT_QNUM | W | 0h | This field controls the default queue number within the selected queue manager onto which teardown descriptors will be queued back to the host.<br>Table<br>98 -Tx Channel N Global Configuration Registers |

### 16.4.5.61 RXGCR14 Register (offset = 9C8h) [reset = 0h]

RXGCR14 is shown in Figure 16-212 and described in Table 16-224.

**Figure 16-212. RXGCR14 Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| RX_ENABLE | RX_TEARDOWN | RX_PAUSE | Reserved | | | | RX_ERROR_HANDLING |
| R/W-0h | R/W-0h | R/W-0h | | | | | W-0h |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| RX_SOP_OFFSET | | | | | | | |
| W-0h | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| RX_DEFAULT_DESC_TYPE | | RX_DEFAULT_RQ_QMGR | | RX_DEFAULT_RQ_QNUM | | | |
| W-0h | | W-0h | | W-0h | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| RX_DEFAULT_RQ_QNUM | | | | | | | |
| W-0h | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-224. RXGCR14 Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31 | RX_ENABLE | R/W | 0h | This field enables or disables the channel<br>0 = channel is disabled<br>1 = channel is enabled This field will be cleared after a channel teardown is complete. |
| 30 | RX_TEARDOWN | R/W | 0h | This field indicates whether or not an Rx teardown operation is complete.<br>This field should be cleared when a channel is initialized.<br>This field will be set after a channel teardown is complete. |
| 29 | RX_PAUSE | R/W | 0h | Setting this bit causes the CPPI DMA to be suspended for rx channels.<br>If a pause is being requested and the channel is not in a packet then drop the credit. |
| 24 | RX_ERROR_HANDLING | W | 0h | This bit controls the error handling mode for the channel and is only used when channel errors (i.e.<br>descriptor or buffer starvation occurs):<br>0 = Starvation errors result in dropping packet and reclaiming any used descriptor or buffer resources back to the original queues/pools they were allocated to<br>1 = Starvation errors result in subsequent re-try of the descriptor allocation operation.<br>In this mode, the DMA will return to the IDLE state without saving it's internal operational state back to the internal state RAM and without issuing an advance operation on the FIFO interface.<br>This results in the DMA re-initiating the FIFO block transfer at a later time with the intention that additional free buffers and/or descriptors will have been added.<br>Regardless of the value of this bit, the DMA will assert the cdma_rx_sof_overrun (for SOP) or cdma_rx_mof_overrun (for non-SOP) when |
| 23-16 | RX_SOP_OFFSET | W | 0h | This field specifies the number of bytes that are to be skipped in the SOP buffer before beginning to write the payload.<br>This value must be less than the minimum size of a buffer in the system.<br>Valid values are<br>0 - 255 bytes. |

**Table 16-224. RXGCR14 Register Field Descriptions (continued)**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 15-14 | RX_DEFAULT_DESC_TYPE | W | 0h | This field indicates the default descriptor type to use:<br>0 = Reserved<br>1 = Host<br>2 = Reserved<br>3 = Reserved The actual descriptor type that will be used for reception can be overridden by information provided in the CPPI FIFO data block. |
| 13-12 | RX_DEFAULT_RQ_QMGR | W | 0h | This field indicates the default receive queue manager that this channel should use.<br>The actual receive queue manager index can be overridden by information provided in the CPPI FIFO data block. |
| 11-0 | RX_DEFAULT_RQ_QNUM | W | 0h | This field indicates the default receive queue that this channel should use.<br>The actual receive queue that will be used for reception can be overridden by information provided in the CPPI FIFO data block.<br>Table<br>99 -Rx Channel N Global Configuration Registers |

### 16.4.5.62 RXHPCRA14 Register (offset = 9CCh) [reset = 0h]

RXHPCRA14 is shown in Figure 16-213 and described in Table 16-225.

**Figure 16-213. RXHPCRA14 Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| Reserved | | RX_HOST_FDQ1_QMGR | | RX_HOST_FDQ1_QNUM | | | |
| | | W-0h | | W-0h | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| RX_HOST_FDQ1_QNUM | | | | | | | |
| W-0h | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| Reserved | | RX_HOST_FDQ0_QMGR | | RX_HOST_FDQ0_QNUM | | | |
| | | W-0h | | W-0h | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| RX_HOST_FDQ0_QNUM | | | | | | | |
| W-0h | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-225. RXHPCRA14 Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 29-28 | RX_HOST_FDQ1_QMGR | W | 0h | This field specifies which Buffer Manager should be used for the second Rx buffer in a host type packet. |
| 27-16 | RX_HOST_FDQ1_QNUM | W | 0h | This field specifies which Free Descriptor / Buffer Pool should be used for the 2nd Rx buffer in a host type packet |
| 13-12 | RX_HOST_FDQ0_QMGR | W | 0h | This field specifies which Buffer Manager should be used for the second Rx buffer in a host type packet. |
| 11-0 | RX_HOST_FDQ0_QNUM | W | 0h | This field specifies which Free Descriptor / Buffer Pool should be used for the 1st Rx buffer in a host type packet Table 100 -Rx Channel N Host Packet Configuration Registers A |

### 16.4.5.63 RXHPCRB14 Register (offset = 9D0h) [reset = 0h]

RXHPCRB14 is shown in Figure 16-214 and described in Table 16-226.

**Figure 16-214. RXHPCRB14 Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| Reserved | | RX_HOST_FDQ3_QMGR | | RX_HOST_FDQ3_QNUM | | | |
| | | W-0h | | W-0h | | | |
| **23** | **22** | **21** | **20** | **19** | **18** | **17** | **16** |
| RX_HOST_FDQ3_QNUM | | | | | | | |
| W-0h | | | | | | | |
| **15** | **14** | **13** | **12** | **11** | **10** | **9** | **8** |
| Reserved | | RX_HOST_FDQ2_QMGR | | RX_HOST_FDQ2_QNUM | | | |
| | | W-0h | | W-0h | | | |
| **7** | **6** | **5** | **4** | **3** | **2** | **1** | **0** |
| RX_HOST_FDQ2_QNUM | | | | | | | |
| W-0h | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-226. RXHPCRB14 Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 29-28 | RX_HOST_FDQ3_QMGR | W | 0h | This field specifies which Manager should be used for the 4th or later Rx buffers in a host type packet |
| 27-16 | RX_HOST_FDQ3_QNUM | W | 0h | This field specifies which Free Descriptor Queue should be used for the 4th or later Rx buffers in a host type packet |
| 13-12 | RX_HOST_FDQ2_QMGR | W | 0h | This field specifies which Buffer Manager should be used for the 3rd Rx buffer in a host type packet |
| 11-0 | RX_HOST_FDQ2_QNUM | W | 0h | This field specifies which Free Descriptor / Buffer Pool should be used for the 3rd Rx buffer in a host type packet Table 101 -Rx Channel N Host Packet Configuration Registers B |

### 16.4.5.64 TXGCR15 Register (offset = 9E0h) [reset = 0h]

TXGCR15 is shown in Figure 16-215 and described in Table 16-227.

**Figure 16-215. TXGCR15 Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| TX_ENABLE | TX_TEARDOWN | Reserved | | | | | |
| R/W-0h | R/W-0h | | | | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| Reserved | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| Reserved | | TX_DEFAULT_QMGR | | TX_DEFAULT_QNUM | | | |
| | | W-0h | | W-0h | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| TX_DEFAULT_QNUM | | | | | | | |
| W-0h | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-227. TXGCR15 Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31 | TX_ENABLE | R/W | 0h | This field enables or disables the channel<br>0 = channel is disabled<br>1 = channel is enabled This field will be cleared after a channel teardown is complete. |
| 30 | TX_TEARDOWN | R/W | 0h | Setting this bit will request the channel to be torn down.<br>This field will remain set after a channel teardown is complete. |
| 13-12 | TX_DEFAULT_QMGR | W | 0h | This field controls the default queue manager number that will be used to queue teardown descriptors back to the host. |
| 11-0 | TX_DEFAULT_QNUM | W | 0h | This field controls the default queue number within the selected queue manager onto which teardown descriptors will be queued back to the host.<br>Table<br>98 -Tx Channel N Global Configuration Registers |

### 16.4.5.65 RXGCR15 Register (offset = 9E8h) [reset = 0h]

RXGCR15 is shown in Figure 16-216 and described in Table 16-228.

**Figure 16-216. RXGCR15 Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| RX_ENABLE | RX_TEARDOWN | RX_PAUSE | Reserved | | | | RX_ERROR_HANDLING |
| R/W-0h | R/W-0h | R/W-0h | | | | | W-0h |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| RX_SOP_OFFSET | | | | | | | |
| W-0h | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| RX_DEFAULT_DESC_TYPE | | RX_DEFAULT_RQ_QMGR | | RX_DEFAULT_RQ_QNUM | | | |
| W-0h | | W-0h | | W-0h | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| RX_DEFAULT_RQ_QNUM | | | | | | | |
| W-0h | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-228. RXGCR15 Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31 | RX_ENABLE | R/W | 0h | This field enables or disables the channel<br>0 = channel is disabled<br>1 = channel is enabled This field will be cleared after a channel teardown is complete. |
| 30 | RX_TEARDOWN | R/W | 0h | This field indicates whether or not an Rx teardown operation is complete.<br>This field should be cleared when a channel is initialized.<br>This field will be set after a channel teardown is complete. |
| 29 | RX_PAUSE | R/W | 0h | Setting this bit causes the CPPI DMA to be suspended for rx channels.<br>If a pause is being requested and the channel is not in a packet then drop the credit. |
| 24 | RX_ERROR_HANDLING | W | 0h | This bit controls the error handling mode for the channel and is only used when channel errors (i.e.<br>descriptor or buffer starvation occurs):<br>0 = Starvation errors result in dropping packet and reclaiming any used descriptor or buffer resources back to the original queues/pools they were allocated to<br>1 = Starvation errors result in subsequent re-try of the descriptor allocation operation.<br>In this mode, the DMA will return to the IDLE state without saving it's internal operational state back to the internal state RAM and without issuing an advance operation on the FIFO interface.<br>This results in the DMA re-initiating the FIFO block transfer at a later time with the intention that additional free buffers and/or descriptors will have been added.<br>Regardless of the value of this bit, the DMA will assert the cdma_rx_sof_overrun (for SOP) or cdma_rx_mof_overrun (for non-SOP) when |
| 23-16 | RX_SOP_OFFSET | W | 0h | This field specifies the number of bytes that are to be skipped in the SOP buffer before beginning to write the payload.<br>This value must be less than the minimum size of a buffer in the system.<br>Valid values are<br>0 - 255 bytes. |

**Table 16-228. RXGCR15 Register Field Descriptions (continued)**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 15-14 | RX_DEFAULT_DESC_TYPE | W | 0h | This field indicates the default descriptor type to use:<br>0 = Reserved<br>1 = Host<br>2 = Reserved<br>3 = Reserved The actual descriptor type that will be used for reception can be overridden by information provided in the CPPI FIFO data block. |
| 13-12 | RX_DEFAULT_RQ_QMGR | W | 0h | This field indicates the default receive queue manager that this channel should use.<br>The actual receive queue manager index can be overridden by information provided in the CPPI FIFO data block. |
| 11-0 | RX_DEFAULT_RQ_QNUM | W | 0h | This field indicates the default receive queue that this channel should use.<br>The actual receive queue that will be used for reception can be overridden by information provided in the CPPI FIFO data block.<br>Table<br>99 -Rx Channel N Global Configuration Registers |

### 16.4.5.66 RXHPCRA15 Register (offset = 9ECh) [reset = 0h]

RXHPCRA15 is shown in Figure 16-217 and described in Table 16-229.

**Figure 16-217. RXHPCRA15 Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| Reserved | | RX_HOST_FDQ1_QMGR | | RX_HOST_FDQ1_QNUM | | | |
| | | W-0h | | W-0h | | | |
| **23** | **22** | **21** | **20** | **19** | **18** | **17** | **16** |
| RX_HOST_FDQ1_QNUM | | | | | | | |
| W-0h | | | | | | | |
| **15** | **14** | **13** | **12** | **11** | **10** | **9** | **8** |
| Reserved | | RX_HOST_FDQ0_QMGR | | RX_HOST_FDQ0_QNUM | | | |
| | | W-0h | | W-0h | | | |
| **7** | **6** | **5** | **4** | **3** | **2** | **1** | **0** |
| RX_HOST_FDQ0_QNUM | | | | | | | |
| W-0h | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-229. RXHPCRA15 Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 29-28 | RX_HOST_FDQ1_QMGR | W | 0h | This field specifies which Buffer Manager should be used for the second Rx buffer in a host type packet. |
| 27-16 | RX_HOST_FDQ1_QNUM | W | 0h | This field specifies which Free Descriptor / Buffer Pool should be used for the 2nd Rx buffer in a host type packet |
| 13-12 | RX_HOST_FDQ0_QMGR | W | 0h | This field specifies which Buffer Manager should be used for the second Rx buffer in a host type packet. |
| 11-0 | RX_HOST_FDQ0_QNUM | W | 0h | This field specifies which Free Descriptor / Buffer Pool should be used for the 1st Rx buffer in a host type packet Table 100 -Rx Channel N Host Packet Configuration Registers A |

### 16.4.5.67 RXHPCRB15 Register (offset = 9F0h) [reset = 0h]

RXHPCRB15 is shown in Figure 16-218 and described in Table 16-230.

**Figure 16-218. RXHPCRB15 Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| Reserved | | RX_HOST_FDQ3_QMGR | | RX_HOST_FDQ3_QNUM | | | |
| | | W-0h | | W-0h | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| RX_HOST_FDQ3_QNUM | | | | | | | |
| W-0h | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| Reserved | | RX_HOST_FDQ2_QMGR | | RX_HOST_FDQ2_QNUM | | | |
| | | W-0h | | W-0h | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| RX_HOST_FDQ2_QNUM | | | | | | | |
| W-0h | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-230. RXHPCRB15 Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 29-28 | RX_HOST_FDQ3_QMGR | W | 0h | This field specifies which Manager should be used for the 4th or later Rx buffers in a host type packet |
| 27-16 | RX_HOST_FDQ3_QNUM | W | 0h | This field specifies which Free Descriptor Queue should be used for the 4th or later Rx buffers in a host type packet |
| 13-12 | RX_HOST_FDQ2_QMGR | W | 0h | This field specifies which Buffer Manager should be used for the 3rd Rx buffer in a host type packet |
| 11-0 | RX_HOST_FDQ2_QNUM | W | 0h | This field specifies which Free Descriptor / Buffer Pool should be used for the 3rd Rx buffer in a host type packet Table 101 -Rx Channel N Host Packet Configuration Registers B |

### 16.4.5.68 TXGCR16 Register (offset = A00h) [reset = 0h]

TXGCR16 is shown in Figure 16-219 and described in Table 16-231.

**Figure 16-219. TXGCR16 Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| TX_ENABLE | TX_TEARDOWN | Reserved | | | | | |
| R/W-0h | R/W-0h | | | | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| Reserved | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| Reserved | | TX_DEFAULT_QMGR | | TX_DEFAULT_QNUM | | | |
| | | W-0h | | W-0h | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| TX_DEFAULT_QNUM | | | | | | | |
| W-0h | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-231. TXGCR16 Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31 | TX_ENABLE | R/W | 0h | This field enables or disables the channel<br>0 = channel is disabled<br>1 = channel is enabled This field will be cleared after a channel teardown is complete. |
| 30 | TX_TEARDOWN | R/W | 0h | Setting this bit will request the channel to be torn down.<br>This field will remain set after a channel teardown is complete. |
| 13-12 | TX_DEFAULT_QMGR | W | 0h | This field controls the default queue manager number that will be used to queue teardown descriptors back to the host. |
| 11-0 | TX_DEFAULT_QNUM | W | 0h | This field controls the default queue number within the selected queue manager onto which teardown descriptors will be queued back to the host.<br>Table<br>98 -Tx Channel N Global Configuration Registers |

### 16.4.5.69 RXGCR16 Register (offset = A08h) [reset = 0h]

RXGCR16 is shown in Figure 16-220 and described in Table 16-232.

**Figure 16-220. RXGCR16 Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| RX_ENABLE | RX_TEARDOWN | RX_PAUSE | Reserved | | | | RX_ERROR_HANDLING |
| R/W-0h | R/W-0h | R/W-0h | | | | | W-0h |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| RX_SOP_OFFSET | | | | | | | |
| W-0h | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| RX_DEFAULT_DESC_TYPE | | RX_DEFAULT_RQ_QMGR | | RX_DEFAULT_RQ_QNUM | | | |
| W-0h | | W-0h | | W-0h | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| RX_DEFAULT_RQ_QNUM | | | | | | | |
| W-0h | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-232. RXGCR16 Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31 | RX_ENABLE | R/W | 0h | This field enables or disables the channel<br>0 = channel is disabled<br>1 = channel is enabled This field will be cleared after a channel teardown is complete. |
| 30 | RX_TEARDOWN | R/W | 0h | This field indicates whether or not an Rx teardown operation is complete.<br>This field should be cleared when a channel is initialized.<br>This field will be set after a channel teardown is complete. |
| 29 | RX_PAUSE | R/W | 0h | Setting this bit causes the CPPI DMA to be suspended for rx channels.<br>If a pause is being requested and the channel is not in a packet then drop the credit. |
| 24 | RX_ERROR_HANDLING | W | 0h | This bit controls the error handling mode for the channel and is only used when channel errors (i.e.<br>descriptor or buffer starvation occurs):<br>0 = Starvation errors result in dropping packet and reclaiming any used descriptor or buffer resources back to the original queues/pools they were allocated to<br>1 = Starvation errors result in subsequent re-try of the descriptor allocation operation.<br>In this mode, the DMA will return to the IDLE state without saving it's internal operational state back to the internal state RAM and without issuing an advance operation on the FIFO interface.<br>This results in the DMA re-initiating the FIFO block transfer at a later time with the intention that additional free buffers and/or descriptors will have been added.<br>Regardless of the value of this bit, the DMA will assert the cdma_rx_sof_overrun (for SOP) or cdma_rx_mof_overrun (for non-SOP) when |
| 23-16 | RX_SOP_OFFSET | W | 0h | This field specifies the number of bytes that are to be skipped in the SOP buffer before beginning to write the payload.<br>This value must be less than the minimum size of a buffer in the system.<br>Valid values are<br>0 - 255 bytes. |

### Table 16-232. RXGCR16 Register Field Descriptions (continued)

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 15-14 | RX_DEFAULT_DESC_TYPE | W | 0h | This field indicates the default descriptor type to use:<br>0 = Reserved<br>1 = Host<br>2 = Reserved<br>3 = Reserved The actual descriptor type that will be used for reception can be overridden by information provided in the CPPI FIFO data block. |
| 13-12 | RX_DEFAULT_RQ_QMGR | W | 0h | This field indicates the default receive queue manager that this channel should use.<br>The actual receive queue manager index can be overridden by information provided in the CPPI FIFO data block. |
| 11-0 | RX_DEFAULT_RQ_QNUM | W | 0h | This field indicates the default receive queue that this channel should use.<br>The actual receive queue that will be used for reception can be overridden by information provided in the CPPI FIFO data block. Table<br>99 -Rx Channel N Global Configuration Registers |

### 16.4.5.70 RXHPCRA16 Register (offset = A0Ch) [reset = 0h]

RXHPCRA16 is shown in Figure 16-221 and described in Table 16-233.

**Figure 16-221. RXHPCRA16 Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| Reserved | | RX_HOST_FDQ1_QMGR | | RX_HOST_FDQ1_QNUM | | | |
| | | W-0h | | W-0h | | | |
| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
| RX_HOST_FDQ1_QNUM | | | | | | | |
| W-0h | | | | | | | |
| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
| Reserved | | RX_HOST_FDQ0_QMGR | | RX_HOST_FDQ0_QNUM | | | |
| | | W-0h | | W-0h | | | |
| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
| RX_HOST_FDQ0_QNUM | | | | | | | |
| W-0h | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-233. RXHPCRA16 Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 29-28 | RX_HOST_FDQ1_QMGR | W | 0h | This field specifies which Buffer Manager should be used for the second Rx buffer in a host type packet. |
| 27-16 | RX_HOST_FDQ1_QNUM | W | 0h | This field specifies which Free Descriptor / Buffer Pool should be used for the 2nd Rx buffer in a host type packet |
| 13-12 | RX_HOST_FDQ0_QMGR | W | 0h | This field specifies which Buffer Manager should be used for the second Rx buffer in a host type packet. |
| 11-0 | RX_HOST_FDQ0_QNUM | W | 0h | This field specifies which Free Descriptor / Buffer Pool should be used for the 1st Rx buffer in a host type packet Table 100 -Rx Channel N Host Packet Configuration Registers A |

### 16.4.5.71 RXHPCRB16 Register (offset = A10h) [reset = 0h]

RXHPCRB16 is shown in Figure 16-222 and described in Table 16-234.

**Figure 16-222. RXHPCRB16 Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| Reserved | | RX_HOST_FDQ3_QMGR | | RX_HOST_FDQ3_QNUM | | | |
| | | W-0h | | W-0h | | | |
| **23** | **22** | **21** | **20** | **19** | **18** | **17** | **16** |
| RX_HOST_FDQ3_QNUM | | | | | | | |
| W-0h | | | | | | | |
| **15** | **14** | **13** | **12** | **11** | **10** | **9** | **8** |
| Reserved | | RX_HOST_FDQ2_QMGR | | RX_HOST_FDQ2_QNUM | | | |
| | | W-0h | | W-0h | | | |
| **7** | **6** | **5** | **4** | **3** | **2** | **1** | **0** |
| RX_HOST_FDQ2_QNUM | | | | | | | |
| W-0h | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-234. RXHPCRB16 Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 29-28 | RX_HOST_FDQ3_QMGR | W | 0h | This field specifies which Manager should be used for the 4th or later Rx buffers in a host type packet |
| 27-16 | RX_HOST_FDQ3_QNUM | W | 0h | This field specifies which Free Descriptor Queue should be used for the 4th or later Rx buffers in a host type packet |
| 13-12 | RX_HOST_FDQ2_QMGR | W | 0h | This field specifies which Buffer Manager should be used for the 3rd Rx buffer in a host type packet |
| 11-0 | RX_HOST_FDQ2_QNUM | W | 0h | This field specifies which Free Descriptor / Buffer Pool should be used for the 3rd Rx buffer in a host type packet Table 101 -Rx Channel N Host Packet Configuration Registers B |

### 16.4.5.72 TXGCR17 Register (offset = A20h) [reset = 0h]

TXGCR17 is shown in Figure 16-223 and described in Table 16-235.

**Figure 16-223. TXGCR17 Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| TX_ENABLE | TX_TEARDOWN | Reserved | | | | | |
| R/W-0h | R/W-0h | | | | | | |
| **23** | **22** | **21** | **20** | **19** | **18** | **17** | **16** |
| Reserved | | | | | | | |
| **15** | **14** | **13** | **12** | **11** | **10** | **9** | **8** |
| Reserved | | TX_DEFAULT_QMGR | | TX_DEFAULT_QNUM | | | |
| | | W-0h | | W-0h | | | |
| **7** | **6** | **5** | **4** | **3** | **2** | **1** | **0** |
| TX_DEFAULT_QNUM | | | | | | | |
| W-0h | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-235. TXGCR17 Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31 | TX_ENABLE | R/W | 0h | This field enables or disables the channel<br>0 = channel is disabled<br>1 = channel is enabled This field will be cleared after a channel teardown is complete. |
| 30 | TX_TEARDOWN | R/W | 0h | Setting this bit will request the channel to be torn down.<br>This field will remain set after a channel teardown is complete. |
| 13-12 | TX_DEFAULT_QMGR | W | 0h | This field controls the default queue manager number that will be used to queue teardown descriptors back to the host. |
| 11-0 | TX_DEFAULT_QNUM | W | 0h | This field controls the default queue number within the selected queue manager onto which teardown descriptors will be queued back to the host.<br>Table<br>98 -Tx Channel N Global Configuration Registers |

### 16.4.5.73 RXGCR17 Register (offset = A28h) [reset = 0h]

RXGCR17 is shown in Figure 16-224 and described in Table 16-236.

**Figure 16-224. RXGCR17 Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| RX_ENABLE | RX_TEARDOWN | RX_PAUSE | Reserved | | | | RX_ERROR_HANDLING |
| R/W-0h | R/W-0h | R/W-0h | | | | | W-0h |
| **23** | **22** | **21** | **20** | **19** | **18** | **17** | **16** |
| RX_SOP_OFFSET | | | | | | | |
| W-0h | | | | | | | |
| **15** | **14** | **13** | **12** | **11** | **10** | **9** | **8** |
| RX_DEFAULT_DESC_TYPE | | RX_DEFAULT_RQ_QMGR | | RX_DEFAULT_RQ_QNUM | | | |
| W-0h | | W-0h | | W-0h | | | |
| **7** | **6** | **5** | **4** | **3** | **2** | **1** | **0** |
| RX_DEFAULT_RQ_QNUM | | | | | | | |
| W-0h | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-236. RXGCR17 Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31 | RX_ENABLE | R/W | 0h | This field enables or disables the channel<br>0 = channel is disabled<br>1 = channel is enabled This field will be cleared after a channel teardown is complete. |
| 30 | RX_TEARDOWN | R/W | 0h | This field indicates whether or not an Rx teardown operation is complete.<br>This field should be cleared when a channel is initialized.<br>This field will be set after a channel teardown is complete. |
| 29 | RX_PAUSE | R/W | 0h | Setting this bit causes the CPPI DMA to be suspended for rx channels.<br>If a pause is being requested and the channel is not in a packet then drop the credit. |
| 24 | RX_ERROR_HANDLING | W | 0h | This bit controls the error handling mode for the channel and is only used when channel errors (i.e.<br>descriptor or buffer starvation occurs):<br>0 = Starvation errors result in dropping packet and reclaiming any used descriptor or buffer resources back to the original queues/pools they were allocated to<br>1 = Starvation errors result in subsequent re-try of the descriptor allocation operation.<br>In this mode, the DMA will return to the IDLE state without saving it's internal operational state back to the internal state RAM and without issuing an advance operation on the FIFO interface.<br>This results in the DMA re-initiating the FIFO block transfer at a later time with the intention that additional free buffers and/or descriptors will have been added.<br>Regardless of the value of this bit, the DMA will assert the cdma_rx_sof_overrun (for SOP) or cdma_rx_mof_overrun (for non-SOP) when |
| 23-16 | RX_SOP_OFFSET | W | 0h | This field specifies the number of bytes that are to be skipped in the SOP buffer before beginning to write the payload.<br>This value must be less than the minimum size of a buffer in the system.<br>Valid values are<br>0 - 255 bytes. |

**Table 16-236. RXGCR17 Register Field Descriptions (continued)**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 15-14 | RX_DEFAULT_DESC_TYPE | W | 0h | This field indicates the default descriptor type to use:<br>0 = Reserved<br>1 = Host<br>2 = Reserved<br>3 = Reserved The actual descriptor type that will be used for reception can be overridden by information provided in the CPPI FIFO data block. |
| 13-12 | RX_DEFAULT_RQ_QMGR | W | 0h | This field indicates the default receive queue manager that this channel should use.<br>The actual receive queue manager index can be overridden by information provided in the CPPI FIFO data block. |
| 11-0 | RX_DEFAULT_RQ_QNUM | W | 0h | This field indicates the default receive queue that this channel should use.<br>The actual receive queue that will be used for reception can be overridden by information provided in the CPPI FIFO data block. Table<br>99 -Rx Channel N Global Configuration Registers |

### 16.4.5.74 RXHPCRA17 Register (offset = A2Ch) [reset = 0h]

RXHPCRA17 is shown in Figure 16-225 and described in Table 16-237.

**Figure 16-225. RXHPCRA17 Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| Reserved | | RX_HOST_FDQ1_QMGR | | RX_HOST_FDQ1_QNUM | | | |
| | | W-0h | | W-0h | | | |
| **23** | **22** | **21** | **20** | **19** | **18** | **17** | **16** |
| RX_HOST_FDQ1_QNUM | | | | | | | |
| W-0h | | | | | | | |
| **15** | **14** | **13** | **12** | **11** | **10** | **9** | **8** |
| Reserved | | RX_HOST_FDQ0_QMGR | | RX_HOST_FDQ0_QNUM | | | |
| | | W-0h | | W-0h | | | |
| **7** | **6** | **5** | **4** | **3** | **2** | **1** | **0** |
| RX_HOST_FDQ0_QNUM | | | | | | | |
| W-0h | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-237. RXHPCRA17 Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 29-28 | RX_HOST_FDQ1_QMGR | W | 0h | This field specifies which Buffer Manager should be used for the second Rx buffer in a host type packet. |
| 27-16 | RX_HOST_FDQ1_QNUM | W | 0h | This field specifies which Free Descriptor / Buffer Pool should be used for the 2nd Rx buffer in a host type packet |
| 13-12 | RX_HOST_FDQ0_QMGR | W | 0h | This field specifies which Buffer Manager should be used for the second Rx buffer in a host type packet. |
| 11-0 | RX_HOST_FDQ0_QNUM | W | 0h | This field specifies which Free Descriptor / Buffer Pool should be used for the 1st Rx buffer in a host type packet Table 100 -Rx Channel N Host Packet Configuration Registers A |

### 16.4.5.75 RXHPCRB17 Register (offset = A30h) [reset = 0h]

RXHPCRB17 is shown in Figure 16-226 and described in Table 16-238.

**Figure 16-226. RXHPCRB17 Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| Reserved | | RX_HOST_FDQ3_QMGR | | RX_HOST_FDQ3_QNUM | | | |
| | | W-0h | | W-0h | | | |
| **23** | **22** | **21** | **20** | **19** | **18** | **17** | **16** |
| RX_HOST_FDQ3_QNUM | | | | | | | |
| W-0h | | | | | | | |
| **15** | **14** | **13** | **12** | **11** | **10** | **9** | **8** |
| Reserved | | RX_HOST_FDQ2_QMGR | | RX_HOST_FDQ2_QNUM | | | |
| | | W-0h | | W-0h | | | |
| **7** | **6** | **5** | **4** | **3** | **2** | **1** | **0** |
| RX_HOST_FDQ2_QNUM | | | | | | | |
| W-0h | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-238. RXHPCRB17 Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 29-28 | RX_HOST_FDQ3_QMGR | W | 0h | This field specifies which Manager should be used for the 4th or later Rx buffers in a host type packet |
| 27-16 | RX_HOST_FDQ3_QNUM | W | 0h | This field specifies which Free Descriptor Queue should be used for the 4th or later Rx buffers in a host type packet |
| 13-12 | RX_HOST_FDQ2_QMGR | W | 0h | This field specifies which Buffer Manager should be used for the 3rd Rx buffer in a host type packet |
| 11-0 | RX_HOST_FDQ2_QNUM | W | 0h | This field specifies which Free Descriptor / Buffer Pool should be used for the 3rd Rx buffer in a host type packet Table 101 -Rx Channel N Host Packet Configuration Registers B |

#### 16.4.5.76 TXGCR18 Register (offset = A40h) [reset = 0h]

TXGCR18 is shown in Figure 16-227 and described in Table 16-239.

**Figure 16-227. TXGCR18 Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| TX_ENABLE | TX_TEARDOWN | Reserved | | | | | |
| R/W-0h | R/W-0h | | | | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| Reserved | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| Reserved | | TX_DEFAULT_QMGR | | TX_DEFAULT_QNUM | | | |
| | | W-0h | | W-0h | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| TX_DEFAULT_QNUM | | | | | | | |
| W-0h | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-239. TXGCR18 Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31 | TX_ENABLE | R/W | 0h | This field enables or disables the channel<br>0 = channel is disabled<br>1 = channel is enabled This field will be cleared after a channel teardown is complete. |
| 30 | TX_TEARDOWN | R/W | 0h | Setting this bit will request the channel to be torn down.<br>This field will remain set after a channel teardown is complete. |
| 13-12 | TX_DEFAULT_QMGR | W | 0h | This field controls the default queue manager number that will be used to queue teardown descriptors back to the host. |
| 11-0 | TX_DEFAULT_QNUM | W | 0h | This field controls the default queue number within the selected queue manager onto which teardown descriptors will be queued back to the host.<br>Table<br>98 -Tx Channel N Global Configuration Registers |

### 16.4.5.77 RXGCR18 Register (offset = A48h) [reset = 0h]

RXGCR18 is shown in Figure 16-228 and described in Table 16-240.

**Figure 16-228. RXGCR18 Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| RX_ENABLE | RX_TEARDOWN | RX_PAUSE | Reserved | | | | RX_ERROR_HANDLING |
| R/W-0h | R/W-0h | R/W-0h | | | | | W-0h |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| RX_SOP_OFFSET | | | | | | | |
| W-0h | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| RX_DEFAULT_DESC_TYPE | | RX_DEFAULT_RQ_QMGR | | RX_DEFAULT_RQ_QNUM | | | |
| W-0h | | W-0h | | W-0h | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| RX_DEFAULT_RQ_QNUM | | | | | | | |
| W-0h | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-240. RXGCR18 Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31 | RX_ENABLE | R/W | 0h | This field enables or disables the channel<br>0 = channel is disabled<br>1 = channel is enabled This field will be cleared after a channel teardown is complete. |
| 30 | RX_TEARDOWN | R/W | 0h | This field indicates whether or not an Rx teardown operation is complete.<br>This field should be cleared when a channel is initialized.<br>This field will be set after a channel teardown is complete. |
| 29 | RX_PAUSE | R/W | 0h | Setting this bit causes the CPPI DMA to be suspended for rx channels.<br>If a pause is being requested and the channel is not in a packet then drop the credit. |
| 24 | RX_ERROR_HANDLING | W | 0h | This bit controls the error handling mode for the channel and is only used when channel errors (i.e.<br>descriptor or buffer starvation occurs):<br>0 = Starvation errors result in dropping packet and reclaiming any used descriptor or buffer resources back to the original queues/pools they were allocated to<br>1 = Starvation errors result in subsequent re-try of the descriptor allocation operation.<br>In this mode, the DMA will return to the IDLE state without saving it's internal operational state back to the internal state RAM and without issuing an advance operation on the FIFO interface.<br>This results in the DMA re-initiating the FIFO block transfer at a later time with the intention that additional free buffers and/or descriptors will have been added.<br>Regardless of the value of this bit, the DMA will assert the cdma_rx_sof_overrun (for SOP) or cdma_rx_mof_overrun (for non-SOP) when |
| 23-16 | RX_SOP_OFFSET | W | 0h | This field specifies the number of bytes that are to be skipped in the SOP buffer before beginning to write the payload.<br>This value must be less than the minimum size of a buffer in the system.<br>Valid values are<br>0 - 255 bytes. |

### Table 16-240. RXGCR18 Register Field Descriptions (continued)

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 15-14 | RX_DEFAULT_DESC_TYPE | W | 0h | This field indicates the default descriptor type to use:<br>0 = Reserved<br>1 = Host<br>2 = Reserved<br>3 = Reserved The actual descriptor type that will be used for reception can be overridden by information provided in the CPPI FIFO data block. |
| 13-12 | RX_DEFAULT_RQ_QMGR | W | 0h | This field indicates the default receive queue manager that this channel should use.<br>The actual receive queue manager index can be overridden by information provided in the CPPI FIFO data block. |
| 11-0 | RX_DEFAULT_RQ_QNUM | W | 0h | This field indicates the default receive queue that this channel should use.<br>The actual receive queue that will be used for reception can be overridden by information provided in the CPPI FIFO data block. Table<br>99 -Rx Channel N Global Configuration Registers |

### 16.4.5.78 RXHPCRA18 Register (offset = A4Ch) [reset = 0h]

RXHPCRA18 is shown in Figure 16-229 and described in Table 16-241.

**Figure 16-229. RXHPCRA18 Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| Reserved | | RX_HOST_FDQ1_QMGR | | RX_HOST_FDQ1_QNUM | | | |
| | | W-0h | | W-0h | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| RX_HOST_FDQ1_QNUM | | | | | | | |
| W-0h | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| Reserved | | RX_HOST_FDQ0_QMGR | | RX_HOST_FDQ0_QNUM | | | |
| | | W-0h | | W-0h | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| RX_HOST_FDQ0_QNUM | | | | | | | |
| W-0h | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-241. RXHPCRA18 Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 29-28 | RX_HOST_FDQ1_QMGR | W | 0h | This field specifies which Buffer Manager should be used for the second Rx buffer in a host type packet. |
| 27-16 | RX_HOST_FDQ1_QNUM | W | 0h | This field specifies which Free Descriptor / Buffer Pool should be used for the 2nd Rx buffer in a host type packet |
| 13-12 | RX_HOST_FDQ0_QMGR | W | 0h | This field specifies which Buffer Manager should be used for the second Rx buffer in a host type packet. |
| 11-0 | RX_HOST_FDQ0_QNUM | W | 0h | This field specifies which Free Descriptor / Buffer Pool should be used for the 1st Rx buffer in a host type packet Table 100 -Rx Channel N Host Packet Configuration Registers A |

### 16.4.5.79 RXHPCRB18 Register (offset = A50h) [reset = 0h]

RXHPCRB18 is shown in Figure 16-230 and described in Table 16-242.

**Figure 16-230. RXHPCRB18 Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| Reserved | | RX_HOST_FDQ3_QMGR | | RX_HOST_FDQ3_QNUM | | | |
| | | W-0h | | W-0h | | | |
| **23** | **22** | **21** | **20** | **19** | **18** | **17** | **16** |
| RX_HOST_FDQ3_QNUM | | | | | | | |
| W-0h | | | | | | | |
| **15** | **14** | **13** | **12** | **11** | **10** | **9** | **8** |
| Reserved | | RX_HOST_FDQ2_QMGR | | RX_HOST_FDQ2_QNUM | | | |
| | | W-0h | | W-0h | | | |
| **7** | **6** | **5** | **4** | **3** | **2** | **1** | **0** |
| RX_HOST_FDQ2_QNUM | | | | | | | |
| W-0h | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-242. RXHPCRB18 Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 29-28 | RX_HOST_FDQ3_QMGR | W | 0h | This field specifies which Manager should be used for the 4th or later Rx buffers in a host type packet |
| 27-16 | RX_HOST_FDQ3_QNUM | W | 0h | This field specifies which Free Descriptor Queue should be used for the 4th or later Rx buffers in a host type packet |
| 13-12 | RX_HOST_FDQ2_QMGR | W | 0h | This field specifies which Buffer Manager should be used for the 3rd Rx buffer in a host type packet |
| 11-0 | RX_HOST_FDQ2_QNUM | W | 0h | This field specifies which Free Descriptor / Buffer Pool should be used for the 3rd Rx buffer in a host type packet Table 101 -Rx Channel N Host Packet Configuration Registers B |

### 16.4.5.80 TXGCR19 Register (offset = A60h) [reset = 0h]

TXGCR19 is shown in Figure 16-231 and described in Table 16-243.

**Figure 16-231. TXGCR19 Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| TX_ENABLE | TX_TEARDOWN | Reserved | | | | | |
| R/W-0h | R/W-0h | | | | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| Reserved | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| Reserved | | TX_DEFAULT_QMGR | | TX_DEFAULT_QNUM | | | |
| | | W-0h | | W-0h | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| TX_DEFAULT_QNUM | | | | | | | |
| W-0h | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-243. TXGCR19 Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31 | TX_ENABLE | R/W | 0h | This field enables or disables the channel<br>0 = channel is disabled<br>1 = channel is enabled This field will be cleared after a channel teardown is complete. |
| 30 | TX_TEARDOWN | R/W | 0h | Setting this bit will request the channel to be torn down.<br>This field will remain set after a channel teardown is complete. |
| 13-12 | TX_DEFAULT_QMGR | W | 0h | This field controls the default queue manager number that will be used to queue teardown descriptors back to the host. |
| 11-0 | TX_DEFAULT_QNUM | W | 0h | This field controls the default queue number within the selected queue manager onto which teardown descriptors will be queued back to the host.<br>Table<br>98 -Tx Channel N Global Configuration Registers |

### 16.4.5.81 RXGCR19 Register (offset = A68h) [reset = 0h]

RXGCR19 is shown in Figure 16-232 and described in Table 16-244.

**Figure 16-232. RXGCR19 Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| RX_ENABLE | RX_TEARDOWN | RX_PAUSE | Reserved | | | | RX_ERROR_HANDLING |
| R/W-0h | R/W-0h | R/W-0h | | | | | W-0h |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| RX_SOP_OFFSET | | | | | | | |
| W-0h | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| RX_DEFAULT_DESC_TYPE | | RX_DEFAULT_RQ_QMGR | | RX_DEFAULT_RQ_QNUM | | | |
| W-0h | | W-0h | | W-0h | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| RX_DEFAULT_RQ_QNUM | | | | | | | |
| W-0h | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-244. RXGCR19 Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31 | RX_ENABLE | R/W | 0h | This field enables or disables the channel<br>0 = channel is disabled<br>1 = channel is enabled This field will be cleared after a channel teardown is complete. |
| 30 | RX_TEARDOWN | R/W | 0h | This field indicates whether or not an Rx teardown operation is complete.<br>This field should be cleared when a channel is initialized.<br>This field will be set after a channel teardown is complete. |
| 29 | RX_PAUSE | R/W | 0h | Setting this bit causes the CPPI DMA to be suspended for rx channels.<br>If a pause is being requested and the channel is not in a packet then drop the credit. |
| 24 | RX_ERROR_HANDLING | W | 0h | This bit controls the error handling mode for the channel and is only used when channel errors (i.e.<br>descriptor or buffer starvation occurs):<br>0 = Starvation errors result in dropping packet and reclaiming any used descriptor or buffer resources back to the original queues/pools they were allocated to<br>1 = Starvation errors result in subsequent re-try of the descriptor allocation operation.<br>In this mode, the DMA will return to the IDLE state without saving it's internal operational state back to the internal state RAM and without issuing an advance operation on the FIFO interface.<br>This results in the DMA re-initiating the FIFO block transfer at a later time with the intention that additional free buffers and/or descriptors will have been added.<br>Regardless of the value of this bit, the DMA will assert the cdma_rx_sof_overrun (for SOP) or cdma_rx_mof_overrun (for non-SOP) when |
| 23-16 | RX_SOP_OFFSET | W | 0h | This field specifies the number of bytes that are to be skipped in the SOP buffer before beginning to write the payload.<br>This value must be less than the minimum size of a buffer in the system.<br>Valid values are<br>0 - 255 bytes. |

**Table 16-244. RXGCR19 Register Field Descriptions (continued)**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 15-14 | RX_DEFAULT_DESC_TYPE | W | 0h | This field indicates the default descriptor type to use:<br>0 = Reserved<br>1 = Host<br>2 = Reserved<br>3 = Reserved The actual descriptor type that will be used for reception can be overridden by information provided in the CPPI FIFO data block. |
| 13-12 | RX_DEFAULT_RQ_QMGR | W | 0h | This field indicates the default receive queue manager that this channel should use.<br>The actual receive queue manager index can be overridden by information provided in the CPPI FIFO data block. |
| 11-0 | RX_DEFAULT_RQ_QNUM | W | 0h | This field indicates the default receive queue that this channel should use.<br>The actual receive queue that will be used for reception can be overridden by information provided in the CPPI FIFO data block. Table<br>99 -Rx Channel N Global Configuration Registers |

### 16.4.5.82 RXHPCRA19 Register (offset = A6Ch) [reset = 0h]

RXHPCRA19 is shown in Figure 16-233 and described in Table 16-245.

**Figure 16-233. RXHPCRA19 Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| Reserved | | RX_HOST_FDQ1_QMGR | | RX_HOST_FDQ1_QNUM | | | |
| | | W-0h | | W-0h | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| RX_HOST_FDQ1_QNUM | | | | | | | |
| W-0h | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| Reserved | | RX_HOST_FDQ0_QMGR | | RX_HOST_FDQ0_QNUM | | | |
| | | W-0h | | W-0h | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| RX_HOST_FDQ0_QNUM | | | | | | | |
| W-0h | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-245. RXHPCRA19 Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 29-28 | RX_HOST_FDQ1_QMGR | W | 0h | This field specifies which Buffer Manager should be used for the second Rx buffer in a host type packet. |
| 27-16 | RX_HOST_FDQ1_QNUM | W | 0h | This field specifies which Free Descriptor / Buffer Pool should be used for the 2nd Rx buffer in a host type packet |
| 13-12 | RX_HOST_FDQ0_QMGR | W | 0h | This field specifies which Buffer Manager should be used for the second Rx buffer in a host type packet. |
| 11-0 | RX_HOST_FDQ0_QNUM | W | 0h | This field specifies which Free Descriptor / Buffer Pool should be used for the 1st Rx buffer in a host type packet Table 100 -Rx Channel N Host Packet Configuration Registers A |

### 16.4.5.83 RXHPCRB19 Register (offset = A70h) [reset = 0h]

RXHPCRB19 is shown in Figure 16-234 and described in Table 16-246.

**Figure 16-234. RXHPCRB19 Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| Reserved | | RX_HOST_FDQ3_QMGR | | RX_HOST_FDQ3_QNUM | | | |
| | | W-0h | | W-0h | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| RX_HOST_FDQ3_QNUM | | | | | | | |
| W-0h | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| Reserved | | RX_HOST_FDQ2_QMGR | | RX_HOST_FDQ2_QNUM | | | |
| | | W-0h | | W-0h | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| RX_HOST_FDQ2_QNUM | | | | | | | |
| W-0h | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-246. RXHPCRB19 Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 29-28 | RX_HOST_FDQ3_QMGR | W | 0h | This field specifies which Manager should be used for the 4th or later Rx buffers in a host type packet |
| 27-16 | RX_HOST_FDQ3_QNUM | W | 0h | This field specifies which Free Descriptor Queue should be used for the 4th or later Rx buffers in a host type packet |
| 13-12 | RX_HOST_FDQ2_QMGR | W | 0h | This field specifies which Buffer Manager should be used for the 3rd Rx buffer in a host type packet |
| 11-0 | RX_HOST_FDQ2_QNUM | W | 0h | This field specifies which Free Descriptor / Buffer Pool should be used for the 3rd Rx buffer in a host type packet Table 101 -Rx Channel N Host Packet Configuration Registers B |

### 16.4.5.84 TXGCR20 Register (offset = A80h) [reset = 0h]

TXGCR20 is shown in Figure 16-235 and described in Table 16-247.

**Figure 16-235. TXGCR20 Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| TX_ENABLE | TX_TEARDOWN | Reserved | | | | | |
| R/W-0h | R/W-0h | | | | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| Reserved | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| Reserved | | TX_DEFAULT_QMGR | | TX_DEFAULT_QNUM | | | |
| | | W-0h | | W-0h | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| TX_DEFAULT_QNUM | | | | | | | |
| W-0h | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-247. TXGCR20 Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31 | TX_ENABLE | R/W | 0h | This field enables or disables the channel<br>0 = channel is disabled<br>1 = channel is enabled This field will be cleared after a channel teardown is complete. |
| 30 | TX_TEARDOWN | R/W | 0h | Setting this bit will request the channel to be torn down.<br>This field will remain set after a channel teardown is complete. |
| 13-12 | TX_DEFAULT_QMGR | W | 0h | This field controls the default queue manager number that will be used to queue teardown descriptors back to the host. |
| 11-0 | TX_DEFAULT_QNUM | W | 0h | This field controls the default queue number within the selected queue manager onto which teardown descriptors will be queued back to the host.<br>Table<br>98 -Tx Channel N Global Configuration Registers |

### 16.4.5.85 RXGCR20 Register (offset = A88h) [reset = 0h]

RXGCR20 is shown in Figure 16-236 and described in Table 16-248.

**Figure 16-236. RXGCR20 Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| RX_ENABLE | RX_TEARDOWN | RX_PAUSE | Reserved | | | | RX_ERROR_HANDLING |
| R/W-0h | R/W-0h | R/W-0h | | | | | W-0h |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| RX_SOP_OFFSET | | | | | | | |
| W-0h | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| RX_DEFAULT_DESC_TYPE | | RX_DEFAULT_RQ_QMGR | | RX_DEFAULT_RQ_QNUM | | | |
| W-0h | | W-0h | | W-0h | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| RX_DEFAULT_RQ_QNUM | | | | | | | |
| W-0h | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-248. RXGCR20 Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31 | RX_ENABLE | R/W | 0h | This field enables or disables the channel<br>0 = channel is disabled<br>1 = channel is enabled This field will be cleared after a channel teardown is complete. |
| 30 | RX_TEARDOWN | R/W | 0h | This field indicates whether or not an Rx teardown operation is complete.<br>This field should be cleared when a channel is initialized.<br>This field will be set after a channel teardown is complete. |
| 29 | RX_PAUSE | R/W | 0h | Setting this bit causes the CPPI DMA to be suspended for rx channels.<br>If a pause is being requested and the channel is not in a packet then drop the credit. |
| 24 | RX_ERROR_HANDLING | W | 0h | This bit controls the error handling mode for the channel and is only used when channel errors (i.e.<br>descriptor or buffer starvation occurs):<br>0 = Starvation errors result in dropping packet and reclaiming any used descriptor or buffer resources back to the original queues/pools they were allocated to<br>1 = Starvation errors result in subsequent re-try of the descriptor allocation operation.<br>In this mode, the DMA will return to the IDLE state without saving it's internal operational state back to the internal state RAM and without issuing an advance operation on the FIFO interface.<br>This results in the DMA re-initiating the FIFO block transfer at a later time with the intention that additional free buffers and/or descriptors will have been added.<br>Regardless of the value of this bit, the DMA will assert the cdma_rx_sof_overrun (for SOP) or cdma_rx_mof_overrun (for non-SOP) when |
| 23-16 | RX_SOP_OFFSET | W | 0h | This field specifies the number of bytes that are to be skipped in the SOP buffer before beginning to write the payload.<br>This value must be less than the minimum size of a buffer in the system.<br>Valid values are<br>0 - 255 bytes. |

### Table 16-248. RXGCR20 Register Field Descriptions (continued)

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 15-14 | RX_DEFAULT_DESC_TYPE | W | 0h | This field indicates the default descriptor type to use:<br>0 = Reserved<br>1 = Host<br>2 = Reserved<br>3 = Reserved The actual descriptor type that will be used for reception can be overridden by information provided in the CPPI FIFO data block. |
| 13-12 | RX_DEFAULT_RQ_QMGR | W | 0h | This field indicates the default receive queue manager that this channel should use.<br>The actual receive queue manager index can be overridden by information provided in the CPPI FIFO data block. |
| 11-0 | RX_DEFAULT_RQ_QNUM | W | 0h | This field indicates the default receive queue that this channel should use.<br>The actual receive queue that will be used for reception can be overridden by information provided in the CPPI FIFO data block. Table<br>99 -Rx Channel N Global Configuration Registers |

### 16.4.5.86 RXHPCRA20 Register (offset = A8Ch) [reset = 0h]

RXHPCRA20 is shown in Figure 16-237 and described in Table 16-249.

**Figure 16-237. RXHPCRA20 Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| Reserved | | RX_HOST_FDQ1_QMGR | | RX_HOST_FDQ1_QNUM | | | |
| | | W-0h | | W-0h | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| RX_HOST_FDQ1_QNUM | | | | | | | |
| W-0h | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| Reserved | | RX_HOST_FDQ0_QMGR | | RX_HOST_FDQ0_QNUM | | | |
| | | W-0h | | W-0h | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| RX_HOST_FDQ0_QNUM | | | | | | | |
| W-0h | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-249. RXHPCRA20 Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 29-28 | RX_HOST_FDQ1_QMGR | W | 0h | This field specifies which Buffer Manager should be used for the second Rx buffer in a host type packet. |
| 27-16 | RX_HOST_FDQ1_QNUM | W | 0h | This field specifies which Free Descriptor / Buffer Pool should be used for the 2nd Rx buffer in a host type packet |
| 13-12 | RX_HOST_FDQ0_QMGR | W | 0h | This field specifies which Buffer Manager should be used for the second Rx buffer in a host type packet. |
| 11-0 | RX_HOST_FDQ0_QNUM | W | 0h | This field specifies which Free Descriptor / Buffer Pool should be used for the 1st Rx buffer in a host type packet Table 100 -Rx Channel N Host Packet Configuration Registers A |

### 16.4.5.87 RXHPCRB20 Register (offset = A90h) [reset = 0h]

RXHPCRB20 is shown in Figure 16-238 and described in Table 16-250.

**Figure 16-238. RXHPCRB20 Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| Reserved | | RX_HOST_FDQ3_QMGR | | RX_HOST_FDQ3_QNUM | | | |
| | | W-0h | | W-0h | | | |
| **23** | **22** | **21** | **20** | **19** | **18** | **17** | **16** |
| RX_HOST_FDQ3_QNUM | | | | | | | |
| W-0h | | | | | | | |
| **15** | **14** | **13** | **12** | **11** | **10** | **9** | **8** |
| Reserved | | RX_HOST_FDQ2_QMGR | | RX_HOST_FDQ2_QNUM | | | |
| | | W-0h | | W-0h | | | |
| **7** | **6** | **5** | **4** | **3** | **2** | **1** | **0** |
| RX_HOST_FDQ2_QNUM | | | | | | | |
| W-0h | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-250. RXHPCRB20 Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 29-28 | RX_HOST_FDQ3_QMGR | W | 0h | This field specifies which Manager should be used for the 4th or later Rx buffers in a host type packet |
| 27-16 | RX_HOST_FDQ3_QNUM | W | 0h | This field specifies which Free Descriptor Queue should be used for the 4th or later Rx buffers in a host type packet |
| 13-12 | RX_HOST_FDQ2_QMGR | W | 0h | This field specifies which Buffer Manager should be used for the 3rd Rx buffer in a host type packet |
| 11-0 | RX_HOST_FDQ2_QNUM | W | 0h | This field specifies which Free Descriptor / Buffer Pool should be used for the 3rd Rx buffer in a host type packet Table 101 -Rx Channel N Host Packet Configuration Registers B |

### 16.4.5.88 TXGCR21 Register (offset = AA0h) [reset = 0h]

TXGCR21 is shown in Figure 16-239 and described in Table 16-251.

**Figure 16-239. TXGCR21 Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| TX_ENABLE | TX_TEARDOWN | Reserved | | | | | |
| R/W-0h | R/W-0h | | | | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| Reserved | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| Reserved | | TX_DEFAULT_QMGR | | TX_DEFAULT_QNUM | | | |
| | | W-0h | | W-0h | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| TX_DEFAULT_QNUM | | | | | | | |
| W-0h | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-251. TXGCR21 Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31 | TX_ENABLE | R/W | 0h | This field enables or disables the channel<br>0 = channel is disabled<br>1 = channel is enabled This field will be cleared after a channel teardown is complete. |
| 30 | TX_TEARDOWN | R/W | 0h | Setting this bit will request the channel to be torn down.<br>This field will remain set after a channel teardown is complete. |
| 13-12 | TX_DEFAULT_QMGR | W | 0h | This field controls the default queue manager number that will be used to queue teardown descriptors back to the host. |
| 11-0 | TX_DEFAULT_QNUM | W | 0h | This field controls the default queue number within the selected queue manager onto which teardown descriptors will be queued back to the host.<br>Table<br>98 -Tx Channel N Global Configuration Registers |

### 16.4.5.89 RXGCR21 Register (offset = AA8h) [reset = 0h]

RXGCR21 is shown in Figure 16-240 and described in Table 16-252.

**Figure 16-240. RXGCR21 Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| RX_ENABLE | RX_TEARDOWN | RX_PAUSE | Reserved | | | | RX_ERROR_HANDLING |
| R/W-0h | R/W-0h | R/W-0h | | | | | W-0h |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| RX_SOP_OFFSET | | | | | | | |
| W-0h | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| RX_DEFAULT_DESC_TYPE | | RX_DEFAULT_RQ_QMGR | | RX_DEFAULT_RQ_QNUM | | | |
| W-0h | | W-0h | | W-0h | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| RX_DEFAULT_RQ_QNUM | | | | | | | |
| W-0h | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-252. RXGCR21 Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31 | RX_ENABLE | R/W | 0h | This field enables or disables the channel<br>0 = channel is disabled<br>1 = channel is enabled This field will be cleared after a channel teardown is complete. |
| 30 | RX_TEARDOWN | R/W | 0h | This field indicates whether or not an Rx teardown operation is complete.<br>This field should be cleared when a channel is initialized.<br>This field will be set after a channel teardown is complete. |
| 29 | RX_PAUSE | R/W | 0h | Setting this bit causes the CPPI DMA to be suspended for rx channels.<br>If a pause is being requested and the channel is not in a packet then drop the credit. |
| 24 | RX_ERROR_HANDLING | W | 0h | This bit controls the error handling mode for the channel and is only used when channel errors (i.e.<br>descriptor or buffer starvation occurs):<br>0 = Starvation errors result in dropping packet and reclaiming any used descriptor or buffer resources back to the original queues/pools they were allocated to<br>1 = Starvation errors result in subsequent re-try of the descriptor allocation operation.<br>In this mode, the DMA will return to the IDLE state without saving it's internal operational state back to the internal state RAM and without issuing an advance operation on the FIFO interface.<br>This results in the DMA re-initiating the FIFO block transfer at a later time with the intention that additional free buffers and/or descriptors will have been added.<br>Regardless of the value of this bit, the DMA will assert the cdma_rx_sof_overrun (for SOP) or cdma_rx_mof_overrun (for non-SOP) when |
| 23-16 | RX_SOP_OFFSET | W | 0h | This field specifies the number of bytes that are to be skipped in the SOP buffer before beginning to write the payload.<br>This value must be less than the minimum size of a buffer in the system.<br>Valid values are<br>0 - 255 bytes. |

**Table 16-252. RXGCR21 Register Field Descriptions (continued)**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 15-14 | RX_DEFAULT_DESC_TYPE | W | 0h | This field indicates the default descriptor type to use:<br>0 = Reserved<br>1 = Host<br>2 = Reserved<br>3 = Reserved The actual descriptor type that will be used for reception can be overridden by information provided in the CPPI FIFO data block. |
| 13-12 | RX_DEFAULT_RQ_QMGR | W | 0h | This field indicates the default receive queue manager that this channel should use.<br>The actual receive queue manager index can be overridden by information provided in the CPPI FIFO data block. |
| 11-0 | RX_DEFAULT_RQ_QNUM | W | 0h | This field indicates the default receive queue that this channel should use.<br>The actual receive queue that will be used for reception can be overridden by information provided in the CPPI FIFO data block. Table<br>99 -Rx Channel N Global Configuration Registers |

### 16.4.5.90 RXHPCRA21 Register (offset = AACh) [reset = 0h]

RXHPCRA21 is shown in Figure 16-241 and described in Table 16-253.

**Figure 16-241. RXHPCRA21 Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| Reserved | | RX_HOST_FDQ1_QMGR | | RX_HOST_FDQ1_QNUM | | | |
| | | W-0h | | W-0h | | | |
| **23** | **22** | **21** | **20** | **19** | **18** | **17** | **16** |
| RX_HOST_FDQ1_QNUM | | | | | | | |
| W-0h | | | | | | | |
| **15** | **14** | **13** | **12** | **11** | **10** | **9** | **8** |
| Reserved | | RX_HOST_FDQ0_QMGR | | RX_HOST_FDQ0_QNUM | | | |
| | | W-0h | | W-0h | | | |
| **7** | **6** | **5** | **4** | **3** | **2** | **1** | **0** |
| RX_HOST_FDQ0_QNUM | | | | | | | |
| W-0h | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-253. RXHPCRA21 Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 29-28 | RX_HOST_FDQ1_QMGR | W | 0h | This field specifies which Buffer Manager should be used for the second Rx buffer in a host type packet. |
| 27-16 | RX_HOST_FDQ1_QNUM | W | 0h | This field specifies which Free Descriptor / Buffer Pool should be used for the 2nd Rx buffer in a host type packet |
| 13-12 | RX_HOST_FDQ0_QMGR | W | 0h | This field specifies which Buffer Manager should be used for the second Rx buffer in a host type packet. |
| 11-0 | RX_HOST_FDQ0_QNUM | W | 0h | This field specifies which Free Descriptor / Buffer Pool should be used for the 1st Rx buffer in a host type packet Table 100 -Rx Channel N Host Packet Configuration Registers A |

### 16.4.5.91 RXHPCRB21 Register (offset = AB0h) [reset = 0h]

RXHPCRB21 is shown in Figure 16-242 and described in Table 16-254.

**Figure 16-242. RXHPCRB21 Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| Reserved | | RX_HOST_FDQ3_QMGR | | RX_HOST_FDQ3_QNUM | | | |
| | | W-0h | | W-0h | | | |
| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
| RX_HOST_FDQ3_QNUM | | | | | | | |
| W-0h | | | | | | | |
| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
| Reserved | | RX_HOST_FDQ2_QMGR | | RX_HOST_FDQ2_QNUM | | | |
| | | W-0h | | W-0h | | | |
| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
| RX_HOST_FDQ2_QNUM | | | | | | | |
| W-0h | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-254. RXHPCRB21 Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 29-28 | RX_HOST_FDQ3_QMGR | W | 0h | This field specifies which Manager should be used for the 4th or later Rx buffers in a host type packet |
| 27-16 | RX_HOST_FDQ3_QNUM | W | 0h | This field specifies which Free Descriptor Queue should be used for the 4th or later Rx buffers in a host type packet |
| 13-12 | RX_HOST_FDQ2_QMGR | W | 0h | This field specifies which Buffer Manager should be used for the 3rd Rx buffer in a host type packet |
| 11-0 | RX_HOST_FDQ2_QNUM | W | 0h | This field specifies which Free Descriptor / Buffer Pool should be used for the 3rd Rx buffer in a host type packet Table 101 -Rx Channel N Host Packet Configuration Registers B |

### 16.4.5.92 TXGCR22 Register (offset = AC0h) [reset = 0h]

TXGCR22 is shown in Figure 16-243 and described in Table 16-255.

**Figure 16-243. TXGCR22 Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| TX_ENABLE | TX_TEARDOWN | Reserved | | | | | |
| R/W-0h | R/W-0h | | | | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| Reserved | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| Reserved | | TX_DEFAULT_QMGR | | TX_DEFAULT_QNUM | | | |
| | | W-0h | | W-0h | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| TX_DEFAULT_QNUM | | | | | | | |
| W-0h | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-255. TXGCR22 Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31 | TX_ENABLE | R/W | 0h | This field enables or disables the channel<br>0 = channel is disabled<br>1 = channel is enabled This field will be cleared after a channel teardown is complete. |
| 30 | TX_TEARDOWN | R/W | 0h | Setting this bit will request the channel to be torn down.<br>This field will remain set after a channel teardown is complete. |
| 13-12 | TX_DEFAULT_QMGR | W | 0h | This field controls the default queue manager number that will be used to queue teardown descriptors back to the host. |
| 11-0 | TX_DEFAULT_QNUM | W | 0h | This field controls the default queue number within the selected queue manager onto which teardown descriptors will be queued back to the host.<br>Table<br>98 -Tx Channel N Global Configuration Registers |

### 16.4.5.93 RXGCR22 Register (offset = AC8h) [reset = 0h]

RXGCR22 is shown in Figure 16-244 and described in Table 16-256.

**Figure 16-244. RXGCR22 Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| RX_ENABLE | RX_TEARDOWN | RX_PAUSE | Reserved | | | | RX_ERROR_HANDLING |
| R/W-0h | R/W-0h | R/W-0h | | | | | W-0h |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| RX_SOP_OFFSET | | | | | | | |
| W-0h | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| RX_DEFAULT_DESC_TYPE | | RX_DEFAULT_RQ_QMGR | | RX_DEFAULT_RQ_QNUM | | | |
| W-0h | | W-0h | | W-0h | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| RX_DEFAULT_RQ_QNUM | | | | | | | |
| W-0h | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-256. RXGCR22 Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31 | RX_ENABLE | R/W | 0h | This field enables or disables the channel<br>0 = channel is disabled<br>1 = channel is enabled This field will be cleared after a channel teardown is complete. |
| 30 | RX_TEARDOWN | R/W | 0h | This field indicates whether or not an Rx teardown operation is complete.<br>This field should be cleared when a channel is initialized.<br>This field will be set after a channel teardown is complete. |
| 29 | RX_PAUSE | R/W | 0h | Setting this bit causes the CPPI DMA to be suspended for rx channels.<br>If a pause is being requested and the channel is not in a packet then drop the credit. |
| 24 | RX_ERROR_HANDLING | W | 0h | This bit controls the error handling mode for the channel and is only used when channel errors (i.e.<br>descriptor or buffer starvation occurs):<br>0 = Starvation errors result in dropping packet and reclaiming any used descriptor or buffer resources back to the original queues/pools they were allocated to<br>1 = Starvation errors result in subsequent re-try of the descriptor allocation operation.<br>In this mode, the DMA will return to the IDLE state without saving it's internal operational state back to the internal state RAM and without issuing an advance operation on the FIFO interface.<br>This results in the DMA re-initiating the FIFO block transfer at a later time with the intention that additional free buffers and/or descriptors will have been added.<br>Regardless of the value of this bit, the DMA will assert the cdma_rx_sof_overrun (for SOP) or cdma_rx_mof_overrun (for non-SOP) when |
| 23-16 | RX_SOP_OFFSET | W | 0h | This field specifies the number of bytes that are to be skipped in the SOP buffer before beginning to write the payload.<br>This value must be less than the minimum size of a buffer in the system.<br>Valid values are<br>0 - 255 bytes. |

### Table 16-256. RXGCR22 Register Field Descriptions (continued)

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 15-14 | RX_DEFAULT_DESC_TYPE | W | 0h | This field indicates the default descriptor type to use:<br>0 = Reserved<br>1 = Host<br>2 = Reserved<br>3 = Reserved The actual descriptor type that will be used for reception can be overridden by information provided in the CPPI FIFO data block. |
| 13-12 | RX_DEFAULT_RQ_QMGR | W | 0h | This field indicates the default receive queue manager that this channel should use.<br>The actual receive queue manager index can be overridden by information provided in the CPPI FIFO data block. |
| 11-0 | RX_DEFAULT_RQ_QNUM | W | 0h | This field indicates the default receive queue that this channel should use.<br>The actual receive queue that will be used for reception can be overridden by information provided in the CPPI FIFO data block. Table<br>99 -Rx Channel N Global Configuration Registers |

### 16.4.5.94 RXHPCRA22 Register (offset = ACCh) [reset = 0h]

RXHPCRA22 is shown in Figure 16-245 and described in Table 16-257.

**Figure 16-245. RXHPCRA22 Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| Reserved | | RX_HOST_FDQ1_QMGR | | RX_HOST_FDQ1_QNUM | | | |
| | | W-0h | | W-0h | | | |
| **23** | **22** | **21** | **20** | **19** | **18** | **17** | **16** |
| RX_HOST_FDQ1_QNUM | | | | | | | |
| W-0h | | | | | | | |
| **15** | **14** | **13** | **12** | **11** | **10** | **9** | **8** |
| Reserved | | RX_HOST_FDQ0_QMGR | | RX_HOST_FDQ0_QNUM | | | |
| | | W-0h | | W-0h | | | |
| **7** | **6** | **5** | **4** | **3** | **2** | **1** | **0** |
| RX_HOST_FDQ0_QNUM | | | | | | | |
| W-0h | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-257. RXHPCRA22 Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 29-28 | RX_HOST_FDQ1_QMGR | W | 0h | This field specifies which Buffer Manager should be used for the second Rx buffer in a host type packet. |
| 27-16 | RX_HOST_FDQ1_QNUM | W | 0h | This field specifies which Free Descriptor / Buffer Pool should be used for the 2nd Rx buffer in a host type packet |
| 13-12 | RX_HOST_FDQ0_QMGR | W | 0h | This field specifies which Buffer Manager should be used for the second Rx buffer in a host type packet. |
| 11-0 | RX_HOST_FDQ0_QNUM | W | 0h | This field specifies which Free Descriptor / Buffer Pool should be used for the 1st Rx buffer in a host type packet Table 100 -Rx Channel N Host Packet Configuration Registers A |

### 16.4.5.95 RXHPCRB22 Register (offset = AD0h) [reset = 0h]

RXHPCRB22 is shown in Figure 16-246 and described in Table 16-258.

**Figure 16-246. RXHPCRB22 Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| Reserved | | RX_HOST_FDQ3_QMGR | | RX_HOST_FDQ3_QNUM | | | |
| | | W-0h | | W-0h | | | |
| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
| RX_HOST_FDQ3_QNUM | | | | | | | |
| W-0h | | | | | | | |
| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
| Reserved | | RX_HOST_FDQ2_QMGR | | RX_HOST_FDQ2_QNUM | | | |
| | | W-0h | | W-0h | | | |
| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
| RX_HOST_FDQ2_QNUM | | | | | | | |
| W-0h | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-258. RXHPCRB22 Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 29-28 | RX_HOST_FDQ3_QMGR | W | 0h | This field specifies which Manager should be used for the 4th or later Rx buffers in a host type packet |
| 27-16 | RX_HOST_FDQ3_QNUM | W | 0h | This field specifies which Free Descriptor Queue should be used for the 4th or later Rx buffers in a host type packet |
| 13-12 | RX_HOST_FDQ2_QMGR | W | 0h | This field specifies which Buffer Manager should be used for the 3rd Rx buffer in a host type packet |
| 11-0 | RX_HOST_FDQ2_QNUM | W | 0h | This field specifies which Free Descriptor / Buffer Pool should be used for the 3rd Rx buffer in a host type packet Table 101 -Rx Channel N Host Packet Configuration Registers B |

### 16.4.5.96 TXGCR23 Register (offset = AE0h) [reset = 0h]

TXGCR23 is shown in Figure 16-247 and described in Table 16-259.

**Figure 16-247. TXGCR23 Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| TX_ENABLE | TX_TEARDOWN | Reserved | | | | | |
| R/W-0h | R/W-0h | | | | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| Reserved | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| Reserved | | TX_DEFAULT_QMGR | | TX_DEFAULT_QNUM | | | |
| | | W-0h | | W-0h | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| TX_DEFAULT_QNUM | | | | | | | |
| W-0h | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-259. TXGCR23 Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31 | TX_ENABLE | R/W | 0h | This field enables or disables the channel<br>0 = channel is disabled<br>1 = channel is enabled This field will be cleared after a channel teardown is complete. |
| 30 | TX_TEARDOWN | R/W | 0h | Setting this bit will request the channel to be torn down.<br>This field will remain set after a channel teardown is complete. |
| 13-12 | TX_DEFAULT_QMGR | W | 0h | This field controls the default queue manager number that will be used to queue teardown descriptors back to the host. |
| 11-0 | TX_DEFAULT_QNUM | W | 0h | This field controls the default queue number within the selected queue manager onto which teardown descriptors will be queued back to the host.<br>Table<br>98 -Tx Channel N Global Configuration Registers |

### 16.4.5.97 RXGCR23 Register (offset = AE8h) [reset = 0h]

RXGCR23 is shown in Figure 16-248 and described in Table 16-260.

**Figure 16-248. RXGCR23 Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| RX_ENABLE | RX_TEARDOWN | RX_PAUSE | Reserved | | | | RX_ERROR_HANDLING |
| R/W-0h | R/W-0h | R/W-0h | | | | | W-0h |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| RX_SOP_OFFSET | | | | | | | |
| W-0h | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| RX_DEFAULT_DESC_TYPE | | RX_DEFAULT_RQ_QMGR | | RX_DEFAULT_RQ_QNUM | | | |
| W-0h | | W-0h | | W-0h | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| RX_DEFAULT_RQ_QNUM | | | | | | | |
| W-0h | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-260. RXGCR23 Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31 | RX_ENABLE | R/W | 0h | This field enables or disables the channel<br>0 = channel is disabled<br>1 = channel is enabled This field will be cleared after a channel teardown is complete. |
| 30 | RX_TEARDOWN | R/W | 0h | This field indicates whether or not an Rx teardown operation is complete.<br>This field should be cleared when a channel is initialized.<br>This field will be set after a channel teardown is complete. |
| 29 | RX_PAUSE | R/W | 0h | Setting this bit causes the CPPI DMA to be suspended for rx channels.<br>If a pause is being requested and the channel is not in a packet then drop the credit. |
| 24 | RX_ERROR_HANDLING | W | 0h | This bit controls the error handling mode for the channel and is only used when channel errors (i.e.<br>descriptor or buffer starvation occurs):<br>0 = Starvation errors result in dropping packet and reclaiming any used descriptor or buffer resources back to the original queues/pools they were allocated to<br>1 = Starvation errors result in subsequent re-try of the descriptor allocation operation.<br>In this mode, the DMA will return to the IDLE state without saving it's internal operational state back to the internal state RAM and without issuing an advance operation on the FIFO interface.<br>This results in the DMA re-initiating the FIFO block transfer at a later time with the intention that additional free buffers and/or descriptors will have been added.<br>Regardless of the value of this bit, the DMA will assert the cdma_rx_sof_overrun (for SOP) or cdma_rx_mof_overrun (for non-SOP) when |
| 23-16 | RX_SOP_OFFSET | W | 0h | This field specifies the number of bytes that are to be skipped in the SOP buffer before beginning to write the payload.<br>This value must be less than the minimum size of a buffer in the system.<br>Valid values are<br>0 - 255 bytes. |

### Table 16-260. RXGCR23 Register Field Descriptions (continued)

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 15-14 | RX_DEFAULT_DESC_TYPE | W | 0h | This field indicates the default descriptor type to use:<br>0 = Reserved<br>1 = Host<br>2 = Reserved<br>3 = Reserved The actual descriptor type that will be used for reception can be overridden by information provided in the CPPI FIFO data block. |
| 13-12 | RX_DEFAULT_RQ_QMGR | W | 0h | This field indicates the default receive queue manager that this channel should use.<br>The actual receive queue manager index can be overridden by information provided in the CPPI FIFO data block. |
| 11-0 | RX_DEFAULT_RQ_QNUM | W | 0h | This field indicates the default receive queue that this channel should use.<br>The actual receive queue that will be used for reception can be overridden by information provided in the CPPI FIFO data block. Table<br>99 -Rx Channel N Global Configuration Registers |

### 16.4.5.98 RXHPCRA23 Register (offset = AECh) [reset = 0h]

RXHPCRA23 is shown in Figure 16-249 and described in Table 16-261.

**Figure 16-249. RXHPCRA23 Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| Reserved | | RX_HOST_FDQ1_QMGR | | RX_HOST_FDQ1_QNUM | | | |
| | | W-0h | | W-0h | | | |
| **23** | **22** | **21** | **20** | **19** | **18** | **17** | **16** |
| RX_HOST_FDQ1_QNUM | | | | | | | |
| W-0h | | | | | | | |
| **15** | **14** | **13** | **12** | **11** | **10** | **9** | **8** |
| Reserved | | RX_HOST_FDQ0_QMGR | | RX_HOST_FDQ0_QNUM | | | |
| | | W-0h | | W-0h | | | |
| **7** | **6** | **5** | **4** | **3** | **2** | **1** | **0** |
| RX_HOST_FDQ0_QNUM | | | | | | | |
| W-0h | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-261. RXHPCRA23 Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 29-28 | RX_HOST_FDQ1_QMGR | W | 0h | This field specifies which Buffer Manager should be used for the second Rx buffer in a host type packet. |
| 27-16 | RX_HOST_FDQ1_QNUM | W | 0h | This field specifies which Free Descriptor / Buffer Pool should be used for the 2nd Rx buffer in a host type packet |
| 13-12 | RX_HOST_FDQ0_QMGR | W | 0h | This field specifies which Buffer Manager should be used for the second Rx buffer in a host type packet. |
| 11-0 | RX_HOST_FDQ0_QNUM | W | 0h | This field specifies which Free Descriptor / Buffer Pool should be used for the 1st Rx buffer in a host type packet Table 100 -Rx Channel N Host Packet Configuration Registers A |

### 16.4.5.99 RXHPCRB23 Register (offset = AF0h) [reset = 0h]

RXHPCRB23 is shown in Figure 16-250 and described in Table 16-262.

**Figure 16-250. RXHPCRB23 Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| Reserved | | RX_HOST_FDQ3_QMGR | | RX_HOST_FDQ3_QNUM | | | |
| | | W-0h | | W-0h | | | |
| **23** | **22** | **21** | **20** | **19** | **18** | **17** | **16** |
| RX_HOST_FDQ3_QNUM | | | | | | | |
| W-0h | | | | | | | |
| **15** | **14** | **13** | **12** | **11** | **10** | **9** | **8** |
| Reserved | | RX_HOST_FDQ2_QMGR | | RX_HOST_FDQ2_QNUM | | | |
| | | W-0h | | W-0h | | | |
| **7** | **6** | **5** | **4** | **3** | **2** | **1** | **0** |
| RX_HOST_FDQ2_QNUM | | | | | | | |
| W-0h | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-262. RXHPCRB23 Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 29-28 | RX_HOST_FDQ3_QMGR | W | 0h | This field specifies which Manager should be used for the 4th or later Rx buffers in a host type packet |
| 27-16 | RX_HOST_FDQ3_QNUM | W | 0h | This field specifies which Free Descriptor Queue should be used for the 4th or later Rx buffers in a host type packet |
| 13-12 | RX_HOST_FDQ2_QMGR | W | 0h | This field specifies which Buffer Manager should be used for the 3rd Rx buffer in a host type packet |
| 11-0 | RX_HOST_FDQ2_QNUM | W | 0h | This field specifies which Free Descriptor / Buffer Pool should be used for the 3rd Rx buffer in a host type packet Table 101 -Rx Channel N Host Packet Configuration Registers B |

### 16.4.5.100 TXGCR24 Register (offset = B00h) [reset = 0h]

TXGCR24 is shown in Figure 16-251 and described in Table 16-263.

**Figure 16-251. TXGCR24 Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| TX_ENABLE | TX_TEARDOWN | Reserved | | | | | |
| R/W-0h | R/W-0h | | | | | | |
| **23** | **22** | **21** | **20** | **19** | **18** | **17** | **16** |
| Reserved | | | | | | | |
| **15** | **14** | **13** | **12** | **11** | **10** | **9** | **8** |
| Reserved | | TX_DEFAULT_QMGR | | TX_DEFAULT_QNUM | | | |
| | | W-0h | | W-0h | | | |
| **7** | **6** | **5** | **4** | **3** | **2** | **1** | **0** |
| TX_DEFAULT_QNUM | | | | | | | |
| W-0h | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-263. TXGCR24 Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31 | TX_ENABLE | R/W | 0h | This field enables or disables the channel<br>0 = channel is disabled<br>1 = channel is enabled This field will be cleared after a channel teardown is complete. |
| 30 | TX_TEARDOWN | R/W | 0h | Setting this bit will request the channel to be torn down.<br>This field will remain set after a channel teardown is complete. |
| 13-12 | TX_DEFAULT_QMGR | W | 0h | This field controls the default queue manager number that will be used to queue teardown descriptors back to the host. |
| 11-0 | TX_DEFAULT_QNUM | W | 0h | This field controls the default queue number within the selected queue manager onto which teardown descriptors will be queued back to the host.<br>Table<br>98 -Tx Channel N Global Configuration Registers |

### 16.4.5.101 RXGCR24 Register (offset = B08h) [reset = 0h]

RXGCR24 is shown in Figure 16-252 and described in Table 16-264.

**Figure 16-252. RXGCR24 Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| RX_ENABLE | RX_TEARDOWN | RX_PAUSE | Reserved | | | | RX_ERROR_HANDLING |
| R/W-0h | R/W-0h | R/W-0h | | | | | W-0h |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| RX_SOP_OFFSET | | | | | | | |
| W-0h | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| RX_DEFAULT_DESC_TYPE | | RX_DEFAULT_RQ_QMGR | | RX_DEFAULT_RQ_QNUM | | | |
| W-0h | | W-0h | | W-0h | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| RX_DEFAULT_RQ_QNUM | | | | | | | |
| W-0h | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-264. RXGCR24 Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31 | RX_ENABLE | R/W | 0h | This field enables or disables the channel<br>0 = channel is disabled<br>1 = channel is enabled This field will be cleared after a channel teardown is complete. |
| 30 | RX_TEARDOWN | R/W | 0h | This field indicates whether or not an Rx teardown operation is complete.<br>This field should be cleared when a channel is initialized.<br>This field will be set after a channel teardown is complete. |
| 29 | RX_PAUSE | R/W | 0h | Setting this bit causes the CPPI DMA to be suspended for rx channels.<br>If a pause is being requested and the channel is not in a packet then drop the credit. |
| 24 | RX_ERROR_HANDLING | W | 0h | This bit controls the error handling mode for the channel and is only used when channel errors (i.e.<br>descriptor or buffer starvation occurs):<br>0 = Starvation errors result in dropping packet and reclaiming any used descriptor or buffer resources back to the original queues/pools they were allocated to<br>1 = Starvation errors result in subsequent re-try of the descriptor allocation operation.<br>In this mode, the DMA will return to the IDLE state without saving it's internal operational state back to the internal state RAM and without issuing an advance operation on the FIFO interface.<br>This results in the DMA re-initiating the FIFO block transfer at a later time with the intention that additional free buffers and/or descriptors will have been added.<br>Regardless of the value of this bit, the DMA will assert the cdma_rx_sof_overrun (for SOP) or cdma_rx_mof_overrun (for non-SOP) when |
| 23-16 | RX_SOP_OFFSET | W | 0h | This field specifies the number of bytes that are to be skipped in the SOP buffer before beginning to write the payload.<br>This value must be less than the minimum size of a buffer in the system.<br>Valid values are<br>0 - 255 bytes. |

**Table 16-264. RXGCR24 Register Field Descriptions (continued)**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 15-14 | RX_DEFAULT_DESC_TYPE | W | 0h | This field indicates the default descriptor type to use:<br>0 = Reserved<br>1 = Host<br>2 = Reserved<br>3 = Reserved The actual descriptor type that will be used for reception can be overridden by information provided in the CPPI FIFO data block. |
| 13-12 | RX_DEFAULT_RQ_QMGR | W | 0h | This field indicates the default receive queue manager that this channel should use.<br>The actual receive queue manager index can be overridden by information provided in the CPPI FIFO data block. |
| 11-0 | RX_DEFAULT_RQ_QNUM | W | 0h | This field indicates the default receive queue that this channel should use.<br>The actual receive queue that will be used for reception can be overridden by information provided in the CPPI FIFO data block.<br>Table<br>99 -Rx Channel N Global Configuration Registers |

### 16.4.5.102 RXHPCRA24 Register (offset = B0Ch) [reset = 0h]

RXHPCRA24 is shown in Figure 16-253 and described in Table 16-265.

**Figure 16-253. RXHPCRA24 Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| Reserved | | RX_HOST_FDQ1_QMGR | | RX_HOST_FDQ1_QNUM | | | |
| | | W-0h | | W-0h | | | |
| **23** | **22** | **21** | **20** | **19** | **18** | **17** | **16** |
| RX_HOST_FDQ1_QNUM | | | | | | | |
| W-0h | | | | | | | |
| **15** | **14** | **13** | **12** | **11** | **10** | **9** | **8** |
| Reserved | | RX_HOST_FDQ0_QMGR | | RX_HOST_FDQ0_QNUM | | | |
| | | W-0h | | W-0h | | | |
| **7** | **6** | **5** | **4** | **3** | **2** | **1** | **0** |
| RX_HOST_FDQ0_QNUM | | | | | | | |
| W-0h | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-265. RXHPCRA24 Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 29-28 | RX_HOST_FDQ1_QMGR | W | 0h | This field specifies which Buffer Manager should be used for the second Rx buffer in a host type packet. |
| 27-16 | RX_HOST_FDQ1_QNUM | W | 0h | This field specifies which Free Descriptor / Buffer Pool should be used for the 2nd Rx buffer in a host type packet |
| 13-12 | RX_HOST_FDQ0_QMGR | W | 0h | This field specifies which Buffer Manager should be used for the second Rx buffer in a host type packet. |
| 11-0 | RX_HOST_FDQ0_QNUM | W | 0h | This field specifies which Free Descriptor / Buffer Pool should be used for the 1st Rx buffer in a host type packet Table 100 -Rx Channel N Host Packet Configuration Registers A |

### 16.4.5.103 RXHPCRB24 Register (offset = B10h) [reset = 0h]

RXHPCRB24 is shown in Figure 16-254 and described in Table 16-266.

**Figure 16-254. RXHPCRB24 Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| Reserved | | RX_HOST_FDQ3_QMGR | | RX_HOST_FDQ3_QNUM | | | |
| | | W-0h | | W-0h | | | |
| **23** | **22** | **21** | **20** | **19** | **18** | **17** | **16** |
| RX_HOST_FDQ3_QNUM | | | | | | | |
| W-0h | | | | | | | |
| **15** | **14** | **13** | **12** | **11** | **10** | **9** | **8** |
| Reserved | | RX_HOST_FDQ2_QMGR | | RX_HOST_FDQ2_QNUM | | | |
| | | W-0h | | W-0h | | | |
| **7** | **6** | **5** | **4** | **3** | **2** | **1** | **0** |
| RX_HOST_FDQ2_QNUM | | | | | | | |
| W-0h | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-266. RXHPCRB24 Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 29-28 | RX_HOST_FDQ3_QMGR | W | 0h | This field specifies which Manager should be used for the 4th or later Rx buffers in a host type packet |
| 27-16 | RX_HOST_FDQ3_QNUM | W | 0h | This field specifies which Free Descriptor Queue should be used for the 4th or later Rx buffers in a host type packet |
| 13-12 | RX_HOST_FDQ2_QMGR | W | 0h | This field specifies which Buffer Manager should be used for the 3rd Rx buffer in a host type packet |
| 11-0 | RX_HOST_FDQ2_QNUM | W | 0h | This field specifies which Free Descriptor / Buffer Pool should be used for the 3rd Rx buffer in a host type packet Table 101 -Rx Channel N Host Packet Configuration Registers B |

### 16.4.5.104 TXGCR25 Register (offset = B20h) [reset = 0h]

TXGCR25 is shown in Figure 16-255 and described in Table 16-267.

**Figure 16-255. TXGCR25 Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| TX_ENABLE | TX_TEARDOWN | Reserved | | | | | |
| R/W-0h | R/W-0h | | | | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| Reserved | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| Reserved | | TX_DEFAULT_QMGR | | TX_DEFAULT_QNUM | | | |
| | | W-0h | | W-0h | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| TX_DEFAULT_QNUM | | | | | | | |
| W-0h | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-267. TXGCR25 Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31 | TX_ENABLE | R/W | 0h | This field enables or disables the channel<br>0 = channel is disabled<br>1 = channel is enabled This field will be cleared after a channel teardown is complete. |
| 30 | TX_TEARDOWN | R/W | 0h | Setting this bit will request the channel to be torn down.<br>This field will remain set after a channel teardown is complete. |
| 13-12 | TX_DEFAULT_QMGR | W | 0h | This field controls the default queue manager number that will be used to queue teardown descriptors back to the host. |
| 11-0 | TX_DEFAULT_QNUM | W | 0h | This field controls the default queue number within the selected queue manager onto which teardown descriptors will be queued back to the host.<br>Table<br>98 -Tx Channel N Global Configuration Registers |

### 16.4.5.105 RXGCR25 Register (offset = B28h) [reset = 0h]

RXGCR25 is shown in Figure 16-256 and described in Table 16-268.

**Figure 16-256. RXGCR25 Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| RX_ENABLE | RX_TEARDOWN | RX_PAUSE | Reserved | | | | RX_ERROR_HANDLING |
| R/W-0h | R/W-0h | R/W-0h | | | | | W-0h |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| RX_SOP_OFFSET | | | | | | | |
| W-0h | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| RX_DEFAULT_DESC_TYPE | | RX_DEFAULT_RQ_QMGR | | RX_DEFAULT_RQ_QNUM | | | |
| W-0h | | W-0h | | W-0h | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| RX_DEFAULT_RQ_QNUM | | | | | | | |
| W-0h | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-268. RXGCR25 Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31 | RX_ENABLE | R/W | 0h | This field enables or disables the channel<br>0 = channel is disabled<br>1 = channel is enabled This field will be cleared after a channel teardown is complete. |
| 30 | RX_TEARDOWN | R/W | 0h | This field indicates whether or not an Rx teardown operation is complete.<br>This field should be cleared when a channel is initialized.<br>This field will be set after a channel teardown is complete. |
| 29 | RX_PAUSE | R/W | 0h | Setting this bit causes the CPPI DMA to be suspended for rx channels.<br>If a pause is being requested and the channel is not in a packet then drop the credit. |
| 24 | RX_ERROR_HANDLING | W | 0h | This bit controls the error handling mode for the channel and is only used when channel errors (i.e.<br>descriptor or buffer starvation occurs):<br>0 = Starvation errors result in dropping packet and reclaiming any used descriptor or buffer resources back to the original queues/pools they were allocated to<br>1 = Starvation errors result in subsequent re-try of the descriptor allocation operation.<br>In this mode, the DMA will return to the IDLE state without saving it's internal operational state back to the internal state RAM and without issuing an advance operation on the FIFO interface.<br>This results in the DMA re-initiating the FIFO block transfer at a later time with the intention that additional free buffers and/or descriptors will have been added.<br>Regardless of the value of this bit, the DMA will assert the cdma_rx_sof_overrun (for SOP) or cdma_rx_mof_overrun (for non-SOP) when |
| 23-16 | RX_SOP_OFFSET | W | 0h | This field specifies the number of bytes that are to be skipped in the SOP buffer before beginning to write the payload.<br>This value must be less than the minimum size of a buffer in the system.<br>Valid values are<br>0 - 255 bytes. |

**Table 16-268. RXGCR25 Register Field Descriptions (continued)**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 15-14 | RX_DEFAULT_DESC_TYPE | W | 0h | This field indicates the default descriptor type to use:<br>0 = Reserved<br>1 = Host<br>2 = Reserved<br>3 = Reserved The actual descriptor type that will be used for reception can be overridden by information provided in the CPPI FIFO data block. |
| 13-12 | RX_DEFAULT_RQ_QMGR | W | 0h | This field indicates the default receive queue manager that this channel should use.<br>The actual receive queue manager index can be overridden by information provided in the CPPI FIFO data block. |
| 11-0 | RX_DEFAULT_RQ_QNUM | W | 0h | This field indicates the default receive queue that this channel should use.<br>The actual receive queue that will be used for reception can be overridden by information provided in the CPPI FIFO data block. Table<br>99 -Rx Channel N Global Configuration Registers |

### 16.4.5.106 RXHPCRA25 Register (offset = B2Ch) [reset = 0h]

RXHPCRA25 is shown in Figure 16-257 and described in Table 16-269.

**Figure 16-257. RXHPCRA25 Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| Reserved | | RX_HOST_FDQ1_QMGR | | RX_HOST_FDQ1_QNUM | | | |
| | | W-0h | | W-0h | | | |
| **23** | **22** | **21** | **20** | **19** | **18** | **17** | **16** |
| RX_HOST_FDQ1_QNUM | | | | | | | |
| W-0h | | | | | | | |
| **15** | **14** | **13** | **12** | **11** | **10** | **9** | **8** |
| Reserved | | RX_HOST_FDQ0_QMGR | | RX_HOST_FDQ0_QNUM | | | |
| | | W-0h | | W-0h | | | |
| **7** | **6** | **5** | **4** | **3** | **2** | **1** | **0** |
| RX_HOST_FDQ0_QNUM | | | | | | | |
| W-0h | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-269. RXHPCRA25 Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 29-28 | RX_HOST_FDQ1_QMGR | W | 0h | This field specifies which Buffer Manager should be used for the second Rx buffer in a host type packet. |
| 27-16 | RX_HOST_FDQ1_QNUM | W | 0h | This field specifies which Free Descriptor / Buffer Pool should be used for the 2nd Rx buffer in a host type packet |
| 13-12 | RX_HOST_FDQ0_QMGR | W | 0h | This field specifies which Buffer Manager should be used for the second Rx buffer in a host type packet. |
| 11-0 | RX_HOST_FDQ0_QNUM | W | 0h | This field specifies which Free Descriptor / Buffer Pool should be used for the 1st Rx buffer in a host type packet Table 100 -Rx Channel N Host Packet Configuration Registers A |

### 16.4.5.107 RXHPCRB25 Register (offset = B30h) [reset = 0h]

RXHPCRB25 is shown in Figure 16-258 and described in Table 16-270.

**Figure 16-258. RXHPCRB25 Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| Reserved | | RX_HOST_FDQ3_QMGR | | RX_HOST_FDQ3_QNUM | | | |
| | | W-0h | | W-0h | | | |
| **23** | **22** | **21** | **20** | **19** | **18** | **17** | **16** |
| RX_HOST_FDQ3_QNUM | | | | | | | |
| W-0h | | | | | | | |
| **15** | **14** | **13** | **12** | **11** | **10** | **9** | **8** |
| Reserved | | RX_HOST_FDQ2_QMGR | | RX_HOST_FDQ2_QNUM | | | |
| | | W-0h | | W-0h | | | |
| **7** | **6** | **5** | **4** | **3** | **2** | **1** | **0** |
| RX_HOST_FDQ2_QNUM | | | | | | | |
| W-0h | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-270. RXHPCRB25 Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 29-28 | RX_HOST_FDQ3_QMGR | W | 0h | This field specifies which Manager should be used for the 4th or later Rx buffers in a host type packet |
| 27-16 | RX_HOST_FDQ3_QNUM | W | 0h | This field specifies which Free Descriptor Queue should be used for the 4th or later Rx buffers in a host type packet |
| 13-12 | RX_HOST_FDQ2_QMGR | W | 0h | This field specifies which Buffer Manager should be used for the 3rd Rx buffer in a host type packet |
| 11-0 | RX_HOST_FDQ2_QNUM | W | 0h | This field specifies which Free Descriptor / Buffer Pool should be used for the 3rd Rx buffer in a host type packet Table 101 -Rx Channel N Host Packet Configuration Registers B |

### 16.4.5.108 TXGCR26 Register (offset = B40h) [reset = 0h]

TXGCR26 is shown in Figure 16-259 and described in Table 16-271.

**Figure 16-259. TXGCR26 Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| TX_ENABLE | TX_TEARDOWN | Reserved | | | | | |
| R/W-0h | R/W-0h | | | | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| Reserved | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| Reserved | | TX_DEFAULT_QMGR | | TX_DEFAULT_QNUM | | | |
| | | W-0h | | W-0h | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| TX_DEFAULT_QNUM | | | | | | | |
| W-0h | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-271. TXGCR26 Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31 | TX_ENABLE | R/W | 0h | This field enables or disables the channel<br>0 = channel is disabled<br>1 = channel is enabled This field will be cleared after a channel teardown is complete. |
| 30 | TX_TEARDOWN | R/W | 0h | Setting this bit will request the channel to be torn down.<br>This field will remain set after a channel teardown is complete. |
| 13-12 | TX_DEFAULT_QMGR | W | 0h | This field controls the default queue manager number that will be used to queue teardown descriptors back to the host. |
| 11-0 | TX_DEFAULT_QNUM | W | 0h | This field controls the default queue number within the selected queue manager onto which teardown descriptors will be queued back to the host.<br>Table<br>98 -Tx Channel N Global Configuration Registers |

### 16.4.5.109 RXGCR26 Register (offset = B48h) [reset = 0h]

RXGCR26 is shown in Figure 16-260 and described in Table 16-272.

**Figure 16-260. RXGCR26 Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| RX_ENABLE | RX_TEARDOWN | RX_PAUSE | Reserved | | | | RX_ERROR_HANDLING |
| R/W-0h | R/W-0h | R/W-0h | | | | | W-0h |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| RX_SOP_OFFSET | | | | | | | |
| W-0h | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| RX_DEFAULT_DESC_TYPE | | RX_DEFAULT_RQ_QMGR | | RX_DEFAULT_RQ_QNUM | | | |
| W-0h | | W-0h | | W-0h | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| RX_DEFAULT_RQ_QNUM | | | | | | | |
| W-0h | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-272. RXGCR26 Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31 | RX_ENABLE | R/W | 0h | This field enables or disables the channel<br>0 = channel is disabled<br>1 = channel is enabled This field will be cleared after a channel teardown is complete. |
| 30 | RX_TEARDOWN | R/W | 0h | This field indicates whether or not an Rx teardown operation is complete.<br>This field should be cleared when a channel is initialized.<br>This field will be set after a channel teardown is complete. |
| 29 | RX_PAUSE | R/W | 0h | Setting this bit causes the CPPI DMA to be suspended for rx channels.<br>If a pause is being requested and the channel is not in a packet then drop the credit. |
| 24 | RX_ERROR_HANDLING | W | 0h | This bit controls the error handling mode for the channel and is only used when channel errors (i.e.<br>descriptor or buffer starvation occurs):<br>0 = Starvation errors result in dropping packet and reclaiming any used descriptor or buffer resources back to the original queues/pools they were allocated to<br>1 = Starvation errors result in subsequent re-try of the descriptor allocation operation.<br>In this mode, the DMA will return to the IDLE state without saving it's internal operational state back to the internal state RAM and without issuing an advance operation on the FIFO interface.<br>This results in the DMA re-initiating the FIFO block transfer at a later time with the intention that additional free buffers and/or descriptors will have been added.<br>Regardless of the value of this bit, the DMA will assert the cdma_rx_sof_overrun (for SOP) or cdma_rx_mof_overrun (for non-SOP) when |
| 23-16 | RX_SOP_OFFSET | W | 0h | This field specifies the number of bytes that are to be skipped in the SOP buffer before beginning to write the payload.<br>This value must be less than the minimum size of a buffer in the system.<br>Valid values are<br>0 - 255 bytes. |

### Table 16-272. RXGCR26 Register Field Descriptions (continued)

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 15-14 | RX_DEFAULT_DESC_TYPE | W | 0h | This field indicates the default descriptor type to use:<br>0 = Reserved<br>1 = Host<br>2 = Reserved<br>3 = Reserved The actual descriptor type that will be used for reception can be overridden by information provided in the CPPI FIFO data block. |
| 13-12 | RX_DEFAULT_RQ_QMGR | W | 0h | This field indicates the default receive queue manager that this channel should use.<br>The actual receive queue manager index can be overridden by information provided in the CPPI FIFO data block. |
| 11-0 | RX_DEFAULT_RQ_QNUM | W | 0h | This field indicates the default receive queue that this channel should use.<br>The actual receive queue that will be used for reception can be overridden by information provided in the CPPI FIFO data block. Table<br>99 -Rx Channel N Global Configuration Registers |

### 16.4.5.110 RXHPCRA26 Register (offset = B4Ch) [reset = 0h]

RXHPCRA26 is shown in Figure 16-261 and described in Table 16-273.

**Figure 16-261. RXHPCRA26 Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| Reserved | | RX_HOST_FDQ1_QMGR | | RX_HOST_FDQ1_QNUM | | | |
| | | W-0h | | W-0h | | | |
| **23** | **22** | **21** | **20** | **19** | **18** | **17** | **16** |
| RX_HOST_FDQ1_QNUM | | | | | | | |
| W-0h | | | | | | | |
| **15** | **14** | **13** | **12** | **11** | **10** | **9** | **8** |
| Reserved | | RX_HOST_FDQ0_QMGR | | RX_HOST_FDQ0_QNUM | | | |
| | | W-0h | | W-0h | | | |
| **7** | **6** | **5** | **4** | **3** | **2** | **1** | **0** |
| RX_HOST_FDQ0_QNUM | | | | | | | |
| W-0h | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-273. RXHPCRA26 Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 29-28 | RX_HOST_FDQ1_QMGR | W | 0h | This field specifies which Buffer Manager should be used for the second Rx buffer in a host type packet. |
| 27-16 | RX_HOST_FDQ1_QNUM | W | 0h | This field specifies which Free Descriptor / Buffer Pool should be used for the 2nd Rx buffer in a host type packet |
| 13-12 | RX_HOST_FDQ0_QMGR | W | 0h | This field specifies which Buffer Manager should be used for the second Rx buffer in a host type packet. |
| 11-0 | RX_HOST_FDQ0_QNUM | W | 0h | This field specifies which Free Descriptor / Buffer Pool should be used for the 1st Rx buffer in a host type packet Table 100 -Rx Channel N Host Packet Configuration Registers A |

### 16.4.5.111 RXHPCRB26 Register (offset = B50h) [reset = 0h]

RXHPCRB26 is shown in Figure 16-262 and described in Table 16-274.

**Figure 16-262. RXHPCRB26 Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| Reserved | | RX_HOST_FDQ3_QMGR | | RX_HOST_FDQ3_QNUM | | | |
| | | W-0h | | W-0h | | | |
| **23** | **22** | **21** | **20** | **19** | **18** | **17** | **16** |
| RX_HOST_FDQ3_QNUM | | | | | | | |
| W-0h | | | | | | | |
| **15** | **14** | **13** | **12** | **11** | **10** | **9** | **8** |
| Reserved | | RX_HOST_FDQ2_QMGR | | RX_HOST_FDQ2_QNUM | | | |
| | | W-0h | | W-0h | | | |
| **7** | **6** | **5** | **4** | **3** | **2** | **1** | **0** |
| RX_HOST_FDQ2_QNUM | | | | | | | |
| W-0h | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-274. RXHPCRB26 Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 29-28 | RX_HOST_FDQ3_QMGR | W | 0h | This field specifies which Manager should be used for the 4th or later Rx buffers in a host type packet |
| 27-16 | RX_HOST_FDQ3_QNUM | W | 0h | This field specifies which Free Descriptor Queue should be used for the 4th or later Rx buffers in a host type packet |
| 13-12 | RX_HOST_FDQ2_QMGR | W | 0h | This field specifies which Buffer Manager should be used for the 3rd Rx buffer in a host type packet |
| 11-0 | RX_HOST_FDQ2_QNUM | W | 0h | This field specifies which Free Descriptor / Buffer Pool should be used for the 3rd Rx buffer in a host type packet Table 101 -Rx Channel N Host Packet Configuration Registers B |

### 16.4.5.112 TXGCR27 Register (offset = B60h) [reset = 0h]

TXGCR27 is shown in Figure 16-263 and described in Table 16-275.

**Figure 16-263. TXGCR27 Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| TX_ENABLE | TX_TEARDOWN | Reserved | | | | | |
| R/W-0h | R/W-0h | | | | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| Reserved | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| Reserved | | TX_DEFAULT_QMGR | | TX_DEFAULT_QNUM | | | |
| | | W-0h | | W-0h | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| TX_DEFAULT_QNUM | | | | | | | |
| W-0h | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-275. TXGCR27 Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31 | TX_ENABLE | R/W | 0h | This field enables or disables the channel<br>0 = channel is disabled<br>1 = channel is enabled This field will be cleared after a channel teardown is complete. |
| 30 | TX_TEARDOWN | R/W | 0h | Setting this bit will request the channel to be torn down.<br>This field will remain set after a channel teardown is complete. |
| 13-12 | TX_DEFAULT_QMGR | W | 0h | This field controls the default queue manager number that will be used to queue teardown descriptors back to the host. |
| 11-0 | TX_DEFAULT_QNUM | W | 0h | This field controls the default queue number within the selected queue manager onto which teardown descriptors will be queued back to the host.<br>Table<br>98 -Tx Channel N Global Configuration Registers |

### 16.4.5.113 RXGCR27 Register (offset = B68h) [reset = 0h]

RXGCR27 is shown in Figure 16-264 and described in Table 16-276.

**Figure 16-264. RXGCR27 Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| RX_ENABLE | RX_TEARDOWN | RX_PAUSE | Reserved | | | | RX_ERROR_HANDLING |
| R/W-0h | R/W-0h | R/W-0h | | | | | W-0h |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| RX_SOP_OFFSET | | | | | | | |
| W-0h | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| RX_DEFAULT_DESC_TYPE | | RX_DEFAULT_RQ_QMGR | | RX_DEFAULT_RQ_QNUM | | | |
| W-0h | | W-0h | | W-0h | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| RX_DEFAULT_RQ_QNUM | | | | | | | |
| W-0h | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-276. RXGCR27 Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31 | RX_ENABLE | R/W | 0h | This field enables or disables the channel<br>0 = channel is disabled<br>1 = channel is enabled This field will be cleared after a channel teardown is complete. |
| 30 | RX_TEARDOWN | R/W | 0h | This field indicates whether or not an Rx teardown operation is complete.<br>This field should be cleared when a channel is initialized.<br>This field will be set after a channel teardown is complete. |
| 29 | RX_PAUSE | R/W | 0h | Setting this bit causes the CPPI DMA to be suspended for rx channels.<br>If a pause is being requested and the channel is not in a packet then drop the credit. |
| 24 | RX_ERROR_HANDLING | W | 0h | This bit controls the error handling mode for the channel and is only used when channel errors (i.e.<br>descriptor or buffer starvation occurs):<br>0 = Starvation errors result in dropping packet and reclaiming any used descriptor or buffer resources back to the original queues/pools they were allocated to<br>1 = Starvation errors result in subsequent re-try of the descriptor allocation operation.<br>In this mode, the DMA will return to the IDLE state without saving it's internal operational state back to the internal state RAM and without issuing an advance operation on the FIFO interface.<br>This results in the DMA re-initiating the FIFO block transfer at a later time with the intention that additional free buffers and/or descriptors will have been added.<br>Regardless of the value of this bit, the DMA will assert the cdma_rx_sof_overrun (for SOP) or cdma_rx_mof_overrun (for non-SOP) when |
| 23-16 | RX_SOP_OFFSET | W | 0h | This field specifies the number of bytes that are to be skipped in the SOP buffer before beginning to write the payload.<br>This value must be less than the minimum size of a buffer in the system.<br>Valid values are<br>0 - 255 bytes. |

**Table 16-276. RXGCR27 Register Field Descriptions (continued)**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 15-14 | RX_DEFAULT_DESC_TYPE | W | 0h | This field indicates the default descriptor type to use:<br>0 = Reserved<br>1 = Host<br>2 = Reserved<br>3 = Reserved The actual descriptor type that will be used for reception can be overridden by information provided in the CPPI FIFO data block. |
| 13-12 | RX_DEFAULT_RQ_QMGR | W | 0h | This field indicates the default receive queue manager that this channel should use.<br>The actual receive queue manager index can be overridden by information provided in the CPPI FIFO data block. |
| 11-0 | RX_DEFAULT_RQ_QNUM | W | 0h | This field indicates the default receive queue that this channel should use.<br>The actual receive queue that will be used for reception can be overridden by information provided in the CPPI FIFO data block.<br>Table<br>99 -Rx Channel N Global Configuration Registers |

### 16.4.5.114 RXHPCRA27 Register (offset = B6Ch) [reset = 0h]

RXHPCRA27 is shown in Figure 16-265 and described in Table 16-277.

**Figure 16-265. RXHPCRA27 Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| Reserved | | RX_HOST_FDQ1_QMGR | | RX_HOST_FDQ1_QNUM | | | |
| | | W-0h | | W-0h | | | |
| **23** | **22** | **21** | **20** | **19** | **18** | **17** | **16** |
| RX_HOST_FDQ1_QNUM | | | | | | | |
| W-0h | | | | | | | |
| **15** | **14** | **13** | **12** | **11** | **10** | **9** | **8** |
| Reserved | | RX_HOST_FDQ0_QMGR | | RX_HOST_FDQ0_QNUM | | | |
| | | W-0h | | W-0h | | | |
| **7** | **6** | **5** | **4** | **3** | **2** | **1** | **0** |
| RX_HOST_FDQ0_QNUM | | | | | | | |
| W-0h | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-277. RXHPCRA27 Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 29-28 | RX_HOST_FDQ1_QMGR | W | 0h | This field specifies which Buffer Manager should be used for the second Rx buffer in a host type packet. |
| 27-16 | RX_HOST_FDQ1_QNUM | W | 0h | This field specifies which Free Descriptor / Buffer Pool should be used for the 2nd Rx buffer in a host type packet |
| 13-12 | RX_HOST_FDQ0_QMGR | W | 0h | This field specifies which Buffer Manager should be used for the second Rx buffer in a host type packet. |
| 11-0 | RX_HOST_FDQ0_QNUM | W | 0h | This field specifies which Free Descriptor / Buffer Pool should be used for the 1st Rx buffer in a host type packet Table 100 -Rx Channel N Host Packet Configuration Registers A |

### 16.4.5.115 RXHPCRB27 Register (offset = B70h) [reset = 0h]

RXHPCRB27 is shown in Figure 16-266 and described in Table 16-278.

**Figure 16-266. RXHPCRB27 Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| Reserved | | RX_HOST_FDQ3_QMGR | | RX_HOST_FDQ3_QNUM | | | |
| | | W-0h | | W-0h | | | |
| **23** | **22** | **21** | **20** | **19** | **18** | **17** | **16** |
| RX_HOST_FDQ3_QNUM | | | | | | | |
| W-0h | | | | | | | |
| **15** | **14** | **13** | **12** | **11** | **10** | **9** | **8** |
| Reserved | | RX_HOST_FDQ2_QMGR | | RX_HOST_FDQ2_QNUM | | | |
| | | W-0h | | W-0h | | | |
| **7** | **6** | **5** | **4** | **3** | **2** | **1** | **0** |
| RX_HOST_FDQ2_QNUM | | | | | | | |
| W-0h | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-278. RXHPCRB27 Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 29-28 | RX_HOST_FDQ3_QMGR | W | 0h | This field specifies which Manager should be used for the 4th or later Rx buffers in a host type packet |
| 27-16 | RX_HOST_FDQ3_QNUM | W | 0h | This field specifies which Free Descriptor Queue should be used for the 4th or later Rx buffers in a host type packet |
| 13-12 | RX_HOST_FDQ2_QMGR | W | 0h | This field specifies which Buffer Manager should be used for the 3rd Rx buffer in a host type packet |
| 11-0 | RX_HOST_FDQ2_QNUM | W | 0h | This field specifies which Free Descriptor / Buffer Pool should be used for the 3rd Rx buffer in a host type packet Table 101 -Rx Channel N Host Packet Configuration Registers B |

### 16.4.5.116 TXGCR28 Register (offset = B80h) [reset = 0h]

TXGCR28 is shown in Figure 16-267 and described in Table 16-279.

**Figure 16-267. TXGCR28 Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| TX_ENABLE | TX_TEARDOWN | Reserved | | | | | |
| R/W-0h | R/W-0h | | | | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| Reserved | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| Reserved | | TX_DEFAULT_QMGR | | TX_DEFAULT_QNUM | | | |
| | | W-0h | | W-0h | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| TX_DEFAULT_QNUM | | | | | | | |
| W-0h | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-279. TXGCR28 Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31 | TX_ENABLE | R/W | 0h | This field enables or disables the channel<br>0 = channel is disabled<br>1 = channel is enabled This field will be cleared after a channel teardown is complete. |
| 30 | TX_TEARDOWN | R/W | 0h | Setting this bit will request the channel to be torn down.<br>This field will remain set after a channel teardown is complete. |
| 13-12 | TX_DEFAULT_QMGR | W | 0h | This field controls the default queue manager number that will be used to queue teardown descriptors back to the host. |
| 11-0 | TX_DEFAULT_QNUM | W | 0h | This field controls the default queue number within the selected queue manager onto which teardown descriptors will be queued back to the host.<br>Table<br>98 -Tx Channel N Global Configuration Registers |

### 16.4.5.117 RXGCR28 Register (offset = B88h) [reset = 0h]

RXGCR28 is shown in Figure 16-268 and described in Table 16-280.

**Figure 16-268. RXGCR28 Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| RX_ENABLE | RX_TEARDOWN | RX_PAUSE | Reserved | | | | RX_ERROR_HANDLING |
| R/W-0h | R/W-0h | R/W-0h | | | | | W-0h |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| RX_SOP_OFFSET | | | | | | | |
| W-0h | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| RX_DEFAULT_DESC_TYPE | | RX_DEFAULT_RQ_QMGR | | RX_DEFAULT_RQ_QNUM | | | |
| W-0h | | W-0h | | W-0h | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| RX_DEFAULT_RQ_QNUM | | | | | | | |
| W-0h | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-280. RXGCR28 Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31 | RX_ENABLE | R/W | 0h | This field enables or disables the channel<br>0 = channel is disabled<br>1 = channel is enabled This field will be cleared after a channel teardown is complete. |
| 30 | RX_TEARDOWN | R/W | 0h | This field indicates whether or not an Rx teardown operation is complete.<br>This field should be cleared when a channel is initialized.<br>This field will be set after a channel teardown is complete. |
| 29 | RX_PAUSE | R/W | 0h | Setting this bit causes the CPPI DMA to be suspended for rx channels.<br>If a pause is being requested and the channel is not in a packet then drop the credit. |
| 24 | RX_ERROR_HANDLING | W | 0h | This bit controls the error handling mode for the channel and is only used when channel errors (i.e.<br>descriptor or buffer starvation occurs):<br>0 = Starvation errors result in dropping packet and reclaiming any used descriptor or buffer resources back to the original queues/pools they were allocated to<br>1 = Starvation errors result in subsequent re-try of the descriptor allocation operation.<br>In this mode, the DMA will return to the IDLE state without saving it's internal operational state back to the internal state RAM and without issuing an advance operation on the FIFO interface.<br>This results in the DMA re-initiating the FIFO block transfer at a later time with the intention that additional free buffers and/or descriptors will have been added.<br>Regardless of the value of this bit, the DMA will assert the cdma_rx_sof_overrun (for SOP) or cdma_rx_mof_overrun (for non-SOP) when |
| 23-16 | RX_SOP_OFFSET | W | 0h | This field specifies the number of bytes that are to be skipped in the SOP buffer before beginning to write the payload.<br>This value must be less than the minimum size of a buffer in the system.<br>Valid values are<br>0 - 255 bytes. |

**Table 16-280. RXGCR28 Register Field Descriptions (continued)**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 15-14 | RX_DEFAULT_DESC_TYPE | W | 0h | This field indicates the default descriptor type to use:<br>0 = Reserved<br>1 = Host<br>2 = Reserved<br>3 = Reserved The actual descriptor type that will be used for reception can be overridden by information provided in the CPPI FIFO data block. |
| 13-12 | RX_DEFAULT_RQ_QMGR | W | 0h | This field indicates the default receive queue manager that this channel should use.<br>The actual receive queue manager index can be overridden by information provided in the CPPI FIFO data block. |
| 11-0 | RX_DEFAULT_RQ_QNUM | W | 0h | This field indicates the default receive queue that this channel should use.<br>The actual receive queue that will be used for reception can be overridden by information provided in the CPPI FIFO data block. Table<br>99 -Rx Channel N Global Configuration Registers |

### 16.4.5.118 RXHPCRA28 Register (offset = B8Ch) [reset = 0h]

RXHPCRA28 is shown in Figure 16-269 and described in Table 16-281.

**Figure 16-269. RXHPCRA28 Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| Reserved | | RX_HOST_FDQ1_QMGR | | RX_HOST_FDQ1_QNUM | | | |
| | | W-0h | | W-0h | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| RX_HOST_FDQ1_QNUM | | | | | | | |
| W-0h | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| Reserved | | RX_HOST_FDQ0_QMGR | | RX_HOST_FDQ0_QNUM | | | |
| | | W-0h | | W-0h | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| RX_HOST_FDQ0_QNUM | | | | | | | |
| W-0h | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-281. RXHPCRA28 Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 29-28 | RX_HOST_FDQ1_QMGR | W | 0h | This field specifies which Buffer Manager should be used for the second Rx buffer in a host type packet. |
| 27-16 | RX_HOST_FDQ1_QNUM | W | 0h | This field specifies which Free Descriptor / Buffer Pool should be used for the 2nd Rx buffer in a host type packet |
| 13-12 | RX_HOST_FDQ0_QMGR | W | 0h | This field specifies which Buffer Manager should be used for the second Rx buffer in a host type packet. |
| 11-0 | RX_HOST_FDQ0_QNUM | W | 0h | This field specifies which Free Descriptor / Buffer Pool should be used for the 1st Rx buffer in a host type packet Table 100 -Rx Channel N Host Packet Configuration Registers A |

### 16.4.5.119 RXHPCRB28 Register (offset = B90h) [reset = 0h]

RXHPCRB28 is shown in Figure 16-270 and described in Table 16-282.

**Figure 16-270. RXHPCRB28 Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| Reserved | | RX_HOST_FDQ3_QMGR | | RX_HOST_FDQ3_QNUM | | | |
| | | W-0h | | W-0h | | | |
| **23** | **22** | **21** | **20** | **19** | **18** | **17** | **16** |
| RX_HOST_FDQ3_QNUM | | | | | | | |
| W-0h | | | | | | | |
| **15** | **14** | **13** | **12** | **11** | **10** | **9** | **8** |
| Reserved | | RX_HOST_FDQ2_QMGR | | RX_HOST_FDQ2_QNUM | | | |
| | | W-0h | | W-0h | | | |
| **7** | **6** | **5** | **4** | **3** | **2** | **1** | **0** |
| RX_HOST_FDQ2_QNUM | | | | | | | |
| W-0h | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-282. RXHPCRB28 Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 29-28 | RX_HOST_FDQ3_QMGR | W | 0h | This field specifies which Manager should be used for the 4th or later Rx buffers in a host type packet |
| 27-16 | RX_HOST_FDQ3_QNUM | W | 0h | This field specifies which Free Descriptor Queue should be used for the 4th or later Rx buffers in a host type packet |
| 13-12 | RX_HOST_FDQ2_QMGR | W | 0h | This field specifies which Buffer Manager should be used for the 3rd Rx buffer in a host type packet |
| 11-0 | RX_HOST_FDQ2_QNUM | W | 0h | This field specifies which Free Descriptor / Buffer Pool should be used for the 3rd Rx buffer in a host type packet Table 101 -Rx Channel N Host Packet Configuration Registers B |

### 16.4.5.120 TXGCR29 Register (offset = BA0h) [reset = 0h]

TXGCR29 is shown in Figure 16-271 and described in Table 16-283.

**Figure 16-271. TXGCR29 Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| TX_ENABLE | TX_TEARDOWN | Reserved | | | | | |
| R/W-0h | R/W-0h | | | | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| Reserved | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| Reserved | | TX_DEFAULT_QMGR | | TX_DEFAULT_QNUM | | | |
| | | W-0h | | W-0h | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| TX_DEFAULT_QNUM | | | | | | | |
| W-0h | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-283. TXGCR29 Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31 | TX_ENABLE | R/W | 0h | This field enables or disables the channel<br>0 = channel is disabled<br>1 = channel is enabled This field will be cleared after a channel teardown is complete. |
| 30 | TX_TEARDOWN | R/W | 0h | Setting this bit will request the channel to be torn down.<br>This field will remain set after a channel teardown is complete. |
| 13-12 | TX_DEFAULT_QMGR | W | 0h | This field controls the default queue manager number that will be used to queue teardown descriptors back to the host. |
| 11-0 | TX_DEFAULT_QNUM | W | 0h | This field controls the default queue number within the selected queue manager onto which teardown descriptors will be queued back to the host.<br>Table<br>98 -Tx Channel N Global Configuration Registers |

### 16.4.5.121 RXGCR29 Register (offset = BA8h) [reset = 0h]

RXGCR29 is shown in Figure 16-272 and described in Table 16-284.

**Figure 16-272. RXGCR29 Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| RX_ENABLE | RX_TEARDOWN | RX_PAUSE | Reserved | | | | RX_ERROR_HANDLING |
| R/W-0h | R/W-0h | R/W-0h | | | | | W-0h |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| RX_SOP_OFFSET | | | | | | | |
| W-0h | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| RX_DEFAULT_DESC_TYPE | | RX_DEFAULT_RQ_QMGR | | RX_DEFAULT_RQ_QNUM | | | |
| W-0h | | W-0h | | W-0h | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| RX_DEFAULT_RQ_QNUM | | | | | | | |
| W-0h | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-284. RXGCR29 Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31 | RX_ENABLE | R/W | 0h | This field enables or disables the channel<br>0 = channel is disabled<br>1 = channel is enabled This field will be cleared after a channel teardown is complete. |
| 30 | RX_TEARDOWN | R/W | 0h | This field indicates whether or not an Rx teardown operation is complete.<br>This field should be cleared when a channel is initialized.<br>This field will be set after a channel teardown is complete. |
| 29 | RX_PAUSE | R/W | 0h | Setting this bit causes the CPPI DMA to be suspended for rx channels.<br>If a pause is being requested and the channel is not in a packet then drop the credit. |
| 24 | RX_ERROR_HANDLING | W | 0h | This bit controls the error handling mode for the channel and is only used when channel errors (i.e.<br>descriptor or buffer starvation occurs):<br>0 = Starvation errors result in dropping packet and reclaiming any used descriptor or buffer resources back to the original queues/pools they were allocated to<br>1 = Starvation errors result in subsequent re-try of the descriptor allocation operation.<br>In this mode, the DMA will return to the IDLE state without saving it's internal operational state back to the internal state RAM and without issuing an advance operation on the FIFO interface.<br>This results in the DMA re-initiating the FIFO block transfer at a later time with the intention that additional free buffers and/or descriptors will have been added.<br>Regardless of the value of this bit, the DMA will assert the cdma_rx_sof_overrun (for SOP) or cdma_rx_mof_overrun (for non-SOP) when |
| 23-16 | RX_SOP_OFFSET | W | 0h | This field specifies the number of bytes that are to be skipped in the SOP buffer before beginning to write the payload.<br>This value must be less than the minimum size of a buffer in the system.<br>Valid values are<br>0 - 255 bytes. |

### Table 16-284. RXGCR29 Register Field Descriptions (continued)

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 15-14 | RX_DEFAULT_DESC_TYPE | W | 0h | This field indicates the default descriptor type to use:<br>0 = Reserved<br>1 = Host<br>2 = Reserved<br>3 = Reserved The actual descriptor type that will be used for reception can be overridden by information provided in the CPPI FIFO data block. |
| 13-12 | RX_DEFAULT_RQ_QMGR | W | 0h | This field indicates the default receive queue manager that this channel should use.<br>The actual receive queue manager index can be overridden by information provided in the CPPI FIFO data block. |
| 11-0 | RX_DEFAULT_RQ_QNUM | W | 0h | This field indicates the default receive queue that this channel should use.<br>The actual receive queue that will be used for reception can be overridden by information provided in the CPPI FIFO data block. Table<br>99 -Rx Channel N Global Configuration Registers |

### 16.4.5.122 RXHPCRA29 Register (offset = BACh) [reset = 0h]

RXHPCRA29 is shown in Figure 16-273 and described in Table 16-285.

**Figure 16-273. RXHPCRA29 Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| Reserved | | RX_HOST_FDQ1_QMGR | | RX_HOST_FDQ1_QNUM | | | |
| | | W-0h | | W-0h | | | |
| **23** | **22** | **21** | **20** | **19** | **18** | **17** | **16** |
| RX_HOST_FDQ1_QNUM | | | | | | | |
| W-0h | | | | | | | |
| **15** | **14** | **13** | **12** | **11** | **10** | **9** | **8** |
| Reserved | | RX_HOST_FDQ0_QMGR | | RX_HOST_FDQ0_QNUM | | | |
| | | W-0h | | W-0h | | | |
| **7** | **6** | **5** | **4** | **3** | **2** | **1** | **0** |
| RX_HOST_FDQ0_QNUM | | | | | | | |
| W-0h | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-285. RXHPCRA29 Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 29-28 | RX_HOST_FDQ1_QMGR | W | 0h | This field specifies which Buffer Manager should be used for the second Rx buffer in a host type packet. |
| 27-16 | RX_HOST_FDQ1_QNUM | W | 0h | This field specifies which Free Descriptor / Buffer Pool should be used for the 2nd Rx buffer in a host type packet |
| 13-12 | RX_HOST_FDQ0_QMGR | W | 0h | This field specifies which Buffer Manager should be used for the second Rx buffer in a host type packet. |
| 11-0 | RX_HOST_FDQ0_QNUM | W | 0h | This field specifies which Free Descriptor / Buffer Pool should be used for the 1st Rx buffer in a host type packet Table 100 -Rx Channel N Host Packet Configuration Registers A |

### 16.4.5.123 RXHPCRB29 Register (offset = BB0h) [reset = 0h]

RXHPCRB29 is shown in Figure 16-274 and described in Table 16-286.

**Figure 16-274. RXHPCRB29 Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| Reserved | | RX_HOST_FDQ3_QMGR | | RX_HOST_FDQ3_QNUM | | | |
| | | W-0h | | W-0h | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| RX_HOST_FDQ3_QNUM | | | | | | | |
| W-0h | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| Reserved | | RX_HOST_FDQ2_QMGR | | RX_HOST_FDQ2_QNUM | | | |
| | | W-0h | | W-0h | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| RX_HOST_FDQ2_QNUM | | | | | | | |
| W-0h | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-286. RXHPCRB29 Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 29-28 | RX_HOST_FDQ3_QMGR | W | 0h | This field specifies which Manager should be used for the 4th or later Rx buffers in a host type packet |
| 27-16 | RX_HOST_FDQ3_QNUM | W | 0h | This field specifies which Free Descriptor Queue should be used for the 4th or later Rx buffers in a host type packet |
| 13-12 | RX_HOST_FDQ2_QMGR | W | 0h | This field specifies which Buffer Manager should be used for the 3rd Rx buffer in a host type packet |
| 11-0 | RX_HOST_FDQ2_QNUM | W | 0h | This field specifies which Free Descriptor / Buffer Pool should be used for the 3rd Rx buffer in a host type packet Table 101 -Rx Channel N Host Packet Configuration Registers B |

## *16.4.6 CPPI_DMA_SCHEDULER Registers*

Table 16-287 lists the memory-mapped registers for the CPPI_DMA_SCHEDULER. All register offset addresses not listed in Table 16-287 should be considered as reserved locations and the register contents should not be modified.

**Table 16-287. CPPI_DMA_SCHEDULER Registers**

| Offset | Acronym | Register Name | Section |
|---|---|---|---|
| 3000h | DMA_SCHED_CTRL | | Section 16.4.6.1 |
| 3800h to 38FCh | WORD_0 to WORD_63 | | Section 16.4.6.2 |

### 16.4.6.1 DMA_SCHED_CTRL Register (offset = 3000h) [reset = 0h]

DMA_SCHED_CTRL is shown in Figure 16-275 and described in Table 16-288.

**Figure 16-275. DMA_SCHED_CTRL Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| ENABLE | RESERVED | | | | | | |
| R/W-0h | R/W-0h | | | | | | |
| **23** | **22** | **21** | **20** | **19** | **18** | **17** | **16** |
| RESERVED | | | | | | | |
| R/W-0h | | | | | | | |
| **15** | **14** | **13** | **12** | **11** | **10** | **9** | **8** |
| RESERVED | | | | | | | |
| R/W-0h | | | | | | | |
| **7** | **6** | **5** | **4** | **3** | **2** | **1** | **0** |
| LAST_ENTRY | | | | | | | |
| R/W-0h | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-288. DMA_SCHED_CTRL Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31 | ENABLE | R/W | 0h | This is the enable bit for the scheduler and is encoded as follows:<br>0 = scheduler is disabled and will no longer fetch entries from the scheduler table or pass credits to the DMA controller<br>1 = scheduler is enabled This bit should only be set after the table has been initialized. |
| 30-8 | RESERVED | R/W | 0h | |
| 7-0 | LAST_ENTRY | R/W | 0h | This field indicates the last valid entry in the scheduler table.<br>There are 64 words in the table and there are 4 entries in each word.<br>The table can be programmed with any integer number of entries from 1 to 256.<br>The corresponding encoding for this field is as follows:<br>0 = 1 entry<br>1 = 2 entries ...<br>254 = 255 entries<br>255 = 256 entries CPPI DMA Scheduler Control Registers |

### 16.4.6.2 WORD_0 to WORD_63 Register (offset = 3800h to 38FCh) [reset = 0h]

WORD_0 to WORD_63 is shown in Figure 16-276 and described in Table 16-289.

**Figure 16-276. WORD_0 to WORD_63 Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| ENTRY3_RXTX | RESERVED | | ENTRY3_CHANNEL | | | | |
| W-0h | W- | | R/W- | | | | |
| **23** | **22** | **21** | **20** | **19** | **18** | **17** | **16** |
| ENTRY2_RXTX | RESERVED | | ENTRY2_CHANNEL | | | | |
| W-0h | W- | | R/W- | | | | |
| **15** | **14** | **13** | **12** | **11** | **10** | **9** | **8** |
| ENTRY1_RXTX | RESERVED | | ENTRY1_CHANNEL | | | | |
| W-0h | W- | | R/W- | | | | |
| **7** | **6** | **5** | **4** | **3** | **2** | **1** | **0** |
| ENTRY0_RXTX | RESERVED | | ENTRY0_CHANNEL | | | | |
| W-0h | W- | | R/W- | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-289. WORD_0 to WORD_63 Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31 | ENTRY3_RXTX | W | 0h | This bit indicates if this entry is for a Tx or an Rx channel and is encoded as follows:<br>0 = Tx Channel<br>1 = Rx Channel |
| 30-29 | RESERVED | W | | |
| 28-24 | ENTRY3_CHANNEL | R/W | | This field indicates the channel number that is to be given an opportunity to transfer data.<br>If this is a Tx entry, the DMA will be presented with a scheduling credit for that exact Tx channel.<br>If this is an Rx entry, the DMA will be presented with a scheduling credit for the Rx FIFO that is associated with this channel.<br>For Rx FIFOs which carry traffic for more than 1 Rx DMA channel, the exact channel number that is given in the Rx credit will actually be the channel number which is currently on the head element of that Rx FIFO, which is not necessarily the channel number given in the scheduler table entry. |
| 23 | ENTRY2_RXTX | W | 0h | This bit indicates if this entry is for a Tx or an Rx channel and is encoded as follows:<br>0 = Tx Channel<br>1 = Rx Channel |
| 22-21 | RESERVED | W | | |
| 20-16 | ENTRY2_CHANNEL | R/W | | This field indicates the channel number that is to be given an opportunity to transfer data.<br>If this is a Tx entry, the DMA will be presented with a scheduling credit for that exact Tx channel.<br>If this is an Rx entry, the DMA will be presented with a scheduling credit for the Rx FIFO that is associated with this channel.<br>For Rx FIFOs which carry traffic for more than 1 Rx DMA channel, the exact channel number that is given in the Rx credit will actually be the channel number which is currently on the head element of that Rx FIFO, which is not necessarily the channel number given in the scheduler table entry. |
| 15 | ENTRY1_RXTX | W | 0h | This bit indicates if this entry is for a Tx or an Rx channel and is encoded as follows:<br>0 = Tx Channel<br>1 = Rx Channel |

**Table 16-289. WORD_0 to WORD_63 Register Field Descriptions (continued)**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 14-13 | RESERVED | W | | |
| 12-8 | ENTRY1_CHANNEL | R/W | | This field indicates the channel number that is to be given an opportunity to transfer data.<br>If this is a Tx entry, the DMA will be presented with a scheduling credit for that exact Tx channel.<br>If this is an Rx entry, the DMA will be presented with a scheduling credit for the Rx FIFO that is associated with this channel.<br>For Rx FIFOs which carry traffic for more than 1 Rx DMA channel, the exact channel number that is given in the Rx credit will actually be the channel number which is currently on the head element of that Rx FIFO, which is not necessarily the channel number given in the scheduler table entry. |
| 7 | ENTRY0_RXTX | W | 0h | This bit indicates if this entry is for a Tx or an Rx channel and is encoded as follows:<br>0 = Tx Channel<br>1 = Rx Channel |
| 6-5 | RESERVED | W | | |
| 4-0 | ENTRY0_CHANNEL | R/W | | This field indicates the channel number that is to be given an opportunity to transfer data.<br>If this is a Tx entry, the DMA will be presented with a scheduling credit for that exact Tx channel.<br>If this is an Rx entry, the DMA will be presented with a scheduling credit for the Rx FIFO that is associated with this channel.<br>For Rx FIFOs which carry traffic for more than 1 Rx DMA channel, the exact channel number that is given in the Rx credit will actually be the channel number which is currently on the head element of that Rx FIFO, which is not necessarily the channel number given in the scheduler table entry. |

### *16.4.7 QUEUE_MGR Registers*

Table 16-290 lists the memory-mapped registers for the QUEUE_MGR. All register offset addresses not listed in Table 16-290 should be considered as reserved locations and the register contents should not be modified.

**Table 16-290. QUEUE_MGR REGISTERS**

| Offset | Acronym | Register Name | Section |
|---|---|---|---|
| 0h | QMGRREVID | | Section 16.4.7.1 |
| 8h | QMGRRST | | Section 16.4.7.2 |
| 20h | FDBSC0 | | Section 16.4.7.3 |
| 24h | FDBSC1 | | Section 16.4.7.4 |
| 28h | FDBSC2 | | Section 16.4.7.5 |
| 2Ch | FDBSC3 | | Section 16.4.7.6 |
| 30h | FDBSC4 | | Section 16.4.7.7 |
| 34h | FDBSC5 | | Section 16.4.7.8 |
| 38h | FDBSC6 | | Section 16.4.7.9 |
| 3Ch | FDBSC7 | | Section 16.4.7.10 |
| 80h | LRAM0BASE | | Section 16.4.7.11 |
| 84h | LRAM0SIZE | | Section 16.4.7.12 |
| 88h | LRAM1BASE | | Section 16.4.7.13 |
| 90h | PEND0 | | Section 16.4.7.14 |
| 94h | PEND1 | | Section 16.4.7.15 |
| 98h | PEND2 | | Section 16.4.7.16 |
| 9Ch | PEND3 | | Section 16.4.7.17 |
| A0h | PEND4 | | Section 16.4.7.18 |

**Table 16-290. QUEUE_MGR REGISTERS (continued)**

| Offset | Acronym | Register Name | Section |
|---|---|---|---|
| 1000h | QMEMRBASE0 | | Section 16.4.7.19 |
| 1004h | QMEMCTRL0 | | Section 16.4.7.20 |
| 1010h | QMEMRBASE1 | | Section 16.4.7.21 |
| 1014h | QMEMCTRL1 | | Section 16.4.7.22 |
| 1020h | QMEMRBASE2 | | Section 16.4.7.23 |
| 1024h | QMEMCTRL2 | | Section 16.4.7.24 |
| 1030h | QMEMRBASE3 | | Section 16.4.7.25 |
| 1034h | QMEMCTRL3 | | Section 16.4.7.26 |
| 1040h | QMEMRBASE4 | | Section 16.4.7.27 |
| 1044h | QMEMCTRL4 | | Section 16.4.7.28 |
| 1050h | QMEMRBASE5 | | Section 16.4.7.29 |
| 1054h | QMEMCTRL5 | | Section 16.4.7.30 |
| 1060h | QMEMRBASE6 | | Section 16.4.7.31 |
| 1064h | QMEMCTRL6 | | Section 16.4.7.32 |
| 1070h | QMEMRBASE7 | | Section 16.4.7.33 |
| 1074h | QMEMCTRL7 | | Section 16.4.7.34 |
| 2000h | QUEUE_0_A | | Section 16.4.7.35 |
| 2004h | QUEUE_0_B | | Section 16.4.7.36 |
| 2008h | QUEUE_0_C | | Section 16.4.7.37 |
| 200Ch | QUEUE_0_D | | Section 16.4.7.38 |
| 2010h | QUEUE_1_A | | Section 16.4.7.39 |
| 2014h | QUEUE_1_B | | Section 16.4.7.40 |
| 2018h | QUEUE_1_C | | Section 16.4.7.41 |
| 201Ch | QUEUE_1_D | | Section 16.4.7.42 |
| 2020h | QUEUE_2_A | | Section 16.4.7.43 |
| 2024h | QUEUE_2_B | | Section 16.4.7.44 |
| 2028h | QUEUE_2_C | | Section 16.4.7.45 |
| 202Ch | QUEUE_2_D | | Section 16.4.7.46 |
| 2030h | QUEUE_3_A | | Section 16.4.7.47 |
| 2034h | QUEUE_3_B | | Section 16.4.7.48 |
| 2038h | QUEUE_3_C | | Section 16.4.7.49 |
| 203Ch | QUEUE_3_D | | Section 16.4.7.50 |
| 2040h | QUEUE_4_A | | Section 16.4.7.51 |
| 2044h | QUEUE_4_B | | Section 16.4.7.52 |
| 2048h | QUEUE_4_C | | Section 16.4.7.53 |
| 204Ch | QUEUE_4_D | | Section 16.4.7.54 |
| 2050h | QUEUE_5_A | | Section 16.4.7.55 |
| 2054h | QUEUE_5_B | | Section 16.4.7.56 |
| 2058h | QUEUE_5_C | | Section 16.4.7.57 |
| 205Ch | QUEUE_5_D | | Section 16.4.7.58 |
| 2060h | QUEUE_6_A | | Section 16.4.7.59 |
| 2064h | QUEUE_6_B | | Section 16.4.7.60 |
| 2068h | QUEUE_6_C | | Section 16.4.7.61 |
| 206Ch | QUEUE_6_D | | Section 16.4.7.62 |
| 2070h | QUEUE_7_A | | Section 16.4.7.63 |
| 2074h | QUEUE_7_B | | Section 16.4.7.64 |
| 2078h | QUEUE_7_C | | Section 16.4.7.65 |

**Table 16-290. QUEUE_MGR REGISTERS (continued)**

| Offset | Acronym | Register Name | Section |
|---|---|---|---|
| 207Ch | QUEUE_7_D | | Section 16.4.7.66 |
| 2080h | QUEUE_8_A | | Section 16.4.7.67 |
| 2084h | QUEUE_8_B | | Section 16.4.7.68 |
| 2088h | QUEUE_8_C | | Section 16.4.7.69 |
| 208Ch | QUEUE_8_D | | Section 16.4.7.70 |
| 2090h | QUEUE_9_A | | Section 16.4.7.71 |
| 2094h | QUEUE_9_B | | Section 16.4.7.72 |
| 2098h | QUEUE_9_C | | Section 16.4.7.73 |
| 209Ch | QUEUE_9_D | | Section 16.4.7.74 |
| 20A0h | QUEUE_10_A | | Section 16.4.7.75 |
| 20A4h | QUEUE_10_B | | Section 16.4.7.76 |
| 20A8h | QUEUE_10_C | | Section 16.4.7.77 |
| 20ACh | QUEUE_10_D | | Section 16.4.7.78 |
| 20B0h | QUEUE_11_A | | Section 16.4.7.79 |
| 20B4h | QUEUE_11_B | | Section 16.4.7.80 |
| 20B8h | QUEUE_11_C | | Section 16.4.7.81 |
| 20BCh | QUEUE_11_D | | Section 16.4.7.82 |
| 20C0h | QUEUE_12_A | | Section 16.4.7.83 |
| 20C4h | QUEUE_12_B | | Section 16.4.7.84 |
| 20C8h | QUEUE_12_C | | Section 16.4.7.85 |
| 20CCh | QUEUE_12_D | | Section 16.4.7.86 |
| 20D0h | QUEUE_13_A | | Section 16.4.7.87 |
| 20D4h | QUEUE_13_B | | Section 16.4.7.88 |
| 20D8h | QUEUE_13_C | | Section 16.4.7.89 |
| 20DCh | QUEUE_13_D | | Section 16.4.7.90 |
| 20E0h | QUEUE_14_A | | Section 16.4.7.91 |
| 20E4h | QUEUE_14_B | | Section 16.4.7.92 |
| 20E8h | QUEUE_14_C | | Section 16.4.7.93 |
| 20ECh | QUEUE_14_D | | Section 16.4.7.94 |
| 20F0h | QUEUE_15_A | | Section 16.4.7.95 |
| 20F4h | QUEUE_15_B | | Section 16.4.7.96 |
| 20F8h | QUEUE_15_C | | Section 16.4.7.97 |
| 20FCh | QUEUE_15_D | | Section 16.4.7.98 |
| 2100h | QUEUE_16_A | | Section 16.4.7.99 |
| 2104h | QUEUE_16_B | | Section 16.4.7.100 |
| 2108h | QUEUE_16_C | | Section 16.4.7.101 |
| 210Ch | QUEUE_16_D | | Section 16.4.7.102 |
| 2110h | QUEUE_17_A | | Section 16.4.7.103 |
| 2114h | QUEUE_17_B | | Section 16.4.7.104 |
| 2118h | QUEUE_17_C | | Section 16.4.7.105 |
| 211Ch | QUEUE_17_D | | Section 16.4.7.106 |
| 2120h | QUEUE_18_A | | Section 16.4.7.107 |
| 2124h | QUEUE_18_B | | Section 16.4.7.108 |
| 2128h | QUEUE_18_C | | Section 16.4.7.109 |
| 212Ch | QUEUE_18_D | | Section 16.4.7.110 |
| 2130h | QUEUE_19_A | | Section 16.4.7.111 |
| 2134h | QUEUE_19_B | | Section 16.4.7.112 |

**Table 16-290. QUEUE_MGR REGISTERS (continued)**

| Offset | Acronym | Register Name | Section |
|---|---|---|---|
| 2138h | QUEUE_19_C | | Section 16.4.7.113 |
| 213Ch | QUEUE_19_D | | Section 16.4.7.114 |
| 2140h | QUEUE_20_A | | Section 16.4.7.115 |
| 2144h | QUEUE_20_B | | Section 16.4.7.116 |
| 2148h | QUEUE_20_C | | Section 16.4.7.117 |
| 214Ch | QUEUE_20_D | | Section 16.4.7.118 |
| 2150h | QUEUE_21_A | | Section 16.4.7.119 |
| 2154h | QUEUE_21_B | | Section 16.4.7.120 |
| 2158h | QUEUE_21_C | | Section 16.4.7.121 |
| 215Ch | QUEUE_21_D | | Section 16.4.7.122 |
| 2160h | QUEUE_22_A | | Section 16.4.7.123 |
| 2164h | QUEUE_22_B | | Section 16.4.7.124 |
| 2168h | QUEUE_22_C | | Section 16.4.7.125 |
| 216Ch | QUEUE_22_D | | Section 16.4.7.126 |
| 2170h | QUEUE_23_A | | Section 16.4.7.127 |
| 2174h | QUEUE_23_B | | Section 16.4.7.128 |
| 2178h | QUEUE_23_C | | Section 16.4.7.129 |
| 217Ch | QUEUE_23_D | | Section 16.4.7.130 |
| 2180h | QUEUE_24_A | | Section 16.4.7.131 |
| 2184h | QUEUE_24_B | | Section 16.4.7.132 |
| 2188h | QUEUE_24_C | | Section 16.4.7.133 |
| 218Ch | QUEUE_24_D | | Section 16.4.7.134 |
| 2190h | QUEUE_25_A | | Section 16.4.7.135 |
| 2194h | QUEUE_25_B | | Section 16.4.7.136 |
| 2198h | QUEUE_25_C | | Section 16.4.7.137 |
| 219Ch | QUEUE_25_D | | Section 16.4.7.138 |
| 21A0h | QUEUE_26_A | | Section 16.4.7.139 |
| 21A4h | QUEUE_26_B | | Section 16.4.7.140 |
| 21A8h | QUEUE_26_C | | Section 16.4.7.141 |
| 21ACh | QUEUE_26_D | | Section 16.4.7.142 |
| 21B0h | QUEUE_27_A | | Section 16.4.7.143 |
| 21B4h | QUEUE_27_B | | Section 16.4.7.144 |
| 21B8h | QUEUE_27_C | | Section 16.4.7.145 |
| 21BCh | QUEUE_27_D | | Section 16.4.7.146 |
| 21C0h | QUEUE_28_A | | Section 16.4.7.147 |
| 21C4h | QUEUE_28_B | | Section 16.4.7.148 |
| 21C8h | QUEUE_28_C | | Section 16.4.7.149 |
| 21CCh | QUEUE_28_D | | Section 16.4.7.150 |
| 21D0h | QUEUE_29_A | | Section 16.4.7.151 |
| 21D4h | QUEUE_29_B | | Section 16.4.7.152 |
| 21D8h | QUEUE_29_C | | Section 16.4.7.153 |
| 21DCh | QUEUE_29_D | | Section 16.4.7.154 |
| 21E0h | QUEUE_30_A | | Section 16.4.7.155 |
| 21E4h | QUEUE_30_B | | Section 16.4.7.156 |
| 21E8h | QUEUE_30_C | | Section 16.4.7.157 |
| 21ECh | QUEUE_30_D | | Section 16.4.7.158 |
| 21F0h | QUEUE_31_A | | Section 16.4.7.159 |

**Table 16-290. QUEUE_MGR REGISTERS (continued)**

| Offset | Acronym | Register Name | Section |
|---|---|---|---|
| 21F4h | QUEUE_31_B | | Section 16.4.7.160 |
| 21F8h | QUEUE_31_C | | Section 16.4.7.161 |
| 21FCh | QUEUE_31_D | | Section 16.4.7.162 |
| 2200h | QUEUE_32_A | | Section 16.4.7.163 |
| 2204h | QUEUE_32_B | | Section 16.4.7.164 |
| 2208h | QUEUE_32_C | | Section 16.4.7.165 |
| 220Ch | QUEUE_32_D | | Section 16.4.7.166 |
| 2210h | QUEUE_33_A | | Section 16.4.7.167 |
| 2214h | QUEUE_33_B | | Section 16.4.7.168 |
| 2218h | QUEUE_33_C | | Section 16.4.7.169 |
| 221Ch | QUEUE_33_D | | Section 16.4.7.170 |
| 2220h | QUEUE_34_A | | Section 16.4.7.171 |
| 2224h | QUEUE_34_B | | Section 16.4.7.172 |
| 2228h | QUEUE_34_C | | Section 16.4.7.173 |
| 222Ch | QUEUE_34_D | | Section 16.4.7.174 |
| 2230h | QUEUE_35_A | | Section 16.4.7.175 |
| 2234h | QUEUE_35_B | | Section 16.4.7.176 |
| 2238h | QUEUE_35_C | | Section 16.4.7.177 |
| 223Ch | QUEUE_35_D | | Section 16.4.7.178 |
| 2240h | QUEUE_36_A | | Section 16.4.7.179 |
| 2244h | QUEUE_36_B | | Section 16.4.7.180 |
| 2248h | QUEUE_36_C | | Section 16.4.7.181 |
| 224Ch | QUEUE_36_D | | Section 16.4.7.182 |
| 2250h | QUEUE_37_A | | Section 16.4.7.183 |
| 2254h | QUEUE_37_B | | Section 16.4.7.184 |
| 2258h | QUEUE_37_C | | Section 16.4.7.185 |
| 225Ch | QUEUE_37_D | | Section 16.4.7.186 |
| 2260h | QUEUE_38_A | | Section 16.4.7.187 |
| 2264h | QUEUE_38_B | | Section 16.4.7.188 |
| 2268h | QUEUE_38_C | | Section 16.4.7.189 |
| 226Ch | QUEUE_38_D | | Section 16.4.7.190 |
| 2270h | QUEUE_39_A | | Section 16.4.7.191 |
| 2274h | QUEUE_39_B | | Section 16.4.7.192 |
| 2278h | QUEUE_39_C | | Section 16.4.7.193 |
| 227Ch | QUEUE_39_D | | Section 16.4.7.194 |
| 2280h | QUEUE_40_A | | Section 16.4.7.195 |
| 2284h | QUEUE_40_B | | Section 16.4.7.196 |
| 2288h | QUEUE_40_C | | Section 16.4.7.197 |
| 228Ch | QUEUE_40_D | | Section 16.4.7.198 |
| 2290h | QUEUE_41_A | | Section 16.4.7.199 |
| 2294h | QUEUE_41_B | | Section 16.4.7.200 |
| 2298h | QUEUE_41_C | | Section 16.4.7.201 |
| 229Ch | QUEUE_41_D | | Section 16.4.7.202 |
| 22A0h | QUEUE_42_A | | Section 16.4.7.203 |
| 22A4h | QUEUE_42_B | | Section 16.4.7.204 |
| 22A8h | QUEUE_42_C | | Section 16.4.7.205 |
| 22ACh | QUEUE_42_D | | Section 16.4.7.206 |

Table 16-290. QUEUE_MGR REGISTERS (continued)

| Offset | Acronym | Register Name | Section |
|---|---|---|---|
| 22B0h | QUEUE_43_A | | Section 16.4.7.207 |
| 22B4h | QUEUE_43_B | | Section 16.4.7.208 |
| 22B8h | QUEUE_43_C | | Section 16.4.7.209 |
| 22BCh | QUEUE_43_D | | Section 16.4.7.210 |
| 22C0h | QUEUE_44_A | | Section 16.4.7.211 |
| 22C4h | QUEUE_44_B | | Section 16.4.7.212 |
| 22C8h | QUEUE_44_C | | Section 16.4.7.213 |
| 22CCh | QUEUE_44_D | | Section 16.4.7.214 |
| 22D0h | QUEUE_45_A | | Section 16.4.7.215 |
| 22D4h | QUEUE_45_B | | Section 16.4.7.216 |
| 22D8h | QUEUE_45_C | | Section 16.4.7.217 |
| 22DCh | QUEUE_45_D | | Section 16.4.7.218 |
| 22E0h | QUEUE_46_A | | Section 16.4.7.219 |
| 22E4h | QUEUE_46_B | | Section 16.4.7.220 |
| 22E8h | QUEUE_46_C | | Section 16.4.7.221 |
| 22ECh | QUEUE_46_D | | Section 16.4.7.222 |
| 22F0h | QUEUE_47_A | | Section 16.4.7.223 |
| 22F4h | QUEUE_47_B | | Section 16.4.7.224 |
| 22F8h | QUEUE_47_C | | Section 16.4.7.225 |
| 22FCh | QUEUE_47_D | | Section 16.4.7.226 |
| 2300h | QUEUE_48_A | | Section 16.4.7.227 |
| 2304h | QUEUE_48_B | | Section 16.4.7.228 |
| 2308h | QUEUE_48_C | | Section 16.4.7.229 |
| 230Ch | QUEUE_48_D | | Section 16.4.7.230 |
| 2310h | QUEUE_49_A | | Section 16.4.7.231 |
| 2314h | QUEUE_49_B | | Section 16.4.7.232 |
| 2318h | QUEUE_49_C | | Section 16.4.7.233 |
| 231Ch | QUEUE_49_D | | Section 16.4.7.234 |
| 2320h | QUEUE_50_A | | Section 16.4.7.235 |
| 2324h | QUEUE_50_B | | Section 16.4.7.236 |
| 2328h | QUEUE_50_C | | Section 16.4.7.237 |
| 232Ch | QUEUE_50_D | | Section 16.4.7.238 |
| 2330h | QUEUE_51_A | | Section 16.4.7.239 |
| 2334h | QUEUE_51_B | | Section 16.4.7.240 |
| 2338h | QUEUE_51_C | | Section 16.4.7.241 |
| 233Ch | QUEUE_51_D | | Section 16.4.7.242 |
| 2340h | QUEUE_52_A | | Section 16.4.7.243 |
| 2344h | QUEUE_52_B | | Section 16.4.7.244 |
| 2348h | QUEUE_52_C | | Section 16.4.7.245 |
| 234Ch | QUEUE_52_D | | Section 16.4.7.246 |
| 2350h | QUEUE_53_A | | Section 16.4.7.247 |
| 2354h | QUEUE_53_B | | Section 16.4.7.248 |
| 2358h | QUEUE_53_C | | Section 16.4.7.249 |
| 235Ch | QUEUE_53_D | | Section 16.4.7.250 |
| 2360h | QUEUE_54_A | | Section 16.4.7.251 |
| 2364h | QUEUE_54_B | | Section 16.4.7.252 |
| 2368h | QUEUE_54_C | | Section 16.4.7.253 |

**Table 16-290. QUEUE_MGR REGISTERS (continued)**

| Offset | Acronym | Register Name | Section |
|---|---|---|---|
| 236Ch | QUEUE_54_D | | Section 16.4.7.254 |
| 2370h | QUEUE_55_A | | Section 16.4.7.255 |
| 2374h | QUEUE_55_B | | Section 16.4.7.256 |
| 2378h | QUEUE_55_C | | Section 16.4.7.257 |
| 237Ch | QUEUE_55_D | | Section 16.4.7.258 |
| 2380h | QUEUE_56_A | | Section 16.4.7.259 |
| 2384h | QUEUE_56_B | | Section 16.4.7.260 |
| 2388h | QUEUE_56_C | | Section 16.4.7.261 |
| 238Ch | QUEUE_56_D | | Section 16.4.7.262 |
| 2390h | QUEUE_57_A | | Section 16.4.7.263 |
| 2394h | QUEUE_57_B | | Section 16.4.7.264 |
| 2398h | QUEUE_57_C | | Section 16.4.7.265 |
| 239Ch | QUEUE_57_D | | Section 16.4.7.266 |
| 23A0h | QUEUE_58_A | | Section 16.4.7.267 |
| 23A4h | QUEUE_58_B | | Section 16.4.7.268 |
| 23A8h | QUEUE_58_C | | Section 16.4.7.269 |
| 23ACh | QUEUE_58_D | | Section 16.4.7.270 |
| 23B0h | QUEUE_59_A | | Section 16.4.7.271 |
| 23B4h | QUEUE_59_B | | Section 16.4.7.272 |
| 23B8h | QUEUE_59_C | | Section 16.4.7.273 |
| 23BCh | QUEUE_59_D | | Section 16.4.7.274 |
| 23C0h | QUEUE_60_A | | Section 16.4.7.275 |
| 23C4h | QUEUE_60_B | | Section 16.4.7.276 |
| 23C8h | QUEUE_60_C | | Section 16.4.7.277 |
| 23CCh | QUEUE_60_D | | Section 16.4.7.278 |
| 23D0h | QUEUE_61_A | | Section 16.4.7.279 |
| 23D4h | QUEUE_61_B | | Section 16.4.7.280 |
| 23D8h | QUEUE_61_C | | Section 16.4.7.281 |
| 23DCh | QUEUE_61_D | | Section 16.4.7.282 |
| 23E0h | QUEUE_62_A | | Section 16.4.7.283 |
| 23E4h | QUEUE_62_B | | Section 16.4.7.284 |
| 23E8h | QUEUE_62_C | | Section 16.4.7.285 |
| 23ECh | QUEUE_62_D | | Section 16.4.7.286 |
| 23F0h | QUEUE_63_A | | Section 16.4.7.287 |
| 23F4h | QUEUE_63_B | | Section 16.4.7.288 |
| 23F8h | QUEUE_63_C | | Section 16.4.7.289 |
| 23FCh | QUEUE_63_D | | Section 16.4.7.290 |
| 2400h | QUEUE_64_A | | Section 16.4.7.291 |
| 2404h | QUEUE_64_B | | Section 16.4.7.292 |
| 2408h | QUEUE_64_C | | Section 16.4.7.293 |
| 240Ch | QUEUE_64_D | | Section 16.4.7.294 |
| 2410h | QUEUE_65_A | | Section 16.4.7.295 |
| 2414h | QUEUE_65_B | | Section 16.4.7.296 |
| 2418h | QUEUE_65_C | | Section 16.4.7.297 |
| 241Ch | QUEUE_65_D | | Section 16.4.7.298 |
| 2420h | QUEUE_66_A | | Section 16.4.7.299 |
| 2424h | QUEUE_66_B | | Section 16.4.7.300 |

**Table 16-290. QUEUE_MGR REGISTERS (continued)**

| Offset | Acronym | Register Name | Section |
|---|---|---|---|
| 2428h | QUEUE_66_C | | Section 16.4.7.301 |
| 242Ch | QUEUE_66_D | | Section 16.4.7.302 |
| 2430h | QUEUE_67_A | | Section 16.4.7.303 |
| 2434h | QUEUE_67_B | | Section 16.4.7.304 |
| 2438h | QUEUE_67_C | | Section 16.4.7.305 |
| 243Ch | QUEUE_67_D | | Section 16.4.7.306 |
| 2440h | QUEUE_68_A | | Section 16.4.7.307 |
| 2444h | QUEUE_68_B | | Section 16.4.7.308 |
| 2448h | QUEUE_68_C | | Section 16.4.7.309 |
| 244Ch | QUEUE_68_D | | Section 16.4.7.310 |
| 2450h | QUEUE_69_A | | Section 16.4.7.311 |
| 2454h | QUEUE_69_B | | Section 16.4.7.312 |
| 2458h | QUEUE_69_C | | Section 16.4.7.313 |
| 245Ch | QUEUE_69_D | | Section 16.4.7.314 |
| 2460h | QUEUE_70_A | | Section 16.4.7.315 |
| 2464h | QUEUE_70_B | | Section 16.4.7.316 |
| 2468h | QUEUE_70_C | | Section 16.4.7.317 |
| 246Ch | QUEUE_70_D | | Section 16.4.7.318 |
| 2470h | QUEUE_71_A | | Section 16.4.7.319 |
| 2474h | QUEUE_71_B | | Section 16.4.7.320 |
| 2478h | QUEUE_71_C | | Section 16.4.7.321 |
| 247Ch | QUEUE_71_D | | Section 16.4.7.322 |
| 2480h | QUEUE_72_A | | Section 16.4.7.323 |
| 2484h | QUEUE_72_B | | Section 16.4.7.324 |
| 2488h | QUEUE_72_C | | Section 16.4.7.325 |
| 248Ch | QUEUE_72_D | | Section 16.4.7.326 |
| 2490h | QUEUE_73_A | | Section 16.4.7.327 |
| 2494h | QUEUE_73_B | | Section 16.4.7.328 |
| 2498h | QUEUE_73_C | | Section 16.4.7.329 |
| 249Ch | QUEUE_73_D | | Section 16.4.7.330 |
| 24A0h | QUEUE_74_A | | Section 16.4.7.331 |
| 24A4h | QUEUE_74_B | | Section 16.4.7.332 |
| 24A8h | QUEUE_74_C | | Section 16.4.7.333 |
| 24ACh | QUEUE_74_D | | Section 16.4.7.334 |
| 24B0h | QUEUE_75_A | | Section 16.4.7.335 |
| 24B4h | QUEUE_75_B | | Section 16.4.7.336 |
| 24B8h | QUEUE_75_C | | Section 16.4.7.337 |
| 24BCh | QUEUE_75_D | | Section 16.4.7.338 |
| 24C0h | QUEUE_76_A | | Section 16.4.7.339 |
| 24C4h | QUEUE_76_B | | Section 16.4.7.340 |
| 24C8h | QUEUE_76_C | | Section 16.4.7.341 |
| 24CCh | QUEUE_76_D | | Section 16.4.7.342 |
| 24D0h | QUEUE_77_A | | Section 16.4.7.343 |
| 24D4h | QUEUE_77_B | | Section 16.4.7.344 |
| 24D8h | QUEUE_77_C | | Section 16.4.7.345 |
| 24DCh | QUEUE_77_D | | Section 16.4.7.346 |
| 24E0h | QUEUE_78_A | | Section 16.4.7.347 |

Table 16-290. QUEUE_MGR REGISTERS (continued)

| Offset | Acronym | Register Name | Section |
|---|---|---|---|
| 24E4h | QUEUE_78_B | | Section 16.4.7.348 |
| 24E8h | QUEUE_78_C | | Section 16.4.7.349 |
| 24ECh | QUEUE_78_D | | Section 16.4.7.350 |
| 24F0h | QUEUE_79_A | | Section 16.4.7.351 |
| 24F4h | QUEUE_79_B | | Section 16.4.7.352 |
| 24F8h | QUEUE_79_C | | Section 16.4.7.353 |
| 24FCh | QUEUE_79_D | | Section 16.4.7.354 |
| 2500h | QUEUE_80_A | | Section 16.4.7.355 |
| 2504h | QUEUE_80_B | | Section 16.4.7.356 |
| 2508h | QUEUE_80_C | | Section 16.4.7.357 |
| 250Ch | QUEUE_80_D | | Section 16.4.7.358 |
| 2510h | QUEUE_81_A | | Section 16.4.7.359 |
| 2514h | QUEUE_81_B | | Section 16.4.7.360 |
| 2518h | QUEUE_81_C | | Section 16.4.7.361 |
| 251Ch | QUEUE_81_D | | Section 16.4.7.362 |
| 2520h | QUEUE_82_A | | Section 16.4.7.363 |
| 2524h | QUEUE_82_B | | Section 16.4.7.364 |
| 2528h | QUEUE_82_C | | Section 16.4.7.365 |
| 252Ch | QUEUE_82_D | | Section 16.4.7.366 |
| 2530h | QUEUE_83_A | | Section 16.4.7.367 |
| 2534h | QUEUE_83_B | | Section 16.4.7.368 |
| 2538h | QUEUE_83_C | | Section 16.4.7.369 |
| 253Ch | QUEUE_83_D | | Section 16.4.7.370 |
| 2540h | QUEUE_84_A | | Section 16.4.7.371 |
| 2544h | QUEUE_84_B | | Section 16.4.7.372 |
| 2548h | QUEUE_84_C | | Section 16.4.7.373 |
| 254Ch | QUEUE_84_D | | Section 16.4.7.374 |
| 2550h | QUEUE_85_A | | Section 16.4.7.375 |
| 2554h | QUEUE_85_B | | Section 16.4.7.376 |
| 2558h | QUEUE_85_C | | Section 16.4.7.377 |
| 255Ch | QUEUE_85_D | | Section 16.4.7.378 |
| 2560h | QUEUE_86_A | | Section 16.4.7.379 |
| 2564h | QUEUE_86_B | | Section 16.4.7.380 |
| 2568h | QUEUE_86_C | | Section 16.4.7.381 |
| 256Ch | QUEUE_86_D | | Section 16.4.7.382 |
| 2570h | QUEUE_87_A | | Section 16.4.7.383 |
| 2574h | QUEUE_87_B | | Section 16.4.7.384 |
| 2578h | QUEUE_87_C | | Section 16.4.7.385 |
| 257Ch | QUEUE_87_D | | Section 16.4.7.386 |
| 2580h | QUEUE_88_A | | Section 16.4.7.387 |
| 2584h | QUEUE_88_B | | Section 16.4.7.388 |
| 2588h | QUEUE_88_C | | Section 16.4.7.389 |
| 258Ch | QUEUE_88_D | | Section 16.4.7.390 |
| 2590h | QUEUE_89_A | | Section 16.4.7.391 |
| 2594h | QUEUE_89_B | | Section 16.4.7.392 |
| 2598h | QUEUE_89_C | | Section 16.4.7.393 |
| 259Ch | QUEUE_89_D | | Section 16.4.7.394 |

**Table 16-290. QUEUE_MGR REGISTERS (continued)**

| Offset | Acronym | Register Name | Section |
|---|---|---|---|
| 25A0h | QUEUE_90_A | | Section 16.4.7.395 |
| 25A4h | QUEUE_90_B | | Section 16.4.7.396 |
| 25A8h | QUEUE_90_C | | Section 16.4.7.397 |
| 25ACh | QUEUE_90_D | | Section 16.4.7.398 |
| 25B0h | QUEUE_91_A | | Section 16.4.7.399 |
| 25B4h | QUEUE_91_B | | Section 16.4.7.400 |
| 25B8h | QUEUE_91_C | | Section 16.4.7.401 |
| 25BCh | QUEUE_91_D | | Section 16.4.7.402 |
| 25C0h | QUEUE_92_A | | Section 16.4.7.403 |
| 25C4h | QUEUE_92_B | | Section 16.4.7.404 |
| 25C8h | QUEUE_92_C | | Section 16.4.7.405 |
| 25CCh | QUEUE_92_D | | Section 16.4.7.406 |
| 25D0h | QUEUE_93_A | | Section 16.4.7.407 |
| 25D4h | QUEUE_93_B | | Section 16.4.7.408 |
| 25D8h | QUEUE_93_C | | Section 16.4.7.409 |
| 25DCh | QUEUE_93_D | | Section 16.4.7.410 |
| 25E0h | QUEUE_94_A | | Section 16.4.7.411 |
| 25E4h | QUEUE_94_B | | Section 16.4.7.412 |
| 25E8h | QUEUE_94_C | | Section 16.4.7.413 |
| 25ECh | QUEUE_94_D | | Section 16.4.7.414 |
| 25F0h | QUEUE_95_A | | Section 16.4.7.415 |
| 25F4h | QUEUE_95_B | | Section 16.4.7.416 |
| 25F8h | QUEUE_95_C | | Section 16.4.7.417 |
| 25FCh | QUEUE_95_D | | Section 16.4.7.418 |
| 2600h | QUEUE_96_A | | Section 16.4.7.419 |
| 2604h | QUEUE_96_B | | Section 16.4.7.420 |
| 2608h | QUEUE_96_C | | Section 16.4.7.421 |
| 260Ch | QUEUE_96_D | | Section 16.4.7.422 |
| 2610h | QUEUE_97_A | | Section 16.4.7.423 |
| 2614h | QUEUE_97_B | | Section 16.4.7.424 |
| 2618h | QUEUE_97_C | | Section 16.4.7.425 |
| 261Ch | QUEUE_97_D | | Section 16.4.7.426 |
| 2620h | QUEUE_98_A | | Section 16.4.7.427 |
| 2624h | QUEUE_98_B | | Section 16.4.7.428 |
| 2628h | QUEUE_98_C | | Section 16.4.7.429 |
| 262Ch | QUEUE_98_D | | Section 16.4.7.430 |
| 2630h | QUEUE_99_A | | Section 16.4.7.431 |
| 2634h | QUEUE_99_B | | Section 16.4.7.432 |
| 2638h | QUEUE_99_C | | Section 16.4.7.433 |
| 263Ch | QUEUE_99_D | | Section 16.4.7.434 |
| 2640h | QUEUE_100_A | | Section 16.4.7.435 |
| 2644h | QUEUE_100_B | | Section 16.4.7.436 |
| 2648h | QUEUE_100_C | | Section 16.4.7.437 |
| 264Ch | QUEUE_100_D | | Section 16.4.7.438 |
| 2650h | QUEUE_101_A | | Section 16.4.7.439 |
| 2654h | QUEUE_101_B | | Section 16.4.7.440 |
| 2658h | QUEUE_101_C | | Section 16.4.7.441 |

**Table 16-290. QUEUE_MGR REGISTERS (continued)**

| Offset | Acronym | Register Name | Section |
|---|---|---|---|
| 265Ch | QUEUE_101_D | | Section 16.4.7.442 |
| 2660h | QUEUE_102_A | | Section 16.4.7.443 |
| 2664h | QUEUE_102_B | | Section 16.4.7.444 |
| 2668h | QUEUE_102_C | | Section 16.4.7.445 |
| 266Ch | QUEUE_102_D | | Section 16.4.7.446 |
| 2670h | QUEUE_103_A | | Section 16.4.7.447 |
| 2674h | QUEUE_103_B | | Section 16.4.7.448 |
| 2678h | QUEUE_103_C | | Section 16.4.7.449 |
| 267Ch | QUEUE_103_D | | Section 16.4.7.450 |
| 2680h | QUEUE_104_A | | Section 16.4.7.451 |
| 2684h | QUEUE_104_B | | Section 16.4.7.452 |
| 2688h | QUEUE_104_C | | Section 16.4.7.453 |
| 268Ch | QUEUE_104_D | | Section 16.4.7.454 |
| 2690h | QUEUE_105_A | | Section 16.4.7.455 |
| 2694h | QUEUE_105_B | | Section 16.4.7.456 |
| 2698h | QUEUE_105_C | | Section 16.4.7.457 |
| 269Ch | QUEUE_105_D | | Section 16.4.7.458 |
| 26A0h | QUEUE_106_A | | Section 16.4.7.459 |
| 26A4h | QUEUE_106_B | | Section 16.4.7.460 |
| 26A8h | QUEUE_106_C | | Section 16.4.7.461 |
| 26ACh | QUEUE_106_D | | Section 16.4.7.462 |
| 26B0h | QUEUE_107_A | | Section 16.4.7.463 |
| 26B4h | QUEUE_107_B | | Section 16.4.7.464 |
| 26B8h | QUEUE_107_C | | Section 16.4.7.465 |
| 26BCh | QUEUE_107_D | | Section 16.4.7.466 |
| 26C0h | QUEUE_108_A | | Section 16.4.7.467 |
| 26C4h | QUEUE_108_B | | Section 16.4.7.468 |
| 26C8h | QUEUE_108_C | | Section 16.4.7.469 |
| 26CCh | QUEUE_108_D | | Section 16.4.7.470 |
| 26D0h | QUEUE_109_A | | Section 16.4.7.471 |
| 26D4h | QUEUE_109_B | | Section 16.4.7.472 |
| 26D8h | QUEUE_109_C | | Section 16.4.7.473 |
| 26DCh | QUEUE_109_D | | Section 16.4.7.474 |
| 26E0h | QUEUE_110_A | | Section 16.4.7.475 |
| 26E4h | QUEUE_110_B | | Section 16.4.7.476 |
| 26E8h | QUEUE_110_C | | Section 16.4.7.477 |
| 26ECh | QUEUE_110_D | | Section 16.4.7.478 |
| 26F0h | QUEUE_111_A | | Section 16.4.7.479 |
| 26F4h | QUEUE_111_B | | Section 16.4.7.480 |
| 26F8h | QUEUE_111_C | | Section 16.4.7.481 |
| 26FCh | QUEUE_111_D | | Section 16.4.7.482 |
| 2700h | QUEUE_112_A | | Section 16.4.7.483 |
| 2704h | QUEUE_112_B | | Section 16.4.7.484 |
| 2708h | QUEUE_112_C | | Section 16.4.7.485 |
| 270Ch | QUEUE_112_D | | Section 16.4.7.486 |
| 2710h | QUEUE_113_A | | Section 16.4.7.487 |
| 2714h | QUEUE_113_B | | Section 16.4.7.488 |

**Table 16-290. QUEUE_MGR REGISTERS (continued)**

| Offset | Acronym | Register Name | Section |
|---|---|---|---|
| 2718h | QUEUE_113_C | | Section 16.4.7.489 |
| 271Ch | QUEUE_113_D | | Section 16.4.7.490 |
| 2720h | QUEUE_114_A | | Section 16.4.7.491 |
| 2724h | QUEUE_114_B | | Section 16.4.7.492 |
| 2728h | QUEUE_114_C | | Section 16.4.7.493 |
| 272Ch | QUEUE_114_D | | Section 16.4.7.494 |
| 2730h | QUEUE_115_A | | Section 16.4.7.495 |
| 2734h | QUEUE_115_B | | Section 16.4.7.496 |
| 2738h | QUEUE_115_C | | Section 16.4.7.497 |
| 273Ch | QUEUE_115_D | | Section 16.4.7.498 |
| 2740h | QUEUE_116_A | | Section 16.4.7.499 |
| 2744h | QUEUE_116_B | | Section 16.4.7.500 |
| 2748h | QUEUE_116_C | | Section 16.4.7.501 |
| 274Ch | QUEUE_116_D | | Section 16.4.7.502 |
| 2750h | QUEUE_117_A | | Section 16.4.7.503 |
| 2754h | QUEUE_117_B | | Section 16.4.7.504 |
| 2758h | QUEUE_117_C | | Section 16.4.7.505 |
| 275Ch | QUEUE_117_D | | Section 16.4.7.506 |
| 2760h | QUEUE_118_A | | Section 16.4.7.507 |
| 2764h | QUEUE_118_B | | Section 16.4.7.508 |
| 2768h | QUEUE_118_C | | Section 16.4.7.509 |
| 276Ch | QUEUE_118_D | | Section 16.4.7.510 |
| 2770h | QUEUE_119_A | | Section 16.4.7.511 |
| 2774h | QUEUE_119_B | | Section 16.4.7.512 |
| 2778h | QUEUE_119_C | | Section 16.4.7.513 |
| 277Ch | QUEUE_119_D | | Section 16.4.7.514 |
| 2780h | QUEUE_120_A | | Section 16.4.7.515 |
| 2784h | QUEUE_120_B | | Section 16.4.7.516 |
| 2788h | QUEUE_120_C | | Section 16.4.7.517 |
| 278Ch | QUEUE_120_D | | Section 16.4.7.518 |
| 2790h | QUEUE_121_A | | Section 16.4.7.519 |
| 2794h | QUEUE_121_B | | Section 16.4.7.520 |
| 2798h | QUEUE_121_C | | Section 16.4.7.521 |
| 279Ch | QUEUE_121_D | | Section 16.4.7.522 |
| 27A0h | QUEUE_122_A | | Section 16.4.7.523 |
| 27A4h | QUEUE_122_B | | Section 16.4.7.524 |
| 27A8h | QUEUE_122_C | | Section 16.4.7.525 |
| 27ACh | QUEUE_122_D | | Section 16.4.7.526 |
| 27B0h | QUEUE_123_A | | Section 16.4.7.527 |
| 27B4h | QUEUE_123_B | | Section 16.4.7.528 |
| 27B8h | QUEUE_123_C | | Section 16.4.7.529 |
| 27BCh | QUEUE_123_D | | Section 16.4.7.530 |
| 27C0h | QUEUE_124_A | | Section 16.4.7.531 |
| 27C4h | QUEUE_124_B | | Section 16.4.7.532 |
| 27C8h | QUEUE_124_C | | Section 16.4.7.533 |
| 27CCh | QUEUE_124_D | | Section 16.4.7.534 |
| 27D0h | QUEUE_125_A | | Section 16.4.7.535 |

**Table 16-290. QUEUE_MGR REGISTERS (continued)**

| Offset | Acronym | Register Name | Section |
|---|---|---|---|
| 27D4h | QUEUE_125_B | | Section 16.4.7.536 |
| 27D8h | QUEUE_125_C | | Section 16.4.7.537 |
| 27DCh | QUEUE_125_D | | Section 16.4.7.538 |
| 27E0h | QUEUE_126_A | | Section 16.4.7.539 |
| 27E4h | QUEUE_126_B | | Section 16.4.7.540 |
| 27E8h | QUEUE_126_C | | Section 16.4.7.541 |
| 27ECh | QUEUE_126_D | | Section 16.4.7.542 |
| 27F0h | QUEUE_127_A | | Section 16.4.7.543 |
| 27F4h | QUEUE_127_B | | Section 16.4.7.544 |
| 27F8h | QUEUE_127_C | | Section 16.4.7.545 |
| 27FCh | QUEUE_127_D | | Section 16.4.7.546 |
| 2800h | QUEUE_128_A | | Section 16.4.7.547 |
| 2804h | QUEUE_128_B | | Section 16.4.7.548 |
| 2808h | QUEUE_128_C | | Section 16.4.7.549 |
| 280Ch | QUEUE_128_D | | Section 16.4.7.550 |
| 2810h | QUEUE_129_A | | Section 16.4.7.551 |
| 2814h | QUEUE_129_B | | Section 16.4.7.552 |
| 2818h | QUEUE_129_C | | Section 16.4.7.553 |
| 281Ch | QUEUE_129_D | | Section 16.4.7.554 |
| 2820h | QUEUE_130_A | | Section 16.4.7.555 |
| 2824h | QUEUE_130_B | | Section 16.4.7.556 |
| 2828h | QUEUE_130_C | | Section 16.4.7.557 |
| 282Ch | QUEUE_130_D | | Section 16.4.7.558 |
| 2830h | QUEUE_131_A | | Section 16.4.7.559 |
| 2834h | QUEUE_131_B | | Section 16.4.7.560 |
| 2838h | QUEUE_131_C | | Section 16.4.7.561 |
| 283Ch | QUEUE_131_D | | Section 16.4.7.562 |
| 2840h | QUEUE_132_A | | Section 16.4.7.563 |
| 2844h | QUEUE_132_B | | Section 16.4.7.564 |
| 2848h | QUEUE_132_C | | Section 16.4.7.565 |
| 284Ch | QUEUE_132_D | | Section 16.4.7.566 |
| 2850h | QUEUE_133_A | | Section 16.4.7.567 |
| 2854h | QUEUE_133_B | | Section 16.4.7.568 |
| 2858h | QUEUE_133_C | | Section 16.4.7.569 |
| 285Ch | QUEUE_133_D | | Section 16.4.7.570 |
| 2860h | QUEUE_134_A | | Section 16.4.7.571 |
| 2864h | QUEUE_134_B | | Section 16.4.7.572 |
| 2868h | QUEUE_134_C | | Section 16.4.7.573 |
| 286Ch | QUEUE_134_D | | Section 16.4.7.574 |
| 2870h | QUEUE_135_A | | Section 16.4.7.575 |
| 2874h | QUEUE_135_B | | Section 16.4.7.576 |
| 2878h | QUEUE_135_C | | Section 16.4.7.577 |
| 287Ch | QUEUE_135_D | | Section 16.4.7.578 |
| 2880h | QUEUE_136_A | | Section 16.4.7.579 |
| 2884h | QUEUE_136_B | | Section 16.4.7.580 |
| 2888h | QUEUE_136_C | | Section 16.4.7.581 |
| 288Ch | QUEUE_136_D | | Section 16.4.7.582 |

**Table 16-290. QUEUE_MGR REGISTERS (continued)**

| Offset | Acronym | Register Name | Section |
|---|---|---|---|
| 2890h | QUEUE_137_A | | Section 16.4.7.583 |
| 2894h | QUEUE_137_B | | Section 16.4.7.584 |
| 2898h | QUEUE_137_C | | Section 16.4.7.585 |
| 289Ch | QUEUE_137_D | | Section 16.4.7.586 |
| 28A0h | QUEUE_138_A | | Section 16.4.7.587 |
| 28A4h | QUEUE_138_B | | Section 16.4.7.588 |
| 28A8h | QUEUE_138_C | | Section 16.4.7.589 |
| 28ACh | QUEUE_138_D | | Section 16.4.7.590 |
| 28B0h | QUEUE_139_A | | Section 16.4.7.591 |
| 28B4h | QUEUE_139_B | | Section 16.4.7.592 |
| 28B8h | QUEUE_139_C | | Section 16.4.7.593 |
| 28BCh | QUEUE_139_D | | Section 16.4.7.594 |
| 28C0h | QUEUE_140_A | | Section 16.4.7.595 |
| 28C4h | QUEUE_140_B | | Section 16.4.7.596 |
| 28C8h | QUEUE_140_C | | Section 16.4.7.597 |
| 28CCh | QUEUE_140_D | | Section 16.4.7.598 |
| 28D0h | QUEUE_141_A | | Section 16.4.7.599 |
| 28D4h | QUEUE_141_B | | Section 16.4.7.600 |
| 28D8h | QUEUE_141_C | | Section 16.4.7.601 |
| 28DCh | QUEUE_141_D | | Section 16.4.7.602 |
| 28E0h | QUEUE_142_A | | Section 16.4.7.603 |
| 28E4h | QUEUE_142_B | | Section 16.4.7.604 |
| 28E8h | QUEUE_142_C | | Section 16.4.7.605 |
| 28ECh | QUEUE_142_D | | Section 16.4.7.606 |
| 28F0h | QUEUE_143_A | | Section 16.4.7.607 |
| 28F4h | QUEUE_143_B | | Section 16.4.7.608 |
| 28F8h | QUEUE_143_C | | Section 16.4.7.609 |
| 28FCh | QUEUE_143_D | | Section 16.4.7.610 |
| 2900h | QUEUE_144_A | | Section 16.4.7.611 |
| 2904h | QUEUE_144_B | | Section 16.4.7.612 |
| 2908h | QUEUE_144_C | | Section 16.4.7.613 |
| 290Ch | QUEUE_144_D | | Section 16.4.7.614 |
| 2910h | QUEUE_145_A | | Section 16.4.7.615 |
| 2914h | QUEUE_145_B | | Section 16.4.7.616 |
| 2918h | QUEUE_145_C | | Section 16.4.7.617 |
| 291Ch | QUEUE_145_D | | Section 16.4.7.618 |
| 2920h | QUEUE_146_A | | Section 16.4.7.619 |
| 2924h | QUEUE_146_B | | Section 16.4.7.620 |
| 2928h | QUEUE_146_C | | Section 16.4.7.621 |
| 292Ch | QUEUE_146_D | | Section 16.4.7.622 |
| 2930h | QUEUE_147_A | | Section 16.4.7.623 |
| 2934h | QUEUE_147_B | | Section 16.4.7.624 |
| 2938h | QUEUE_147_C | | Section 16.4.7.625 |
| 293Ch | QUEUE_147_D | | Section 16.4.7.626 |
| 2940h | QUEUE_148_A | | Section 16.4.7.627 |
| 2944h | QUEUE_148_B | | Section 16.4.7.628 |
| 2948h | QUEUE_148_C | | Section 16.4.7.629 |

Table 16-290. QUEUE_MGR REGISTERS (continued)

| Offset | Acronym | Register Name | Section |
|---|---|---|---|
| 294Ch | QUEUE_148_D | | Section 16.4.7.630 |
| 2950h | QUEUE_149_A | | Section 16.4.7.631 |
| 2954h | QUEUE_149_B | | Section 16.4.7.632 |
| 2958h | QUEUE_149_C | | Section 16.4.7.633 |
| 295Ch | QUEUE_149_D | | Section 16.4.7.634 |
| 2960h | QUEUE_150_A | | Section 16.4.7.635 |
| 2964h | QUEUE_150_B | | Section 16.4.7.636 |
| 2968h | QUEUE_150_C | | Section 16.4.7.637 |
| 296Ch | QUEUE_150_D | | Section 16.4.7.638 |
| 2970h | QUEUE_151_A | | Section 16.4.7.639 |
| 2974h | QUEUE_151_B | | Section 16.4.7.640 |
| 2978h | QUEUE_151_C | | Section 16.4.7.641 |
| 297Ch | QUEUE_151_D | | Section 16.4.7.642 |
| 2980h | QUEUE_152_A | | Section 16.4.7.643 |
| 2984h | QUEUE_152_B | | Section 16.4.7.644 |
| 2988h | QUEUE_152_C | | Section 16.4.7.645 |
| 298Ch | QUEUE_152_D | | Section 16.4.7.646 |
| 2990h | QUEUE_153_A | | Section 16.4.7.647 |
| 2994h | QUEUE_153_B | | Section 16.4.7.648 |
| 2998h | QUEUE_153_C | | Section 16.4.7.649 |
| 299Ch | QUEUE_153_D | | Section 16.4.7.650 |
| 29A0h | QUEUE_154_A | | Section 16.4.7.651 |
| 29A4h | QUEUE_154_B | | Section 16.4.7.652 |
| 29A8h | QUEUE_154_C | | Section 16.4.7.653 |
| 29ACh | QUEUE_154_D | | Section 16.4.7.654 |
| 29B0h | QUEUE_155_A | | Section 16.4.7.655 |
| 29B4h | QUEUE_155_B | | Section 16.4.7.656 |
| 29B8h | QUEUE_155_C | | Section 16.4.7.657 |
| 29BCh | QUEUE_155_D | | Section 16.4.7.658 |
| 3000h | QUEUE_0_STATUS_A | | Section 16.4.7.659 |
| 3004h | QUEUE_0_STATUS_B | | Section 16.4.7.660 |
| 3008h | QUEUE_0_STATUS_C | | Section 16.4.7.661 |
| 3010h | QUEUE_1_STATUS_A | | Section 16.4.7.662 |
| 3014h | QUEUE_1_STATUS_B | | Section 16.4.7.663 |
| 3018h | QUEUE_1_STATUS_C | | Section 16.4.7.664 |
| 3020h | QUEUE_2_STATUS_A | | Section 16.4.7.665 |
| 3024h | QUEUE_2_STATUS_B | | Section 16.4.7.666 |
| 3028h | QUEUE_2_STATUS_C | | Section 16.4.7.667 |
| 3030h | QUEUE_3_STATUS_A | | Section 16.4.7.668 |
| 3034h | QUEUE_3_STATUS_B | | Section 16.4.7.669 |
| 3038h | QUEUE_3_STATUS_C | | Section 16.4.7.670 |
| 3040h | QUEUE_4_STATUS_A | | Section 16.4.7.671 |
| 3044h | QUEUE_4_STATUS_B | | Section 16.4.7.672 |
| 3048h | QUEUE_4_STATUS_C | | Section 16.4.7.673 |
| 3050h | QUEUE_5_STATUS_A | | Section 16.4.7.674 |
| 3054h | QUEUE_5_STATUS_B | | Section 16.4.7.675 |
| 3058h | QUEUE_5_STATUS_C | | Section 16.4.7.676 |

**Table 16-290. QUEUE_MGR REGISTERS (continued)**

| Offset | Acronym | Register Name | Section |
|---|---|---|---|
| 3060h | QUEUE_6_STATUS_A | | Section 16.4.7.677 |
| 3064h | QUEUE_6_STATUS_B | | Section 16.4.7.678 |
| 3068h | QUEUE_6_STATUS_C | | Section 16.4.7.679 |
| 3070h | QUEUE_7_STATUS_A | | Section 16.4.7.680 |
| 3074h | QUEUE_7_STATUS_B | | Section 16.4.7.681 |
| 3078h | QUEUE_7_STATUS_C | | Section 16.4.7.682 |
| 3080h | QUEUE_8_STATUS_A | | Section 16.4.7.683 |
| 3084h | QUEUE_8_STATUS_B | | Section 16.4.7.684 |
| 3088h | QUEUE_8_STATUS_C | | Section 16.4.7.685 |
| 3090h | QUEUE_9_STATUS_A | | Section 16.4.7.686 |
| 3094h | QUEUE_9_STATUS_B | | Section 16.4.7.687 |
| 3098h | QUEUE_9_STATUS_C | | Section 16.4.7.688 |
| 30A0h | QUEUE_10_STATUS_A | | Section 16.4.7.689 |
| 30A4h | QUEUE_10_STATUS_B | | Section 16.4.7.690 |
| 30A8h | QUEUE_10_STATUS_C | | Section 16.4.7.691 |
| 30B0h | QUEUE_11_STATUS_A | | Section 16.4.7.692 |
| 30B4h | QUEUE_11_STATUS_B | | Section 16.4.7.693 |
| 30B8h | QUEUE_11_STATUS_C | | Section 16.4.7.694 |
| 30C0h | QUEUE_12_STATUS_A | | Section 16.4.7.695 |
| 30C4h | QUEUE_12_STATUS_B | | Section 16.4.7.696 |
| 30C8h | QUEUE_12_STATUS_C | | Section 16.4.7.697 |
| 30D0h | QUEUE_13_STATUS_A | | Section 16.4.7.698 |
| 30D4h | QUEUE_13_STATUS_B | | Section 16.4.7.699 |
| 30D8h | QUEUE_13_STATUS_C | | Section 16.4.7.700 |
| 30E0h | QUEUE_14_STATUS_A | | Section 16.4.7.701 |
| 30E4h | QUEUE_14_STATUS_B | | Section 16.4.7.702 |
| 30E8h | QUEUE_14_STATUS_C | | Section 16.4.7.703 |
| 30F0h | QUEUE_15_STATUS_A | | Section 16.4.7.704 |
| 30F4h | QUEUE_15_STATUS_B | | Section 16.4.7.705 |
| 30F8h | QUEUE_15_STATUS_C | | Section 16.4.7.706 |
| 3100h | QUEUE_16_STATUS_A | | Section 16.4.7.707 |
| 3104h | QUEUE_16_STATUS_B | | Section 16.4.7.708 |
| 3108h | QUEUE_16_STATUS_C | | Section 16.4.7.709 |
| 3110h | QUEUE_17_STATUS_A | | Section 16.4.7.710 |
| 3114h | QUEUE_17_STATUS_B | | Section 16.4.7.711 |
| 3118h | QUEUE_17_STATUS_C | | Section 16.4.7.712 |
| 3120h | QUEUE_18_STATUS_A | | Section 16.4.7.713 |
| 3124h | QUEUE_18_STATUS_B | | Section 16.4.7.714 |
| 3128h | QUEUE_18_STATUS_C | | Section 16.4.7.715 |
| 3130h | QUEUE_19_STATUS_A | | Section 16.4.7.716 |
| 3134h | QUEUE_19_STATUS_B | | Section 16.4.7.717 |
| 3138h | QUEUE_19_STATUS_C | | Section 16.4.7.718 |
| 3140h | QUEUE_20_STATUS_A | | Section 16.4.7.719 |
| 3144h | QUEUE_20_STATUS_B | | Section 16.4.7.720 |
| 3148h | QUEUE_20_STATUS_C | | Section 16.4.7.721 |
| 3150h | QUEUE_21_STATUS_A | | Section 16.4.7.722 |
| 3154h | QUEUE_21_STATUS_B | | Section 16.4.7.723 |

**Table 16-290. QUEUE_MGR REGISTERS (continued)**

| Offset | Acronym | Register Name | Section |
|---|---|---|---|
| 3158h | QUEUE_21_STATUS_C | | Section 16.4.7.724 |
| 3160h | QUEUE_22_STATUS_A | | Section 16.4.7.725 |
| 3164h | QUEUE_22_STATUS_B | | Section 16.4.7.726 |
| 3168h | QUEUE_22_STATUS_C | | Section 16.4.7.727 |
| 3170h | QUEUE_23_STATUS_A | | Section 16.4.7.728 |
| 3174h | QUEUE_23_STATUS_B | | Section 16.4.7.729 |
| 3178h | QUEUE_23_STATUS_C | | Section 16.4.7.730 |
| 3180h | QUEUE_24_STATUS_A | | Section 16.4.7.731 |
| 3184h | QUEUE_24_STATUS_B | | Section 16.4.7.732 |
| 3188h | QUEUE_24_STATUS_C | | Section 16.4.7.733 |
| 3190h | QUEUE_25_STATUS_A | | Section 16.4.7.734 |
| 3194h | QUEUE_25_STATUS_B | | Section 16.4.7.735 |
| 3198h | QUEUE_25_STATUS_C | | Section 16.4.7.736 |
| 31A0h | QUEUE_26_STATUS_A | | Section 16.4.7.737 |
| 31A4h | QUEUE_26_STATUS_B | | Section 16.4.7.738 |
| 31A8h | QUEUE_26_STATUS_C | | Section 16.4.7.739 |
| 31B0h | QUEUE_27_STATUS_A | | Section 16.4.7.740 |
| 31B4h | QUEUE_27_STATUS_B | | Section 16.4.7.741 |
| 31B8h | QUEUE_27_STATUS_C | | Section 16.4.7.742 |
| 31C0h | QUEUE_28_STATUS_A | | Section 16.4.7.743 |
| 31C4h | QUEUE_28_STATUS_B | | Section 16.4.7.744 |
| 31C8h | QUEUE_28_STATUS_C | | Section 16.4.7.745 |
| 31D0h | QUEUE_29_STATUS_A | | Section 16.4.7.746 |
| 31D4h | QUEUE_29_STATUS_B | | Section 16.4.7.747 |
| 31D8h | QUEUE_29_STATUS_C | | Section 16.4.7.748 |
| 31E0h | QUEUE_30_STATUS_A | | Section 16.4.7.749 |
| 31E4h | QUEUE_30_STATUS_B | | Section 16.4.7.750 |
| 31E8h | QUEUE_30_STATUS_C | | Section 16.4.7.751 |
| 31F0h | QUEUE_31_STATUS_A | | Section 16.4.7.752 |
| 31F4h | QUEUE_31_STATUS_B | | Section 16.4.7.753 |
| 31F8h | QUEUE_31_STATUS_C | | Section 16.4.7.754 |
| 3200h | QUEUE_32_STATUS_A | | Section 16.4.7.755 |
| 3204h | QUEUE_32_STATUS_B | | Section 16.4.7.756 |
| 3208h | QUEUE_32_STATUS_C | | Section 16.4.7.757 |
| 3210h | QUEUE_33_STATUS_A | | Section 16.4.7.758 |
| 3214h | QUEUE_33_STATUS_B | | Section 16.4.7.759 |
| 3218h | QUEUE_33_STATUS_C | | Section 16.4.7.760 |
| 3220h | QUEUE_34_STATUS_A | | Section 16.4.7.761 |
| 3224h | QUEUE_34_STATUS_B | | Section 16.4.7.762 |
| 3228h | QUEUE_34_STATUS_C | | Section 16.4.7.763 |
| 3230h | QUEUE_35_STATUS_A | | Section 16.4.7.764 |
| 3234h | QUEUE_35_STATUS_B | | Section 16.4.7.765 |
| 3238h | QUEUE_35_STATUS_C | | Section 16.4.7.766 |
| 3240h | QUEUE_36_STATUS_A | | Section 16.4.7.767 |
| 3244h | QUEUE_36_STATUS_B | | Section 16.4.7.768 |
| 3248h | QUEUE_36_STATUS_C | | Section 16.4.7.769 |
| 3250h | QUEUE_37_STATUS_A | | Section 16.4.7.770 |

**Table 16-290. QUEUE_MGR REGISTERS (continued)**

| Offset | Acronym | Register Name | Section |
|---|---|---|---|
| 3254h | QUEUE_37_STATUS_B | | Section 16.4.7.771 |
| 3258h | QUEUE_37_STATUS_C | | Section 16.4.7.772 |
| 3260h | QUEUE_38_STATUS_A | | Section 16.4.7.773 |
| 3264h | QUEUE_38_STATUS_B | | Section 16.4.7.774 |
| 3268h | QUEUE_38_STATUS_C | | Section 16.4.7.775 |
| 3270h | QUEUE_39_STATUS_A | | Section 16.4.7.776 |
| 3274h | QUEUE_39_STATUS_B | | Section 16.4.7.777 |
| 3278h | QUEUE_39_STATUS_C | | Section 16.4.7.778 |
| 3280h | QUEUE_40_STATUS_A | | Section 16.4.7.779 |
| 3284h | QUEUE_40_STATUS_B | | Section 16.4.7.780 |
| 3288h | QUEUE_40_STATUS_C | | Section 16.4.7.781 |
| 3290h | QUEUE_41_STATUS_A | | Section 16.4.7.782 |
| 3294h | QUEUE_41_STATUS_B | | Section 16.4.7.783 |
| 3298h | QUEUE_41_STATUS_C | | Section 16.4.7.784 |
| 32A0h | QUEUE_42_STATUS_A | | Section 16.4.7.785 |
| 32A4h | QUEUE_42_STATUS_B | | Section 16.4.7.786 |
| 32A8h | QUEUE_42_STATUS_C | | Section 16.4.7.787 |
| 32B0h | QUEUE_43_STATUS_A | | Section 16.4.7.788 |
| 32B4h | QUEUE_43_STATUS_B | | Section 16.4.7.789 |
| 32B8h | QUEUE_43_STATUS_C | | Section 16.4.7.790 |
| 32C0h | QUEUE_44_STATUS_A | | Section 16.4.7.791 |
| 32C4h | QUEUE_44_STATUS_B | | Section 16.4.7.792 |
| 32C8h | QUEUE_44_STATUS_C | | Section 16.4.7.793 |
| 32D0h | QUEUE_45_STATUS_A | | Section 16.4.7.794 |
| 32D4h | QUEUE_45_STATUS_B | | Section 16.4.7.795 |
| 32D8h | QUEUE_45_STATUS_C | | Section 16.4.7.796 |
| 32E0h | QUEUE_46_STATUS_A | | Section 16.4.7.797 |
| 32E4h | QUEUE_46_STATUS_B | | Section 16.4.7.798 |
| 32E8h | QUEUE_46_STATUS_C | | Section 16.4.7.799 |
| 32F0h | QUEUE_47_STATUS_A | | Section 16.4.7.800 |
| 32F4h | QUEUE_47_STATUS_B | | Section 16.4.7.801 |
| 32F8h | QUEUE_47_STATUS_C | | Section 16.4.7.802 |
| 3300h | QUEUE_48_STATUS_A | | Section 16.4.7.803 |
| 3304h | QUEUE_48_STATUS_B | | Section 16.4.7.804 |
| 3308h | QUEUE_48_STATUS_C | | Section 16.4.7.805 |
| 3310h | QUEUE_49_STATUS_A | | Section 16.4.7.806 |
| 3314h | QUEUE_49_STATUS_B | | Section 16.4.7.807 |
| 3318h | QUEUE_49_STATUS_C | | Section 16.4.7.808 |
| 3320h | QUEUE_50_STATUS_A | | Section 16.4.7.809 |
| 3324h | QUEUE_50_STATUS_B | | Section 16.4.7.810 |
| 3328h | QUEUE_50_STATUS_C | | Section 16.4.7.811 |
| 3330h | QUEUE_51_STATUS_A | | Section 16.4.7.812 |
| 3334h | QUEUE_51_STATUS_B | | Section 16.4.7.813 |
| 3338h | QUEUE_51_STATUS_C | | Section 16.4.7.814 |
| 3340h | QUEUE_52_STATUS_A | | Section 16.4.7.815 |
| 3344h | QUEUE_52_STATUS_B | | Section 16.4.7.816 |
| 3348h | QUEUE_52_STATUS_C | | Section 16.4.7.817 |

**Table 16-290. QUEUE_MGR REGISTERS (continued)**

| Offset | Acronym | Register Name | Section |
|---|---|---|---|
| 3350h | QUEUE_53_STATUS_A | | Section 16.4.7.818 |
| 3354h | QUEUE_53_STATUS_B | | Section 16.4.7.819 |
| 3358h | QUEUE_53_STATUS_C | | Section 16.4.7.820 |
| 3360h | QUEUE_54_STATUS_A | | Section 16.4.7.821 |
| 3364h | QUEUE_54_STATUS_B | | Section 16.4.7.822 |
| 3368h | QUEUE_54_STATUS_C | | Section 16.4.7.823 |
| 3370h | QUEUE_55_STATUS_A | | Section 16.4.7.824 |
| 3374h | QUEUE_55_STATUS_B | | Section 16.4.7.825 |
| 3378h | QUEUE_55_STATUS_C | | Section 16.4.7.826 |
| 3380h | QUEUE_56_STATUS_A | | Section 16.4.7.827 |
| 3384h | QUEUE_56_STATUS_B | | Section 16.4.7.828 |
| 3388h | QUEUE_56_STATUS_C | | Section 16.4.7.829 |
| 3390h | QUEUE_57_STATUS_A | | Section 16.4.7.830 |
| 3394h | QUEUE_57_STATUS_B | | Section 16.4.7.831 |
| 3398h | QUEUE_57_STATUS_C | | Section 16.4.7.832 |
| 33A0h | QUEUE_58_STATUS_A | | Section 16.4.7.833 |
| 33A4h | QUEUE_58_STATUS_B | | Section 16.4.7.834 |
| 33A8h | QUEUE_58_STATUS_C | | Section 16.4.7.835 |
| 33B0h | QUEUE_59_STATUS_A | | Section 16.4.7.836 |
| 33B4h | QUEUE_59_STATUS_B | | Section 16.4.7.837 |
| 33B8h | QUEUE_59_STATUS_C | | Section 16.4.7.838 |
| 33C0h | QUEUE_60_STATUS_A | | Section 16.4.7.839 |
| 33C4h | QUEUE_60_STATUS_B | | Section 16.4.7.840 |
| 33C8h | QUEUE_60_STATUS_C | | Section 16.4.7.841 |
| 33D0h | QUEUE_61_STATUS_A | | Section 16.4.7.842 |
| 33D4h | QUEUE_61_STATUS_B | | Section 16.4.7.843 |
| 33D8h | QUEUE_61_STATUS_C | | Section 16.4.7.844 |
| 33E0h | QUEUE_62_STATUS_A | | Section 16.4.7.845 |
| 33E4h | QUEUE_62_STATUS_B | | Section 16.4.7.846 |
| 33E8h | QUEUE_62_STATUS_C | | Section 16.4.7.847 |
| 33F0h | QUEUE_63_STATUS_A | | Section 16.4.7.848 |
| 33F4h | QUEUE_63_STATUS_B | | Section 16.4.7.849 |
| 33F8h | QUEUE_63_STATUS_C | | Section 16.4.7.850 |
| 3400h | QUEUE_64_STATUS_A | | Section 16.4.7.851 |
| 3404h | QUEUE_64_STATUS_B | | Section 16.4.7.852 |
| 3408h | QUEUE_64_STATUS_C | | Section 16.4.7.853 |
| 3410h | QUEUE_65_STATUS_A | | Section 16.4.7.854 |
| 3414h | QUEUE_65_STATUS_B | | Section 16.4.7.855 |
| 3418h | QUEUE_65_STATUS_C | | Section 16.4.7.856 |
| 3420h | QUEUE_66_STATUS_A | | Section 16.4.7.857 |
| 3424h | QUEUE_66_STATUS_B | | Section 16.4.7.858 |
| 3428h | QUEUE_66_STATUS_C | | Section 16.4.7.859 |
| 3430h | QUEUE_67_STATUS_A | | Section 16.4.7.860 |
| 3434h | QUEUE_67_STATUS_B | | Section 16.4.7.861 |
| 3438h | QUEUE_67_STATUS_C | | Section 16.4.7.862 |
| 3440h | QUEUE_68_STATUS_A | | Section 16.4.7.863 |
| 3444h | QUEUE_68_STATUS_B | | Section 16.4.7.864 |

**Table 16-290. QUEUE_MGR REGISTERS (continued)**

| Offset | Acronym | Register Name | Section |
|---|---|---|---|
| 3448h | QUEUE_68_STATUS_C | | Section 16.4.7.865 |
| 3450h | QUEUE_69_STATUS_A | | Section 16.4.7.866 |
| 3454h | QUEUE_69_STATUS_B | | Section 16.4.7.867 |
| 3458h | QUEUE_69_STATUS_C | | Section 16.4.7.868 |
| 3460h | QUEUE_70_STATUS_A | | Section 16.4.7.869 |
| 3464h | QUEUE_70_STATUS_B | | Section 16.4.7.870 |
| 3468h | QUEUE_70_STATUS_C | | Section 16.4.7.871 |
| 3470h | QUEUE_71_STATUS_A | | Section 16.4.7.872 |
| 3474h | QUEUE_71_STATUS_B | | Section 16.4.7.873 |
| 3478h | QUEUE_71_STATUS_C | | Section 16.4.7.874 |
| 3480h | QUEUE_72_STATUS_A | | Section 16.4.7.875 |
| 3484h | QUEUE_72_STATUS_B | | Section 16.4.7.876 |
| 3488h | QUEUE_72_STATUS_C | | Section 16.4.7.877 |
| 3490h | QUEUE_73_STATUS_A | | Section 16.4.7.878 |
| 3494h | QUEUE_73_STATUS_B | | Section 16.4.7.879 |
| 3498h | QUEUE_73_STATUS_C | | Section 16.4.7.880 |
| 34A0h | QUEUE_74_STATUS_A | | Section 16.4.7.881 |
| 34A4h | QUEUE_74_STATUS_B | | Section 16.4.7.882 |
| 34A8h | QUEUE_74_STATUS_C | | Section 16.4.7.883 |
| 34B0h | QUEUE_75_STATUS_A | | Section 16.4.7.884 |
| 34B4h | QUEUE_75_STATUS_B | | Section 16.4.7.885 |
| 34B8h | QUEUE_75_STATUS_C | | Section 16.4.7.886 |
| 34C0h | QUEUE_76_STATUS_A | | Section 16.4.7.887 |
| 34C4h | QUEUE_76_STATUS_B | | Section 16.4.7.888 |
| 34C8h | QUEUE_76_STATUS_C | | Section 16.4.7.889 |
| 34D0h | QUEUE_77_STATUS_A | | Section 16.4.7.890 |
| 34D4h | QUEUE_77_STATUS_B | | Section 16.4.7.891 |
| 34D8h | QUEUE_77_STATUS_C | | Section 16.4.7.892 |
| 34E0h | QUEUE_78_STATUS_A | | Section 16.4.7.893 |
| 34E4h | QUEUE_78_STATUS_B | | Section 16.4.7.894 |
| 34E8h | QUEUE_78_STATUS_C | | Section 16.4.7.895 |
| 34F0h | QUEUE_79_STATUS_A | | Section 16.4.7.896 |
| 34F4h | QUEUE_79_STATUS_B | | Section 16.4.7.897 |
| 34F8h | QUEUE_79_STATUS_C | | Section 16.4.7.898 |
| 3500h | QUEUE_80_STATUS_A | | Section 16.4.7.899 |
| 3504h | QUEUE_80_STATUS_B | | Section 16.4.7.900 |
| 3508h | QUEUE_80_STATUS_C | | Section 16.4.7.901 |
| 3510h | QUEUE_81_STATUS_A | | Section 16.4.7.902 |
| 3514h | QUEUE_81_STATUS_B | | Section 16.4.7.903 |
| 3518h | QUEUE_81_STATUS_C | | Section 16.4.7.904 |
| 3520h | QUEUE_82_STATUS_A | | Section 16.4.7.905 |
| 3524h | QUEUE_82_STATUS_B | | Section 16.4.7.906 |
| 3528h | QUEUE_82_STATUS_C | | Section 16.4.7.907 |
| 3530h | QUEUE_83_STATUS_A | | Section 16.4.7.908 |
| 3534h | QUEUE_83_STATUS_B | | Section 16.4.7.909 |
| 3538h | QUEUE_83_STATUS_C | | Section 16.4.7.910 |
| 3540h | QUEUE_84_STATUS_A | | Section 16.4.7.911 |

**Table 16-290. QUEUE_MGR REGISTERS (continued)**

| Offset | Acronym | Register Name | Section |
|---|---|---|---|
| 3544h | QUEUE_84_STATUS_B | | Section 16.4.7.912 |
| 3548h | QUEUE_84_STATUS_C | | Section 16.4.7.913 |
| 3550h | QUEUE_85_STATUS_A | | Section 16.4.7.914 |
| 3554h | QUEUE_85_STATUS_B | | Section 16.4.7.915 |
| 3558h | QUEUE_85_STATUS_C | | Section 16.4.7.916 |
| 3560h | QUEUE_86_STATUS_A | | Section 16.4.7.917 |
| 3564h | QUEUE_86_STATUS_B | | Section 16.4.7.918 |
| 3568h | QUEUE_86_STATUS_C | | Section 16.4.7.919 |
| 3570h | QUEUE_87_STATUS_A | | Section 16.4.7.920 |
| 3574h | QUEUE_87_STATUS_B | | Section 16.4.7.921 |
| 3578h | QUEUE_87_STATUS_C | | Section 16.4.7.922 |
| 3580h | QUEUE_88_STATUS_A | | Section 16.4.7.923 |
| 3584h | QUEUE_88_STATUS_B | | Section 16.4.7.924 |
| 3588h | QUEUE_88_STATUS_C | | Section 16.4.7.925 |
| 3590h | QUEUE_89_STATUS_A | | Section 16.4.7.926 |
| 3594h | QUEUE_89_STATUS_B | | Section 16.4.7.927 |
| 3598h | QUEUE_89_STATUS_C | | Section 16.4.7.928 |
| 35A0h | QUEUE_90_STATUS_A | | Section 16.4.7.929 |
| 35A4h | QUEUE_90_STATUS_B | | Section 16.4.7.930 |
| 35A8h | QUEUE_90_STATUS_C | | Section 16.4.7.931 |
| 35B0h | QUEUE_91_STATUS_A | | Section 16.4.7.932 |
| 35B4h | QUEUE_91_STATUS_B | | Section 16.4.7.933 |
| 35B8h | QUEUE_91_STATUS_C | | Section 16.4.7.934 |
| 35C0h | QUEUE_92_STATUS_A | | Section 16.4.7.935 |
| 35C4h | QUEUE_92_STATUS_B | | Section 16.4.7.936 |
| 35C8h | QUEUE_92_STATUS_C | | Section 16.4.7.937 |
| 35D0h | QUEUE_93_STATUS_A | | Section 16.4.7.938 |
| 35D4h | QUEUE_93_STATUS_B | | Section 16.4.7.939 |
| 35D8h | QUEUE_93_STATUS_C | | Section 16.4.7.940 |
| 35E0h | QUEUE_94_STATUS_A | | Section 16.4.7.941 |
| 35E4h | QUEUE_94_STATUS_B | | Section 16.4.7.942 |
| 35E8h | QUEUE_94_STATUS_C | | Section 16.4.7.943 |
| 35F0h | QUEUE_95_STATUS_A | | Section 16.4.7.944 |
| 35F4h | QUEUE_95_STATUS_B | | Section 16.4.7.945 |
| 35F8h | QUEUE_95_STATUS_C | | Section 16.4.7.946 |
| 3600h | QUEUE_96_STATUS_A | | Section 16.4.7.947 |
| 3604h | QUEUE_96_STATUS_B | | Section 16.4.7.948 |
| 3608h | QUEUE_96_STATUS_C | | Section 16.4.7.949 |
| 3610h | QUEUE_97_STATUS_A | | Section 16.4.7.950 |
| 3614h | QUEUE_97_STATUS_B | | Section 16.4.7.951 |
| 3618h | QUEUE_97_STATUS_C | | Section 16.4.7.952 |
| 3620h | QUEUE_98_STATUS_A | | Section 16.4.7.953 |
| 3624h | QUEUE_98_STATUS_B | | Section 16.4.7.954 |
| 3628h | QUEUE_98_STATUS_C | | Section 16.4.7.955 |
| 3630h | QUEUE_99_STATUS_A | | Section 16.4.7.956 |
| 3634h | QUEUE_99_STATUS_B | | Section 16.4.7.957 |
| 3638h | QUEUE_99_STATUS_C | | Section 16.4.7.958 |

**Table 16-290. QUEUE_MGR REGISTERS (continued)**

| Offset | Acronym | Register Name | Section |
|---|---|---|---|
| 3640h | QUEUE_100_STATUS_A | | Section 16.4.7.959 |
| 3644h | QUEUE_100_STATUS_B | | Section 16.4.7.960 |
| 3648h | QUEUE_100_STATUS_C | | Section 16.4.7.961 |
| 3650h | QUEUE_101_STATUS_A | | Section 16.4.7.962 |
| 3654h | QUEUE_101_STATUS_B | | Section 16.4.7.963 |
| 3658h | QUEUE_101_STATUS_C | | Section 16.4.7.964 |
| 3660h | QUEUE_102_STATUS_A | | Section 16.4.7.965 |
| 3664h | QUEUE_102_STATUS_B | | Section 16.4.7.966 |
| 3668h | QUEUE_102_STATUS_C | | Section 16.4.7.967 |
| 3670h | QUEUE_103_STATUS_A | | Section 16.4.7.968 |
| 3674h | QUEUE_103_STATUS_B | | Section 16.4.7.969 |
| 3678h | QUEUE_103_STATUS_C | | Section 16.4.7.970 |
| 3680h | QUEUE_104_STATUS_A | | Section 16.4.7.971 |
| 3684h | QUEUE_104_STATUS_B | | Section 16.4.7.972 |
| 3688h | QUEUE_104_STATUS_C | | Section 16.4.7.973 |
| 3690h | QUEUE_105_STATUS_A | | Section 16.4.7.974 |
| 3694h | QUEUE_105_STATUS_B | | Section 16.4.7.975 |
| 3698h | QUEUE_105_STATUS_C | | Section 16.4.7.976 |
| 36A0h | QUEUE_106_STATUS_A | | Section 16.4.7.977 |
| 36A4h | QUEUE_106_STATUS_B | | Section 16.4.7.978 |
| 36A8h | QUEUE_106_STATUS_C | | Section 16.4.7.979 |
| 36B0h | QUEUE_107_STATUS_A | | Section 16.4.7.980 |
| 36B4h | QUEUE_107_STATUS_B | | Section 16.4.7.981 |
| 36B8h | QUEUE_107_STATUS_C | | Section 16.4.7.982 |
| 36C0h | QUEUE_108_STATUS_A | | Section 16.4.7.983 |
| 36C4h | QUEUE_108_STATUS_B | | Section 16.4.7.984 |
| 36C8h | QUEUE_108_STATUS_C | | Section 16.4.7.985 |
| 36D0h | QUEUE_109_STATUS_A | | Section 16.4.7.986 |
| 36D4h | QUEUE_109_STATUS_B | | Section 16.4.7.987 |
| 36D8h | QUEUE_109_STATUS_C | | Section 16.4.7.988 |
| 36E0h | QUEUE_110_STATUS_A | | Section 16.4.7.989 |
| 36E4h | QUEUE_110_STATUS_B | | Section 16.4.7.990 |
| 36E8h | QUEUE_110_STATUS_C | | Section 16.4.7.991 |
| 36F0h | QUEUE_111_STATUS_A | | Section 16.4.7.992 |
| 36F4h | QUEUE_111_STATUS_B | | Section 16.4.7.993 |
| 36F8h | QUEUE_111_STATUS_C | | Section 16.4.7.994 |
| 3700h | QUEUE_112_STATUS_A | | Section 16.4.7.995 |
| 3704h | QUEUE_112_STATUS_B | | Section 16.4.7.996 |
| 3708h | QUEUE_112_STATUS_C | | Section 16.4.7.997 |
| 3710h | QUEUE_113_STATUS_A | | Section 16.4.7.998 |
| 3714h | QUEUE_113_STATUS_B | | Section 16.4.7.999 |
| 3718h | QUEUE_113_STATUS_C | | Section 16.4.7.1000 |
| 3720h | QUEUE_114_STATUS_A | | Section 16.4.7.1001 |
| 3724h | QUEUE_114_STATUS_B | | Section 16.4.7.1002 |
| 3728h | QUEUE_114_STATUS_C | | Section 16.4.7.1003 |
| 3730h | QUEUE_115_STATUS_A | | Section 16.4.7.1004 |
| 3734h | QUEUE_115_STATUS_B | | Section 16.4.7.1005 |

**Table 16-290. QUEUE_MGR REGISTERS (continued)**

| Offset | Acronym | Register Name | Section |
|---|---|---|---|
| 3738h | QUEUE_115_STATUS_C | | Section 16.4.7.1006 |
| 3740h | QUEUE_116_STATUS_A | | Section 16.4.7.1007 |
| 3744h | QUEUE_116_STATUS_B | | Section 16.4.7.1008 |
| 3748h | QUEUE_116_STATUS_C | | Section 16.4.7.1009 |
| 3750h | QUEUE_117_STATUS_A | | Section 16.4.7.1010 |
| 3754h | QUEUE_117_STATUS_B | | Section 16.4.7.1011 |
| 3758h | QUEUE_117_STATUS_C | | Section 16.4.7.1012 |
| 3760h | QUEUE_118_STATUS_A | | Section 16.4.7.1013 |
| 3764h | QUEUE_118_STATUS_B | | Section 16.4.7.1014 |
| 3768h | QUEUE_118_STATUS_C | | Section 16.4.7.1015 |
| 3770h | QUEUE_119_STATUS_A | | Section 16.4.7.1016 |
| 3774h | QUEUE_119_STATUS_B | | Section 16.4.7.1017 |
| 3778h | QUEUE_119_STATUS_C | | Section 16.4.7.1018 |
| 3780h | QUEUE_120_STATUS_A | | Section 16.4.7.1019 |
| 3784h | QUEUE_120_STATUS_B | | Section 16.4.7.1020 |
| 3788h | QUEUE_120_STATUS_C | | Section 16.4.7.1021 |
| 3790h | QUEUE_121_STATUS_A | | Section 16.4.7.1022 |
| 3794h | QUEUE_121_STATUS_B | | Section 16.4.7.1023 |
| 3798h | QUEUE_121_STATUS_C | | Section 16.4.7.1024 |
| 37A0h | QUEUE_122_STATUS_A | | Section 16.4.7.1025 |
| 37A4h | QUEUE_122_STATUS_B | | Section 16.4.7.1026 |
| 37A8h | QUEUE_122_STATUS_C | | Section 16.4.7.1027 |
| 37B0h | QUEUE_123_STATUS_A | | Section 16.4.7.1028 |
| 37B4h | QUEUE_123_STATUS_B | | Section 16.4.7.1029 |
| 37B8h | QUEUE_123_STATUS_C | | Section 16.4.7.1030 |
| 37C0h | QUEUE_124_STATUS_A | | Section 16.4.7.1031 |
| 37C4h | QUEUE_124_STATUS_B | | Section 16.4.7.1032 |
| 37C8h | QUEUE_124_STATUS_C | | Section 16.4.7.1033 |
| 37D0h | QUEUE_125_STATUS_A | | Section 16.4.7.1034 |
| 37D4h | QUEUE_125_STATUS_B | | Section 16.4.7.1035 |
| 37D8h | QUEUE_125_STATUS_C | | Section 16.4.7.1036 |
| 37E0h | QUEUE_126_STATUS_A | | Section 16.4.7.1037 |
| 37E4h | QUEUE_126_STATUS_B | | Section 16.4.7.1038 |
| 37E8h | QUEUE_126_STATUS_C | | Section 16.4.7.1039 |
| 37F0h | QUEUE_127_STATUS_A | | Section 16.4.7.1040 |
| 37F4h | QUEUE_127_STATUS_B | | Section 16.4.7.1041 |
| 37F8h | QUEUE_127_STATUS_C | | Section 16.4.7.1042 |
| 3800h | QUEUE_128_STATUS_A | | Section 16.4.7.1043 |
| 3804h | QUEUE_128_STATUS_B | | Section 16.4.7.1044 |
| 3808h | QUEUE_128_STATUS_C | | Section 16.4.7.1045 |
| 3810h | QUEUE_129_STATUS_A | | Section 16.4.7.1046 |
| 3814h | QUEUE_129_STATUS_B | | Section 16.4.7.1047 |
| 3818h | QUEUE_129_STATUS_C | | Section 16.4.7.1048 |
| 3820h | QUEUE_130_STATUS_A | | Section 16.4.7.1049 |
| 3824h | QUEUE_130_STATUS_B | | Section 16.4.7.1050 |
| 3828h | QUEUE_130_STATUS_C | | Section 16.4.7.1051 |
| 3830h | QUEUE_131_STATUS_A | | Section 16.4.7.1052 |

**Table 16-290. QUEUE_MGR REGISTERS (continued)**

| Offset | Acronym | Register Name | Section |
|---|---|---|---|
| 3834h | QUEUE_131_STATUS_B | | Section 16.4.7.1053 |
| 3838h | QUEUE_131_STATUS_C | | Section 16.4.7.1054 |
| 3840h | QUEUE_132_STATUS_A | | Section 16.4.7.1055 |
| 3844h | QUEUE_132_STATUS_B | | Section 16.4.7.1056 |
| 3848h | QUEUE_132_STATUS_C | | Section 16.4.7.1057 |
| 3850h | QUEUE_133_STATUS_A | | Section 16.4.7.1058 |
| 3854h | QUEUE_133_STATUS_B | | Section 16.4.7.1059 |
| 3858h | QUEUE_133_STATUS_C | | Section 16.4.7.1060 |
| 3860h | QUEUE_134_STATUS_A | | Section 16.4.7.1061 |
| 3864h | QUEUE_134_STATUS_B | | Section 16.4.7.1062 |
| 3868h | QUEUE_134_STATUS_C | | Section 16.4.7.1063 |
| 3870h | QUEUE_135_STATUS_A | | Section 16.4.7.1064 |
| 3874h | QUEUE_135_STATUS_B | | Section 16.4.7.1065 |
| 3878h | QUEUE_135_STATUS_C | | Section 16.4.7.1066 |
| 3880h | QUEUE_136_STATUS_A | | Section 16.4.7.1067 |
| 3884h | QUEUE_136_STATUS_B | | Section 16.4.7.1068 |
| 3888h | QUEUE_136_STATUS_C | | Section 16.4.7.1069 |
| 3890h | QUEUE_137_STATUS_A | | Section 16.4.7.1070 |
| 3894h | QUEUE_137_STATUS_B | | Section 16.4.7.1071 |
| 3898h | QUEUE_137_STATUS_C | | Section 16.4.7.1072 |
| 38A0h | QUEUE_138_STATUS_A | | Section 16.4.7.1073 |
| 38A4h | QUEUE_138_STATUS_B | | Section 16.4.7.1074 |
| 38A8h | QUEUE_138_STATUS_C | | Section 16.4.7.1075 |
| 38B0h | QUEUE_139_STATUS_A | | Section 16.4.7.1076 |
| 38B4h | QUEUE_139_STATUS_B | | Section 16.4.7.1077 |
| 38B8h | QUEUE_139_STATUS_C | | Section 16.4.7.1078 |
| 38C0h | QUEUE_140_STATUS_A | | Section 16.4.7.1079 |
| 38C4h | QUEUE_140_STATUS_B | | Section 16.4.7.1080 |
| 38C8h | QUEUE_140_STATUS_C | | Section 16.4.7.1081 |
| 38D0h | QUEUE_141_STATUS_A | | Section 16.4.7.1082 |
| 38D4h | QUEUE_141_STATUS_B | | Section 16.4.7.1083 |
| 38D8h | QUEUE_141_STATUS_C | | Section 16.4.7.1084 |
| 38E0h | QUEUE_142_STATUS_A | | Section 16.4.7.1085 |
| 38E4h | QUEUE_142_STATUS_B | | Section 16.4.7.1086 |
| 38E8h | QUEUE_142_STATUS_C | | Section 16.4.7.1087 |
| 38F0h | QUEUE_143_STATUS_A | | Section 16.4.7.1088 |
| 38F4h | QUEUE_143_STATUS_B | | Section 16.4.7.1089 |
| 38F8h | QUEUE_143_STATUS_C | | Section 16.4.7.1090 |
| 3900h | QUEUE_144_STATUS_A | | Section 16.4.7.1091 |
| 3904h | QUEUE_144_STATUS_B | | Section 16.4.7.1092 |
| 3908h | QUEUE_144_STATUS_C | | Section 16.4.7.1093 |
| 3910h | QUEUE_145_STATUS_A | | Section 16.4.7.1094 |
| 3914h | QUEUE_145_STATUS_B | | Section 16.4.7.1095 |
| 3918h | QUEUE_145_STATUS_C | | Section 16.4.7.1096 |
| 3920h | QUEUE_146_STATUS_A | | Section 16.4.7.1097 |
| 3924h | QUEUE_146_STATUS_B | | Section 16.4.7.1098 |
| 3928h | QUEUE_146_STATUS_C | | Section 16.4.7.1099 |

**Table 16-290. QUEUE_MGR REGISTERS (continued)**

| Offset | Acronym | Register Name | Section |
|---|---|---|---|
| 3930h | QUEUE_147_STATUS_A | | Section 16.4.7.1100 |
| 3934h | QUEUE_147_STATUS_B | | Section 16.4.7.1101 |
| 3938h | QUEUE_147_STATUS_C | | Section 16.4.7.1102 |
| 3940h | QUEUE_148_STATUS_A | | Section 16.4.7.1103 |
| 3944h | QUEUE_148_STATUS_B | | Section 16.4.7.1104 |
| 3948h | QUEUE_148_STATUS_C | | Section 16.4.7.1105 |
| 3950h | QUEUE_149_STATUS_A | | Section 16.4.7.1106 |
| 3954h | QUEUE_149_STATUS_B | | Section 16.4.7.1107 |
| 3958h | QUEUE_149_STATUS_C | | Section 16.4.7.1108 |
| 3960h | QUEUE_150_STATUS_A | | Section 16.4.7.1109 |
| 3964h | QUEUE_150_STATUS_B | | Section 16.4.7.1110 |
| 3968h | QUEUE_150_STATUS_C | | Section 16.4.7.1111 |
| 3970h | QUEUE_151_STATUS_A | | Section 16.4.7.1112 |
| 3974h | QUEUE_151_STATUS_B | | Section 16.4.7.1113 |
| 3978h | QUEUE_151_STATUS_C | | Section 16.4.7.1114 |
| 3980h | QUEUE_152_STATUS_A | | Section 16.4.7.1115 |
| 3984h | QUEUE_152_STATUS_B | | Section 16.4.7.1116 |
| 3988h | QUEUE_152_STATUS_C | | Section 16.4.7.1117 |
| 3990h | QUEUE_153_STATUS_A | | Section 16.4.7.1118 |
| 3994h | QUEUE_153_STATUS_B | | Section 16.4.7.1119 |
| 3998h | QUEUE_153_STATUS_C | | Section 16.4.7.1120 |
| 39A0h | QUEUE_154_STATUS_A | | Section 16.4.7.1121 |
| 39A4h | QUEUE_154_STATUS_B | | Section 16.4.7.1122 |
| 39A8h | QUEUE_154_STATUS_C | | Section 16.4.7.1123 |
| 39B0h | QUEUE_155_STATUS_A | | Section 16.4.7.1124 |
| 39B4h | QUEUE_155_STATUS_B | | Section 16.4.7.1125 |
| 39B8h | QUEUE_155_STATUS_C | | Section 16.4.7.1126 |

### 16.4.7.1 QMGRREVID Register (offset = 0h) [reset = 4E530800h]

QMGRREVID is shown in Figure 16-277 and described in Table 16-291.

**Figure 16-277. QMGRREVID Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| SCHEME | | Reserved | | FUNCTION | | | |
| R-0 | | | | R-0 | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| FUNCTION | | | | | | | |
| R-0 | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| REVRTL | | | | | REVMAJ | | |
| R-0 | | | | | R-0 | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| REVCUSTOM | | REVMIN | | | | | |
| R-0 | | R-0 | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-291. QMGRREVID Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-30 | SCHEME | R-0 | 0 | Scheme that this register is compliant with |
| 27-16 | FUNCTION | R-0 | 0 | Function |
| 15-11 | REVRTL | R-0 | 0 | RTL revision |
| 10-8 | REVMAJ | R-0 | 0 | Major revision |
| 7-6 | REVCUSTOM | R-0 | 0 | Custom revision |
| 5-0 | REVMIN | R-0 | 0 | Minor revision Queue Manager Revision Register |

### 16.4.7.2 QMGRRST Register (offset = 8h) [reset = 0h]

QMGRRST is shown in Figure 16-278 and described in Table 16-292.

**Figure 16-278. QMGRRST Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| HEAD_TAIL | Reserved | DEST_QNUM | | | | | |
| W-0 | | W-0 | | | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| DEST_QNUM | | | | | | | |
| W-0 | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| Reserved | | SOURCE_QNUM | | | | | |
| | | W-0 | | | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| SOURCE_QNUM | | | | | | | |
| W-0 | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-292. QMGRRST Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31 | HEAD_TAIL | W-0 | 0 | Indicates whether queue contents should be merged on to head or tail of destination queue.<br>Clear this field for head and set for tail. |
| 29-16 | DEST_QNUM | W-0 | 0 | Destination Queue Number |
| 13-0 | SOURCE_QNUM | W-0 | 0 | Source Queue Number Queue Manager Queue Diversion Register<br>Note : CBA write transactions to this register cause the QMGR to start processing an internal state machine.<br>This disables CBA read transactions.while it is busy processing the state machine.<br>Once the state machine is back in the idle state<br>CBA read transactions are available again. |

### 16.4.7.3 FDBSC0 Register (offset = 20h) [reset = 0h]

FDBSC0 is shown in Figure 16-279 and described in Table 16-293.

**Figure 16-279. FDBSC0 Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| FDBQ3_STARVE_CNT | | | | | | | |
| R-0 | | | | | | | |
| **23** | **22** | **21** | **20** | **19** | **18** | **17** | **16** |
| FDBQ2_STARVE_CNT | | | | | | | |
| R-0 | | | | | | | |
| **15** | **14** | **13** | **12** | **11** | **10** | **9** | **8** |
| FDBQ1_STARVE_CNT | | | | | | | |
| R-0 | | | | | | | |
| **7** | **6** | **5** | **4** | **3** | **2** | **1** | **0** |
| FDBQ0_STARVE_CNT | | | | | | | |
| R-0 | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-293. FDBSC0 Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-24 | FDBQ3_STARVE_CNT | R-0 | 0 | This field increments each time the Free Descriptor/Buffer Queue 3 is read while it is empty via the CPPI DMA.<br>This field is cleared when read via the cpu. |
| 23-16 | FDBQ2_STARVE_CNT | R-0 | 0 | This field increments each time the Free Descriptor/Buffer Queue 2 is read while it is empty via the CPPI DMA.<br>This field is cleared when read via the cpu. |
| 15-8 | FDBQ1_STARVE_CNT | R-0 | 0 | This field increments each time the Free Descriptor/Buffer Queue 1 is read while it is empty via the CPPI DMA.<br>This field is cleared when read via the cpu. |
| 7-0 | FDBQ0_STARVE_CNT | R-0 | 0 | This field increments each time the Free Descriptor/Buffer Queue 0 is read while it is empty via the CPPI DMA.<br>This field is cleared when read via the cpu.<br>Queue_Manager_Free_Descriptor_Buffer_Starvation_Count Register 0 |

### 16.4.7.4 FDBSC1 Register (offset = 24h) [reset = 0h]

FDBSC1 is shown in Figure 16-280 and described in Table 16-294.

**Figure 16-280. FDBSC1 Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| FDBQ7_STARVE_CNT | | | | | | | |
| R-0 | | | | | | | |
| **23** | **22** | **21** | **20** | **19** | **18** | **17** | **16** |
| FDBQ6_STARVE_CNT | | | | | | | |
| R-0 | | | | | | | |
| **15** | **14** | **13** | **12** | **11** | **10** | **9** | **8** |
| FDBQ5_STARVE_CNT | | | | | | | |
| R-0 | | | | | | | |
| **7** | **6** | **5** | **4** | **3** | **2** | **1** | **0** |
| FDBQ4_STARVE_CNT | | | | | | | |
| R-0 | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-294. FDBSC1 Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-24 | FDBQ7_STARVE_CNT | R-0 | 0 | This field increments each time the Free Descriptor/Buffer Queue 7 is read while it is empty via the CPPI DMA.<br>This field is cleared when read via the cpu. |
| 23-16 | FDBQ6_STARVE_CNT | R-0 | 0 | This field increments each time the Free Descriptor/Buffer Queue 6 is read while it is empty via the CPPI DMA.<br>This field is cleared when read via the cpu. |
| 15-8 | FDBQ5_STARVE_CNT | R-0 | 0 | This field increments each time the Free Descriptor/Buffer Queue 5 is read while it is empty via the CPPI DMA.<br>This field is cleared when read via the cpu. |
| 7-0 | FDBQ4_STARVE_CNT | R-0 | 0 | This field increments each time the Free Descriptor/Buffer Queue 4 is read while it is empty via the CPPI DMA.<br>This field is cleared when read via the cpu.<br>Queue_Manager_Free_Descriptor_Buffer_Starvation_Count Register 1 |

### 16.4.7.5 FDBSC2 Register (offset = 28h) [reset = 0h]

FDBSC2 is shown in Figure 16-281 and described in Table 16-295.

**Figure 16-281. FDBSC2 Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| FDBQ11_STARVE_CNT | | | | | | | |
| R-0 | | | | | | | |
| **23** | **22** | **21** | **20** | **19** | **18** | **17** | **16** |
| FDBQ10_STARVE_CNT | | | | | | | |
| R-0 | | | | | | | |
| **15** | **14** | **13** | **12** | **11** | **10** | **9** | **8** |
| FDBQ9_STARVE_CNT | | | | | | | |
| R-0 | | | | | | | |
| **7** | **6** | **5** | **4** | **3** | **2** | **1** | **0** |
| FDBQ8_STARVE_CNT | | | | | | | |
| R-0 | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-295. FDBSC2 Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-24 | FDBQ11_STARVE_CNT | R-0 | 0 | This field increments each time the Free Descriptor/Buffer Queue 11 is read while it is empty via the CPPI DMA.<br>This field is cleared when read via the cpu. |
| 23-16 | FDBQ10_STARVE_CNT | R-0 | 0 | This field increments each time the Free Descriptor/Buffer Queue 10 is read while it is empty via the CPPI DMA.<br>This field is cleared when read via the cpu. |
| 15-8 | FDBQ9_STARVE_CNT | R-0 | 0 | This field increments each time the Free Descriptor/Buffer Queue 9 is read while it is empty via the CPPI DMA.<br>This field is cleared when read via the cpu. |
| 7-0 | FDBQ8_STARVE_CNT | R-0 | 0 | This field increments each time the Free Descriptor/Buffer Queue 8 is read while it is empty via the CPPI DMA.<br>This field is cleared when read via the cpu.<br>Queue_Manager_Free_Descriptor_Buffer_Starvation_Count Register 2 |

### 16.4.7.6 FDBSC3 Register (offset = 2Ch) [reset = 0h]

FDBSC3 is shown in Figure 16-282 and described in Table 16-296.

**Figure 16-282. FDBSC3 Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| FDBQ15_STARVE_CNT | | | | | | | |
| R-0 | | | | | | | |
| **23** | **22** | **21** | **20** | **19** | **18** | **17** | **16** |
| FDBQ14_STARVE_CNT | | | | | | | |
| R-0 | | | | | | | |
| **15** | **14** | **13** | **12** | **11** | **10** | **9** | **8** |
| FDBQ13_STARVE_CNT | | | | | | | |
| R-0 | | | | | | | |
| **7** | **6** | **5** | **4** | **3** | **2** | **1** | **0** |
| FDBQ12_STARVE_CNT | | | | | | | |
| R-0 | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-296. FDBSC3 Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-24 | FDBQ15_STARVE_CNT | R-0 | 0 | This field increments each time the Free Descriptor/Buffer Queue 15 is read while it is empty via the CPPI DMA.<br>This field is cleared when read via the cpu. |
| 23-16 | FDBQ14_STARVE_CNT | R-0 | 0 | This field increments each time the Free Descriptor/Buffer Queue 14 is read while it is empty via the CPPI DMA.<br>This field is cleared when read via the cpu. |
| 15-8 | FDBQ13_STARVE_CNT | R-0 | 0 | This field increments each time the Free Descriptor/Buffer Queue 13 is read while it is empty via the CPPI DMA.<br>This field is cleared when read via the cpu. |
| 7-0 | FDBQ12_STARVE_CNT | R-0 | 0 | This field increments each time the Free Descriptor/Buffer Queue 12 is read while it is empty via the CPPI DMA.<br>This field is cleared when read via the cpu.<br>Table<br>109 - QMGR_Free_Descriptor_Buffer_Starvation_Count Register 3 |

### 16.4.7.7 FDBSC4 Register (offset = 30h) [reset = 0h]

FDBSC4 is shown in Figure 16-283 and described in Table 16-297.

**Figure 16-283. FDBSC4 Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| FDBQ19_STARVE_CNT | | | | | | | |
| R-0 | | | | | | | |
| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
| FDBQ18_STARVE_CNT | | | | | | | |
| R-0 | | | | | | | |
| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
| FDBQ17_STARVE_CNT | | | | | | | |
| R-0 | | | | | | | |
| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
| FDBQ16_STARVE_CNT | | | | | | | |
| R-0 | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-297. FDBSC4 Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-24 | FDBQ19_STARVE_CNT | R-0 | 0 | This field increments each time the Free Descriptor/Buffer Queue 19 is read while it is empty via the CPPI DMA.<br>This field is cleared when read via the cpu. |
| 23-16 | FDBQ18_STARVE_CNT | R-0 | 0 | This field increments each time the Free Descriptor/Buffer Queue 18 is read while it is empty via the CPPI DMA.<br>This field is cleared when read via the cpu. |
| 15-8 | FDBQ17_STARVE_CNT | R-0 | 0 | This field increments each time the Free Descriptor/Buffer Queue 17 is read while it is empty via the CPPI DMA.<br>This field is cleared when read via the cpu. |
| 7-0 | FDBQ16_STARVE_CNT | R-0 | 0 | This field increments each time the Free Descriptor/Buffer Queue 16 is read while it is empty via the CPPI DMA.<br>This field is cleared when read via the cpu.<br>Queue_Manager_Free_Descriptor_Buffer_Starvation_Count Register 4 |

### 16.4.7.8 FDBSC5 Register (offset = 34h) [reset = 0h]

FDBSC5 is shown in Figure 16-284 and described in Table 16-298.

**Figure 16-284. FDBSC5 Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| FDBQ23_STARVE_CNT | | | | | | | |
| R-0 | | | | | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| FDBQ22_STARVE_CNT | | | | | | | |
| R-0 | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| FDBQ21_STARVE_CNT | | | | | | | |
| R-0 | | | | | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| FDBQ20_STARVE_CNT | | | | | | | |
| R-0 | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-298. FDBSC5 Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-24 | FDBQ23_STARVE_CNT | R-0 | 0 | This field increments each time the Free Descriptor/Buffer Queue 23 is read while it is empty via the CPPI DMA.<br>This field is cleared when read via the cpu. |
| 23-16 | FDBQ22_STARVE_CNT | R-0 | 0 | This field increments each time the Free Descriptor/Buffer Queue 22 is read while it is empty via the CPPI DMA.<br>This field is cleared when read via the cpu. |
| 15-8 | FDBQ21_STARVE_CNT | R-0 | 0 | This field increments each time the Free Descriptor/Buffer Queue 21 is read while it is empty via the CPPI DMA.<br>This field is cleared when read via the cpu. |
| 7-0 | FDBQ20_STARVE_CNT | R-0 | 0 | This field increments each time the Free Descriptor/Buffer Queue 20 is read while it is empty via the CPPI DMA.<br>This field is cleared when read via the cpu.<br>Queue_Manager_Free_Descriptor_Buffer_Starvation_Count Register 5 |

### 16.4.7.9 FDBSC6 Register (offset = 38h) [reset = 0h]

FDBSC6 is shown in Figure 16-285 and described in Table 16-299.

**Figure 16-285. FDBSC6 Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| FDBQ27_STARVE_CNT | | | | | | | |
| R-0 | | | | | | | |
| **23** | **22** | **21** | **20** | **19** | **18** | **17** | **16** |
| FDBQ26_STARVE_CNT | | | | | | | |
| R-0 | | | | | | | |
| **15** | **14** | **13** | **12** | **11** | **10** | **9** | **8** |
| FDBQ25_STARVE_CNT | | | | | | | |
| R-0 | | | | | | | |
| **7** | **6** | **5** | **4** | **3** | **2** | **1** | **0** |
| FDBQ24_STARVE_CNT | | | | | | | |
| R-0 | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-299. FDBSC6 Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-24 | FDBQ27_STARVE_CNT | R-0 | 0 | This field increments each time the Free Descriptor/Buffer Queue 27 is read while it is empty via the CPPI DMA.<br>This field is cleared when read via the cpu. |
| 23-16 | FDBQ26_STARVE_CNT | R-0 | 0 | This field increments each time the Free Descriptor/Buffer Queue 26 is read while it is empty via the CPPI DMA.<br>This field is cleared when read via the cpu. |
| 15-8 | FDBQ25_STARVE_CNT | R-0 | 0 | This field increments each time the Free Descriptor/Buffer Queue 25 is read while it is empty via the CPPI DMA.<br>This field is cleared when read via the cpu. |
| 7-0 | FDBQ24_STARVE_CNT | R-0 | 0 | This field increments each time the Free Descriptor/Buffer Queue 24 is read while it is empty via the CPPI DMA.<br>This field is cleared when read via the cpu.<br>Queue_Manager_Free_Descriptor_Buffer_Starvation_Count Register 6 |

### 16.4.7.10 FDBSC7 Register (offset = 3Ch) [reset = 0h]

FDBSC7 is shown in Figure 16-286 and described in Table 16-300.

**Figure 16-286. FDBSC7 Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| FDBQ31_STARVE_CNT | | | | | | | |
| R-0 | | | | | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| FDBQ30_STARVE_CNT | | | | | | | |
| R-0 | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| FDBQ29_STARVE_CNT | | | | | | | |
| R-0 | | | | | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| FDBQ28_STARVE_CNT | | | | | | | |
| R-0 | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-300. FDBSC7 Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-24 | FDBQ31_STARVE_CNT | R-0 | 0 | This field increments each time the Free Descriptor/Buffer Queue 31 is read while it is empty via the CPPI DMA.<br>This field is cleared when read via the cpu. |
| 23-16 | FDBQ30_STARVE_CNT | R-0 | 0 | This field increments each time the Free Descriptor/Buffer Queue 30 is read while it is empty via the CPPI DMA.<br>This field is cleared when read via the cpu. |
| 15-8 | FDBQ29_STARVE_CNT | R-0 | 0 | This field increments each time the Free Descriptor/Buffer Queue 29 is read while it is empty via the CPPI DMA.<br>This field is cleared when read via the cpu. |
| 7-0 | FDBQ28_STARVE_CNT | R-0 | 0 | This field increments each time the Free Descriptor/Buffer Queue 28 is read while it is empty via the CPPI DMA.<br>This field is cleared when read via the cpu.<br>Queue_Manager_Free_Descriptor_Buffer_Starvation_Count Register 7 |

### 16.4.7.11 LRAM0BASE Register (offset = 80h) [reset = 0h]

LRAM0BASE is shown in Figure 16-287 and described in Table 16-301.

**Figure 16-287. LRAM0BASE Register**

| 31–2 | 1–0 |
|---|---|
| REGION0_BASE | Reserved |
| R/W-0 | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-301. LRAM0BASE Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-2 | REGION0_BASE | R/W-0 | 0 | This field stores the base address for the first region of the linking RAM.<br>This may be anywhere in<br>32-bit address space but would be typically located in on-chip memory. |

### 16.4.7.12 LRAM0SIZE Register (offset = 84h) [reset = 0h]

LRAM0SIZE is shown in Figure 16-288 and described in Table 16-302.

**Figure 16-288. LRAM0SIZE Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | REGION0_SIZE |
| | R/W-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-302. LRAM0SIZE Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | REGION0_SIZE | R/W-0 | 0 | This field indicates the number of entries that are contained in the linking RAM region 0.<br>A descriptor with index less than region0_size value has its linking location in region 0.<br>A descriptor with index greater than region0_size has its linking location in region 1.<br>The queue manager will add the index (left shifted by 2 bits) to the appropriate regionX_base_addr to get the absolute 32-bit address to the linking location for a descriptor.<br>Queue Manager Linking Ram Region 0 Size Register |

### 16.4.7.13 LRAM1BASE Register (offset = 88h) [reset = 0h]

LRAM1BASE is shown in Figure 16-289 and described in Table 16-303.

**Figure 16-289. LRAM1BASE Register**

| 31–2 | 1–0 |
|---|---|
| REGION1_BASE | Reserved |
| R/W-0 | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-303. LRAM1BASE Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-2 | REGION1_BASE | R/W-0 | 0 | This field stores the base address for the second region of the linking RAM.<br>This may be anywhere in<br>32- bit address space but would be typically located in off- chip memory. |

### 16.4.7.14 PEND0 Register (offset = 90h) [reset = 0h]

PEND0 is shown in Figure 16-290 and described in Table 16-304.

**Figure 16-290. PEND0 Register**

| 31–0 |
|---|
| QPEND0 |
| R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-304. PEND0 Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-0 | QPEND0 | R-0 | 0 | This field indicates the queue pending status for queues[ 31:0]. |

### 16.4.7.15 PEND1 Register (offset = 94h) [reset = 0h]

PEND1 is shown in Figure 16-291 and described in Table 16-305.

**Figure 16-291. PEND1 Register**

| 31-0 |
|---|
| QPEND1 |
| R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-305. PEND1 Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-0 | QPEND1 | R-0 | 0 | This field indicates the queue pending status for queues[ 63:32].<br>Table<br>118 - QMGR_Queue_Pending_1 Register 1 |

### 16.4.7.16 PEND2 Register (offset = 98h) [reset = 0h]

PEND2 is shown in Figure 16-292 and described in Table 16-306.

**Figure 16-292. PEND2 Register**

| 31–0 |
|---|
| QPEND2 |
| R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-306. PEND2 Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-0 | QPEND2 | R-0 | 0 | This field indicates the queue pending status for queues[ 95:64].<br>Queue_Manager_Queue_Pending_2 Register 2 |

### 16.4.7.17 PEND3 Register (offset = 9Ch) [reset = 0h]

PEND3 is shown in Figure 16-293 and described in Table 16-307.

**Figure 16-293. PEND3 Register**

| 31–0 |
|---|
| QPEND3 |
| R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-307. PEND3 Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-0 | QPEND3 | R-0 | 0 | This field indicates the queue pending status for queues[ 127:96].<br>Queue_Manager_Queue_Pending_3 Register 3 |

### 16.4.7.18 PEND4 Register (offset = A0h) [reset = 0h]

PEND4 is shown in Figure 16-294 and described in Table 16-308.

**Figure 16-294. PEND4 Register**

| 31–0 |
| --- |
| QPEND4 |
| R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-308. PEND4 Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
| --- | --- | --- | --- | --- |
| 31-0 | QPEND4 | R-0 | 0 | This field indicates the queue pending status for queues[ 159:128]. Queue_Manager_Queue_Pending_4 Register 4 |

### 16.4.7.19 QMEMRBASE0 Register (offset = 1000h) [reset = 0h]

QMEMRBASE0 is shown in Figure 16-295 and described in Table 16-309.

**Figure 16-295. QMEMRBASE0 Register**

| 31-5 | 4-0 |
|---|---|
| REG | Reserved |
| R/W-0 | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-309. QMEMRBASE0 Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-5 | REG | R/W-0 | 0 | This field contains the base address of the memory region R. |

### 16.4.7.20 QMEMCTRL0 Register (offset = 1004h) [reset = 0h]

QMEMCTRL0 is shown in Figure 16-296 and described in Table 16-310.

**Figure 16-296. QMEMCTRL0 Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| Reserved | | START_INDEX | | | | | |
| | | R/W-0 | | | | | |
| **23** | **22** | **21** | **20** | **19** | **18** | **17** | **16** |
| START_INDEX | | | | | | | |
| R/W-0 | | | | | | | |
| **15** | **14** | **13** | **12** | **11** | **10** | **9** | **8** |
| Reserved | | | | DESC_SIZE | | | |
| | | | | R/W-0 | | | |
| **7** | **6** | **5** | **4** | **3** | **2** | **1** | **0** |
| Reserved | | | | | REG_SIZE | | |
| | | | | | R/W-0 | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-310. QMEMCTRL0 Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 29-16 | START_INDEX | R/W-0 | 0 | This field indicates where in linking RAM does the descriptor linking information corresponding to memory region R starts. |
| 11-8 | DESC_SIZE | R/W-0 | 0 | This field indicates the size of each descriptor in this memory region.<br>It is an encoded value that specifies descriptor size as 2^(5+desc_size) number of bytes.<br>The settings of desc_size from<br>9-15 are reserved. |
| 2-0 | REG_SIZE | R/W-0 | 0 | This field indicates the size of the memory region (in terms of number of descriptors).<br>It is an encoded value that specifies region size as 2^(5+reg_size) number of descriptors.<br>Queue Manager Memory Region R Control Registers The following sections describe each of the four register locations that may be present for each queue in the queues region.<br>For reasons of implementation and area efficiency, these registers are not actually implemented as a huge array of flip flops but are instead implemented as a single set of mailbox registers which use the LSBs of the provided address as a queue index.<br>Due to this implementation all accesses to these registers need to be performed as a single burst write for each packet push operation or a single burst read for each packet pop operation.<br>The length of a burst to push or pop a packet will vary depending on the optional features that the queue supports which may be 4, 8, 12 or 16 bytes.<br>Queue N Register D must always be written / read in the burst but the preceding words are optional depending on the required queue functionality. |

### 16.4.7.21 QMEMRBASE1 Register (offset = 1010h) [reset = 0h]

QMEMRBASE1 is shown in Figure 16-297 and described in Table 16-311.

**Figure 16-297. QMEMRBASE1 Register**

| 31–5 | 4–0 |
|---|---|
| REG | Reserved |
| R/W-0 | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-311. QMEMRBASE1 Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-5 | REG | R/W-0 | 0 | This field contains the base address of the memory region R. |

### 16.4.7.22 QMEMCTRL1 Register (offset = 1014h) [reset = 0h]

QMEMCTRL1 is shown in Figure 16-298 and described in Table 16-312.

**Figure 16-298. QMEMCTRL1 Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| Reserved | | START_INDEX | | | | | |
| | | R/W-0 | | | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| START_INDEX | | | | | | | |
| R/W-0 | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| Reserved | | | | DESC_SIZE | | | |
| | | | | R/W-0 | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| Reserved | | | | | REG_SIZE | | |
| | | | | | R/W-0 | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-312. QMEMCTRL1 Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 29-16 | START_INDEX | R/W-0 | 0 | This field indicates where in linking RAM does the descriptor linking information corresponding to memory region R starts. |
| 11-8 | DESC_SIZE | R/W-0 | 0 | This field indicates the size of each descriptor in this memory region.<br>It is an encoded value that specifies descriptor size as 2^(5+desc_size) number of bytes.<br>The settings of desc_size from<br>9-15 are reserved. |
| 2-0 | REG_SIZE | R/W-0 | 0 | This field indicates the size of the memory region (in terms of number of descriptors).<br>It is an encoded value that specifies region size as 2^(5+reg_size) number of descriptors.<br>Queue Manager Memory Region R Control Registers The following sections describe each of the four register locations that may be present for each queue in the queues region.<br>For reasons of implementation and area efficiency, these registers are not actually implemented as a huge array of flip flops but are instead implemented as a single set of mailbox registers which use the LSBs of the provided address as a queue index.<br>Due to this implementation all accesses to these registers need to be performed as a single burst write for each packet push operation or a single burst read for each packet pop operation.<br>The length of a burst to push or pop a packet will vary depending on the optional features that the queue supports which may be 4, 8, 12 or 16 bytes.<br>Queue N Register D must always be written / read in the burst but the preceding words are optional depending on the required queue functionality. |

### 16.4.7.23 QMEMRBASE2 Register (offset = 1020h) [reset = 0h]

QMEMRBASE2 is shown in Figure 16-299 and described in Table 16-313.

**Figure 16-299. QMEMRBASE2 Register**

| 31–5 | 4–0 |
|---|---|
| REG | Reserved |
| R/W-0 | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-313. QMEMRBASE2 Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-5 | REG | R/W-0 | 0 | This field contains the base address of the memory region R. |

### 16.4.7.24 QMEMCTRL2 Register (offset = 1024h) [reset = 0h]

QMEMCTRL2 is shown in Figure 16-300 and described in Table 16-314.

**Figure 16-300. QMEMCTRL2 Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| Reserved | | START_INDEX | | | | | |
| | | R/W-0 | | | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| START_INDEX | | | | | | | |
| R/W-0 | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| Reserved | | | | DESC_SIZE | | | |
| | | | | R/W-0 | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| Reserved | | | | | REG_SIZE | | |
| | | | | | R/W-0 | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-314. QMEMCTRL2 Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 29-16 | START_INDEX | R/W-0 | 0 | This field indicates where in linking RAM does the descriptor linking information corresponding to memory region R starts. |
| 11-8 | DESC_SIZE | R/W-0 | 0 | This field indicates the size of each descriptor in this memory region.<br>It is an encoded value that specifies descriptor size as 2^(5+desc_size) number of bytes.<br>The settings of desc_size from<br>9-15 are reserved. |
| 2-0 | REG_SIZE | R/W-0 | 0 | This field indicates the size of the memory region (in terms of number of descriptors).<br>It is an encoded value that specifies region size as 2^(5+reg_size) number of descriptors.<br>Queue Manager Memory Region R Control Registers The following sections describe each of the four register locations that may be present for each queue in the queues region.<br>For reasons of implementation and area efficiency, these registers are not actually implemented as a huge array of flip flops but are instead implemented as a single set of mailbox registers which use the LSBs of the provided address as a queue index.<br>Due to this implementation all accesses to these registers need to be performed as a single burst write for each packet push operation or a single burst read for each packet pop operation.<br>The length of a burst to push or pop a packet will vary depending on the optional features that the queue supports which may be 4, 8, 12 or 16 bytes.<br>Queue N Register D must always be written / read in the burst but the preceding words are optional depending on the required queue functionality. |

### 16.4.7.25 QMEMRBASE3 Register (offset = 1030h) [reset = 0h]

QMEMRBASE3 is shown in Figure 16-301 and described in Table 16-315.

**Figure 16-301. QMEMRBASE3 Register**

| 31–5 | 4–0 |
|---|---|
| REG | Reserved |
| R/W-0 | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-315. QMEMRBASE3 Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-5 | REG | R/W-0 | 0 | This field contains the base address of the memory region R. |

### 16.4.7.26 QMEMCTRL3 Register (offset = 1034h) [reset = 0h]

QMEMCTRL3 is shown in Figure 16-302 and described in Table 16-316.

**Figure 16-302. QMEMCTRL3 Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| Reserved | | START_INDEX | | | | | |
| | | R/W-0 | | | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| START_INDEX | | | | | | | |
| R/W-0 | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| Reserved | | | | DESC_SIZE | | | |
| | | | | R/W-0 | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| Reserved | | | | | REG_SIZE | | |
| | | | | | R/W-0 | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-316. QMEMCTRL3 Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 29-16 | START_INDEX | R/W-0 | 0 | This field indicates where in linking RAM does the descriptor linking information corresponding to memory region R starts. |
| 11-8 | DESC_SIZE | R/W-0 | 0 | This field indicates the size of each descriptor in this memory region.<br>It is an encoded value that specifies descriptor size as 2^(5+desc_size) number of bytes.<br>The settings of desc_size from<br>9-15 are reserved. |
| 2-0 | REG_SIZE | R/W-0 | 0 | This field indicates the size of the memory region (in terms of number of descriptors).<br>It is an encoded value that specifies region size as 2^(5+reg_size) number of descriptors.<br>Queue Manager Memory Region R Control Registers The following sections describe each of the four register locations that may be present for each queue in the queues region.<br>For reasons of implementation and area efficiency, these registers are not actually implemented as a huge array of flip flops but are instead implemented as a single set of mailbox registers which use the LSBs of the provided address as a queue index.<br>Due to this implementation all accesses to these registers need to be performed as a single burst write for each packet push operation or a single burst read for each packet pop operation.<br>The length of a burst to push or pop a packet will vary depending on the optional features that the queue supports which may be 4, 8, 12 or 16 bytes.<br>Queue N Register D must always be written / read in the burst but the preceding words are optional depending on the required queue functionality. |

### 16.4.7.27 QMEMRBASE4 Register (offset = 1040h) [reset = 0h]

QMEMRBASE4 is shown in Figure 16-303 and described in Table 16-317.

**Figure 16-303. QMEMRBASE4 Register**

| 31–5 | 4–0 |
|---|---|
| REG | Reserved |
| R/W-0 | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-317. QMEMRBASE4 Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-5 | REG | R/W-0 | 0 | This field contains the base address of the memory region R. |

### 16.4.7.28 QMEMCTRL4 Register (offset = 1044h) [reset = 0h]

QMEMCTRL4 is shown in Figure 16-304 and described in Table 16-318.

**Figure 16-304. QMEMCTRL4 Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| Reserved | | START_INDEX | | | | | |
| | | R/W-0 | | | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| START_INDEX | | | | | | | |
| R/W-0 | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| Reserved | | | | DESC_SIZE | | | |
| | | | | R/W-0 | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| Reserved | | | | | REG_SIZE | | |
| | | | | | R/W-0 | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-318. QMEMCTRL4 Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 29-16 | START_INDEX | R/W-0 | 0 | This field indicates where in linking RAM does the descriptor linking information corresponding to memory region R starts. |
| 11-8 | DESC_SIZE | R/W-0 | 0 | This field indicates the size of each descriptor in this memory region.<br>It is an encoded value that specifies descriptor size as 2^(5+desc_size) number of bytes.<br>The settings of desc_size from<br>9-15 are reserved. |
| 2-0 | REG_SIZE | R/W-0 | 0 | This field indicates the size of the memory region (in terms of number of descriptors).<br>It is an encoded value that specifies region size as 2^(5+reg_size) number of descriptors.<br>Queue Manager Memory Region R Control Registers The following sections describe each of the four register locations that may be present for each queue in the queues region.<br>For reasons of implementation and area efficiency, these registers are not actually implemented as a huge array of flip flops but are instead implemented as a single set of mailbox registers which use the LSBs of the provided address as a queue index.<br>Due to this implementation all accesses to these registers need to be performed as a single burst write for each packet push operation or a single burst read for each packet pop operation.<br>The length of a burst to push or pop a packet will vary depending on the optional features that the queue supports which may be 4, 8, 12 or 16 bytes.<br>Queue N Register D must always be written / read in the burst but the preceding words are optional depending on the required queue functionality. |

### 16.4.7.29 QMEMRBASE5 Register (offset = 1050h) [reset = 0h]

QMEMRBASE5 is shown in Figure 16-305 and described in Table 16-319.

**Figure 16-305. QMEMRBASE5 Register**

| 31–5 | 4–0 |
|---|---|
| REG | Reserved |
| R/W-0 | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-319. QMEMRBASE5 Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-5 | REG | R/W-0 | 0 | This field contains the base address of the memory region R. |

### 16.4.7.30 QMEMCTRL5 Register (offset = 1054h) [reset = 0h]

QMEMCTRL5 is shown in Figure 16-306 and described in Table 16-320.

**Figure 16-306. QMEMCTRL5 Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| Reserved | | START_INDEX | | | | | |
| | | R/W-0 | | | | | |
| **23** | **22** | **21** | **20** | **19** | **18** | **17** | **16** |
| START_INDEX | | | | | | | |
| R/W-0 | | | | | | | |
| **15** | **14** | **13** | **12** | **11** | **10** | **9** | **8** |
| Reserved | | | | DESC_SIZE | | | |
| | | | | R/W-0 | | | |
| **7** | **6** | **5** | **4** | **3** | **2** | **1** | **0** |
| Reserved | | | | | REG_SIZE | | |
| | | | | | R/W-0 | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-320. QMEMCTRL5 Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 29-16 | START_INDEX | R/W-0 | 0 | This field indicates where in linking RAM does the descriptor linking information corresponding to memory region R starts. |
| 11-8 | DESC_SIZE | R/W-0 | 0 | This field indicates the size of each descriptor in this memory region.<br>It is an encoded value that specifies descriptor size as 2^(5+desc_size) number of bytes.<br>The settings of desc_size from 9-15 are reserved. |
| 2-0 | REG_SIZE | R/W-0 | 0 | This field indicates the size of the memory region (in terms of number of descriptors).<br>It is an encoded value that specifies region size as 2^(5+reg_size) number of descriptors.<br>Queue Manager Memory Region R Control Registers The following sections describe each of the four register locations that may be present for each queue in the queues region.<br>For reasons of implementation and area efficiency, these registers are not actually implemented as a huge array of flip flops but are instead implemented as a single set of mailbox registers which use the LSBs of the provided address as a queue index.<br>Due to this implementation all accesses to these registers need to be performed as a single burst write for each packet push operation or a single burst read for each packet pop operation.<br>The length of a burst to push or pop a packet will vary depending on the optional features that the queue supports which may be 4, 8, 12 or 16 bytes.<br>Queue N Register D must always be written / read in the burst but the preceding words are optional depending on the required queue functionality. |

### 16.4.7.31 QMEMRBASE6 Register (offset = 1060h) [reset = 0h]

QMEMRBASE6 is shown in Figure 16-307 and described in Table 16-321.

**Figure 16-307. QMEMRBASE6 Register**

| 31-5 | 4-0 |
|---|---|
| REG | Reserved |
| R/W-0 | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-321. QMEMRBASE6 Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-5 | REG | R/W-0 | 0 | This field contains the base address of the memory region R. |

### 16.4.7.32 QMEMCTRL6 Register (offset = 1064h) [reset = 0h]

QMEMCTRL6 is shown in Figure 16-308 and described in Table 16-322.

**Figure 16-308. QMEMCTRL6 Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| Reserved | | START_INDEX | | | | | |
| | | R/W-0 | | | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| START_INDEX | | | | | | | |
| R/W-0 | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| Reserved | | | | DESC_SIZE | | | |
| | | | | R/W-0 | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| Reserved | | | | | REG_SIZE | | |
| | | | | | R/W-0 | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-322. QMEMCTRL6 Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 29-16 | START_INDEX | R/W-0 | 0 | This field indicates where in linking RAM does the descriptor linking information corresponding to memory region R starts. |
| 11-8 | DESC_SIZE | R/W-0 | 0 | This field indicates the size of each descriptor in this memory region.<br>It is an encoded value that specifies descriptor size as 2^(5+desc_size) number of bytes.<br>The settings of desc_size from<br>9-15 are reserved. |
| 2-0 | REG_SIZE | R/W-0 | 0 | This field indicates the size of the memory region (in terms of number of descriptors).<br>It is an encoded value that specifies region size as 2^(5+reg_size) number of descriptors.<br>Queue Manager Memory Region R Control Registers The following sections describe each of the four register locations that may be present for each queue in the queues region.<br>For reasons of implementation and area efficiency, these registers are not actually implemented as a huge array of flip flops but are instead implemented as a single set of mailbox registers which use the LSBs of the provided address as a queue index.<br>Due to this implementation all accesses to these registers need to be performed as a single burst write for each packet push operation or a single burst read for each packet pop operation.<br>The length of a burst to push or pop a packet will vary depending on the optional features that the queue supports which may be 4, 8, 12 or 16 bytes.<br>Queue N Register D must always be written / read in the burst but the preceding words are optional depending on the required queue functionality. |

### 16.4.7.33 QMEMRBASE7 Register (offset = 1070h) [reset = 0h]

QMEMRBASE7 is shown in Figure 16-309 and described in Table 16-323.

**Figure 16-309. QMEMRBASE7 Register**

| 31–5 | 4–0 |
|---|---|
| REG | Reserved |
| R/W-0 | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-323. QMEMRBASE7 Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-5 | REG | R/W-0 | 0 | This field contains the base address of the memory region R. |

### 16.4.7.34 QMEMCTRL7 Register (offset = 1074h) [reset = 0h]

QMEMCTRL7 is shown in Figure 16-310 and described in Table 16-324.

**Figure 16-310. QMEMCTRL7 Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| Reserved | | START_INDEX | | | | | |
| | | R/W-0 | | | | | |
| **23** | **22** | **21** | **20** | **19** | **18** | **17** | **16** |
| START_INDEX | | | | | | | |
| R/W-0 | | | | | | | |
| **15** | **14** | **13** | **12** | **11** | **10** | **9** | **8** |
| Reserved | | | | DESC_SIZE | | | |
| | | | | R/W-0 | | | |
| **7** | **6** | **5** | **4** | **3** | **2** | **1** | **0** |
| Reserved | | | | | REG_SIZE | | |
| | | | | | R/W-0 | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-324. QMEMCTRL7 Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 29-16 | START_INDEX | R/W-0 | 0 | This field indicates where in linking RAM does the descriptor linking information corresponding to memory region R starts. |
| 11-8 | DESC_SIZE | R/W-0 | 0 | This field indicates the size of each descriptor in this memory region.<br>It is an encoded value that specifies descriptor size as 2^(5+desc_size) number of bytes.<br>The settings of desc_size from<br>9-15 are reserved. |
| 2-0 | REG_SIZE | R/W-0 | 0 | This field indicates the size of the memory region (in terms of number of descriptors).<br>It is an encoded value that specifies region size as 2^(5+reg_size) number of descriptors.<br>Queue Manager Memory Region R Control Registers The following sections describe each of the four register locations that may be present for each queue in the queues region.<br>For reasons of implementation and area efficiency, these registers are not actually implemented as a huge array of flip flops but are instead implemented as a single set of mailbox registers which use the LSBs of the provided address as a queue index.<br>Due to this implementation all accesses to these registers need to be performed as a single burst write for each packet push operation or a single burst read for each packet pop operation.<br>The length of a burst to push or pop a packet will vary depending on the optional features that the queue supports which may be 4, 8, 12 or 16 bytes.<br>Queue N Register D must always be written / read in the burst but the preceding words are optional depending on the required queue functionality. |

### 16.4.7.35 QUEUE_0_A Register (offset = 2000h) [reset = 0h]

QUEUE_0_A is shown in Figure 16-311 and described in Table 16-325.

**Figure 16-311. QUEUE_0_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-325. QUEUE_0_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>This count is incremented by 1 whenever a packet is added to the queue.<br>This count is decremented by 1 whenever a packet is popped from the queue. |

### 16.4.7.36 QUEUE_0_B Register (offset = 2004h) [reset = 0h]

QUEUE_0_B is shown in Figure 16-312 and described in Table 16-326.

**Figure 16-312. QUEUE_0_B Register**

| 31–28 | 27–0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-326. QUEUE_0_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. |

### 16.4.7.37 QUEUE_0_C Register (offset = 2008h) [reset = 0h]

QUEUE_0_C is shown in Figure 16-313 and described in Table 16-327.

**Figure 16-313. QUEUE_0_C Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| HEAD_TAIL | Reserved | | | | | | |
| W-0 | | | | | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| Reserved | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| Reserved | | PACKET_SIZE | | | | | |
| | | R/W-0 | | | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| PACKET_SIZE | | | | | | | |
| R/W-0 | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-327. QUEUE_0_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31 | HEAD_TAIL | W-0 | 0 | Head/Tail Push Control.<br>Set to zero in order to push packet onto tail of queue and set to one in order to push packet onto head of queue. |
| 13-0 | PACKET_SIZE | R/W-0 | 0 | packet_size This field indicates packet size and is assumed to be zero on each packet add unless the value is explicitly overwritten. This field indicates packet size for packet pop operation. |

#### 16.4.7.38 QUEUE_0_D Register (offset = 200Ch) [reset = 0h]

QUEUE_0_D is shown in Figure 16-314 and described in Table 16-328.

**Figure 16-314. QUEUE_0_D Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |
| **23** | **22** | **21** | **20** | **19** | **18** | **17** | **16** |
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |
| **15** | **14** | **13** | **12** | **11** | **10** | **9** | **8** |
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |
| **7** | **6** | **5** | **4** | **3** | **2** | **1** | **0** |
| DESC_PTR | | | DESC_SIZE | | | | |
| R/W-0 | | | R/W-0 | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-328. QUEUE_0_D Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-5 | DESC_PTR | R/W-0 | 0 | Descriptor pointer.<br>It will be read as zero if the queue is empty.<br>It will indicate a<br>32-bit aligned address that points to a descriptor when the queue is not empty. |
| 4-0 | DESC_SIZE | R/W-0 | 0 | Descriptor Size.<br>It is encoded in<br>4-byte increments with values 0 to 31 representing 24 and so on to 148 bytes.<br>This field will return a 0x0 when an empty queue is read.<br>Queue Manager Queue N Registers D To save hardware resources, the queue manager internally stores descriptor size (desc_size) information in four bits.<br>However, register D has five LSBs that specify descriptor size.<br>As a consequence, the value of desc_size that is pushed may not be same as that is read during a pop.<br>The value that is read back is equal to always rounded to an odd number.<br>So, for even values, the value read back is one more than what was written.<br>For odd values, the value read back is same as the value that was written.<br>Note that this<br>5-bit field (desc_size) is unrelated to the code for size of descriptors in a descriptor region.<br>It is just a place holder for a<br>5-bit value that is maintained across the push and pop operations for every descriptor managed by the queue manager. |

### 16.4.7.39 QUEUE_1_A Register (offset = 2010h) [reset = 0h]

QUEUE_1_A is shown in Figure 16-315 and described in Table 16-329.

**Figure 16-315. QUEUE_1_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-329. QUEUE_1_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>This count is incremented by 1 whenever a packet is added to the queue.<br>This count is decremented by 1 whenever a packet is popped from the queue. |

### 16.4.7.40 QUEUE_1_B Register (offset = 2014h) [reset = 0h]

QUEUE_1_B is shown in Figure 16-316 and described in Table 16-330.

**Figure 16-316. QUEUE_1_B Register**

| 31–28 | 27–0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-330. QUEUE_1_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. |

### 16.4.7.41 QUEUE_1_C Register (offset = 2018h) [reset = 0h]

QUEUE_1_C is shown in Figure 16-317 and described in Table 16-331.

**Figure 16-317. QUEUE_1_C Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| HEAD_TAIL | Reserved | | | | | | |
| W-0 | | | | | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| Reserved | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| Reserved | | PACKET_SIZE | | | | | |
| | | R/W-0 | | | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| PACKET_SIZE | | | | | | | |
| R/W-0 | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-331. QUEUE_1_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31 | HEAD_TAIL | W-0 | 0 | Head/Tail Push Control.<br>Set to zero in order to push packet onto tail of queue and set to one in order to push packet onto head of queue. |
| 13-0 | PACKET_SIZE | R/W-0 | 0 | packet_size This field indicates packet size and is assumed to be zero on each packet add unless the value is explicitly overwritten. This field indicates packet size for packet pop operation. |

### 16.4.7.42 QUEUE_1_D Register (offset = 201Ch) [reset = 0h]

QUEUE_1_D is shown in Figure 16-318 and described in Table 16-332.

**Figure 16-318. QUEUE_1_D Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |
| **23** | **22** | **21** | **20** | **19** | **18** | **17** | **16** |
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |
| **15** | **14** | **13** | **12** | **11** | **10** | **9** | **8** |
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |
| **7** | **6** | **5** | **4** | **3** | **2** | **1** | **0** |
| DESC_PTR | | | DESC_SIZE | | | | |
| R/W-0 | | | R/W-0 | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-332. QUEUE_1_D Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-5 | DESC_PTR | R/W-0 | 0 | Descriptor pointer.<br>It will be read as zero if the queue is empty.<br>It will indicate a<br>32-bit aligned address that points to a descriptor when the queue is not empty. |
| 4-0 | DESC_SIZE | R/W-0 | 0 | Descriptor Size.<br>It is encoded in<br>4-byte increments with values 0 to 31 representing 24 and so on to 148 bytes.<br>This field will return a 0x0 when an empty queue is read.<br>Queue Manager Queue N Registers D To save hardware resources, the queue manager internally stores descriptor size (desc_size) information in four bits.<br>However, register D has five LSBs that specify descriptor size.<br>As a consequence, the value of desc_size that is pushed may not be same as that is read during a pop.<br>The value that is read back is equal to always rounded to an odd number.<br>So, for even values, the value read back is one more than what was written.<br>For odd values, the value read back is same as the value that was written.<br>Note that this<br>5-bit field (desc_size) is unrelated to the code for size of descriptors in a descriptor region.<br>It is just a place holder for a<br>5-bit value that is maintained across the push and pop operations for every descriptor managed by the queue manager. |

### 16.4.7.43 QUEUE_2_A Register (offset = 2020h) [reset = 0h]

QUEUE_2_A is shown in Figure 16-319 and described in Table 16-333.

**Figure 16-319. QUEUE_2_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-333. QUEUE_2_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>This count is incremented by 1 whenever a packet is added to the queue.<br>This count is decremented by 1 whenever a packet is popped from the queue. |

### 16.4.7.44 QUEUE_2_B Register (offset = 2024h) [reset = 0h]

QUEUE_2_B is shown in Figure 16-320 and described in Table 16-334.

**Figure 16-320. QUEUE_2_B Register**

| 31–28 | 27–0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-334. QUEUE_2_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. |

### 16.4.7.45 QUEUE_2_C Register (offset = 2028h) [reset = 0h]

QUEUE_2_C is shown in Figure 16-321 and described in Table 16-335.

**Figure 16-321. QUEUE_2_C Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| HEAD_TAIL | Reserved | | | | | | |
| W-0 | | | | | | | |
| **23** | **22** | **21** | **20** | **19** | **18** | **17** | **16** |
| Reserved | | | | | | | |
| **15** | **14** | **13** | **12** | **11** | **10** | **9** | **8** |
| Reserved | | PACKET_SIZE | | | | | |
| | | R/W-0 | | | | | |
| **7** | **6** | **5** | **4** | **3** | **2** | **1** | **0** |
| PACKET_SIZE | | | | | | | |
| R/W-0 | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-335. QUEUE_2_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31 | HEAD_TAIL | W-0 | 0 | Head/Tail Push Control.<br>Set to zero in order to push packet onto tail of queue and set to one in order to push packet onto head of queue. |
| 13-0 | PACKET_SIZE | R/W-0 | 0 | packet_size This field indicates packet size and is assumed to be zero on each packet add unless the value is explicitly overwritten. This field indicates packet size for packet pop operation. |

### 16.4.7.46 QUEUE_2_D Register (offset = 202Ch) [reset = 0h]

QUEUE_2_D is shown in Figure 16-322 and described in Table 16-336.

**Figure 16-322. QUEUE_2_D Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | DESC_SIZE | | | | |
| R/W-0 | | | R/W-0 | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-336. QUEUE_2_D Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-5 | DESC_PTR | R/W-0 | 0 | Descriptor pointer.<br>It will be read as zero if the queue is empty.<br>It will indicate a<br>32-bit aligned address that points to a descriptor when the queue is not empty. |
| 4-0 | DESC_SIZE | R/W-0 | 0 | Descriptor Size.<br>It is encoded in<br>4-byte increments with values 0 to 31 representing 24 and so on to 148 bytes.<br>This field will return a 0x0 when an empty queue is read.<br>Queue Manager Queue N Registers D To save hardware resources, the queue manager internally stores descriptor size (desc_size) information in four bits.<br>However, register D has five LSBs that specify descriptor size.<br>As a consequence, the value of desc_size that is pushed may not be same as that is read during a pop.<br>The value that is read back is equal to always rounded to an odd number.<br>So, for even values, the value read back is one more than what was written.<br>For odd values, the value read back is same as the value that was written.<br>Note that this<br>5-bit field (desc_size) is unrelated to the code for size of descriptors in a descriptor region.<br>It is just a place holder for a<br>5-bit value that is maintained across the push and pop operations for every descriptor managed by the queue manager. |

### 16.4.7.47 QUEUE_3_A Register (offset = 2030h) [reset = 0h]

QUEUE_3_A is shown in Figure 16-323 and described in Table 16-337.

**Figure 16-323. QUEUE_3_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-337. QUEUE_3_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>This count is incremented by 1 whenever a packet is added to the queue.<br>This count is decremented by 1 whenever a packet is popped from the queue. |

### 16.4.7.48 QUEUE_3_B Register (offset = 2034h) [reset = 0h]

QUEUE_3_B is shown in Figure 16-324 and described in Table 16-338.

**Figure 16-324. QUEUE_3_B Register**

| 31–28 | 27–0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-338. QUEUE_3_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. |

### 16.4.7.49 QUEUE_3_C Register (offset = 2038h) [reset = 0h]

QUEUE_3_C is shown in Figure 16-325 and described in Table 16-339.

**Figure 16-325. QUEUE_3_C Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| HEAD_TAIL | Reserved | | | | | | |
| W-0 | | | | | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| Reserved | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| Reserved | | PACKET_SIZE | | | | | |
| | | R/W-0 | | | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| PACKET_SIZE | | | | | | | |
| R/W-0 | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-339. QUEUE_3_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31 | HEAD_TAIL | W-0 | 0 | Head/Tail Push Control.<br>Set to zero in order to push packet onto tail of queue and set to one in order to push packet onto head of queue. |
| 13-0 | PACKET_SIZE | R/W-0 | 0 | packet_size This field indicates packet size and is assumed to be zero on each packet add unless the value is explicitly overwritten. This field indicates packet size for packet pop operation. |

### 16.4.7.50 QUEUE_3_D Register (offset = 203Ch) [reset = 0h]

QUEUE_3_D is shown in Figure 16-326 and described in Table 16-340.

**Figure 16-326. QUEUE_3_D Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | DESC_SIZE | | | | |
| R/W-0 | | | R/W-0 | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-340. QUEUE_3_D Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-5 | DESC_PTR | R/W-0 | 0 | Descriptor pointer.<br>It will be read as zero if the queue is empty.<br>It will indicate a<br>32-bit aligned address that points to a descriptor when the queue is not empty. |
| 4-0 | DESC_SIZE | R/W-0 | 0 | Descriptor Size.<br>It is encoded in<br>4-byte increments with values 0 to 31 representing 24 and so on to 148 bytes.<br>This field will return a 0x0 when an empty queue is read.<br>Queue Manager Queue N Registers D To save hardware resources, the queue manager internally stores descriptor size (desc_size) information in four bits.<br>However, register D has five LSBs that specify descriptor size.<br>As a consequence, the value of desc_size that is pushed may not be same as that is read during a pop.<br>The value that is read back is equal to always rounded to an odd number.<br>So, for even values, the value read back is one more than what was written.<br>For odd values, the value read back is same as the value that was written.<br>Note that this<br>5-bit field (desc_size) is unrelated to the code for size of descriptors in a descriptor region.<br>It is just a place holder for a<br>5-bit value that is maintained across the push and pop operations for every descriptor managed by the queue manager. |

### 16.4.7.51 QUEUE_4_A Register (offset = 2040h) [reset = 0h]

QUEUE_4_A is shown in Figure 16-327 and described in Table 16-341.

**Figure 16-327. QUEUE_4_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-341. QUEUE_4_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>This count is incremented by 1 whenever a packet is added to the queue.<br>This count is decremented by 1 whenever a packet is popped from the queue. |

### 16.4.7.52 QUEUE_4_B Register (offset = 2044h) [reset = 0h]

QUEUE_4_B is shown in Figure 16-328 and described in Table 16-342.

**Figure 16-328. QUEUE_4_B Register**

| 31–28 | 27–0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-342. QUEUE_4_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. |

### 16.4.7.53 QUEUE_4_C Register (offset = 2048h) [reset = 0h]

QUEUE_4_C is shown in Figure 16-329 and described in Table 16-343.

**Figure 16-329. QUEUE_4_C Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| HEAD_TAIL | Reserved | | | | | | |
| W-0 | | | | | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| Reserved | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| Reserved | | PACKET_SIZE | | | | | |
| | | R/W-0 | | | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| PACKET_SIZE | | | | | | | |
| R/W-0 | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-343. QUEUE_4_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31 | HEAD_TAIL | W-0 | 0 | Head/Tail Push Control.<br>Set to zero in order to push packet onto tail of queue and set to one in order to push packet onto head of queue. |
| 13-0 | PACKET_SIZE | R/W-0 | 0 | packet_size This field indicates packet size and is assumed to be zero on each packet add unless the value is explicitly overwritten. This field indicates packet size for packet pop operation. |

### 16.4.7.54 QUEUE_4_D Register (offset = 204Ch) [reset = 0h]

QUEUE_4_D is shown in Figure 16-330 and described in Table 16-344.

**Figure 16-330. QUEUE_4_D Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | DESC_SIZE | | | | |
| R/W-0 | | | R/W-0 | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-344. QUEUE_4_D Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-5 | DESC_PTR | R/W-0 | 0 | Descriptor pointer.<br>It will be read as zero if the queue is empty.<br>It will indicate a<br>32-bit aligned address that points to a descriptor when the queue is not empty. |
| 4-0 | DESC_SIZE | R/W-0 | 0 | Descriptor Size.<br>It is encoded in<br>4-byte increments with values 0 to 31 representing 24 and so on to 148 bytes.<br>This field will return a 0x0 when an empty queue is read.<br>Queue Manager Queue N Registers D To save hardware resources, the queue manager internally stores descriptor size (desc_size) information in four bits.<br>However, register D has five LSBs that specify descriptor size.<br>As a consequence, the value of desc_size that is pushed may not be same as that is read during a pop.<br>The value that is read back is equal to always rounded to an odd number.<br>So, for even values, the value read back is one more than what was written.<br>For odd values, the value read back is same as the value that was written.<br>Note that this<br>5-bit field (desc_size) is unrelated to the code for size of descriptors in a descriptor region.<br>It is just a place holder for a<br>5-bit value that is maintained across the push and pop operations for every descriptor managed by the queue manager. |

### 16.4.7.55 QUEUE_5_A Register (offset = 2050h) [reset = 0h]

QUEUE_5_A is shown in Figure 16-331 and described in Table 16-345.

**Figure 16-331. QUEUE_5_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-345. QUEUE_5_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>This count is incremented by 1 whenever a packet is added to the queue.<br>This count is decremented by 1 whenever a packet is popped from the queue. |

### 16.4.7.56 QUEUE_5_B Register (offset = 2054h) [reset = 0h]

QUEUE_5_B is shown in Figure 16-332 and described in Table 16-346.

**Figure 16-332. QUEUE_5_B Register**

| 31-28 | 27-0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-346. QUEUE_5_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. |

### 16.4.7.57 QUEUE_5_C Register (offset = 2058h) [reset = 0h]

QUEUE_5_C is shown in Figure 16-333 and described in Table 16-347.

**Figure 16-333. QUEUE_5_C Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| HEAD_TAIL | Reserved | | | | | | |
| W-0 | | | | | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| Reserved | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| Reserved | | PACKET_SIZE | | | | | |
| | | R/W-0 | | | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| PACKET_SIZE | | | | | | | |
| R/W-0 | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-347. QUEUE_5_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31 | HEAD_TAIL | W-0 | 0 | Head/Tail Push Control.<br>Set to zero in order to push packet onto tail of queue and set to one in order to push packet onto head of queue. |
| 13-0 | PACKET_SIZE | R/W-0 | 0 | packet_size This field indicates packet size and is assumed to be zero on each packet add unless the value is explicitly overwritten. This field indicates packet size for packet pop operation. |

### 16.4.7.58 QUEUE_5_D Register (offset = 205Ch) [reset = 0h]

QUEUE_5_D is shown in Figure 16-334 and described in Table 16-348.

**Figure 16-334. QUEUE_5_D Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |
| **23** | **22** | **21** | **20** | **19** | **18** | **17** | **16** |
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |
| **15** | **14** | **13** | **12** | **11** | **10** | **9** | **8** |
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |
| **7** | **6** | **5** | **4** | **3** | **2** | **1** | **0** |
| DESC_PTR | | | DESC_SIZE | | | | |
| R/W-0 | | | R/W-0 | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-348. QUEUE_5_D Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-5 | DESC_PTR | R/W-0 | 0 | Descriptor pointer.<br>It will be read as zero if the queue is empty.<br>It will indicate a<br>32-bit aligned address that points to a descriptor when the queue is not empty. |
| 4-0 | DESC_SIZE | R/W-0 | 0 | Descriptor Size.<br>It is encoded in<br>4-byte increments with values 0 to 31 representing 24 and so on to 148 bytes.<br>This field will return a 0x0 when an empty queue is read.<br>Queue Manager Queue N Registers D To save hardware resources, the queue manager internally stores descriptor size (desc_size) information in four bits.<br>However, register D has five LSBs that specify descriptor size.<br>As a consequence, the value of desc_size that is pushed may not be same as that is read during a pop.<br>The value that is read back is equal to always rounded to an odd number.<br>So, for even values, the value read back is one more than what was written.<br>For odd values, the value read back is same as the value that was written.<br>Note that this<br>5-bit field (desc_size) is unrelated to the code for size of descriptors in a descriptor region.<br>It is just a place holder for a<br>5-bit value that is maintained across the push and pop operations for every descriptor managed by the queue manager. |

### 16.4.7.59 QUEUE_6_A Register (offset = 2060h) [reset = 0h]

QUEUE_6_A is shown in Figure 16-335 and described in Table 16-349.

**Figure 16-335. QUEUE_6_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-349. QUEUE_6_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>This count is incremented by 1 whenever a packet is added to the queue.<br>This count is decremented by 1 whenever a packet is popped from the queue. |

#### 16.4.7.60 QUEUE_6_B Register (offset = 2064h) [reset = 0h]

QUEUE_6_B is shown in Figure 16-336 and described in Table 16-350.

**Figure 16-336. QUEUE_6_B Register**

| 31–28 | 27–0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-350. QUEUE_6_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. |

### 16.4.7.61 QUEUE_6_C Register (offset = 2068h) [reset = 0h]

QUEUE_6_C is shown in Figure 16-337 and described in Table 16-351.

**Figure 16-337. QUEUE_6_C Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| HEAD_TAIL | Reserved | | | | | | |
| W-0 | | | | | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| Reserved | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| Reserved | | PACKET_SIZE | | | | | |
| | | R/W-0 | | | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| PACKET_SIZE | | | | | | | |
| R/W-0 | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-351. QUEUE_6_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31 | HEAD_TAIL | W-0 | 0 | Head/Tail Push Control.<br>Set to zero in order to push packet onto tail of queue and set to one in order to push packet onto head of queue. |
| 13-0 | PACKET_SIZE | R/W-0 | 0 | packet_size This field indicates packet size and is assumed to be zero on each packet add unless the value is explicitly overwritten. This field indicates packet size for packet pop operation. |

### 16.4.7.62 QUEUE_6_D Register (offset = 206Ch) [reset = 0h]

QUEUE_6_D is shown in Figure 16-338 and described in Table 16-352.

**Figure 16-338. QUEUE_6_D Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |
| **23** | **22** | **21** | **20** | **19** | **18** | **17** | **16** |
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |
| **15** | **14** | **13** | **12** | **11** | **10** | **9** | **8** |
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |
| **7** | **6** | **5** | **4** | **3** | **2** | **1** | **0** |
| DESC_PTR | | | DESC_SIZE | | | | |
| R/W-0 | | | R/W-0 | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-352. QUEUE_6_D Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-5 | DESC_PTR | R/W-0 | 0 | Descriptor pointer.<br>It will be read as zero if the queue is empty.<br>It will indicate a<br>32-bit aligned address that points to a descriptor when the queue is not empty. |
| 4-0 | DESC_SIZE | R/W-0 | 0 | Descriptor Size.<br>It is encoded in<br>4-byte increments with values 0 to 31 representing 24 and so on to 148 bytes.<br>This field will return a 0x0 when an empty queue is read.<br>Queue Manager Queue N Registers D To save hardware resources, the queue manager internally stores descriptor size (desc_size) information in four bits.<br>However, register D has five LSBs that specify descriptor size.<br>As a consequence, the value of desc_size that is pushed may not be same as that is read during a pop.<br>The value that is read back is equal to always rounded to an odd number.<br>So, for even values, the value read back is one more than what was written.<br>For odd values, the value read back is same as the value that was written.<br>Note that this<br>5-bit field (desc_size) is unrelated to the code for size of descriptors in a descriptor region.<br>It is just a place holder for a<br>5-bit value that is maintained across the push and pop operations for every descriptor managed by the queue manager. |

### 16.4.7.63 QUEUE_7_A Register (offset = 2070h) [reset = 0h]

QUEUE_7_A is shown in Figure 16-339 and described in Table 16-353.

**Figure 16-339. QUEUE_7_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-353. QUEUE_7_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>This count is incremented by 1 whenever a packet is added to the queue.<br>This count is decremented by 1 whenever a packet is popped from the queue. |

### 16.4.7.64 QUEUE_7_B Register (offset = 2074h) [reset = 0h]

QUEUE_7_B is shown in Figure 16-340 and described in Table 16-354.

**Figure 16-340. QUEUE_7_B Register**

| 31-28 | 27-0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-354. QUEUE_7_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. |

### 16.4.7.65 QUEUE_7_C Register (offset = 2078h) [reset = 0h]

QUEUE_7_C is shown in Figure 16-341 and described in Table 16-355.

**Figure 16-341. QUEUE_7_C Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| HEAD_TAIL | Reserved | | | | | | |
| W-0 | | | | | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| Reserved | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| Reserved | | PACKET_SIZE | | | | | |
| | | R/W-0 | | | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| PACKET_SIZE | | | | | | | |
| R/W-0 | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-355. QUEUE_7_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31 | HEAD_TAIL | W-0 | 0 | Head/Tail Push Control.<br>Set to zero in order to push packet onto tail of queue and set to one in order to push packet onto head of queue. |
| 13-0 | PACKET_SIZE | R/W-0 | 0 | packet_size This field indicates packet size and is assumed to be zero on each packet add unless the value is explicitly overwritten. This field indicates packet size for packet pop operation. |

### 16.4.7.66 QUEUE_7_D Register (offset = 207Ch) [reset = 0h]

QUEUE_7_D is shown in Figure 16-342 and described in Table 16-356.

**Figure 16-342. QUEUE_7_D Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | DESC_SIZE | | | | |
| R/W-0 | | | R/W-0 | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-356. QUEUE_7_D Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-5 | DESC_PTR | R/W-0 | 0 | Descriptor pointer.<br>It will be read as zero if the queue is empty.<br>It will indicate a<br>32-bit aligned address that points to a descriptor when the queue is not empty. |
| 4-0 | DESC_SIZE | R/W-0 | 0 | Descriptor Size.<br>It is encoded in<br>4-byte increments with values 0 to 31 representing 24 and so on to 148 bytes.<br>This field will return a 0x0 when an empty queue is read.<br>Queue Manager Queue N Registers D To save hardware resources, the queue manager internally stores descriptor size (desc_size) information in four bits.<br>However, register D has five LSBs that specify descriptor size.<br>As a consequence, the value of desc_size that is pushed may not be same as that is read during a pop.<br>The value that is read back is equal to always rounded to an odd number.<br>So, for even values, the value read back is one more than what was written.<br>For odd values, the value read back is same as the value that was written.<br>Note that this<br>5-bit field (desc_size) is unrelated to the code for size of descriptors in a descriptor region.<br>It is just a place holder for a<br>5-bit value that is maintained across the push and pop operations for every descriptor managed by the queue manager. |

### 16.4.7.67 QUEUE_8_A Register (offset = 2080h) [reset = 0h]

QUEUE_8_A is shown in Figure 16-343 and described in Table 16-357.

**Figure 16-343. QUEUE_8_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-357. QUEUE_8_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>This count is incremented by 1 whenever a packet is added to the queue.<br>This count is decremented by 1 whenever a packet is popped from the queue. |

### 16.4.7.68 QUEUE_8_B Register (offset = 2084h) [reset = 0h]

QUEUE_8_B is shown in Figure 16-344 and described in Table 16-358.

**Figure 16-344. QUEUE_8_B Register**

| 31–28 | 27–0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-358. QUEUE_8_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. |

### 16.4.7.69 QUEUE_8_C Register (offset = 2088h) [reset = 0h]

QUEUE_8_C is shown in Figure 16-345 and described in Table 16-359.

**Figure 16-345. QUEUE_8_C Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| HEAD_TAIL | Reserved | | | | | | |
| W-0 | | | | | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| Reserved | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| Reserved | | PACKET_SIZE | | | | | |
| | | R/W-0 | | | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| PACKET_SIZE | | | | | | | |
| R/W-0 | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-359. QUEUE_8_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31 | HEAD_TAIL | W-0 | 0 | Head/Tail Push Control.<br>Set to zero in order to push packet onto tail of queue and set to one in order to push packet onto head of queue. |
| 13-0 | PACKET_SIZE | R/W-0 | 0 | packet_size This field indicates packet size and is assumed to be zero on each packet add unless the value is explicitly overwritten. This field indicates packet size for packet pop operation. |

### 16.4.7.70 QUEUE_8_D Register (offset = 208Ch) [reset = 0h]

QUEUE_8_D is shown in Figure 16-346 and described in Table 16-360.

**Figure 16-346. QUEUE_8_D Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | DESC_SIZE | | | | |
| R/W-0 | | | R/W-0 | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-360. QUEUE_8_D Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-5 | DESC_PTR | R/W-0 | 0 | Descriptor pointer.<br>It will be read as zero if the queue is empty.<br>It will indicate a<br>32-bit aligned address that points to a descriptor when the queue is not empty. |
| 4-0 | DESC_SIZE | R/W-0 | 0 | Descriptor Size.<br>It is encoded in<br>4-byte increments with values 0 to 31 representing 24 and so on to 148 bytes.<br>This field will return a 0x0 when an empty queue is read.<br>Queue Manager Queue N Registers D To save hardware resources, the queue manager internally stores descriptor size (desc_size) information in four bits.<br>However, register D has five LSBs that specify descriptor size.<br>As a consequence, the value of desc_size that is pushed may not be same as that is read during a pop.<br>The value that is read back is equal to always rounded to an odd number.<br>So, for even values, the value read back is one more than what was written.<br>For odd values, the value read back is same as the value that was written.<br>Note that this<br>5-bit field (desc_size) is unrelated to the code for size of descriptors in a descriptor region.<br>It is just a place holder for a<br>5-bit value that is maintained across the push and pop operations for every descriptor managed by the queue manager. |

### 16.4.7.71 QUEUE_9_A Register (offset = 2090h) [reset = 0h]

QUEUE_9_A is shown in Figure 16-347 and described in Table 16-361.

**Figure 16-347. QUEUE_9_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-361. QUEUE_9_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>This count is incremented by 1 whenever a packet is added to the queue.<br>This count is decremented by 1 whenever a packet is popped from the queue. |

### 16.4.7.72 QUEUE_9_B Register (offset = 2094h) [reset = 0h]

QUEUE_9_B is shown in Figure 16-348 and described in Table 16-362.

**Figure 16-348. QUEUE_9_B Register**

| 31-28 | 27-0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-362. QUEUE_9_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. |

### 16.4.7.73 QUEUE_9_C Register (offset = 2098h) [reset = 0h]

QUEUE_9_C is shown in Figure 16-349 and described in Table 16-363.

**Figure 16-349. QUEUE_9_C Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| HEAD_TAIL | Reserved | | | | | | |
| W-0 | | | | | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| Reserved | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| Reserved | | PACKET_SIZE | | | | | |
| | | R/W-0 | | | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| PACKET_SIZE | | | | | | | |
| R/W-0 | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-363. QUEUE_9_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31 | HEAD_TAIL | W-0 | 0 | Head/Tail Push Control.<br>Set to zero in order to push packet onto tail of queue and set to one in order to push packet onto head of queue. |
| 13-0 | PACKET_SIZE | R/W-0 | 0 | packet_size This field indicates packet size and is assumed to be zero on each packet add unless the value is explicitly overwritten. This field indicates packet size for packet pop operation. |

### 16.4.7.74 QUEUE_9_D Register (offset = 209Ch) [reset = 0h]

QUEUE_9_D is shown in Figure 16-350 and described in Table 16-364.

**Figure 16-350. QUEUE_9_D Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |
| **23** | **22** | **21** | **20** | **19** | **18** | **17** | **16** |
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |
| **15** | **14** | **13** | **12** | **11** | **10** | **9** | **8** |
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |
| **7** | **6** | **5** | **4** | **3** | **2** | **1** | **0** |
| DESC_PTR | | | DESC_SIZE | | | | |
| R/W-0 | | | R/W-0 | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-364. QUEUE_9_D Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-5 | DESC_PTR | R/W-0 | 0 | Descriptor pointer.<br>It will be read as zero if the queue is empty.<br>It will indicate a<br>32-bit aligned address that points to a descriptor when the queue is not empty. |
| 4-0 | DESC_SIZE | R/W-0 | 0 | Descriptor Size.<br>It is encoded in<br>4-byte increments with values 0 to 31 representing 24 and so on to 148 bytes.<br>This field will return a 0x0 when an empty queue is read.<br>Queue Manager Queue N Registers D To save hardware resources, the queue manager internally stores descriptor size (desc_size) information in four bits.<br>However, register D has five LSBs that specify descriptor size.<br>As a consequence, the value of desc_size that is pushed may not be same as that is read during a pop.<br>The value that is read back is equal to always rounded to an odd number.<br>So, for even values, the value read back is one more than what was written.<br>For odd values, the value read back is same as the value that was written.<br>Note that this<br>5-bit field (desc_size) is unrelated to the code for size of descriptors in a descriptor region.<br>It is just a place holder for a<br>5-bit value that is maintained across the push and pop operations for every descriptor managed by the queue manager. |

### 16.4.7.75 QUEUE_10_A Register (offset = 20A0h) [reset = 0h]

QUEUE_10_A is shown in Figure 16-351 and described in Table 16-365.

**Figure 16-351. QUEUE_10_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-365. QUEUE_10_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>This count is incremented by 1 whenever a packet is added to the queue.<br>This count is decremented by 1 whenever a packet is popped from the queue. |

### 16.4.7.76 QUEUE_10_B Register (offset = 20A4h) [reset = 0h]

QUEUE_10_B is shown in Figure 16-352 and described in Table 16-366.

**Figure 16-352. QUEUE_10_B Register**

| 31–28 | 27–0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-366. QUEUE_10_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. |

### 16.4.7.77 QUEUE_10_C Register (offset = 20A8h) [reset = 0h]

QUEUE_10_C is shown in Figure 16-353 and described in Table 16-367.

**Figure 16-353. QUEUE_10_C Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| HEAD_TAIL | Reserved | | | | | | |
| W-0 | | | | | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| Reserved | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| Reserved | | PACKET_SIZE | | | | | |
| | | R/W-0 | | | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| PACKET_SIZE | | | | | | | |
| R/W-0 | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-367. QUEUE_10_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31 | HEAD_TAIL | W-0 | 0 | Head/Tail Push Control.<br>Set to zero in order to push packet onto tail of queue and set to one in order to push packet onto head of queue. |
| 13-0 | PACKET_SIZE | R/W-0 | 0 | packet_size This field indicates packet size and is assumed to be zero on each packet add unless the value is explicitly overwritten. This field indicates packet size for packet pop operation. |

### 16.4.7.78 QUEUE_10_D Register (offset = 20ACh) [reset = 0h]

QUEUE_10_D is shown in Figure 16-354 and described in Table 16-368.

**Figure 16-354. QUEUE_10_D Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |
| **23** | **22** | **21** | **20** | **19** | **18** | **17** | **16** |
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |
| **15** | **14** | **13** | **12** | **11** | **10** | **9** | **8** |
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |
| **7** | **6** | **5** | **4** | **3** | **2** | **1** | **0** |
| DESC_PTR | | | DESC_SIZE | | | | |
| R/W-0 | | | R/W-0 | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-368. QUEUE_10_D Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-5 | DESC_PTR | R/W-0 | 0 | Descriptor pointer.<br>It will be read as zero if the queue is empty.<br>It will indicate a<br>32-bit aligned address that points to a descriptor when the queue is not empty. |
| 4-0 | DESC_SIZE | R/W-0 | 0 | Descriptor Size.<br>It is encoded in<br>4-byte increments with values 0 to 31 representing 24 and so on to 148 bytes.<br>This field will return a 0x0 when an empty queue is read.<br>Queue Manager Queue N Registers D To save hardware resources, the queue manager internally stores descriptor size (desc_size) information in four bits.<br>However, register D has five LSBs that specify descriptor size.<br>As a consequence, the value of desc_size that is pushed may not be same as that is read during a pop.<br>The value that is read back is equal to always rounded to an odd number.<br>So, for even values, the value read back is one more than what was written.<br>For odd values, the value read back is same as the value that was written.<br>Note that this<br>5-bit field (desc_size) is unrelated to the code for size of descriptors in a descriptor region.<br>It is just a place holder for a<br>5-bit value that is maintained across the push and pop operations for every descriptor managed by the queue manager. |

### 16.4.7.79 QUEUE_11_A Register (offset = 20B0h) [reset = 0h]

QUEUE_11_A is shown in Figure 16-355 and described in Table 16-369.

**Figure 16-355. QUEUE_11_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-369. QUEUE_11_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>This count is incremented by 1 whenever a packet is added to the queue.<br>This count is decremented by 1 whenever a packet is popped from the queue. |

### 16.4.7.80 QUEUE_11_B Register (offset = 20B4h) [reset = 0h]

QUEUE_11_B is shown in Figure 16-356 and described in Table 16-370.

**Figure 16-356. QUEUE_11_B Register**

| 31–28 | 27–0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-370. QUEUE_11_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. |

### 16.4.7.81 QUEUE_11_C Register (offset = 20B8h) [reset = 0h]

QUEUE_11_C is shown in Figure 16-357 and described in Table 16-371.

**Figure 16-357. QUEUE_11_C Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| HEAD_TAIL | Reserved | | | | | | |
| W-0 | | | | | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| Reserved | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| Reserved | | PACKET_SIZE | | | | | |
| | | R/W-0 | | | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| PACKET_SIZE | | | | | | | |
| R/W-0 | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-371. QUEUE_11_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31 | HEAD_TAIL | W-0 | 0 | Head/Tail Push Control.<br>Set to zero in order to push packet onto tail of queue and set to one in order to push packet onto head of queue. |
| 13-0 | PACKET_SIZE | R/W-0 | 0 | packet_size This field indicates packet size and is assumed to be zero on each packet add unless the value is explicitly overwritten. This field indicates packet size for packet pop operation. |

### 16.4.7.82 QUEUE_11_D Register (offset = 20BCh) [reset = 0h]

QUEUE_11_D is shown in Figure 16-358 and described in Table 16-372.

**Figure 16-358. QUEUE_11_D Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |
| **23** | **22** | **21** | **20** | **19** | **18** | **17** | **16** |
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |
| **15** | **14** | **13** | **12** | **11** | **10** | **9** | **8** |
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |
| **7** | **6** | **5** | **4** | **3** | **2** | **1** | **0** |
| DESC_PTR | | | DESC_SIZE | | | | |
| R/W-0 | | | R/W-0 | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-372. QUEUE_11_D Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-5 | DESC_PTR | R/W-0 | 0 | Descriptor pointer.<br>It will be read as zero if the queue is empty.<br>It will indicate a<br>32-bit aligned address that points to a descriptor when the queue is not empty. |
| 4-0 | DESC_SIZE | R/W-0 | 0 | Descriptor Size.<br>It is encoded in<br>4-byte increments with values 0 to 31 representing 24 and so on to 148 bytes.<br>This field will return a 0x0 when an empty queue is read.<br>Queue Manager Queue N Registers D To save hardware resources, the queue manager internally stores descriptor size (desc_size) information in four bits.<br>However, register D has five LSBs that specify descriptor size.<br>As a consequence, the value of desc_size that is pushed may not be same as that is read during a pop.<br>The value that is read back is equal to always rounded to an odd number.<br>So, for even values, the value read back is one more than what was written.<br>For odd values, the value read back is same as the value that was written.<br>Note that this<br>5-bit field (desc_size) is unrelated to the code for size of descriptors in a descriptor region.<br>It is just a place holder for a<br>5-bit value that is maintained across the push and pop operations for every descriptor managed by the queue manager. |

### 16.4.7.83 QUEUE_12_A Register (offset = 20C0h) [reset = 0h]

QUEUE_12_A is shown in Figure 16-359 and described in Table 16-373.

**Figure 16-359. QUEUE_12_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-373. QUEUE_12_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>This count is incremented by 1 whenever a packet is added to the queue.<br>This count is decremented by 1 whenever a packet is popped from the queue. |

### 16.4.7.84 QUEUE_12_B Register (offset = 20C4h) [reset = 0h]

QUEUE_12_B is shown in Figure 16-360 and described in Table 16-374.

**Figure 16-360. QUEUE_12_B Register**

| 31–28 | 27–0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-374. QUEUE_12_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. |

### 16.4.7.85 QUEUE_12_C Register (offset = 20C8h) [reset = 0h]

QUEUE_12_C is shown in Figure 16-361 and described in Table 16-375.

**Figure 16-361. QUEUE_12_C Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| HEAD_TAIL | Reserved | | | | | | |
| W-0 | | | | | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| Reserved | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| Reserved | | PACKET_SIZE | | | | | |
| | | R/W-0 | | | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| PACKET_SIZE | | | | | | | |
| R/W-0 | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-375. QUEUE_12_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31 | HEAD_TAIL | W-0 | 0 | Head/Tail Push Control.<br>Set to zero in order to push packet onto tail of queue and set to one in order to push packet onto head of queue. |
| 13-0 | PACKET_SIZE | R/W-0 | 0 | packet_size This field indicates packet size and is assumed to be zero on each packet add unless the value is explicitly overwritten. This field indicates packet size for packet pop operation. |

### 16.4.7.86 QUEUE_12_D Register (offset = 20CCh) [reset = 0h]

QUEUE_12_D is shown in Figure 16-362 and described in Table 16-376.

**Figure 16-362. QUEUE_12_D Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |
| **23** | **22** | **21** | **20** | **19** | **18** | **17** | **16** |
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |
| **15** | **14** | **13** | **12** | **11** | **10** | **9** | **8** |
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |
| **7** | **6** | **5** | **4** | **3** | **2** | **1** | **0** |
| DESC_PTR | | | DESC_SIZE | | | | |
| R/W-0 | | | R/W-0 | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-376. QUEUE_12_D Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-5 | DESC_PTR | R/W-0 | 0 | Descriptor pointer.<br>It will be read as zero if the queue is empty.<br>It will indicate a<br>32-bit aligned address that points to a descriptor when the queue is not empty. |
| 4-0 | DESC_SIZE | R/W-0 | 0 | Descriptor Size.<br>It is encoded in<br>4-byte increments with values 0 to 31 representing 24 and so on to 148 bytes.<br>This field will return a 0x0 when an empty queue is read.<br>Queue Manager Queue N Registers D To save hardware resources, the queue manager internally stores descriptor size (desc_size) information in four bits.<br>However, register D has five LSBs that specify descriptor size.<br>As a consequence, the value of desc_size that is pushed may not be same as that is read during a pop.<br>The value that is read back is equal to always rounded to an odd number.<br>So, for even values, the value read back is one more than what was written.<br>For odd values, the value read back is same as the value that was written.<br>Note that this<br>5-bit field (desc_size) is unrelated to the code for size of descriptors in a descriptor region.<br>It is just a place holder for a<br>5-bit value that is maintained across the push and pop operations for every descriptor managed by the queue manager. |

### 16.4.7.87 QUEUE_13_A Register (offset = 20D0h) [reset = 0h]

QUEUE_13_A is shown in Figure 16-363 and described in Table 16-377.

**Figure 16-363. QUEUE_13_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-377. QUEUE_13_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>This count is incremented by 1 whenever a packet is added to the queue.<br>This count is decremented by 1 whenever a packet is popped from the queue. |

### 16.4.7.88 QUEUE_13_B Register (offset = 20D4h) [reset = 0h]

QUEUE_13_B is shown in Figure 16-364 and described in Table 16-378.

**Figure 16-364. QUEUE_13_B Register**

| 31-28 | 27-0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-378. QUEUE_13_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. |

### 16.4.7.89 QUEUE_13_C Register (offset = 20D8h) [reset = 0h]

QUEUE_13_C is shown in Figure 16-365 and described in Table 16-379.

**Figure 16-365. QUEUE_13_C Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| HEAD_TAIL | Reserved | | | | | | |
| W-0 | | | | | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| Reserved | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| Reserved | | PACKET_SIZE | | | | | |
| | | R/W-0 | | | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| PACKET_SIZE | | | | | | | |
| R/W-0 | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-379. QUEUE_13_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31 | HEAD_TAIL | W-0 | 0 | Head/Tail Push Control.<br>Set to zero in order to push packet onto tail of queue and set to one in order to push packet onto head of queue. |
| 13-0 | PACKET_SIZE | R/W-0 | 0 | packet_size This field indicates packet size and is assumed to be zero on each packet add unless the value is explicitly overwritten. This field indicates packet size for packet pop operation. |

### 16.4.7.90 QUEUE_13_D Register (offset = 20DCh) [reset = 0h]

QUEUE_13_D is shown in Figure 16-366 and described in Table 16-380.

**Figure 16-366. QUEUE_13_D Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |
| **23** | **22** | **21** | **20** | **19** | **18** | **17** | **16** |
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |
| **15** | **14** | **13** | **12** | **11** | **10** | **9** | **8** |
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |
| **7** | **6** | **5** | **4** | **3** | **2** | **1** | **0** |
| DESC_PTR | | | DESC_SIZE | | | | |
| R/W-0 | | | R/W-0 | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-380. QUEUE_13_D Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-5 | DESC_PTR | R/W-0 | 0 | Descriptor pointer.<br>It will be read as zero if the queue is empty.<br>It will indicate a<br>32-bit aligned address that points to a descriptor when the queue is not empty. |
| 4-0 | DESC_SIZE | R/W-0 | 0 | Descriptor Size.<br>It is encoded in<br>4-byte increments with values 0 to 31 representing 24 and so on to 148 bytes.<br>This field will return a 0x0 when an empty queue is read.<br>Queue Manager Queue N Registers D To save hardware resources, the queue manager internally stores descriptor size (desc_size) information in four bits.<br>However, register D has five LSBs that specify descriptor size.<br>As a consequence, the value of desc_size that is pushed may not be same as that is read during a pop.<br>The value that is read back is equal to always rounded to an odd number.<br>So, for even values, the value read back is one more than what was written.<br>For odd values, the value read back is same as the value that was written.<br>Note that this<br>5-bit field (desc_size) is unrelated to the code for size of descriptors in a descriptor region.<br>It is just a place holder for a<br>5-bit value that is maintained across the push and pop operations for every descriptor managed by the queue manager. |

### 16.4.7.91 QUEUE_14_A Register (offset = 20E0h) [reset = 0h]

QUEUE_14_A is shown in Figure 16-367 and described in Table 16-381.

**Figure 16-367. QUEUE_14_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-381. QUEUE_14_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>This count is incremented by 1 whenever a packet is added to the queue.<br>This count is decremented by 1 whenever a packet is popped from the queue. |

### 16.4.7.92 QUEUE_14_B Register (offset = 20E4h) [reset = 0h]

QUEUE_14_B is shown in Figure 16-368 and described in Table 16-382.

**Figure 16-368. QUEUE_14_B Register**

| 31–28 | 27–0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-382. QUEUE_14_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. |

### 16.4.7.93 QUEUE_14_C Register (offset = 20E8h) [reset = 0h]

QUEUE_14_C is shown in Figure 16-369 and described in Table 16-383.

**Figure 16-369. QUEUE_14_C Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| HEAD_TAIL | Reserved | | | | | | |
| W-0 | | | | | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| Reserved | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| Reserved | | PACKET_SIZE | | | | | |
| | | R/W-0 | | | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| PACKET_SIZE | | | | | | | |
| R/W-0 | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-383. QUEUE_14_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31 | HEAD_TAIL | W-0 | 0 | Head/Tail Push Control.<br>Set to zero in order to push packet onto tail of queue and set to one in order to push packet onto head of queue. |
| 13-0 | PACKET_SIZE | R/W-0 | 0 | packet_size This field indicates packet size and is assumed to be zero on each packet add unless the value is explicitly overwritten. This field indicates packet size for packet pop operation. |

### 16.4.7.94 QUEUE_14_D Register (offset = 20ECh) [reset = 0h]

QUEUE_14_D is shown in Figure 16-370 and described in Table 16-384.

**Figure 16-370. QUEUE_14_D Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |
| **23** | **22** | **21** | **20** | **19** | **18** | **17** | **16** |
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |
| **15** | **14** | **13** | **12** | **11** | **10** | **9** | **8** |
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |
| **7** | **6** | **5** | **4** | **3** | **2** | **1** | **0** |
| DESC_PTR | | | DESC_SIZE | | | | |
| R/W-0 | | | R/W-0 | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-384. QUEUE_14_D Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-5 | DESC_PTR | R/W-0 | 0 | Descriptor pointer.<br>It will be read as zero if the queue is empty.<br>It will indicate a<br>32-bit aligned address that points to a descriptor when the queue is not empty. |
| 4-0 | DESC_SIZE | R/W-0 | 0 | Descriptor Size.<br>It is encoded in<br>4-byte increments with values 0 to 31 representing 24 and so on to 148 bytes.<br>This field will return a 0x0 when an empty queue is read.<br>Queue Manager Queue N Registers D To save hardware resources, the queue manager internally stores descriptor size (desc_size) information in four bits.<br>However, register D has five LSBs that specify descriptor size.<br>As a consequence, the value of desc_size that is pushed may not be same as that is read during a pop.<br>The value that is read back is equal to always rounded to an odd number.<br>So, for even values, the value read back is one more than what was written.<br>For odd values, the value read back is same as the value that was written.<br>Note that this<br>5-bit field (desc_size) is unrelated to the code for size of descriptors in a descriptor region.<br>It is just a place holder for a<br>5-bit value that is maintained across the push and pop operations for every descriptor managed by the queue manager. |

### 16.4.7.95 QUEUE_15_A Register (offset = 20F0h) [reset = 0h]

QUEUE_15_A is shown in Figure 16-371 and described in Table 16-385.

**Figure 16-371. QUEUE_15_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-385. QUEUE_15_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>This count is incremented by 1 whenever a packet is added to the queue.<br>This count is decremented by 1 whenever a packet is popped from the queue. |

### 16.4.7.96 QUEUE_15_B Register (offset = 20F4h) [reset = 0h]

QUEUE_15_B is shown in Figure 16-372 and described in Table 16-386.

**Figure 16-372. QUEUE_15_B Register**

| 31–28 | 27–0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-386. QUEUE_15_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. |

### 16.4.7.97 QUEUE_15_C Register (offset = 20F8h) [reset = 0h]

QUEUE_15_C is shown in Figure 16-373 and described in Table 16-387.

**Figure 16-373. QUEUE_15_C Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| HEAD_TAIL | Reserved | | | | | | |
| W-0 | | | | | | | |
| **23** | **22** | **21** | **20** | **19** | **18** | **17** | **16** |
| Reserved | | | | | | | |
| **15** | **14** | **13** | **12** | **11** | **10** | **9** | **8** |
| Reserved | | PACKET_SIZE | | | | | |
| | | R/W-0 | | | | | |
| **7** | **6** | **5** | **4** | **3** | **2** | **1** | **0** |
| PACKET_SIZE | | | | | | | |
| R/W-0 | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-387. QUEUE_15_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31 | HEAD_TAIL | W-0 | 0 | Head/Tail Push Control.<br>Set to zero in order to push packet onto tail of queue and set to one in order to push packet onto head of queue. |
| 13-0 | PACKET_SIZE | R/W-0 | 0 | packet_size This field indicates packet size and is assumed to be zero on each packet add unless the value is explicitly overwritten. This field indicates packet size for packet pop operation. |

### 16.4.7.98 QUEUE_15_D Register (offset = 20FCh) [reset = 0h]

QUEUE_15_D is shown in Figure 16-374 and described in Table 16-388.

**Figure 16-374. QUEUE_15_D Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | DESC_SIZE | | | | |
| R/W-0 | | | R/W-0 | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-388. QUEUE_15_D Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-5 | DESC_PTR | R/W-0 | 0 | Descriptor pointer.<br>It will be read as zero if the queue is empty.<br>It will indicate a<br>32-bit aligned address that points to a descriptor when the queue is not empty. |
| 4-0 | DESC_SIZE | R/W-0 | 0 | Descriptor Size.<br>It is encoded in<br>4-byte increments with values 0 to 31 representing 24 and so on to 148 bytes.<br>This field will return a 0x0 when an empty queue is read.<br>Queue Manager Queue N Registers D To save hardware resources, the queue manager internally stores descriptor size (desc_size) information in four bits.<br>However, register D has five LSBs that specify descriptor size.<br>As a consequence, the value of desc_size that is pushed may not be same as that is read during a pop.<br>The value that is read back is equal to always rounded to an odd number.<br>So, for even values, the value read back is one more than what was written.<br>For odd values, the value read back is same as the value that was written.<br>Note that this<br>5-bit field (desc_size) is unrelated to the code for size of descriptors in a descriptor region.<br>It is just a place holder for a<br>5-bit value that is maintained across the push and pop operations for every descriptor managed by the queue manager. |

### 16.4.7.99 QUEUE_16_A Register (offset = 2100h) [reset = 0h]

QUEUE_16_A is shown in Figure 16-375 and described in Table 16-389.

**Figure 16-375. QUEUE_16_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-389. QUEUE_16_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>This count is incremented by 1 whenever a packet is added to the queue.<br>This count is decremented by 1 whenever a packet is popped from the queue. |

### 16.4.7.100 QUEUE_16_B Register (offset = 2104h) [reset = 0h]

QUEUE_16_B is shown in Figure 16-376 and described in Table 16-390.

**Figure 16-376. QUEUE_16_B Register**

| 31–28 | 27–0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-390. QUEUE_16_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. |

### 16.4.7.101 QUEUE_16_C Register (offset = 2108h) [reset = 0h]

QUEUE_16_C is shown in Figure 16-377 and described in Table 16-391.

**Figure 16-377. QUEUE_16_C Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| HEAD_TAIL | Reserved | | | | | | |
| W-0 | | | | | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| Reserved | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| Reserved | | PACKET_SIZE | | | | | |
| | | R/W-0 | | | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| PACKET_SIZE | | | | | | | |
| R/W-0 | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-391. QUEUE_16_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31 | HEAD_TAIL | W-0 | 0 | Head/Tail Push Control.<br>Set to zero in order to push packet onto tail of queue and set to one in order to push packet onto head of queue. |
| 13-0 | PACKET_SIZE | R/W-0 | 0 | packet_size This field indicates packet size and is assumed to be zero on each packet add unless the value is explicitly overwritten. This field indicates packet size for packet pop operation. |

### 16.4.7.102 QUEUE_16_D Register (offset = 210Ch) [reset = 0h]

QUEUE_16_D is shown in Figure 16-378 and described in Table 16-392.

**Figure 16-378. QUEUE_16_D Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | DESC_SIZE | | | | |
| R/W-0 | | | R/W-0 | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-392. QUEUE_16_D Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-5 | DESC_PTR | R/W-0 | 0 | Descriptor pointer.<br>It will be read as zero if the queue is empty.<br>It will indicate a<br>32-bit aligned address that points to a descriptor when the queue is not empty. |
| 4-0 | DESC_SIZE | R/W-0 | 0 | Descriptor Size.<br>It is encoded in<br>4-byte increments with values 0 to 31 representing 24 and so on to 148 bytes.<br>This field will return a 0x0 when an empty queue is read.<br>Queue Manager Queue N Registers D To save hardware resources, the queue manager internally stores descriptor size (desc_size) information in four bits.<br>However, register D has five LSBs that specify descriptor size.<br>As a consequence, the value of desc_size that is pushed may not be same as that is read during a pop.<br>The value that is read back is equal to always rounded to an odd number.<br>So, for even values, the value read back is one more than what was written.<br>For odd values, the value read back is same as the value that was written.<br>Note that this<br>5-bit field (desc_size) is unrelated to the code for size of descriptors in a descriptor region.<br>It is just a place holder for a<br>5-bit value that is maintained across the push and pop operations for every descriptor managed by the queue manager. |

### 16.4.7.103 QUEUE_17_A Register (offset = 2110h) [reset = 0h]

QUEUE_17_A is shown in Figure 16-379 and described in Table 16-393.

**Figure 16-379. QUEUE_17_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-393. QUEUE_17_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>This count is incremented by 1 whenever a packet is added to the queue.<br>This count is decremented by 1 whenever a packet is popped from the queue. |

### 16.4.7.104 QUEUE_17_B Register (offset = 2114h) [reset = 0h]

QUEUE_17_B is shown in Figure 16-380 and described in Table 16-394.

**Figure 16-380. QUEUE_17_B Register**

| 31-28 | 27-0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-394. QUEUE_17_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. |

### 16.4.7.105 QUEUE_17_C Register (offset = 2118h) [reset = 0h]

QUEUE_17_C is shown in Figure 16-381 and described in Table 16-395.

**Figure 16-381. QUEUE_17_C Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| HEAD_TAIL | Reserved | | | | | | |
| W-0 | | | | | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| Reserved | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| Reserved | | PACKET_SIZE | | | | | |
| | | R/W-0 | | | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| PACKET_SIZE | | | | | | | |
| R/W-0 | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-395. QUEUE_17_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31 | HEAD_TAIL | W-0 | 0 | Head/Tail Push Control.<br>Set to zero in order to push packet onto tail of queue and set to one in order to push packet onto head of queue. |
| 13-0 | PACKET_SIZE | R/W-0 | 0 | packet_size This field indicates packet size and is assumed to be zero on each packet add unless the value is explicitly overwritten. This field indicates packet size for packet pop operation. |

### 16.4.7.106 QUEUE_17_D Register (offset = 211Ch) [reset = 0h]

QUEUE_17_D is shown in Figure 16-382 and described in Table 16-396.

**Figure 16-382. QUEUE_17_D Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | DESC_SIZE | | | | |
| R/W-0 | | | R/W-0 | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-396. QUEUE_17_D Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-5 | DESC_PTR | R/W-0 | 0 | Descriptor pointer.<br>It will be read as zero if the queue is empty.<br>It will indicate a<br>32-bit aligned address that points to a descriptor when the queue is not empty. |
| 4-0 | DESC_SIZE | R/W-0 | 0 | Descriptor Size.<br>It is encoded in<br>4-byte increments with values 0 to 31 representing 24 and so on to 148 bytes.<br>This field will return a 0x0 when an empty queue is read.<br>Queue Manager Queue N Registers D To save hardware resources, the queue manager internally stores descriptor size (desc_size) information in four bits.<br>However, register D has five LSBs that specify descriptor size.<br>As a consequence, the value of desc_size that is pushed may not be same as that is read during a pop.<br>The value that is read back is equal to always rounded to an odd number.<br>So, for even values, the value read back is one more than what was written.<br>For odd values, the value read back is same as the value that was written.<br>Note that this<br>5-bit field (desc_size) is unrelated to the code for size of descriptors in a descriptor region.<br>It is just a place holder for a<br>5-bit value that is maintained across the push and pop operations for every descriptor managed by the queue manager. |

### 16.4.7.107 QUEUE_18_A Register (offset = 2120h) [reset = 0h]

QUEUE_18_A is shown in Figure 16-383 and described in Table 16-397.

**Figure 16-383. QUEUE_18_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-397. QUEUE_18_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>This count is incremented by 1 whenever a packet is added to the queue.<br>This count is decremented by 1 whenever a packet is popped from the queue. |

### 16.4.7.108 QUEUE_18_B Register (offset = 2124h) [reset = 0h]

QUEUE_18_B is shown in Figure 16-384 and described in Table 16-398.

**Figure 16-384. QUEUE_18_B Register**

| 31–28 | 27–0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-398. QUEUE_18_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. |

### 16.4.7.109 QUEUE_18_C Register (offset = 2128h) [reset = 0h]

QUEUE_18_C is shown in Figure 16-385 and described in Table 16-399.

**Figure 16-385. QUEUE_18_C Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| HEAD_TAIL | Reserved | | | | | | |
| W-0 | | | | | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| Reserved | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| Reserved | | PACKET_SIZE | | | | | |
| | | R/W-0 | | | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| PACKET_SIZE | | | | | | | |
| R/W-0 | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-399. QUEUE_18_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31 | HEAD_TAIL | W-0 | 0 | Head/Tail Push Control.<br>Set to zero in order to push packet onto tail of queue and set to one in order to push packet onto head of queue. |
| 13-0 | PACKET_SIZE | R/W-0 | 0 | packet_size This field indicates packet size and is assumed to be zero on each packet add unless the value is explicitly overwritten. This field indicates packet size for packet pop operation. |

### 16.4.7.110 QUEUE_18_D Register (offset = 212Ch) [reset = 0h]

QUEUE_18_D is shown in Figure 16-386 and described in Table 16-400.

**Figure 16-386. QUEUE_18_D Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | DESC_SIZE | | | | |
| R/W-0 | | | R/W-0 | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-400. QUEUE_18_D Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-5 | DESC_PTR | R/W-0 | 0 | Descriptor pointer.<br>It will be read as zero if the queue is empty.<br>It will indicate a<br>32-bit aligned address that points to a descriptor when the queue is not empty. |
| 4-0 | DESC_SIZE | R/W-0 | 0 | Descriptor Size.<br>It is encoded in<br>4-byte increments with values 0 to 31 representing 24 and so on to 148 bytes.<br>This field will return a 0x0 when an empty queue is read.<br>Queue Manager Queue N Registers D To save hardware resources, the queue manager internally stores descriptor size (desc_size) information in four bits.<br>However, register D has five LSBs that specify descriptor size.<br>As a consequence, the value of desc_size that is pushed may not be same as that is read during a pop.<br>The value that is read back is equal to always rounded to an odd number.<br>So, for even values, the value read back is one more than what was written.<br>For odd values, the value read back is same as the value that was written.<br>Note that this<br>5-bit field (desc_size) is unrelated to the code for size of descriptors in a descriptor region.<br>It is just a place holder for a<br>5-bit value that is maintained across the push and pop operations for every descriptor managed by the queue manager. |

### 16.4.7.111 QUEUE_19_A Register (offset = 2130h) [reset = 0h]

QUEUE_19_A is shown in Figure 16-387 and described in Table 16-401.

**Figure 16-387. QUEUE_19_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-401. QUEUE_19_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>This count is incremented by 1 whenever a packet is added to the queue.<br>This count is decremented by 1 whenever a packet is popped from the queue. |

### 16.4.7.112 QUEUE_19_B Register (offset = 2134h) [reset = 0h]

QUEUE_19_B is shown in Figure 16-388 and described in Table 16-402.

**Figure 16-388. QUEUE_19_B Register**

| 31-28 | 27-0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-402. QUEUE_19_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. |

### 16.4.7.113 QUEUE_19_C Register (offset = 2138h) [reset = 0h]

QUEUE_19_C is shown in Figure 16-389 and described in Table 16-403.

**Figure 16-389. QUEUE_19_C Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| HEAD_TAIL | Reserved | | | | | | |
| W-0 | | | | | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| Reserved | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| Reserved | | PACKET_SIZE | | | | | |
| | | R/W-0 | | | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| PACKET_SIZE | | | | | | | |
| R/W-0 | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-403. QUEUE_19_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31 | HEAD_TAIL | W-0 | 0 | Head/Tail Push Control.<br>Set to zero in order to push packet onto tail of queue and set to one in order to push packet onto head of queue. |
| 13-0 | PACKET_SIZE | R/W-0 | 0 | packet_size This field indicates packet size and is assumed to be zero on each packet add unless the value is explicitly overwritten. This field indicates packet size for packet pop operation. |

### 16.4.7.114 QUEUE_19_D Register (offset = 213Ch) [reset = 0h]

QUEUE_19_D is shown in Figure 16-390 and described in Table 16-404.

**Figure 16-390. QUEUE_19_D Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |
| **23** | **22** | **21** | **20** | **19** | **18** | **17** | **16** |
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |
| **15** | **14** | **13** | **12** | **11** | **10** | **9** | **8** |
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |
| **7** | **6** | **5** | **4** | **3** | **2** | **1** | **0** |
| DESC_PTR | | | DESC_SIZE | | | | |
| R/W-0 | | | R/W-0 | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-404. QUEUE_19_D Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-5 | DESC_PTR | R/W-0 | 0 | Descriptor pointer.<br>It will be read as zero if the queue is empty.<br>It will indicate a<br>32-bit aligned address that points to a descriptor when the queue is not empty. |
| 4-0 | DESC_SIZE | R/W-0 | 0 | Descriptor Size.<br>It is encoded in<br>4-byte increments with values 0 to 31 representing 24 and so on to 148 bytes.<br>This field will return a 0x0 when an empty queue is read.<br>Queue Manager Queue N Registers D To save hardware resources, the queue manager internally stores descriptor size (desc_size) information in four bits.<br>However, register D has five LSBs that specify descriptor size.<br>As a consequence, the value of desc_size that is pushed may not be same as that is read during a pop.<br>The value that is read back is equal to always rounded to an odd number.<br>So, for even values, the value read back is one more than what was written.<br>For odd values, the value read back is same as the value that was written.<br>Note that this<br>5-bit field (desc_size) is unrelated to the code for size of descriptors in a descriptor region.<br>It is just a place holder for a<br>5-bit value that is maintained across the push and pop operations for every descriptor managed by the queue manager. |

### 16.4.7.115 QUEUE_20_A Register (offset = 2140h) [reset = 0h]

QUEUE_20_A is shown in Figure 16-391 and described in Table 16-405.

**Figure 16-391. QUEUE_20_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-405. QUEUE_20_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>This count is incremented by 1 whenever a packet is added to the queue.<br>This count is decremented by 1 whenever a packet is popped from the queue. |

### 16.4.7.116 QUEUE_20_B Register (offset = 2144h) [reset = 0h]

QUEUE_20_B is shown in Figure 16-392 and described in Table 16-406.

**Figure 16-392. QUEUE_20_B Register**

| 31–28 | 27–0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-406. QUEUE_20_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. |

### 16.4.7.117 QUEUE_20_C Register (offset = 2148h) [reset = 0h]

QUEUE_20_C is shown in Figure 16-393 and described in Table 16-407.

**Figure 16-393. QUEUE_20_C Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| HEAD_TAIL | Reserved | | | | | | |
| W-0 | | | | | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| Reserved | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| Reserved | | PACKET_SIZE | | | | | |
| | | R/W-0 | | | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| PACKET_SIZE | | | | | | | |
| R/W-0 | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-407. QUEUE_20_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31 | HEAD_TAIL | W-0 | 0 | Head/Tail Push Control.<br>Set to zero in order to push packet onto tail of queue and set to one in order to push packet onto head of queue. |
| 13-0 | PACKET_SIZE | R/W-0 | 0 | packet_size This field indicates packet size and is assumed to be zero on each packet add unless the value is explicitly overwritten. This field indicates packet size for packet pop operation. |

### 16.4.7.118 QUEUE_20_D Register (offset = 214Ch) [reset = 0h]

QUEUE_20_D is shown in Figure 16-394 and described in Table 16-408.

**Figure 16-394. QUEUE_20_D Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | DESC_SIZE | | | | |
| R/W-0 | | | R/W-0 | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-408. QUEUE_20_D Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-5 | DESC_PTR | R/W-0 | 0 | Descriptor pointer.<br>It will be read as zero if the queue is empty.<br>It will indicate a<br>32-bit aligned address that points to a descriptor when the queue is not empty. |
| 4-0 | DESC_SIZE | R/W-0 | 0 | Descriptor Size.<br>It is encoded in<br>4-byte increments with values 0 to 31 representing 24 and so on to 148 bytes.<br>This field will return a 0x0 when an empty queue is read.<br>Queue Manager Queue N Registers D To save hardware resources, the queue manager internally stores descriptor size (desc_size) information in four bits.<br>However, register D has five LSBs that specify descriptor size.<br>As a consequence, the value of desc_size that is pushed may not be same as that is read during a pop.<br>The value that is read back is equal to always rounded to an odd number.<br>So, for even values, the value read back is one more than what was written.<br>For odd values, the value read back is same as the value that was written.<br>Note that this<br>5-bit field (desc_size) is unrelated to the code for size of descriptors in a descriptor region.<br>It is just a place holder for a<br>5-bit value that is maintained across the push and pop operations for every descriptor managed by the queue manager. |

### 16.4.7.119 QUEUE_21_A Register (offset = 2150h) [reset = 0h]

QUEUE_21_A is shown in Figure 16-395 and described in Table 16-409.

**Figure 16-395. QUEUE_21_A Register**

| 31–14 | 13–0 |
| --- | --- |
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-409. QUEUE_21_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
| --- | --- | --- | --- | --- |
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>This count is incremented by 1 whenever a packet is added to the queue.<br>This count is decremented by 1 whenever a packet is popped from the queue. |

### 16.4.7.120 QUEUE_21_B Register (offset = 2154h) [reset = 0h]

QUEUE_21_B is shown in Figure 16-396 and described in Table 16-410.

**Figure 16-396. QUEUE_21_B Register**

| 31–28 | 27–0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-410. QUEUE_21_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. |

### 16.4.7.121 QUEUE_21_C Register (offset = 2158h) [reset = 0h]

QUEUE_21_C is shown in Figure 16-397 and described in Table 16-411.

**Figure 16-397. QUEUE_21_C Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| HEAD_TAIL | Reserved | | | | | | |
| W-0 | | | | | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| Reserved | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| Reserved | | PACKET_SIZE | | | | | |
| | | R/W-0 | | | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| PACKET_SIZE | | | | | | | |
| R/W-0 | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-411. QUEUE_21_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31 | HEAD_TAIL | W-0 | 0 | Head/Tail Push Control.<br>Set to zero in order to push packet onto tail of queue and set to one in order to push packet onto head of queue. |
| 13-0 | PACKET_SIZE | R/W-0 | 0 | packet_size This field indicates packet size and is assumed to be zero on each packet add unless the value is explicitly overwritten. This field indicates packet size for packet pop operation. |

### 16.4.7.122 QUEUE_21_D Register (offset = 215Ch) [reset = 0h]

QUEUE_21_D is shown in Figure 16-398 and described in Table 16-412.

**Figure 16-398. QUEUE_21_D Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |
| **23** | **22** | **21** | **20** | **19** | **18** | **17** | **16** |
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |
| **15** | **14** | **13** | **12** | **11** | **10** | **9** | **8** |
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |
| **7** | **6** | **5** | **4** | **3** | **2** | **1** | **0** |
| DESC_PTR | | | DESC_SIZE | | | | |
| R/W-0 | | | R/W-0 | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-412. QUEUE_21_D Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-5 | DESC_PTR | R/W-0 | 0 | Descriptor pointer.<br>It will be read as zero if the queue is empty.<br>It will indicate a<br>32-bit aligned address that points to a descriptor when the queue is not empty. |
| 4-0 | DESC_SIZE | R/W-0 | 0 | Descriptor Size.<br>It is encoded in<br>4-byte increments with values 0 to 31 representing 24 and so on to 148 bytes.<br>This field will return a 0x0 when an empty queue is read.<br>Queue Manager Queue N Registers D To save hardware resources, the queue manager internally stores descriptor size (desc_size) information in four bits.<br>However, register D has five LSBs that specify descriptor size.<br>As a consequence, the value of desc_size that is pushed may not be same as that is read during a pop.<br>The value that is read back is equal to always rounded to an odd number.<br>So, for even values, the value read back is one more than what was written.<br>For odd values, the value read back is same as the value that was written.<br>Note that this<br>5-bit field (desc_size) is unrelated to the code for size of descriptors in a descriptor region.<br>It is just a place holder for a<br>5-bit value that is maintained across the push and pop operations for every descriptor managed by the queue manager. |

### 16.4.7.123 QUEUE_22_A Register (offset = 2160h) [reset = 0h]

QUEUE_22_A is shown in Figure 16-399 and described in Table 16-413.

**Figure 16-399. QUEUE_22_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-413. QUEUE_22_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>This count is incremented by 1 whenever a packet is added to the queue.<br>This count is decremented by 1 whenever a packet is popped from the queue. |

### 16.4.7.124 QUEUE_22_B Register (offset = 2164h) [reset = 0h]

QUEUE_22_B is shown in Figure 16-400 and described in Table 16-414.

**Figure 16-400. QUEUE_22_B Register**

| 31–28 | 27–0 |
| --- | --- |
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-414. QUEUE_22_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
| --- | --- | --- | --- | --- |
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. |

### 16.4.7.125 QUEUE_22_C Register (offset = 2168h) [reset = 0h]

QUEUE_22_C is shown in Figure 16-401 and described in Table 16-415.

**Figure 16-401. QUEUE_22_C Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| HEAD_TAIL | Reserved | | | | | | |
| W-0 | | | | | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| Reserved | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| Reserved | | PACKET_SIZE | | | | | |
| | | R/W-0 | | | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| PACKET_SIZE | | | | | | | |
| R/W-0 | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-415. QUEUE_22_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31 | HEAD_TAIL | W-0 | 0 | Head/Tail Push Control.<br>Set to zero in order to push packet onto tail of queue and set to one in order to push packet onto head of queue. |
| 13-0 | PACKET_SIZE | R/W-0 | 0 | packet_size This field indicates packet size and is assumed to be zero on each packet add unless the value is explicitly overwritten. This field indicates packet size for packet pop operation. |

### 16.4.7.126 QUEUE_22_D Register (offset = 216Ch) [reset = 0h]

QUEUE_22_D is shown in Figure 16-402 and described in Table 16-416.

**Figure 16-402. QUEUE_22_D Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |
| **23** | **22** | **21** | **20** | **19** | **18** | **17** | **16** |
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |
| **15** | **14** | **13** | **12** | **11** | **10** | **9** | **8** |
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |
| **7** | **6** | **5** | **4** | **3** | **2** | **1** | **0** |
| DESC_PTR | | | DESC_SIZE | | | | |
| R/W-0 | | | R/W-0 | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-416. QUEUE_22_D Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-5 | DESC_PTR | R/W-0 | 0 | Descriptor pointer.<br>It will be read as zero if the queue is empty.<br>It will indicate a<br>32-bit aligned address that points to a descriptor when the queue is not empty. |
| 4-0 | DESC_SIZE | R/W-0 | 0 | Descriptor Size.<br>It is encoded in<br>4-byte increments with values 0 to 31 representing 24 and so on to 148 bytes.<br>This field will return a 0x0 when an empty queue is read.<br>Queue Manager Queue N Registers D To save hardware resources, the queue manager internally stores descriptor size (desc_size) information in four bits.<br>However, register D has five LSBs that specify descriptor size.<br>As a consequence, the value of desc_size that is pushed may not be same as that is read during a pop.<br>The value that is read back is equal to always rounded to an odd number.<br>So, for even values, the value read back is one more than what was written.<br>For odd values, the value read back is same as the value that was written.<br>Note that this<br>5-bit field (desc_size) is unrelated to the code for size of descriptors in a descriptor region.<br>It is just a place holder for a<br>5-bit value that is maintained across the push and pop operations for every descriptor managed by the queue manager. |

### 16.4.7.127 QUEUE_23_A Register (offset = 2170h) [reset = 0h]

QUEUE_23_A is shown in Figure 16-403 and described in Table 16-417.

**Figure 16-403. QUEUE_23_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-417. QUEUE_23_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>This count is incremented by 1 whenever a packet is added to the queue.<br>This count is decremented by 1 whenever a packet is popped from the queue. |

### 16.4.7.128 QUEUE_23_B Register (offset = 2174h) [reset = 0h]

QUEUE_23_B is shown in Figure 16-404 and described in Table 16-418.

**Figure 16-404. QUEUE_23_B Register**

| 31-28 | 27-0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-418. QUEUE_23_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. |

### 16.4.7.129 QUEUE_23_C Register (offset = 2178h) [reset = 0h]

QUEUE_23_C is shown in Figure 16-405 and described in Table 16-419.

**Figure 16-405. QUEUE_23_C Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| HEAD_TAIL | Reserved | | | | | | |
| W-0 | | | | | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| Reserved | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| Reserved | | PACKET_SIZE | | | | | |
| | | R/W-0 | | | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| PACKET_SIZE | | | | | | | |
| R/W-0 | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-419. QUEUE_23_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31 | HEAD_TAIL | W-0 | 0 | Head/Tail Push Control.<br>Set to zero in order to push packet onto tail of queue and set to one in order to push packet onto head of queue. |
| 13-0 | PACKET_SIZE | R/W-0 | 0 | packet_size This field indicates packet size and is assumed to be zero on each packet add unless the value is explicitly overwritten. This field indicates packet size for packet pop operation. |

### 16.4.7.130 QUEUE_23_D Register (offset = 217Ch) [reset = 0h]

QUEUE_23_D is shown in Figure 16-406 and described in Table 16-420.

**Figure 16-406. QUEUE_23_D Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | DESC_SIZE | | | | |
| R/W-0 | | | R/W-0 | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-420. QUEUE_23_D Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-5 | DESC_PTR | R/W-0 | 0 | Descriptor pointer.<br>It will be read as zero if the queue is empty.<br>It will indicate a<br>32-bit aligned address that points to a descriptor when the queue is not empty. |
| 4-0 | DESC_SIZE | R/W-0 | 0 | Descriptor Size.<br>It is encoded in<br>4-byte increments with values 0 to 31 representing 24 and so on to 148 bytes.<br>This field will return a 0x0 when an empty queue is read.<br>Queue Manager Queue N Registers D To save hardware resources, the queue manager internally stores descriptor size (desc_size) information in four bits.<br>However, register D has five LSBs that specify descriptor size.<br>As a consequence, the value of desc_size that is pushed may not be same as that is read during a pop.<br>The value that is read back is equal to always rounded to an odd number.<br>So, for even values, the value read back is one more than what was written.<br>For odd values, the value read back is same as the value that was written.<br>Note that this<br>5-bit field (desc_size) is unrelated to the code for size of descriptors in a descriptor region.<br>It is just a place holder for a<br>5-bit value that is maintained across the push and pop operations for every descriptor managed by the queue manager. |

### 16.4.7.131 QUEUE_24_A Register (offset = 2180h) [reset = 0h]

QUEUE_24_A is shown in Figure 16-407 and described in Table 16-421.

**Figure 16-407. QUEUE_24_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-421. QUEUE_24_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>This count is incremented by 1 whenever a packet is added to the queue.<br>This count is decremented by 1 whenever a packet is popped from the queue. |

### 16.4.7.132 QUEUE_24_B Register (offset = 2184h) [reset = 0h]

QUEUE_24_B is shown in Figure 16-408 and described in Table 16-422.

**Figure 16-408. QUEUE_24_B Register**

| 31-28 | 27-0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-422. QUEUE_24_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. |

#### 16.4.7.133 QUEUE_24_C Register (offset = 2188h) [reset = 0h]

QUEUE_24_C is shown in Figure 16-409 and described in Table 16-423.

**Figure 16-409. QUEUE_24_C Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| HEAD_TAIL | Reserved | | | | | | |
| W-0 | | | | | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| Reserved | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| Reserved | | PACKET_SIZE | | | | | |
| | | R/W-0 | | | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| PACKET_SIZE | | | | | | | |
| R/W-0 | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-423. QUEUE_24_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31 | HEAD_TAIL | W-0 | 0 | Head/Tail Push Control.<br>Set to zero in order to push packet onto tail of queue and set to one in order to push packet onto head of queue. |
| 13-0 | PACKET_SIZE | R/W-0 | 0 | packet_size This field indicates packet size and is assumed to be zero on each packet add unless the value is explicitly overwritten. This field indicates packet size for packet pop operation. |

### 16.4.7.134 QUEUE_24_D Register (offset = 218Ch) [reset = 0h]

QUEUE_24_D is shown in Figure 16-410 and described in Table 16-424.

**Figure 16-410. QUEUE_24_D Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | DESC_SIZE | | | | |
| R/W-0 | | | R/W-0 | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-424. QUEUE_24_D Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-5 | DESC_PTR | R/W-0 | 0 | Descriptor pointer.<br>It will be read as zero if the queue is empty.<br>It will indicate a<br>32-bit aligned address that points to a descriptor when the queue is not empty. |
| 4-0 | DESC_SIZE | R/W-0 | 0 | Descriptor Size.<br>It is encoded in<br>4-byte increments with values 0 to 31 representing 24 and so on to 148 bytes.<br>This field will return a 0x0 when an empty queue is read.<br>Queue Manager Queue N Registers D To save hardware resources, the queue manager internally stores descriptor size (desc_size) information in four bits.<br>However, register D has five LSBs that specify descriptor size.<br>As a consequence, the value of desc_size that is pushed may not be same as that is read during a pop.<br>The value that is read back is equal to always rounded to an odd number.<br>So, for even values, the value read back is one more than what was written.<br>For odd values, the value read back is same as the value that was written.<br>Note that this<br>5-bit field (desc_size) is unrelated to the code for size of descriptors in a descriptor region.<br>It is just a place holder for a<br>5-bit value that is maintained across the push and pop operations for every descriptor managed by the queue manager. |

### 16.4.7.135 QUEUE_25_A Register (offset = 2190h) [reset = 0h]

QUEUE_25_A is shown in Figure 16-411 and described in Table 16-425.

**Figure 16-411. QUEUE_25_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-425. QUEUE_25_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>This count is incremented by 1 whenever a packet is added to the queue.<br>This count is decremented by 1 whenever a packet is popped from the queue. |

### 16.4.7.136 QUEUE_25_B Register (offset = 2194h) [reset = 0h]

QUEUE_25_B is shown in Figure 16-412 and described in Table 16-426.

**Figure 16-412. QUEUE_25_B Register**

| 31–28 | 27–0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-426. QUEUE_25_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. |

### 16.4.7.137 QUEUE_25_C Register (offset = 2198h) [reset = 0h]

QUEUE_25_C is shown in Figure 16-413 and described in Table 16-427.

**Figure 16-413. QUEUE_25_C Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| HEAD_TAIL | Reserved | | | | | | |
| W-0 | | | | | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| Reserved | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| Reserved | | PACKET_SIZE | | | | | |
| | | R/W-0 | | | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| PACKET_SIZE | | | | | | | |
| R/W-0 | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-427. QUEUE_25_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31 | HEAD_TAIL | W-0 | 0 | Head/Tail Push Control.<br>Set to zero in order to push packet onto tail of queue and set to one in order to push packet onto head of queue. |
| 13-0 | PACKET_SIZE | R/W-0 | 0 | packet_size This field indicates packet size and is assumed to be zero on each packet add unless the value is explicitly overwritten. This field indicates packet size for packet pop operation. |

### 16.4.7.138 QUEUE_25_D Register (offset = 219Ch) [reset = 0h]

QUEUE_25_D is shown in Figure 16-414 and described in Table 16-428.

**Figure 16-414. QUEUE_25_D Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | DESC_SIZE | | | | |
| R/W-0 | | | R/W-0 | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-428. QUEUE_25_D Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-5 | DESC_PTR | R/W-0 | 0 | Descriptor pointer.<br>It will be read as zero if the queue is empty.<br>It will indicate a<br>32-bit aligned address that points to a descriptor when the queue is not empty. |
| 4-0 | DESC_SIZE | R/W-0 | 0 | Descriptor Size.<br>It is encoded in<br>4-byte increments with values 0 to 31 representing 24 and so on to 148 bytes.<br>This field will return a 0x0 when an empty queue is read.<br>Queue Manager Queue N Registers D To save hardware resources, the queue manager internally stores descriptor size (desc_size) information in four bits.<br>However, register D has five LSBs that specify descriptor size.<br>As a consequence, the value of desc_size that is pushed may not be same as that is read during a pop.<br>The value that is read back is equal to always rounded to an odd number.<br>So, for even values, the value read back is one more than what was written.<br>For odd values, the value read back is same as the value that was written.<br>Note that this<br>5-bit field (desc_size) is unrelated to the code for size of descriptors in a descriptor region.<br>It is just a place holder for a<br>5-bit value that is maintained across the push and pop operations for every descriptor managed by the queue manager. |

### 16.4.7.139 QUEUE_26_A Register (offset = 21A0h) [reset = 0h]

QUEUE_26_A is shown in Figure 16-415 and described in Table 16-429.

**Figure 16-415. QUEUE_26_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-429. QUEUE_26_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>This count is incremented by 1 whenever a packet is added to the queue.<br>This count is decremented by 1 whenever a packet is popped from the queue. |

### 16.4.7.140 QUEUE_26_B Register (offset = 21A4h) [reset = 0h]

QUEUE_26_B is shown in Figure 16-416 and described in Table 16-430.

**Figure 16-416. QUEUE_26_B Register**

| 31–28 | 27–0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-430. QUEUE_26_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. |

### 16.4.7.141 QUEUE_26_C Register (offset = 21A8h) [reset = 0h]

QUEUE_26_C is shown in Figure 16-417 and described in Table 16-431.

**Figure 16-417. QUEUE_26_C Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| HEAD_TAIL | Reserved | | | | | | |
| W-0 | | | | | | | |
| **23** | **22** | **21** | **20** | **19** | **18** | **17** | **16** |
| Reserved | | | | | | | |
| **15** | **14** | **13** | **12** | **11** | **10** | **9** | **8** |
| Reserved | | PACKET_SIZE | | | | | |
| | | R/W-0 | | | | | |
| **7** | **6** | **5** | **4** | **3** | **2** | **1** | **0** |
| PACKET_SIZE | | | | | | | |
| R/W-0 | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-431. QUEUE_26_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31 | HEAD_TAIL | W-0 | 0 | Head/Tail Push Control.<br>Set to zero in order to push packet onto tail of queue and set to one in order to push packet onto head of queue. |
| 13-0 | PACKET_SIZE | R/W-0 | 0 | packet_size This field indicates packet size and is assumed to be zero on each packet add unless the value is explicitly overwritten. This field indicates packet size for packet pop operation. |

### 16.4.7.142 QUEUE_26_D Register (offset = 21ACh) [reset = 0h]

QUEUE_26_D is shown in Figure 16-418 and described in Table 16-432.

**Figure 16-418. QUEUE_26_D Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | DESC_SIZE | | | | |
| R/W-0 | | | R/W-0 | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-432. QUEUE_26_D Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-5 | DESC_PTR | R/W-0 | 0 | Descriptor pointer.<br>It will be read as zero if the queue is empty.<br>It will indicate a<br>32-bit aligned address that points to a descriptor when the queue is not empty. |
| 4-0 | DESC_SIZE | R/W-0 | 0 | Descriptor Size.<br>It is encoded in<br>4-byte increments with values 0 to 31 representing 24 and so on to 148 bytes.<br>This field will return a 0x0 when an empty queue is read.<br>Queue Manager Queue N Registers D To save hardware resources, the queue manager internally stores descriptor size (desc_size) information in four bits.<br>However, register D has five LSBs that specify descriptor size.<br>As a consequence, the value of desc_size that is pushed may not be same as that is read during a pop.<br>The value that is read back is equal to always rounded to an odd number.<br>So, for even values, the value read back is one more than what was written.<br>For odd values, the value read back is same as the value that was written.<br>Note that this<br>5-bit field (desc_size) is unrelated to the code for size of descriptors in a descriptor region.<br>It is just a place holder for a<br>5-bit value that is maintained across the push and pop operations for every descriptor managed by the queue manager. |

### 16.4.7.143 QUEUE_27_A Register (offset = 21B0h) [reset = 0h]

QUEUE_27_A is shown in Figure 16-419 and described in Table 16-433.

**Figure 16-419. QUEUE_27_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-433. QUEUE_27_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>This count is incremented by 1 whenever a packet is added to the queue.<br>This count is decremented by 1 whenever a packet is popped from the queue. |

### 16.4.7.144 QUEUE_27_B Register (offset = 21B4h) [reset = 0h]

QUEUE_27_B is shown in Figure 16-420 and described in Table 16-434.

**Figure 16-420. QUEUE_27_B Register**

| 31-28 | 27-0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-434. QUEUE_27_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. |

### 16.4.7.145 QUEUE_27_C Register (offset = 21B8h) [reset = 0h]

QUEUE_27_C is shown in Figure 16-421 and described in Table 16-435.

**Figure 16-421. QUEUE_27_C Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| HEAD_TAIL | Reserved | | | | | | |
| W-0 | | | | | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| Reserved | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| Reserved | | PACKET_SIZE | | | | | |
| | | R/W-0 | | | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| PACKET_SIZE | | | | | | | |
| R/W-0 | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-435. QUEUE_27_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31 | HEAD_TAIL | W-0 | 0 | Head/Tail Push Control.<br>Set to zero in order to push packet onto tail of queue and set to one in order to push packet onto head of queue. |
| 13-0 | PACKET_SIZE | R/W-0 | 0 | packet_size This field indicates packet size and is assumed to be zero on each packet add unless the value is explicitly overwritten. This field indicates packet size for packet pop operation. |

### 16.4.7.146 QUEUE_27_D Register (offset = 21BCh) [reset = 0h]

QUEUE_27_D is shown in Figure 16-422 and described in Table 16-436.

**Figure 16-422. QUEUE_27_D Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | DESC_SIZE | | | | |
| R/W-0 | | | R/W-0 | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-436. QUEUE_27_D Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-5 | DESC_PTR | R/W-0 | 0 | Descriptor pointer.<br>It will be read as zero if the queue is empty.<br>It will indicate a<br>32-bit aligned address that points to a descriptor when the queue is not empty. |
| 4-0 | DESC_SIZE | R/W-0 | 0 | Descriptor Size.<br>It is encoded in<br>4-byte increments with values 0 to 31 representing 24 and so on to 148 bytes.<br>This field will return a 0x0 when an empty queue is read.<br>Queue Manager Queue N Registers D To save hardware resources, the queue manager internally stores descriptor size (desc_size) information in four bits.<br>However, register D has five LSBs that specify descriptor size.<br>As a consequence, the value of desc_size that is pushed may not be same as that is read during a pop.<br>The value that is read back is equal to always rounded to an odd number.<br>So, for even values, the value read back is one more than what was written.<br>For odd values, the value read back is same as the value that was written.<br>Note that this<br>5-bit field (desc_size) is unrelated to the code for size of descriptors in a descriptor region.<br>It is just a place holder for a<br>5-bit value that is maintained across the push and pop operations for every descriptor managed by the queue manager. |

### 16.4.7.147 QUEUE_28_A Register (offset = 21C0h) [reset = 0h]

QUEUE_28_A is shown in Figure 16-423 and described in Table 16-437.

**Figure 16-423. QUEUE_28_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-437. QUEUE_28_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>This count is incremented by 1 whenever a packet is added to the queue.<br>This count is decremented by 1 whenever a packet is popped from the queue. |

### 16.4.7.148 QUEUE_28_B Register (offset = 21C4h) [reset = 0h]

QUEUE_28_B is shown in Figure 16-424 and described in Table 16-438.

**Figure 16-424. QUEUE_28_B Register**

| 31–28 | 27–0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-438. QUEUE_28_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. |

### 16.4.7.149 QUEUE_28_C Register (offset = 21C8h) [reset = 0h]

QUEUE_28_C is shown in Figure 16-425 and described in Table 16-439.

**Figure 16-425. QUEUE_28_C Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| HEAD_TAIL | Reserved | | | | | | |
| W-0 | | | | | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| Reserved | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| Reserved | | PACKET_SIZE | | | | | |
| | | R/W-0 | | | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| PACKET_SIZE | | | | | | | |
| R/W-0 | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-439. QUEUE_28_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31 | HEAD_TAIL | W-0 | 0 | Head/Tail Push Control.<br>Set to zero in order to push packet onto tail of queue and set to one in order to push packet onto head of queue. |
| 13-0 | PACKET_SIZE | R/W-0 | 0 | packet_size This field indicates packet size and is assumed to be zero on each packet add unless the value is explicitly overwritten. This field indicates packet size for packet pop operation. |

### 16.4.7.150 QUEUE_28_D Register (offset = 21CCh) [reset = 0h]

QUEUE_28_D is shown in Figure 16-426 and described in Table 16-440.

**Figure 16-426. QUEUE_28_D Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | DESC_SIZE | | | | |
| R/W-0 | | | R/W-0 | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-440. QUEUE_28_D Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-5 | DESC_PTR | R/W-0 | 0 | Descriptor pointer.<br>It will be read as zero if the queue is empty.<br>It will indicate a<br>32-bit aligned address that points to a descriptor when the queue is not empty. |
| 4-0 | DESC_SIZE | R/W-0 | 0 | Descriptor Size.<br>It is encoded in<br>4-byte increments with values 0 to 31 representing 24 and so on to 148 bytes.<br>This field will return a 0x0 when an empty queue is read.<br>Queue Manager Queue N Registers D To save hardware resources, the queue manager internally stores descriptor size (desc_size) information in four bits.<br>However, register D has five LSBs that specify descriptor size.<br>As a consequence, the value of desc_size that is pushed may not be same as that is read during a pop.<br>The value that is read back is equal to always rounded to an odd number.<br>So, for even values, the value read back is one more than what was written.<br>For odd values, the value read back is same as the value that was written.<br>Note that this<br>5-bit field (desc_size) is unrelated to the code for size of descriptors in a descriptor region.<br>It is just a place holder for a<br>5-bit value that is maintained across the push and pop operations for every descriptor managed by the queue manager. |

### 16.4.7.151 QUEUE_29_A Register (offset = 21D0h) [reset = 0h]

QUEUE_29_A is shown in Figure 16-427 and described in Table 16-441.

**Figure 16-427. QUEUE_29_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-441. QUEUE_29_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>This count is incremented by 1 whenever a packet is added to the queue.<br>This count is decremented by 1 whenever a packet is popped from the queue. |

### 16.4.7.152 QUEUE_29_B Register (offset = 21D4h) [reset = 0h]

QUEUE_29_B is shown in Figure 16-428 and described in Table 16-442.

**Figure 16-428. QUEUE_29_B Register**

| 31-28 | 27-0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-442. QUEUE_29_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. |

### 16.4.7.153 QUEUE_29_C Register (offset = 21D8h) [reset = 0h]

QUEUE_29_C is shown in Figure 16-429 and described in Table 16-443.

**Figure 16-429. QUEUE_29_C Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| HEAD_TAIL | Reserved | | | | | | |
| W-0 | | | | | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| Reserved | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| Reserved | | PACKET_SIZE | | | | | |
| | | R/W-0 | | | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| PACKET_SIZE | | | | | | | |
| R/W-0 | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-443. QUEUE_29_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31 | HEAD_TAIL | W-0 | 0 | Head/Tail Push Control.<br>Set to zero in order to push packet onto tail of queue and set to one in order to push packet onto head of queue. |
| 13-0 | PACKET_SIZE | R/W-0 | 0 | packet_size This field indicates packet size and is assumed to be zero on each packet add unless the value is explicitly overwritten. This field indicates packet size for packet pop operation. |

### 16.4.7.154 QUEUE_29_D Register (offset = 21DCh) [reset = 0h]

QUEUE_29_D is shown in Figure 16-430 and described in Table 16-444.

**Figure 16-430. QUEUE_29_D Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | DESC_SIZE | | | | |
| R/W-0 | | | R/W-0 | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-444. QUEUE_29_D Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-5 | DESC_PTR | R/W-0 | 0 | Descriptor pointer.<br>It will be read as zero if the queue is empty.<br>It will indicate a<br>32-bit aligned address that points to a descriptor when the queue is not empty. |
| 4-0 | DESC_SIZE | R/W-0 | 0 | Descriptor Size.<br>It is encoded in<br>4-byte increments with values 0 to 31 representing 24 and so on to 148 bytes.<br>This field will return a 0x0 when an empty queue is read.<br>Queue Manager Queue N Registers D To save hardware resources, the queue manager internally stores descriptor size (desc_size) information in four bits.<br>However, register D has five LSBs that specify descriptor size.<br>As a consequence, the value of desc_size that is pushed may not be same as that is read during a pop.<br>The value that is read back is equal to always rounded to an odd number.<br>So, for even values, the value read back is one more than what was written.<br>For odd values, the value read back is same as the value that was written.<br>Note that this<br>5-bit field (desc_size) is unrelated to the code for size of descriptors in a descriptor region.<br>It is just a place holder for a<br>5-bit value that is maintained across the push and pop operations for every descriptor managed by the queue manager. |

### 16.4.7.155 QUEUE_30_A Register (offset = 21E0h) [reset = 0h]

QUEUE_30_A is shown in Figure 16-431 and described in Table 16-445.

**Figure 16-431. QUEUE_30_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-445. QUEUE_30_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>This count is incremented by 1 whenever a packet is added to the queue.<br>This count is decremented by 1 whenever a packet is popped from the queue. |

### 16.4.7.156 QUEUE_30_B Register (offset = 21E4h) [reset = 0h]

QUEUE_30_B is shown in Figure 16-432 and described in Table 16-446.

**Figure 16-432. QUEUE_30_B Register**

| 31–28 | 27–0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-446. QUEUE_30_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. |

### 16.4.7.157 QUEUE_30_C Register (offset = 21E8h) [reset = 0h]

QUEUE_30_C is shown in Figure 16-433 and described in Table 16-447.

**Figure 16-433. QUEUE_30_C Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| HEAD_TAIL | Reserved | | | | | | |
| W-0 | | | | | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| Reserved | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| Reserved | | PACKET_SIZE | | | | | |
| | | R/W-0 | | | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| PACKET_SIZE | | | | | | | |
| R/W-0 | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-447. QUEUE_30_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31 | HEAD_TAIL | W-0 | 0 | Head/Tail Push Control.<br>Set to zero in order to push packet onto tail of queue and set to one in order to push packet onto head of queue. |
| 13-0 | PACKET_SIZE | R/W-0 | 0 | packet_size This field indicates packet size and is assumed to be zero on each packet add unless the value is explicitly overwritten. This field indicates packet size for packet pop operation. |

### 16.4.7.158 QUEUE_30_D Register (offset = 21ECh) [reset = 0h]

QUEUE_30_D is shown in Figure 16-434 and described in Table 16-448.

**Figure 16-434. QUEUE_30_D Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | DESC_SIZE | | | | |
| R/W-0 | | | R/W-0 | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-448. QUEUE_30_D Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-5 | DESC_PTR | R/W-0 | 0 | Descriptor pointer.<br>It will be read as zero if the queue is empty.<br>It will indicate a<br>32-bit aligned address that points to a descriptor when the queue is not empty. |
| 4-0 | DESC_SIZE | R/W-0 | 0 | Descriptor Size.<br>It is encoded in<br>4-byte increments with values 0 to 31 representing 24 and so on to 148 bytes.<br>This field will return a 0x0 when an empty queue is read.<br>Queue Manager Queue N Registers D To save hardware resources, the queue manager internally stores descriptor size (desc_size) information in four bits.<br>However, register D has five LSBs that specify descriptor size.<br>As a consequence, the value of desc_size that is pushed may not be same as that is read during a pop.<br>The value that is read back is equal to always rounded to an odd number.<br>So, for even values, the value read back is one more than what was written.<br>For odd values, the value read back is same as the value that was written.<br>Note that this<br>5-bit field (desc_size) is unrelated to the code for size of descriptors in a descriptor region.<br>It is just a place holder for a<br>5-bit value that is maintained across the push and pop operations for every descriptor managed by the queue manager. |

### 16.4.7.159 QUEUE_31_A Register (offset = 21F0h) [reset = 0h]

QUEUE_31_A is shown in Figure 16-435 and described in Table 16-449.

**Figure 16-435. QUEUE_31_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-449. QUEUE_31_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>This count is incremented by 1 whenever a packet is added to the queue.<br>This count is decremented by 1 whenever a packet is popped from the queue. |

### 16.4.7.160 QUEUE_31_B Register (offset = 21F4h) [reset = 0h]

QUEUE_31_B is shown in Figure 16-436 and described in Table 16-450.

**Figure 16-436. QUEUE_31_B Register**

| 31–28 | 27–0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-450. QUEUE_31_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. |

### 16.4.7.161 QUEUE_31_C Register (offset = 21F8h) [reset = 0h]

QUEUE_31_C is shown in Figure 16-437 and described in Table 16-451.

**Figure 16-437. QUEUE_31_C Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| HEAD_TAIL | Reserved | | | | | | |
| W-0 | | | | | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| Reserved | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| Reserved | | PACKET_SIZE | | | | | |
| | | R/W-0 | | | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| PACKET_SIZE | | | | | | | |
| R/W-0 | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-451. QUEUE_31_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31 | HEAD_TAIL | W-0 | 0 | Head/Tail Push Control.<br>Set to zero in order to push packet onto tail of queue and set to one in order to push packet onto head of queue. |
| 13-0 | PACKET_SIZE | R/W-0 | 0 | packet_size This field indicates packet size and is assumed to be zero on each packet add unless the value is explicitly overwritten. This field indicates packet size for packet pop operation. |

### 16.4.7.162 QUEUE_31_D Register (offset = 21FCh) [reset = 0h]

QUEUE_31_D is shown in Figure 16-438 and described in Table 16-452.

**Figure 16-438. QUEUE_31_D Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | DESC_SIZE | | | | |
| R/W-0 | | | R/W-0 | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-452. QUEUE_31_D Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-5 | DESC_PTR | R/W-0 | 0 | Descriptor pointer.<br>It will be read as zero if the queue is empty.<br>It will indicate a<br>32-bit aligned address that points to a descriptor when the queue is not empty. |
| 4-0 | DESC_SIZE | R/W-0 | 0 | Descriptor Size.<br>It is encoded in<br>4-byte increments with values 0 to 31 representing 24 and so on to 148 bytes.<br>This field will return a 0x0 when an empty queue is read.<br>Queue Manager Queue N Registers D To save hardware resources, the queue manager internally stores descriptor size (desc_size) information in four bits.<br>However, register D has five LSBs that specify descriptor size.<br>As a consequence, the value of desc_size that is pushed may not be same as that is read during a pop.<br>The value that is read back is equal to always rounded to an odd number.<br>So, for even values, the value read back is one more than what was written.<br>For odd values, the value read back is same as the value that was written.<br>Note that this<br>5-bit field (desc_size) is unrelated to the code for size of descriptors in a descriptor region.<br>It is just a place holder for a<br>5-bit value that is maintained across the push and pop operations for every descriptor managed by the queue manager. |

### 16.4.7.163 QUEUE_32_A Register (offset = 2200h) [reset = 0h]

QUEUE_32_A is shown in Figure 16-439 and described in Table 16-453.

**Figure 16-439. QUEUE_32_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-453. QUEUE_32_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>This count is incremented by 1 whenever a packet is added to the queue.<br>This count is decremented by 1 whenever a packet is popped from the queue. |

### 16.4.7.164 QUEUE_32_B Register (offset = 2204h) [reset = 0h]

QUEUE_32_B is shown in Figure 16-440 and described in Table 16-454.

**Figure 16-440. QUEUE_32_B Register**

| 31–28 | 27–0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-454. QUEUE_32_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. |

### 16.4.7.165 QUEUE_32_C Register (offset = 2208h) [reset = 0h]

QUEUE_32_C is shown in Figure 16-441 and described in Table 16-455.

**Figure 16-441. QUEUE_32_C Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| HEAD_TAIL | Reserved | | | | | | |
| W-0 | | | | | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| Reserved | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| Reserved | | PACKET_SIZE | | | | | |
| | | R/W-0 | | | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| PACKET_SIZE | | | | | | | |
| R/W-0 | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-455. QUEUE_32_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31 | HEAD_TAIL | W-0 | 0 | Head/Tail Push Control.<br>Set to zero in order to push packet onto tail of queue and set to one in order to push packet onto head of queue. |
| 13-0 | PACKET_SIZE | R/W-0 | 0 | packet_size This field indicates packet size and is assumed to be zero on each packet add unless the value is explicitly overwritten. This field indicates packet size for packet pop operation. |

### 16.4.7.166 QUEUE_32_D Register (offset = 220Ch) [reset = 0h]

QUEUE_32_D is shown in Figure 16-442 and described in Table 16-456.

**Figure 16-442. QUEUE_32_D Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | DESC_SIZE | | | | |
| R/W-0 | | | R/W-0 | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-456. QUEUE_32_D Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-5 | DESC_PTR | R/W-0 | 0 | Descriptor pointer.<br>It will be read as zero if the queue is empty.<br>It will indicate a<br>32-bit aligned address that points to a descriptor when the queue is not empty. |
| 4-0 | DESC_SIZE | R/W-0 | 0 | Descriptor Size.<br>It is encoded in<br>4-byte increments with values 0 to 31 representing 24 and so on to 148 bytes.<br>This field will return a 0x0 when an empty queue is read.<br>Queue Manager Queue N Registers D To save hardware resources, the queue manager internally stores descriptor size (desc_size) information in four bits.<br>However, register D has five LSBs that specify descriptor size.<br>As a consequence, the value of desc_size that is pushed may not be same as that is read during a pop.<br>The value that is read back is equal to always rounded to an odd number.<br>So, for even values, the value read back is one more than what was written.<br>For odd values, the value read back is same as the value that was written.<br>Note that this<br>5-bit field (desc_size) is unrelated to the code for size of descriptors in a descriptor region.<br>It is just a place holder for a<br>5-bit value that is maintained across the push and pop operations for every descriptor managed by the queue manager. |

### 16.4.7.167 QUEUE_33_A Register (offset = 2210h) [reset = 0h]

QUEUE_33_A is shown in Figure 16-443 and described in Table 16-457.

**Figure 16-443. QUEUE_33_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-457. QUEUE_33_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>This count is incremented by 1 whenever a packet is added to the queue.<br>This count is decremented by 1 whenever a packet is popped from the queue. |

### 16.4.7.168 QUEUE_33_B Register (offset = 2214h) [reset = 0h]

QUEUE_33_B is shown in Figure 16-444 and described in Table 16-458.

**Figure 16-444. QUEUE_33_B Register**

| 31–28 | 27–0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-458. QUEUE_33_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. |

### 16.4.7.169 QUEUE_33_C Register (offset = 2218h) [reset = 0h]

QUEUE_33_C is shown in Figure 16-445 and described in Table 16-459.

**Figure 16-445. QUEUE_33_C Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| HEAD_TAIL | Reserved | | | | | | |
| W-0 | | | | | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| Reserved | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| Reserved | | PACKET_SIZE | | | | | |
| | | R/W-0 | | | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| PACKET_SIZE | | | | | | | |
| R/W-0 | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-459. QUEUE_33_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31 | HEAD_TAIL | W-0 | 0 | Head/Tail Push Control.<br>Set to zero in order to push packet onto tail of queue and set to one in order to push packet onto head of queue. |
| 13-0 | PACKET_SIZE | R/W-0 | 0 | packet_size This field indicates packet size and is assumed to be zero on each packet add unless the value is explicitly overwritten. This field indicates packet size for packet pop operation. |

### 16.4.7.170 QUEUE_33_D Register (offset = 221Ch) [reset = 0h]

QUEUE_33_D is shown in Figure 16-446 and described in Table 16-460.

**Figure 16-446. QUEUE_33_D Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |
| **23** | **22** | **21** | **20** | **19** | **18** | **17** | **16** |
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |
| **15** | **14** | **13** | **12** | **11** | **10** | **9** | **8** |
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |
| **7** | **6** | **5** | **4** | **3** | **2** | **1** | **0** |
| DESC_PTR | | | DESC_SIZE | | | | |
| R/W-0 | | | R/W-0 | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-460. QUEUE_33_D Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-5 | DESC_PTR | R/W-0 | 0 | Descriptor pointer.<br>It will be read as zero if the queue is empty.<br>It will indicate a<br>32-bit aligned address that points to a descriptor when the queue is not empty. |
| 4-0 | DESC_SIZE | R/W-0 | 0 | Descriptor Size.<br>It is encoded in<br>4-byte increments with values 0 to 31 representing 24 and so on to 148 bytes.<br>This field will return a 0x0 when an empty queue is read.<br>Queue Manager Queue N Registers D To save hardware resources, the queue manager internally stores descriptor size (desc_size) information in four bits.<br>However, register D has five LSBs that specify descriptor size.<br>As a consequence, the value of desc_size that is pushed may not be same as that is read during a pop.<br>The value that is read back is equal to always rounded to an odd number.<br>So, for even values, the value read back is one more than what was written.<br>For odd values, the value read back is same as the value that was written.<br>Note that this<br>5-bit field (desc_size) is unrelated to the code for size of descriptors in a descriptor region.<br>It is just a place holder for a<br>5-bit value that is maintained across the push and pop operations for every descriptor managed by the queue manager. |

### 16.4.7.171 QUEUE_34_A Register (offset = 2220h) [reset = 0h]

QUEUE_34_A is shown in Figure 16-447 and described in Table 16-461.

**Figure 16-447. QUEUE_34_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-461. QUEUE_34_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>This count is incremented by 1 whenever a packet is added to the queue.<br>This count is decremented by 1 whenever a packet is popped from the queue. |

### 16.4.7.172 QUEUE_34_B Register (offset = 2224h) [reset = 0h]

QUEUE_34_B is shown in Figure 16-448 and described in Table 16-462.

**Figure 16-448. QUEUE_34_B Register**

| 31–28 | 27–0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-462. QUEUE_34_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. |

### 16.4.7.173 QUEUE_34_C Register (offset = 2228h) [reset = 0h]

QUEUE_34_C is shown in Figure 16-449 and described in Table 16-463.

**Figure 16-449. QUEUE_34_C Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| HEAD_TAIL | Reserved | | | | | | |
| W-0 | | | | | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| Reserved | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| Reserved | | PACKET_SIZE | | | | | |
| | | R/W-0 | | | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| PACKET_SIZE | | | | | | | |
| R/W-0 | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-463. QUEUE_34_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31 | HEAD_TAIL | W-0 | 0 | Head/Tail Push Control.<br>Set to zero in order to push packet onto tail of queue and set to one in order to push packet onto head of queue. |
| 13-0 | PACKET_SIZE | R/W-0 | 0 | packet_size This field indicates packet size and is assumed to be zero on each packet add unless the value is explicitly overwritten. This field indicates packet size for packet pop operation. |

### 16.4.7.174 QUEUE_34_D Register (offset = 222Ch) [reset = 0h]

QUEUE_34_D is shown in Figure 16-450 and described in Table 16-464.

**Figure 16-450. QUEUE_34_D Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | DESC_SIZE | | | | |
| R/W-0 | | | R/W-0 | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-464. QUEUE_34_D Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-5 | DESC_PTR | R/W-0 | 0 | Descriptor pointer.<br>It will be read as zero if the queue is empty.<br>It will indicate a<br>32-bit aligned address that points to a descriptor when the queue is not empty. |
| 4-0 | DESC_SIZE | R/W-0 | 0 | Descriptor Size.<br>It is encoded in<br>4-byte increments with values 0 to 31 representing 24 and so on to 148 bytes.<br>This field will return a 0x0 when an empty queue is read.<br>Queue Manager Queue N Registers D To save hardware resources, the queue manager internally stores descriptor size (desc_size) information in four bits.<br>However, register D has five LSBs that specify descriptor size.<br>As a consequence, the value of desc_size that is pushed may not be same as that is read during a pop.<br>The value that is read back is equal to always rounded to an odd number.<br>So, for even values, the value read back is one more than what was written.<br>For odd values, the value read back is same as the value that was written.<br>Note that this<br>5-bit field (desc_size) is unrelated to the code for size of descriptors in a descriptor region.<br>It is just a place holder for a<br>5-bit value that is maintained across the push and pop operations for every descriptor managed by the queue manager. |

### 16.4.7.175 QUEUE_35_A Register (offset = 2230h) [reset = 0h]

QUEUE_35_A is shown in Figure 16-451 and described in Table 16-465.

**Figure 16-451. QUEUE_35_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-465. QUEUE_35_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>This count is incremented by 1 whenever a packet is added to the queue.<br>This count is decremented by 1 whenever a packet is popped from the queue. |

### 16.4.7.176 QUEUE_35_B Register (offset = 2234h) [reset = 0h]

QUEUE_35_B is shown in Figure 16-452 and described in Table 16-466.

**Figure 16-452. QUEUE_35_B Register**

| 31–28 | 27–0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-466. QUEUE_35_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. |

### 16.4.7.177 QUEUE_35_C Register (offset = 2238h) [reset = 0h]

QUEUE_35_C is shown in Figure 16-453 and described in Table 16-467.

**Figure 16-453. QUEUE_35_C Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| HEAD_TAIL | Reserved | | | | | | |
| W-0 | | | | | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| Reserved | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| Reserved | | PACKET_SIZE | | | | | |
| | | R/W-0 | | | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| PACKET_SIZE | | | | | | | |
| R/W-0 | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-467. QUEUE_35_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31 | HEAD_TAIL | W-0 | 0 | Head/Tail Push Control.<br>Set to zero in order to push packet onto tail of queue and set to one in order to push packet onto head of queue. |
| 13-0 | PACKET_SIZE | R/W-0 | 0 | packet_size This field indicates packet size and is assumed to be zero on each packet add unless the value is explicitly overwritten. This field indicates packet size for packet pop operation. |

### 16.4.7.178 QUEUE_35_D Register (offset = 223Ch) [reset = 0h]

QUEUE_35_D is shown in Figure 16-454 and described in Table 16-468.

**Figure 16-454. QUEUE_35_D Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | DESC_SIZE | | | | |
| R/W-0 | | | R/W-0 | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-468. QUEUE_35_D Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-5 | DESC_PTR | R/W-0 | 0 | Descriptor pointer.<br>It will be read as zero if the queue is empty.<br>It will indicate a<br>32-bit aligned address that points to a descriptor when the queue is not empty. |
| 4-0 | DESC_SIZE | R/W-0 | 0 | Descriptor Size.<br>It is encoded in<br>4-byte increments with values 0 to 31 representing 24 and so on to 148 bytes.<br>This field will return a 0x0 when an empty queue is read.<br>Queue Manager Queue N Registers D To save hardware resources, the queue manager internally stores descriptor size (desc_size) information in four bits.<br>However, register D has five LSBs that specify descriptor size.<br>As a consequence, the value of desc_size that is pushed may not be same as that is read during a pop.<br>The value that is read back is equal to always rounded to an odd number.<br>So, for even values, the value read back is one more than what was written.<br>For odd values, the value read back is same as the value that was written.<br>Note that this<br>5-bit field (desc_size) is unrelated to the code for size of descriptors in a descriptor region.<br>It is just a place holder for a<br>5-bit value that is maintained across the push and pop operations for every descriptor managed by the queue manager. |

### 16.4.7.179 QUEUE_36_A Register (offset = 2240h) [reset = 0h]

QUEUE_36_A is shown in Figure 16-455 and described in Table 16-469.

**Figure 16-455. QUEUE_36_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-469. QUEUE_36_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>This count is incremented by 1 whenever a packet is added to the queue.<br>This count is decremented by 1 whenever a packet is popped from the queue. |

### 16.4.7.180 QUEUE_36_B Register (offset = 2244h) [reset = 0h]

QUEUE_36_B is shown in Figure 16-456 and described in Table 16-470.

**Figure 16-456. QUEUE_36_B Register**

| 31–28 | 27–0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-470. QUEUE_36_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. |

#### 16.4.7.181 QUEUE_36_C Register (offset = 2248h) [reset = 0h]

QUEUE_36_C is shown in Figure 16-457 and described in Table 16-471.

**Figure 16-457. QUEUE_36_C Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| HEAD_TAIL | Reserved | | | | | | |
| W-0 | | | | | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| Reserved | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| Reserved | | PACKET_SIZE | | | | | |
| | | R/W-0 | | | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| PACKET_SIZE | | | | | | | |
| R/W-0 | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-471. QUEUE_36_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31 | HEAD_TAIL | W-0 | 0 | Head/Tail Push Control.<br>Set to zero in order to push packet onto tail of queue and set to one in order to push packet onto head of queue. |
| 13-0 | PACKET_SIZE | R/W-0 | 0 | packet_size This field indicates packet size and is assumed to be zero on each packet add unless the value is explicitly overwritten. This field indicates packet size for packet pop operation. |

### 16.4.7.182 QUEUE_36_D Register (offset = 224Ch) [reset = 0h]

QUEUE_36_D is shown in Figure 16-458 and described in Table 16-472.

**Figure 16-458. QUEUE_36_D Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | DESC_SIZE | | | | |
| R/W-0 | | | R/W-0 | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-472. QUEUE_36_D Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-5 | DESC_PTR | R/W-0 | 0 | Descriptor pointer.<br>It will be read as zero if the queue is empty.<br>It will indicate a<br>32-bit aligned address that points to a descriptor when the queue is not empty. |
| 4-0 | DESC_SIZE | R/W-0 | 0 | Descriptor Size.<br>It is encoded in<br>4-byte increments with values 0 to 31 representing 24 and so on to 148 bytes.<br>This field will return a 0x0 when an empty queue is read.<br>Queue Manager Queue N Registers D To save hardware resources, the queue manager internally stores descriptor size (desc_size) information in four bits.<br>However, register D has five LSBs that specify descriptor size.<br>As a consequence, the value of desc_size that is pushed may not be same as that is read during a pop.<br>The value that is read back is equal to always rounded to an odd number.<br>So, for even values, the value read back is one more than what was written.<br>For odd values, the value read back is same as the value that was written.<br>Note that this<br>5-bit field (desc_size) is unrelated to the code for size of descriptors in a descriptor region.<br>It is just a place holder for a<br>5-bit value that is maintained across the push and pop operations for every descriptor managed by the queue manager. |

### 16.4.7.183 QUEUE_37_A Register (offset = 2250h) [reset = 0h]

QUEUE_37_A is shown in Figure 16-459 and described in Table 16-473.

**Figure 16-459. QUEUE_37_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-473. QUEUE_37_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>This count is incremented by 1 whenever a packet is added to the queue.<br>This count is decremented by 1 whenever a packet is popped from the queue. |

### 16.4.7.184 QUEUE_37_B Register (offset = 2254h) [reset = 0h]

QUEUE_37_B is shown in Figure 16-460 and described in Table 16-474.

**Figure 16-460. QUEUE_37_B Register**

| 31–28 | 27–0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-474. QUEUE_37_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. |

### 16.4.7.185 QUEUE_37_C Register (offset = 2258h) [reset = 0h]

QUEUE_37_C is shown in Figure 16-461 and described in Table 16-475.

**Figure 16-461. QUEUE_37_C Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| HEAD_TAIL | Reserved | | | | | | |
| W-0 | | | | | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| Reserved | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| Reserved | | PACKET_SIZE | | | | | |
| | | R/W-0 | | | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| PACKET_SIZE | | | | | | | |
| R/W-0 | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-475. QUEUE_37_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31 | HEAD_TAIL | W-0 | 0 | Head/Tail Push Control.<br>Set to zero in order to push packet onto tail of queue and set to one in order to push packet onto head of queue. |
| 13-0 | PACKET_SIZE | R/W-0 | 0 | packet_size This field indicates packet size and is assumed to be zero on each packet add unless the value is explicitly overwritten. This field indicates packet size for packet pop operation. |

### 16.4.7.186 QUEUE_37_D Register (offset = 225Ch) [reset = 0h]

QUEUE_37_D is shown in Figure 16-462 and described in Table 16-476.

**Figure 16-462. QUEUE_37_D Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |
| **23** | **22** | **21** | **20** | **19** | **18** | **17** | **16** |
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |
| **15** | **14** | **13** | **12** | **11** | **10** | **9** | **8** |
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |
| **7** | **6** | **5** | **4** | **3** | **2** | **1** | **0** |
| DESC_PTR | | | DESC_SIZE | | | | |
| R/W-0 | | | R/W-0 | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-476. QUEUE_37_D Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-5 | DESC_PTR | R/W-0 | 0 | Descriptor pointer.<br>It will be read as zero if the queue is empty.<br>It will indicate a<br>32-bit aligned address that points to a descriptor when the queue is not empty. |
| 4-0 | DESC_SIZE | R/W-0 | 0 | Descriptor Size.<br>It is encoded in<br>4-byte increments with values 0 to 31 representing 24 and so on to 148 bytes.<br>This field will return a 0x0 when an empty queue is read.<br>Queue Manager Queue N Registers D To save hardware resources, the queue manager internally stores descriptor size (desc_size) information in four bits.<br>However, register D has five LSBs that specify descriptor size.<br>As a consequence, the value of desc_size that is pushed may not be same as that is read during a pop.<br>The value that is read back is equal to always rounded to an odd number.<br>So, for even values, the value read back is one more than what was written.<br>For odd values, the value read back is same as the value that was written.<br>Note that this<br>5-bit field (desc_size) is unrelated to the code for size of descriptors in a descriptor region.<br>It is just a place holder for a<br>5-bit value that is maintained across the push and pop operations for every descriptor managed by the queue manager. |

### 16.4.7.187 QUEUE_38_A Register (offset = 2260h) [reset = 0h]

QUEUE_38_A is shown in Figure 16-463 and described in Table 16-477.

**Figure 16-463. QUEUE_38_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-477. QUEUE_38_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>This count is incremented by 1 whenever a packet is added to the queue.<br>This count is decremented by 1 whenever a packet is popped from the queue. |

#### 16.4.7.188 QUEUE_38_B Register (offset = 2264h) [reset = 0h]

QUEUE_38_B is shown in Figure 16-464 and described in Table 16-478.

**Figure 16-464. QUEUE_38_B Register**

| 31–28 | 27–0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-478. QUEUE_38_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. |

### 16.4.7.189 QUEUE_38_C Register (offset = 2268h) [reset = 0h]

QUEUE_38_C is shown in Figure 16-465 and described in Table 16-479.

**Figure 16-465. QUEUE_38_C Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| HEAD_TAIL | Reserved | | | | | | |
| W-0 | | | | | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| Reserved | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| Reserved | | PACKET_SIZE | | | | | |
| | | R/W-0 | | | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| PACKET_SIZE | | | | | | | |
| R/W-0 | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-479. QUEUE_38_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31 | HEAD_TAIL | W-0 | 0 | Head/Tail Push Control.<br>Set to zero in order to push packet onto tail of queue and set to one in order to push packet onto head of queue. |
| 13-0 | PACKET_SIZE | R/W-0 | 0 | packet_size This field indicates packet size and is assumed to be zero on each packet add unless the value is explicitly overwritten. This field indicates packet size for packet pop operation. |

### 16.4.7.190 QUEUE_38_D Register (offset = 226Ch) [reset = 0h]

QUEUE_38_D is shown in Figure 16-466 and described in Table 16-480.

**Figure 16-466. QUEUE_38_D Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |
| **23** | **22** | **21** | **20** | **19** | **18** | **17** | **16** |
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |
| **15** | **14** | **13** | **12** | **11** | **10** | **9** | **8** |
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |
| **7** | **6** | **5** | **4** | **3** | **2** | **1** | **0** |
| DESC_PTR | | | DESC_SIZE | | | | |
| R/W-0 | | | R/W-0 | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-480. QUEUE_38_D Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-5 | DESC_PTR | R/W-0 | 0 | Descriptor pointer.<br>It will be read as zero if the queue is empty.<br>It will indicate a<br>32-bit aligned address that points to a descriptor when the queue is not empty. |
| 4-0 | DESC_SIZE | R/W-0 | 0 | Descriptor Size.<br>It is encoded in<br>4-byte increments with values 0 to 31 representing 24 and so on to 148 bytes.<br>This field will return a 0x0 when an empty queue is read.<br>Queue Manager Queue N Registers D To save hardware resources, the queue manager internally stores descriptor size (desc_size) information in four bits.<br>However, register D has five LSBs that specify descriptor size.<br>As a consequence, the value of desc_size that is pushed may not be same as that is read during a pop.<br>The value that is read back is equal to always rounded to an odd number.<br>So, for even values, the value read back is one more than what was written.<br>For odd values, the value read back is same as the value that was written.<br>Note that this<br>5-bit field (desc_size) is unrelated to the code for size of descriptors in a descriptor region.<br>It is just a place holder for a<br>5-bit value that is maintained across the push and pop operations for every descriptor managed by the queue manager. |

### 16.4.7.191 QUEUE_39_A Register (offset = 2270h) [reset = 0h]

QUEUE_39_A is shown in Figure 16-467 and described in Table 16-481.

**Figure 16-467. QUEUE_39_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-481. QUEUE_39_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>This count is incremented by 1 whenever a packet is added to the queue.<br>This count is decremented by 1 whenever a packet is popped from the queue. |

### 16.4.7.192 QUEUE_39_B Register (offset = 2274h) [reset = 0h]

QUEUE_39_B is shown in Figure 16-468 and described in Table 16-482.

**Figure 16-468. QUEUE_39_B Register**

| 31–28 | 27–0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-482. QUEUE_39_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. |

### 16.4.7.193 QUEUE_39_C Register (offset = 2278h) [reset = 0h]

QUEUE_39_C is shown in Figure 16-469 and described in Table 16-483.

**Figure 16-469. QUEUE_39_C Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| HEAD_TAIL | Reserved | | | | | | |
| W-0 | | | | | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| Reserved | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| Reserved | | PACKET_SIZE | | | | | |
| | | R/W-0 | | | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| PACKET_SIZE | | | | | | | |
| R/W-0 | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-483. QUEUE_39_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31 | HEAD_TAIL | W-0 | 0 | Head/Tail Push Control.<br>Set to zero in order to push packet onto tail of queue and set to one in order to push packet onto head of queue. |
| 13-0 | PACKET_SIZE | R/W-0 | 0 | packet_size This field indicates packet size and is assumed to be zero on each packet add unless the value is explicitly overwritten. This field indicates packet size for packet pop operation. |

### 16.4.7.194 QUEUE_39_D Register (offset = 227Ch) [reset = 0h]

QUEUE_39_D is shown in Figure 16-470 and described in Table 16-484.

**Figure 16-470. QUEUE_39_D Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | DESC_SIZE | | | | |
| R/W-0 | | | R/W-0 | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-484. QUEUE_39_D Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-5 | DESC_PTR | R/W-0 | 0 | Descriptor pointer.<br>It will be read as zero if the queue is empty.<br>It will indicate a<br>32-bit aligned address that points to a descriptor when the queue is not empty. |
| 4-0 | DESC_SIZE | R/W-0 | 0 | Descriptor Size.<br>It is encoded in<br>4-byte increments with values 0 to 31 representing 24 and so on to 148 bytes.<br>This field will return a 0x0 when an empty queue is read.<br>Queue Manager Queue N Registers D To save hardware resources, the queue manager internally stores descriptor size (desc_size) information in four bits.<br>However, register D has five LSBs that specify descriptor size.<br>As a consequence, the value of desc_size that is pushed may not be same as that is read during a pop.<br>The value that is read back is equal to always rounded to an odd number.<br>So, for even values, the value read back is one more than what was written.<br>For odd values, the value read back is same as the value that was written.<br>Note that this<br>5-bit field (desc_size) is unrelated to the code for size of descriptors in a descriptor region.<br>It is just a place holder for a<br>5-bit value that is maintained across the push and pop operations for every descriptor managed by the queue manager. |

### 16.4.7.195 QUEUE_40_A Register (offset = 2280h) [reset = 0h]

QUEUE_40_A is shown in Figure 16-471 and described in Table 16-485.

**Figure 16-471. QUEUE_40_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-485. QUEUE_40_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>This count is incremented by 1 whenever a packet is added to the queue.<br>This count is decremented by 1 whenever a packet is popped from the queue. |

### 16.4.7.196 QUEUE_40_B Register (offset = 2284h) [reset = 0h]

QUEUE_40_B is shown in Figure 16-472 and described in Table 16-486.

**Figure 16-472. QUEUE_40_B Register**

| 31–28 | 27–0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-486. QUEUE_40_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. |

### 16.4.7.197 QUEUE_40_C Register (offset = 2288h) [reset = 0h]

QUEUE_40_C is shown in Figure 16-473 and described in Table 16-487.

**Figure 16-473. QUEUE_40_C Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| HEAD_TAIL | Reserved | | | | | | |
| W-0 | | | | | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| Reserved | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| Reserved | | PACKET_SIZE | | | | | |
| | | R/W-0 | | | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| PACKET_SIZE | | | | | | | |
| R/W-0 | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-487. QUEUE_40_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31 | HEAD_TAIL | W-0 | 0 | Head/Tail Push Control.<br>Set to zero in order to push packet onto tail of queue and set to one in order to push packet onto head of queue. |
| 13-0 | PACKET_SIZE | R/W-0 | 0 | packet_size This field indicates packet size and is assumed to be zero on each packet add unless the value is explicitly overwritten.<br>This field indicates packet size for packet pop operation. |

### 16.4.7.198 QUEUE_40_D Register (offset = 228Ch) [reset = 0h]

QUEUE_40_D is shown in Figure 16-474 and described in Table 16-488.

**Figure 16-474. QUEUE_40_D Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | DESC_SIZE | | | | |
| R/W-0 | | | R/W-0 | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-488. QUEUE_40_D Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-5 | DESC_PTR | R/W-0 | 0 | Descriptor pointer.<br>It will be read as zero if the queue is empty.<br>It will indicate a<br>32-bit aligned address that points to a descriptor when the queue is not empty. |
| 4-0 | DESC_SIZE | R/W-0 | 0 | Descriptor Size.<br>It is encoded in<br>4-byte increments with values 0 to 31 representing 24 and so on to 148 bytes.<br>This field will return a 0x0 when an empty queue is read.<br>Queue Manager Queue N Registers D To save hardware resources, the queue manager internally stores descriptor size (desc_size) information in four bits.<br>However, register D has five LSBs that specify descriptor size.<br>As a consequence, the value of desc_size that is pushed may not be same as that is read during a pop.<br>The value that is read back is equal to always rounded to an odd number.<br>So, for even values, the value read back is one more than what was written.<br>For odd values, the value read back is same as the value that was written.<br>Note that this<br>5-bit field (desc_size) is unrelated to the code for size of descriptors in a descriptor region.<br>It is just a place holder for a<br>5-bit value that is maintained across the push and pop operations for every descriptor managed by the queue manager. |

### 16.4.7.199 QUEUE_41_A Register (offset = 2290h) [reset = 0h]

QUEUE_41_A is shown in Figure 16-475 and described in Table 16-489.

**Figure 16-475. QUEUE_41_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-489. QUEUE_41_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>This count is incremented by 1 whenever a packet is added to the queue.<br>This count is decremented by 1 whenever a packet is popped from the queue. |

### 16.4.7.200 QUEUE_41_B Register (offset = 2294h) [reset = 0h]

QUEUE_41_B is shown in Figure 16-476 and described in Table 16-490.

**Figure 16-476. QUEUE_41_B Register**

| 31–28 | 27–0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-490. QUEUE_41_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. |

### 16.4.7.201 QUEUE_41_C Register (offset = 2298h) [reset = 0h]

QUEUE_41_C is shown in Figure 16-477 and described in Table 16-491.

**Figure 16-477. QUEUE_41_C Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| HEAD_TAIL | Reserved | | | | | | |
| W-0 | | | | | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| Reserved | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| Reserved | | PACKET_SIZE | | | | | |
| | | R/W-0 | | | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| PACKET_SIZE | | | | | | | |
| R/W-0 | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-491. QUEUE_41_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31 | HEAD_TAIL | W-0 | 0 | Head/Tail Push Control.<br>Set to zero in order to push packet onto tail of queue and set to one in order to push packet onto head of queue. |
| 13-0 | PACKET_SIZE | R/W-0 | 0 | packet_size This field indicates packet size and is assumed to be zero on each packet add unless the value is explicitly overwritten. This field indicates packet size for packet pop operation. |

### 16.4.7.202 QUEUE_41_D Register (offset = 229Ch) [reset = 0h]

QUEUE_41_D is shown in Figure 16-478 and described in Table 16-492.

**Figure 16-478. QUEUE_41_D Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |
| **23** | **22** | **21** | **20** | **19** | **18** | **17** | **16** |
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |
| **15** | **14** | **13** | **12** | **11** | **10** | **9** | **8** |
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |
| **7** | **6** | **5** | **4** | **3** | **2** | **1** | **0** |
| DESC_PTR | | | DESC_SIZE | | | | |
| R/W-0 | | | R/W-0 | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-492. QUEUE_41_D Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-5 | DESC_PTR | R/W-0 | 0 | Descriptor pointer.<br>It will be read as zero if the queue is empty.<br>It will indicate a<br>32-bit aligned address that points to a descriptor when the queue is not empty. |
| 4-0 | DESC_SIZE | R/W-0 | 0 | Descriptor Size.<br>It is encoded in<br>4-byte increments with values 0 to 31 representing 24 and so on to 148 bytes.<br>This field will return a 0x0 when an empty queue is read.<br>Queue Manager Queue N Registers D To save hardware resources, the queue manager internally stores descriptor size (desc_size) information in four bits.<br>However, register D has five LSBs that specify descriptor size.<br>As a consequence, the value of desc_size that is pushed may not be same as that is read during a pop.<br>The value that is read back is equal to always rounded to an odd number.<br>So, for even values, the value read back is one more than what was written.<br>For odd values, the value read back is same as the value that was written.<br>Note that this<br>5-bit field (desc_size) is unrelated to the code for size of descriptors in a descriptor region.<br>It is just a place holder for a<br>5-bit value that is maintained across the push and pop operations for every descriptor managed by the queue manager. |

### 16.4.7.203 QUEUE_42_A Register (offset = 22A0h) [reset = 0h]

QUEUE_42_A is shown in Figure 16-479 and described in Table 16-493.

**Figure 16-479. QUEUE_42_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-493. QUEUE_42_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>This count is incremented by 1 whenever a packet is added to the queue.<br>This count is decremented by 1 whenever a packet is popped from the queue. |

### 16.4.7.204 QUEUE_42_B Register (offset = 22A4h) [reset = 0h]

QUEUE_42_B is shown in Figure 16-480 and described in Table 16-494.

**Figure 16-480. QUEUE_42_B Register**

| 31–28 | 27–0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-494. QUEUE_42_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. |

### 16.4.7.205 QUEUE_42_C Register (offset = 22A8h) [reset = 0h]

QUEUE_42_C is shown in Figure 16-481 and described in Table 16-495.

**Figure 16-481. QUEUE_42_C Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| HEAD_TAIL | Reserved | | | | | | |
| W-0 | | | | | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| Reserved | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| Reserved | | PACKET_SIZE | | | | | |
| | | R/W-0 | | | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| PACKET_SIZE | | | | | | | |
| R/W-0 | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-495. QUEUE_42_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31 | HEAD_TAIL | W-0 | 0 | Head/Tail Push Control.<br>Set to zero in order to push packet onto tail of queue and set to one in order to push packet onto head of queue. |
| 13-0 | PACKET_SIZE | R/W-0 | 0 | packet_size This field indicates packet size and is assumed to be zero on each packet add unless the value is explicitly overwritten. This field indicates packet size for packet pop operation. |

### 16.4.7.206 QUEUE_42_D Register (offset = 22ACh) [reset = 0h]

QUEUE_42_D is shown in Figure 16-482 and described in Table 16-496.

**Figure 16-482. QUEUE_42_D Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |
| **23** | **22** | **21** | **20** | **19** | **18** | **17** | **16** |
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |
| **15** | **14** | **13** | **12** | **11** | **10** | **9** | **8** |
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |
| **7** | **6** | **5** | **4** | **3** | **2** | **1** | **0** |
| DESC_PTR | | | DESC_SIZE | | | | |
| R/W-0 | | | R/W-0 | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-496. QUEUE_42_D Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-5 | DESC_PTR | R/W-0 | 0 | Descriptor pointer.<br>It will be read as zero if the queue is empty.<br>It will indicate a<br>32-bit aligned address that points to a descriptor when the queue is not empty. |
| 4-0 | DESC_SIZE | R/W-0 | 0 | Descriptor Size.<br>It is encoded in<br>4-byte increments with values 0 to 31 representing 24 and so on to 148 bytes.<br>This field will return a 0x0 when an empty queue is read.<br>Queue Manager Queue N Registers D To save hardware resources, the queue manager internally stores descriptor size (desc_size) information in four bits.<br>However, register D has five LSBs that specify descriptor size.<br>As a consequence, the value of desc_size that is pushed may not be same as that is read during a pop.<br>The value that is read back is equal to always rounded to an odd number.<br>So, for even values, the value read back is one more than what was written.<br>For odd values, the value read back is same as the value that was written.<br>Note that this<br>5-bit field (desc_size) is unrelated to the code for size of descriptors in a descriptor region.<br>It is just a place holder for a<br>5-bit value that is maintained across the push and pop operations for every descriptor managed by the queue manager. |

### 16.4.7.207 QUEUE_43_A Register (offset = 22B0h) [reset = 0h]

QUEUE_43_A is shown in Figure 16-483 and described in Table 16-497.

**Figure 16-483. QUEUE_43_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-497. QUEUE_43_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>This count is incremented by 1 whenever a packet is added to the queue.<br>This count is decremented by 1 whenever a packet is popped from the queue. |

### 16.4.7.208 QUEUE_43_B Register (offset = 22B4h) [reset = 0h]

QUEUE_43_B is shown in Figure 16-484 and described in Table 16-498.

**Figure 16-484. QUEUE_43_B Register**

| 31–28 | 27–0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-498. QUEUE_43_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. |

### 16.4.7.209 QUEUE_43_C Register (offset = 22B8h) [reset = 0h]

QUEUE_43_C is shown in Figure 16-485 and described in Table 16-499.

**Figure 16-485. QUEUE_43_C Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| HEAD_TAIL | Reserved | | | | | | |
| W-0 | | | | | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| Reserved | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| Reserved | | PACKET_SIZE | | | | | |
| | | R/W-0 | | | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| PACKET_SIZE | | | | | | | |
| R/W-0 | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-499. QUEUE_43_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31 | HEAD_TAIL | W-0 | 0 | Head/Tail Push Control.<br>Set to zero in order to push packet onto tail of queue and set to one in order to push packet onto head of queue. |
| 13-0 | PACKET_SIZE | R/W-0 | 0 | packet_size This field indicates packet size and is assumed to be zero on each packet add unless the value is explicitly overwritten. This field indicates packet size for packet pop operation. |

### 16.4.7.210 QUEUE_43_D Register (offset = 22BCh) [reset = 0h]

QUEUE_43_D is shown in Figure 16-486 and described in Table 16-500.

**Figure 16-486. QUEUE_43_D Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |
| **23** | **22** | **21** | **20** | **19** | **18** | **17** | **16** |
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |
| **15** | **14** | **13** | **12** | **11** | **10** | **9** | **8** |
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |
| **7** | **6** | **5** | **4** | **3** | **2** | **1** | **0** |
| DESC_PTR | | | DESC_SIZE | | | | |
| R/W-0 | | | R/W-0 | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-500. QUEUE_43_D Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-5 | DESC_PTR | R/W-0 | 0 | Descriptor pointer.<br>It will be read as zero if the queue is empty.<br>It will indicate a<br>32-bit aligned address that points to a descriptor when the queue is not empty. |
| 4-0 | DESC_SIZE | R/W-0 | 0 | Descriptor Size.<br>It is encoded in<br>4-byte increments with values 0 to 31 representing 24 and so on to 148 bytes.<br>This field will return a 0x0 when an empty queue is read.<br>Queue Manager Queue N Registers D To save hardware resources, the queue manager internally stores descriptor size (desc_size) information in four bits.<br>However, register D has five LSBs that specify descriptor size.<br>As a consequence, the value of desc_size that is pushed may not be same as that is read during a pop.<br>The value that is read back is equal to always rounded to an odd number.<br>So, for even values, the value read back is one more than what was written.<br>For odd values, the value read back is same as the value that was written.<br>Note that this<br>5-bit field (desc_size) is unrelated to the code for size of descriptors in a descriptor region.<br>It is just a place holder for a<br>5-bit value that is maintained across the push and pop operations for every descriptor managed by the queue manager. |

### 16.4.7.211 QUEUE_44_A Register (offset = 22C0h) [reset = 0h]

QUEUE_44_A is shown in Figure 16-487 and described in Table 16-501.

**Figure 16-487. QUEUE_44_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-501. QUEUE_44_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>This count is incremented by 1 whenever a packet is added to the queue.<br>This count is decremented by 1 whenever a packet is popped from the queue. |

### 16.4.7.212 QUEUE_44_B Register (offset = 22C4h) [reset = 0h]

QUEUE_44_B is shown in Figure 16-488 and described in Table 16-502.

**Figure 16-488. QUEUE_44_B Register**

| 31-28 | 27-0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-502. QUEUE_44_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. |

### 16.4.7.213 QUEUE_44_C Register (offset = 22C8h) [reset = 0h]

QUEUE_44_C is shown in Figure 16-489 and described in Table 16-503.

**Figure 16-489. QUEUE_44_C Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| HEAD_TAIL | Reserved | | | | | | |
| W-0 | | | | | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| Reserved | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| Reserved | | PACKET_SIZE | | | | | |
| | | R/W-0 | | | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| PACKET_SIZE | | | | | | | |
| R/W-0 | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-503. QUEUE_44_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31 | HEAD_TAIL | W-0 | 0 | Head/Tail Push Control.<br>Set to zero in order to push packet onto tail of queue and set to one in order to push packet onto head of queue. |
| 13-0 | PACKET_SIZE | R/W-0 | 0 | packet_size This field indicates packet size and is assumed to be zero on each packet add unless the value is explicitly overwritten. This field indicates packet size for packet pop operation. |

### 16.4.7.214 QUEUE_44_D Register (offset = 22CCh) [reset = 0h]

QUEUE_44_D is shown in Figure 16-490 and described in Table 16-504.

**Figure 16-490. QUEUE_44_D Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |
| **23** | **22** | **21** | **20** | **19** | **18** | **17** | **16** |
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |
| **15** | **14** | **13** | **12** | **11** | **10** | **9** | **8** |
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |
| **7** | **6** | **5** | **4** | **3** | **2** | **1** | **0** |
| DESC_PTR | | | DESC_SIZE | | | | |
| R/W-0 | | | R/W-0 | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-504. QUEUE_44_D Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-5 | DESC_PTR | R/W-0 | 0 | Descriptor pointer.<br>It will be read as zero if the queue is empty.<br>It will indicate a<br>32-bit aligned address that points to a descriptor when the queue is not empty. |
| 4-0 | DESC_SIZE | R/W-0 | 0 | Descriptor Size.<br>It is encoded in<br>4-byte increments with values 0 to 31 representing 24 and so on to 148 bytes.<br>This field will return a 0x0 when an empty queue is read.<br>Queue Manager Queue N Registers D To save hardware resources, the queue manager internally stores descriptor size (desc_size) information in four bits.<br>However, register D has five LSBs that specify descriptor size.<br>As a consequence, the value of desc_size that is pushed may not be same as that is read during a pop.<br>The value that is read back is equal to always rounded to an odd number.<br>So, for even values, the value read back is one more than what was written.<br>For odd values, the value read back is same as the value that was written.<br>Note that this<br>5-bit field (desc_size) is unrelated to the code for size of descriptors in a descriptor region.<br>It is just a place holder for a<br>5-bit value that is maintained across the push and pop operations for every descriptor managed by the queue manager. |

### 16.4.7.215 QUEUE_45_A Register (offset = 22D0h) [reset = 0h]

QUEUE_45_A is shown in Figure 16-491 and described in Table 16-505.

**Figure 16-491. QUEUE_45_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-505. QUEUE_45_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>This count is incremented by 1 whenever a packet is added to the queue.<br>This count is decremented by 1 whenever a packet is popped from the queue. |

### 16.4.7.216 QUEUE_45_B Register (offset = 22D4h) [reset = 0h]

QUEUE_45_B is shown in Figure 16-492 and described in Table 16-506.

**Figure 16-492. QUEUE_45_B Register**

| 31–28 | 27–0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-506. QUEUE_45_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. |

### 16.4.7.217 QUEUE_45_C Register (offset = 22D8h) [reset = 0h]

QUEUE_45_C is shown in Figure 16-493 and described in Table 16-507.

**Figure 16-493. QUEUE_45_C Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| HEAD_TAIL | Reserved | | | | | | |
| W-0 | | | | | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| Reserved | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| Reserved | | PACKET_SIZE | | | | | |
| | | R/W-0 | | | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| PACKET_SIZE | | | | | | | |
| R/W-0 | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-507. QUEUE_45_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31 | HEAD_TAIL | W-0 | 0 | Head/Tail Push Control.<br>Set to zero in order to push packet onto tail of queue and set to one in order to push packet onto head of queue. |
| 13-0 | PACKET_SIZE | R/W-0 | 0 | packet_size This field indicates packet size and is assumed to be zero on each packet add unless the value is explicitly overwritten. This field indicates packet size for packet pop operation. |

### 16.4.7.218 QUEUE_45_D Register (offset = 22DCh) [reset = 0h]

QUEUE_45_D is shown in Figure 16-494 and described in Table 16-508.

**Figure 16-494. QUEUE_45_D Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |
| **23** | **22** | **21** | **20** | **19** | **18** | **17** | **16** |
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |
| **15** | **14** | **13** | **12** | **11** | **10** | **9** | **8** |
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |
| **7** | **6** | **5** | **4** | **3** | **2** | **1** | **0** |
| DESC_PTR | | | DESC_SIZE | | | | |
| R/W-0 | | | R/W-0 | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-508. QUEUE_45_D Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-5 | DESC_PTR | R/W-0 | 0 | Descriptor pointer.<br>It will be read as zero if the queue is empty.<br>It will indicate a<br>32-bit aligned address that points to a descriptor when the queue is not empty. |
| 4-0 | DESC_SIZE | R/W-0 | 0 | Descriptor Size.<br>It is encoded in<br>4-byte increments with values 0 to 31 representing 24 and so on to 148 bytes.<br>This field will return a 0x0 when an empty queue is read.<br>Queue Manager Queue N Registers D To save hardware resources, the queue manager internally stores descriptor size (desc_size) information in four bits.<br>However, register D has five LSBs that specify descriptor size.<br>As a consequence, the value of desc_size that is pushed may not be same as that is read during a pop.<br>The value that is read back is equal to always rounded to an odd number.<br>So, for even values, the value read back is one more than what was written.<br>For odd values, the value read back is same as the value that was written.<br>Note that this<br>5-bit field (desc_size) is unrelated to the code for size of descriptors in a descriptor region.<br>It is just a place holder for a<br>5-bit value that is maintained across the push and pop operations for every descriptor managed by the queue manager. |

### 16.4.7.219 QUEUE_46_A Register (offset = 22E0h) [reset = 0h]

QUEUE_46_A is shown in Figure 16-495 and described in Table 16-509.

**Figure 16-495. QUEUE_46_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-509. QUEUE_46_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>This count is incremented by 1 whenever a packet is added to the queue.<br>This count is decremented by 1 whenever a packet is popped from the queue. |

### 16.4.7.220 QUEUE_46_B Register (offset = 22E4h) [reset = 0h]

QUEUE_46_B is shown in Figure 16-496 and described in Table 16-510.

**Figure 16-496. QUEUE_46_B Register**

| 31–28 | 27–0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-510. QUEUE_46_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. |

### 16.4.7.221 QUEUE_46_C Register (offset = 22E8h) [reset = 0h]

QUEUE_46_C is shown in Figure 16-497 and described in Table 16-511.

**Figure 16-497. QUEUE_46_C Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| HEAD_TAIL | Reserved | | | | | | |
| W-0 | | | | | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| Reserved | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| Reserved | | PACKET_SIZE | | | | | |
| | | R/W-0 | | | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| PACKET_SIZE | | | | | | | |
| R/W-0 | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-511. QUEUE_46_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31 | HEAD_TAIL | W-0 | 0 | Head/Tail Push Control.<br>Set to zero in order to push packet onto tail of queue and set to one in order to push packet onto head of queue. |
| 13-0 | PACKET_SIZE | R/W-0 | 0 | packet_size This field indicates packet size and is assumed to be zero on each packet add unless the value is explicitly overwritten. This field indicates packet size for packet pop operation. |

### 16.4.7.222 QUEUE_46_D Register (offset = 22ECh) [reset = 0h]

QUEUE_46_D is shown in Figure 16-498 and described in Table 16-512.

**Figure 16-498. QUEUE_46_D Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |
| **23** | **22** | **21** | **20** | **19** | **18** | **17** | **16** |
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |
| **15** | **14** | **13** | **12** | **11** | **10** | **9** | **8** |
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |
| **7** | **6** | **5** | **4** | **3** | **2** | **1** | **0** |
| DESC_PTR | | | DESC_SIZE | | | | |
| R/W-0 | | | R/W-0 | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-512. QUEUE_46_D Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-5 | DESC_PTR | R/W-0 | 0 | Descriptor pointer.<br>It will be read as zero if the queue is empty.<br>It will indicate a<br>32-bit aligned address that points to a descriptor when the queue is not empty. |
| 4-0 | DESC_SIZE | R/W-0 | 0 | Descriptor Size.<br>It is encoded in<br>4-byte increments with values 0 to 31 representing 24 and so on to 148 bytes.<br>This field will return a 0x0 when an empty queue is read.<br>Queue Manager Queue N Registers D To save hardware resources, the queue manager internally stores descriptor size (desc_size) information in four bits.<br>However, register D has five LSBs that specify descriptor size.<br>As a consequence, the value of desc_size that is pushed may not be same as that is read during a pop.<br>The value that is read back is equal to always rounded to an odd number.<br>So, for even values, the value read back is one more than what was written.<br>For odd values, the value read back is same as the value that was written.<br>Note that this<br>5-bit field (desc_size) is unrelated to the code for size of descriptors in a descriptor region.<br>It is just a place holder for a<br>5-bit value that is maintained across the push and pop operations for every descriptor managed by the queue manager. |

### 16.4.7.223 QUEUE_47_A Register (offset = 22F0h) [reset = 0h]

QUEUE_47_A is shown in Figure 16-499 and described in Table 16-513.

**Figure 16-499. QUEUE_47_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-513. QUEUE_47_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>This count is incremented by 1 whenever a packet is added to the queue.<br>This count is decremented by 1 whenever a packet is popped from the queue. |

### 16.4.7.224 QUEUE_47_B Register (offset = 22F4h) [reset = 0h]

QUEUE_47_B is shown in Figure 16-500 and described in Table 16-514.

**Figure 16-500. QUEUE_47_B Register**

| 31-28 | 27-0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-514. QUEUE_47_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. |

### 16.4.7.225 QUEUE_47_C Register (offset = 22F8h) [reset = 0h]

QUEUE_47_C is shown in Figure 16-501 and described in Table 16-515.

**Figure 16-501. QUEUE_47_C Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| HEAD_TAIL | Reserved | | | | | | |
| W-0 | | | | | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| Reserved | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| Reserved | | PACKET_SIZE | | | | | |
| | | R/W-0 | | | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| PACKET_SIZE | | | | | | | |
| R/W-0 | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-515. QUEUE_47_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31 | HEAD_TAIL | W-0 | 0 | Head/Tail Push Control.<br>Set to zero in order to push packet onto tail of queue and set to one in order to push packet onto head of queue. |
| 13-0 | PACKET_SIZE | R/W-0 | 0 | packet_size This field indicates packet size and is assumed to be zero on each packet add unless the value is explicitly overwritten. This field indicates packet size for packet pop operation. |

### 16.4.7.226 QUEUE_47_D Register (offset = 22FCh) [reset = 0h]

QUEUE_47_D is shown in Figure 16-502 and described in Table 16-516.

**Figure 16-502. QUEUE_47_D Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |
| **23** | **22** | **21** | **20** | **19** | **18** | **17** | **16** |
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |
| **15** | **14** | **13** | **12** | **11** | **10** | **9** | **8** |
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |
| **7** | **6** | **5** | **4** | **3** | **2** | **1** | **0** |
| DESC_PTR | | | DESC_SIZE | | | | |
| R/W-0 | | | R/W-0 | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-516. QUEUE_47_D Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-5 | DESC_PTR | R/W-0 | 0 | Descriptor pointer.<br>It will be read as zero if the queue is empty.<br>It will indicate a<br>32-bit aligned address that points to a descriptor when the queue is not empty. |
| 4-0 | DESC_SIZE | R/W-0 | 0 | Descriptor Size.<br>It is encoded in<br>4-byte increments with values 0 to 31 representing 24 and so on to 148 bytes.<br>This field will return a 0x0 when an empty queue is read.<br>Queue Manager Queue N Registers D To save hardware resources, the queue manager internally stores descriptor size (desc_size) information in four bits.<br>However, register D has five LSBs that specify descriptor size.<br>As a consequence, the value of desc_size that is pushed may not be same as that is read during a pop.<br>The value that is read back is equal to always rounded to an odd number.<br>So, for even values, the value read back is one more than what was written.<br>For odd values, the value read back is same as the value that was written.<br>Note that this<br>5-bit field (desc_size) is unrelated to the code for size of descriptors in a descriptor region.<br>It is just a place holder for a<br>5-bit value that is maintained across the push and pop operations for every descriptor managed by the queue manager. |

### 16.4.7.227 QUEUE_48_A Register (offset = 2300h) [reset = 0h]

QUEUE_48_A is shown in Figure 16-503 and described in Table 16-517.

**Figure 16-503. QUEUE_48_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-517. QUEUE_48_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>This count is incremented by 1 whenever a packet is added to the queue.<br>This count is decremented by 1 whenever a packet is popped from the queue. |

### 16.4.7.228 QUEUE_48_B Register (offset = 2304h) [reset = 0h]

QUEUE_48_B is shown in Figure 16-504 and described in Table 16-518.

**Figure 16-504. QUEUE_48_B Register**

| 31–28 | 27–0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-518. QUEUE_48_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. |

### 16.4.7.229 QUEUE_48_C Register (offset = 2308h) [reset = 0h]

QUEUE_48_C is shown in Figure 16-505 and described in Table 16-519.

**Figure 16-505. QUEUE_48_C Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| HEAD_TAIL | Reserved | | | | | | |
| W-0 | | | | | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| Reserved | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| Reserved | | PACKET_SIZE | | | | | |
| | | R/W-0 | | | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| PACKET_SIZE | | | | | | | |
| R/W-0 | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-519. QUEUE_48_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31 | HEAD_TAIL | W-0 | 0 | Head/Tail Push Control.<br>Set to zero in order to push packet onto tail of queue and set to one in order to push packet onto head of queue. |
| 13-0 | PACKET_SIZE | R/W-0 | 0 | packet_size This field indicates packet size and is assumed to be zero on each packet add unless the value is explicitly overwritten. This field indicates packet size for packet pop operation. |

### 16.4.7.230 QUEUE_48_D Register (offset = 230Ch) [reset = 0h]

QUEUE_48_D is shown in Figure 16-506 and described in Table 16-520.

**Figure 16-506. QUEUE_48_D Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |
| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |
| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |
| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
| DESC_PTR | | | DESC_SIZE | | | | |
| R/W-0 | | | R/W-0 | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-520. QUEUE_48_D Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-5 | DESC_PTR | R/W-0 | 0 | Descriptor pointer.<br>It will be read as zero if the queue is empty.<br>It will indicate a<br>32-bit aligned address that points to a descriptor when the queue is not empty. |
| 4-0 | DESC_SIZE | R/W-0 | 0 | Descriptor Size.<br>It is encoded in<br>4-byte increments with values 0 to 31 representing 24 and so on to 148 bytes.<br>This field will return a 0x0 when an empty queue is read.<br>Queue Manager Queue N Registers D To save hardware resources, the queue manager internally stores descriptor size (desc_size) information in four bits.<br>However, register D has five LSBs that specify descriptor size.<br>As a consequence, the value of desc_size that is pushed may not be same as that is read during a pop.<br>The value that is read back is equal to always rounded to an odd number.<br>So, for even values, the value read back is one more than what was written.<br>For odd values, the value read back is same as the value that was written.<br>Note that this<br>5-bit field (desc_size) is unrelated to the code for size of descriptors in a descriptor region.<br>It is just a place holder for a<br>5-bit value that is maintained across the push and pop operations for every descriptor managed by the queue manager. |

### 16.4.7.231 QUEUE_49_A Register (offset = 2310h) [reset = 0h]

QUEUE_49_A is shown in Figure 16-507 and described in Table 16-521.

**Figure 16-507. QUEUE_49_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-521. QUEUE_49_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>This count is incremented by 1 whenever a packet is added to the queue.<br>This count is decremented by 1 whenever a packet is popped from the queue. |

### 16.4.7.232 QUEUE_49_B Register (offset = 2314h) [reset = 0h]

QUEUE_49_B is shown in Figure 16-508 and described in Table 16-522.

**Figure 16-508. QUEUE_49_B Register**

| 31-28 | 27-0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-522. QUEUE_49_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. |

### 16.4.7.233 QUEUE_49_C Register (offset = 2318h) [reset = 0h]

QUEUE_49_C is shown in Figure 16-509 and described in Table 16-523.

**Figure 16-509. QUEUE_49_C Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| HEAD_TAIL | Reserved | | | | | | |
| W-0 | | | | | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| Reserved | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| Reserved | | PACKET_SIZE | | | | | |
| | | R/W-0 | | | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| PACKET_SIZE | | | | | | | |
| R/W-0 | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-523. QUEUE_49_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31 | HEAD_TAIL | W-0 | 0 | Head/Tail Push Control.<br>Set to zero in order to push packet onto tail of queue and set to one in order to push packet onto head of queue. |
| 13-0 | PACKET_SIZE | R/W-0 | 0 | packet_size This field indicates packet size and is assumed to be zero on each packet add unless the value is explicitly overwritten. This field indicates packet size for packet pop operation. |

### 16.4.7.234 QUEUE_49_D Register (offset = 231Ch) [reset = 0h]

QUEUE_49_D is shown in Figure 16-510 and described in Table 16-524.

**Figure 16-510. QUEUE_49_D Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | DESC_SIZE | | | | |
| R/W-0 | | | R/W-0 | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-524. QUEUE_49_D Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-5 | DESC_PTR | R/W-0 | 0 | Descriptor pointer.<br>It will be read as zero if the queue is empty.<br>It will indicate a<br>32-bit aligned address that points to a descriptor when the queue is not empty. |
| 4-0 | DESC_SIZE | R/W-0 | 0 | Descriptor Size.<br>It is encoded in<br>4-byte increments with values 0 to 31 representing 24 and so on to 148 bytes.<br>This field will return a 0x0 when an empty queue is read.<br>Queue Manager Queue N Registers D To save hardware resources, the queue manager internally stores descriptor size (desc_size) information in four bits.<br>However, register D has five LSBs that specify descriptor size.<br>As a consequence, the value of desc_size that is pushed may not be same as that is read during a pop.<br>The value that is read back is equal to always rounded to an odd number.<br>So, for even values, the value read back is one more than what was written.<br>For odd values, the value read back is same as the value that was written.<br>Note that this<br>5-bit field (desc_size) is unrelated to the code for size of descriptors in a descriptor region.<br>It is just a place holder for a<br>5-bit value that is maintained across the push and pop operations for every descriptor managed by the queue manager. |

### 16.4.7.235 QUEUE_50_A Register (offset = 2320h) [reset = 0h]

QUEUE_50_A is shown in Figure 16-511 and described in Table 16-525.

**Figure 16-511. QUEUE_50_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-525. QUEUE_50_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>This count is incremented by 1 whenever a packet is added to the queue.<br>This count is decremented by 1 whenever a packet is popped from the queue. |

### 16.4.7.236 QUEUE_50_B Register (offset = 2324h) [reset = 0h]

QUEUE_50_B is shown in Figure 16-512 and described in Table 16-526.

**Figure 16-512. QUEUE_50_B Register**

| 31–28 | 27–0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-526. QUEUE_50_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. |

### 16.4.7.237 QUEUE_50_C Register (offset = 2328h) [reset = 0h]

QUEUE_50_C is shown in Figure 16-513 and described in Table 16-527.

**Figure 16-513. QUEUE_50_C Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| HEAD_TAIL | Reserved | | | | | | |
| W-0 | | | | | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| Reserved | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| Reserved | | PACKET_SIZE | | | | | |
| | | R/W-0 | | | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| PACKET_SIZE | | | | | | | |
| R/W-0 | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-527. QUEUE_50_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31 | HEAD_TAIL | W-0 | 0 | Head/Tail Push Control.<br>Set to zero in order to push packet onto tail of queue and set to one in order to push packet onto head of queue. |
| 13-0 | PACKET_SIZE | R/W-0 | 0 | packet_size This field indicates packet size and is assumed to be zero on each packet add unless the value is explicitly overwritten. This field indicates packet size for packet pop operation. |

### 16.4.7.238 QUEUE_50_D Register (offset = 232Ch) [reset = 0h]

QUEUE_50_D is shown in Figure 16-514 and described in Table 16-528.

**Figure 16-514. QUEUE_50_D Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |
| **23** | **22** | **21** | **20** | **19** | **18** | **17** | **16** |
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |
| **15** | **14** | **13** | **12** | **11** | **10** | **9** | **8** |
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |
| **7** | **6** | **5** | **4** | **3** | **2** | **1** | **0** |
| DESC_PTR | | | DESC_SIZE | | | | |
| R/W-0 | | | R/W-0 | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-528. QUEUE_50_D Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-5 | DESC_PTR | R/W-0 | 0 | Descriptor pointer.<br>It will be read as zero if the queue is empty.<br>It will indicate a<br>32-bit aligned address that points to a descriptor when the queue is not empty. |
| 4-0 | DESC_SIZE | R/W-0 | 0 | Descriptor Size.<br>It is encoded in<br>4-byte increments with values 0 to 31 representing 24 and so on to 148 bytes.<br>This field will return a 0x0 when an empty queue is read.<br>Queue Manager Queue N Registers D To save hardware resources, the queue manager internally stores descriptor size (desc_size) information in four bits.<br>However, register D has five LSBs that specify descriptor size.<br>As a consequence, the value of desc_size that is pushed may not be same as that is read during a pop.<br>The value that is read back is equal to always rounded to an odd number.<br>So, for even values, the value read back is one more than what was written.<br>For odd values, the value read back is same as the value that was written.<br>Note that this<br>5-bit field (desc_size) is unrelated to the code for size of descriptors in a descriptor region.<br>It is just a place holder for a<br>5-bit value that is maintained across the push and pop operations for every descriptor managed by the queue manager. |

### 16.4.7.239 QUEUE_51_A Register (offset = 2330h) [reset = 0h]

QUEUE_51_A is shown in Figure 16-515 and described in Table 16-529.

**Figure 16-515. QUEUE_51_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-529. QUEUE_51_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>This count is incremented by 1 whenever a packet is added to the queue.<br>This count is decremented by 1 whenever a packet is popped from the queue. |

### 16.4.7.240 QUEUE_51_B Register (offset = 2334h) [reset = 0h]

QUEUE_51_B is shown in Figure 16-516 and described in Table 16-530.

**Figure 16-516. QUEUE_51_B Register**

| 31–28 | 27–0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-530. QUEUE_51_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. |

### 16.4.7.241 QUEUE_51_C Register (offset = 2338h) [reset = 0h]

QUEUE_51_C is shown in Figure 16-517 and described in Table 16-531.

**Figure 16-517. QUEUE_51_C Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| HEAD_TAIL | Reserved | | | | | | |
| W-0 | | | | | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| Reserved | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| Reserved | | PACKET_SIZE | | | | | |
| | | R/W-0 | | | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| PACKET_SIZE | | | | | | | |
| R/W-0 | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-531. QUEUE_51_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31 | HEAD_TAIL | W-0 | 0 | Head/Tail Push Control.<br>Set to zero in order to push packet onto tail of queue and set to one in order to push packet onto head of queue. |
| 13-0 | PACKET_SIZE | R/W-0 | 0 | packet_size This field indicates packet size and is assumed to be zero on each packet add unless the value is explicitly overwritten.<br>This field indicates packet size for packet pop operation. |

### 16.4.7.242 QUEUE_51_D Register (offset = 233Ch) [reset = 0h]

QUEUE_51_D is shown in Figure 16-518 and described in Table 16-532.

**Figure 16-518. QUEUE_51_D Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |
| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |
| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |
| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
| DESC_PTR | | | DESC_SIZE | | | | |
| R/W-0 | | | R/W-0 | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-532. QUEUE_51_D Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-5 | DESC_PTR | R/W-0 | 0 | Descriptor pointer.<br>It will be read as zero if the queue is empty.<br>It will indicate a<br>32-bit aligned address that points to a descriptor when the queue is not empty. |
| 4-0 | DESC_SIZE | R/W-0 | 0 | Descriptor Size.<br>It is encoded in<br>4-byte increments with values 0 to 31 representing 24 and so on to 148 bytes.<br>This field will return a 0x0 when an empty queue is read.<br>Queue Manager Queue N Registers D To save hardware resources, the queue manager internally stores descriptor size (desc_size) information in four bits.<br>However, register D has five LSBs that specify descriptor size.<br>As a consequence, the value of desc_size that is pushed may not be same as that is read during a pop.<br>The value that is read back is equal to always rounded to an odd number.<br>So, for even values, the value read back is one more than what was written.<br>For odd values, the value read back is same as the value that was written.<br>Note that this<br>5-bit field (desc_size) is unrelated to the code for size of descriptors in a descriptor region.<br>It is just a place holder for a<br>5-bit value that is maintained across the push and pop operations for every descriptor managed by the queue manager. |

#### 16.4.7.243 QUEUE_52_A Register (offset = 2340h) [reset = 0h]

QUEUE_52_A is shown in Figure 16-519 and described in Table 16-533.

**Figure 16-519. QUEUE_52_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-533. QUEUE_52_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>This count is incremented by 1 whenever a packet is added to the queue.<br>This count is decremented by 1 whenever a packet is popped from the queue. |

### 16.4.7.244 QUEUE_52_B Register (offset = 2344h) [reset = 0h]

QUEUE_52_B is shown in Figure 16-520 and described in Table 16-534.

**Figure 16-520. QUEUE_52_B Register**

| 31-28 | 27-0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-534. QUEUE_52_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. |

### 16.4.7.245 QUEUE_52_C Register (offset = 2348h) [reset = 0h]

QUEUE_52_C is shown in Figure 16-521 and described in Table 16-535.

**Figure 16-521. QUEUE_52_C Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| HEAD_TAIL | Reserved | | | | | | |
| W-0 | | | | | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| Reserved | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| Reserved | | PACKET_SIZE | | | | | |
| | | R/W-0 | | | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| PACKET_SIZE | | | | | | | |
| R/W-0 | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-535. QUEUE_52_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31 | HEAD_TAIL | W-0 | 0 | Head/Tail Push Control.<br>Set to zero in order to push packet onto tail of queue and set to one in order to push packet onto head of queue. |
| 13-0 | PACKET_SIZE | R/W-0 | 0 | packet_size This field indicates packet size and is assumed to be zero on each packet add unless the value is explicitly overwritten. This field indicates packet size for packet pop operation. |

### 16.4.7.246 QUEUE_52_D Register (offset = 234Ch) [reset = 0h]

QUEUE_52_D is shown in Figure 16-522 and described in Table 16-536.

**Figure 16-522. QUEUE_52_D Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |
| **23** | **22** | **21** | **20** | **19** | **18** | **17** | **16** |
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |
| **15** | **14** | **13** | **12** | **11** | **10** | **9** | **8** |
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |
| **7** | **6** | **5** | **4** | **3** | **2** | **1** | **0** |
| DESC_PTR | | | DESC_SIZE | | | | |
| R/W-0 | | | R/W-0 | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-536. QUEUE_52_D Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-5 | DESC_PTR | R/W-0 | 0 | Descriptor pointer.<br>It will be read as zero if the queue is empty.<br>It will indicate a<br>32-bit aligned address that points to a descriptor when the queue is not empty. |
| 4-0 | DESC_SIZE | R/W-0 | 0 | Descriptor Size.<br>It is encoded in<br>4-byte increments with values 0 to 31 representing 24 and so on to 148 bytes.<br>This field will return a 0x0 when an empty queue is read.<br>Queue Manager Queue N Registers D To save hardware resources, the queue manager internally stores descriptor size (desc_size) information in four bits.<br>However, register D has five LSBs that specify descriptor size.<br>As a consequence, the value of desc_size that is pushed may not be same as that is read during a pop.<br>The value that is read back is equal to always rounded to an odd number.<br>So, for even values, the value read back is one more than what was written.<br>For odd values, the value read back is same as the value that was written.<br>Note that this<br>5-bit field (desc_size) is unrelated to the code for size of descriptors in a descriptor region.<br>It is just a place holder for a<br>5-bit value that is maintained across the push and pop operations for every descriptor managed by the queue manager. |

### 16.4.7.247 QUEUE_53_A Register (offset = 2350h) [reset = 0h]

QUEUE_53_A is shown in Figure 16-523 and described in Table 16-537.

**Figure 16-523. QUEUE_53_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-537. QUEUE_53_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>This count is incremented by 1 whenever a packet is added to the queue.<br>This count is decremented by 1 whenever a packet is popped from the queue. |

### 16.4.7.248 QUEUE_53_B Register (offset = 2354h) [reset = 0h]

QUEUE_53_B is shown in Figure 16-524 and described in Table 16-538.

**Figure 16-524. QUEUE_53_B Register**

| 31-28 | 27-0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-538. QUEUE_53_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. |

### 16.4.7.249 QUEUE_53_C Register (offset = 2358h) [reset = 0h]

QUEUE_53_C is shown in Figure 16-525 and described in Table 16-539.

**Figure 16-525. QUEUE_53_C Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| HEAD_TAIL | Reserved | | | | | | |
| W-0 | | | | | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| Reserved | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| Reserved | | PACKET_SIZE | | | | | |
| | | R/W-0 | | | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| PACKET_SIZE | | | | | | | |
| R/W-0 | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-539. QUEUE_53_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31 | HEAD_TAIL | W-0 | 0 | Head/Tail Push Control.<br>Set to zero in order to push packet onto tail of queue and set to one in order to push packet onto head of queue. |
| 13-0 | PACKET_SIZE | R/W-0 | 0 | packet_size This field indicates packet size and is assumed to be zero on each packet add unless the value is explicitly overwritten. This field indicates packet size for packet pop operation. |

### 16.4.7.250 QUEUE_53_D Register (offset = 235Ch) [reset = 0h]

QUEUE_53_D is shown in Figure 16-526 and described in Table 16-540.

**Figure 16-526. QUEUE_53_D Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | DESC_SIZE | | | | |
| R/W-0 | | | R/W-0 | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-540. QUEUE_53_D Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-5 | DESC_PTR | R/W-0 | 0 | Descriptor pointer.<br>It will be read as zero if the queue is empty.<br>It will indicate a<br>32-bit aligned address that points to a descriptor when the queue is not empty. |
| 4-0 | DESC_SIZE | R/W-0 | 0 | Descriptor Size.<br>It is encoded in<br>4-byte increments with values 0 to 31 representing 24 and so on to 148 bytes.<br>This field will return a 0x0 when an empty queue is read.<br>Queue Manager Queue N Registers D To save hardware resources, the queue manager internally stores descriptor size (desc_size) information in four bits.<br>However, register D has five LSBs that specify descriptor size.<br>As a consequence, the value of desc_size that is pushed may not be same as that is read during a pop.<br>The value that is read back is equal to always rounded to an odd number.<br>So, for even values, the value read back is one more than what was written.<br>For odd values, the value read back is same as the value that was written.<br>Note that this<br>5-bit field (desc_size) is unrelated to the code for size of descriptors in a descriptor region.<br>It is just a place holder for a<br>5-bit value that is maintained across the push and pop operations for every descriptor managed by the queue manager. |

### 16.4.7.251 QUEUE_54_A Register (offset = 2360h) [reset = 0h]

QUEUE_54_A is shown in Figure 16-527 and described in Table 16-541.

**Figure 16-527. QUEUE_54_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-541. QUEUE_54_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>This count is incremented by 1 whenever a packet is added to the queue.<br>This count is decremented by 1 whenever a packet is popped from the queue. |

### 16.4.7.252 QUEUE_54_B Register (offset = 2364h) [reset = 0h]

QUEUE_54_B is shown in Figure 16-528 and described in Table 16-542.

**Figure 16-528. QUEUE_54_B Register**

| 31-28 | 27-0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-542. QUEUE_54_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. |

### 16.4.7.253 QUEUE_54_C Register (offset = 2368h) [reset = 0h]

QUEUE_54_C is shown in Figure 16-529 and described in Table 16-543.

**Figure 16-529. QUEUE_54_C Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| HEAD_TAIL | Reserved | | | | | | |
| W-0 | | | | | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| Reserved | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| Reserved | | PACKET_SIZE | | | | | |
| | | R/W-0 | | | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| PACKET_SIZE | | | | | | | |
| R/W-0 | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-543. QUEUE_54_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31 | HEAD_TAIL | W-0 | 0 | Head/Tail Push Control.<br>Set to zero in order to push packet onto tail of queue and set to one in order to push packet onto head of queue. |
| 13-0 | PACKET_SIZE | R/W-0 | 0 | packet_size This field indicates packet size and is assumed to be zero on each packet add unless the value is explicitly overwritten. This field indicates packet size for packet pop operation. |

### 16.4.7.254 QUEUE_54_D Register (offset = 236Ch) [reset = 0h]

QUEUE_54_D is shown in Figure 16-530 and described in Table 16-544.

**Figure 16-530. QUEUE_54_D Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | DESC_SIZE | | | | |
| R/W-0 | | | R/W-0 | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-544. QUEUE_54_D Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-5 | DESC_PTR | R/W-0 | 0 | Descriptor pointer.<br>It will be read as zero if the queue is empty.<br>It will indicate a<br>32-bit aligned address that points to a descriptor when the queue is not empty. |
| 4-0 | DESC_SIZE | R/W-0 | 0 | Descriptor Size.<br>It is encoded in<br>4-byte increments with values 0 to 31 representing 24 and so on to 148 bytes.<br>This field will return a 0x0 when an empty queue is read.<br>Queue Manager Queue N Registers D To save hardware resources, the queue manager internally stores descriptor size (desc_size) information in four bits.<br>However, register D has five LSBs that specify descriptor size.<br>As a consequence, the value of desc_size that is pushed may not be same as that is read during a pop.<br>The value that is read back is equal to always rounded to an odd number.<br>So, for even values, the value read back is one more than what was written.<br>For odd values, the value read back is same as the value that was written.<br>Note that this<br>5-bit field (desc_size) is unrelated to the code for size of descriptors in a descriptor region.<br>It is just a place holder for a<br>5-bit value that is maintained across the push and pop operations for every descriptor managed by the queue manager. |

### 16.4.7.255 QUEUE_55_A Register (offset = 2370h) [reset = 0h]

QUEUE_55_A is shown in Figure 16-531 and described in Table 16-545.

**Figure 16-531. QUEUE_55_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-545. QUEUE_55_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>This count is incremented by 1 whenever a packet is added to the queue.<br>This count is decremented by 1 whenever a packet is popped from the queue. |

### 16.4.7.256 QUEUE_55_B Register (offset = 2374h) [reset = 0h]

QUEUE_55_B is shown in Figure 16-532 and described in Table 16-546.

**Figure 16-532. QUEUE_55_B Register**

| 31–28 | 27–0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-546. QUEUE_55_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. |

### 16.4.7.257 QUEUE_55_C Register (offset = 2378h) [reset = 0h]

QUEUE_55_C is shown in Figure 16-533 and described in Table 16-547.

**Figure 16-533. QUEUE_55_C Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| HEAD_TAIL | Reserved | | | | | | |
| W-0 | | | | | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| Reserved | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| Reserved | | PACKET_SIZE | | | | | |
| | | R/W-0 | | | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| PACKET_SIZE | | | | | | | |
| R/W-0 | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-547. QUEUE_55_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31 | HEAD_TAIL | W-0 | 0 | Head/Tail Push Control.<br>Set to zero in order to push packet onto tail of queue and set to one in order to push packet onto head of queue. |
| 13-0 | PACKET_SIZE | R/W-0 | 0 | packet_size This field indicates packet size and is assumed to be zero on each packet add unless the value is explicitly overwritten. This field indicates packet size for packet pop operation. |

### 16.4.7.258 QUEUE_55_D Register (offset = 237Ch) [reset = 0h]

QUEUE_55_D is shown in Figure 16-534 and described in Table 16-548.

**Figure 16-534. QUEUE_55_D Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |
| **23** | **22** | **21** | **20** | **19** | **18** | **17** | **16** |
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |
| **15** | **14** | **13** | **12** | **11** | **10** | **9** | **8** |
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |
| **7** | **6** | **5** | **4** | **3** | **2** | **1** | **0** |
| DESC_PTR | | | DESC_SIZE | | | | |
| R/W-0 | | | R/W-0 | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-548. QUEUE_55_D Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-5 | DESC_PTR | R/W-0 | 0 | Descriptor pointer.<br>It will be read as zero if the queue is empty.<br>It will indicate a<br>32-bit aligned address that points to a descriptor when the queue is not empty. |
| 4-0 | DESC_SIZE | R/W-0 | 0 | Descriptor Size.<br>It is encoded in<br>4-byte increments with values 0 to 31 representing 24 and so on to 148 bytes.<br>This field will return a 0x0 when an empty queue is read.<br>Queue Manager Queue N Registers D To save hardware resources, the queue manager internally stores descriptor size (desc_size) information in four bits.<br>However, register D has five LSBs that specify descriptor size.<br>As a consequence, the value of desc_size that is pushed may not be same as that is read during a pop.<br>The value that is read back is equal to always rounded to an odd number.<br>So, for even values, the value read back is one more than what was written.<br>For odd values, the value read back is same as the value that was written.<br>Note that this<br>5-bit field (desc_size) is unrelated to the code for size of descriptors in a descriptor region.<br>It is just a place holder for a<br>5-bit value that is maintained across the push and pop operations for every descriptor managed by the queue manager. |

### 16.4.7.259 QUEUE_56_A Register (offset = 2380h) [reset = 0h]

QUEUE_56_A is shown in Figure 16-535 and described in Table 16-549.

**Figure 16-535. QUEUE_56_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-549. QUEUE_56_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>This count is incremented by 1 whenever a packet is added to the queue.<br>This count is decremented by 1 whenever a packet is popped from the queue. |

### 16.4.7.260 QUEUE_56_B Register (offset = 2384h) [reset = 0h]

QUEUE_56_B is shown in Figure 16-536 and described in Table 16-550.

**Figure 16-536. QUEUE_56_B Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 | 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 | 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 | 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| Reserved | | | | QUEUE_BYTE_COUNT | | | | | | | | | | | | | | | | | | | | | | | | | | | |
| | | | | R-0 | | | | | | | | | | | | | | | | | | | | | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-550. QUEUE_56_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. |

### 16.4.7.261 QUEUE_56_C Register (offset = 2388h) [reset = 0h]

QUEUE_56_C is shown in Figure 16-537 and described in Table 16-551.

**Figure 16-537. QUEUE_56_C Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| HEAD_TAIL | Reserved | | | | | | |
| W-0 | | | | | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| Reserved | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| Reserved | | PACKET_SIZE | | | | | |
| | | R/W-0 | | | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| PACKET_SIZE | | | | | | | |
| R/W-0 | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-551. QUEUE_56_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31 | HEAD_TAIL | W-0 | 0 | Head/Tail Push Control.<br>Set to zero in order to push packet onto tail of queue and set to one in order to push packet onto head of queue. |
| 13-0 | PACKET_SIZE | R/W-0 | 0 | packet_size This field indicates packet size and is assumed to be zero on each packet add unless the value is explicitly overwritten. This field indicates packet size for packet pop operation. |

### 16.4.7.262 QUEUE_56_D Register (offset = 238Ch) [reset = 0h]

QUEUE_56_D is shown in Figure 16-538 and described in Table 16-552.

**Figure 16-538. QUEUE_56_D Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |
| **23** | **22** | **21** | **20** | **19** | **18** | **17** | **16** |
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |
| **15** | **14** | **13** | **12** | **11** | **10** | **9** | **8** |
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |
| **7** | **6** | **5** | **4** | **3** | **2** | **1** | **0** |
| DESC_PTR | | | DESC_SIZE | | | | |
| R/W-0 | | | R/W-0 | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-552. QUEUE_56_D Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-5 | DESC_PTR | R/W-0 | 0 | Descriptor pointer.<br>It will be read as zero if the queue is empty.<br>It will indicate a<br>32-bit aligned address that points to a descriptor when the queue is not empty. |
| 4-0 | DESC_SIZE | R/W-0 | 0 | Descriptor Size.<br>It is encoded in<br>4-byte increments with values 0 to 31 representing 24 and so on to 148 bytes.<br>This field will return a 0x0 when an empty queue is read.<br>Queue Manager Queue N Registers D To save hardware resources, the queue manager internally stores descriptor size (desc_size) information in four bits.<br>However, register D has five LSBs that specify descriptor size.<br>As a consequence, the value of desc_size that is pushed may not be same as that is read during a pop.<br>The value that is read back is equal to always rounded to an odd number.<br>So, for even values, the value read back is one more than what was written.<br>For odd values, the value read back is same as the value that was written.<br>Note that this<br>5-bit field (desc_size) is unrelated to the code for size of descriptors in a descriptor region.<br>It is just a place holder for a<br>5-bit value that is maintained across the push and pop operations for every descriptor managed by the queue manager. |

### 16.4.7.263 QUEUE_57_A Register (offset = 2390h) [reset = 0h]

QUEUE_57_A is shown in Figure 16-539 and described in Table 16-553.

**Figure 16-539. QUEUE_57_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-553. QUEUE_57_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>This count is incremented by 1 whenever a packet is added to the queue.<br>This count is decremented by 1 whenever a packet is popped from the queue. |

### 16.4.7.264 QUEUE_57_B Register (offset = 2394h) [reset = 0h]

QUEUE_57_B is shown in Figure 16-540 and described in Table 16-554.

**Figure 16-540. QUEUE_57_B Register**

| 31–28 | 27–0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-554. QUEUE_57_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. |

### 16.4.7.265 QUEUE_57_C Register (offset = 2398h) [reset = 0h]

QUEUE_57_C is shown in Figure 16-541 and described in Table 16-555.

**Figure 16-541. QUEUE_57_C Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| HEAD_TAIL | Reserved | | | | | | |
| W-0 | | | | | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| Reserved | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| Reserved | | PACKET_SIZE | | | | | |
| | | R/W-0 | | | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| PACKET_SIZE | | | | | | | |
| R/W-0 | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-555. QUEUE_57_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31 | HEAD_TAIL | W-0 | 0 | Head/Tail Push Control.<br>Set to zero in order to push packet onto tail of queue and set to one in order to push packet onto head of queue. |
| 13-0 | PACKET_SIZE | R/W-0 | 0 | packet_size This field indicates packet size and is assumed to be zero on each packet add unless the value is explicitly overwritten. This field indicates packet size for packet pop operation. |

### 16.4.7.266 QUEUE_57_D Register (offset = 239Ch) [reset = 0h]

QUEUE_57_D is shown in Figure 16-542 and described in Table 16-556.

**Figure 16-542. QUEUE_57_D Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |
| **23** | **22** | **21** | **20** | **19** | **18** | **17** | **16** |
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |
| **15** | **14** | **13** | **12** | **11** | **10** | **9** | **8** |
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |
| **7** | **6** | **5** | **4** | **3** | **2** | **1** | **0** |
| DESC_PTR | | | DESC_SIZE | | | | |
| R/W-0 | | | R/W-0 | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-556. QUEUE_57_D Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-5 | DESC_PTR | R/W-0 | 0 | Descriptor pointer.<br>It will be read as zero if the queue is empty.<br>It will indicate a<br>32-bit aligned address that points to a descriptor when the queue is not empty. |
| 4-0 | DESC_SIZE | R/W-0 | 0 | Descriptor Size.<br>It is encoded in<br>4-byte increments with values 0 to 31 representing 24 and so on to 148 bytes.<br>This field will return a 0x0 when an empty queue is read.<br>Queue Manager Queue N Registers D To save hardware resources, the queue manager internally stores descriptor size (desc_size) information in four bits.<br>However, register D has five LSBs that specify descriptor size.<br>As a consequence, the value of desc_size that is pushed may not be same as that is read during a pop.<br>The value that is read back is equal to always rounded to an odd number.<br>So, for even values, the value read back is one more than what was written.<br>For odd values, the value read back is same as the value that was written.<br>Note that this<br>5-bit field (desc_size) is unrelated to the code for size of descriptors in a descriptor region.<br>It is just a place holder for a<br>5-bit value that is maintained across the push and pop operations for every descriptor managed by the queue manager. |

### 16.4.7.267 QUEUE_58_A Register (offset = 23A0h) [reset = 0h]

QUEUE_58_A is shown in Figure 16-543 and described in Table 16-557.

**Figure 16-543. QUEUE_58_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-557. QUEUE_58_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>This count is incremented by 1 whenever a packet is added to the queue.<br>This count is decremented by 1 whenever a packet is popped from the queue. |

### 16.4.7.268 QUEUE_58_B Register (offset = 23A4h) [reset = 0h]

QUEUE_58_B is shown in Figure 16-544 and described in Table 16-558.

**Figure 16-544. QUEUE_58_B Register**

| 31-28 | 27-0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-558. QUEUE_58_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. |

### 16.4.7.269 QUEUE_58_C Register (offset = 23A8h) [reset = 0h]

QUEUE_58_C is shown in Figure 16-545 and described in Table 16-559.

**Figure 16-545. QUEUE_58_C Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| HEAD_TAIL | Reserved | | | | | | |
| W-0 | | | | | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| Reserved | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| Reserved | | PACKET_SIZE | | | | | |
| | | R/W-0 | | | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| PACKET_SIZE | | | | | | | |
| R/W-0 | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-559. QUEUE_58_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31 | HEAD_TAIL | W-0 | 0 | Head/Tail Push Control.<br>Set to zero in order to push packet onto tail of queue and set to one in order to push packet onto head of queue. |
| 13-0 | PACKET_SIZE | R/W-0 | 0 | packet_size This field indicates packet size and is assumed to be zero on each packet add unless the value is explicitly overwritten. This field indicates packet size for packet pop operation. |

### 16.4.7.270 QUEUE_58_D Register (offset = 23ACh) [reset = 0h]

QUEUE_58_D is shown in Figure 16-546 and described in Table 16-560.

**Figure 16-546. QUEUE_58_D Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | DESC_SIZE | | | | |
| R/W-0 | | | R/W-0 | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-560. QUEUE_58_D Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-5 | DESC_PTR | R/W-0 | 0 | Descriptor pointer.<br>It will be read as zero if the queue is empty.<br>It will indicate a<br>32-bit aligned address that points to a descriptor when the queue is not empty. |
| 4-0 | DESC_SIZE | R/W-0 | 0 | Descriptor Size.<br>It is encoded in<br>4-byte increments with values 0 to 31 representing 24 and so on to 148 bytes.<br>This field will return a 0x0 when an empty queue is read.<br>Queue Manager Queue N Registers D To save hardware resources, the queue manager internally stores descriptor size (desc_size) information in four bits.<br>However, register D has five LSBs that specify descriptor size.<br>As a consequence, the value of desc_size that is pushed may not be same as that is read during a pop.<br>The value that is read back is equal to always rounded to an odd number.<br>So, for even values, the value read back is one more than what was written.<br>For odd values, the value read back is same as the value that was written.<br>Note that this<br>5-bit field (desc_size) is unrelated to the code for size of descriptors in a descriptor region.<br>It is just a place holder for a<br>5-bit value that is maintained across the push and pop operations for every descriptor managed by the queue manager. |

### 16.4.7.271 QUEUE_59_A Register (offset = 23B0h) [reset = 0h]

QUEUE_59_A is shown in Figure 16-547 and described in Table 16-561.

**Figure 16-547. QUEUE_59_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-561. QUEUE_59_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>This count is incremented by 1 whenever a packet is added to the queue.<br>This count is decremented by 1 whenever a packet is popped from the queue. |

### 16.4.7.272 QUEUE_59_B Register (offset = 23B4h) [reset = 0h]

QUEUE_59_B is shown in Figure 16-548 and described in Table 16-562.

**Figure 16-548. QUEUE_59_B Register**

| 31-28 | 27-0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-562. QUEUE_59_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. |

### 16.4.7.273 QUEUE_59_C Register (offset = 23B8h) [reset = 0h]

QUEUE_59_C is shown in Figure 16-549 and described in Table 16-563.

**Figure 16-549. QUEUE_59_C Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| HEAD_TAIL | Reserved | | | | | | |
| W-0 | | | | | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| Reserved | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| Reserved | | PACKET_SIZE | | | | | |
| | | R/W-0 | | | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| PACKET_SIZE | | | | | | | |
| R/W-0 | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-563. QUEUE_59_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31 | HEAD_TAIL | W-0 | 0 | Head/Tail Push Control.<br>Set to zero in order to push packet onto tail of queue and set to one in order to push packet onto head of queue. |
| 13-0 | PACKET_SIZE | R/W-0 | 0 | packet_size This field indicates packet size and is assumed to be zero on each packet add unless the value is explicitly overwritten.<br>This field indicates packet size for packet pop operation. |

### 16.4.7.274 QUEUE_59_D Register (offset = 23BCh) [reset = 0h]

QUEUE_59_D is shown in Figure 16-550 and described in Table 16-564.

**Figure 16-550. QUEUE_59_D Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | DESC_SIZE | | | | |
| R/W-0 | | | R/W-0 | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-564. QUEUE_59_D Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-5 | DESC_PTR | R/W-0 | 0 | Descriptor pointer.<br>It will be read as zero if the queue is empty.<br>It will indicate a<br>32-bit aligned address that points to a descriptor when the queue is not empty. |
| 4-0 | DESC_SIZE | R/W-0 | 0 | Descriptor Size.<br>It is encoded in<br>4-byte increments with values 0 to 31 representing 24 and so on to 148 bytes.<br>This field will return a 0x0 when an empty queue is read.<br>Queue Manager Queue N Registers D To save hardware resources, the queue manager internally stores descriptor size (desc_size) information in four bits.<br>However, register D has five LSBs that specify descriptor size.<br>As a consequence, the value of desc_size that is pushed may not be same as that is read during a pop.<br>The value that is read back is equal to always rounded to an odd number.<br>So, for even values, the value read back is one more than what was written.<br>For odd values, the value read back is same as the value that was written.<br>Note that this<br>5-bit field (desc_size) is unrelated to the code for size of descriptors in a descriptor region.<br>It is just a place holder for a<br>5-bit value that is maintained across the push and pop operations for every descriptor managed by the queue manager. |

### 16.4.7.275 QUEUE_60_A Register (offset = 23C0h) [reset = 0h]

QUEUE_60_A is shown in Figure 16-551 and described in Table 16-565.

**Figure 16-551. QUEUE_60_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-565. QUEUE_60_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>This count is incremented by 1 whenever a packet is added to the queue.<br>This count is decremented by 1 whenever a packet is popped from the queue. |

### 16.4.7.276 QUEUE_60_B Register (offset = 23C4h) [reset = 0h]

QUEUE_60_B is shown in Figure 16-552 and described in Table 16-566.

**Figure 16-552. QUEUE_60_B Register**

| 31–28 | 27–0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-566. QUEUE_60_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. |

### 16.4.7.277 QUEUE_60_C Register (offset = 23C8h) [reset = 0h]

QUEUE_60_C is shown in Figure 16-553 and described in Table 16-567.

**Figure 16-553. QUEUE_60_C Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| HEAD_TAIL | Reserved | | | | | | |
| W-0 | | | | | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| Reserved | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| Reserved | | PACKET_SIZE | | | | | |
| | | R/W-0 | | | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| PACKET_SIZE | | | | | | | |
| R/W-0 | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-567. QUEUE_60_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31 | HEAD_TAIL | W-0 | 0 | Head/Tail Push Control.<br>Set to zero in order to push packet onto tail of queue and set to one in order to push packet onto head of queue. |
| 13-0 | PACKET_SIZE | R/W-0 | 0 | packet_size This field indicates packet size and is assumed to be zero on each packet add unless the value is explicitly overwritten. This field indicates packet size for packet pop operation. |

### 16.4.7.278 QUEUE_60_D Register (offset = 23CCh) [reset = 0h]

QUEUE_60_D is shown in Figure 16-554 and described in Table 16-568.

**Figure 16-554. QUEUE_60_D Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |
| **23** | **22** | **21** | **20** | **19** | **18** | **17** | **16** |
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |
| **15** | **14** | **13** | **12** | **11** | **10** | **9** | **8** |
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |
| **7** | **6** | **5** | **4** | **3** | **2** | **1** | **0** |
| DESC_PTR | | | DESC_SIZE | | | | |
| R/W-0 | | | R/W-0 | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-568. QUEUE_60_D Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-5 | DESC_PTR | R/W-0 | 0 | Descriptor pointer.<br>It will be read as zero if the queue is empty.<br>It will indicate a<br>32-bit aligned address that points to a descriptor when the queue is not empty. |
| 4-0 | DESC_SIZE | R/W-0 | 0 | Descriptor Size.<br>It is encoded in<br>4-byte increments with values 0 to 31 representing 24 and so on to 148 bytes.<br>This field will return a 0x0 when an empty queue is read.<br>Queue Manager Queue N Registers D To save hardware resources, the queue manager internally stores descriptor size (desc_size) information in four bits.<br>However, register D has five LSBs that specify descriptor size.<br>As a consequence, the value of desc_size that is pushed may not be same as that is read during a pop.<br>The value that is read back is equal to always rounded to an odd number.<br>So, for even values, the value read back is one more than what was written.<br>For odd values, the value read back is same as the value that was written.<br>Note that this<br>5-bit field (desc_size) is unrelated to the code for size of descriptors in a descriptor region.<br>It is just a place holder for a<br>5-bit value that is maintained across the push and pop operations for every descriptor managed by the queue manager. |

### 16.4.7.279 QUEUE_61_A Register (offset = 23D0h) [reset = 0h]

QUEUE_61_A is shown in Figure 16-555 and described in Table 16-569.

**Figure 16-555. QUEUE_61_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-569. QUEUE_61_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>This count is incremented by 1 whenever a packet is added to the queue.<br>This count is decremented by 1 whenever a packet is popped from the queue. |

### 16.4.7.280 QUEUE_61_B Register (offset = 23D4h) [reset = 0h]

QUEUE_61_B is shown in Figure 16-556 and described in Table 16-570.

**Figure 16-556. QUEUE_61_B Register**

| 31–28 | 27–0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-570. QUEUE_61_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. |

### 16.4.7.281 QUEUE_61_C Register (offset = 23D8h) [reset = 0h]

QUEUE_61_C is shown in Figure 16-557 and described in Table 16-571.

**Figure 16-557. QUEUE_61_C Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| HEAD_TAIL | Reserved | | | | | | |
| W-0 | | | | | | | |
| **23** | **22** | **21** | **20** | **19** | **18** | **17** | **16** |
| Reserved | | | | | | | |
| **15** | **14** | **13** | **12** | **11** | **10** | **9** | **8** |
| Reserved | | PACKET_SIZE | | | | | |
| | | R/W-0 | | | | | |
| **7** | **6** | **5** | **4** | **3** | **2** | **1** | **0** |
| PACKET_SIZE | | | | | | | |
| R/W-0 | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-571. QUEUE_61_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31 | HEAD_TAIL | W-0 | 0 | Head/Tail Push Control.<br>Set to zero in order to push packet onto tail of queue and set to one in order to push packet onto head of queue. |
| 13-0 | PACKET_SIZE | R/W-0 | 0 | packet_size This field indicates packet size and is assumed to be zero on each packet add unless the value is explicitly overwritten. This field indicates packet size for packet pop operation. |

### 16.4.7.282 QUEUE_61_D Register (offset = 23DCh) [reset = 0h]

QUEUE_61_D is shown in Figure 16-558 and described in Table 16-572.

**Figure 16-558. QUEUE_61_D Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | DESC_SIZE | | | | |
| R/W-0 | | | R/W-0 | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-572. QUEUE_61_D Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-5 | DESC_PTR | R/W-0 | 0 | Descriptor pointer.<br>It will be read as zero if the queue is empty.<br>It will indicate a<br>32-bit aligned address that points to a descriptor when the queue is not empty. |
| 4-0 | DESC_SIZE | R/W-0 | 0 | Descriptor Size.<br>It is encoded in<br>4-byte increments with values 0 to 31 representing 24 and so on to 148 bytes.<br>This field will return a 0x0 when an empty queue is read.<br>Queue Manager Queue N Registers D To save hardware resources, the queue manager internally stores descriptor size (desc_size) information in four bits.<br>However, register D has five LSBs that specify descriptor size.<br>As a consequence, the value of desc_size that is pushed may not be same as that is read during a pop.<br>The value that is read back is equal to always rounded to an odd number.<br>So, for even values, the value read back is one more than what was written.<br>For odd values, the value read back is same as the value that was written.<br>Note that this<br>5-bit field (desc_size) is unrelated to the code for size of descriptors in a descriptor region.<br>It is just a place holder for a<br>5-bit value that is maintained across the push and pop operations for every descriptor managed by the queue manager. |

### 16.4.7.283 QUEUE_62_A Register (offset = 23E0h) [reset = 0h]

QUEUE_62_A is shown in Figure 16-559 and described in Table 16-573.

**Figure 16-559. QUEUE_62_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-573. QUEUE_62_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>This count is incremented by 1 whenever a packet is added to the queue.<br>This count is decremented by 1 whenever a packet is popped from the queue. |

### 16.4.7.284 QUEUE_62_B Register (offset = 23E4h) [reset = 0h]

QUEUE_62_B is shown in Figure 16-560 and described in Table 16-574.

**Figure 16-560. QUEUE_62_B Register**

| 31–28 | 27–0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-574. QUEUE_62_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. |

### 16.4.7.285 QUEUE_62_C Register (offset = 23E8h) [reset = 0h]

QUEUE_62_C is shown in Figure 16-561 and described in Table 16-575.

**Figure 16-561. QUEUE_62_C Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| HEAD_TAIL | Reserved | | | | | | |
| W-0 | | | | | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| Reserved | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| Reserved | | PACKET_SIZE | | | | | |
| | | R/W-0 | | | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| PACKET_SIZE | | | | | | | |
| R/W-0 | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-575. QUEUE_62_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31 | HEAD_TAIL | W-0 | 0 | Head/Tail Push Control.<br>Set to zero in order to push packet onto tail of queue and set to one in order to push packet onto head of queue. |
| 13-0 | PACKET_SIZE | R/W-0 | 0 | packet_size This field indicates packet size and is assumed to be zero on each packet add unless the value is explicitly overwritten. This field indicates packet size for packet pop operation. |

### 16.4.7.286 QUEUE_62_D Register (offset = 23ECh) [reset = 0h]

QUEUE_62_D is shown in Figure 16-562 and described in Table 16-576.

**Figure 16-562. QUEUE_62_D Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |
| **23** | **22** | **21** | **20** | **19** | **18** | **17** | **16** |
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |
| **15** | **14** | **13** | **12** | **11** | **10** | **9** | **8** |
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |
| **7** | **6** | **5** | **4** | **3** | **2** | **1** | **0** |
| DESC_PTR | | | DESC_SIZE | | | | |
| R/W-0 | | | R/W-0 | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-576. QUEUE_62_D Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-5 | DESC_PTR | R/W-0 | 0 | Descriptor pointer.<br>It will be read as zero if the queue is empty.<br>It will indicate a<br>32-bit aligned address that points to a descriptor when the queue is not empty. |
| 4-0 | DESC_SIZE | R/W-0 | 0 | Descriptor Size.<br>It is encoded in<br>4-byte increments with values 0 to 31 representing 24 and so on to 148 bytes.<br>This field will return a 0x0 when an empty queue is read.<br>Queue Manager Queue N Registers D To save hardware resources, the queue manager internally stores descriptor size (desc_size) information in four bits.<br>However, register D has five LSBs that specify descriptor size.<br>As a consequence, the value of desc_size that is pushed may not be same as that is read during a pop.<br>The value that is read back is equal to always rounded to an odd number.<br>So, for even values, the value read back is one more than what was written.<br>For odd values, the value read back is same as the value that was written.<br>Note that this<br>5-bit field (desc_size) is unrelated to the code for size of descriptors in a descriptor region.<br>It is just a place holder for a<br>5-bit value that is maintained across the push and pop operations for every descriptor managed by the queue manager. |

### 16.4.7.287 QUEUE_63_A Register (offset = 23F0h) [reset = 0h]

QUEUE_63_A is shown in Figure 16-563 and described in Table 16-577.

**Figure 16-563. QUEUE_63_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-577. QUEUE_63_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>This count is incremented by 1 whenever a packet is added to the queue.<br>This count is decremented by 1 whenever a packet is popped from the queue. |

### 16.4.7.288 QUEUE_63_B Register (offset = 23F4h) [reset = 0h]

QUEUE_63_B is shown in Figure 16-564 and described in Table 16-578.

**Figure 16-564. QUEUE_63_B Register**

| 31–28 | 27–0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-578. QUEUE_63_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. |

### 16.4.7.289 QUEUE_63_C Register (offset = 23F8h) [reset = 0h]

QUEUE_63_C is shown in Figure 16-565 and described in Table 16-579.

**Figure 16-565. QUEUE_63_C Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| HEAD_TAIL | Reserved | | | | | | |
| W-0 | | | | | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| Reserved | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| Reserved | | PACKET_SIZE | | | | | |
| | | R/W-0 | | | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| PACKET_SIZE | | | | | | | |
| R/W-0 | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-579. QUEUE_63_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31 | HEAD_TAIL | W-0 | 0 | Head/Tail Push Control.<br>Set to zero in order to push packet onto tail of queue and set to one in order to push packet onto head of queue. |
| 13-0 | PACKET_SIZE | R/W-0 | 0 | packet_size This field indicates packet size and is assumed to be zero on each packet add unless the value is explicitly overwritten. This field indicates packet size for packet pop operation. |

### 16.4.7.290 QUEUE_63_D Register (offset = 23FCh) [reset = 0h]

QUEUE_63_D is shown in Figure 16-566 and described in Table 16-580.

**Figure 16-566. QUEUE_63_D Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |
| **23** | **22** | **21** | **20** | **19** | **18** | **17** | **16** |
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |
| **15** | **14** | **13** | **12** | **11** | **10** | **9** | **8** |
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |
| **7** | **6** | **5** | **4** | **3** | **2** | **1** | **0** |
| DESC_PTR | | | DESC_SIZE | | | | |
| R/W-0 | | | R/W-0 | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-580. QUEUE_63_D Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-5 | DESC_PTR | R/W-0 | 0 | Descriptor pointer.<br>It will be read as zero if the queue is empty.<br>It will indicate a<br>32-bit aligned address that points to a descriptor when the queue is not empty. |
| 4-0 | DESC_SIZE | R/W-0 | 0 | Descriptor Size.<br>It is encoded in<br>4-byte increments with values 0 to 31 representing 24 and so on to 148 bytes.<br>This field will return a 0x0 when an empty queue is read.<br>Queue Manager Queue N Registers D To save hardware resources, the queue manager internally stores descriptor size (desc_size) information in four bits.<br>However, register D has five LSBs that specify descriptor size.<br>As a consequence, the value of desc_size that is pushed may not be same as that is read during a pop.<br>The value that is read back is equal to always rounded to an odd number.<br>So, for even values, the value read back is one more than what was written.<br>For odd values, the value read back is same as the value that was written.<br>Note that this<br>5-bit field (desc_size) is unrelated to the code for size of descriptors in a descriptor region.<br>It is just a place holder for a<br>5-bit value that is maintained across the push and pop operations for every descriptor managed by the queue manager. |

### 16.4.7.291 QUEUE_64_A Register (offset = 2400h) [reset = 0h]

QUEUE_64_A is shown in Figure 16-567 and described in Table 16-581.

**Figure 16-567. QUEUE_64_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-581. QUEUE_64_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>This count is incremented by 1 whenever a packet is added to the queue.<br>This count is decremented by 1 whenever a packet is popped from the queue. |

### 16.4.7.292 QUEUE_64_B Register (offset = 2404h) [reset = 0h]

QUEUE_64_B is shown in Figure 16-568 and described in Table 16-582.

**Figure 16-568. QUEUE_64_B Register**

| 31–28 | 27–0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-582. QUEUE_64_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. |

### 16.4.7.293 QUEUE_64_C Register (offset = 2408h) [reset = 0h]

QUEUE_64_C is shown in Figure 16-569 and described in Table 16-583.

**Figure 16-569. QUEUE_64_C Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| HEAD_TAIL | Reserved | | | | | | |
| W-0 | | | | | | | |
| **23** | **22** | **21** | **20** | **19** | **18** | **17** | **16** |
| Reserved | | | | | | | |
| **15** | **14** | **13** | **12** | **11** | **10** | **9** | **8** |
| Reserved | | PACKET_SIZE | | | | | |
| | | R/W-0 | | | | | |
| **7** | **6** | **5** | **4** | **3** | **2** | **1** | **0** |
| PACKET_SIZE | | | | | | | |
| R/W-0 | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-583. QUEUE_64_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31 | HEAD_TAIL | W-0 | 0 | Head/Tail Push Control.<br>Set to zero in order to push packet onto tail of queue and set to one in order to push packet onto head of queue. |
| 13-0 | PACKET_SIZE | R/W-0 | 0 | packet_size This field indicates packet size and is assumed to be zero on each packet add unless the value is explicitly overwritten. This field indicates packet size for packet pop operation. |

### 16.4.7.294 QUEUE_64_D Register (offset = 240Ch) [reset = 0h]

QUEUE_64_D is shown in Figure 16-570 and described in Table 16-584.

**Figure 16-570. QUEUE_64_D Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |
| **23** | **22** | **21** | **20** | **19** | **18** | **17** | **16** |
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |
| **15** | **14** | **13** | **12** | **11** | **10** | **9** | **8** |
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |
| **7** | **6** | **5** | **4** | **3** | **2** | **1** | **0** |
| DESC_PTR | | | DESC_SIZE | | | | |
| R/W-0 | | | R/W-0 | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-584. QUEUE_64_D Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-5 | DESC_PTR | R/W-0 | 0 | Descriptor pointer.<br>It will be read as zero if the queue is empty.<br>It will indicate a<br>32-bit aligned address that points to a descriptor when the queue is not empty. |
| 4-0 | DESC_SIZE | R/W-0 | 0 | Descriptor Size.<br>It is encoded in<br>4-byte increments with values 0 to 31 representing 24 and so on to 148 bytes.<br>This field will return a 0x0 when an empty queue is read.<br>Queue Manager Queue N Registers D To save hardware resources, the queue manager internally stores descriptor size (desc_size) information in four bits.<br>However, register D has five LSBs that specify descriptor size.<br>As a consequence, the value of desc_size that is pushed may not be same as that is read during a pop.<br>The value that is read back is equal to always rounded to an odd number.<br>So, for even values, the value read back is one more than what was written.<br>For odd values, the value read back is same as the value that was written.<br>Note that this<br>5-bit field (desc_size) is unrelated to the code for size of descriptors in a descriptor region.<br>It is just a place holder for a<br>5-bit value that is maintained across the push and pop operations for every descriptor managed by the queue manager. |

### 16.4.7.295 QUEUE_65_A Register (offset = 2410h) [reset = 0h]

QUEUE_65_A is shown in Figure 16-571 and described in Table 16-585.

**Figure 16-571. QUEUE_65_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-585. QUEUE_65_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>This count is incremented by 1 whenever a packet is added to the queue.<br>This count is decremented by 1 whenever a packet is popped from the queue. |

### 16.4.7.296 QUEUE_65_B Register (offset = 2414h) [reset = 0h]

QUEUE_65_B is shown in Figure 16-572 and described in Table 16-586.

**Figure 16-572. QUEUE_65_B Register**

| 31-28 | 27-0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-586. QUEUE_65_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. |

### 16.4.7.297 QUEUE_65_C Register (offset = 2418h) [reset = 0h]

QUEUE_65_C is shown in Figure 16-573 and described in Table 16-587.

**Figure 16-573. QUEUE_65_C Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| HEAD_TAIL | Reserved | | | | | | |
| W-0 | | | | | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| Reserved | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| Reserved | | PACKET_SIZE | | | | | |
| | | R/W-0 | | | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| PACKET_SIZE | | | | | | | |
| R/W-0 | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-587. QUEUE_65_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31 | HEAD_TAIL | W-0 | 0 | Head/Tail Push Control.<br>Set to zero in order to push packet onto tail of queue and set to one in order to push packet onto head of queue. |
| 13-0 | PACKET_SIZE | R/W-0 | 0 | packet_size This field indicates packet size and is assumed to be zero on each packet add unless the value is explicitly overwritten. This field indicates packet size for packet pop operation. |

### 16.4.7.298 QUEUE_65_D Register (offset = 241Ch) [reset = 0h]

QUEUE_65_D is shown in Figure 16-574 and described in Table 16-588.

**Figure 16-574. QUEUE_65_D Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | DESC_SIZE | | | | |
| R/W-0 | | | R/W-0 | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-588. QUEUE_65_D Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-5 | DESC_PTR | R/W-0 | 0 | Descriptor pointer.<br>It will be read as zero if the queue is empty.<br>It will indicate a<br>32-bit aligned address that points to a descriptor when the queue is not empty. |
| 4-0 | DESC_SIZE | R/W-0 | 0 | Descriptor Size.<br>It is encoded in<br>4-byte increments with values 0 to 31 representing 24 and so on to 148 bytes.<br>This field will return a 0x0 when an empty queue is read.<br>Queue Manager Queue N Registers D To save hardware resources, the queue manager internally stores descriptor size (desc_size) information in four bits.<br>However, register D has five LSBs that specify descriptor size.<br>As a consequence, the value of desc_size that is pushed may not be same as that is read during a pop.<br>The value that is read back is equal to always rounded to an odd number.<br>So, for even values, the value read back is one more than what was written.<br>For odd values, the value read back is same as the value that was written.<br>Note that this<br>5-bit field (desc_size) is unrelated to the code for size of descriptors in a descriptor region.<br>It is just a place holder for a<br>5-bit value that is maintained across the push and pop operations for every descriptor managed by the queue manager. |

### 16.4.7.299 QUEUE_66_A Register (offset = 2420h) [reset = 0h]

QUEUE_66_A is shown in Figure 16-575 and described in Table 16-589.

**Figure 16-575. QUEUE_66_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-589. QUEUE_66_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>This count is incremented by 1 whenever a packet is added to the queue.<br>This count is decremented by 1 whenever a packet is popped from the queue. |

### 16.4.7.300 QUEUE_66_B Register (offset = 2424h) [reset = 0h]

QUEUE_66_B is shown in Figure 16-576 and described in Table 16-590.

**Figure 16-576. QUEUE_66_B Register**

| 31–28 | 27–0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-590. QUEUE_66_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. |

### 16.4.7.301 QUEUE_66_C Register (offset = 2428h) [reset = 0h]

QUEUE_66_C is shown in Figure 16-577 and described in Table 16-591.

**Figure 16-577. QUEUE_66_C Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| HEAD_TAIL | Reserved | | | | | | |
| W-0 | | | | | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| Reserved | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| Reserved | | PACKET_SIZE | | | | | |
| | | R/W-0 | | | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| PACKET_SIZE | | | | | | | |
| R/W-0 | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-591. QUEUE_66_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31 | HEAD_TAIL | W-0 | 0 | Head/Tail Push Control.<br>Set to zero in order to push packet onto tail of queue and set to one in order to push packet onto head of queue. |
| 13-0 | PACKET_SIZE | R/W-0 | 0 | packet_size This field indicates packet size and is assumed to be zero on each packet add unless the value is explicitly overwritten. This field indicates packet size for packet pop operation. |

### 16.4.7.302 QUEUE_66_D Register (offset = 242Ch) [reset = 0h]

QUEUE_66_D is shown in Figure 16-578 and described in Table 16-592.

**Figure 16-578. QUEUE_66_D Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | DESC_SIZE | | | | |
| R/W-0 | | | R/W-0 | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-592. QUEUE_66_D Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-5 | DESC_PTR | R/W-0 | 0 | Descriptor pointer.<br>It will be read as zero if the queue is empty.<br>It will indicate a<br>32-bit aligned address that points to a descriptor when the queue is not empty. |
| 4-0 | DESC_SIZE | R/W-0 | 0 | Descriptor Size.<br>It is encoded in<br>4-byte increments with values 0 to 31 representing 24 and so on to 148 bytes.<br>This field will return a 0x0 when an empty queue is read.<br>Queue Manager Queue N Registers D To save hardware resources, the queue manager internally stores descriptor size (desc_size) information in four bits.<br>However, register D has five LSBs that specify descriptor size.<br>As a consequence, the value of desc_size that is pushed may not be same as that is read during a pop.<br>The value that is read back is equal to always rounded to an odd number.<br>So, for even values, the value read back is one more than what was written.<br>For odd values, the value read back is same as the value that was written.<br>Note that this<br>5-bit field (desc_size) is unrelated to the code for size of descriptors in a descriptor region.<br>It is just a place holder for a<br>5-bit value that is maintained across the push and pop operations for every descriptor managed by the queue manager. |

### 16.4.7.303 QUEUE_67_A Register (offset = 2430h) [reset = 0h]

QUEUE_67_A is shown in Figure 16-579 and described in Table 16-593.

**Figure 16-579. QUEUE_67_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-593. QUEUE_67_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>This count is incremented by 1 whenever a packet is added to the queue.<br>This count is decremented by 1 whenever a packet is popped from the queue. |

### 16.4.7.304 QUEUE_67_B Register (offset = 2434h) [reset = 0h]

QUEUE_67_B is shown in Figure 16-580 and described in Table 16-594.

**Figure 16-580. QUEUE_67_B Register**

| 31–28 | 27–0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-594. QUEUE_67_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. |

### 16.4.7.305 QUEUE_67_C Register (offset = 2438h) [reset = 0h]

QUEUE_67_C is shown in Figure 16-581 and described in Table 16-595.

**Figure 16-581. QUEUE_67_C Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| HEAD_TAIL | Reserved | | | | | | |
| W-0 | | | | | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| Reserved | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| Reserved | | PACKET_SIZE | | | | | |
| | | R/W-0 | | | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| PACKET_SIZE | | | | | | | |
| R/W-0 | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-595. QUEUE_67_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31 | HEAD_TAIL | W-0 | 0 | Head/Tail Push Control.<br>Set to zero in order to push packet onto tail of queue and set to one in order to push packet onto head of queue. |
| 13-0 | PACKET_SIZE | R/W-0 | 0 | packet_size This field indicates packet size and is assumed to be zero on each packet add unless the value is explicitly overwritten. This field indicates packet size for packet pop operation. |

### 16.4.7.306 QUEUE_67_D Register (offset = 243Ch) [reset = 0h]

QUEUE_67_D is shown in Figure 16-582 and described in Table 16-596.

**Figure 16-582. QUEUE_67_D Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | DESC_SIZE | | | | |
| R/W-0 | | | R/W-0 | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-596. QUEUE_67_D Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-5 | DESC_PTR | R/W-0 | 0 | Descriptor pointer.<br>It will be read as zero if the queue is empty.<br>It will indicate a<br>32-bit aligned address that points to a descriptor when the queue is not empty. |
| 4-0 | DESC_SIZE | R/W-0 | 0 | Descriptor Size.<br>It is encoded in<br>4-byte increments with values 0 to 31 representing 24 and so on to 148 bytes.<br>This field will return a 0x0 when an empty queue is read.<br>Queue Manager Queue N Registers D To save hardware resources, the queue manager internally stores descriptor size (desc_size) information in four bits.<br>However, register D has five LSBs that specify descriptor size.<br>As a consequence, the value of desc_size that is pushed may not be same as that is read during a pop.<br>The value that is read back is equal to always rounded to an odd number.<br>So, for even values, the value read back is one more than what was written.<br>For odd values, the value read back is same as the value that was written.<br>Note that this<br>5-bit field (desc_size) is unrelated to the code for size of descriptors in a descriptor region.<br>It is just a place holder for a<br>5-bit value that is maintained across the push and pop operations for every descriptor managed by the queue manager. |

### 16.4.7.307 QUEUE_68_A Register (offset = 2440h) [reset = 0h]

QUEUE_68_A is shown in Figure 16-583 and described in Table 16-597.

**Figure 16-583. QUEUE_68_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-597. QUEUE_68_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>This count is incremented by 1 whenever a packet is added to the queue.<br>This count is decremented by 1 whenever a packet is popped from the queue. |

### 16.4.7.308 QUEUE_68_B Register (offset = 2444h) [reset = 0h]

QUEUE_68_B is shown in Figure 16-584 and described in Table 16-598.

**Figure 16-584. QUEUE_68_B Register**

| 31–28 | 27–0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-598. QUEUE_68_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. |

### 16.4.7.309 QUEUE_68_C Register (offset = 2448h) [reset = 0h]

QUEUE_68_C is shown in Figure 16-585 and described in Table 16-599.

**Figure 16-585. QUEUE_68_C Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| HEAD_TAIL | Reserved | | | | | | |
| W-0 | | | | | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| Reserved | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| Reserved | | PACKET_SIZE | | | | | |
| | | R/W-0 | | | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| PACKET_SIZE | | | | | | | |
| R/W-0 | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-599. QUEUE_68_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31 | HEAD_TAIL | W-0 | 0 | Head/Tail Push Control.<br>Set to zero in order to push packet onto tail of queue and set to one in order to push packet onto head of queue. |
| 13-0 | PACKET_SIZE | R/W-0 | 0 | packet_size This field indicates packet size and is assumed to be zero on each packet add unless the value is explicitly overwritten. This field indicates packet size for packet pop operation. |

### 16.4.7.310 QUEUE_68_D Register (offset = 244Ch) [reset = 0h]

QUEUE_68_D is shown in Figure 16-586 and described in Table 16-600.

**Figure 16-586. QUEUE_68_D Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |
| **23** | **22** | **21** | **20** | **19** | **18** | **17** | **16** |
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |
| **15** | **14** | **13** | **12** | **11** | **10** | **9** | **8** |
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |
| **7** | **6** | **5** | **4** | **3** | **2** | **1** | **0** |
| DESC_PTR | | | DESC_SIZE | | | | |
| R/W-0 | | | R/W-0 | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-600. QUEUE_68_D Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-5 | DESC_PTR | R/W-0 | 0 | Descriptor pointer.<br>It will be read as zero if the queue is empty.<br>It will indicate a<br>32-bit aligned address that points to a descriptor when the queue is not empty. |
| 4-0 | DESC_SIZE | R/W-0 | 0 | Descriptor Size.<br>It is encoded in<br>4-byte increments with values 0 to 31 representing 24 and so on to 148 bytes.<br>This field will return a 0x0 when an empty queue is read.<br>Queue Manager Queue N Registers D To save hardware resources, the queue manager internally stores descriptor size (desc_size) information in four bits.<br>However, register D has five LSBs that specify descriptor size.<br>As a consequence, the value of desc_size that is pushed may not be same as that is read during a pop.<br>The value that is read back is equal to always rounded to an odd number.<br>So, for even values, the value read back is one more than what was written.<br>For odd values, the value read back is same as the value that was written.<br>Note that this<br>5-bit field (desc_size) is unrelated to the code for size of descriptors in a descriptor region.<br>It is just a place holder for a<br>5-bit value that is maintained across the push and pop operations for every descriptor managed by the queue manager. |

### 16.4.7.311 QUEUE_69_A Register (offset = 2450h) [reset = 0h]

QUEUE_69_A is shown in Figure 16-587 and described in Table 16-601.

**Figure 16-587. QUEUE_69_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-601. QUEUE_69_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>This count is incremented by 1 whenever a packet is added to the queue.<br>This count is decremented by 1 whenever a packet is popped from the queue. |

### 16.4.7.312 QUEUE_69_B Register (offset = 2454h) [reset = 0h]

QUEUE_69_B is shown in Figure 16-588 and described in Table 16-602.

**Figure 16-588. QUEUE_69_B Register**

| 31-28 | 27-0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-602. QUEUE_69_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. |

### 16.4.7.313 QUEUE_69_C Register (offset = 2458h) [reset = 0h]

QUEUE_69_C is shown in Figure 16-589 and described in Table 16-603.

**Figure 16-589. QUEUE_69_C Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| HEAD_TAIL | Reserved | | | | | | |
| W-0 | | | | | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| Reserved | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| Reserved | | PACKET_SIZE | | | | | |
| | | R/W-0 | | | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| PACKET_SIZE | | | | | | | |
| R/W-0 | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-603. QUEUE_69_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31 | HEAD_TAIL | W-0 | 0 | Head/Tail Push Control.<br>Set to zero in order to push packet onto tail of queue and set to one in order to push packet onto head of queue. |
| 13-0 | PACKET_SIZE | R/W-0 | 0 | packet_size This field indicates packet size and is assumed to be zero on each packet add unless the value is explicitly overwritten. This field indicates packet size for packet pop operation. |

### 16.4.7.314 QUEUE_69_D Register (offset = 245Ch) [reset = 0h]

QUEUE_69_D is shown in Figure 16-590 and described in Table 16-604.

**Figure 16-590. QUEUE_69_D Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | DESC_SIZE | | | | |
| R/W-0 | | | R/W-0 | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-604. QUEUE_69_D Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-5 | DESC_PTR | R/W-0 | 0 | Descriptor pointer.<br>It will be read as zero if the queue is empty.<br>It will indicate a<br>32-bit aligned address that points to a descriptor when the queue is not empty. |
| 4-0 | DESC_SIZE | R/W-0 | 0 | Descriptor Size.<br>It is encoded in<br>4-byte increments with values 0 to 31 representing 24 and so on to 148 bytes.<br>This field will return a 0x0 when an empty queue is read.<br>Queue Manager Queue N Registers D To save hardware resources, the queue manager internally stores descriptor size (desc_size) information in four bits.<br>However, register D has five LSBs that specify descriptor size.<br>As a consequence, the value of desc_size that is pushed may not be same as that is read during a pop.<br>The value that is read back is equal to always rounded to an odd number.<br>So, for even values, the value read back is one more than what was written.<br>For odd values, the value read back is same as the value that was written.<br>Note that this<br>5-bit field (desc_size) is unrelated to the code for size of descriptors in a descriptor region.<br>It is just a place holder for a<br>5-bit value that is maintained across the push and pop operations for every descriptor managed by the queue manager. |

### 16.4.7.315 QUEUE_70_A Register (offset = 2460h) [reset = 0h]

QUEUE_70_A is shown in Figure 16-591 and described in Table 16-605.

**Figure 16-591. QUEUE_70_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-605. QUEUE_70_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>This count is incremented by 1 whenever a packet is added to the queue.<br>This count is decremented by 1 whenever a packet is popped from the queue. |

### 16.4.7.316 QUEUE_70_B Register (offset = 2464h) [reset = 0h]

QUEUE_70_B is shown in Figure 16-592 and described in Table 16-606.

**Figure 16-592. QUEUE_70_B Register**

| 31–28 | 27–0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-606. QUEUE_70_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. |

### 16.4.7.317 QUEUE_70_C Register (offset = 2468h) [reset = 0h]

QUEUE_70_C is shown in Figure 16-593 and described in Table 16-607.

**Figure 16-593. QUEUE_70_C Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| HEAD_TAIL | Reserved | | | | | | |
| W-0 | | | | | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| Reserved | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| Reserved | | PACKET_SIZE | | | | | |
| | | R/W-0 | | | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| PACKET_SIZE | | | | | | | |
| R/W-0 | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-607. QUEUE_70_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31 | HEAD_TAIL | W-0 | 0 | Head/Tail Push Control.<br>Set to zero in order to push packet onto tail of queue and set to one in order to push packet onto head of queue. |
| 13-0 | PACKET_SIZE | R/W-0 | 0 | packet_size This field indicates packet size and is assumed to be zero on each packet add unless the value is explicitly overwritten. This field indicates packet size for packet pop operation. |

### 16.4.7.318 QUEUE_70_D Register (offset = 246Ch) [reset = 0h]

QUEUE_70_D is shown in Figure 16-594 and described in Table 16-608.

**Figure 16-594. QUEUE_70_D Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |
| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |
| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |
| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
| DESC_PTR | | | DESC_SIZE | | | | |
| R/W-0 | | | R/W-0 | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-608. QUEUE_70_D Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-5 | DESC_PTR | R/W-0 | 0 | Descriptor pointer.<br>It will be read as zero if the queue is empty.<br>It will indicate a<br>32-bit aligned address that points to a descriptor when the queue is not empty. |
| 4-0 | DESC_SIZE | R/W-0 | 0 | Descriptor Size.<br>It is encoded in<br>4-byte increments with values 0 to 31 representing 24 and so on to 148 bytes.<br>This field will return a 0x0 when an empty queue is read.<br>Queue Manager Queue N Registers D To save hardware resources, the queue manager internally stores descriptor size (desc_size) information in four bits.<br>However, register D has five LSBs that specify descriptor size.<br>As a consequence, the value of desc_size that is pushed may not be same as that is read during a pop.<br>The value that is read back is equal to always rounded to an odd number.<br>So, for even values, the value read back is one more than what was written.<br>For odd values, the value read back is same as the value that was written.<br>Note that this<br>5-bit field (desc_size) is unrelated to the code for size of descriptors in a descriptor region.<br>It is just a place holder for a<br>5-bit value that is maintained across the push and pop operations for every descriptor managed by the queue manager. |

### 16.4.7.319 QUEUE_71_A Register (offset = 2470h) [reset = 0h]

QUEUE_71_A is shown in Figure 16-595 and described in Table 16-609.

**Figure 16-595. QUEUE_71_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-609. QUEUE_71_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>This count is incremented by 1 whenever a packet is added to the queue.<br>This count is decremented by 1 whenever a packet is popped from the queue. |

### 16.4.7.320 QUEUE_71_B Register (offset = 2474h) [reset = 0h]

QUEUE_71_B is shown in Figure 16-596 and described in Table 16-610.

**Figure 16-596. QUEUE_71_B Register**

| 31–28 | 27–0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-610. QUEUE_71_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. |

### 16.4.7.321 QUEUE_71_C Register (offset = 2478h) [reset = 0h]

QUEUE_71_C is shown in Figure 16-597 and described in Table 16-611.

**Figure 16-597. QUEUE_71_C Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| HEAD_TAIL | Reserved | | | | | | |
| W-0 | | | | | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| Reserved | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| Reserved | | PACKET_SIZE | | | | | |
| | | R/W-0 | | | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| PACKET_SIZE | | | | | | | |
| R/W-0 | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-611. QUEUE_71_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31 | HEAD_TAIL | W-0 | 0 | Head/Tail Push Control.<br>Set to zero in order to push packet onto tail of queue and set to one in order to push packet onto head of queue. |
| 13-0 | PACKET_SIZE | R/W-0 | 0 | packet_size This field indicates packet size and is assumed to be zero on each packet add unless the value is explicitly overwritten. This field indicates packet size for packet pop operation. |

### 16.4.7.322 QUEUE_71_D Register (offset = 247Ch) [reset = 0h]

QUEUE_71_D is shown in Figure 16-598 and described in Table 16-612.

**Figure 16-598. QUEUE_71_D Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |
| **23** | **22** | **21** | **20** | **19** | **18** | **17** | **16** |
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |
| **15** | **14** | **13** | **12** | **11** | **10** | **9** | **8** |
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |
| **7** | **6** | **5** | **4** | **3** | **2** | **1** | **0** |
| DESC_PTR | | | DESC_SIZE | | | | |
| R/W-0 | | | R/W-0 | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-612. QUEUE_71_D Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-5 | DESC_PTR | R/W-0 | 0 | Descriptor pointer.<br>It will be read as zero if the queue is empty.<br>It will indicate a<br>32-bit aligned address that points to a descriptor when the queue is not empty. |
| 4-0 | DESC_SIZE | R/W-0 | 0 | Descriptor Size.<br>It is encoded in<br>4-byte increments with values 0 to 31 representing 24 and so on to 148 bytes.<br>This field will return a 0x0 when an empty queue is read.<br>Queue Manager Queue N Registers D To save hardware resources, the queue manager internally stores descriptor size (desc_size) information in four bits.<br>However, register D has five LSBs that specify descriptor size.<br>As a consequence, the value of desc_size that is pushed may not be same as that is read during a pop.<br>The value that is read back is equal to always rounded to an odd number.<br>So, for even values, the value read back is one more than what was written.<br>For odd values, the value read back is same as the value that was written.<br>Note that this<br>5-bit field (desc_size) is unrelated to the code for size of descriptors in a descriptor region.<br>It is just a place holder for a<br>5-bit value that is maintained across the push and pop operations for every descriptor managed by the queue manager. |

### 16.4.7.323 QUEUE_72_A Register (offset = 2480h) [reset = 0h]

QUEUE_72_A is shown in Figure 16-599 and described in Table 16-613.

**Figure 16-599. QUEUE_72_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-613. QUEUE_72_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>This count is incremented by 1 whenever a packet is added to the queue.<br>This count is decremented by 1 whenever a packet is popped from the queue. |

### 16.4.7.324 QUEUE_72_B Register (offset = 2484h) [reset = 0h]

QUEUE_72_B is shown in Figure 16-600 and described in Table 16-614.

**Figure 16-600. QUEUE_72_B Register**

| 31–28 | 27–0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-614. QUEUE_72_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. |

### 16.4.7.325 QUEUE_72_C Register (offset = 2488h) [reset = 0h]

QUEUE_72_C is shown in Figure 16-601 and described in Table 16-615.

**Figure 16-601. QUEUE_72_C Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| HEAD_TAIL | Reserved | | | | | | |
| W-0 | | | | | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| Reserved | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| Reserved | | PACKET_SIZE | | | | | |
| | | R/W-0 | | | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| PACKET_SIZE | | | | | | | |
| R/W-0 | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-615. QUEUE_72_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31 | HEAD_TAIL | W-0 | 0 | Head/Tail Push Control.<br>Set to zero in order to push packet onto tail of queue and set to one in order to push packet onto head of queue. |
| 13-0 | PACKET_SIZE | R/W-0 | 0 | packet_size This field indicates packet size and is assumed to be zero on each packet add unless the value is explicitly overwritten. This field indicates packet size for packet pop operation. |

### 16.4.7.326 QUEUE_72_D Register (offset = 248Ch) [reset = 0h]

QUEUE_72_D is shown in Figure 16-602 and described in Table 16-616.

**Figure 16-602. QUEUE_72_D Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | DESC_SIZE | | | | |
| R/W-0 | | | R/W-0 | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-616. QUEUE_72_D Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-5 | DESC_PTR | R/W-0 | 0 | Descriptor pointer.<br>It will be read as zero if the queue is empty.<br>It will indicate a<br>32-bit aligned address that points to a descriptor when the queue is not empty. |
| 4-0 | DESC_SIZE | R/W-0 | 0 | Descriptor Size.<br>It is encoded in<br>4-byte increments with values 0 to 31 representing 24 and so on to 148 bytes.<br>This field will return a 0x0 when an empty queue is read.<br>Queue Manager Queue N Registers D To save hardware resources, the queue manager internally stores descriptor size (desc_size) information in four bits.<br>However, register D has five LSBs that specify descriptor size.<br>As a consequence, the value of desc_size that is pushed may not be same as that is read during a pop.<br>The value that is read back is equal to always rounded to an odd number.<br>So, for even values, the value read back is one more than what was written.<br>For odd values, the value read back is same as the value that was written.<br>Note that this<br>5-bit field (desc_size) is unrelated to the code for size of descriptors in a descriptor region.<br>It is just a place holder for a<br>5-bit value that is maintained across the push and pop operations for every descriptor managed by the queue manager. |

### 16.4.7.327 QUEUE_73_A Register (offset = 2490h) [reset = 0h]

QUEUE_73_A is shown in Figure 16-603 and described in Table 16-617.

**Figure 16-603. QUEUE_73_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-617. QUEUE_73_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>This count is incremented by 1 whenever a packet is added to the queue.<br>This count is decremented by 1 whenever a packet is popped from the queue. |

### 16.4.7.328 QUEUE_73_B Register (offset = 2494h) [reset = 0h]

QUEUE_73_B is shown in Figure 16-604 and described in Table 16-618.

**Figure 16-604. QUEUE_73_B Register**

| 31–28 | 27–0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-618. QUEUE_73_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. |

### 16.4.7.329 QUEUE_73_C Register (offset = 2498h) [reset = 0h]

QUEUE_73_C is shown in Figure 16-605 and described in Table 16-619.

**Figure 16-605. QUEUE_73_C Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| HEAD_TAIL | Reserved | | | | | | |
| W-0 | | | | | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| Reserved | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| Reserved | | PACKET_SIZE | | | | | |
| | | R/W-0 | | | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| PACKET_SIZE | | | | | | | |
| R/W-0 | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-619. QUEUE_73_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31 | HEAD_TAIL | W-0 | 0 | Head/Tail Push Control.<br>Set to zero in order to push packet onto tail of queue and set to one in order to push packet onto head of queue. |
| 13-0 | PACKET_SIZE | R/W-0 | 0 | packet_size This field indicates packet size and is assumed to be zero on each packet add unless the value is explicitly overwritten. This field indicates packet size for packet pop operation. |

### 16.4.7.330 QUEUE_73_D Register (offset = 249Ch) [reset = 0h]

QUEUE_73_D is shown in Figure 16-606 and described in Table 16-620.

**Figure 16-606. QUEUE_73_D Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | DESC_SIZE | | | | |
| R/W-0 | | | R/W-0 | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-620. QUEUE_73_D Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-5 | DESC_PTR | R/W-0 | 0 | Descriptor pointer.<br>It will be read as zero if the queue is empty.<br>It will indicate a<br>32-bit aligned address that points to a descriptor when the queue is not empty. |
| 4-0 | DESC_SIZE | R/W-0 | 0 | Descriptor Size.<br>It is encoded in<br>4-byte increments with values 0 to 31 representing 24 and so on to 148 bytes.<br>This field will return a 0x0 when an empty queue is read.<br>Queue Manager Queue N Registers D To save hardware resources, the queue manager internally stores descriptor size (desc_size) information in four bits.<br>However, register D has five LSBs that specify descriptor size.<br>As a consequence, the value of desc_size that is pushed may not be same as that is read during a pop.<br>The value that is read back is equal to always rounded to an odd number.<br>So, for even values, the value read back is one more than what was written.<br>For odd values, the value read back is same as the value that was written.<br>Note that this<br>5-bit field (desc_size) is unrelated to the code for size of descriptors in a descriptor region.<br>It is just a place holder for a<br>5-bit value that is maintained across the push and pop operations for every descriptor managed by the queue manager. |

### 16.4.7.331 QUEUE_74_A Register (offset = 24A0h) [reset = 0h]

QUEUE_74_A is shown in Figure 16-607 and described in Table 16-621.

**Figure 16-607. QUEUE_74_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-621. QUEUE_74_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>This count is incremented by 1 whenever a packet is added to the queue.<br>This count is decremented by 1 whenever a packet is popped from the queue. |

### 16.4.7.332 QUEUE_74_B Register (offset = 24A4h) [reset = 0h]

QUEUE_74_B is shown in Figure 16-608 and described in Table 16-622.

**Figure 16-608. QUEUE_74_B Register**

| 31–28 | 27–0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-622. QUEUE_74_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. |

### 16.4.7.333 QUEUE_74_C Register (offset = 24A8h) [reset = 0h]

QUEUE_74_C is shown in Figure 16-609 and described in Table 16-623.

**Figure 16-609. QUEUE_74_C Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| HEAD_TAIL | Reserved | | | | | | |
| W-0 | | | | | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| Reserved | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| Reserved | | PACKET_SIZE | | | | | |
| | | R/W-0 | | | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| PACKET_SIZE | | | | | | | |
| R/W-0 | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-623. QUEUE_74_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31 | HEAD_TAIL | W-0 | 0 | Head/Tail Push Control.<br>Set to zero in order to push packet onto tail of queue and set to one in order to push packet onto head of queue. |
| 13-0 | PACKET_SIZE | R/W-0 | 0 | packet_size This field indicates packet size and is assumed to be zero on each packet add unless the value is explicitly overwritten. This field indicates packet size for packet pop operation. |

### 16.4.7.334 QUEUE_74_D Register (offset = 24ACh) [reset = 0h]

QUEUE_74_D is shown in Figure 16-610 and described in Table 16-624.

**Figure 16-610. QUEUE_74_D Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |
| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |
| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |
| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
| DESC_PTR | | | DESC_SIZE | | | | |
| R/W-0 | | | R/W-0 | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-624. QUEUE_74_D Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-5 | DESC_PTR | R/W-0 | 0 | Descriptor pointer.<br>It will be read as zero if the queue is empty.<br>It will indicate a<br>32-bit aligned address that points to a descriptor when the queue is not empty. |
| 4-0 | DESC_SIZE | R/W-0 | 0 | Descriptor Size.<br>It is encoded in<br>4-byte increments with values 0 to 31 representing 24 and so on to 148 bytes.<br>This field will return a 0x0 when an empty queue is read.<br>Queue Manager Queue N Registers D To save hardware resources, the queue manager internally stores descriptor size (desc_size) information in four bits.<br>However, register D has five LSBs that specify descriptor size.<br>As a consequence, the value of desc_size that is pushed may not be same as that is read during a pop.<br>The value that is read back is equal to always rounded to an odd number.<br>So, for even values, the value read back is one more than what was written.<br>For odd values, the value read back is same as the value that was written.<br>Note that this<br>5-bit field (desc_size) is unrelated to the code for size of descriptors in a descriptor region.<br>It is just a place holder for a<br>5-bit value that is maintained across the push and pop operations for every descriptor managed by the queue manager. |

### 16.4.7.335 QUEUE_75_A Register (offset = 24B0h) [reset = 0h]

QUEUE_75_A is shown in Figure 16-611 and described in Table 16-625.

**Figure 16-611. QUEUE_75_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-625. QUEUE_75_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>This count is incremented by 1 whenever a packet is added to the queue.<br>This count is decremented by 1 whenever a packet is popped from the queue. |

### 16.4.7.336 QUEUE_75_B Register (offset = 24B4h) [reset = 0h]

QUEUE_75_B is shown in Figure 16-612 and described in Table 16-626.

**Figure 16-612. QUEUE_75_B Register**

| 31–28 | 27–0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-626. QUEUE_75_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. |

### 16.4.7.337 QUEUE_75_C Register (offset = 24B8h) [reset = 0h]

QUEUE_75_C is shown in Figure 16-613 and described in Table 16-627.

**Figure 16-613. QUEUE_75_C Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| HEAD_TAIL | Reserved | | | | | | |
| W-0 | | | | | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| Reserved | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| Reserved | | PACKET_SIZE | | | | | |
| | | R/W-0 | | | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| PACKET_SIZE | | | | | | | |
| R/W-0 | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-627. QUEUE_75_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31 | HEAD_TAIL | W-0 | 0 | Head/Tail Push Control.<br>Set to zero in order to push packet onto tail of queue and set to one in order to push packet onto head of queue. |
| 13-0 | PACKET_SIZE | R/W-0 | 0 | packet_size This field indicates packet size and is assumed to be zero on each packet add unless the value is explicitly overwritten. This field indicates packet size for packet pop operation. |

### 16.4.7.338 QUEUE_75_D Register (offset = 24BCh) [reset = 0h]

QUEUE_75_D is shown in Figure 16-614 and described in Table 16-628.

**Figure 16-614. QUEUE_75_D Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |
| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |
| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |
| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
| DESC_PTR | | | DESC_SIZE | | | | |
| R/W-0 | | | R/W-0 | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-628. QUEUE_75_D Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-5 | DESC_PTR | R/W-0 | 0 | Descriptor pointer.<br>It will be read as zero if the queue is empty.<br>It will indicate a<br>32-bit aligned address that points to a descriptor when the queue is not empty. |
| 4-0 | DESC_SIZE | R/W-0 | 0 | Descriptor Size.<br>It is encoded in<br>4-byte increments with values 0 to 31 representing 24 and so on to 148 bytes.<br>This field will return a 0x0 when an empty queue is read.<br>Queue Manager Queue N Registers D To save hardware resources, the queue manager internally stores descriptor size (desc_size) information in four bits.<br>However, register D has five LSBs that specify descriptor size.<br>As a consequence, the value of desc_size that is pushed may not be same as that is read during a pop.<br>The value that is read back is equal to always rounded to an odd number.<br>So, for even values, the value read back is one more than what was written.<br>For odd values, the value read back is same as the value that was written.<br>Note that this<br>5-bit field (desc_size) is unrelated to the code for size of descriptors in a descriptor region.<br>It is just a place holder for a<br>5-bit value that is maintained across the push and pop operations for every descriptor managed by the queue manager. |

### 16.4.7.339 QUEUE_76_A Register (offset = 24C0h) [reset = 0h]

QUEUE_76_A is shown in Figure 16-615 and described in Table 16-629.

**Figure 16-615. QUEUE_76_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-629. QUEUE_76_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>This count is incremented by 1 whenever a packet is added to the queue.<br>This count is decremented by 1 whenever a packet is popped from the queue. |

### 16.4.7.340 QUEUE_76_B Register (offset = 24C4h) [reset = 0h]

QUEUE_76_B is shown in Figure 16-616 and described in Table 16-630.

**Figure 16-616. QUEUE_76_B Register**

| 31–28 | 27–0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-630. QUEUE_76_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. |

### 16.4.7.341 QUEUE_76_C Register (offset = 24C8h) [reset = 0h]

QUEUE_76_C is shown in Figure 16-617 and described in Table 16-631.

**Figure 16-617. QUEUE_76_C Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| HEAD_TAIL | Reserved | | | | | | |
| W-0 | | | | | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| Reserved | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| Reserved | | PACKET_SIZE | | | | | |
| | | R/W-0 | | | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| PACKET_SIZE | | | | | | | |
| R/W-0 | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-631. QUEUE_76_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31 | HEAD_TAIL | W-0 | 0 | Head/Tail Push Control.<br>Set to zero in order to push packet onto tail of queue and set to one in order to push packet onto head of queue. |
| 13-0 | PACKET_SIZE | R/W-0 | 0 | packet_size This field indicates packet size and is assumed to be zero on each packet add unless the value is explicitly overwritten.<br>This field indicates packet size for packet pop operation. |

### 16.4.7.342 QUEUE_76_D Register (offset = 24CCh) [reset = 0h]

QUEUE_76_D is shown in Figure 16-618 and described in Table 16-632.

**Figure 16-618. QUEUE_76_D Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | DESC_SIZE | | | | |
| R/W-0 | | | R/W-0 | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-632. QUEUE_76_D Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-5 | DESC_PTR | R/W-0 | 0 | Descriptor pointer.<br>It will be read as zero if the queue is empty.<br>It will indicate a<br>32-bit aligned address that points to a descriptor when the queue is not empty. |
| 4-0 | DESC_SIZE | R/W-0 | 0 | Descriptor Size.<br>It is encoded in<br>4-byte increments with values 0 to 31 representing 24 and so on to 148 bytes.<br>This field will return a 0x0 when an empty queue is read.<br>Queue Manager Queue N Registers D To save hardware resources, the queue manager internally stores descriptor size (desc_size) information in four bits.<br>However, register D has five LSBs that specify descriptor size.<br>As a consequence, the value of desc_size that is pushed may not be same as that is read during a pop.<br>The value that is read back is equal to always rounded to an odd number.<br>So, for even values, the value read back is one more than what was written.<br>For odd values, the value read back is same as the value that was written.<br>Note that this<br>5-bit field (desc_size) is unrelated to the code for size of descriptors in a descriptor region.<br>It is just a place holder for a<br>5-bit value that is maintained across the push and pop operations for every descriptor managed by the queue manager. |

### 16.4.7.343 QUEUE_77_A Register (offset = 24D0h) [reset = 0h]

QUEUE_77_A is shown in Figure 16-619 and described in Table 16-633.

**Figure 16-619. QUEUE_77_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-633. QUEUE_77_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>This count is incremented by 1 whenever a packet is added to the queue.<br>This count is decremented by 1 whenever a packet is popped from the queue. |

### 16.4.7.344 QUEUE_77_B Register (offset = 24D4h) [reset = 0h]

QUEUE_77_B is shown in Figure 16-620 and described in Table 16-634.

**Figure 16-620. QUEUE_77_B Register**

| 31–28 | 27–0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-634. QUEUE_77_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. |

### 16.4.7.345 QUEUE_77_C Register (offset = 24D8h) [reset = 0h]

QUEUE_77_C is shown in Figure 16-621 and described in Table 16-635.

**Figure 16-621. QUEUE_77_C Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| HEAD_TAIL | Reserved | | | | | | |
| W-0 | | | | | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| Reserved | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| Reserved | | PACKET_SIZE | | | | | |
| | | R/W-0 | | | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| PACKET_SIZE | | | | | | | |
| R/W-0 | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-635. QUEUE_77_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31 | HEAD_TAIL | W-0 | 0 | Head/Tail Push Control.<br>Set to zero in order to push packet onto tail of queue and set to one in order to push packet onto head of queue. |
| 13-0 | PACKET_SIZE | R/W-0 | 0 | packet_size This field indicates packet size and is assumed to be zero on each packet add unless the value is explicitly overwritten. This field indicates packet size for packet pop operation. |

### 16.4.7.346 QUEUE_77_D Register (offset = 24DCh) [reset = 0h]

QUEUE_77_D is shown in Figure 16-622 and described in Table 16-636.

**Figure 16-622. QUEUE_77_D Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |
| **23** | **22** | **21** | **20** | **19** | **18** | **17** | **16** |
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |
| **15** | **14** | **13** | **12** | **11** | **10** | **9** | **8** |
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |
| **7** | **6** | **5** | **4** | **3** | **2** | **1** | **0** |
| DESC_PTR | | | DESC_SIZE | | | | |
| R/W-0 | | | R/W-0 | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-636. QUEUE_77_D Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-5 | DESC_PTR | R/W-0 | 0 | Descriptor pointer.<br>It will be read as zero if the queue is empty.<br>It will indicate a<br>32-bit aligned address that points to a descriptor when the queue is not empty. |
| 4-0 | DESC_SIZE | R/W-0 | 0 | Descriptor Size.<br>It is encoded in<br>4-byte increments with values 0 to 31 representing 24 and so on to 148 bytes.<br>This field will return a 0x0 when an empty queue is read.<br>Queue Manager Queue N Registers D To save hardware resources, the queue manager internally stores descriptor size (desc_size) information in four bits.<br>However, register D has five LSBs that specify descriptor size.<br>As a consequence, the value of desc_size that is pushed may not be same as that is read during a pop.<br>The value that is read back is equal to always rounded to an odd number.<br>So, for even values, the value read back is one more than what was written.<br>For odd values, the value read back is same as the value that was written.<br>Note that this<br>5-bit field (desc_size) is unrelated to the code for size of descriptors in a descriptor region.<br>It is just a place holder for a<br>5-bit value that is maintained across the push and pop operations for every descriptor managed by the queue manager. |

### 16.4.7.347 QUEUE_78_A Register (offset = 24E0h) [reset = 0h]

QUEUE_78_A is shown in Figure 16-623 and described in Table 16-637.

**Figure 16-623. QUEUE_78_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-637. QUEUE_78_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>This count is incremented by 1 whenever a packet is added to the queue.<br>This count is decremented by 1 whenever a packet is popped from the queue. |

### 16.4.7.348 QUEUE_78_B Register (offset = 24E4h) [reset = 0h]

QUEUE_78_B is shown in Figure 16-624 and described in Table 16-638.

**Figure 16-624. QUEUE_78_B Register**

| 31–28 | 27–0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-638. QUEUE_78_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. |

### 16.4.7.349 QUEUE_78_C Register (offset = 24E8h) [reset = 0h]

QUEUE_78_C is shown in Figure 16-625 and described in Table 16-639.

**Figure 16-625. QUEUE_78_C Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| HEAD_TAIL | Reserved | | | | | | |
| W-0 | | | | | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| Reserved | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| Reserved | | PACKET_SIZE | | | | | |
| | | R/W-0 | | | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| PACKET_SIZE | | | | | | | |
| R/W-0 | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-639. QUEUE_78_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31 | HEAD_TAIL | W-0 | 0 | Head/Tail Push Control.<br>Set to zero in order to push packet onto tail of queue and set to one in order to push packet onto head of queue. |
| 13-0 | PACKET_SIZE | R/W-0 | 0 | packet_size This field indicates packet size and is assumed to be zero on each packet add unless the value is explicitly overwritten. This field indicates packet size for packet pop operation. |

### 16.4.7.350 QUEUE_78_D Register (offset = 24ECh) [reset = 0h]

QUEUE_78_D is shown in Figure 16-626 and described in Table 16-640.

**Figure 16-626. QUEUE_78_D Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |
| **23** | **22** | **21** | **20** | **19** | **18** | **17** | **16** |
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |
| **15** | **14** | **13** | **12** | **11** | **10** | **9** | **8** |
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |
| **7** | **6** | **5** | **4** | **3** | **2** | **1** | **0** |
| DESC_PTR | | | DESC_SIZE | | | | |
| R/W-0 | | | R/W-0 | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-640. QUEUE_78_D Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-5 | DESC_PTR | R/W-0 | 0 | Descriptor pointer.<br>It will be read as zero if the queue is empty.<br>It will indicate a<br>32-bit aligned address that points to a descriptor when the queue is not empty. |
| 4-0 | DESC_SIZE | R/W-0 | 0 | Descriptor Size.<br>It is encoded in<br>4-byte increments with values 0 to 31 representing 24 and so on to 148 bytes.<br>This field will return a 0x0 when an empty queue is read.<br>Queue Manager Queue N Registers D To save hardware resources, the queue manager internally stores descriptor size (desc_size) information in four bits.<br>However, register D has five LSBs that specify descriptor size.<br>As a consequence, the value of desc_size that is pushed may not be same as that is read during a pop.<br>The value that is read back is equal to always rounded to an odd number.<br>So, for even values, the value read back is one more than what was written.<br>For odd values, the value read back is same as the value that was written.<br>Note that this<br>5-bit field (desc_size) is unrelated to the code for size of descriptors in a descriptor region.<br>It is just a place holder for a<br>5-bit value that is maintained across the push and pop operations for every descriptor managed by the queue manager. |

### 16.4.7.351 QUEUE_79_A Register (offset = 24F0h) [reset = 0h]

QUEUE_79_A is shown in Figure 16-627 and described in Table 16-641.

**Figure 16-627. QUEUE_79_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-641. QUEUE_79_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>This count is incremented by 1 whenever a packet is added to the queue.<br>This count is decremented by 1 whenever a packet is popped from the queue. |

### 16.4.7.352 QUEUE_79_B Register (offset = 24F4h) [reset = 0h]

QUEUE_79_B is shown in Figure 16-628 and described in Table 16-642.

**Figure 16-628. QUEUE_79_B Register**

| 31–28 | 27–0 |
| --- | --- |
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-642. QUEUE_79_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
| --- | --- | --- | --- | --- |
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. |

### 16.4.7.353 QUEUE_79_C Register (offset = 24F8h) [reset = 0h]

QUEUE_79_C is shown in Figure 16-629 and described in Table 16-643.

**Figure 16-629. QUEUE_79_C Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| HEAD_TAIL | Reserved | | | | | | |
| W-0 | | | | | | | |
| **23** | **22** | **21** | **20** | **19** | **18** | **17** | **16** |
| Reserved | | | | | | | |
| **15** | **14** | **13** | **12** | **11** | **10** | **9** | **8** |
| Reserved | | PACKET_SIZE | | | | | |
| | | R/W-0 | | | | | |
| **7** | **6** | **5** | **4** | **3** | **2** | **1** | **0** |
| PACKET_SIZE | | | | | | | |
| R/W-0 | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-643. QUEUE_79_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31 | HEAD_TAIL | W-0 | 0 | Head/Tail Push Control.<br>Set to zero in order to push packet onto tail of queue and set to one in order to push packet onto head of queue. |
| 13-0 | PACKET_SIZE | R/W-0 | 0 | packet_size This field indicates packet size and is assumed to be zero on each packet add unless the value is explicitly overwritten. This field indicates packet size for packet pop operation. |

### 16.4.7.354 QUEUE_79_D Register (offset = 24FCh) [reset = 0h]

QUEUE_79_D is shown in Figure 16-630 and described in Table 16-644.

**Figure 16-630. QUEUE_79_D Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |
| **23** | **22** | **21** | **20** | **19** | **18** | **17** | **16** |
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |
| **15** | **14** | **13** | **12** | **11** | **10** | **9** | **8** |
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |
| **7** | **6** | **5** | **4** | **3** | **2** | **1** | **0** |
| DESC_PTR | | | DESC_SIZE | | | | |
| R/W-0 | | | R/W-0 | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-644. QUEUE_79_D Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-5 | DESC_PTR | R/W-0 | 0 | Descriptor pointer.<br>It will be read as zero if the queue is empty.<br>It will indicate a<br>32-bit aligned address that points to a descriptor when the queue is not empty. |
| 4-0 | DESC_SIZE | R/W-0 | 0 | Descriptor Size.<br>It is encoded in<br>4-byte increments with values 0 to 31 representing 24 and so on to 148 bytes.<br>This field will return a 0x0 when an empty queue is read.<br>Queue Manager Queue N Registers D To save hardware resources, the queue manager internally stores descriptor size (desc_size) information in four bits.<br>However, register D has five LSBs that specify descriptor size.<br>As a consequence, the value of desc_size that is pushed may not be same as that is read during a pop.<br>The value that is read back is equal to always rounded to an odd number.<br>So, for even values, the value read back is one more than what was written.<br>For odd values, the value read back is same as the value that was written.<br>Note that this<br>5-bit field (desc_size) is unrelated to the code for size of descriptors in a descriptor region.<br>It is just a place holder for a<br>5-bit value that is maintained across the push and pop operations for every descriptor managed by the queue manager. |

### 16.4.7.355 QUEUE_80_A Register (offset = 2500h) [reset = 0h]

QUEUE_80_A is shown in Figure 16-631 and described in Table 16-645.

**Figure 16-631. QUEUE_80_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-645. QUEUE_80_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>This count is incremented by 1 whenever a packet is added to the queue.<br>This count is decremented by 1 whenever a packet is popped from the queue. |

### 16.4.7.356 QUEUE_80_B Register (offset = 2504h) [reset = 0h]

QUEUE_80_B is shown in Figure 16-632 and described in Table 16-646.

**Figure 16-632. QUEUE_80_B Register**

| 31–28 | 27–0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-646. QUEUE_80_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. |

### 16.4.7.357 QUEUE_80_C Register (offset = 2508h) [reset = 0h]

QUEUE_80_C is shown in Figure 16-633 and described in Table 16-647.

**Figure 16-633. QUEUE_80_C Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| HEAD_TAIL | Reserved | | | | | | |
| W-0 | | | | | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| Reserved | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| Reserved | | PACKET_SIZE | | | | | |
| | | R/W-0 | | | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| PACKET_SIZE | | | | | | | |
| R/W-0 | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-647. QUEUE_80_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31 | HEAD_TAIL | W-0 | 0 | Head/Tail Push Control.<br>Set to zero in order to push packet onto tail of queue and set to one in order to push packet onto head of queue. |
| 13-0 | PACKET_SIZE | R/W-0 | 0 | packet_size This field indicates packet size and is assumed to be zero on each packet add unless the value is explicitly overwritten. This field indicates packet size for packet pop operation. |

### 16.4.7.358 QUEUE_80_D Register (offset = 250Ch) [reset = 0h]

QUEUE_80_D is shown in Figure 16-634 and described in Table 16-648.

**Figure 16-634. QUEUE_80_D Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | DESC_SIZE | | | | |
| R/W-0 | | | R/W-0 | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-648. QUEUE_80_D Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-5 | DESC_PTR | R/W-0 | 0 | Descriptor pointer.<br>It will be read as zero if the queue is empty.<br>It will indicate a<br>32-bit aligned address that points to a descriptor when the queue is not empty. |
| 4-0 | DESC_SIZE | R/W-0 | 0 | Descriptor Size.<br>It is encoded in<br>4-byte increments with values 0 to 31 representing 24 and so on to 148 bytes.<br>This field will return a 0x0 when an empty queue is read.<br>Queue Manager Queue N Registers D To save hardware resources, the queue manager internally stores descriptor size (desc_size) information in four bits.<br>However, register D has five LSBs that specify descriptor size.<br>As a consequence, the value of desc_size that is pushed may not be same as that is read during a pop.<br>The value that is read back is equal to always rounded to an odd number.<br>So, for even values, the value read back is one more than what was written.<br>For odd values, the value read back is same as the value that was written.<br>Note that this<br>5-bit field (desc_size) is unrelated to the code for size of descriptors in a descriptor region.<br>It is just a place holder for a<br>5-bit value that is maintained across the push and pop operations for every descriptor managed by the queue manager. |

### 16.4.7.359 QUEUE_81_A Register (offset = 2510h) [reset = 0h]

QUEUE_81_A is shown in Figure 16-635 and described in Table 16-649.

**Figure 16-635. QUEUE_81_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-649. QUEUE_81_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>This count is incremented by 1 whenever a packet is added to the queue.<br>This count is decremented by 1 whenever a packet is popped from the queue. |

### 16.4.7.360 QUEUE_81_B Register (offset = 2514h) [reset = 0h]

QUEUE_81_B is shown in Figure 16-636 and described in Table 16-650.

**Figure 16-636. QUEUE_81_B Register**

| 31–28 | 27–0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-650. QUEUE_81_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. |

### 16.4.7.361 QUEUE_81_C Register (offset = 2518h) [reset = 0h]

QUEUE_81_C is shown in Figure 16-637 and described in Table 16-651.

**Figure 16-637. QUEUE_81_C Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| HEAD_TAIL | Reserved | | | | | | |
| W-0 | | | | | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| Reserved | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| Reserved | | PACKET_SIZE | | | | | |
| | | R/W-0 | | | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| PACKET_SIZE | | | | | | | |
| R/W-0 | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-651. QUEUE_81_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31 | HEAD_TAIL | W-0 | 0 | Head/Tail Push Control.<br>Set to zero in order to push packet onto tail of queue and set to one in order to push packet onto head of queue. |
| 13-0 | PACKET_SIZE | R/W-0 | 0 | packet_size This field indicates packet size and is assumed to be zero on each packet add unless the value is explicitly overwritten. This field indicates packet size for packet pop operation. |

### 16.4.7.362 QUEUE_81_D Register (offset = 251Ch) [reset = 0h]

QUEUE_81_D is shown in Figure 16-638 and described in Table 16-652.

**Figure 16-638. QUEUE_81_D Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |
| **23** | **22** | **21** | **20** | **19** | **18** | **17** | **16** |
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |
| **15** | **14** | **13** | **12** | **11** | **10** | **9** | **8** |
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |
| **7** | **6** | **5** | **4** | **3** | **2** | **1** | **0** |
| DESC_PTR | | | DESC_SIZE | | | | |
| R/W-0 | | | R/W-0 | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-652. QUEUE_81_D Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-5 | DESC_PTR | R/W-0 | 0 | Descriptor pointer.<br>It will be read as zero if the queue is empty.<br>It will indicate a<br>32-bit aligned address that points to a descriptor when the queue is not empty. |
| 4-0 | DESC_SIZE | R/W-0 | 0 | Descriptor Size.<br>It is encoded in<br>4-byte increments with values 0 to 31 representing 24 and so on to 148 bytes.<br>This field will return a 0x0 when an empty queue is read.<br>Queue Manager Queue N Registers D To save hardware resources, the queue manager internally stores descriptor size (desc_size) information in four bits.<br>However, register D has five LSBs that specify descriptor size.<br>As a consequence, the value of desc_size that is pushed may not be same as that is read during a pop.<br>The value that is read back is equal to always rounded to an odd number.<br>So, for even values, the value read back is one more than what was written.<br>For odd values, the value read back is same as the value that was written.<br>Note that this<br>5-bit field (desc_size) is unrelated to the code for size of descriptors in a descriptor region.<br>It is just a place holder for a<br>5-bit value that is maintained across the push and pop operations for every descriptor managed by the queue manager. |

### 16.4.7.363 QUEUE_82_A Register (offset = 2520h) [reset = 0h]

QUEUE_82_A is shown in Figure 16-639 and described in Table 16-653.

**Figure 16-639. QUEUE_82_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-653. QUEUE_82_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>This count is incremented by 1 whenever a packet is added to the queue.<br>This count is decremented by 1 whenever a packet is popped from the queue. |

### 16.4.7.364 QUEUE_82_B Register (offset = 2524h) [reset = 0h]

QUEUE_82_B is shown in Figure 16-640 and described in Table 16-654.

**Figure 16-640. QUEUE_82_B Register**

| 31-28 | 27-0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-654. QUEUE_82_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. |

### 16.4.7.365 QUEUE_82_C Register (offset = 2528h) [reset = 0h]

QUEUE_82_C is shown in Figure 16-641 and described in Table 16-655.

**Figure 16-641. QUEUE_82_C Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| HEAD_TAIL | Reserved | | | | | | |
| W-0 | | | | | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| Reserved | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| Reserved | | PACKET_SIZE | | | | | |
| | | R/W-0 | | | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| PACKET_SIZE | | | | | | | |
| R/W-0 | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-655. QUEUE_82_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31 | HEAD_TAIL | W-0 | 0 | Head/Tail Push Control.<br>Set to zero in order to push packet onto tail of queue and set to one in order to push packet onto head of queue. |
| 13-0 | PACKET_SIZE | R/W-0 | 0 | packet_size This field indicates packet size and is assumed to be zero on each packet add unless the value is explicitly overwritten. This field indicates packet size for packet pop operation. |

### 16.4.7.366 QUEUE_82_D Register (offset = 252Ch) [reset = 0h]

QUEUE_82_D is shown in Figure 16-642 and described in Table 16-656.

**Figure 16-642. QUEUE_82_D Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |
| **23** | **22** | **21** | **20** | **19** | **18** | **17** | **16** |
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |
| **15** | **14** | **13** | **12** | **11** | **10** | **9** | **8** |
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |
| **7** | **6** | **5** | **4** | **3** | **2** | **1** | **0** |
| DESC_PTR | | | DESC_SIZE | | | | |
| R/W-0 | | | R/W-0 | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-656. QUEUE_82_D Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-5 | DESC_PTR | R/W-0 | 0 | Descriptor pointer.<br>It will be read as zero if the queue is empty.<br>It will indicate a<br>32-bit aligned address that points to a descriptor when the queue is not empty. |
| 4-0 | DESC_SIZE | R/W-0 | 0 | Descriptor Size.<br>It is encoded in<br>4-byte increments with values 0 to 31 representing 24 and so on to 148 bytes.<br>This field will return a 0x0 when an empty queue is read.<br>Queue Manager Queue N Registers D To save hardware resources, the queue manager internally stores descriptor size (desc_size) information in four bits.<br>However, register D has five LSBs that specify descriptor size.<br>As a consequence, the value of desc_size that is pushed may not be same as that is read during a pop.<br>The value that is read back is equal to always rounded to an odd number.<br>So, for even values, the value read back is one more than what was written.<br>For odd values, the value read back is same as the value that was written.<br>Note that this<br>5-bit field (desc_size) is unrelated to the code for size of descriptors in a descriptor region.<br>It is just a place holder for a<br>5-bit value that is maintained across the push and pop operations for every descriptor managed by the queue manager. |

### 16.4.7.367 QUEUE_83_A Register (offset = 2530h) [reset = 0h]

QUEUE_83_A is shown in Figure 16-643 and described in Table 16-657.

**Figure 16-643. QUEUE_83_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-657. QUEUE_83_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>This count is incremented by 1 whenever a packet is added to the queue.<br>This count is decremented by 1 whenever a packet is popped from the queue. |

### 16.4.7.368 QUEUE_83_B Register (offset = 2534h) [reset = 0h]

QUEUE_83_B is shown in Figure 16-644 and described in Table 16-658.

**Figure 16-644. QUEUE_83_B Register**

| 31-28 | 27-0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-658. QUEUE_83_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. |

### 16.4.7.369 QUEUE_83_C Register (offset = 2538h) [reset = 0h]

QUEUE_83_C is shown in Figure 16-645 and described in Table 16-659.

**Figure 16-645. QUEUE_83_C Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| HEAD_TAIL | Reserved | | | | | | |
| W-0 | | | | | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| Reserved | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| Reserved | | PACKET_SIZE | | | | | |
| | | R/W-0 | | | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| PACKET_SIZE | | | | | | | |
| R/W-0 | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-659. QUEUE_83_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31 | HEAD_TAIL | W-0 | 0 | Head/Tail Push Control.<br>Set to zero in order to push packet onto tail of queue and set to one in order to push packet onto head of queue. |
| 13-0 | PACKET_SIZE | R/W-0 | 0 | packet_size This field indicates packet size and is assumed to be zero on each packet add unless the value is explicitly overwritten. This field indicates packet size for packet pop operation. |

### 16.4.7.370 QUEUE_83_D Register (offset = 253Ch) [reset = 0h]

QUEUE_83_D is shown in Figure 16-646 and described in Table 16-660.

**Figure 16-646. QUEUE_83_D Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | DESC_SIZE | | | | |
| R/W-0 | | | R/W-0 | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-660. QUEUE_83_D Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-5 | DESC_PTR | R/W-0 | 0 | Descriptor pointer.<br>It will be read as zero if the queue is empty.<br>It will indicate a<br>32-bit aligned address that points to a descriptor when the queue is not empty. |
| 4-0 | DESC_SIZE | R/W-0 | 0 | Descriptor Size.<br>It is encoded in<br>4-byte increments with values 0 to 31 representing 24 and so on to 148 bytes.<br>This field will return a 0x0 when an empty queue is read.<br>Queue Manager Queue N Registers D To save hardware resources, the queue manager internally stores descriptor size (desc_size) information in four bits.<br>However, register D has five LSBs that specify descriptor size.<br>As a consequence, the value of desc_size that is pushed may not be same as that is read during a pop.<br>The value that is read back is equal to always rounded to an odd number.<br>So, for even values, the value read back is one more than what was written.<br>For odd values, the value read back is same as the value that was written.<br>Note that this<br>5-bit field (desc_size) is unrelated to the code for size of descriptors in a descriptor region.<br>It is just a place holder for a<br>5-bit value that is maintained across the push and pop operations for every descriptor managed by the queue manager. |

### 16.4.7.371 QUEUE_84_A Register (offset = 2540h) [reset = 0h]

QUEUE_84_A is shown in Figure 16-647 and described in Table 16-661.

**Figure 16-647. QUEUE_84_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-661. QUEUE_84_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>This count is incremented by 1 whenever a packet is added to the queue.<br>This count is decremented by 1 whenever a packet is popped from the queue. |

### 16.4.7.372 QUEUE_84_B Register (offset = 2544h) [reset = 0h]

QUEUE_84_B is shown in Figure 16-648 and described in Table 16-662.

**Figure 16-648. QUEUE_84_B Register**

| 31-28 | 27-0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-662. QUEUE_84_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. |

### 16.4.7.373 QUEUE_84_C Register (offset = 2548h) [reset = 0h]

QUEUE_84_C is shown in Figure 16-649 and described in Table 16-663.

**Figure 16-649. QUEUE_84_C Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| HEAD_TAIL | Reserved | | | | | | |
| W-0 | | | | | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| Reserved | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| Reserved | | PACKET_SIZE | | | | | |
| | | R/W-0 | | | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| PACKET_SIZE | | | | | | | |
| R/W-0 | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-663. QUEUE_84_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31 | HEAD_TAIL | W-0 | 0 | Head/Tail Push Control.<br>Set to zero in order to push packet onto tail of queue and set to one in order to push packet onto head of queue. |
| 13-0 | PACKET_SIZE | R/W-0 | 0 | packet_size This field indicates packet size and is assumed to be zero on each packet add unless the value is explicitly overwritten. This field indicates packet size for packet pop operation. |

### 16.4.7.374 QUEUE_84_D Register (offset = 254Ch) [reset = 0h]

QUEUE_84_D is shown in Figure 16-650 and described in Table 16-664.

**Figure 16-650. QUEUE_84_D Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | DESC_SIZE | | | | |
| R/W-0 | | | R/W-0 | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-664. QUEUE_84_D Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-5 | DESC_PTR | R/W-0 | 0 | Descriptor pointer.<br>It will be read as zero if the queue is empty.<br>It will indicate a<br>32-bit aligned address that points to a descriptor when the queue is not empty. |
| 4-0 | DESC_SIZE | R/W-0 | 0 | Descriptor Size.<br>It is encoded in<br>4-byte increments with values 0 to 31 representing 24 and so on to 148 bytes.<br>This field will return a 0x0 when an empty queue is read.<br>Queue Manager Queue N Registers D To save hardware resources, the queue manager internally stores descriptor size (desc_size) information in four bits.<br>However, register D has five LSBs that specify descriptor size.<br>As a consequence, the value of desc_size that is pushed may not be same as that is read during a pop.<br>The value that is read back is equal to always rounded to an odd number.<br>So, for even values, the value read back is one more than what was written.<br>For odd values, the value read back is same as the value that was written.<br>Note that this<br>5-bit field (desc_size) is unrelated to the code for size of descriptors in a descriptor region.<br>It is just a place holder for a<br>5-bit value that is maintained across the push and pop operations for every descriptor managed by the queue manager. |

### 16.4.7.375 QUEUE_85_A Register (offset = 2550h) [reset = 0h]

QUEUE_85_A is shown in Figure 16-651 and described in Table 16-665.

**Figure 16-651. QUEUE_85_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-665. QUEUE_85_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>This count is incremented by 1 whenever a packet is added to the queue.<br>This count is decremented by 1 whenever a packet is popped from the queue. |

### 16.4.7.376 QUEUE_85_B Register (offset = 2554h) [reset = 0h]

QUEUE_85_B is shown in Figure 16-652 and described in Table 16-666.

**Figure 16-652. QUEUE_85_B Register**

| 31–28 | 27–0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-666. QUEUE_85_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. |

### 16.4.7.377 QUEUE_85_C Register (offset = 2558h) [reset = 0h]

QUEUE_85_C is shown in Figure 16-653 and described in Table 16-667.

**Figure 16-653. QUEUE_85_C Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| HEAD_TAIL | Reserved | | | | | | |
| W-0 | | | | | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| Reserved | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| Reserved | | PACKET_SIZE | | | | | |
| | | R/W-0 | | | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| PACKET_SIZE | | | | | | | |
| R/W-0 | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-667. QUEUE_85_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31 | HEAD_TAIL | W-0 | 0 | Head/Tail Push Control.<br>Set to zero in order to push packet onto tail of queue and set to one in order to push packet onto head of queue. |
| 13-0 | PACKET_SIZE | R/W-0 | 0 | packet_size This field indicates packet size and is assumed to be zero on each packet add unless the value is explicitly overwritten. This field indicates packet size for packet pop operation. |

### 16.4.7.378 QUEUE_85_D Register (offset = 255Ch) [reset = 0h]

QUEUE_85_D is shown in Figure 16-654 and described in Table 16-668.

**Figure 16-654. QUEUE_85_D Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |
| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |
| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |
| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
| DESC_PTR | | | DESC_SIZE | | | | |
| R/W-0 | | | R/W-0 | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-668. QUEUE_85_D Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-5 | DESC_PTR | R/W-0 | 0 | Descriptor pointer.<br>It will be read as zero if the queue is empty.<br>It will indicate a<br>32-bit aligned address that points to a descriptor when the queue is not empty. |
| 4-0 | DESC_SIZE | R/W-0 | 0 | Descriptor Size.<br>It is encoded in<br>4-byte increments with values 0 to 31 representing 24 and so on to 148 bytes.<br>This field will return a 0x0 when an empty queue is read.<br>Queue Manager Queue N Registers D To save hardware resources, the queue manager internally stores descriptor size (desc_size) information in four bits.<br>However, register D has five LSBs that specify descriptor size.<br>As a consequence, the value of desc_size that is pushed may not be same as that is read during a pop.<br>The value that is read back is equal to always rounded to an odd number.<br>So, for even values, the value read back is one more than what was written.<br>For odd values, the value read back is same as the value that was written.<br>Note that this<br>5-bit field (desc_size) is unrelated to the code for size of descriptors in a descriptor region.<br>It is just a place holder for a<br>5-bit value that is maintained across the push and pop operations for every descriptor managed by the queue manager. |

### 16.4.7.379 QUEUE_86_A Register (offset = 2560h) [reset = 0h]

QUEUE_86_A is shown in Figure 16-655 and described in Table 16-669.

**Figure 16-655. QUEUE_86_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-669. QUEUE_86_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>This count is incremented by 1 whenever a packet is added to the queue.<br>This count is decremented by 1 whenever a packet is popped from the queue. |

### 16.4.7.380 QUEUE_86_B Register (offset = 2564h) [reset = 0h]

QUEUE_86_B is shown in Figure 16-656 and described in Table 16-670.

**Figure 16-656. QUEUE_86_B Register**

| 31–28 | 27–0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-670. QUEUE_86_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. |

### 16.4.7.381 QUEUE_86_C Register (offset = 2568h) [reset = 0h]

QUEUE_86_C is shown in Figure 16-657 and described in Table 16-671.

**Figure 16-657. QUEUE_86_C Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| HEAD_TAIL | Reserved | | | | | | |
| W-0 | | | | | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| Reserved | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| Reserved | | PACKET_SIZE | | | | | |
| | | R/W-0 | | | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| PACKET_SIZE | | | | | | | |
| R/W-0 | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-671. QUEUE_86_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31 | HEAD_TAIL | W-0 | 0 | Head/Tail Push Control.<br>Set to zero in order to push packet onto tail of queue and set to one in order to push packet onto head of queue. |
| 13-0 | PACKET_SIZE | R/W-0 | 0 | packet_size This field indicates packet size and is assumed to be zero on each packet add unless the value is explicitly overwritten. This field indicates packet size for packet pop operation. |

### 16.4.7.382 QUEUE_86_D Register (offset = 256Ch) [reset = 0h]

QUEUE_86_D is shown in Figure 16-658 and described in Table 16-672.

**Figure 16-658. QUEUE_86_D Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | DESC_SIZE | | | | |
| R/W-0 | | | R/W-0 | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-672. QUEUE_86_D Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-5 | DESC_PTR | R/W-0 | 0 | Descriptor pointer.<br>It will be read as zero if the queue is empty.<br>It will indicate a<br>32-bit aligned address that points to a descriptor when the queue is not empty. |
| 4-0 | DESC_SIZE | R/W-0 | 0 | Descriptor Size.<br>It is encoded in<br>4-byte increments with values 0 to 31 representing 24 and so on to 148 bytes.<br>This field will return a 0x0 when an empty queue is read.<br>Queue Manager Queue N Registers D To save hardware resources, the queue manager internally stores descriptor size (desc_size) information in four bits.<br>However, register D has five LSBs that specify descriptor size.<br>As a consequence, the value of desc_size that is pushed may not be same as that is read during a pop.<br>The value that is read back is equal to always rounded to an odd number.<br>So, for even values, the value read back is one more than what was written.<br>For odd values, the value read back is same as the value that was written.<br>Note that this<br>5-bit field (desc_size) is unrelated to the code for size of descriptors in a descriptor region.<br>It is just a place holder for a<br>5-bit value that is maintained across the push and pop operations for every descriptor managed by the queue manager. |

### 16.4.7.383 QUEUE_87_A Register (offset = 2570h) [reset = 0h]

QUEUE_87_A is shown in Figure 16-659 and described in Table 16-673.

**Figure 16-659. QUEUE_87_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-673. QUEUE_87_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>This count is incremented by 1 whenever a packet is added to the queue.<br>This count is decremented by 1 whenever a packet is popped from the queue. |

### 16.4.7.384 QUEUE_87_B Register (offset = 2574h) [reset = 0h]

QUEUE_87_B is shown in Figure 16-660 and described in Table 16-674.

**Figure 16-660. QUEUE_87_B Register**

| 31-28 | 27-0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-674. QUEUE_87_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. |

### 16.4.7.385 QUEUE_87_C Register (offset = 2578h) [reset = 0h]

QUEUE_87_C is shown in Figure 16-661 and described in Table 16-675.

**Figure 16-661. QUEUE_87_C Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| HEAD_TAIL | Reserved | | | | | | |
| W-0 | | | | | | | |
| **23** | **22** | **21** | **20** | **19** | **18** | **17** | **16** |
| Reserved | | | | | | | |
| **15** | **14** | **13** | **12** | **11** | **10** | **9** | **8** |
| Reserved | | PACKET_SIZE | | | | | |
| | | R/W-0 | | | | | |
| **7** | **6** | **5** | **4** | **3** | **2** | **1** | **0** |
| PACKET_SIZE | | | | | | | |
| R/W-0 | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-675. QUEUE_87_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31 | HEAD_TAIL | W-0 | 0 | Head/Tail Push Control.<br>Set to zero in order to push packet onto tail of queue and set to one in order to push packet onto head of queue. |
| 13-0 | PACKET_SIZE | R/W-0 | 0 | packet_size This field indicates packet size and is assumed to be zero on each packet add unless the value is explicitly overwritten. This field indicates packet size for packet pop operation. |

### 16.4.7.386 QUEUE_87_D Register (offset = 257Ch) [reset = 0h]

QUEUE_87_D is shown in Figure 16-662 and described in Table 16-676.

**Figure 16-662. QUEUE_87_D Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | DESC_SIZE | | | | |
| R/W-0 | | | R/W-0 | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-676. QUEUE_87_D Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-5 | DESC_PTR | R/W-0 | 0 | Descriptor pointer.<br>It will be read as zero if the queue is empty.<br>It will indicate a<br>32-bit aligned address that points to a descriptor when the queue is not empty. |
| 4-0 | DESC_SIZE | R/W-0 | 0 | Descriptor Size.<br>It is encoded in<br>4-byte increments with values 0 to 31 representing 24 and so on to 148 bytes.<br>This field will return a 0x0 when an empty queue is read.<br>Queue Manager Queue N Registers D To save hardware resources, the queue manager internally stores descriptor size (desc_size) information in four bits.<br>However, register D has five LSBs that specify descriptor size.<br>As a consequence, the value of desc_size that is pushed may not be same as that is read during a pop.<br>The value that is read back is equal to always rounded to an odd number.<br>So, for even values, the value read back is one more than what was written.<br>For odd values, the value read back is same as the value that was written.<br>Note that this<br>5-bit field (desc_size) is unrelated to the code for size of descriptors in a descriptor region.<br>It is just a place holder for a<br>5-bit value that is maintained across the push and pop operations for every descriptor managed by the queue manager. |

### 16.4.7.387 QUEUE_88_A Register (offset = 2580h) [reset = 0h]

QUEUE_88_A is shown in Figure 16-663 and described in Table 16-677.

**Figure 16-663. QUEUE_88_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-677. QUEUE_88_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>This count is incremented by 1 whenever a packet is added to the queue.<br>This count is decremented by 1 whenever a packet is popped from the queue. |

### 16.4.7.388 QUEUE_88_B Register (offset = 2584h) [reset = 0h]

QUEUE_88_B is shown in Figure 16-664 and described in Table 16-678.

**Figure 16-664. QUEUE_88_B Register**

| 31-28 | 27-0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-678. QUEUE_88_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. |

### 16.4.7.389 QUEUE_88_C Register (offset = 2588h) [reset = 0h]

QUEUE_88_C is shown in Figure 16-665 and described in Table 16-679.

**Figure 16-665. QUEUE_88_C Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| HEAD_TAIL | Reserved | | | | | | |
| W-0 | | | | | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| Reserved | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| Reserved | | PACKET_SIZE | | | | | |
| | | R/W-0 | | | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| PACKET_SIZE | | | | | | | |
| R/W-0 | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-679. QUEUE_88_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31 | HEAD_TAIL | W-0 | 0 | Head/Tail Push Control.<br>Set to zero in order to push packet onto tail of queue and set to one in order to push packet onto head of queue. |
| 13-0 | PACKET_SIZE | R/W-0 | 0 | packet_size This field indicates packet size and is assumed to be zero on each packet add unless the value is explicitly overwritten. This field indicates packet size for packet pop operation. |

### 16.4.7.390 QUEUE_88_D Register (offset = 258Ch) [reset = 0h]

QUEUE_88_D is shown in Figure 16-666 and described in Table 16-680.

**Figure 16-666. QUEUE_88_D Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | DESC_SIZE | | | | |
| R/W-0 | | | R/W-0 | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-680. QUEUE_88_D Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-5 | DESC_PTR | R/W-0 | 0 | Descriptor pointer.<br>It will be read as zero if the queue is empty.<br>It will indicate a<br>32-bit aligned address that points to a descriptor when the queue is not empty. |
| 4-0 | DESC_SIZE | R/W-0 | 0 | Descriptor Size.<br>It is encoded in<br>4-byte increments with values 0 to 31 representing 24 and so on to 148 bytes.<br>This field will return a 0x0 when an empty queue is read.<br>Queue Manager Queue N Registers D To save hardware resources, the queue manager internally stores descriptor size (desc_size) information in four bits.<br>However, register D has five LSBs that specify descriptor size.<br>As a consequence, the value of desc_size that is pushed may not be same as that is read during a pop.<br>The value that is read back is equal to always rounded to an odd number.<br>So, for even values, the value read back is one more than what was written.<br>For odd values, the value read back is same as the value that was written.<br>Note that this<br>5-bit field (desc_size) is unrelated to the code for size of descriptors in a descriptor region.<br>It is just a place holder for a<br>5-bit value that is maintained across the push and pop operations for every descriptor managed by the queue manager. |

### 16.4.7.391 QUEUE_89_A Register (offset = 2590h) [reset = 0h]

QUEUE_89_A is shown in Figure 16-667 and described in Table 16-681.

**Figure 16-667. QUEUE_89_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-681. QUEUE_89_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>This count is incremented by 1 whenever a packet is added to the queue.<br>This count is decremented by 1 whenever a packet is popped from the queue. |

### 16.4.7.392 QUEUE_89_B Register (offset = 2594h) [reset = 0h]

QUEUE_89_B is shown in Figure 16-668 and described in Table 16-682.

**Figure 16-668. QUEUE_89_B Register**

| 31–28 | 27–0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-682. QUEUE_89_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. |

### 16.4.7.393 QUEUE_89_C Register (offset = 2598h) [reset = 0h]

QUEUE_89_C is shown in Figure 16-669 and described in Table 16-683.

**Figure 16-669. QUEUE_89_C Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| HEAD_TAIL | Reserved | | | | | | |
| W-0 | | | | | | | |
| **23** | **22** | **21** | **20** | **19** | **18** | **17** | **16** |
| Reserved | | | | | | | |
| **15** | **14** | **13** | **12** | **11** | **10** | **9** | **8** |
| Reserved | | PACKET_SIZE | | | | | |
| | | R/W-0 | | | | | |
| **7** | **6** | **5** | **4** | **3** | **2** | **1** | **0** |
| PACKET_SIZE | | | | | | | |
| R/W-0 | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-683. QUEUE_89_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31 | HEAD_TAIL | W-0 | 0 | Head/Tail Push Control.<br>Set to zero in order to push packet onto tail of queue and set to one in order to push packet onto head of queue. |
| 13-0 | PACKET_SIZE | R/W-0 | 0 | packet_size This field indicates packet size and is assumed to be zero on each packet add unless the value is explicitly overwritten. This field indicates packet size for packet pop operation. |

### 16.4.7.394 QUEUE_89_D Register (offset = 259Ch) [reset = 0h]

QUEUE_89_D is shown in Figure 16-670 and described in Table 16-684.

**Figure 16-670. QUEUE_89_D Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |
| **23** | **22** | **21** | **20** | **19** | **18** | **17** | **16** |
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |
| **15** | **14** | **13** | **12** | **11** | **10** | **9** | **8** |
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |
| **7** | **6** | **5** | **4** | **3** | **2** | **1** | **0** |
| DESC_PTR | | | DESC_SIZE | | | | |
| R/W-0 | | | R/W-0 | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-684. QUEUE_89_D Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-5 | DESC_PTR | R/W-0 | 0 | Descriptor pointer.<br>It will be read as zero if the queue is empty.<br>It will indicate a<br>32-bit aligned address that points to a descriptor when the queue is not empty. |
| 4-0 | DESC_SIZE | R/W-0 | 0 | Descriptor Size.<br>It is encoded in<br>4-byte increments with values 0 to 31 representing 24 and so on to 148 bytes.<br>This field will return a 0x0 when an empty queue is read.<br>Queue Manager Queue N Registers D To save hardware resources, the queue manager internally stores descriptor size (desc_size) information in four bits.<br>However, register D has five LSBs that specify descriptor size.<br>As a consequence, the value of desc_size that is pushed may not be same as that is read during a pop.<br>The value that is read back is equal to always rounded to an odd number.<br>So, for even values, the value read back is one more than what was written.<br>For odd values, the value read back is same as the value that was written.<br>Note that this<br>5-bit field (desc_size) is unrelated to the code for size of descriptors in a descriptor region.<br>It is just a place holder for a<br>5-bit value that is maintained across the push and pop operations for every descriptor managed by the queue manager. |

### 16.4.7.395 QUEUE_90_A Register (offset = 25A0h) [reset = 0h]

QUEUE_90_A is shown in Figure 16-671 and described in Table 16-685.

**Figure 16-671. QUEUE_90_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-685. QUEUE_90_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>This count is incremented by 1 whenever a packet is added to the queue.<br>This count is decremented by 1 whenever a packet is popped from the queue. |

### 16.4.7.396 QUEUE_90_B Register (offset = 25A4h) [reset = 0h]

QUEUE_90_B is shown in Figure 16-672 and described in Table 16-686.

**Figure 16-672. QUEUE_90_B Register**

| 31–28 | 27–0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-686. QUEUE_90_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. |

### 16.4.7.397 QUEUE_90_C Register (offset = 25A8h) [reset = 0h]

QUEUE_90_C is shown in Figure 16-673 and described in Table 16-687.

**Figure 16-673. QUEUE_90_C Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| HEAD_TAIL | Reserved | | | | | | |
| W-0 | | | | | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| Reserved | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| Reserved | | PACKET_SIZE | | | | | |
| | | R/W-0 | | | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| PACKET_SIZE | | | | | | | |
| R/W-0 | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-687. QUEUE_90_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31 | HEAD_TAIL | W-0 | 0 | Head/Tail Push Control.<br>Set to zero in order to push packet onto tail of queue and set to one in order to push packet onto head of queue. |
| 13-0 | PACKET_SIZE | R/W-0 | 0 | packet_size This field indicates packet size and is assumed to be zero on each packet add unless the value is explicitly overwritten. This field indicates packet size for packet pop operation. |

### 16.4.7.398 QUEUE_90_D Register (offset = 25ACh) [reset = 0h]

QUEUE_90_D is shown in Figure 16-674 and described in Table 16-688.

**Figure 16-674. QUEUE_90_D Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | DESC_SIZE | | | | |
| R/W-0 | | | R/W-0 | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-688. QUEUE_90_D Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-5 | DESC_PTR | R/W-0 | 0 | Descriptor pointer.<br>It will be read as zero if the queue is empty.<br>It will indicate a<br>32-bit aligned address that points to a descriptor when the queue is not empty. |
| 4-0 | DESC_SIZE | R/W-0 | 0 | Descriptor Size.<br>It is encoded in<br>4-byte increments with values 0 to 31 representing 24 and so on to 148 bytes.<br>This field will return a 0x0 when an empty queue is read.<br>Queue Manager Queue N Registers D To save hardware resources, the queue manager internally stores descriptor size (desc_size) information in four bits.<br>However, register D has five LSBs that specify descriptor size.<br>As a consequence, the value of desc_size that is pushed may not be same as that is read during a pop.<br>The value that is read back is equal to always rounded to an odd number.<br>So, for even values, the value read back is one more than what was written.<br>For odd values, the value read back is same as the value that was written.<br>Note that this<br>5-bit field (desc_size) is unrelated to the code for size of descriptors in a descriptor region.<br>It is just a place holder for a<br>5-bit value that is maintained across the push and pop operations for every descriptor managed by the queue manager. |

### 16.4.7.399 QUEUE_91_A Register (offset = 25B0h) [reset = 0h]

QUEUE_91_A is shown in Figure 16-675 and described in Table 16-689.

**Figure 16-675. QUEUE_91_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-689. QUEUE_91_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>This count is incremented by 1 whenever a packet is added to the queue.<br>This count is decremented by 1 whenever a packet is popped from the queue. |

### 16.4.7.400 QUEUE_91_B Register (offset = 25B4h) [reset = 0h]

QUEUE_91_B is shown in Figure 16-676 and described in Table 16-690.

**Figure 16-676. QUEUE_91_B Register**

| 31–28 | 27–0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-690. QUEUE_91_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. |

### 16.4.7.401 QUEUE_91_C Register (offset = 25B8h) [reset = 0h]

QUEUE_91_C is shown in Figure 16-677 and described in Table 16-691.

**Figure 16-677. QUEUE_91_C Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| HEAD_TAIL | Reserved | | | | | | |
| W-0 | | | | | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| Reserved | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| Reserved | | PACKET_SIZE | | | | | |
| | | R/W-0 | | | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| PACKET_SIZE | | | | | | | |
| R/W-0 | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-691. QUEUE_91_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31 | HEAD_TAIL | W-0 | 0 | Head/Tail Push Control.<br>Set to zero in order to push packet onto tail of queue and set to one in order to push packet onto head of queue. |
| 13-0 | PACKET_SIZE | R/W-0 | 0 | packet_size This field indicates packet size and is assumed to be zero on each packet add unless the value is explicitly overwritten. This field indicates packet size for packet pop operation. |

### 16.4.7.402 QUEUE_91_D Register (offset = 25BCh) [reset = 0h]

QUEUE_91_D is shown in Figure 16-678 and described in Table 16-692.

**Figure 16-678. QUEUE_91_D Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | DESC_SIZE | | | | |
| R/W-0 | | | R/W-0 | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-692. QUEUE_91_D Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-5 | DESC_PTR | R/W-0 | 0 | Descriptor pointer.<br>It will be read as zero if the queue is empty.<br>It will indicate a<br>32-bit aligned address that points to a descriptor when the queue is not empty. |
| 4-0 | DESC_SIZE | R/W-0 | 0 | Descriptor Size.<br>It is encoded in<br>4-byte increments with values 0 to 31 representing 24 and so on to 148 bytes.<br>This field will return a 0x0 when an empty queue is read.<br>Queue Manager Queue N Registers D To save hardware resources, the queue manager internally stores descriptor size (desc_size) information in four bits.<br>However, register D has five LSBs that specify descriptor size.<br>As a consequence, the value of desc_size that is pushed may not be same as that is read during a pop.<br>The value that is read back is equal to always rounded to an odd number.<br>So, for even values, the value read back is one more than what was written.<br>For odd values, the value read back is same as the value that was written.<br>Note that this<br>5-bit field (desc_size) is unrelated to the code for size of descriptors in a descriptor region.<br>It is just a place holder for a<br>5-bit value that is maintained across the push and pop operations for every descriptor managed by the queue manager. |

### 16.4.7.403 QUEUE_92_A Register (offset = 25C0h) [reset = 0h]

QUEUE_92_A is shown in Figure 16-679 and described in Table 16-693.

**Figure 16-679. QUEUE_92_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-693. QUEUE_92_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>This count is incremented by 1 whenever a packet is added to the queue.<br>This count is decremented by 1 whenever a packet is popped from the queue. |

### 16.4.7.404 QUEUE_92_B Register (offset = 25C4h) [reset = 0h]

QUEUE_92_B is shown in Figure 16-680 and described in Table 16-694.

**Figure 16-680. QUEUE_92_B Register**

| 31-28 | 27-0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-694. QUEUE_92_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. |

### 16.4.7.405 QUEUE_92_C Register (offset = 25C8h) [reset = 0h]

QUEUE_92_C is shown in Figure 16-681 and described in Table 16-695.

**Figure 16-681. QUEUE_92_C Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| HEAD_TAIL | Reserved | | | | | | |
| W-0 | | | | | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| Reserved | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| Reserved | | PACKET_SIZE | | | | | |
| | | R/W-0 | | | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| PACKET_SIZE | | | | | | | |
| R/W-0 | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-695. QUEUE_92_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31 | HEAD_TAIL | W-0 | 0 | Head/Tail Push Control.<br>Set to zero in order to push packet onto tail of queue and set to one in order to push packet onto head of queue. |
| 13-0 | PACKET_SIZE | R/W-0 | 0 | packet_size This field indicates packet size and is assumed to be zero on each packet add unless the value is explicitly overwritten. This field indicates packet size for packet pop operation. |

### 16.4.7.406 QUEUE_92_D Register (offset = 25CCh) [reset = 0h]

QUEUE_92_D is shown in Figure 16-682 and described in Table 16-696.

**Figure 16-682. QUEUE_92_D Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | DESC_SIZE | | | | |
| R/W-0 | | | R/W-0 | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-696. QUEUE_92_D Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-5 | DESC_PTR | R/W-0 | 0 | Descriptor pointer.<br>It will be read as zero if the queue is empty.<br>It will indicate a<br>32-bit aligned address that points to a descriptor when the queue is not empty. |
| 4-0 | DESC_SIZE | R/W-0 | 0 | Descriptor Size.<br>It is encoded in<br>4-byte increments with values 0 to 31 representing 24 and so on to 148 bytes.<br>This field will return a 0x0 when an empty queue is read.<br>Queue Manager Queue N Registers D To save hardware resources, the queue manager internally stores descriptor size (desc_size) information in four bits.<br>However, register D has five LSBs that specify descriptor size.<br>As a consequence, the value of desc_size that is pushed may not be same as that is read during a pop.<br>The value that is read back is equal to always rounded to an odd number.<br>So, for even values, the value read back is one more than what was written.<br>For odd values, the value read back is same as the value that was written.<br>Note that this<br>5-bit field (desc_size) is unrelated to the code for size of descriptors in a descriptor region.<br>It is just a place holder for a<br>5-bit value that is maintained across the push and pop operations for every descriptor managed by the queue manager. |

### 16.4.7.407 QUEUE_93_A Register (offset = 25D0h) [reset = 0h]

QUEUE_93_A is shown in Figure 16-683 and described in Table 16-697.

**Figure 16-683. QUEUE_93_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-697. QUEUE_93_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>This count is incremented by 1 whenever a packet is added to the queue.<br>This count is decremented by 1 whenever a packet is popped from the queue. |

#### 16.4.7.408 QUEUE_93_B Register (offset = 25D4h) [reset = 0h]

QUEUE_93_B is shown in Figure 16-684 and described in Table 16-698.

**Figure 16-684. QUEUE_93_B Register**

| 31–28 | 27–0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-698. QUEUE_93_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. |

### 16.4.7.409 QUEUE_93_C Register (offset = 25D8h) [reset = 0h]

QUEUE_93_C is shown in Figure 16-685 and described in Table 16-699.

**Figure 16-685. QUEUE_93_C Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| HEAD_TAIL | Reserved | | | | | | |
| W-0 | | | | | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| Reserved | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| Reserved | | PACKET_SIZE | | | | | |
| | | R/W-0 | | | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| PACKET_SIZE | | | | | | | |
| R/W-0 | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-699. QUEUE_93_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31 | HEAD_TAIL | W-0 | 0 | Head/Tail Push Control.<br>Set to zero in order to push packet onto tail of queue and set to one in order to push packet onto head of queue. |
| 13-0 | PACKET_SIZE | R/W-0 | 0 | packet_size This field indicates packet size and is assumed to be zero on each packet add unless the value is explicitly overwritten. This field indicates packet size for packet pop operation. |

### 16.4.7.410 QUEUE_93_D Register (offset = 25DCh) [reset = 0h]

QUEUE_93_D is shown in Figure 16-686 and described in Table 16-700.

**Figure 16-686. QUEUE_93_D Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | DESC_SIZE | | | | |
| R/W-0 | | | R/W-0 | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-700. QUEUE_93_D Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-5 | DESC_PTR | R/W-0 | 0 | Descriptor pointer.<br>It will be read as zero if the queue is empty.<br>It will indicate a<br>32-bit aligned address that points to a descriptor when the queue is not empty. |
| 4-0 | DESC_SIZE | R/W-0 | 0 | Descriptor Size.<br>It is encoded in<br>4-byte increments with values 0 to 31 representing 24 and so on to 148 bytes.<br>This field will return a 0x0 when an empty queue is read.<br>Queue Manager Queue N Registers D To save hardware resources, the queue manager internally stores descriptor size (desc_size) information in four bits.<br>However, register D has five LSBs that specify descriptor size.<br>As a consequence, the value of desc_size that is pushed may not be same as that is read during a pop.<br>The value that is read back is equal to always rounded to an odd number.<br>So, for even values, the value read back is one more than what was written.<br>For odd values, the value read back is same as the value that was written.<br>Note that this<br>5-bit field (desc_size) is unrelated to the code for size of descriptors in a descriptor region.<br>It is just a place holder for a<br>5-bit value that is maintained across the push and pop operations for every descriptor managed by the queue manager. |

### 16.4.7.411 QUEUE_94_A Register (offset = 25E0h) [reset = 0h]

QUEUE_94_A is shown in Figure 16-687 and described in Table 16-701.

**Figure 16-687. QUEUE_94_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-701. QUEUE_94_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>This count is incremented by 1 whenever a packet is added to the queue.<br>This count is decremented by 1 whenever a packet is popped from the queue. |

### 16.4.7.412 QUEUE_94_B Register (offset = 25E4h) [reset = 0h]

QUEUE_94_B is shown in Figure 16-688 and described in Table 16-702.

**Figure 16-688. QUEUE_94_B Register**

| 31-28 | 27-0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-702. QUEUE_94_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. |

### 16.4.7.413 QUEUE_94_C Register (offset = 25E8h) [reset = 0h]

QUEUE_94_C is shown in Figure 16-689 and described in Table 16-703.

**Figure 16-689. QUEUE_94_C Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| HEAD_TAIL | Reserved | | | | | | |
| W-0 | | | | | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| Reserved | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| Reserved | | PACKET_SIZE | | | | | |
| | | R/W-0 | | | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| PACKET_SIZE | | | | | | | |
| R/W-0 | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-703. QUEUE_94_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31 | HEAD_TAIL | W-0 | 0 | Head/Tail Push Control.<br>Set to zero in order to push packet onto tail of queue and set to one in order to push packet onto head of queue. |
| 13-0 | PACKET_SIZE | R/W-0 | 0 | packet_size This field indicates packet size and is assumed to be zero on each packet add unless the value is explicitly overwritten. This field indicates packet size for packet pop operation. |

### 16.4.7.414 QUEUE_94_D Register (offset = 25ECh) [reset = 0h]

QUEUE_94_D is shown in Figure 16-690 and described in Table 16-704.

**Figure 16-690. QUEUE_94_D Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |
| **23** | **22** | **21** | **20** | **19** | **18** | **17** | **16** |
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |
| **15** | **14** | **13** | **12** | **11** | **10** | **9** | **8** |
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |
| **7** | **6** | **5** | **4** | **3** | **2** | **1** | **0** |
| DESC_PTR | | | DESC_SIZE | | | | |
| R/W-0 | | | R/W-0 | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-704. QUEUE_94_D Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-5 | DESC_PTR | R/W-0 | 0 | Descriptor pointer.<br>It will be read as zero if the queue is empty.<br>It will indicate a<br>32-bit aligned address that points to a descriptor when the queue is not empty. |
| 4-0 | DESC_SIZE | R/W-0 | 0 | Descriptor Size.<br>It is encoded in<br>4-byte increments with values 0 to 31 representing 24 and so on to 148 bytes.<br>This field will return a 0x0 when an empty queue is read.<br>Queue Manager Queue N Registers D To save hardware resources, the queue manager internally stores descriptor size (desc_size) information in four bits.<br>However, register D has five LSBs that specify descriptor size.<br>As a consequence, the value of desc_size that is pushed may not be same as that is read during a pop.<br>The value that is read back is equal to always rounded to an odd number.<br>So, for even values, the value read back is one more than what was written.<br>For odd values, the value read back is same as the value that was written.<br>Note that this<br>5-bit field (desc_size) is unrelated to the code for size of descriptors in a descriptor region.<br>It is just a place holder for a<br>5-bit value that is maintained across the push and pop operations for every descriptor managed by the queue manager. |

### 16.4.7.415 QUEUE_95_A Register (offset = 25F0h) [reset = 0h]

QUEUE_95_A is shown in Figure 16-691 and described in Table 16-705.

**Figure 16-691. QUEUE_95_A Register**

| 31-14 | 13-0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-705. QUEUE_95_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>This count is incremented by 1 whenever a packet is added to the queue.<br>This count is decremented by 1 whenever a packet is popped from the queue. |

### 16.4.7.416 QUEUE_95_B Register (offset = 25F4h) [reset = 0h]

QUEUE_95_B is shown in Figure 16-692 and described in Table 16-706.

**Figure 16-692. QUEUE_95_B Register**

| 31–28 | 27–0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-706. QUEUE_95_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. |

### 16.4.7.417 QUEUE_95_C Register (offset = 25F8h) [reset = 0h]

QUEUE_95_C is shown in Figure 16-693 and described in Table 16-707.

**Figure 16-693. QUEUE_95_C Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| HEAD_TAIL | Reserved | | | | | | |
| W-0 | | | | | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| Reserved | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| Reserved | | PACKET_SIZE | | | | | |
| | | R/W-0 | | | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| PACKET_SIZE | | | | | | | |
| R/W-0 | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-707. QUEUE_95_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31 | HEAD_TAIL | W-0 | 0 | Head/Tail Push Control.<br>Set to zero in order to push packet onto tail of queue and set to one in order to push packet onto head of queue. |
| 13-0 | PACKET_SIZE | R/W-0 | 0 | packet_size This field indicates packet size and is assumed to be zero on each packet add unless the value is explicitly overwritten. This field indicates packet size for packet pop operation. |

### 16.4.7.418 QUEUE_95_D Register (offset = 25FCh) [reset = 0h]

QUEUE_95_D is shown in Figure 16-694 and described in Table 16-708.

**Figure 16-694. QUEUE_95_D Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |
| **23** | **22** | **21** | **20** | **19** | **18** | **17** | **16** |
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |
| **15** | **14** | **13** | **12** | **11** | **10** | **9** | **8** |
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |
| **7** | **6** | **5** | **4** | **3** | **2** | **1** | **0** |
| DESC_PTR | | | DESC_SIZE | | | | |
| R/W-0 | | | R/W-0 | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-708. QUEUE_95_D Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-5 | DESC_PTR | R/W-0 | 0 | Descriptor pointer.<br>It will be read as zero if the queue is empty.<br>It will indicate a<br>32-bit aligned address that points to a descriptor when the queue is not empty. |
| 4-0 | DESC_SIZE | R/W-0 | 0 | Descriptor Size.<br>It is encoded in<br>4-byte increments with values 0 to 31 representing 24 and so on to 148 bytes.<br>This field will return a 0x0 when an empty queue is read.<br>Queue Manager Queue N Registers D To save hardware resources, the queue manager internally stores descriptor size (desc_size) information in four bits.<br>However, register D has five LSBs that specify descriptor size.<br>As a consequence, the value of desc_size that is pushed may not be same as that is read during a pop.<br>The value that is read back is equal to always rounded to an odd number.<br>So, for even values, the value read back is one more than what was written.<br>For odd values, the value read back is same as the value that was written.<br>Note that this<br>5-bit field (desc_size) is unrelated to the code for size of descriptors in a descriptor region.<br>It is just a place holder for a<br>5-bit value that is maintained across the push and pop operations for every descriptor managed by the queue manager. |

### 16.4.7.419 QUEUE_96_A Register (offset = 2600h) [reset = 0h]

QUEUE_96_A is shown in Figure 16-695 and described in Table 16-709.

**Figure 16-695. QUEUE_96_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-709. QUEUE_96_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>This count is incremented by 1 whenever a packet is added to the queue.<br>This count is decremented by 1 whenever a packet is popped from the queue. |

### 16.4.7.420 QUEUE_96_B Register (offset = 2604h) [reset = 0h]

QUEUE_96_B is shown in Figure 16-696 and described in Table 16-710.

**Figure 16-696. QUEUE_96_B Register**

| 31-28 | 27-0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-710. QUEUE_96_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. |

### 16.4.7.421 QUEUE_96_C Register (offset = 2608h) [reset = 0h]

QUEUE_96_C is shown in Figure 16-697 and described in Table 16-711.

**Figure 16-697. QUEUE_96_C Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| HEAD_TAIL | Reserved | | | | | | |
| W-0 | | | | | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| Reserved | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| Reserved | | PACKET_SIZE | | | | | |
| | | R/W-0 | | | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| PACKET_SIZE | | | | | | | |
| R/W-0 | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-711. QUEUE_96_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31 | HEAD_TAIL | W-0 | 0 | Head/Tail Push Control.<br>Set to zero in order to push packet onto tail of queue and set to one in order to push packet onto head of queue. |
| 13-0 | PACKET_SIZE | R/W-0 | 0 | packet_size This field indicates packet size and is assumed to be zero on each packet add unless the value is explicitly overwritten. This field indicates packet size for packet pop operation. |

### 16.4.7.422 QUEUE_96_D Register (offset = 260Ch) [reset = 0h]

QUEUE_96_D is shown in Figure 16-698 and described in Table 16-712.

**Figure 16-698. QUEUE_96_D Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | DESC_SIZE | | | | |
| R/W-0 | | | R/W-0 | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-712. QUEUE_96_D Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-5 | DESC_PTR | R/W-0 | 0 | Descriptor pointer.<br>It will be read as zero if the queue is empty.<br>It will indicate a<br>32-bit aligned address that points to a descriptor when the queue is not empty. |
| 4-0 | DESC_SIZE | R/W-0 | 0 | Descriptor Size.<br>It is encoded in<br>4-byte increments with values 0 to 31 representing 24 and so on to 148 bytes.<br>This field will return a 0x0 when an empty queue is read.<br>Queue Manager Queue N Registers D To save hardware resources, the queue manager internally stores descriptor size (desc_size) information in four bits.<br>However, register D has five LSBs that specify descriptor size.<br>As a consequence, the value of desc_size that is pushed may not be same as that is read during a pop.<br>The value that is read back is equal to always rounded to an odd number.<br>So, for even values, the value read back is one more than what was written.<br>For odd values, the value read back is same as the value that was written.<br>Note that this<br>5-bit field (desc_size) is unrelated to the code for size of descriptors in a descriptor region.<br>It is just a place holder for a<br>5-bit value that is maintained across the push and pop operations for every descriptor managed by the queue manager. |

### 16.4.7.423 QUEUE_97_A Register (offset = 2610h) [reset = 0h]

QUEUE_97_A is shown in Figure 16-699 and described in Table 16-713.

**Figure 16-699. QUEUE_97_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-713. QUEUE_97_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>This count is incremented by 1 whenever a packet is added to the queue.<br>This count is decremented by 1 whenever a packet is popped from the queue. |

### 16.4.7.424 QUEUE_97_B Register (offset = 2614h) [reset = 0h]

QUEUE_97_B is shown in Figure 16-700 and described in Table 16-714.

**Figure 16-700. QUEUE_97_B Register**

| 31-28 | 27-0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-714. QUEUE_97_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. |

### 16.4.7.425 QUEUE_97_C Register (offset = 2618h) [reset = 0h]

QUEUE_97_C is shown in Figure 16-701 and described in Table 16-715.

**Figure 16-701. QUEUE_97_C Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| HEAD_TAIL | Reserved | | | | | | |
| W-0 | | | | | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| Reserved | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| Reserved | | PACKET_SIZE | | | | | |
| | | R/W-0 | | | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| PACKET_SIZE | | | | | | | |
| R/W-0 | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-715. QUEUE_97_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31 | HEAD_TAIL | W-0 | 0 | Head/Tail Push Control.<br>Set to zero in order to push packet onto tail of queue and set to one in order to push packet onto head of queue. |
| 13-0 | PACKET_SIZE | R/W-0 | 0 | packet_size This field indicates packet size and is assumed to be zero on each packet add unless the value is explicitly overwritten. This field indicates packet size for packet pop operation. |

### 16.4.7.426 QUEUE_97_D Register (offset = 261Ch) [reset = 0h]

QUEUE_97_D is shown in Figure 16-702 and described in Table 16-716.

**Figure 16-702. QUEUE_97_D Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |
| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |
| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |
| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
| DESC_PTR | | | DESC_SIZE | | | | |
| R/W-0 | | | R/W-0 | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-716. QUEUE_97_D Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-5 | DESC_PTR | R/W-0 | 0 | Descriptor pointer.<br>It will be read as zero if the queue is empty.<br>It will indicate a<br>32-bit aligned address that points to a descriptor when the queue is not empty. |
| 4-0 | DESC_SIZE | R/W-0 | 0 | Descriptor Size.<br>It is encoded in<br>4-byte increments with values 0 to 31 representing 24 and so on to 148 bytes.<br>This field will return a 0x0 when an empty queue is read.<br>Queue Manager Queue N Registers D To save hardware resources, the queue manager internally stores descriptor size (desc_size) information in four bits.<br>However, register D has five LSBs that specify descriptor size.<br>As a consequence, the value of desc_size that is pushed may not be same as that is read during a pop.<br>The value that is read back is equal to always rounded to an odd number.<br>So, for even values, the value read back is one more than what was written.<br>For odd values, the value read back is same as the value that was written.<br>Note that this<br>5-bit field (desc_size) is unrelated to the code for size of descriptors in a descriptor region.<br>It is just a place holder for a<br>5-bit value that is maintained across the push and pop operations for every descriptor managed by the queue manager. |

### 16.4.7.427 QUEUE_98_A Register (offset = 2620h) [reset = 0h]

QUEUE_98_A is shown in Figure 16-703 and described in Table 16-717.

**Figure 16-703. QUEUE_98_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-717. QUEUE_98_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>This count is incremented by 1 whenever a packet is added to the queue.<br>This count is decremented by 1 whenever a packet is popped from the queue. |

### 16.4.7.428 QUEUE_98_B Register (offset = 2624h) [reset = 0h]

QUEUE_98_B is shown in Figure 16-704 and described in Table 16-718.

**Figure 16-704. QUEUE_98_B Register**

| 31–28 | 27–0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-718. QUEUE_98_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. |

### 16.4.7.429 QUEUE_98_C Register (offset = 2628h) [reset = 0h]

QUEUE_98_C is shown in Figure 16-705 and described in Table 16-719.

**Figure 16-705. QUEUE_98_C Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| HEAD_TAIL | Reserved | | | | | | |
| W-0 | | | | | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| Reserved | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| Reserved | | PACKET_SIZE | | | | | |
| | | R/W-0 | | | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| PACKET_SIZE | | | | | | | |
| R/W-0 | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-719. QUEUE_98_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31 | HEAD_TAIL | W-0 | 0 | Head/Tail Push Control.<br>Set to zero in order to push packet onto tail of queue and set to one in order to push packet onto head of queue. |
| 13-0 | PACKET_SIZE | R/W-0 | 0 | packet_size This field indicates packet size and is assumed to be zero on each packet add unless the value is explicitly overwritten. This field indicates packet size for packet pop operation. |

### 16.4.7.430 QUEUE_98_D Register (offset = 262Ch) [reset = 0h]

QUEUE_98_D is shown in Figure 16-706 and described in Table 16-720.

**Figure 16-706. QUEUE_98_D Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |
| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |
| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |
| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
| DESC_PTR | | | DESC_SIZE | | | | |
| R/W-0 | | | R/W-0 | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-720. QUEUE_98_D Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-5 | DESC_PTR | R/W-0 | 0 | Descriptor pointer.<br>It will be read as zero if the queue is empty.<br>It will indicate a<br>32-bit aligned address that points to a descriptor when the queue is not empty. |
| 4-0 | DESC_SIZE | R/W-0 | 0 | Descriptor Size.<br>It is encoded in<br>4-byte increments with values 0 to 31 representing 24 and so on to 148 bytes.<br>This field will return a 0x0 when an empty queue is read.<br>Queue Manager Queue N Registers D To save hardware resources, the queue manager internally stores descriptor size (desc_size) information in four bits.<br>However, register D has five LSBs that specify descriptor size.<br>As a consequence, the value of desc_size that is pushed may not be same as that is read during a pop.<br>The value that is read back is equal to always rounded to an odd number.<br>So, for even values, the value read back is one more than what was written.<br>For odd values, the value read back is same as the value that was written.<br>Note that this<br>5-bit field (desc_size) is unrelated to the code for size of descriptors in a descriptor region.<br>It is just a place holder for a<br>5-bit value that is maintained across the push and pop operations for every descriptor managed by the queue manager. |

### 16.4.7.431 QUEUE_99_A Register (offset = 2630h) [reset = 0h]

QUEUE_99_A is shown in Figure 16-707 and described in Table 16-721.

**Figure 16-707. QUEUE_99_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-721. QUEUE_99_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>This count is incremented by 1 whenever a packet is added to the queue.<br>This count is decremented by 1 whenever a packet is popped from the queue. |

### 16.4.7.432 QUEUE_99_B Register (offset = 2634h) [reset = 0h]

QUEUE_99_B is shown in Figure 16-708 and described in Table 16-722.

**Figure 16-708. QUEUE_99_B Register**

| 31–28 | 27–0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-722. QUEUE_99_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. |

### 16.4.7.433 QUEUE_99_C Register (offset = 2638h) [reset = 0h]

QUEUE_99_C is shown in Figure 16-709 and described in Table 16-723.

**Figure 16-709. QUEUE_99_C Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| HEAD_TAIL | Reserved | | | | | | |
| W-0 | | | | | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| Reserved | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| Reserved | | PACKET_SIZE | | | | | |
| | | R/W-0 | | | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| PACKET_SIZE | | | | | | | |
| R/W-0 | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-723. QUEUE_99_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31 | HEAD_TAIL | W-0 | 0 | Head/Tail Push Control.<br>Set to zero in order to push packet onto tail of queue and set to one in order to push packet onto head of queue. |
| 13-0 | PACKET_SIZE | R/W-0 | 0 | packet_size This field indicates packet size and is assumed to be zero on each packet add unless the value is explicitly overwritten. This field indicates packet size for packet pop operation. |

### 16.4.7.434 QUEUE_99_D Register (offset = 263Ch) [reset = 0h]

QUEUE_99_D is shown in Figure 16-710 and described in Table 16-724.

**Figure 16-710. QUEUE_99_D Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |
| **23** | **22** | **21** | **20** | **19** | **18** | **17** | **16** |
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |
| **15** | **14** | **13** | **12** | **11** | **10** | **9** | **8** |
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |
| **7** | **6** | **5** | **4** | **3** | **2** | **1** | **0** |
| DESC_PTR | | | DESC_SIZE | | | | |
| R/W-0 | | | R/W-0 | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-724. QUEUE_99_D Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-5 | DESC_PTR | R/W-0 | 0 | Descriptor pointer.<br>It will be read as zero if the queue is empty.<br>It will indicate a<br>32-bit aligned address that points to a descriptor when the queue is not empty. |
| 4-0 | DESC_SIZE | R/W-0 | 0 | Descriptor Size.<br>It is encoded in<br>4-byte increments with values 0 to 31 representing 24 and so on to 148 bytes.<br>This field will return a 0x0 when an empty queue is read.<br>Queue Manager Queue N Registers D To save hardware resources, the queue manager internally stores descriptor size (desc_size) information in four bits.<br>However, register D has five LSBs that specify descriptor size.<br>As a consequence, the value of desc_size that is pushed may not be same as that is read during a pop.<br>The value that is read back is equal to always rounded to an odd number.<br>So, for even values, the value read back is one more than what was written.<br>For odd values, the value read back is same as the value that was written.<br>Note that this<br>5-bit field (desc_size) is unrelated to the code for size of descriptors in a descriptor region.<br>It is just a place holder for a<br>5-bit value that is maintained across the push and pop operations for every descriptor managed by the queue manager. |

### 16.4.7.435 QUEUE_100_A Register (offset = 2640h) [reset = 0h]

QUEUE_100_A is shown in Figure 16-711 and described in Table 16-725.

**Figure 16-711. QUEUE_100_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-725. QUEUE_100_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>This count is incremented by 1 whenever a packet is added to the queue.<br>This count is decremented by 1 whenever a packet is popped from the queue. |

### 16.4.7.436 QUEUE_100_B Register (offset = 2644h) [reset = 0h]

QUEUE_100_B is shown in Figure 16-712 and described in Table 16-726.

**Figure 16-712. QUEUE_100_B Register**

| 31–28 | 27–0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-726. QUEUE_100_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. |

### 16.4.7.437 QUEUE_100_C Register (offset = 2648h) [reset = 0h]

QUEUE_100_C is shown in Figure 16-713 and described in Table 16-727.

**Figure 16-713. QUEUE_100_C Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| HEAD_TAIL | Reserved | | | | | | |
| W-0 | | | | | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| Reserved | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| Reserved | | PACKET_SIZE | | | | | |
| | | R/W-0 | | | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| PACKET_SIZE | | | | | | | |
| R/W-0 | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-727. QUEUE_100_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31 | HEAD_TAIL | W-0 | 0 | Head/Tail Push Control.<br>Set to zero in order to push packet onto tail of queue and set to one in order to push packet onto head of queue. |
| 13-0 | PACKET_SIZE | R/W-0 | 0 | packet_size This field indicates packet size and is assumed to be zero on each packet add unless the value is explicitly overwritten. This field indicates packet size for packet pop operation. |

### 16.4.7.438 QUEUE_100_D Register (offset = 264Ch) [reset = 0h]

QUEUE_100_D is shown in Figure 16-714 and described in Table 16-728.

**Figure 16-714. QUEUE_100_D Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |
| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |
| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |
| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
| DESC_PTR | | | DESC_SIZE | | | | |
| R/W-0 | | | R/W-0 | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-728. QUEUE_100_D Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-5 | DESC_PTR | R/W-0 | 0 | Descriptor pointer.<br>It will be read as zero if the queue is empty.<br>It will indicate a<br>32-bit aligned address that points to a descriptor when the queue is not empty. |
| 4-0 | DESC_SIZE | R/W-0 | 0 | Descriptor Size.<br>It is encoded in<br>4-byte increments with values 0 to 31 representing 24 and so on to 148 bytes.<br>This field will return a 0x0 when an empty queue is read.<br>Queue Manager Queue N Registers D To save hardware resources, the queue manager internally stores descriptor size (desc_size) information in four bits.<br>However, register D has five LSBs that specify descriptor size.<br>As a consequence, the value of desc_size that is pushed may not be same as that is read during a pop.<br>The value that is read back is equal to always rounded to an odd number.<br>So, for even values, the value read back is one more than what was written.<br>For odd values, the value read back is same as the value that was written.<br>Note that this<br>5-bit field (desc_size) is unrelated to the code for size of descriptors in a descriptor region.<br>It is just a place holder for a<br>5-bit value that is maintained across the push and pop operations for every descriptor managed by the queue manager. |

### 16.4.7.439 QUEUE_101_A Register (offset = 2650h) [reset = 0h]

QUEUE_101_A is shown in Figure 16-715 and described in Table 16-729.

**Figure 16-715. QUEUE_101_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-729. QUEUE_101_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>This count is incremented by 1 whenever a packet is added to the queue.<br>This count is decremented by 1 whenever a packet is popped from the queue. |

### 16.4.7.440 QUEUE_101_B Register (offset = 2654h) [reset = 0h]

QUEUE_101_B is shown in Figure 16-716 and described in Table 16-730.

**Figure 16-716. QUEUE_101_B Register**

| 31-28 | 27-0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-730. QUEUE_101_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. |

### 16.4.7.441 QUEUE_101_C Register (offset = 2658h) [reset = 0h]

QUEUE_101_C is shown in Figure 16-717 and described in Table 16-731.

**Figure 16-717. QUEUE_101_C Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| HEAD_TAIL | Reserved | | | | | | |
| W-0 | | | | | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| Reserved | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| Reserved | | PACKET_SIZE | | | | | |
| | | R/W-0 | | | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| PACKET_SIZE | | | | | | | |
| R/W-0 | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-731. QUEUE_101_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31 | HEAD_TAIL | W-0 | 0 | Head/Tail Push Control.<br>Set to zero in order to push packet onto tail of queue and set to one in order to push packet onto head of queue. |
| 13-0 | PACKET_SIZE | R/W-0 | 0 | packet_size This field indicates packet size and is assumed to be zero on each packet add unless the value is explicitly overwritten. This field indicates packet size for packet pop operation. |

### 16.4.7.442 QUEUE_101_D Register (offset = 265Ch) [reset = 0h]

QUEUE_101_D is shown in Figure 16-718 and described in Table 16-732.

**Figure 16-718. QUEUE_101_D Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |
| **23** | **22** | **21** | **20** | **19** | **18** | **17** | **16** |
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |
| **15** | **14** | **13** | **12** | **11** | **10** | **9** | **8** |
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |
| **7** | **6** | **5** | **4** | **3** | **2** | **1** | **0** |
| DESC_PTR | | | DESC_SIZE | | | | |
| R/W-0 | | | R/W-0 | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-732. QUEUE_101_D Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-5 | DESC_PTR | R/W-0 | 0 | Descriptor pointer.<br>It will be read as zero if the queue is empty.<br>It will indicate a<br>32-bit aligned address that points to a descriptor when the queue is not empty. |
| 4-0 | DESC_SIZE | R/W-0 | 0 | Descriptor Size.<br>It is encoded in<br>4-byte increments with values 0 to 31 representing 24 and so on to 148 bytes.<br>This field will return a 0x0 when an empty queue is read.<br>Queue Manager Queue N Registers D To save hardware resources, the queue manager internally stores descriptor size (desc_size) information in four bits.<br>However, register D has five LSBs that specify descriptor size.<br>As a consequence, the value of desc_size that is pushed may not be same as that is read during a pop.<br>The value that is read back is equal to always rounded to an odd number.<br>So, for even values, the value read back is one more than what was written.<br>For odd values, the value read back is same as the value that was written.<br>Note that this<br>5-bit field (desc_size) is unrelated to the code for size of descriptors in a descriptor region.<br>It is just a place holder for a<br>5-bit value that is maintained across the push and pop operations for every descriptor managed by the queue manager. |

### 16.4.7.443 QUEUE_102_A Register (offset = 2660h) [reset = 0h]

QUEUE_102_A is shown in Figure 16-719 and described in Table 16-733.

**Figure 16-719. QUEUE_102_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-733. QUEUE_102_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>This count is incremented by 1 whenever a packet is added to the queue.<br>This count is decremented by 1 whenever a packet is popped from the queue. |

### 16.4.7.444 QUEUE_102_B Register (offset = 2664h) [reset = 0h]

QUEUE_102_B is shown in Figure 16-720 and described in Table 16-734.

**Figure 16-720. QUEUE_102_B Register**

| 31–28 | 27–0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-734. QUEUE_102_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. |

### 16.4.7.445 QUEUE_102_C Register (offset = 2668h) [reset = 0h]

QUEUE_102_C is shown in Figure 16-721 and described in Table 16-735.

**Figure 16-721. QUEUE_102_C Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| HEAD_TAIL | Reserved | | | | | | |
| W-0 | | | | | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| Reserved | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| Reserved | | PACKET_SIZE | | | | | |
| | | R/W-0 | | | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| PACKET_SIZE | | | | | | | |
| R/W-0 | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-735. QUEUE_102_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31 | HEAD_TAIL | W-0 | 0 | Head/Tail Push Control.<br>Set to zero in order to push packet onto tail of queue and set to one in order to push packet onto head of queue. |
| 13-0 | PACKET_SIZE | R/W-0 | 0 | packet_size This field indicates packet size and is assumed to be zero on each packet add unless the value is explicitly overwritten. This field indicates packet size for packet pop operation. |

### 16.4.7.446 QUEUE_102_D Register (offset = 266Ch) [reset = 0h]

QUEUE_102_D is shown in Figure 16-722 and described in Table 16-736.

**Figure 16-722. QUEUE_102_D Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | DESC_SIZE | | | | |
| R/W-0 | | | R/W-0 | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-736. QUEUE_102_D Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-5 | DESC_PTR | R/W-0 | 0 | Descriptor pointer.<br>It will be read as zero if the queue is empty.<br>It will indicate a<br>32-bit aligned address that points to a descriptor when the queue is not empty. |
| 4-0 | DESC_SIZE | R/W-0 | 0 | Descriptor Size.<br>It is encoded in<br>4-byte increments with values 0 to 31 representing 24 and so on to 148 bytes.<br>This field will return a 0x0 when an empty queue is read.<br>Queue Manager Queue N Registers D To save hardware resources, the queue manager internally stores descriptor size (desc_size) information in four bits.<br>However, register D has five LSBs that specify descriptor size.<br>As a consequence, the value of desc_size that is pushed may not be same as that is read during a pop.<br>The value that is read back is equal to always rounded to an odd number.<br>So, for even values, the value read back is one more than what was written.<br>For odd values, the value read back is same as the value that was written.<br>Note that this<br>5-bit field (desc_size) is unrelated to the code for size of descriptors in a descriptor region.<br>It is just a place holder for a<br>5-bit value that is maintained across the push and pop operations for every descriptor managed by the queue manager. |

### 16.4.7.447 QUEUE_103_A Register (offset = 2670h) [reset = 0h]

QUEUE_103_A is shown in Figure 16-723 and described in Table 16-737.

**Figure 16-723. QUEUE_103_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-737. QUEUE_103_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>This count is incremented by 1 whenever a packet is added to the queue.<br>This count is decremented by 1 whenever a packet is popped from the queue. |

### 16.4.7.448 QUEUE_103_B Register (offset = 2674h) [reset = 0h]

QUEUE_103_B is shown in Figure 16-724 and described in Table 16-738.

**Figure 16-724. QUEUE_103_B Register**

| 31-28 | 27-0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-738. QUEUE_103_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. |

### 16.4.7.449 QUEUE_103_C Register (offset = 2678h) [reset = 0h]

QUEUE_103_C is shown in Figure 16-725 and described in Table 16-739.

**Figure 16-725. QUEUE_103_C Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| HEAD_TAIL | Reserved | | | | | | |
| W-0 | | | | | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| Reserved | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| Reserved | | PACKET_SIZE | | | | | |
| | | R/W-0 | | | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| PACKET_SIZE | | | | | | | |
| R/W-0 | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-739. QUEUE_103_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31 | HEAD_TAIL | W-0 | 0 | Head/Tail Push Control.<br>Set to zero in order to push packet onto tail of queue and set to one in order to push packet onto head of queue. |
| 13-0 | PACKET_SIZE | R/W-0 | 0 | packet_size This field indicates packet size and is assumed to be zero on each packet add unless the value is explicitly overwritten. This field indicates packet size for packet pop operation. |

### 16.4.7.450 QUEUE_103_D Register (offset = 267Ch) [reset = 0h]

QUEUE_103_D is shown in Figure 16-726 and described in Table 16-740.

**Figure 16-726. QUEUE_103_D Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |
| **23** | **22** | **21** | **20** | **19** | **18** | **17** | **16** |
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |
| **15** | **14** | **13** | **12** | **11** | **10** | **9** | **8** |
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |
| **7** | **6** | **5** | **4** | **3** | **2** | **1** | **0** |
| DESC_PTR | | | DESC_SIZE | | | | |
| R/W-0 | | | R/W-0 | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-740. QUEUE_103_D Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-5 | DESC_PTR | R/W-0 | 0 | Descriptor pointer.<br>It will be read as zero if the queue is empty.<br>It will indicate a<br>32-bit aligned address that points to a descriptor when the queue is not empty. |
| 4-0 | DESC_SIZE | R/W-0 | 0 | Descriptor Size.<br>It is encoded in<br>4-byte increments with values 0 to 31 representing 24 and so on to 148 bytes.<br>This field will return a 0x0 when an empty queue is read.<br>Queue Manager Queue N Registers D To save hardware resources, the queue manager internally stores descriptor size (desc_size) information in four bits.<br>However, register D has five LSBs that specify descriptor size.<br>As a consequence, the value of desc_size that is pushed may not be same as that is read during a pop.<br>The value that is read back is equal to always rounded to an odd number.<br>So, for even values, the value read back is one more than what was written.<br>For odd values, the value read back is same as the value that was written.<br>Note that this<br>5-bit field (desc_size) is unrelated to the code for size of descriptors in a descriptor region.<br>It is just a place holder for a<br>5-bit value that is maintained across the push and pop operations for every descriptor managed by the queue manager. |

### 16.4.7.451 QUEUE_104_A Register (offset = 2680h) [reset = 0h]

QUEUE_104_A is shown in Figure 16-727 and described in Table 16-741.

**Figure 16-727. QUEUE_104_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-741. QUEUE_104_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>This count is incremented by 1 whenever a packet is added to the queue.<br>This count is decremented by 1 whenever a packet is popped from the queue. |

### 16.4.7.452 QUEUE_104_B Register (offset = 2684h) [reset = 0h]

QUEUE_104_B is shown in Figure 16-728 and described in Table 16-742.

**Figure 16-728. QUEUE_104_B Register**

| 31–28 | 27–0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-742. QUEUE_104_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. |

### 16.4.7.453 QUEUE_104_C Register (offset = 2688h) [reset = 0h]

QUEUE_104_C is shown in Figure 16-729 and described in Table 16-743.

**Figure 16-729. QUEUE_104_C Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| HEAD_TAIL | Reserved | | | | | | |
| W-0 | | | | | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| Reserved | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| Reserved | | PACKET_SIZE | | | | | |
| | | R/W-0 | | | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| PACKET_SIZE | | | | | | | |
| R/W-0 | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-743. QUEUE_104_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31 | HEAD_TAIL | W-0 | 0 | Head/Tail Push Control.<br>Set to zero in order to push packet onto tail of queue and set to one in order to push packet onto head of queue. |
| 13-0 | PACKET_SIZE | R/W-0 | 0 | packet_size This field indicates packet size and is assumed to be zero on each packet add unless the value is explicitly overwritten. This field indicates packet size for packet pop operation. |

### 16.4.7.454 QUEUE_104_D Register (offset = 268Ch) [reset = 0h]

QUEUE_104_D is shown in Figure 16-730 and described in Table 16-744.

**Figure 16-730. QUEUE_104_D Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |
| **23** | **22** | **21** | **20** | **19** | **18** | **17** | **16** |
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |
| **15** | **14** | **13** | **12** | **11** | **10** | **9** | **8** |
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |
| **7** | **6** | **5** | **4** | **3** | **2** | **1** | **0** |
| DESC_PTR | | | DESC_SIZE | | | | |
| R/W-0 | | | R/W-0 | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-744. QUEUE_104_D Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-5 | DESC_PTR | R/W-0 | 0 | Descriptor pointer.<br>It will be read as zero if the queue is empty.<br>It will indicate a<br>32-bit aligned address that points to a descriptor when the queue is not empty. |
| 4-0 | DESC_SIZE | R/W-0 | 0 | Descriptor Size.<br>It is encoded in<br>4-byte increments with values 0 to 31 representing 24 and so on to 148 bytes.<br>This field will return a 0x0 when an empty queue is read.<br>Queue Manager Queue N Registers D To save hardware resources, the queue manager internally stores descriptor size (desc_size) information in four bits.<br>However, register D has five LSBs that specify descriptor size.<br>As a consequence, the value of desc_size that is pushed may not be same as that is read during a pop.<br>The value that is read back is equal to always rounded to an odd number.<br>So, for even values, the value read back is one more than what was written.<br>For odd values, the value read back is same as the value that was written.<br>Note that this<br>5-bit field (desc_size) is unrelated to the code for size of descriptors in a descriptor region.<br>It is just a place holder for a<br>5-bit value that is maintained across the push and pop operations for every descriptor managed by the queue manager. |

### 16.4.7.455 QUEUE_105_A Register (offset = 2690h) [reset = 0h]

QUEUE_105_A is shown in Figure 16-731 and described in Table 16-745.

**Figure 16-731. QUEUE_105_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-745. QUEUE_105_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>This count is incremented by 1 whenever a packet is added to the queue.<br>This count is decremented by 1 whenever a packet is popped from the queue. |

### 16.4.7.456 QUEUE_105_B Register (offset = 2694h) [reset = 0h]

QUEUE_105_B is shown in Figure 16-732 and described in Table 16-746.

**Figure 16-732. QUEUE_105_B Register**

| 31–28 | 27–0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-746. QUEUE_105_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. |

### 16.4.7.457 QUEUE_105_C Register (offset = 2698h) [reset = 0h]

QUEUE_105_C is shown in Figure 16-733 and described in Table 16-747.

**Figure 16-733. QUEUE_105_C Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| HEAD_TAIL | Reserved | | | | | | |
| W-0 | | | | | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| Reserved | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| Reserved | | PACKET_SIZE | | | | | |
| | | R/W-0 | | | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| PACKET_SIZE | | | | | | | |
| R/W-0 | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-747. QUEUE_105_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31 | HEAD_TAIL | W-0 | 0 | Head/Tail Push Control.<br>Set to zero in order to push packet onto tail of queue and set to one in order to push packet onto head of queue. |
| 13-0 | PACKET_SIZE | R/W-0 | 0 | packet_size This field indicates packet size and is assumed to be zero on each packet add unless the value is explicitly overwritten. This field indicates packet size for packet pop operation. |

### 16.4.7.458 QUEUE_105_D Register (offset = 269Ch) [reset = 0h]

QUEUE_105_D is shown in Figure 16-734 and described in Table 16-748.

**Figure 16-734. QUEUE_105_D Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | DESC_SIZE | | | | |
| R/W-0 | | | R/W-0 | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-748. QUEUE_105_D Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-5 | DESC_PTR | R/W-0 | 0 | Descriptor pointer.<br>It will be read as zero if the queue is empty.<br>It will indicate a<br>32-bit aligned address that points to a descriptor when the queue is not empty. |
| 4-0 | DESC_SIZE | R/W-0 | 0 | Descriptor Size.<br>It is encoded in<br>4-byte increments with values 0 to 31 representing 24 and so on to 148 bytes.<br>This field will return a 0x0 when an empty queue is read.<br>Queue Manager Queue N Registers D To save hardware resources, the queue manager internally stores descriptor size (desc_size) information in four bits.<br>However, register D has five LSBs that specify descriptor size.<br>As a consequence, the value of desc_size that is pushed may not be same as that is read during a pop.<br>The value that is read back is equal to always rounded to an odd number.<br>So, for even values, the value read back is one more than what was written.<br>For odd values, the value read back is same as the value that was written.<br>Note that this<br>5-bit field (desc_size) is unrelated to the code for size of descriptors in a descriptor region.<br>It is just a place holder for a<br>5-bit value that is maintained across the push and pop operations for every descriptor managed by the queue manager. |

### 16.4.7.459 QUEUE_106_A Register (offset = 26A0h) [reset = 0h]

QUEUE_106_A is shown in Figure 16-735 and described in Table 16-749.

**Figure 16-735. QUEUE_106_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-749. QUEUE_106_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>This count is incremented by 1 whenever a packet is added to the queue.<br>This count is decremented by 1 whenever a packet is popped from the queue. |

### 16.4.7.460 QUEUE_106_B Register (offset = 26A4h) [reset = 0h]

QUEUE_106_B is shown in Figure 16-736 and described in Table 16-750.

**Figure 16-736. QUEUE_106_B Register**

| 31–28 | 27–0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-750. QUEUE_106_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. |

### 16.4.7.461 QUEUE_106_C Register (offset = 26A8h) [reset = 0h]

QUEUE_106_C is shown in Figure 16-737 and described in Table 16-751.

**Figure 16-737. QUEUE_106_C Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| HEAD_TAIL | Reserved | | | | | | |
| W-0 | | | | | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| Reserved | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| Reserved | | PACKET_SIZE | | | | | |
| | | R/W-0 | | | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| PACKET_SIZE | | | | | | | |
| R/W-0 | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-751. QUEUE_106_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31 | HEAD_TAIL | W-0 | 0 | Head/Tail Push Control.<br>Set to zero in order to push packet onto tail of queue and set to one in order to push packet onto head of queue. |
| 13-0 | PACKET_SIZE | R/W-0 | 0 | packet_size This field indicates packet size and is assumed to be zero on each packet add unless the value is explicitly overwritten. This field indicates packet size for packet pop operation. |

### 16.4.7.462 QUEUE_106_D Register (offset = 26ACh) [reset = 0h]

QUEUE_106_D is shown in Figure 16-738 and described in Table 16-752.

**Figure 16-738. QUEUE_106_D Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |
| **23** | **22** | **21** | **20** | **19** | **18** | **17** | **16** |
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |
| **15** | **14** | **13** | **12** | **11** | **10** | **9** | **8** |
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |
| **7** | **6** | **5** | **4** | **3** | **2** | **1** | **0** |
| DESC_PTR | | | DESC_SIZE | | | | |
| R/W-0 | | | R/W-0 | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-752. QUEUE_106_D Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-5 | DESC_PTR | R/W-0 | 0 | Descriptor pointer.<br>It will be read as zero if the queue is empty.<br>It will indicate a<br>32-bit aligned address that points to a descriptor when the queue is not empty. |
| 4-0 | DESC_SIZE | R/W-0 | 0 | Descriptor Size.<br>It is encoded in<br>4-byte increments with values 0 to 31 representing 24 and so on to 148 bytes.<br>This field will return a 0x0 when an empty queue is read.<br>Queue Manager Queue N Registers D To save hardware resources, the queue manager internally stores descriptor size (desc_size) information in four bits.<br>However, register D has five LSBs that specify descriptor size.<br>As a consequence, the value of desc_size that is pushed may not be same as that is read during a pop.<br>The value that is read back is equal to always rounded to an odd number.<br>So, for even values, the value read back is one more than what was written.<br>For odd values, the value read back is same as the value that was written.<br>Note that this<br>5-bit field (desc_size) is unrelated to the code for size of descriptors in a descriptor region.<br>It is just a place holder for a<br>5-bit value that is maintained across the push and pop operations for every descriptor managed by the queue manager. |

### 16.4.7.463 QUEUE_107_A Register (offset = 26B0h) [reset = 0h]

QUEUE_107_A is shown in Figure 16-739 and described in Table 16-753.

**Figure 16-739. QUEUE_107_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-753. QUEUE_107_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>This count is incremented by 1 whenever a packet is added to the queue.<br>This count is decremented by 1 whenever a packet is popped from the queue. |

### 16.4.7.464 QUEUE_107_B Register (offset = 26B4h) [reset = 0h]

QUEUE_107_B is shown in Figure 16-740 and described in Table 16-754.

**Figure 16-740. QUEUE_107_B Register**

| 31-28 | 27-0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-754. QUEUE_107_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. |

### 16.4.7.465 QUEUE_107_C Register (offset = 26B8h) [reset = 0h]

QUEUE_107_C is shown in Figure 16-741 and described in Table 16-755.

**Figure 16-741. QUEUE_107_C Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| HEAD_TAIL | Reserved | | | | | | |
| W-0 | | | | | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| Reserved | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| Reserved | | PACKET_SIZE | | | | | |
| | | R/W-0 | | | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| PACKET_SIZE | | | | | | | |
| R/W-0 | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-755. QUEUE_107_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31 | HEAD_TAIL | W-0 | 0 | Head/Tail Push Control.<br>Set to zero in order to push packet onto tail of queue and set to one in order to push packet onto head of queue. |
| 13-0 | PACKET_SIZE | R/W-0 | 0 | packet_size This field indicates packet size and is assumed to be zero on each packet add unless the value is explicitly overwritten. This field indicates packet size for packet pop operation. |

### 16.4.7.466 QUEUE_107_D Register (offset = 26BCh) [reset = 0h]

QUEUE_107_D is shown in Figure 16-742 and described in Table 16-756.

**Figure 16-742. QUEUE_107_D Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | DESC_SIZE | | | | |
| R/W-0 | | | R/W-0 | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-756. QUEUE_107_D Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-5 | DESC_PTR | R/W-0 | 0 | Descriptor pointer.<br>It will be read as zero if the queue is empty.<br>It will indicate a<br>32-bit aligned address that points to a descriptor when the queue is not empty. |
| 4-0 | DESC_SIZE | R/W-0 | 0 | Descriptor Size.<br>It is encoded in<br>4-byte increments with values 0 to 31 representing 24 and so on to 148 bytes.<br>This field will return a 0x0 when an empty queue is read.<br>Queue Manager Queue N Registers D To save hardware resources, the queue manager internally stores descriptor size (desc_size) information in four bits.<br>However, register D has five LSBs that specify descriptor size.<br>As a consequence, the value of desc_size that is pushed may not be same as that is read during a pop.<br>The value that is read back is equal to always rounded to an odd number.<br>So, for even values, the value read back is one more than what was written.<br>For odd values, the value read back is same as the value that was written.<br>Note that this<br>5-bit field (desc_size) is unrelated to the code for size of descriptors in a descriptor region.<br>It is just a place holder for a<br>5-bit value that is maintained across the push and pop operations for every descriptor managed by the queue manager. |

### 16.4.7.467 QUEUE_108_A Register (offset = 26C0h) [reset = 0h]

QUEUE_108_A is shown in Figure 16-743 and described in Table 16-757.

**Figure 16-743. QUEUE_108_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-757. QUEUE_108_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>This count is incremented by 1 whenever a packet is added to the queue.<br>This count is decremented by 1 whenever a packet is popped from the queue. |

### 16.4.7.468 QUEUE_108_B Register (offset = 26C4h) [reset = 0h]

QUEUE_108_B is shown in Figure 16-744 and described in Table 16-758.

**Figure 16-744. QUEUE_108_B Register**

| 31–28 | 27–0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-758. QUEUE_108_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. |

### 16.4.7.469 QUEUE_108_C Register (offset = 26C8h) [reset = 0h]

QUEUE_108_C is shown in Figure 16-745 and described in Table 16-759.

**Figure 16-745. QUEUE_108_C Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| HEAD_TAIL | Reserved | | | | | | |
| W-0 | | | | | | | |
| **23** | **22** | **21** | **20** | **19** | **18** | **17** | **16** |
| Reserved | | | | | | | |
| **15** | **14** | **13** | **12** | **11** | **10** | **9** | **8** |
| Reserved | | PACKET_SIZE | | | | | |
| | | R/W-0 | | | | | |
| **7** | **6** | **5** | **4** | **3** | **2** | **1** | **0** |
| PACKET_SIZE | | | | | | | |
| R/W-0 | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-759. QUEUE_108_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31 | HEAD_TAIL | W-0 | 0 | Head/Tail Push Control.<br>Set to zero in order to push packet onto tail of queue and set to one in order to push packet onto head of queue. |
| 13-0 | PACKET_SIZE | R/W-0 | 0 | packet_size This field indicates packet size and is assumed to be zero on each packet add unless the value is explicitly overwritten. This field indicates packet size for packet pop operation. |

### 16.4.7.470 QUEUE_108_D Register (offset = 26CCh) [reset = 0h]

QUEUE_108_D is shown in Figure 16-746 and described in Table 16-760.

**Figure 16-746. QUEUE_108_D Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | DESC_SIZE | | | | |
| R/W-0 | | | R/W-0 | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-760. QUEUE_108_D Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-5 | DESC_PTR | R/W-0 | 0 | Descriptor pointer.<br>It will be read as zero if the queue is empty.<br>It will indicate a<br>32-bit aligned address that points to a descriptor when the queue is not empty. |
| 4-0 | DESC_SIZE | R/W-0 | 0 | Descriptor Size.<br>It is encoded in<br>4-byte increments with values 0 to 31 representing 24 and so on to 148 bytes.<br>This field will return a 0x0 when an empty queue is read.<br>Queue Manager Queue N Registers D To save hardware resources, the queue manager internally stores descriptor size (desc_size) information in four bits.<br>However, register D has five LSBs that specify descriptor size.<br>As a consequence, the value of desc_size that is pushed may not be same as that is read during a pop.<br>The value that is read back is equal to always rounded to an odd number.<br>So, for even values, the value read back is one more than what was written.<br>For odd values, the value read back is same as the value that was written.<br>Note that this<br>5-bit field (desc_size) is unrelated to the code for size of descriptors in a descriptor region.<br>It is just a place holder for a<br>5-bit value that is maintained across the push and pop operations for every descriptor managed by the queue manager. |

### 16.4.7.471 QUEUE_109_A Register (offset = 26D0h) [reset = 0h]

QUEUE_109_A is shown in Figure 16-747 and described in Table 16-761.

**Figure 16-747. QUEUE_109_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-761. QUEUE_109_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>This count is incremented by 1 whenever a packet is added to the queue.<br>This count is decremented by 1 whenever a packet is popped from the queue. |

### 16.4.7.472 QUEUE_109_B Register (offset = 26D4h) [reset = 0h]

QUEUE_109_B is shown in Figure 16-748 and described in Table 16-762.

**Figure 16-748. QUEUE_109_B Register**

| 31–28 | 27–0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-762. QUEUE_109_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. |

### 16.4.7.473 QUEUE_109_C Register (offset = 26D8h) [reset = 0h]

QUEUE_109_C is shown in Figure 16-749 and described in Table 16-763.

**Figure 16-749. QUEUE_109_C Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| HEAD_TAIL | Reserved | | | | | | |
| W-0 | | | | | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| Reserved | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| Reserved | | PACKET_SIZE | | | | | |
| | | R/W-0 | | | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| PACKET_SIZE | | | | | | | |
| R/W-0 | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-763. QUEUE_109_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31 | HEAD_TAIL | W-0 | 0 | Head/Tail Push Control.<br>Set to zero in order to push packet onto tail of queue and set to one in order to push packet onto head of queue. |
| 13-0 | PACKET_SIZE | R/W-0 | 0 | packet_size This field indicates packet size and is assumed to be zero on each packet add unless the value is explicitly overwritten. This field indicates packet size for packet pop operation. |

### 16.4.7.474 QUEUE_109_D Register (offset = 26DCh) [reset = 0h]

QUEUE_109_D is shown in Figure 16-750 and described in Table 16-764.

**Figure 16-750. QUEUE_109_D Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |
| **23** | **22** | **21** | **20** | **19** | **18** | **17** | **16** |
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |
| **15** | **14** | **13** | **12** | **11** | **10** | **9** | **8** |
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |
| **7** | **6** | **5** | **4** | **3** | **2** | **1** | **0** |
| DESC_PTR | | | DESC_SIZE | | | | |
| R/W-0 | | | R/W-0 | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-764. QUEUE_109_D Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-5 | DESC_PTR | R/W-0 | 0 | Descriptor pointer.<br>It will be read as zero if the queue is empty.<br>It will indicate a<br>32-bit aligned address that points to a descriptor when the queue is not empty. |
| 4-0 | DESC_SIZE | R/W-0 | 0 | Descriptor Size.<br>It is encoded in<br>4-byte increments with values 0 to 31 representing 24 and so on to 148 bytes.<br>This field will return a 0x0 when an empty queue is read.<br>Queue Manager Queue N Registers D To save hardware resources, the queue manager internally stores descriptor size (desc_size) information in four bits.<br>However, register D has five LSBs that specify descriptor size.<br>As a consequence, the value of desc_size that is pushed may not be same as that is read during a pop.<br>The value that is read back is equal to always rounded to an odd number.<br>So, for even values, the value read back is one more than what was written.<br>For odd values, the value read back is same as the value that was written.<br>Note that this<br>5-bit field (desc_size) is unrelated to the code for size of descriptors in a descriptor region.<br>It is just a place holder for a<br>5-bit value that is maintained across the push and pop operations for every descriptor managed by the queue manager. |

### 16.4.7.475 QUEUE_110_A Register (offset = 26E0h) [reset = 0h]

QUEUE_110_A is shown in Figure 16-751 and described in Table 16-765.

**Figure 16-751. QUEUE_110_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-765. QUEUE_110_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>This count is incremented by 1 whenever a packet is added to the queue.<br>This count is decremented by 1 whenever a packet is popped from the queue. |

### 16.4.7.476 QUEUE_110_B Register (offset = 26E4h) [reset = 0h]

QUEUE_110_B is shown in Figure 16-752 and described in Table 16-766.

**Figure 16-752. QUEUE_110_B Register**

| 31-28 | 27-0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-766. QUEUE_110_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. |

### 16.4.7.477 QUEUE_110_C Register (offset = 26E8h) [reset = 0h]

QUEUE_110_C is shown in Figure 16-753 and described in Table 16-767.

**Figure 16-753. QUEUE_110_C Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| HEAD_TAIL | Reserved | | | | | | |
| W-0 | | | | | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| Reserved | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| Reserved | | PACKET_SIZE | | | | | |
| | | R/W-0 | | | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| PACKET_SIZE | | | | | | | |
| R/W-0 | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-767. QUEUE_110_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31 | HEAD_TAIL | W-0 | 0 | Head/Tail Push Control.<br>Set to zero in order to push packet onto tail of queue and set to one in order to push packet onto head of queue. |
| 13-0 | PACKET_SIZE | R/W-0 | 0 | packet_size This field indicates packet size and is assumed to be zero on each packet add unless the value is explicitly overwritten. This field indicates packet size for packet pop operation. |

### 16.4.7.478 QUEUE_110_D Register (offset = 26ECh) [reset = 0h]

QUEUE_110_D is shown in Figure 16-754 and described in Table 16-768.

**Figure 16-754. QUEUE_110_D Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |
| **23** | **22** | **21** | **20** | **19** | **18** | **17** | **16** |
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |
| **15** | **14** | **13** | **12** | **11** | **10** | **9** | **8** |
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |
| **7** | **6** | **5** | **4** | **3** | **2** | **1** | **0** |
| DESC_PTR | | | DESC_SIZE | | | | |
| R/W-0 | | | R/W-0 | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-768. QUEUE_110_D Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-5 | DESC_PTR | R/W-0 | 0 | Descriptor pointer.<br>It will be read as zero if the queue is empty.<br>It will indicate a<br>32-bit aligned address that points to a descriptor when the queue is not empty. |
| 4-0 | DESC_SIZE | R/W-0 | 0 | Descriptor Size.<br>It is encoded in<br>4-byte increments with values 0 to 31 representing 24 and so on to 148 bytes.<br>This field will return a 0x0 when an empty queue is read.<br>Queue Manager Queue N Registers D To save hardware resources, the queue manager internally stores descriptor size (desc_size) information in four bits.<br>However, register D has five LSBs that specify descriptor size.<br>As a consequence, the value of desc_size that is pushed may not be same as that is read during a pop.<br>The value that is read back is equal to always rounded to an odd number.<br>So, for even values, the value read back is one more than what was written.<br>For odd values, the value read back is same as the value that was written.<br>Note that this<br>5-bit field (desc_size) is unrelated to the code for size of descriptors in a descriptor region.<br>It is just a place holder for a<br>5-bit value that is maintained across the push and pop operations for every descriptor managed by the queue manager. |

### 16.4.7.479 QUEUE_111_A Register (offset = 26F0h) [reset = 0h]

QUEUE_111_A is shown in Figure 16-755 and described in Table 16-769.

**Figure 16-755. QUEUE_111_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-769. QUEUE_111_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>This count is incremented by 1 whenever a packet is added to the queue.<br>This count is decremented by 1 whenever a packet is popped from the queue. |

### 16.4.7.480 QUEUE_111_B Register (offset = 26F4h) [reset = 0h]

QUEUE_111_B is shown in Figure 16-756 and described in Table 16-770.

**Figure 16-756. QUEUE_111_B Register**

| 31-28 | 27-0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-770. QUEUE_111_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. |

### 16.4.7.481 QUEUE_111_C Register (offset = 26F8h) [reset = 0h]

QUEUE_111_C is shown in Figure 16-757 and described in Table 16-771.

**Figure 16-757. QUEUE_111_C Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| HEAD_TAIL | Reserved | | | | | | |
| W-0 | | | | | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| Reserved | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| Reserved | | PACKET_SIZE | | | | | |
| | | R/W-0 | | | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| PACKET_SIZE | | | | | | | |
| R/W-0 | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-771. QUEUE_111_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31 | HEAD_TAIL | W-0 | 0 | Head/Tail Push Control.<br>Set to zero in order to push packet onto tail of queue and set to one in order to push packet onto head of queue. |
| 13-0 | PACKET_SIZE | R/W-0 | 0 | packet_size This field indicates packet size and is assumed to be zero on each packet add unless the value is explicitly overwritten. This field indicates packet size for packet pop operation. |

### 16.4.7.482 QUEUE_111_D Register (offset = 26FCh) [reset = 0h]

QUEUE_111_D is shown in Figure 16-758 and described in Table 16-772.

**Figure 16-758. QUEUE_111_D Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |
| **23** | **22** | **21** | **20** | **19** | **18** | **17** | **16** |
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |
| **15** | **14** | **13** | **12** | **11** | **10** | **9** | **8** |
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |
| **7** | **6** | **5** | **4** | **3** | **2** | **1** | **0** |
| DESC_PTR | | | DESC_SIZE | | | | |
| R/W-0 | | | R/W-0 | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-772. QUEUE_111_D Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-5 | DESC_PTR | R/W-0 | 0 | Descriptor pointer.<br>It will be read as zero if the queue is empty.<br>It will indicate a<br>32-bit aligned address that points to a descriptor when the queue is not empty. |
| 4-0 | DESC_SIZE | R/W-0 | 0 | Descriptor Size.<br>It is encoded in<br>4-byte increments with values 0 to 31 representing 24 and so on to 148 bytes.<br>This field will return a 0x0 when an empty queue is read.<br>Queue Manager Queue N Registers D To save hardware resources, the queue manager internally stores descriptor size (desc_size) information in four bits.<br>However, register D has five LSBs that specify descriptor size.<br>As a consequence, the value of desc_size that is pushed may not be same as that is read during a pop.<br>The value that is read back is equal to always rounded to an odd number.<br>So, for even values, the value read back is one more than what was written.<br>For odd values, the value read back is same as the value that was written.<br>Note that this<br>5-bit field (desc_size) is unrelated to the code for size of descriptors in a descriptor region.<br>It is just a place holder for a<br>5-bit value that is maintained across the push and pop operations for every descriptor managed by the queue manager. |

### 16.4.7.483 QUEUE_112_A Register (offset = 2700h) [reset = 0h]

QUEUE_112_A is shown in Figure 16-759 and described in Table 16-773.

**Figure 16-759. QUEUE_112_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-773. QUEUE_112_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>This count is incremented by 1 whenever a packet is added to the queue.<br>This count is decremented by 1 whenever a packet is popped from the queue. |

### 16.4.7.484 QUEUE_112_B Register (offset = 2704h) [reset = 0h]

QUEUE_112_B is shown in Figure 16-760 and described in Table 16-774.

**Figure 16-760. QUEUE_112_B Register**

| 31-28 | 27-0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-774. QUEUE_112_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. |

### 16.4.7.485 QUEUE_112_C Register (offset = 2708h) [reset = 0h]

QUEUE_112_C is shown in Figure 16-761 and described in Table 16-775.

**Figure 16-761. QUEUE_112_C Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| HEAD_TAIL | Reserved | | | | | | |
| W-0 | | | | | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| Reserved | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| Reserved | | PACKET_SIZE | | | | | |
| | | R/W-0 | | | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| PACKET_SIZE | | | | | | | |
| R/W-0 | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-775. QUEUE_112_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31 | HEAD_TAIL | W-0 | 0 | Head/Tail Push Control.<br>Set to zero in order to push packet onto tail of queue and set to one in order to push packet onto head of queue. |
| 13-0 | PACKET_SIZE | R/W-0 | 0 | packet_size This field indicates packet size and is assumed to be zero on each packet add unless the value is explicitly overwritten. This field indicates packet size for packet pop operation. |

### 16.4.7.486 QUEUE_112_D Register (offset = 270Ch) [reset = 0h]

QUEUE_112_D is shown in Figure 16-762 and described in Table 16-776.

**Figure 16-762. QUEUE_112_D Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | DESC_SIZE | | | | |
| R/W-0 | | | R/W-0 | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-776. QUEUE_112_D Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-5 | DESC_PTR | R/W-0 | 0 | Descriptor pointer.<br>It will be read as zero if the queue is empty.<br>It will indicate a<br>32-bit aligned address that points to a descriptor when the queue is not empty. |
| 4-0 | DESC_SIZE | R/W-0 | 0 | Descriptor Size.<br>It is encoded in<br>4-byte increments with values 0 to 31 representing 24 and so on to 148 bytes.<br>This field will return a 0x0 when an empty queue is read.<br>Queue Manager Queue N Registers D To save hardware resources, the queue manager internally stores descriptor size (desc_size) information in four bits.<br>However, register D has five LSBs that specify descriptor size.<br>As a consequence, the value of desc_size that is pushed may not be same as that is read during a pop.<br>The value that is read back is equal to always rounded to an odd number.<br>So, for even values, the value read back is one more than what was written.<br>For odd values, the value read back is same as the value that was written.<br>Note that this<br>5-bit field (desc_size) is unrelated to the code for size of descriptors in a descriptor region.<br>It is just a place holder for a<br>5-bit value that is maintained across the push and pop operations for every descriptor managed by the queue manager. |

### 16.4.7.487 QUEUE_113_A Register (offset = 2710h) [reset = 0h]

QUEUE_113_A is shown in Figure 16-763 and described in Table 16-777.

**Figure 16-763. QUEUE_113_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-777. QUEUE_113_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>This count is incremented by 1 whenever a packet is added to the queue.<br>This count is decremented by 1 whenever a packet is popped from the queue. |

### 16.4.7.488 QUEUE_113_B Register (offset = 2714h) [reset = 0h]

QUEUE_113_B is shown in Figure 16-764 and described in Table 16-778.

**Figure 16-764. QUEUE_113_B Register**

| 31-28 | 27-0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-778. QUEUE_113_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. |

### 16.4.7.489 QUEUE_113_C Register (offset = 2718h) [reset = 0h]

QUEUE_113_C is shown in Figure 16-765 and described in Table 16-779.

**Figure 16-765. QUEUE_113_C Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| HEAD_TAIL | Reserved | | | | | | |
| W-0 | | | | | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| Reserved | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| Reserved | | PACKET_SIZE | | | | | |
| | | R/W-0 | | | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| PACKET_SIZE | | | | | | | |
| R/W-0 | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-779. QUEUE_113_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31 | HEAD_TAIL | W-0 | 0 | Head/Tail Push Control.<br>Set to zero in order to push packet onto tail of queue and set to one in order to push packet onto head of queue. |
| 13-0 | PACKET_SIZE | R/W-0 | 0 | packet_size This field indicates packet size and is assumed to be zero on each packet add unless the value is explicitly overwritten. This field indicates packet size for packet pop operation. |

### 16.4.7.490 QUEUE_113_D Register (offset = 271Ch) [reset = 0h]

QUEUE_113_D is shown in Figure 16-766 and described in Table 16-780.

**Figure 16-766. QUEUE_113_D Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | DESC_SIZE | | | | |
| R/W-0 | | | R/W-0 | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-780. QUEUE_113_D Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-5 | DESC_PTR | R/W-0 | 0 | Descriptor pointer.<br>It will be read as zero if the queue is empty.<br>It will indicate a<br>32-bit aligned address that points to a descriptor when the queue is not empty. |
| 4-0 | DESC_SIZE | R/W-0 | 0 | Descriptor Size.<br>It is encoded in<br>4-byte increments with values 0 to 31 representing 24 and so on to 148 bytes.<br>This field will return a 0x0 when an empty queue is read.<br>Queue Manager Queue N Registers D To save hardware resources, the queue manager internally stores descriptor size (desc_size) information in four bits.<br>However, register D has five LSBs that specify descriptor size.<br>As a consequence, the value of desc_size that is pushed may not be same as that is read during a pop.<br>The value that is read back is equal to always rounded to an odd number.<br>So, for even values, the value read back is one more than what was written.<br>For odd values, the value read back is same as the value that was written.<br>Note that this<br>5-bit field (desc_size) is unrelated to the code for size of descriptors in a descriptor region.<br>It is just a place holder for a<br>5-bit value that is maintained across the push and pop operations for every descriptor managed by the queue manager. |

### 16.4.7.491 QUEUE_114_A Register (offset = 2720h) [reset = 0h]

QUEUE_114_A is shown in Figure 16-767 and described in Table 16-781.

**Figure 16-767. QUEUE_114_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-781. QUEUE_114_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>This count is incremented by 1 whenever a packet is added to the queue.<br>This count is decremented by 1 whenever a packet is popped from the queue. |

### 16.4.7.492 QUEUE_114_B Register (offset = 2724h) [reset = 0h]

QUEUE_114_B is shown in Figure 16-768 and described in Table 16-782.

**Figure 16-768. QUEUE_114_B Register**

| 31-28 | 27-0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-782. QUEUE_114_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. |

### 16.4.7.493 QUEUE_114_C Register (offset = 2728h) [reset = 0h]

QUEUE_114_C is shown in Figure 16-769 and described in Table 16-783.

**Figure 16-769. QUEUE_114_C Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| HEAD_TAIL | Reserved | | | | | | |
| W-0 | | | | | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| Reserved | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| Reserved | | PACKET_SIZE | | | | | |
| | | R/W-0 | | | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| PACKET_SIZE | | | | | | | |
| R/W-0 | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-783. QUEUE_114_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31 | HEAD_TAIL | W-0 | 0 | Head/Tail Push Control.<br>Set to zero in order to push packet onto tail of queue and set to one in order to push packet onto head of queue. |
| 13-0 | PACKET_SIZE | R/W-0 | 0 | packet_size This field indicates packet size and is assumed to be zero on each packet add unless the value is explicitly overwritten.<br>This field indicates packet size for packet pop operation. |

### 16.4.7.494 QUEUE_114_D Register (offset = 272Ch) [reset = 0h]

QUEUE_114_D is shown in Figure 16-770 and described in Table 16-784.

**Figure 16-770. QUEUE_114_D Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | DESC_SIZE | | | | |
| R/W-0 | | | R/W-0 | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-784. QUEUE_114_D Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-5 | DESC_PTR | R/W-0 | 0 | Descriptor pointer.<br>It will be read as zero if the queue is empty.<br>It will indicate a<br>32-bit aligned address that points to a descriptor when the queue is not empty. |
| 4-0 | DESC_SIZE | R/W-0 | 0 | Descriptor Size.<br>It is encoded in<br>4-byte increments with values 0 to 31 representing 24 and so on to 148 bytes.<br>This field will return a 0x0 when an empty queue is read.<br>Queue Manager Queue N Registers D To save hardware resources, the queue manager internally stores descriptor size (desc_size) information in four bits.<br>However, register D has five LSBs that specify descriptor size.<br>As a consequence, the value of desc_size that is pushed may not be same as that is read during a pop.<br>The value that is read back is equal to always rounded to an odd number.<br>So, for even values, the value read back is one more than what was written.<br>For odd values, the value read back is same as the value that was written.<br>Note that this<br>5-bit field (desc_size) is unrelated to the code for size of descriptors in a descriptor region.<br>It is just a place holder for a<br>5-bit value that is maintained across the push and pop operations for every descriptor managed by the queue manager. |

### 16.4.7.495 QUEUE_115_A Register (offset = 2730h) [reset = 0h]

QUEUE_115_A is shown in Figure 16-771 and described in Table 16-785.

**Figure 16-771. QUEUE_115_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-785. QUEUE_115_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>This count is incremented by 1 whenever a packet is added to the queue.<br>This count is decremented by 1 whenever a packet is popped from the queue. |

### 16.4.7.496 QUEUE_115_B Register (offset = 2734h) [reset = 0h]

QUEUE_115_B is shown in Figure 16-772 and described in Table 16-786.

**Figure 16-772. QUEUE_115_B Register**

| 31-28 | 27-0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-786. QUEUE_115_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. |

### 16.4.7.497 QUEUE_115_C Register (offset = 2738h) [reset = 0h]

QUEUE_115_C is shown in Figure 16-773 and described in Table 16-787.

**Figure 16-773. QUEUE_115_C Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| HEAD_TAIL | Reserved | | | | | | |
| W-0 | | | | | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| Reserved | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| Reserved | | PACKET_SIZE | | | | | |
| | | R/W-0 | | | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| PACKET_SIZE | | | | | | | |
| R/W-0 | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-787. QUEUE_115_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31 | HEAD_TAIL | W-0 | 0 | Head/Tail Push Control.<br>Set to zero in order to push packet onto tail of queue and set to one in order to push packet onto head of queue. |
| 13-0 | PACKET_SIZE | R/W-0 | 0 | packet_size This field indicates packet size and is assumed to be zero on each packet add unless the value is explicitly overwritten. This field indicates packet size for packet pop operation. |

### 16.4.7.498 QUEUE_115_D Register (offset = 273Ch) [reset = 0h]

QUEUE_115_D is shown in Figure 16-774 and described in Table 16-788.

**Figure 16-774. QUEUE_115_D Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |
| **23** | **22** | **21** | **20** | **19** | **18** | **17** | **16** |
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |
| **15** | **14** | **13** | **12** | **11** | **10** | **9** | **8** |
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |
| **7** | **6** | **5** | **4** | **3** | **2** | **1** | **0** |
| DESC_PTR | | | DESC_SIZE | | | | |
| R/W-0 | | | R/W-0 | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-788. QUEUE_115_D Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-5 | DESC_PTR | R/W-0 | 0 | Descriptor pointer.<br>It will be read as zero if the queue is empty.<br>It will indicate a<br>32-bit aligned address that points to a descriptor when the queue is not empty. |
| 4-0 | DESC_SIZE | R/W-0 | 0 | Descriptor Size.<br>It is encoded in<br>4-byte increments with values 0 to 31 representing 24 and so on to 148 bytes.<br>This field will return a 0x0 when an empty queue is read.<br>Queue Manager Queue N Registers D To save hardware resources, the queue manager internally stores descriptor size (desc_size) information in four bits.<br>However, register D has five LSBs that specify descriptor size.<br>As a consequence, the value of desc_size that is pushed may not be same as that is read during a pop.<br>The value that is read back is equal to always rounded to an odd number.<br>So, for even values, the value read back is one more than what was written.<br>For odd values, the value read back is same as the value that was written.<br>Note that this<br>5-bit field (desc_size) is unrelated to the code for size of descriptors in a descriptor region.<br>It is just a place holder for a<br>5-bit value that is maintained across the push and pop operations for every descriptor managed by the queue manager. |

### 16.4.7.499 QUEUE_116_A Register (offset = 2740h) [reset = 0h]

QUEUE_116_A is shown in Figure 16-775 and described in Table 16-789.

**Figure 16-775. QUEUE_116_A Register**

| 31–14 | 13–0 |
| --- | --- |
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-789. QUEUE_116_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
| --- | --- | --- | --- | --- |
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>This count is incremented by 1 whenever a packet is added to the queue.<br>This count is decremented by 1 whenever a packet is popped from the queue. |

### 16.4.7.500 QUEUE_116_B Register (offset = 2744h) [reset = 0h]

QUEUE_116_B is shown in Figure 16-776 and described in Table 16-790.

**Figure 16-776. QUEUE_116_B Register**

| 31–28 | 27–0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-790. QUEUE_116_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. |

### 16.4.7.501 QUEUE_116_C Register (offset = 2748h) [reset = 0h]

QUEUE_116_C is shown in Figure 16-777 and described in Table 16-791.

**Figure 16-777. QUEUE_116_C Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| HEAD_TAIL | Reserved | | | | | | |
| W-0 | | | | | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| Reserved | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| Reserved | | PACKET_SIZE | | | | | |
| | | R/W-0 | | | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| PACKET_SIZE | | | | | | | |
| R/W-0 | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-791. QUEUE_116_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31 | HEAD_TAIL | W-0 | 0 | Head/Tail Push Control.<br>Set to zero in order to push packet onto tail of queue and set to one in order to push packet onto head of queue. |
| 13-0 | PACKET_SIZE | R/W-0 | 0 | packet_size This field indicates packet size and is assumed to be zero on each packet add unless the value is explicitly overwritten. This field indicates packet size for packet pop operation. |

### 16.4.7.502 QUEUE_116_D Register (offset = 274Ch) [reset = 0h]

QUEUE_116_D is shown in Figure 16-778 and described in Table 16-792.

**Figure 16-778. QUEUE_116_D Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | DESC_SIZE | | | | |
| R/W-0 | | | R/W-0 | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-792. QUEUE_116_D Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-5 | DESC_PTR | R/W-0 | 0 | Descriptor pointer.<br>It will be read as zero if the queue is empty.<br>It will indicate a<br>32-bit aligned address that points to a descriptor when the queue is not empty. |
| 4-0 | DESC_SIZE | R/W-0 | 0 | Descriptor Size.<br>It is encoded in<br>4-byte increments with values 0 to 31 representing 24 and so on to 148 bytes.<br>This field will return a 0x0 when an empty queue is read.<br>Queue Manager Queue N Registers D To save hardware resources, the queue manager internally stores descriptor size (desc_size) information in four bits.<br>However, register D has five LSBs that specify descriptor size.<br>As a consequence, the value of desc_size that is pushed may not be same as that is read during a pop.<br>The value that is read back is equal to always rounded to an odd number.<br>So, for even values, the value read back is one more than what was written.<br>For odd values, the value read back is same as the value that was written.<br>Note that this<br>5-bit field (desc_size) is unrelated to the code for size of descriptors in a descriptor region.<br>It is just a place holder for a<br>5-bit value that is maintained across the push and pop operations for every descriptor managed by the queue manager. |

### 16.4.7.503 QUEUE_117_A Register (offset = 2750h) [reset = 0h]

QUEUE_117_A is shown in Figure 16-779 and described in Table 16-793.

**Figure 16-779. QUEUE_117_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-793. QUEUE_117_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>This count is incremented by 1 whenever a packet is added to the queue.<br>This count is decremented by 1 whenever a packet is popped from the queue. |

### 16.4.7.504 QUEUE_117_B Register (offset = 2754h) [reset = 0h]

QUEUE_117_B is shown in Figure 16-780 and described in Table 16-794.

**Figure 16-780. QUEUE_117_B Register**

| 31–28 | 27–0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-794. QUEUE_117_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. |

### 16.4.7.505 QUEUE_117_C Register (offset = 2758h) [reset = 0h]

QUEUE_117_C is shown in Figure 16-781 and described in Table 16-795.

**Figure 16-781. QUEUE_117_C Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| HEAD_TAIL | Reserved | | | | | | |
| W-0 | | | | | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| Reserved | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| Reserved | | PACKET_SIZE | | | | | |
| | | R/W-0 | | | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| PACKET_SIZE | | | | | | | |
| R/W-0 | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-795. QUEUE_117_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31 | HEAD_TAIL | W-0 | 0 | Head/Tail Push Control.<br>Set to zero in order to push packet onto tail of queue and set to one in order to push packet onto head of queue. |
| 13-0 | PACKET_SIZE | R/W-0 | 0 | packet_size This field indicates packet size and is assumed to be zero on each packet add unless the value is explicitly overwritten. This field indicates packet size for packet pop operation. |

### 16.4.7.506 QUEUE_117_D Register (offset = 275Ch) [reset = 0h]

QUEUE_117_D is shown in Figure 16-782 and described in Table 16-796.

**Figure 16-782. QUEUE_117_D Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |
| **23** | **22** | **21** | **20** | **19** | **18** | **17** | **16** |
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |
| **15** | **14** | **13** | **12** | **11** | **10** | **9** | **8** |
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |
| **7** | **6** | **5** | **4** | **3** | **2** | **1** | **0** |
| DESC_PTR | | | DESC_SIZE | | | | |
| R/W-0 | | | R/W-0 | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-796. QUEUE_117_D Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-5 | DESC_PTR | R/W-0 | 0 | Descriptor pointer.<br>It will be read as zero if the queue is empty.<br>It will indicate a<br>32-bit aligned address that points to a descriptor when the queue is not empty. |
| 4-0 | DESC_SIZE | R/W-0 | 0 | Descriptor Size.<br>It is encoded in<br>4-byte increments with values 0 to 31 representing 24 and so on to 148 bytes.<br>This field will return a 0x0 when an empty queue is read.<br>Queue Manager Queue N Registers D To save hardware resources, the queue manager internally stores descriptor size (desc_size) information in four bits.<br>However, register D has five LSBs that specify descriptor size.<br>As a consequence, the value of desc_size that is pushed may not be same as that is read during a pop.<br>The value that is read back is equal to always rounded to an odd number.<br>So, for even values, the value read back is one more than what was written.<br>For odd values, the value read back is same as the value that was written.<br>Note that this<br>5-bit field (desc_size) is unrelated to the code for size of descriptors in a descriptor region.<br>It is just a place holder for a<br>5-bit value that is maintained across the push and pop operations for every descriptor managed by the queue manager. |

### 16.4.7.507 QUEUE_118_A Register (offset = 2760h) [reset = 0h]

QUEUE_118_A is shown in Figure 16-783 and described in Table 16-797.

**Figure 16-783. QUEUE_118_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-797. QUEUE_118_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>This count is incremented by 1 whenever a packet is added to the queue.<br>This count is decremented by 1 whenever a packet is popped from the queue. |

### 16.4.7.508 QUEUE_118_B Register (offset = 2764h) [reset = 0h]

QUEUE_118_B is shown in Figure 16-784 and described in Table 16-798.

**Figure 16-784. QUEUE_118_B Register**

| 31–28 | 27–0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-798. QUEUE_118_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. |

### 16.4.7.509 QUEUE_118_C Register (offset = 2768h) [reset = 0h]

QUEUE_118_C is shown in Figure 16-785 and described in Table 16-799.

**Figure 16-785. QUEUE_118_C Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| HEAD_TAIL | Reserved | | | | | | |
| W-0 | | | | | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| Reserved | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| Reserved | | PACKET_SIZE | | | | | |
| | | R/W-0 | | | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| PACKET_SIZE | | | | | | | |
| R/W-0 | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-799. QUEUE_118_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31 | HEAD_TAIL | W-0 | 0 | Head/Tail Push Control.<br>Set to zero in order to push packet onto tail of queue and set to one in order to push packet onto head of queue. |
| 13-0 | PACKET_SIZE | R/W-0 | 0 | packet_size This field indicates packet size and is assumed to be zero on each packet add unless the value is explicitly overwritten. This field indicates packet size for packet pop operation. |

### 16.4.7.510 QUEUE_118_D Register (offset = 276Ch) [reset = 0h]

QUEUE_118_D is shown in Figure 16-786 and described in Table 16-800.

**Figure 16-786. QUEUE_118_D Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |
| **23** | **22** | **21** | **20** | **19** | **18** | **17** | **16** |
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |
| **15** | **14** | **13** | **12** | **11** | **10** | **9** | **8** |
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |
| **7** | **6** | **5** | **4** | **3** | **2** | **1** | **0** |
| DESC_PTR | | | DESC_SIZE | | | | |
| R/W-0 | | | R/W-0 | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-800. QUEUE_118_D Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-5 | DESC_PTR | R/W-0 | 0 | Descriptor pointer.<br>It will be read as zero if the queue is empty.<br>It will indicate a<br>32-bit aligned address that points to a descriptor when the queue is not empty. |
| 4-0 | DESC_SIZE | R/W-0 | 0 | Descriptor Size.<br>It is encoded in<br>4-byte increments with values 0 to 31 representing 24 and so on to 148 bytes.<br>This field will return a 0x0 when an empty queue is read.<br>Queue Manager Queue N Registers D To save hardware resources, the queue manager internally stores descriptor size (desc_size) information in four bits.<br>However, register D has five LSBs that specify descriptor size.<br>As a consequence, the value of desc_size that is pushed may not be same as that is read during a pop.<br>The value that is read back is equal to always rounded to an odd number.<br>So, for even values, the value read back is one more than what was written.<br>For odd values, the value read back is same as the value that was written.<br>Note that this<br>5-bit field (desc_size) is unrelated to the code for size of descriptors in a descriptor region.<br>It is just a place holder for a<br>5-bit value that is maintained across the push and pop operations for every descriptor managed by the queue manager. |

### 16.4.7.511 QUEUE_119_A Register (offset = 2770h) [reset = 0h]

QUEUE_119_A is shown in Figure 16-787 and described in Table 16-801.

**Figure 16-787. QUEUE_119_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-801. QUEUE_119_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>This count is incremented by 1 whenever a packet is added to the queue.<br>This count is decremented by 1 whenever a packet is popped from the queue. |

### 16.4.7.512 QUEUE_119_B Register (offset = 2774h) [reset = 0h]

QUEUE_119_B is shown in Figure 16-788 and described in Table 16-802.

**Figure 16-788. QUEUE_119_B Register**

| 31-28 | 27-0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-802. QUEUE_119_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. |

### 16.4.7.513 QUEUE_119_C Register (offset = 2778h) [reset = 0h]

QUEUE_119_C is shown in Figure 16-789 and described in Table 16-803.

**Figure 16-789. QUEUE_119_C Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| HEAD_TAIL | Reserved | | | | | | |
| W-0 | | | | | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| Reserved | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| Reserved | | PACKET_SIZE | | | | | |
| | | R/W-0 | | | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| PACKET_SIZE | | | | | | | |
| R/W-0 | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-803. QUEUE_119_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31 | HEAD_TAIL | W-0 | 0 | Head/Tail Push Control.<br>Set to zero in order to push packet onto tail of queue and set to one in order to push packet onto head of queue. |
| 13-0 | PACKET_SIZE | R/W-0 | 0 | packet_size This field indicates packet size and is assumed to be zero on each packet add unless the value is explicitly overwritten. This field indicates packet size for packet pop operation. |

### 16.4.7.514 QUEUE_119_D Register (offset = 277Ch) [reset = 0h]

QUEUE_119_D is shown in Figure 16-790 and described in Table 16-804.

**Figure 16-790. QUEUE_119_D Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |
| **23** | **22** | **21** | **20** | **19** | **18** | **17** | **16** |
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |
| **15** | **14** | **13** | **12** | **11** | **10** | **9** | **8** |
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |
| **7** | **6** | **5** | **4** | **3** | **2** | **1** | **0** |
| DESC_PTR | | | DESC_SIZE | | | | |
| R/W-0 | | | R/W-0 | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-804. QUEUE_119_D Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-5 | DESC_PTR | R/W-0 | 0 | Descriptor pointer.<br>It will be read as zero if the queue is empty.<br>It will indicate a<br>32-bit aligned address that points to a descriptor when the queue is not empty. |
| 4-0 | DESC_SIZE | R/W-0 | 0 | Descriptor Size.<br>It is encoded in<br>4-byte increments with values 0 to 31 representing 24 and so on to 148 bytes.<br>This field will return a 0x0 when an empty queue is read.<br>Queue Manager Queue N Registers D To save hardware resources, the queue manager internally stores descriptor size (desc_size) information in four bits.<br>However, register D has five LSBs that specify descriptor size.<br>As a consequence, the value of desc_size that is pushed may not be same as that is read during a pop.<br>The value that is read back is equal to always rounded to an odd number.<br>So, for even values, the value read back is one more than what was written.<br>For odd values, the value read back is same as the value that was written.<br>Note that this<br>5-bit field (desc_size) is unrelated to the code for size of descriptors in a descriptor region.<br>It is just a place holder for a<br>5-bit value that is maintained across the push and pop operations for every descriptor managed by the queue manager. |

### 16.4.7.515 QUEUE_120_A Register (offset = 2780h) [reset = 0h]

QUEUE_120_A is shown in Figure 16-791 and described in Table 16-805.

**Figure 16-791. QUEUE_120_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-805. QUEUE_120_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>This count is incremented by 1 whenever a packet is added to the queue.<br>This count is decremented by 1 whenever a packet is popped from the queue. |

### 16.4.7.516 QUEUE_120_B Register (offset = 2784h) [reset = 0h]

QUEUE_120_B is shown in Figure 16-792 and described in Table 16-806.

**Figure 16-792. QUEUE_120_B Register**

| 31–28 | 27–0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-806. QUEUE_120_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. |

### 16.4.7.517 QUEUE_120_C Register (offset = 2788h) [reset = 0h]

QUEUE_120_C is shown in Figure 16-793 and described in Table 16-807.

**Figure 16-793. QUEUE_120_C Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| HEAD_TAIL | Reserved | | | | | | |
| W-0 | | | | | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| Reserved | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| Reserved | | PACKET_SIZE | | | | | |
| | | R/W-0 | | | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| PACKET_SIZE | | | | | | | |
| R/W-0 | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-807. QUEUE_120_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31 | HEAD_TAIL | W-0 | 0 | Head/Tail Push Control.<br>Set to zero in order to push packet onto tail of queue and set to one in order to push packet onto head of queue. |
| 13-0 | PACKET_SIZE | R/W-0 | 0 | packet_size This field indicates packet size and is assumed to be zero on each packet add unless the value is explicitly overwritten. This field indicates packet size for packet pop operation. |

### 16.4.7.518 QUEUE_120_D Register (offset = 278Ch) [reset = 0h]

QUEUE_120_D is shown in Figure 16-794 and described in Table 16-808.

**Figure 16-794. QUEUE_120_D Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |
| **23** | **22** | **21** | **20** | **19** | **18** | **17** | **16** |
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |
| **15** | **14** | **13** | **12** | **11** | **10** | **9** | **8** |
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |
| **7** | **6** | **5** | **4** | **3** | **2** | **1** | **0** |
| DESC_PTR | | | DESC_SIZE | | | | |
| R/W-0 | | | R/W-0 | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-808. QUEUE_120_D Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-5 | DESC_PTR | R/W-0 | 0 | Descriptor pointer.<br>It will be read as zero if the queue is empty.<br>It will indicate a<br>32-bit aligned address that points to a descriptor when the queue is not empty. |
| 4-0 | DESC_SIZE | R/W-0 | 0 | Descriptor Size.<br>It is encoded in<br>4-byte increments with values 0 to 31 representing 24 and so on to 148 bytes.<br>This field will return a 0x0 when an empty queue is read.<br>Queue Manager Queue N Registers D To save hardware resources, the queue manager internally stores descriptor size (desc_size) information in four bits.<br>However, register D has five LSBs that specify descriptor size.<br>As a consequence, the value of desc_size that is pushed may not be same as that is read during a pop.<br>The value that is read back is equal to always rounded to an odd number.<br>So, for even values, the value read back is one more than what was written.<br>For odd values, the value read back is same as the value that was written.<br>Note that this<br>5-bit field (desc_size) is unrelated to the code for size of descriptors in a descriptor region.<br>It is just a place holder for a<br>5-bit value that is maintained across the push and pop operations for every descriptor managed by the queue manager. |

### 16.4.7.519 QUEUE_121_A Register (offset = 2790h) [reset = 0h]

QUEUE_121_A is shown in Figure 16-795 and described in Table 16-809.

**Figure 16-795. QUEUE_121_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-809. QUEUE_121_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>This count is incremented by 1 whenever a packet is added to the queue.<br>This count is decremented by 1 whenever a packet is popped from the queue. |

### 16.4.7.520 QUEUE_121_B Register (offset = 2794h) [reset = 0h]

QUEUE_121_B is shown in Figure 16-796 and described in Table 16-810.

**Figure 16-796. QUEUE_121_B Register**

| 31-28 | 27-0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-810. QUEUE_121_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. |

### 16.4.7.521 QUEUE_121_C Register (offset = 2798h) [reset = 0h]

QUEUE_121_C is shown in Figure 16-797 and described in Table 16-811.

**Figure 16-797. QUEUE_121_C Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| HEAD_TAIL | Reserved | | | | | | |
| W-0 | | | | | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| Reserved | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| Reserved | | PACKET_SIZE | | | | | |
| | | R/W-0 | | | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| PACKET_SIZE | | | | | | | |
| R/W-0 | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-811. QUEUE_121_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31 | HEAD_TAIL | W-0 | 0 | Head/Tail Push Control.<br>Set to zero in order to push packet onto tail of queue and set to one in order to push packet onto head of queue. |
| 13-0 | PACKET_SIZE | R/W-0 | 0 | packet_size This field indicates packet size and is assumed to be zero on each packet add unless the value is explicitly overwritten. This field indicates packet size for packet pop operation. |

### 16.4.7.522 QUEUE_121_D Register (offset = 279Ch) [reset = 0h]

QUEUE_121_D is shown in Figure 16-798 and described in Table 16-812.

**Figure 16-798. QUEUE_121_D Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | DESC_SIZE | | | | |
| R/W-0 | | | R/W-0 | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-812. QUEUE_121_D Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-5 | DESC_PTR | R/W-0 | 0 | Descriptor pointer.<br>It will be read as zero if the queue is empty.<br>It will indicate a<br>32-bit aligned address that points to a descriptor when the queue is not empty. |
| 4-0 | DESC_SIZE | R/W-0 | 0 | Descriptor Size.<br>It is encoded in<br>4-byte increments with values 0 to 31 representing 24 and so on to 148 bytes.<br>This field will return a 0x0 when an empty queue is read.<br>Queue Manager Queue N Registers D To save hardware resources, the queue manager internally stores descriptor size (desc_size) information in four bits.<br>However, register D has five LSBs that specify descriptor size.<br>As a consequence, the value of desc_size that is pushed may not be same as that is read during a pop.<br>The value that is read back is equal to always rounded to an odd number.<br>So, for even values, the value read back is one more than what was written.<br>For odd values, the value read back is same as the value that was written.<br>Note that this<br>5-bit field (desc_size) is unrelated to the code for size of descriptors in a descriptor region.<br>It is just a place holder for a<br>5-bit value that is maintained across the push and pop operations for every descriptor managed by the queue manager. |

### 16.4.7.523 QUEUE_122_A Register (offset = 27A0h) [reset = 0h]

QUEUE_122_A is shown in Figure 16-799 and described in Table 16-813.

**Figure 16-799. QUEUE_122_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-813. QUEUE_122_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>This count is incremented by 1 whenever a packet is added to the queue.<br>This count is decremented by 1 whenever a packet is popped from the queue. |

### 16.4.7.524 QUEUE_122_B Register (offset = 27A4h) [reset = 0h]

QUEUE_122_B is shown in Figure 16-800 and described in Table 16-814.

**Figure 16-800. QUEUE_122_B Register**

| 31–28 | 27–0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-814. QUEUE_122_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. |

### 16.4.7.525 QUEUE_122_C Register (offset = 27A8h) [reset = 0h]

QUEUE_122_C is shown in Figure 16-801 and described in Table 16-815.

**Figure 16-801. QUEUE_122_C Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| HEAD_TAIL | Reserved | | | | | | |
| W-0 | | | | | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| Reserved | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| Reserved | | PACKET_SIZE | | | | | |
| | | R/W-0 | | | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| PACKET_SIZE | | | | | | | |
| R/W-0 | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-815. QUEUE_122_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31 | HEAD_TAIL | W-0 | 0 | Head/Tail Push Control.<br>Set to zero in order to push packet onto tail of queue and set to one in order to push packet onto head of queue. |
| 13-0 | PACKET_SIZE | R/W-0 | 0 | packet_size This field indicates packet size and is assumed to be zero on each packet add unless the value is explicitly overwritten. This field indicates packet size for packet pop operation. |

### 16.4.7.526 QUEUE_122_D Register (offset = 27ACh) [reset = 0h]

QUEUE_122_D is shown in Figure 16-802 and described in Table 16-816.

**Figure 16-802. QUEUE_122_D Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | DESC_SIZE | | | | |
| R/W-0 | | | R/W-0 | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-816. QUEUE_122_D Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-5 | DESC_PTR | R/W-0 | 0 | Descriptor pointer.<br>It will be read as zero if the queue is empty.<br>It will indicate a<br>32-bit aligned address that points to a descriptor when the queue is not empty. |
| 4-0 | DESC_SIZE | R/W-0 | 0 | Descriptor Size.<br>It is encoded in<br>4-byte increments with values 0 to 31 representing 24 and so on to 148 bytes.<br>This field will return a 0x0 when an empty queue is read.<br>Queue Manager Queue N Registers D To save hardware resources, the queue manager internally stores descriptor size (desc_size) information in four bits.<br>However, register D has five LSBs that specify descriptor size.<br>As a consequence, the value of desc_size that is pushed may not be same as that is read during a pop.<br>The value that is read back is equal to always rounded to an odd number.<br>So, for even values, the value read back is one more than what was written.<br>For odd values, the value read back is same as the value that was written.<br>Note that this<br>5-bit field (desc_size) is unrelated to the code for size of descriptors in a descriptor region.<br>It is just a place holder for a<br>5-bit value that is maintained across the push and pop operations for every descriptor managed by the queue manager. |

### 16.4.7.527 QUEUE_123_A Register (offset = 27B0h) [reset = 0h]

QUEUE_123_A is shown in Figure 16-803 and described in Table 16-817.

**Figure 16-803. QUEUE_123_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-817. QUEUE_123_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>This count is incremented by 1 whenever a packet is added to the queue.<br>This count is decremented by 1 whenever a packet is popped from the queue. |

### 16.4.7.528 QUEUE_123_B Register (offset = 27B4h) [reset = 0h]

QUEUE_123_B is shown in Figure 16-804 and described in Table 16-818.

**Figure 16-804. QUEUE_123_B Register**

| 31–28 | 27–0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-818. QUEUE_123_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. |

### 16.4.7.529 QUEUE_123_C Register (offset = 27B8h) [reset = 0h]

QUEUE_123_C is shown in Figure 16-805 and described in Table 16-819.

**Figure 16-805. QUEUE_123_C Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| HEAD_TAIL | Reserved | | | | | | |
| W-0 | | | | | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| Reserved | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| Reserved | | PACKET_SIZE | | | | | |
| | | R/W-0 | | | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| PACKET_SIZE | | | | | | | |
| R/W-0 | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-819. QUEUE_123_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31 | HEAD_TAIL | W-0 | 0 | Head/Tail Push Control.<br>Set to zero in order to push packet onto tail of queue and set to one in order to push packet onto head of queue. |
| 13-0 | PACKET_SIZE | R/W-0 | 0 | packet_size This field indicates packet size and is assumed to be zero on each packet add unless the value is explicitly overwritten. This field indicates packet size for packet pop operation. |

### 16.4.7.530 QUEUE_123_D Register (offset = 27BCh) [reset = 0h]

QUEUE_123_D is shown in Figure 16-806 and described in Table 16-820.

**Figure 16-806. QUEUE_123_D Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | DESC_SIZE | | | | |
| R/W-0 | | | R/W-0 | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-820. QUEUE_123_D Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-5 | DESC_PTR | R/W-0 | 0 | Descriptor pointer.<br>It will be read as zero if the queue is empty.<br>It will indicate a<br>32-bit aligned address that points to a descriptor when the queue is not empty. |
| 4-0 | DESC_SIZE | R/W-0 | 0 | Descriptor Size.<br>It is encoded in<br>4-byte increments with values 0 to 31 representing 24 and so on to 148 bytes.<br>This field will return a 0x0 when an empty queue is read.<br>Queue Manager Queue N Registers D To save hardware resources, the queue manager internally stores descriptor size (desc_size) information in four bits.<br>However, register D has five LSBs that specify descriptor size.<br>As a consequence, the value of desc_size that is pushed may not be same as that is read during a pop.<br>The value that is read back is equal to always rounded to an odd number.<br>So, for even values, the value read back is one more than what was written.<br>For odd values, the value read back is same as the value that was written.<br>Note that this<br>5-bit field (desc_size) is unrelated to the code for size of descriptors in a descriptor region.<br>It is just a place holder for a<br>5-bit value that is maintained across the push and pop operations for every descriptor managed by the queue manager. |

### 16.4.7.531 QUEUE_124_A Register (offset = 27C0h) [reset = 0h]

QUEUE_124_A is shown in Figure 16-807 and described in Table 16-821.

**Figure 16-807. QUEUE_124_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-821. QUEUE_124_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>This count is incremented by 1 whenever a packet is added to the queue.<br>This count is decremented by 1 whenever a packet is popped from the queue. |

### 16.4.7.532 QUEUE_124_B Register (offset = 27C4h) [reset = 0h]

QUEUE_124_B is shown in Figure 16-808 and described in Table 16-822.

**Figure 16-808. QUEUE_124_B Register**

| 31–28 | 27–0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-822. QUEUE_124_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. |

### 16.4.7.533 QUEUE_124_C Register (offset = 27C8h) [reset = 0h]

QUEUE_124_C is shown in Figure 16-809 and described in Table 16-823.

**Figure 16-809. QUEUE_124_C Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| HEAD_TAIL | Reserved | | | | | | |
| W-0 | | | | | | | |
| **23** | **22** | **21** | **20** | **19** | **18** | **17** | **16** |
| Reserved | | | | | | | |
| **15** | **14** | **13** | **12** | **11** | **10** | **9** | **8** |
| Reserved | | PACKET_SIZE | | | | | |
| | | R/W-0 | | | | | |
| **7** | **6** | **5** | **4** | **3** | **2** | **1** | **0** |
| PACKET_SIZE | | | | | | | |
| R/W-0 | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-823. QUEUE_124_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31 | HEAD_TAIL | W-0 | 0 | Head/Tail Push Control.<br>Set to zero in order to push packet onto tail of queue and set to one in order to push packet onto head of queue. |
| 13-0 | PACKET_SIZE | R/W-0 | 0 | packet_size This field indicates packet size and is assumed to be zero on each packet add unless the value is explicitly overwritten. This field indicates packet size for packet pop operation. |

### 16.4.7.534 QUEUE_124_D Register (offset = 27CCh) [reset = 0h]

QUEUE_124_D is shown in Figure 16-810 and described in Table 16-824.

**Figure 16-810. QUEUE_124_D Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |
| **23** | **22** | **21** | **20** | **19** | **18** | **17** | **16** |
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |
| **15** | **14** | **13** | **12** | **11** | **10** | **9** | **8** |
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |
| **7** | **6** | **5** | **4** | **3** | **2** | **1** | **0** |
| DESC_PTR | | | DESC_SIZE | | | | |
| R/W-0 | | | R/W-0 | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-824. QUEUE_124_D Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-5 | DESC_PTR | R/W-0 | 0 | Descriptor pointer.<br>It will be read as zero if the queue is empty.<br>It will indicate a<br>32-bit aligned address that points to a descriptor when the queue is not empty. |
| 4-0 | DESC_SIZE | R/W-0 | 0 | Descriptor Size.<br>It is encoded in<br>4-byte increments with values 0 to 31 representing 24 and so on to 148 bytes.<br>This field will return a 0x0 when an empty queue is read.<br>Queue Manager Queue N Registers D To save hardware resources, the queue manager internally stores descriptor size (desc_size) information in four bits.<br>However, register D has five LSBs that specify descriptor size.<br>As a consequence, the value of desc_size that is pushed may not be same as that is read during a pop.<br>The value that is read back is equal to always rounded to an odd number.<br>So, for even values, the value read back is one more than what was written.<br>For odd values, the value read back is same as the value that was written.<br>Note that this<br>5-bit field (desc_size) is unrelated to the code for size of descriptors in a descriptor region.<br>It is just a place holder for a<br>5-bit value that is maintained across the push and pop operations for every descriptor managed by the queue manager. |

### 16.4.7.535 QUEUE_125_A Register (offset = 27D0h) [reset = 0h]

QUEUE_125_A is shown in Figure 16-811 and described in Table 16-825.

**Figure 16-811. QUEUE_125_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-825. QUEUE_125_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>This count is incremented by 1 whenever a packet is added to the queue.<br>This count is decremented by 1 whenever a packet is popped from the queue. |

### 16.4.7.536 QUEUE_125_B Register (offset = 27D4h) [reset = 0h]

QUEUE_125_B is shown in Figure 16-812 and described in Table 16-826.

**Figure 16-812. QUEUE_125_B Register**

| 31–28 | 27–0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-826. QUEUE_125_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. |

### 16.4.7.537 QUEUE_125_C Register (offset = 27D8h) [reset = 0h]

QUEUE_125_C is shown in Figure 16-813 and described in Table 16-827.

**Figure 16-813. QUEUE_125_C Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| HEAD_TAIL | Reserved | | | | | | |
| W-0 | | | | | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| Reserved | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| Reserved | | PACKET_SIZE | | | | | |
| | | R/W-0 | | | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| PACKET_SIZE | | | | | | | |
| R/W-0 | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-827. QUEUE_125_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31 | HEAD_TAIL | W-0 | 0 | Head/Tail Push Control.<br>Set to zero in order to push packet onto tail of queue and set to one in order to push packet onto head of queue. |
| 13-0 | PACKET_SIZE | R/W-0 | 0 | packet_size This field indicates packet size and is assumed to be zero on each packet add unless the value is explicitly overwritten. This field indicates packet size for packet pop operation. |

### 16.4.7.538 QUEUE_125_D Register (offset = 27DCh) [reset = 0h]

QUEUE_125_D is shown in Figure 16-814 and described in Table 16-828.

**Figure 16-814. QUEUE_125_D Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | DESC_SIZE | | | | |
| R/W-0 | | | R/W-0 | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-828. QUEUE_125_D Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-5 | DESC_PTR | R/W-0 | 0 | Descriptor pointer.<br>It will be read as zero if the queue is empty.<br>It will indicate a<br>32-bit aligned address that points to a descriptor when the queue is not empty. |
| 4-0 | DESC_SIZE | R/W-0 | 0 | Descriptor Size.<br>It is encoded in<br>4-byte increments with values 0 to 31 representing 24 and so on to 148 bytes.<br>This field will return a 0x0 when an empty queue is read.<br>Queue Manager Queue N Registers D To save hardware resources, the queue manager internally stores descriptor size (desc_size) information in four bits.<br>However, register D has five LSBs that specify descriptor size.<br>As a consequence, the value of desc_size that is pushed may not be same as that is read during a pop.<br>The value that is read back is equal to always rounded to an odd number.<br>So, for even values, the value read back is one more than what was written.<br>For odd values, the value read back is same as the value that was written.<br>Note that this<br>5-bit field (desc_size) is unrelated to the code for size of descriptors in a descriptor region.<br>It is just a place holder for a<br>5-bit value that is maintained across the push and pop operations for every descriptor managed by the queue manager. |

### 16.4.7.539 QUEUE_126_A Register (offset = 27E0h) [reset = 0h]

QUEUE_126_A is shown in Figure 16-815 and described in Table 16-829.

**Figure 16-815. QUEUE_126_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-829. QUEUE_126_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>This count is incremented by 1 whenever a packet is added to the queue.<br>This count is decremented by 1 whenever a packet is popped from the queue. |

### 16.4.7.540 QUEUE_126_B Register (offset = 27E4h) [reset = 0h]

QUEUE_126_B is shown in Figure 16-816 and described in Table 16-830.

**Figure 16-816. QUEUE_126_B Register**

| 31-28 | 27-0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-830. QUEUE_126_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. |

### 16.4.7.541 QUEUE_126_C Register (offset = 27E8h) [reset = 0h]

QUEUE_126_C is shown in Figure 16-817 and described in Table 16-831.

**Figure 16-817. QUEUE_126_C Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| HEAD_TAIL | Reserved | | | | | | |
| W-0 | | | | | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| Reserved | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| Reserved | | PACKET_SIZE | | | | | |
| | | R/W-0 | | | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| PACKET_SIZE | | | | | | | |
| R/W-0 | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-831. QUEUE_126_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31 | HEAD_TAIL | W-0 | 0 | Head/Tail Push Control.<br>Set to zero in order to push packet onto tail of queue and set to one in order to push packet onto head of queue. |
| 13-0 | PACKET_SIZE | R/W-0 | 0 | packet_size This field indicates packet size and is assumed to be zero on each packet add unless the value is explicitly overwritten. This field indicates packet size for packet pop operation. |

### 16.4.7.542 QUEUE_126_D Register (offset = 27ECh) [reset = 0h]

QUEUE_126_D is shown in Figure 16-818 and described in Table 16-832.

**Figure 16-818. QUEUE_126_D Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | DESC_SIZE | | | | |
| R/W-0 | | | R/W-0 | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-832. QUEUE_126_D Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-5 | DESC_PTR | R/W-0 | 0 | Descriptor pointer.<br>It will be read as zero if the queue is empty.<br>It will indicate a<br>32-bit aligned address that points to a descriptor when the queue is not empty. |
| 4-0 | DESC_SIZE | R/W-0 | 0 | Descriptor Size.<br>It is encoded in<br>4-byte increments with values 0 to 31 representing 24 and so on to 148 bytes.<br>This field will return a 0x0 when an empty queue is read.<br>Queue Manager Queue N Registers D To save hardware resources, the queue manager internally stores descriptor size (desc_size) information in four bits.<br>However, register D has five LSBs that specify descriptor size.<br>As a consequence, the value of desc_size that is pushed may not be same as that is read during a pop.<br>The value that is read back is equal to always rounded to an odd number.<br>So, for even values, the value read back is one more than what was written.<br>For odd values, the value read back is same as the value that was written.<br>Note that this<br>5-bit field (desc_size) is unrelated to the code for size of descriptors in a descriptor region.<br>It is just a place holder for a<br>5-bit value that is maintained across the push and pop operations for every descriptor managed by the queue manager. |

### 16.4.7.543 QUEUE_127_A Register (offset = 27F0h) [reset = 0h]

QUEUE_127_A is shown in Figure 16-819 and described in Table 16-833.

**Figure 16-819. QUEUE_127_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-833. QUEUE_127_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>This count is incremented by 1 whenever a packet is added to the queue.<br>This count is decremented by 1 whenever a packet is popped from the queue. |

### 16.4.7.544 QUEUE_127_B Register (offset = 27F4h) [reset = 0h]

QUEUE_127_B is shown in Figure 16-820 and described in Table 16-834.

**Figure 16-820. QUEUE_127_B Register**

| 31-28 | 27-0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-834. QUEUE_127_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. |

### 16.4.7.545 QUEUE_127_C Register (offset = 27F8h) [reset = 0h]

QUEUE_127_C is shown in Figure 16-821 and described in Table 16-835.

**Figure 16-821. QUEUE_127_C Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| HEAD_TAIL | Reserved | | | | | | |
| W-0 | | | | | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| Reserved | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| Reserved | | PACKET_SIZE | | | | | |
| | | R/W-0 | | | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| PACKET_SIZE | | | | | | | |
| R/W-0 | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-835. QUEUE_127_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31 | HEAD_TAIL | W-0 | 0 | Head/Tail Push Control.<br>Set to zero in order to push packet onto tail of queue and set to one in order to push packet onto head of queue. |
| 13-0 | PACKET_SIZE | R/W-0 | 0 | packet_size This field indicates packet size and is assumed to be zero on each packet add unless the value is explicitly overwritten.<br>This field indicates packet size for packet pop operation. |

### 16.4.7.546 QUEUE_127_D Register (offset = 27FCh) [reset = 0h]

QUEUE_127_D is shown in Figure 16-822 and described in Table 16-836.

**Figure 16-822. QUEUE_127_D Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | DESC_SIZE | | | | |
| R/W-0 | | | R/W-0 | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-836. QUEUE_127_D Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-5 | DESC_PTR | R/W-0 | 0 | Descriptor pointer.<br>It will be read as zero if the queue is empty.<br>It will indicate a<br>32-bit aligned address that points to a descriptor when the queue is not empty. |
| 4-0 | DESC_SIZE | R/W-0 | 0 | Descriptor Size.<br>It is encoded in<br>4-byte increments with values 0 to 31 representing 24 and so on to 148 bytes.<br>This field will return a 0x0 when an empty queue is read.<br>Queue Manager Queue N Registers D To save hardware resources, the queue manager internally stores descriptor size (desc_size) information in four bits.<br>However, register D has five LSBs that specify descriptor size.<br>As a consequence, the value of desc_size that is pushed may not be same as that is read during a pop.<br>The value that is read back is equal to always rounded to an odd number.<br>So, for even values, the value read back is one more than what was written.<br>For odd values, the value read back is same as the value that was written.<br>Note that this<br>5-bit field (desc_size) is unrelated to the code for size of descriptors in a descriptor region.<br>It is just a place holder for a<br>5-bit value that is maintained across the push and pop operations for every descriptor managed by the queue manager. |

### 16.4.7.547 QUEUE_128_A Register (offset = 2800h) [reset = 0h]

QUEUE_128_A is shown in Figure 16-823 and described in Table 16-837.

**Figure 16-823. QUEUE_128_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-837. QUEUE_128_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>This count is incremented by 1 whenever a packet is added to the queue.<br>This count is decremented by 1 whenever a packet is popped from the queue. |

### 16.4.7.548 QUEUE_128_B Register (offset = 2804h) [reset = 0h]

QUEUE_128_B is shown in Figure 16-824 and described in Table 16-838.

**Figure 16-824. QUEUE_128_B Register**

| 31-28 | 27-0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-838. QUEUE_128_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. |

### 16.4.7.549 QUEUE_128_C Register (offset = 2808h) [reset = 0h]

QUEUE_128_C is shown in Figure 16-825 and described in Table 16-839.

**Figure 16-825. QUEUE_128_C Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| HEAD_TAIL | Reserved | | | | | | |
| W-0 | | | | | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| Reserved | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| Reserved | | PACKET_SIZE | | | | | |
| | | R/W-0 | | | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| PACKET_SIZE | | | | | | | |
| R/W-0 | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-839. QUEUE_128_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31 | HEAD_TAIL | W-0 | 0 | Head/Tail Push Control.<br>Set to zero in order to push packet onto tail of queue and set to one in order to push packet onto head of queue. |
| 13-0 | PACKET_SIZE | R/W-0 | 0 | packet_size This field indicates packet size and is assumed to be zero on each packet add unless the value is explicitly overwritten. This field indicates packet size for packet pop operation. |

### 16.4.7.550 QUEUE_128_D Register (offset = 280Ch) [reset = 0h]

QUEUE_128_D is shown in Figure 16-826 and described in Table 16-840.

**Figure 16-826. QUEUE_128_D Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | DESC_SIZE | | | | |
| R/W-0 | | | R/W-0 | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-840. QUEUE_128_D Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-5 | DESC_PTR | R/W-0 | 0 | Descriptor pointer.<br>It will be read as zero if the queue is empty.<br>It will indicate a<br>32-bit aligned address that points to a descriptor when the queue is not empty. |
| 4-0 | DESC_SIZE | R/W-0 | 0 | Descriptor Size.<br>It is encoded in<br>4-byte increments with values 0 to 31 representing 24 and so on to 148 bytes.<br>This field will return a 0x0 when an empty queue is read.<br>Queue Manager Queue N Registers D To save hardware resources, the queue manager internally stores descriptor size (desc_size) information in four bits.<br>However, register D has five LSBs that specify descriptor size.<br>As a consequence, the value of desc_size that is pushed may not be same as that is read during a pop.<br>The value that is read back is equal to always rounded to an odd number.<br>So, for even values, the value read back is one more than what was written.<br>For odd values, the value read back is same as the value that was written.<br>Note that this<br>5-bit field (desc_size) is unrelated to the code for size of descriptors in a descriptor region.<br>It is just a place holder for a<br>5-bit value that is maintained across the push and pop operations for every descriptor managed by the queue manager. |

### 16.4.7.551 QUEUE_129_A Register (offset = 2810h) [reset = 0h]

QUEUE_129_A is shown in Figure 16-827 and described in Table 16-841.

**Figure 16-827. QUEUE_129_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-841. QUEUE_129_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>This count is incremented by 1 whenever a packet is added to the queue.<br>This count is decremented by 1 whenever a packet is popped from the queue. |

### 16.4.7.552 QUEUE_129_B Register (offset = 2814h) [reset = 0h]

QUEUE_129_B is shown in Figure 16-828 and described in Table 16-842.

**Figure 16-828. QUEUE_129_B Register**

| 31–28 | 27–0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-842. QUEUE_129_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. |

### 16.4.7.553 QUEUE_129_C Register (offset = 2818h) [reset = 0h]

QUEUE_129_C is shown in Figure 16-829 and described in Table 16-843.

**Figure 16-829. QUEUE_129_C Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| HEAD_TAIL | Reserved | | | | | | |
| W-0 | | | | | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| Reserved | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| Reserved | | PACKET_SIZE | | | | | |
| | | R/W-0 | | | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| PACKET_SIZE | | | | | | | |
| R/W-0 | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-843. QUEUE_129_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31 | HEAD_TAIL | W-0 | 0 | Head/Tail Push Control.<br>Set to zero in order to push packet onto tail of queue and set to one in order to push packet onto head of queue. |
| 13-0 | PACKET_SIZE | R/W-0 | 0 | packet_size This field indicates packet size and is assumed to be zero on each packet add unless the value is explicitly overwritten. This field indicates packet size for packet pop operation. |

### 16.4.7.554 QUEUE_129_D Register (offset = 281Ch) [reset = 0h]

QUEUE_129_D is shown in Figure 16-830 and described in Table 16-844.

**Figure 16-830. QUEUE_129_D Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | DESC_SIZE | | | | |
| R/W-0 | | | R/W-0 | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-844. QUEUE_129_D Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-5 | DESC_PTR | R/W-0 | 0 | Descriptor pointer.<br>It will be read as zero if the queue is empty.<br>It will indicate a<br>32-bit aligned address that points to a descriptor when the queue is not empty. |
| 4-0 | DESC_SIZE | R/W-0 | 0 | Descriptor Size.<br>It is encoded in<br>4-byte increments with values 0 to 31 representing 24 and so on to 148 bytes.<br>This field will return a 0x0 when an empty queue is read.<br>Queue Manager Queue N Registers D To save hardware resources, the queue manager internally stores descriptor size (desc_size) information in four bits.<br>However, register D has five LSBs that specify descriptor size.<br>As a consequence, the value of desc_size that is pushed may not be same as that is read during a pop.<br>The value that is read back is equal to always rounded to an odd number.<br>So, for even values, the value read back is one more than what was written.<br>For odd values, the value read back is same as the value that was written.<br>Note that this<br>5-bit field (desc_size) is unrelated to the code for size of descriptors in a descriptor region.<br>It is just a place holder for a<br>5-bit value that is maintained across the push and pop operations for every descriptor managed by the queue manager. |

### 16.4.7.555 QUEUE_130_A Register (offset = 2820h) [reset = 0h]

QUEUE_130_A is shown in Figure 16-831 and described in Table 16-845.

**Figure 16-831. QUEUE_130_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-845. QUEUE_130_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>This count is incremented by 1 whenever a packet is added to the queue.<br>This count is decremented by 1 whenever a packet is popped from the queue. |

### 16.4.7.556 QUEUE_130_B Register (offset = 2824h) [reset = 0h]

QUEUE_130_B is shown in Figure 16-832 and described in Table 16-846.

**Figure 16-832. QUEUE_130_B Register**

| 31–28 | 27–0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-846. QUEUE_130_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. |

#### 16.4.7.557 QUEUE_130_C Register (offset = 2828h) [reset = 0h]

QUEUE_130_C is shown in Figure 16-833 and described in Table 16-847.

**Figure 16-833. QUEUE_130_C Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| HEAD_TAIL | Reserved | | | | | | |
| W-0 | | | | | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| Reserved | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| Reserved | | PACKET_SIZE | | | | | |
| | | R/W-0 | | | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| PACKET_SIZE | | | | | | | |
| R/W-0 | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-847. QUEUE_130_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31 | HEAD_TAIL | W-0 | 0 | Head/Tail Push Control.<br>Set to zero in order to push packet onto tail of queue and set to one in order to push packet onto head of queue. |
| 13-0 | PACKET_SIZE | R/W-0 | 0 | packet_size This field indicates packet size and is assumed to be zero on each packet add unless the value is explicitly overwritten. This field indicates packet size for packet pop operation. |

### 16.4.7.558 QUEUE_130_D Register (offset = 282Ch) [reset = 0h]

QUEUE_130_D is shown in Figure 16-834 and described in Table 16-848.

**Figure 16-834. QUEUE_130_D Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |
| **23** | **22** | **21** | **20** | **19** | **18** | **17** | **16** |
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |
| **15** | **14** | **13** | **12** | **11** | **10** | **9** | **8** |
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |
| **7** | **6** | **5** | **4** | **3** | **2** | **1** | **0** |
| DESC_PTR | | | DESC_SIZE | | | | |
| R/W-0 | | | R/W-0 | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-848. QUEUE_130_D Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-5 | DESC_PTR | R/W-0 | 0 | Descriptor pointer.<br>It will be read as zero if the queue is empty.<br>It will indicate a<br>32-bit aligned address that points to a descriptor when the queue is not empty. |
| 4-0 | DESC_SIZE | R/W-0 | 0 | Descriptor Size.<br>It is encoded in<br>4-byte increments with values 0 to 31 representing 24 and so on to 148 bytes.<br>This field will return a 0x0 when an empty queue is read.<br>Queue Manager Queue N Registers D To save hardware resources, the queue manager internally stores descriptor size (desc_size) information in four bits.<br>However, register D has five LSBs that specify descriptor size.<br>As a consequence, the value of desc_size that is pushed may not be same as that is read during a pop.<br>The value that is read back is equal to always rounded to an odd number.<br>So, for even values, the value read back is one more than what was written.<br>For odd values, the value read back is same as the value that was written.<br>Note that this<br>5-bit field (desc_size) is unrelated to the code for size of descriptors in a descriptor region.<br>It is just a place holder for a<br>5-bit value that is maintained across the push and pop operations for every descriptor managed by the queue manager. |

### 16.4.7.559 QUEUE_131_A Register (offset = 2830h) [reset = 0h]

QUEUE_131_A is shown in Figure 16-835 and described in Table 16-849.

**Figure 16-835. QUEUE_131_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-849. QUEUE_131_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>This count is incremented by 1 whenever a packet is added to the queue.<br>This count is decremented by 1 whenever a packet is popped from the queue. |

### 16.4.7.560 QUEUE_131_B Register (offset = 2834h) [reset = 0h]

QUEUE_131_B is shown in Figure 16-836 and described in Table 16-850.

**Figure 16-836. QUEUE_131_B Register**

| 31-28 | 27-0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-850. QUEUE_131_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. |

### 16.4.7.561 QUEUE_131_C Register (offset = 2838h) [reset = 0h]

QUEUE_131_C is shown in Figure 16-837 and described in Table 16-851.

**Figure 16-837. QUEUE_131_C Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| HEAD_TAIL | Reserved | | | | | | |
| W-0 | | | | | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| Reserved | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| Reserved | | PACKET_SIZE | | | | | |
| | | R/W-0 | | | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| PACKET_SIZE | | | | | | | |
| R/W-0 | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-851. QUEUE_131_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31 | HEAD_TAIL | W-0 | 0 | Head/Tail Push Control.<br>Set to zero in order to push packet onto tail of queue and set to one in order to push packet onto head of queue. |
| 13-0 | PACKET_SIZE | R/W-0 | 0 | packet_size This field indicates packet size and is assumed to be zero on each packet add unless the value is explicitly overwritten. This field indicates packet size for packet pop operation. |

### 16.4.7.562 QUEUE_131_D Register (offset = 283Ch) [reset = 0h]

QUEUE_131_D is shown in Figure 16-838 and described in Table 16-852.

**Figure 16-838. QUEUE_131_D Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | DESC_SIZE | | | | |
| R/W-0 | | | R/W-0 | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-852. QUEUE_131_D Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-5 | DESC_PTR | R/W-0 | 0 | Descriptor pointer.<br>It will be read as zero if the queue is empty.<br>It will indicate a<br>32-bit aligned address that points to a descriptor when the queue is not empty. |
| 4-0 | DESC_SIZE | R/W-0 | 0 | Descriptor Size.<br>It is encoded in<br>4-byte increments with values 0 to 31 representing 24 and so on to 148 bytes.<br>This field will return a 0x0 when an empty queue is read.<br>Queue Manager Queue N Registers D To save hardware resources, the queue manager internally stores descriptor size (desc_size) information in four bits.<br>However, register D has five LSBs that specify descriptor size.<br>As a consequence, the value of desc_size that is pushed may not be same as that is read during a pop.<br>The value that is read back is equal to always rounded to an odd number.<br>So, for even values, the value read back is one more than what was written.<br>For odd values, the value read back is same as the value that was written.<br>Note that this<br>5-bit field (desc_size) is unrelated to the code for size of descriptors in a descriptor region.<br>It is just a place holder for a<br>5-bit value that is maintained across the push and pop operations for every descriptor managed by the queue manager. |

### 16.4.7.563 QUEUE_132_A Register (offset = 2840h) [reset = 0h]

QUEUE_132_A is shown in Figure 16-839 and described in Table 16-853.

**Figure 16-839. QUEUE_132_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-853. QUEUE_132_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>This count is incremented by 1 whenever a packet is added to the queue.<br>This count is decremented by 1 whenever a packet is popped from the queue. |

### 16.4.7.564 QUEUE_132_B Register (offset = 2844h) [reset = 0h]

QUEUE_132_B is shown in Figure 16-840 and described in Table 16-854.

**Figure 16-840. QUEUE_132_B Register**

| 31-28 | 27-0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-854. QUEUE_132_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. |

### 16.4.7.565 QUEUE_132_C Register (offset = 2848h) [reset = 0h]

QUEUE_132_C is shown in Figure 16-841 and described in Table 16-855.

**Figure 16-841. QUEUE_132_C Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| HEAD_TAIL | Reserved | | | | | | |
| W-0 | | | | | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| Reserved | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| Reserved | | PACKET_SIZE | | | | | |
| | | R/W-0 | | | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| PACKET_SIZE | | | | | | | |
| R/W-0 | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-855. QUEUE_132_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31 | HEAD_TAIL | W-0 | 0 | Head/Tail Push Control.<br>Set to zero in order to push packet onto tail of queue and set to one in order to push packet onto head of queue. |
| 13-0 | PACKET_SIZE | R/W-0 | 0 | packet_size This field indicates packet size and is assumed to be zero on each packet add unless the value is explicitly overwritten. This field indicates packet size for packet pop operation. |

### 16.4.7.566 QUEUE_132_D Register (offset = 284Ch) [reset = 0h]

QUEUE_132_D is shown in Figure 16-842 and described in Table 16-856.

**Figure 16-842. QUEUE_132_D Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | DESC_SIZE | | | | |
| R/W-0 | | | R/W-0 | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-856. QUEUE_132_D Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-5 | DESC_PTR | R/W-0 | 0 | Descriptor pointer.<br>It will be read as zero if the queue is empty.<br>It will indicate a<br>32-bit aligned address that points to a descriptor when the queue is not empty. |
| 4-0 | DESC_SIZE | R/W-0 | 0 | Descriptor Size.<br>It is encoded in<br>4-byte increments with values 0 to 31 representing 24 and so on to 148 bytes.<br>This field will return a 0x0 when an empty queue is read.<br>Queue Manager Queue N Registers D To save hardware resources, the queue manager internally stores descriptor size (desc_size) information in four bits.<br>However, register D has five LSBs that specify descriptor size.<br>As a consequence, the value of desc_size that is pushed may not be same as that is read during a pop.<br>The value that is read back is equal to always rounded to an odd number.<br>So, for even values, the value read back is one more than what was written.<br>For odd values, the value read back is same as the value that was written.<br>Note that this<br>5-bit field (desc_size) is unrelated to the code for size of descriptors in a descriptor region.<br>It is just a place holder for a<br>5-bit value that is maintained across the push and pop operations for every descriptor managed by the queue manager. |

### 16.4.7.567 QUEUE_133_A Register (offset = 2850h) [reset = 0h]

QUEUE_133_A is shown in Figure 16-843 and described in Table 16-857.

**Figure 16-843. QUEUE_133_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-857. QUEUE_133_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>This count is incremented by 1 whenever a packet is added to the queue.<br>This count is decremented by 1 whenever a packet is popped from the queue. |

### 16.4.7.568 QUEUE_133_B Register (offset = 2854h) [reset = 0h]

QUEUE_133_B is shown in Figure 16-844 and described in Table 16-858.

**Figure 16-844. QUEUE_133_B Register**

| 31-28 | 27-0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-858. QUEUE_133_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. |

### 16.4.7.569 QUEUE_133_C Register (offset = 2858h) [reset = 0h]

QUEUE_133_C is shown in Figure 16-845 and described in Table 16-859.

**Figure 16-845. QUEUE_133_C Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| HEAD_TAIL | Reserved | | | | | | |
| W-0 | | | | | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| Reserved | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| Reserved | | PACKET_SIZE | | | | | |
| | | R/W-0 | | | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| PACKET_SIZE | | | | | | | |
| R/W-0 | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-859. QUEUE_133_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31 | HEAD_TAIL | W-0 | 0 | Head/Tail Push Control.<br>Set to zero in order to push packet onto tail of queue and set to one in order to push packet onto head of queue. |
| 13-0 | PACKET_SIZE | R/W-0 | 0 | packet_size This field indicates packet size and is assumed to be zero on each packet add unless the value is explicitly overwritten. This field indicates packet size for packet pop operation. |

### 16.4.7.570 QUEUE_133_D Register (offset = 285Ch) [reset = 0h]

QUEUE_133_D is shown in Figure 16-846 and described in Table 16-860.

**Figure 16-846. QUEUE_133_D Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | DESC_SIZE | | | | |
| R/W-0 | | | R/W-0 | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-860. QUEUE_133_D Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-5 | DESC_PTR | R/W-0 | 0 | Descriptor pointer.<br>It will be read as zero if the queue is empty.<br>It will indicate a<br>32-bit aligned address that points to a descriptor when the queue is not empty. |
| 4-0 | DESC_SIZE | R/W-0 | 0 | Descriptor Size.<br>It is encoded in<br>4-byte increments with values 0 to 31 representing 24 and so on to 148 bytes.<br>This field will return a 0x0 when an empty queue is read.<br>Queue Manager Queue N Registers D To save hardware resources, the queue manager internally stores descriptor size (desc_size) information in four bits.<br>However, register D has five LSBs that specify descriptor size.<br>As a consequence, the value of desc_size that is pushed may not be same as that is read during a pop.<br>The value that is read back is equal to always rounded to an odd number.<br>So, for even values, the value read back is one more than what was written.<br>For odd values, the value read back is same as the value that was written.<br>Note that this<br>5-bit field (desc_size) is unrelated to the code for size of descriptors in a descriptor region.<br>It is just a place holder for a<br>5-bit value that is maintained across the push and pop operations for every descriptor managed by the queue manager. |

### 16.4.7.571 QUEUE_134_A Register (offset = 2860h) [reset = 0h]

QUEUE_134_A is shown in Figure 16-847 and described in Table 16-861.

**Figure 16-847. QUEUE_134_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-861. QUEUE_134_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>This count is incremented by 1 whenever a packet is added to the queue.<br>This count is decremented by 1 whenever a packet is popped from the queue. |

### 16.4.7.572 QUEUE_134_B Register (offset = 2864h) [reset = 0h]

QUEUE_134_B is shown in Figure 16-848 and described in Table 16-862.

**Figure 16-848. QUEUE_134_B Register**

| 31-28 | 27-0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-862. QUEUE_134_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. |

### 16.4.7.573 QUEUE_134_C Register (offset = 2868h) [reset = 0h]

QUEUE_134_C is shown in Figure 16-849 and described in Table 16-863.

**Figure 16-849. QUEUE_134_C Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| HEAD_TAIL | Reserved | | | | | | |
| W-0 | | | | | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| Reserved | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| Reserved | | PACKET_SIZE | | | | | |
| | | R/W-0 | | | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| PACKET_SIZE | | | | | | | |
| R/W-0 | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-863. QUEUE_134_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31 | HEAD_TAIL | W-0 | 0 | Head/Tail Push Control.<br>Set to zero in order to push packet onto tail of queue and set to one in order to push packet onto head of queue. |
| 13-0 | PACKET_SIZE | R/W-0 | 0 | packet_size This field indicates packet size and is assumed to be zero on each packet add unless the value is explicitly overwritten. This field indicates packet size for packet pop operation. |

### 16.4.7.574 QUEUE_134_D Register (offset = 286Ch) [reset = 0h]

QUEUE_134_D is shown in Figure 16-850 and described in Table 16-864.

**Figure 16-850. QUEUE_134_D Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |
| **23** | **22** | **21** | **20** | **19** | **18** | **17** | **16** |
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |
| **15** | **14** | **13** | **12** | **11** | **10** | **9** | **8** |
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |
| **7** | **6** | **5** | **4** | **3** | **2** | **1** | **0** |
| DESC_PTR | | | DESC_SIZE | | | | |
| R/W-0 | | | R/W-0 | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-864. QUEUE_134_D Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-5 | DESC_PTR | R/W-0 | 0 | Descriptor pointer.<br>It will be read as zero if the queue is empty.<br>It will indicate a<br>32-bit aligned address that points to a descriptor when the queue is not empty. |
| 4-0 | DESC_SIZE | R/W-0 | 0 | Descriptor Size.<br>It is encoded in<br>4-byte increments with values 0 to 31 representing 24 and so on to 148 bytes.<br>This field will return a 0x0 when an empty queue is read.<br>Queue Manager Queue N Registers D To save hardware resources, the queue manager internally stores descriptor size (desc_size) information in four bits.<br>However, register D has five LSBs that specify descriptor size.<br>As a consequence, the value of desc_size that is pushed may not be same as that is read during a pop.<br>The value that is read back is equal to always rounded to an odd number.<br>So, for even values, the value read back is one more than what was written.<br>For odd values, the value read back is same as the value that was written.<br>Note that this<br>5-bit field (desc_size) is unrelated to the code for size of descriptors in a descriptor region.<br>It is just a place holder for a<br>5-bit value that is maintained across the push and pop operations for every descriptor managed by the queue manager. |

### 16.4.7.575 QUEUE_135_A Register (offset = 2870h) [reset = 0h]

QUEUE_135_A is shown in Figure 16-851 and described in Table 16-865.

**Figure 16-851. QUEUE_135_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-865. QUEUE_135_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>This count is incremented by 1 whenever a packet is added to the queue.<br>This count is decremented by 1 whenever a packet is popped from the queue. |

### 16.4.7.576 QUEUE_135_B Register (offset = 2874h) [reset = 0h]

QUEUE_135_B is shown in Figure 16-852 and described in Table 16-866.

**Figure 16-852. QUEUE_135_B Register**

| 31–28 | 27–0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-866. QUEUE_135_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. |

### 16.4.7.577 QUEUE_135_C Register (offset = 2878h) [reset = 0h]

QUEUE_135_C is shown in Figure 16-853 and described in Table 16-867.

**Figure 16-853. QUEUE_135_C Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| HEAD_TAIL | Reserved | | | | | | |
| W-0 | | | | | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| Reserved | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| Reserved | | PACKET_SIZE | | | | | |
| | | R/W-0 | | | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| PACKET_SIZE | | | | | | | |
| R/W-0 | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-867. QUEUE_135_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31 | HEAD_TAIL | W-0 | 0 | Head/Tail Push Control.<br>Set to zero in order to push packet onto tail of queue and set to one in order to push packet onto head of queue. |
| 13-0 | PACKET_SIZE | R/W-0 | 0 | packet_size This field indicates packet size and is assumed to be zero on each packet add unless the value is explicitly overwritten. This field indicates packet size for packet pop operation. |

### 16.4.7.578 QUEUE_135_D Register (offset = 287Ch) [reset = 0h]

QUEUE_135_D is shown in Figure 16-854 and described in Table 16-868.

**Figure 16-854. QUEUE_135_D Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |
| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |
| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |
| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
| DESC_PTR | | | DESC_SIZE | | | | |
| R/W-0 | | | R/W-0 | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-868. QUEUE_135_D Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-5 | DESC_PTR | R/W-0 | 0 | Descriptor pointer.<br>It will be read as zero if the queue is empty.<br>It will indicate a<br>32-bit aligned address that points to a descriptor when the queue is not empty. |
| 4-0 | DESC_SIZE | R/W-0 | 0 | Descriptor Size.<br>It is encoded in<br>4-byte increments with values 0 to 31 representing 24 and so on to 148 bytes.<br>This field will return a 0x0 when an empty queue is read.<br>Queue Manager Queue N Registers D To save hardware resources, the queue manager internally stores descriptor size (desc_size) information in four bits.<br>However, register D has five LSBs that specify descriptor size.<br>As a consequence, the value of desc_size that is pushed may not be same as that is read during a pop.<br>The value that is read back is equal to always rounded to an odd number.<br>So, for even values, the value read back is one more than what was written.<br>For odd values, the value read back is same as the value that was written.<br>Note that this<br>5-bit field (desc_size) is unrelated to the code for size of descriptors in a descriptor region.<br>It is just a place holder for a<br>5-bit value that is maintained across the push and pop operations for every descriptor managed by the queue manager. |

### 16.4.7.579 QUEUE_136_A Register (offset = 2880h) [reset = 0h]

QUEUE_136_A is shown in Figure 16-855 and described in Table 16-869.

**Figure 16-855. QUEUE_136_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-869. QUEUE_136_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>This count is incremented by 1 whenever a packet is added to the queue.<br>This count is decremented by 1 whenever a packet is popped from the queue. |

### 16.4.7.580 QUEUE_136_B Register (offset = 2884h) [reset = 0h]

QUEUE_136_B is shown in Figure 16-856 and described in Table 16-870.

**Figure 16-856. QUEUE_136_B Register**

| 31-28 | 27-0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-870. QUEUE_136_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. |

### 16.4.7.581 QUEUE_136_C Register (offset = 2888h) [reset = 0h]

QUEUE_136_C is shown in Figure 16-857 and described in Table 16-871.

**Figure 16-857. QUEUE_136_C Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| HEAD_TAIL | Reserved | | | | | | |
| W-0 | | | | | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| Reserved | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| Reserved | | PACKET_SIZE | | | | | |
| | | R/W-0 | | | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| PACKET_SIZE | | | | | | | |
| R/W-0 | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-871. QUEUE_136_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31 | HEAD_TAIL | W-0 | 0 | Head/Tail Push Control.<br>Set to zero in order to push packet onto tail of queue and set to one in order to push packet onto head of queue. |
| 13-0 | PACKET_SIZE | R/W-0 | 0 | packet_size This field indicates packet size and is assumed to be zero on each packet add unless the value is explicitly overwritten. This field indicates packet size for packet pop operation. |

### 16.4.7.582 QUEUE_136_D Register (offset = 288Ch) [reset = 0h]

QUEUE_136_D is shown in Figure 16-858 and described in Table 16-872.

**Figure 16-858. QUEUE_136_D Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | DESC_SIZE | | | | |
| R/W-0 | | | R/W-0 | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-872. QUEUE_136_D Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-5 | DESC_PTR | R/W-0 | 0 | Descriptor pointer.<br>It will be read as zero if the queue is empty.<br>It will indicate a<br>32-bit aligned address that points to a descriptor when the queue is not empty. |
| 4-0 | DESC_SIZE | R/W-0 | 0 | Descriptor Size.<br>It is encoded in<br>4-byte increments with values 0 to 31 representing 24 and so on to 148 bytes.<br>This field will return a 0x0 when an empty queue is read.<br>Queue Manager Queue N Registers D To save hardware resources, the queue manager internally stores descriptor size (desc_size) information in four bits.<br>However, register D has five LSBs that specify descriptor size.<br>As a consequence, the value of desc_size that is pushed may not be same as that is read during a pop.<br>The value that is read back is equal to always rounded to an odd number.<br>So, for even values, the value read back is one more than what was written.<br>For odd values, the value read back is same as the value that was written.<br>Note that this<br>5-bit field (desc_size) is unrelated to the code for size of descriptors in a descriptor region.<br>It is just a place holder for a<br>5-bit value that is maintained across the push and pop operations for every descriptor managed by the queue manager. |

### 16.4.7.583 QUEUE_137_A Register (offset = 2890h) [reset = 0h]

QUEUE_137_A is shown in Figure 16-859 and described in Table 16-873.

**Figure 16-859. QUEUE_137_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-873. QUEUE_137_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>This count is incremented by 1 whenever a packet is added to the queue.<br>This count is decremented by 1 whenever a packet is popped from the queue. |

### 16.4.7.584 QUEUE_137_B Register (offset = 2894h) [reset = 0h]

QUEUE_137_B is shown in Figure 16-860 and described in Table 16-874.

**Figure 16-860. QUEUE_137_B Register**

| 31–28 | 27–0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-874. QUEUE_137_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. |

### 16.4.7.585 QUEUE_137_C Register (offset = 2898h) [reset = 0h]

QUEUE_137_C is shown in Figure 16-861 and described in Table 16-875.

**Figure 16-861. QUEUE_137_C Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| HEAD_TAIL | Reserved | | | | | | |
| W-0 | | | | | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| Reserved | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| Reserved | | PACKET_SIZE | | | | | |
| | | R/W-0 | | | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| PACKET_SIZE | | | | | | | |
| R/W-0 | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-875. QUEUE_137_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31 | HEAD_TAIL | W-0 | 0 | Head/Tail Push Control.<br>Set to zero in order to push packet onto tail of queue and set to one in order to push packet onto head of queue. |
| 13-0 | PACKET_SIZE | R/W-0 | 0 | packet_size This field indicates packet size and is assumed to be zero on each packet add unless the value is explicitly overwritten. This field indicates packet size for packet pop operation. |

### 16.4.7.586 QUEUE_137_D Register (offset = 289Ch) [reset = 0h]

QUEUE_137_D is shown in Figure 16-862 and described in Table 16-876.

**Figure 16-862. QUEUE_137_D Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |
| **23** | **22** | **21** | **20** | **19** | **18** | **17** | **16** |
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |
| **15** | **14** | **13** | **12** | **11** | **10** | **9** | **8** |
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |
| **7** | **6** | **5** | **4** | **3** | **2** | **1** | **0** |
| DESC_PTR | | | DESC_SIZE | | | | |
| R/W-0 | | | R/W-0 | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-876. QUEUE_137_D Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-5 | DESC_PTR | R/W-0 | 0 | Descriptor pointer.<br>It will be read as zero if the queue is empty.<br>It will indicate a<br>32-bit aligned address that points to a descriptor when the queue is not empty. |
| 4-0 | DESC_SIZE | R/W-0 | 0 | Descriptor Size.<br>It is encoded in<br>4-byte increments with values 0 to 31 representing 24 and so on to 148 bytes.<br>This field will return a 0x0 when an empty queue is read.<br>Queue Manager Queue N Registers D To save hardware resources, the queue manager internally stores descriptor size (desc_size) information in four bits.<br>However, register D has five LSBs that specify descriptor size.<br>As a consequence, the value of desc_size that is pushed may not be same as that is read during a pop.<br>The value that is read back is equal to always rounded to an odd number.<br>So, for even values, the value read back is one more than what was written.<br>For odd values, the value read back is same as the value that was written.<br>Note that this<br>5-bit field (desc_size) is unrelated to the code for size of descriptors in a descriptor region.<br>It is just a place holder for a<br>5-bit value that is maintained across the push and pop operations for every descriptor managed by the queue manager. |

### 16.4.7.587 QUEUE_138_A Register (offset = 28A0h) [reset = 0h]

QUEUE_138_A is shown in Figure 16-863 and described in Table 16-877.

**Figure 16-863. QUEUE_138_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-877. QUEUE_138_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>This count is incremented by 1 whenever a packet is added to the queue.<br>This count is decremented by 1 whenever a packet is popped from the queue. |

### 16.4.7.588 QUEUE_138_B Register (offset = 28A4h) [reset = 0h]

QUEUE_138_B is shown in Figure 16-864 and described in Table 16-878.

**Figure 16-864. QUEUE_138_B Register**

| 31–28 | 27–0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-878. QUEUE_138_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. |

### 16.4.7.589 QUEUE_138_C Register (offset = 28A8h) [reset = 0h]

QUEUE_138_C is shown in Figure 16-865 and described in Table 16-879.

**Figure 16-865. QUEUE_138_C Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| HEAD_TAIL | Reserved | | | | | | |
| W-0 | | | | | | | |
| **23** | **22** | **21** | **20** | **19** | **18** | **17** | **16** |
| Reserved | | | | | | | |
| **15** | **14** | **13** | **12** | **11** | **10** | **9** | **8** |
| Reserved | | PACKET_SIZE | | | | | |
| | | R/W-0 | | | | | |
| **7** | **6** | **5** | **4** | **3** | **2** | **1** | **0** |
| PACKET_SIZE | | | | | | | |
| R/W-0 | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-879. QUEUE_138_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31 | HEAD_TAIL | W-0 | 0 | Head/Tail Push Control.<br>Set to zero in order to push packet onto tail of queue and set to one in order to push packet onto head of queue. |
| 13-0 | PACKET_SIZE | R/W-0 | 0 | packet_size This field indicates packet size and is assumed to be zero on each packet add unless the value is explicitly overwritten. This field indicates packet size for packet pop operation. |

### 16.4.7.590 QUEUE_138_D Register (offset = 28ACh) [reset = 0h]

QUEUE_138_D is shown in Figure 16-866 and described in Table 16-880.

**Figure 16-866. QUEUE_138_D Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | DESC_SIZE | | | | |
| R/W-0 | | | R/W-0 | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-880. QUEUE_138_D Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-5 | DESC_PTR | R/W-0 | 0 | Descriptor pointer.<br>It will be read as zero if the queue is empty.<br>It will indicate a<br>32-bit aligned address that points to a descriptor when the queue is not empty. |
| 4-0 | DESC_SIZE | R/W-0 | 0 | Descriptor Size.<br>It is encoded in<br>4-byte increments with values 0 to 31 representing 24 and so on to 148 bytes.<br>This field will return a 0x0 when an empty queue is read.<br>Queue Manager Queue N Registers D To save hardware resources, the queue manager internally stores descriptor size (desc_size) information in four bits.<br>However, register D has five LSBs that specify descriptor size.<br>As a consequence, the value of desc_size that is pushed may not be same as that is read during a pop.<br>The value that is read back is equal to always rounded to an odd number.<br>So, for even values, the value read back is one more than what was written.<br>For odd values, the value read back is same as the value that was written.<br>Note that this<br>5-bit field (desc_size) is unrelated to the code for size of descriptors in a descriptor region.<br>It is just a place holder for a<br>5-bit value that is maintained across the push and pop operations for every descriptor managed by the queue manager. |

### 16.4.7.591 QUEUE_139_A Register (offset = 28B0h) [reset = 0h]

QUEUE_139_A is shown in Figure 16-867 and described in Table 16-881.

**Figure 16-867. QUEUE_139_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-881. QUEUE_139_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>This count is incremented by 1 whenever a packet is added to the queue.<br>This count is decremented by 1 whenever a packet is popped from the queue. |

### 16.4.7.592 QUEUE_139_B Register (offset = 28B4h) [reset = 0h]

QUEUE_139_B is shown in Figure 16-868 and described in Table 16-882.

**Figure 16-868. QUEUE_139_B Register**

| 31–28 | 27–0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-882. QUEUE_139_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. |

### 16.4.7.593 QUEUE_139_C Register (offset = 28B8h) [reset = 0h]

QUEUE_139_C is shown in Figure 16-869 and described in Table 16-883.

**Figure 16-869. QUEUE_139_C Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| HEAD_TAIL | Reserved | | | | | | |
| W-0 | | | | | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| Reserved | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| Reserved | | PACKET_SIZE | | | | | |
| | | R/W-0 | | | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| PACKET_SIZE | | | | | | | |
| R/W-0 | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-883. QUEUE_139_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31 | HEAD_TAIL | W-0 | 0 | Head/Tail Push Control.<br>Set to zero in order to push packet onto tail of queue and set to one in order to push packet onto head of queue. |
| 13-0 | PACKET_SIZE | R/W-0 | 0 | packet_size This field indicates packet size and is assumed to be zero on each packet add unless the value is explicitly overwritten.<br>This field indicates packet size for packet pop operation. |

### 16.4.7.594 QUEUE_139_D Register (offset = 28BCh) [reset = 0h]

QUEUE_139_D is shown in Figure 16-870 and described in Table 16-884.

**Figure 16-870. QUEUE_139_D Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |
| **23** | **22** | **21** | **20** | **19** | **18** | **17** | **16** |
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |
| **15** | **14** | **13** | **12** | **11** | **10** | **9** | **8** |
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |
| **7** | **6** | **5** | **4** | **3** | **2** | **1** | **0** |
| DESC_PTR | | | DESC_SIZE | | | | |
| R/W-0 | | | R/W-0 | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-884. QUEUE_139_D Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-5 | DESC_PTR | R/W-0 | 0 | Descriptor pointer.<br>It will be read as zero if the queue is empty.<br>It will indicate a<br>32-bit aligned address that points to a descriptor when the queue is not empty. |
| 4-0 | DESC_SIZE | R/W-0 | 0 | Descriptor Size.<br>It is encoded in<br>4-byte increments with values 0 to 31 representing 24 and so on to 148 bytes.<br>This field will return a 0x0 when an empty queue is read.<br>Queue Manager Queue N Registers D To save hardware resources, the queue manager internally stores descriptor size (desc_size) information in four bits.<br>However, register D has five LSBs that specify descriptor size.<br>As a consequence, the value of desc_size that is pushed may not be same as that is read during a pop.<br>The value that is read back is equal to always rounded to an odd number.<br>So, for even values, the value read back is one more than what was written.<br>For odd values, the value read back is same as the value that was written.<br>Note that this<br>5-bit field (desc_size) is unrelated to the code for size of descriptors in a descriptor region.<br>It is just a place holder for a<br>5-bit value that is maintained across the push and pop operations for every descriptor managed by the queue manager. |

### 16.4.7.595 QUEUE_140_A Register (offset = 28C0h) [reset = 0h]

QUEUE_140_A is shown in Figure 16-871 and described in Table 16-885.

**Figure 16-871. QUEUE_140_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-885. QUEUE_140_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>This count is incremented by 1 whenever a packet is added to the queue.<br>This count is decremented by 1 whenever a packet is popped from the queue. |

### 16.4.7.596 QUEUE_140_B Register (offset = 28C4h) [reset = 0h]

QUEUE_140_B is shown in Figure 16-872 and described in Table 16-886.

**Figure 16-872. QUEUE_140_B Register**

| 31-28 | 27-0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-886. QUEUE_140_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. |

### 16.4.7.597 QUEUE_140_C Register (offset = 28C8h) [reset = 0h]

QUEUE_140_C is shown in Figure 16-873 and described in Table 16-887.

**Figure 16-873. QUEUE_140_C Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| HEAD_TAIL | Reserved | | | | | | |
| W-0 | | | | | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| Reserved | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| Reserved | | PACKET_SIZE | | | | | |
| | | R/W-0 | | | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| PACKET_SIZE | | | | | | | |
| R/W-0 | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-887. QUEUE_140_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31 | HEAD_TAIL | W-0 | 0 | Head/Tail Push Control.<br>Set to zero in order to push packet onto tail of queue and set to one in order to push packet onto head of queue. |
| 13-0 | PACKET_SIZE | R/W-0 | 0 | packet_size This field indicates packet size and is assumed to be zero on each packet add unless the value is explicitly overwritten. This field indicates packet size for packet pop operation. |

### 16.4.7.598 QUEUE_140_D Register (offset = 28CCh) [reset = 0h]

QUEUE_140_D is shown in Figure 16-874 and described in Table 16-888.

**Figure 16-874. QUEUE_140_D Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |
| **23** | **22** | **21** | **20** | **19** | **18** | **17** | **16** |
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |
| **15** | **14** | **13** | **12** | **11** | **10** | **9** | **8** |
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |
| **7** | **6** | **5** | **4** | **3** | **2** | **1** | **0** |
| DESC_PTR | | | DESC_SIZE | | | | |
| R/W-0 | | | R/W-0 | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-888. QUEUE_140_D Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-5 | DESC_PTR | R/W-0 | 0 | Descriptor pointer.<br>It will be read as zero if the queue is empty.<br>It will indicate a<br>32-bit aligned address that points to a descriptor when the queue is not empty. |
| 4-0 | DESC_SIZE | R/W-0 | 0 | Descriptor Size.<br>It is encoded in<br>4-byte increments with values 0 to 31 representing 24 and so on to 148 bytes.<br>This field will return a 0x0 when an empty queue is read.<br>Queue Manager Queue N Registers D To save hardware resources, the queue manager internally stores descriptor size (desc_size) information in four bits.<br>However, register D has five LSBs that specify descriptor size.<br>As a consequence, the value of desc_size that is pushed may not be same as that is read during a pop.<br>The value that is read back is equal to always rounded to an odd number.<br>So, for even values, the value read back is one more than what was written.<br>For odd values, the value read back is same as the value that was written.<br>Note that this<br>5-bit field (desc_size) is unrelated to the code for size of descriptors in a descriptor region.<br>It is just a place holder for a<br>5-bit value that is maintained across the push and pop operations for every descriptor managed by the queue manager. |

### 16.4.7.599 QUEUE_141_A Register (offset = 28D0h) [reset = 0h]

QUEUE_141_A is shown in Figure 16-875 and described in Table 16-889.

**Figure 16-875. QUEUE_141_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-889. QUEUE_141_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>This count is incremented by 1 whenever a packet is added to the queue.<br>This count is decremented by 1 whenever a packet is popped from the queue. |

### 16.4.7.600 QUEUE_141_B Register (offset = 28D4h) [reset = 0h]

QUEUE_141_B is shown in Figure 16-876 and described in Table 16-890.

**Figure 16-876. QUEUE_141_B Register**

| 31–28 | 27–0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-890. QUEUE_141_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. |

### 16.4.7.601 QUEUE_141_C Register (offset = 28D8h) [reset = 0h]

QUEUE_141_C is shown in Figure 16-877 and described in Table 16-891.

**Figure 16-877. QUEUE_141_C Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| HEAD_TAIL | Reserved | | | | | | |
| W-0 | | | | | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| Reserved | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| Reserved | | PACKET_SIZE | | | | | |
| | | R/W-0 | | | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| PACKET_SIZE | | | | | | | |
| R/W-0 | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-891. QUEUE_141_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31 | HEAD_TAIL | W-0 | 0 | Head/Tail Push Control.<br>Set to zero in order to push packet onto tail of queue and set to one in order to push packet onto head of queue. |
| 13-0 | PACKET_SIZE | R/W-0 | 0 | packet_size This field indicates packet size and is assumed to be zero on each packet add unless the value is explicitly overwritten. This field indicates packet size for packet pop operation. |

### 16.4.7.602 QUEUE_141_D Register (offset = 28DCh) [reset = 0h]

QUEUE_141_D is shown in Figure 16-878 and described in Table 16-892.

**Figure 16-878. QUEUE_141_D Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |
| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |
| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |
| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
| DESC_PTR | | | DESC_SIZE | | | | |
| R/W-0 | | | R/W-0 | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-892. QUEUE_141_D Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-5 | DESC_PTR | R/W-0 | 0 | Descriptor pointer.<br>It will be read as zero if the queue is empty.<br>It will indicate a<br>32-bit aligned address that points to a descriptor when the queue is not empty. |
| 4-0 | DESC_SIZE | R/W-0 | 0 | Descriptor Size.<br>It is encoded in<br>4-byte increments with values 0 to 31 representing 24 and so on to 148 bytes.<br>This field will return a 0x0 when an empty queue is read.<br>Queue Manager Queue N Registers D To save hardware resources, the queue manager internally stores descriptor size (desc_size) information in four bits.<br>However, register D has five LSBs that specify descriptor size.<br>As a consequence, the value of desc_size that is pushed may not be same as that is read during a pop.<br>The value that is read back is equal to always rounded to an odd number.<br>So, for even values, the value read back is one more than what was written.<br>For odd values, the value read back is same as the value that was written.<br>Note that this<br>5-bit field (desc_size) is unrelated to the code for size of descriptors in a descriptor region.<br>It is just a place holder for a<br>5-bit value that is maintained across the push and pop operations for every descriptor managed by the queue manager. |

### 16.4.7.603 QUEUE_142_A Register (offset = 28E0h) [reset = 0h]

QUEUE_142_A is shown in Figure 16-879 and described in Table 16-893.

**Figure 16-879. QUEUE_142_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-893. QUEUE_142_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>This count is incremented by 1 whenever a packet is added to the queue.<br>This count is decremented by 1 whenever a packet is popped from the queue. |

### 16.4.7.604 QUEUE_142_B Register (offset = 28E4h) [reset = 0h]

QUEUE_142_B is shown in Figure 16-880 and described in Table 16-894.

**Figure 16-880. QUEUE_142_B Register**

| 31–28 | 27–0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-894. QUEUE_142_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. |

### 16.4.7.605 QUEUE_142_C Register (offset = 28E8h) [reset = 0h]

QUEUE_142_C is shown in Figure 16-881 and described in Table 16-895.

**Figure 16-881. QUEUE_142_C Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| HEAD_TAIL | Reserved | | | | | | |
| W-0 | | | | | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| Reserved | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| Reserved | | PACKET_SIZE | | | | | |
| | | R/W-0 | | | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| PACKET_SIZE | | | | | | | |
| R/W-0 | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-895. QUEUE_142_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31 | HEAD_TAIL | W-0 | 0 | Head/Tail Push Control.<br>Set to zero in order to push packet onto tail of queue and set to one in order to push packet onto head of queue. |
| 13-0 | PACKET_SIZE | R/W-0 | 0 | packet_size This field indicates packet size and is assumed to be zero on each packet add unless the value is explicitly overwritten. This field indicates packet size for packet pop operation. |

### 16.4.7.606 QUEUE_142_D Register (offset = 28ECh) [reset = 0h]

QUEUE_142_D is shown in Figure 16-882 and described in Table 16-896.

**Figure 16-882. QUEUE_142_D Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |
| **23** | **22** | **21** | **20** | **19** | **18** | **17** | **16** |
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |
| **15** | **14** | **13** | **12** | **11** | **10** | **9** | **8** |
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |
| **7** | **6** | **5** | **4** | **3** | **2** | **1** | **0** |
| DESC_PTR | | | DESC_SIZE | | | | |
| R/W-0 | | | R/W-0 | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-896. QUEUE_142_D Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-5 | DESC_PTR | R/W-0 | 0 | Descriptor pointer.<br>It will be read as zero if the queue is empty.<br>It will indicate a<br>32-bit aligned address that points to a descriptor when the queue is not empty. |
| 4-0 | DESC_SIZE | R/W-0 | 0 | Descriptor Size.<br>It is encoded in<br>4-byte increments with values 0 to 31 representing 24 and so on to 148 bytes.<br>This field will return a 0x0 when an empty queue is read.<br>Queue Manager Queue N Registers D To save hardware resources, the queue manager internally stores descriptor size (desc_size) information in four bits.<br>However, register D has five LSBs that specify descriptor size.<br>As a consequence, the value of desc_size that is pushed may not be same as that is read during a pop.<br>The value that is read back is equal to always rounded to an odd number.<br>So, for even values, the value read back is one more than what was written.<br>For odd values, the value read back is same as the value that was written.<br>Note that this<br>5-bit field (desc_size) is unrelated to the code for size of descriptors in a descriptor region.<br>It is just a place holder for a<br>5-bit value that is maintained across the push and pop operations for every descriptor managed by the queue manager. |

### 16.4.7.607 QUEUE_143_A Register (offset = 28F0h) [reset = 0h]

QUEUE_143_A is shown in Figure 16-883 and described in Table 16-897.

**Figure 16-883. QUEUE_143_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-897. QUEUE_143_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>This count is incremented by 1 whenever a packet is added to the queue.<br>This count is decremented by 1 whenever a packet is popped from the queue. |

### 16.4.7.608 QUEUE_143_B Register (offset = 28F4h) [reset = 0h]

QUEUE_143_B is shown in Figure 16-884 and described in Table 16-898.

**Figure 16-884. QUEUE_143_B Register**

| 31-28 | 27-0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-898. QUEUE_143_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. |

### 16.4.7.609 QUEUE_143_C Register (offset = 28F8h) [reset = 0h]

QUEUE_143_C is shown in Figure 16-885 and described in Table 16-899.

**Figure 16-885. QUEUE_143_C Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| HEAD_TAIL | Reserved | | | | | | |
| W-0 | | | | | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| Reserved | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| Reserved | | PACKET_SIZE | | | | | |
| | | R/W-0 | | | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| PACKET_SIZE | | | | | | | |
| R/W-0 | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-899. QUEUE_143_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31 | HEAD_TAIL | W-0 | 0 | Head/Tail Push Control.<br>Set to zero in order to push packet onto tail of queue and set to one in order to push packet onto head of queue. |
| 13-0 | PACKET_SIZE | R/W-0 | 0 | packet_size This field indicates packet size and is assumed to be zero on each packet add unless the value is explicitly overwritten. This field indicates packet size for packet pop operation. |

### 16.4.7.610 QUEUE_143_D Register (offset = 28FCh) [reset = 0h]

QUEUE_143_D is shown in Figure 16-886 and described in Table 16-900.

**Figure 16-886. QUEUE_143_D Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |
| **23** | **22** | **21** | **20** | **19** | **18** | **17** | **16** |
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |
| **15** | **14** | **13** | **12** | **11** | **10** | **9** | **8** |
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |
| **7** | **6** | **5** | **4** | **3** | **2** | **1** | **0** |
| DESC_PTR | | | DESC_SIZE | | | | |
| R/W-0 | | | R/W-0 | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-900. QUEUE_143_D Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-5 | DESC_PTR | R/W-0 | 0 | Descriptor pointer.<br>It will be read as zero if the queue is empty.<br>It will indicate a<br>32-bit aligned address that points to a descriptor when the queue is not empty. |
| 4-0 | DESC_SIZE | R/W-0 | 0 | Descriptor Size.<br>It is encoded in<br>4-byte increments with values 0 to 31 representing 24 and so on to 148 bytes.<br>This field will return a 0x0 when an empty queue is read.<br>Queue Manager Queue N Registers D To save hardware resources, the queue manager internally stores descriptor size (desc_size) information in four bits.<br>However, register D has five LSBs that specify descriptor size.<br>As a consequence, the value of desc_size that is pushed may not be same as that is read during a pop.<br>The value that is read back is equal to always rounded to an odd number.<br>So, for even values, the value read back is one more than what was written.<br>For odd values, the value read back is same as the value that was written.<br>Note that this<br>5-bit field (desc_size) is unrelated to the code for size of descriptors in a descriptor region.<br>It is just a place holder for a<br>5-bit value that is maintained across the push and pop operations for every descriptor managed by the queue manager. |

### 16.4.7.611 QUEUE_144_A Register (offset = 2900h) [reset = 0h]

QUEUE_144_A is shown in Figure 16-887 and described in Table 16-901.

**Figure 16-887. QUEUE_144_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-901. QUEUE_144_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>This count is incremented by 1 whenever a packet is added to the queue.<br>This count is decremented by 1 whenever a packet is popped from the queue. |

### 16.4.7.612 QUEUE_144_B Register (offset = 2904h) [reset = 0h]

QUEUE_144_B is shown in Figure 16-888 and described in Table 16-902.

**Figure 16-888. QUEUE_144_B Register**

| 31–28 | 27–0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-902. QUEUE_144_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. |

### 16.4.7.613 QUEUE_144_C Register (offset = 2908h) [reset = 0h]

QUEUE_144_C is shown in Figure 16-889 and described in Table 16-903.

**Figure 16-889. QUEUE_144_C Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| HEAD_TAIL | Reserved | | | | | | |
| W-0 | | | | | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| Reserved | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| Reserved | | PACKET_SIZE | | | | | |
| | | R/W-0 | | | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| PACKET_SIZE | | | | | | | |
| R/W-0 | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-903. QUEUE_144_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31 | HEAD_TAIL | W-0 | 0 | Head/Tail Push Control.<br>Set to zero in order to push packet onto tail of queue and set to one in order to push packet onto head of queue. |
| 13-0 | PACKET_SIZE | R/W-0 | 0 | packet_size This field indicates packet size and is assumed to be zero on each packet add unless the value is explicitly overwritten. This field indicates packet size for packet pop operation. |

### 16.4.7.614 QUEUE_144_D Register (offset = 290Ch) [reset = 0h]

QUEUE_144_D is shown in Figure 16-890 and described in Table 16-904.

**Figure 16-890. QUEUE_144_D Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | DESC_SIZE | | | | |
| R/W-0 | | | R/W-0 | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-904. QUEUE_144_D Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-5 | DESC_PTR | R/W-0 | 0 | Descriptor pointer.<br>It will be read as zero if the queue is empty.<br>It will indicate a<br>32-bit aligned address that points to a descriptor when the queue is not empty. |
| 4-0 | DESC_SIZE | R/W-0 | 0 | Descriptor Size.<br>It is encoded in<br>4-byte increments with values 0 to 31 representing 24 and so on to 148 bytes.<br>This field will return a 0x0 when an empty queue is read.<br>Queue Manager Queue N Registers D To save hardware resources, the queue manager internally stores descriptor size (desc_size) information in four bits.<br>However, register D has five LSBs that specify descriptor size.<br>As a consequence, the value of desc_size that is pushed may not be same as that is read during a pop.<br>The value that is read back is equal to always rounded to an odd number.<br>So, for even values, the value read back is one more than what was written.<br>For odd values, the value read back is same as the value that was written.<br>Note that this<br>5-bit field (desc_size) is unrelated to the code for size of descriptors in a descriptor region.<br>It is just a place holder for a<br>5-bit value that is maintained across the push and pop operations for every descriptor managed by the queue manager. |

### 16.4.7.615 QUEUE_145_A Register (offset = 2910h) [reset = 0h]

QUEUE_145_A is shown in Figure 16-891 and described in Table 16-905.

**Figure 16-891. QUEUE_145_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-905. QUEUE_145_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>This count is incremented by 1 whenever a packet is added to the queue.<br>This count is decremented by 1 whenever a packet is popped from the queue. |

### 16.4.7.616 QUEUE_145_B Register (offset = 2914h) [reset = 0h]

QUEUE_145_B is shown in Figure 16-892 and described in Table 16-906.

**Figure 16-892. QUEUE_145_B Register**

| 31-28 | 27-0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-906. QUEUE_145_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. |

### 16.4.7.617 QUEUE_145_C Register (offset = 2918h) [reset = 0h]

QUEUE_145_C is shown in Figure 16-893 and described in Table 16-907.

**Figure 16-893. QUEUE_145_C Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| HEAD_TAIL | Reserved | | | | | | |
| W-0 | | | | | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| Reserved | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| Reserved | | PACKET_SIZE | | | | | |
| | | R/W-0 | | | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| PACKET_SIZE | | | | | | | |
| R/W-0 | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-907. QUEUE_145_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31 | HEAD_TAIL | W-0 | 0 | Head/Tail Push Control.<br>Set to zero in order to push packet onto tail of queue and set to one in order to push packet onto head of queue. |
| 13-0 | PACKET_SIZE | R/W-0 | 0 | packet_size This field indicates packet size and is assumed to be zero on each packet add unless the value is explicitly overwritten. This field indicates packet size for packet pop operation. |

### 16.4.7.618 QUEUE_145_D Register (offset = 291Ch) [reset = 0h]

QUEUE_145_D is shown in Figure 16-894 and described in Table 16-908.

**Figure 16-894. QUEUE_145_D Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |
| **23** | **22** | **21** | **20** | **19** | **18** | **17** | **16** |
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |
| **15** | **14** | **13** | **12** | **11** | **10** | **9** | **8** |
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |
| **7** | **6** | **5** | **4** | **3** | **2** | **1** | **0** |
| DESC_PTR | | | DESC_SIZE | | | | |
| R/W-0 | | | R/W-0 | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-908. QUEUE_145_D Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-5 | DESC_PTR | R/W-0 | 0 | Descriptor pointer.<br>It will be read as zero if the queue is empty.<br>It will indicate a<br>32-bit aligned address that points to a descriptor when the queue is not empty. |
| 4-0 | DESC_SIZE | R/W-0 | 0 | Descriptor Size.<br>It is encoded in<br>4-byte increments with values 0 to 31 representing 24 and so on to 148 bytes.<br>This field will return a 0x0 when an empty queue is read.<br>Queue Manager Queue N Registers D To save hardware resources, the queue manager internally stores descriptor size (desc_size) information in four bits.<br>However, register D has five LSBs that specify descriptor size.<br>As a consequence, the value of desc_size that is pushed may not be same as that is read during a pop.<br>The value that is read back is equal to always rounded to an odd number.<br>So, for even values, the value read back is one more than what was written.<br>For odd values, the value read back is same as the value that was written.<br>Note that this<br>5-bit field (desc_size) is unrelated to the code for size of descriptors in a descriptor region.<br>It is just a place holder for a<br>5-bit value that is maintained across the push and pop operations for every descriptor managed by the queue manager. |

### 16.4.7.619 QUEUE_146_A Register (offset = 2920h) [reset = 0h]

QUEUE_146_A is shown in Figure 16-895 and described in Table 16-909.

**Figure 16-895. QUEUE_146_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-909. QUEUE_146_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>This count is incremented by 1 whenever a packet is added to the queue.<br>This count is decremented by 1 whenever a packet is popped from the queue. |

### 16.4.7.620 QUEUE_146_B Register (offset = 2924h) [reset = 0h]

QUEUE_146_B is shown in Figure 16-896 and described in Table 16-910.

**Figure 16-896. QUEUE_146_B Register**

| 31–28 | 27–0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-910. QUEUE_146_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. |

### 16.4.7.621 QUEUE_146_C Register (offset = 2928h) [reset = 0h]

QUEUE_146_C is shown in Figure 16-897 and described in Table 16-911.

**Figure 16-897. QUEUE_146_C Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| HEAD_TAIL | Reserved | | | | | | |
| W-0 | | | | | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| Reserved | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| Reserved | | PACKET_SIZE | | | | | |
| | | R/W-0 | | | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| PACKET_SIZE | | | | | | | |
| R/W-0 | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-911. QUEUE_146_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31 | HEAD_TAIL | W-0 | 0 | Head/Tail Push Control.<br>Set to zero in order to push packet onto tail of queue and set to one in order to push packet onto head of queue. |
| 13-0 | PACKET_SIZE | R/W-0 | 0 | packet_size This field indicates packet size and is assumed to be zero on each packet add unless the value is explicitly overwritten. This field indicates packet size for packet pop operation. |

### 16.4.7.622 QUEUE_146_D Register (offset = 292Ch) [reset = 0h]

QUEUE_146_D is shown in Figure 16-898 and described in Table 16-912.

**Figure 16-898. QUEUE_146_D Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |
| **23** | **22** | **21** | **20** | **19** | **18** | **17** | **16** |
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |
| **15** | **14** | **13** | **12** | **11** | **10** | **9** | **8** |
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |
| **7** | **6** | **5** | **4** | **3** | **2** | **1** | **0** |
| DESC_PTR | | | DESC_SIZE | | | | |
| R/W-0 | | | R/W-0 | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-912. QUEUE_146_D Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-5 | DESC_PTR | R/W-0 | 0 | Descriptor pointer.<br>It will be read as zero if the queue is empty.<br>It will indicate a<br>32-bit aligned address that points to a descriptor when the queue is not empty. |
| 4-0 | DESC_SIZE | R/W-0 | 0 | Descriptor Size.<br>It is encoded in<br>4-byte increments with values 0 to 31 representing 24 and so on to 148 bytes.<br>This field will return a 0x0 when an empty queue is read.<br>Queue Manager Queue N Registers D To save hardware resources, the queue manager internally stores descriptor size (desc_size) information in four bits.<br>However, register D has five LSBs that specify descriptor size.<br>As a consequence, the value of desc_size that is pushed may not be same as that is read during a pop.<br>The value that is read back is equal to always rounded to an odd number.<br>So, for even values, the value read back is one more than what was written.<br>For odd values, the value read back is same as the value that was written.<br>Note that this<br>5-bit field (desc_size) is unrelated to the code for size of descriptors in a descriptor region.<br>It is just a place holder for a<br>5-bit value that is maintained across the push and pop operations for every descriptor managed by the queue manager. |

### 16.4.7.623 QUEUE_147_A Register (offset = 2930h) [reset = 0h]

QUEUE_147_A is shown in Figure 16-899 and described in Table 16-913.

**Figure 16-899. QUEUE_147_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-913. QUEUE_147_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>This count is incremented by 1 whenever a packet is added to the queue.<br>This count is decremented by 1 whenever a packet is popped from the queue. |

### 16.4.7.624 QUEUE_147_B Register (offset = 2934h) [reset = 0h]

QUEUE_147_B is shown in Figure 16-900 and described in Table 16-914.

**Figure 16-900. QUEUE_147_B Register**

| 31-28 | 27-0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-914. QUEUE_147_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. |

### 16.4.7.625 QUEUE_147_C Register (offset = 2938h) [reset = 0h]

QUEUE_147_C is shown in Figure 16-901 and described in Table 16-915.

**Figure 16-901. QUEUE_147_C Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| HEAD_TAIL | Reserved | | | | | | |
| W-0 | | | | | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| Reserved | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| Reserved | | PACKET_SIZE | | | | | |
| | | R/W-0 | | | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| PACKET_SIZE | | | | | | | |
| R/W-0 | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-915. QUEUE_147_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31 | HEAD_TAIL | W-0 | 0 | Head/Tail Push Control.<br>Set to zero in order to push packet onto tail of queue and set to one in order to push packet onto head of queue. |
| 13-0 | PACKET_SIZE | R/W-0 | 0 | packet_size This field indicates packet size and is assumed to be zero on each packet add unless the value is explicitly overwritten. This field indicates packet size for packet pop operation. |

### 16.4.7.626 QUEUE_147_D Register (offset = 293Ch) [reset = 0h]

QUEUE_147_D is shown in Figure 16-902 and described in Table 16-916.

**Figure 16-902. QUEUE_147_D Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | DESC_SIZE | | | | |
| R/W-0 | | | R/W-0 | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-916. QUEUE_147_D Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-5 | DESC_PTR | R/W-0 | 0 | Descriptor pointer.<br>It will be read as zero if the queue is empty.<br>It will indicate a<br>32-bit aligned address that points to a descriptor when the queue is not empty. |
| 4-0 | DESC_SIZE | R/W-0 | 0 | Descriptor Size.<br>It is encoded in<br>4-byte increments with values 0 to 31 representing 24 and so on to 148 bytes.<br>This field will return a 0x0 when an empty queue is read.<br>Queue Manager Queue N Registers D To save hardware resources, the queue manager internally stores descriptor size (desc_size) information in four bits.<br>However, register D has five LSBs that specify descriptor size.<br>As a consequence, the value of desc_size that is pushed may not be same as that is read during a pop.<br>The value that is read back is equal to always rounded to an odd number.<br>So, for even values, the value read back is one more than what was written.<br>For odd values, the value read back is same as the value that was written.<br>Note that this<br>5-bit field (desc_size) is unrelated to the code for size of descriptors in a descriptor region.<br>It is just a place holder for a<br>5-bit value that is maintained across the push and pop operations for every descriptor managed by the queue manager. |

### 16.4.7.627 QUEUE_148_A Register (offset = 2940h) [reset = 0h]

QUEUE_148_A is shown in Figure 16-903 and described in Table 16-917.

**Figure 16-903. QUEUE_148_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-917. QUEUE_148_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>This count is incremented by 1 whenever a packet is added to the queue.<br>This count is decremented by 1 whenever a packet is popped from the queue. |

### 16.4.7.628 QUEUE_148_B Register (offset = 2944h) [reset = 0h]

QUEUE_148_B is shown in Figure 16-904 and described in Table 16-918.

**Figure 16-904. QUEUE_148_B Register**

| 31-28 | 27-0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-918. QUEUE_148_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. |

### 16.4.7.629 QUEUE_148_C Register (offset = 2948h) [reset = 0h]

QUEUE_148_C is shown in Figure 16-905 and described in Table 16-919.

**Figure 16-905. QUEUE_148_C Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| HEAD_TAIL | Reserved | | | | | | |
| W-0 | | | | | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| Reserved | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| Reserved | | PACKET_SIZE | | | | | |
| | | R/W-0 | | | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| PACKET_SIZE | | | | | | | |
| R/W-0 | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-919. QUEUE_148_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31 | HEAD_TAIL | W-0 | 0 | Head/Tail Push Control.<br>Set to zero in order to push packet onto tail of queue and set to one in order to push packet onto head of queue. |
| 13-0 | PACKET_SIZE | R/W-0 | 0 | packet_size This field indicates packet size and is assumed to be zero on each packet add unless the value is explicitly overwritten. This field indicates packet size for packet pop operation. |

### 16.4.7.630 QUEUE_148_D Register (offset = 294Ch) [reset = 0h]

QUEUE_148_D is shown in Figure 16-906 and described in Table 16-920.

**Figure 16-906. QUEUE_148_D Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | DESC_SIZE | | | | |
| R/W-0 | | | R/W-0 | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-920. QUEUE_148_D Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-5 | DESC_PTR | R/W-0 | 0 | Descriptor pointer.<br>It will be read as zero if the queue is empty.<br>It will indicate a<br>32-bit aligned address that points to a descriptor when the queue is not empty. |
| 4-0 | DESC_SIZE | R/W-0 | 0 | Descriptor Size.<br>It is encoded in<br>4-byte increments with values 0 to 31 representing 24 and so on to 148 bytes.<br>This field will return a 0x0 when an empty queue is read.<br>Queue Manager Queue N Registers D To save hardware resources, the queue manager internally stores descriptor size (desc_size) information in four bits.<br>However, register D has five LSBs that specify descriptor size.<br>As a consequence, the value of desc_size that is pushed may not be same as that is read during a pop.<br>The value that is read back is equal to always rounded to an odd number.<br>So, for even values, the value read back is one more than what was written.<br>For odd values, the value read back is same as the value that was written.<br>Note that this<br>5-bit field (desc_size) is unrelated to the code for size of descriptors in a descriptor region.<br>It is just a place holder for a<br>5-bit value that is maintained across the push and pop operations for every descriptor managed by the queue manager. |

### 16.4.7.631 QUEUE_149_A Register (offset = 2950h) [reset = 0h]

QUEUE_149_A is shown in Figure 16-907 and described in Table 16-921.

**Figure 16-907. QUEUE_149_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-921. QUEUE_149_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>This count is incremented by 1 whenever a packet is added to the queue.<br>This count is decremented by 1 whenever a packet is popped from the queue. |

### 16.4.7.632 QUEUE_149_B Register (offset = 2954h) [reset = 0h]

QUEUE_149_B is shown in Figure 16-908 and described in Table 16-922.

**Figure 16-908. QUEUE_149_B Register**

| 31-28 | 27-0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-922. QUEUE_149_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. |

### 16.4.7.633 QUEUE_149_C Register (offset = 2958h) [reset = 0h]

QUEUE_149_C is shown in Figure 16-909 and described in Table 16-923.

**Figure 16-909. QUEUE_149_C Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| HEAD_TAIL | Reserved | | | | | | |
| W-0 | | | | | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| Reserved | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| Reserved | | PACKET_SIZE | | | | | |
| | | R/W-0 | | | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| PACKET_SIZE | | | | | | | |
| R/W-0 | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-923. QUEUE_149_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31 | HEAD_TAIL | W-0 | 0 | Head/Tail Push Control.<br>Set to zero in order to push packet onto tail of queue and set to one in order to push packet onto head of queue. |
| 13-0 | PACKET_SIZE | R/W-0 | 0 | packet_size This field indicates packet size and is assumed to be zero on each packet add unless the value is explicitly overwritten. This field indicates packet size for packet pop operation. |

### 16.4.7.634 QUEUE_149_D Register (offset = 295Ch) [reset = 0h]

QUEUE_149_D is shown in Figure 16-910 and described in Table 16-924.

**Figure 16-910. QUEUE_149_D Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |
| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |
| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |
| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
| DESC_PTR | | | DESC_SIZE | | | | |
| R/W-0 | | | R/W-0 | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-924. QUEUE_149_D Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-5 | DESC_PTR | R/W-0 | 0 | Descriptor pointer.<br>It will be read as zero if the queue is empty.<br>It will indicate a<br>32-bit aligned address that points to a descriptor when the queue is not empty. |
| 4-0 | DESC_SIZE | R/W-0 | 0 | Descriptor Size.<br>It is encoded in<br>4-byte increments with values 0 to 31 representing 24 and so on to 148 bytes.<br>This field will return a 0x0 when an empty queue is read.<br>Queue Manager Queue N Registers D To save hardware resources, the queue manager internally stores descriptor size (desc_size) information in four bits.<br>However, register D has five LSBs that specify descriptor size.<br>As a consequence, the value of desc_size that is pushed may not be same as that is read during a pop.<br>The value that is read back is equal to always rounded to an odd number.<br>So, for even values, the value read back is one more than what was written.<br>For odd values, the value read back is same as the value that was written.<br>Note that this<br>5-bit field (desc_size) is unrelated to the code for size of descriptors in a descriptor region.<br>It is just a place holder for a<br>5-bit value that is maintained across the push and pop operations for every descriptor managed by the queue manager. |

### 16.4.7.635 QUEUE_150_A Register (offset = 2960h) [reset = 0h]

QUEUE_150_A is shown in Figure 16-911 and described in Table 16-925.

**Figure 16-911. QUEUE_150_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-925. QUEUE_150_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>This count is incremented by 1 whenever a packet is added to the queue.<br>This count is decremented by 1 whenever a packet is popped from the queue. |

### 16.4.7.636 QUEUE_150_B Register (offset = 2964h) [reset = 0h]

QUEUE_150_B is shown in Figure 16-912 and described in Table 16-926.

**Figure 16-912. QUEUE_150_B Register**

| 31–28 | 27–0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-926. QUEUE_150_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. |

### 16.4.7.637 QUEUE_150_C Register (offset = 2968h) [reset = 0h]

QUEUE_150_C is shown in Figure 16-913 and described in Table 16-927.

**Figure 16-913. QUEUE_150_C Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| HEAD_TAIL | Reserved | | | | | | |
| W-0 | | | | | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| Reserved | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| Reserved | | PACKET_SIZE | | | | | |
| | | R/W-0 | | | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| PACKET_SIZE | | | | | | | |
| R/W-0 | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-927. QUEUE_150_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31 | HEAD_TAIL | W-0 | 0 | Head/Tail Push Control.<br>Set to zero in order to push packet onto tail of queue and set to one in order to push packet onto head of queue. |
| 13-0 | PACKET_SIZE | R/W-0 | 0 | packet_size This field indicates packet size and is assumed to be zero on each packet add unless the value is explicitly overwritten. This field indicates packet size for packet pop operation. |

### 16.4.7.638 QUEUE_150_D Register (offset = 296Ch) [reset = 0h]

QUEUE_150_D is shown in Figure 16-914 and described in Table 16-928.

**Figure 16-914. QUEUE_150_D Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |
| **23** | **22** | **21** | **20** | **19** | **18** | **17** | **16** |
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |
| **15** | **14** | **13** | **12** | **11** | **10** | **9** | **8** |
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |
| **7** | **6** | **5** | **4** | **3** | **2** | **1** | **0** |
| DESC_PTR | | | DESC_SIZE | | | | |
| R/W-0 | | | R/W-0 | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-928. QUEUE_150_D Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-5 | DESC_PTR | R/W-0 | 0 | Descriptor pointer.<br>It will be read as zero if the queue is empty.<br>It will indicate a<br>32-bit aligned address that points to a descriptor when the queue is not empty. |
| 4-0 | DESC_SIZE | R/W-0 | 0 | Descriptor Size.<br>It is encoded in<br>4-byte increments with values 0 to 31 representing 24 and so on to 148 bytes.<br>This field will return a 0x0 when an empty queue is read.<br>Queue Manager Queue N Registers D To save hardware resources, the queue manager internally stores descriptor size (desc_size) information in four bits.<br>However, register D has five LSBs that specify descriptor size.<br>As a consequence, the value of desc_size that is pushed may not be same as that is read during a pop.<br>The value that is read back is equal to always rounded to an odd number.<br>So, for even values, the value read back is one more than what was written.<br>For odd values, the value read back is same as the value that was written.<br>Note that this<br>5-bit field (desc_size) is unrelated to the code for size of descriptors in a descriptor region.<br>It is just a place holder for a<br>5-bit value that is maintained across the push and pop operations for every descriptor managed by the queue manager. |

### 16.4.7.639 QUEUE_151_A Register (offset = 2970h) [reset = 0h]

QUEUE_151_A is shown in Figure 16-915 and described in Table 16-929.

**Figure 16-915. QUEUE_151_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-929. QUEUE_151_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>This count is incremented by 1 whenever a packet is added to the queue.<br>This count is decremented by 1 whenever a packet is popped from the queue. |

### 16.4.7.640 QUEUE_151_B Register (offset = 2974h) [reset = 0h]

QUEUE_151_B is shown in Figure 16-916 and described in Table 16-930.

**Figure 16-916. QUEUE_151_B Register**

| 31-28 | 27-0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-930. QUEUE_151_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. |

### 16.4.7.641 QUEUE_151_C Register (offset = 2978h) [reset = 0h]

QUEUE_151_C is shown in Figure 16-917 and described in Table 16-931.

**Figure 16-917. QUEUE_151_C Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| HEAD_TAIL | Reserved | | | | | | |
| W-0 | | | | | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| Reserved | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| Reserved | | PACKET_SIZE | | | | | |
| | | R/W-0 | | | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| PACKET_SIZE | | | | | | | |
| R/W-0 | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-931. QUEUE_151_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31 | HEAD_TAIL | W-0 | 0 | Head/Tail Push Control.<br>Set to zero in order to push packet onto tail of queue and set to one in order to push packet onto head of queue. |
| 13-0 | PACKET_SIZE | R/W-0 | 0 | packet_size This field indicates packet size and is assumed to be zero on each packet add unless the value is explicitly overwritten. This field indicates packet size for packet pop operation. |

### 16.4.7.642 QUEUE_151_D Register (offset = 297Ch) [reset = 0h]

QUEUE_151_D is shown in Figure 16-918 and described in Table 16-932.

**Figure 16-918. QUEUE_151_D Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | DESC_SIZE | | | | |
| R/W-0 | | | R/W-0 | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-932. QUEUE_151_D Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-5 | DESC_PTR | R/W-0 | 0 | Descriptor pointer.<br>It will be read as zero if the queue is empty.<br>It will indicate a<br>32-bit aligned address that points to a descriptor when the queue is not empty. |
| 4-0 | DESC_SIZE | R/W-0 | 0 | Descriptor Size.<br>It is encoded in<br>4-byte increments with values 0 to 31 representing 24 and so on to 148 bytes.<br>This field will return a 0x0 when an empty queue is read.<br>Queue Manager Queue N Registers D To save hardware resources, the queue manager internally stores descriptor size (desc_size) information in four bits.<br>However, register D has five LSBs that specify descriptor size.<br>As a consequence, the value of desc_size that is pushed may not be same as that is read during a pop.<br>The value that is read back is equal to always rounded to an odd number.<br>So, for even values, the value read back is one more than what was written.<br>For odd values, the value read back is same as the value that was written.<br>Note that this<br>5-bit field (desc_size) is unrelated to the code for size of descriptors in a descriptor region.<br>It is just a place holder for a<br>5-bit value that is maintained across the push and pop operations for every descriptor managed by the queue manager. |

### 16.4.7.643 QUEUE_152_A Register (offset = 2980h) [reset = 0h]

QUEUE_152_A is shown in Figure 16-919 and described in Table 16-933.

**Figure 16-919. QUEUE_152_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-933. QUEUE_152_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>This count is incremented by 1 whenever a packet is added to the queue.<br>This count is decremented by 1 whenever a packet is popped from the queue. |

### 16.4.7.644 QUEUE_152_B Register (offset = 2984h) [reset = 0h]

QUEUE_152_B is shown in Figure 16-920 and described in Table 16-934.

**Figure 16-920. QUEUE_152_B Register**

| 31-28 | 27-0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-934. QUEUE_152_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. |

### 16.4.7.645 QUEUE_152_C Register (offset = 2988h) [reset = 0h]

QUEUE_152_C is shown in Figure 16-921 and described in Table 16-935.

**Figure 16-921. QUEUE_152_C Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| HEAD_TAIL | Reserved | | | | | | |
| W-0 | | | | | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| Reserved | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| Reserved | | PACKET_SIZE | | | | | |
| | | R/W-0 | | | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| PACKET_SIZE | | | | | | | |
| R/W-0 | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-935. QUEUE_152_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31 | HEAD_TAIL | W-0 | 0 | Head/Tail Push Control.<br>Set to zero in order to push packet onto tail of queue and set to one in order to push packet onto head of queue. |
| 13-0 | PACKET_SIZE | R/W-0 | 0 | packet_size This field indicates packet size and is assumed to be zero on each packet add unless the value is explicitly overwritten. This field indicates packet size for packet pop operation. |

### 16.4.7.646 QUEUE_152_D Register (offset = 298Ch) [reset = 0h]

QUEUE_152_D is shown in Figure 16-922 and described in Table 16-936.

**Figure 16-922. QUEUE_152_D Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |
| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |
| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |
| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
| DESC_PTR | | | DESC_SIZE | | | | |
| R/W-0 | | | R/W-0 | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-936. QUEUE_152_D Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-5 | DESC_PTR | R/W-0 | 0 | Descriptor pointer.<br>It will be read as zero if the queue is empty.<br>It will indicate a<br>32-bit aligned address that points to a descriptor when the queue is not empty. |
| 4-0 | DESC_SIZE | R/W-0 | 0 | Descriptor Size.<br>It is encoded in<br>4-byte increments with values 0 to 31 representing 24 and so on to 148 bytes.<br>This field will return a 0x0 when an empty queue is read.<br>Queue Manager Queue N Registers D To save hardware resources, the queue manager internally stores descriptor size (desc_size) information in four bits.<br>However, register D has five LSBs that specify descriptor size.<br>As a consequence, the value of desc_size that is pushed may not be same as that is read during a pop.<br>The value that is read back is equal to always rounded to an odd number.<br>So, for even values, the value read back is one more than what was written.<br>For odd values, the value read back is same as the value that was written.<br>Note that this<br>5-bit field (desc_size) is unrelated to the code for size of descriptors in a descriptor region.<br>It is just a place holder for a<br>5-bit value that is maintained across the push and pop operations for every descriptor managed by the queue manager. |

### 16.4.7.647 QUEUE_153_A Register (offset = 2990h) [reset = 0h]

QUEUE_153_A is shown in Figure 16-923 and described in Table 16-937.

**Figure 16-923. QUEUE_153_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-937. QUEUE_153_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>This count is incremented by 1 whenever a packet is added to the queue.<br>This count is decremented by 1 whenever a packet is popped from the queue. |

### 16.4.7.648 QUEUE_153_B Register (offset = 2994h) [reset = 0h]

QUEUE_153_B is shown in Figure 16-924 and described in Table 16-938.

**Figure 16-924. QUEUE_153_B Register**

| 31–28 | 27–0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-938. QUEUE_153_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. |

### 16.4.7.649 QUEUE_153_C Register (offset = 2998h) [reset = 0h]

QUEUE_153_C is shown in Figure 16-925 and described in Table 16-939.

**Figure 16-925. QUEUE_153_C Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| HEAD_TAIL | Reserved | | | | | | |
| W-0 | | | | | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| Reserved | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| Reserved | | PACKET_SIZE | | | | | |
| | | R/W-0 | | | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| PACKET_SIZE | | | | | | | |
| R/W-0 | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-939. QUEUE_153_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31 | HEAD_TAIL | W-0 | 0 | Head/Tail Push Control.<br>Set to zero in order to push packet onto tail of queue and set to one in order to push packet onto head of queue. |
| 13-0 | PACKET_SIZE | R/W-0 | 0 | packet_size This field indicates packet size and is assumed to be zero on each packet add unless the value is explicitly overwritten. This field indicates packet size for packet pop operation. |

### 16.4.7.650 QUEUE_153_D Register (offset = 299Ch) [reset = 0h]

QUEUE_153_D is shown in Figure 16-926 and described in Table 16-940.

**Figure 16-926. QUEUE_153_D Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | DESC_SIZE | | | | |
| R/W-0 | | | R/W-0 | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-940. QUEUE_153_D Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-5 | DESC_PTR | R/W-0 | 0 | Descriptor pointer.<br>It will be read as zero if the queue is empty.<br>It will indicate a<br>32-bit aligned address that points to a descriptor when the queue is not empty. |
| 4-0 | DESC_SIZE | R/W-0 | 0 | Descriptor Size.<br>It is encoded in<br>4-byte increments with values 0 to 31 representing 24 and so on to 148 bytes.<br>This field will return a 0x0 when an empty queue is read.<br>Queue Manager Queue N Registers D To save hardware resources, the queue manager internally stores descriptor size (desc_size) information in four bits.<br>However, register D has five LSBs that specify descriptor size.<br>As a consequence, the value of desc_size that is pushed may not be same as that is read during a pop.<br>The value that is read back is equal to always rounded to an odd number.<br>So, for even values, the value read back is one more than what was written.<br>For odd values, the value read back is same as the value that was written.<br>Note that this<br>5-bit field (desc_size) is unrelated to the code for size of descriptors in a descriptor region.<br>It is just a place holder for a<br>5-bit value that is maintained across the push and pop operations for every descriptor managed by the queue manager. |

### 16.4.7.651 QUEUE_154_A Register (offset = 29A0h) [reset = 0h]

QUEUE_154_A is shown in Figure 16-927 and described in Table 16-941.

**Figure 16-927. QUEUE_154_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-941. QUEUE_154_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>This count is incremented by 1 whenever a packet is added to the queue.<br>This count is decremented by 1 whenever a packet is popped from the queue. |

### 16.4.7.652 QUEUE_154_B Register (offset = 29A4h) [reset = 0h]

QUEUE_154_B is shown in Figure 16-928 and described in Table 16-942.

**Figure 16-928. QUEUE_154_B Register**

| 31-28 | 27-0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-942. QUEUE_154_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. |

### 16.4.7.653 QUEUE_154_C Register (offset = 29A8h) [reset = 0h]

QUEUE_154_C is shown in Figure 16-929 and described in Table 16-943.

**Figure 16-929. QUEUE_154_C Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| HEAD_TAIL | Reserved | | | | | | |
| W-0 | | | | | | | |
| **23** | **22** | **21** | **20** | **19** | **18** | **17** | **16** |
| Reserved | | | | | | | |
| **15** | **14** | **13** | **12** | **11** | **10** | **9** | **8** |
| Reserved | | PACKET_SIZE | | | | | |
| | | R/W-0 | | | | | |
| **7** | **6** | **5** | **4** | **3** | **2** | **1** | **0** |
| PACKET_SIZE | | | | | | | |
| R/W-0 | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-943. QUEUE_154_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31 | HEAD_TAIL | W-0 | 0 | Head/Tail Push Control.<br>Set to zero in order to push packet onto tail of queue and set to one in order to push packet onto head of queue. |
| 13-0 | PACKET_SIZE | R/W-0 | 0 | packet_size This field indicates packet size and is assumed to be zero on each packet add unless the value is explicitly overwritten. This field indicates packet size for packet pop operation. |

### 16.4.7.654 QUEUE_154_D Register (offset = 29ACh) [reset = 0h]

QUEUE_154_D is shown in Figure 16-930 and described in Table 16-944.

**Figure 16-930. QUEUE_154_D Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |
| **23** | **22** | **21** | **20** | **19** | **18** | **17** | **16** |
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |
| **15** | **14** | **13** | **12** | **11** | **10** | **9** | **8** |
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |
| **7** | **6** | **5** | **4** | **3** | **2** | **1** | **0** |
| DESC_PTR | | | DESC_SIZE | | | | |
| R/W-0 | | | R/W-0 | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-944. QUEUE_154_D Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-5 | DESC_PTR | R/W-0 | 0 | Descriptor pointer.<br>It will be read as zero if the queue is empty.<br>It will indicate a<br>32-bit aligned address that points to a descriptor when the queue is not empty. |
| 4-0 | DESC_SIZE | R/W-0 | 0 | Descriptor Size.<br>It is encoded in<br>4-byte increments with values 0 to 31 representing 24 and so on to 148 bytes.<br>This field will return a 0x0 when an empty queue is read.<br>Queue Manager Queue N Registers D To save hardware resources, the queue manager internally stores descriptor size (desc_size) information in four bits.<br>However, register D has five LSBs that specify descriptor size.<br>As a consequence, the value of desc_size that is pushed may not be same as that is read during a pop.<br>The value that is read back is equal to always rounded to an odd number.<br>So, for even values, the value read back is one more than what was written.<br>For odd values, the value read back is same as the value that was written.<br>Note that this<br>5-bit field (desc_size) is unrelated to the code for size of descriptors in a descriptor region.<br>It is just a place holder for a<br>5-bit value that is maintained across the push and pop operations for every descriptor managed by the queue manager. |

### 16.4.7.655 QUEUE_155_A Register (offset = 29B0h) [reset = 0h]

QUEUE_155_A is shown in Figure 16-931 and described in Table 16-945.

**Figure 16-931. QUEUE_155_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-945. QUEUE_155_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>This count is incremented by 1 whenever a packet is added to the queue.<br>This count is decremented by 1 whenever a packet is popped from the queue. |

### 16.4.7.656 QUEUE_155_B Register (offset = 29B4h) [reset = 0h]

QUEUE_155_B is shown in Figure 16-932 and described in Table 16-946.

**Figure 16-932. QUEUE_155_B Register**

| 31–28 | 27–0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-946. QUEUE_155_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. |

### 16.4.7.657 QUEUE_155_C Register (offset = 29B8h) [reset = 0h]

QUEUE_155_C is shown in Figure 16-933 and described in Table 16-947.

**Figure 16-933. QUEUE_155_C Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| HEAD_TAIL | Reserved | | | | | | |
| W-0 | | | | | | | |

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| Reserved | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| Reserved | | PACKET_SIZE | | | | | |
| | | R/W-0 | | | | | |

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| PACKET_SIZE | | | | | | | |
| R/W-0 | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-947. QUEUE_155_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31 | HEAD_TAIL | W-0 | 0 | Head/Tail Push Control.<br>Set to zero in order to push packet onto tail of queue and set to one in order to push packet onto head of queue. |
| 13-0 | PACKET_SIZE | R/W-0 | 0 | packet_size This field indicates packet size and is assumed to be zero on each packet add unless the value is explicitly overwritten. This field indicates packet size for packet pop operation. |

### 16.4.7.658 QUEUE_155_D Register (offset = 29BCh) [reset = 0h]

QUEUE_155_D is shown in Figure 16-934 and described in Table 16-948.

**Figure 16-934. QUEUE_155_D Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |
| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |
| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
| DESC_PTR | | | | | | | |
| R/W-0 | | | | | | | |
| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
| DESC_PTR | | | DESC_SIZE | | | | |
| R/W-0 | | | R/W-0 | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-948. QUEUE_155_D Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 31-5 | DESC_PTR | R/W-0 | 0 | Descriptor pointer.<br>It will be read as zero if the queue is empty.<br>It will indicate a<br>32-bit aligned address that points to a descriptor when the queue is not empty. |
| 4-0 | DESC_SIZE | R/W-0 | 0 | Descriptor Size.<br>It is encoded in<br>4-byte increments with values 0 to 31 representing 24 and so on to 148 bytes.<br>This field will return a 0x0 when an empty queue is read.<br>Queue Manager Queue N Registers D To save hardware resources, the queue manager internally stores descriptor size (desc_size) information in four bits.<br>However, register D has five LSBs that specify descriptor size.<br>As a consequence, the value of desc_size that is pushed may not be same as that is read during a pop.<br>The value that is read back is equal to always rounded to an odd number.<br>So, for even values, the value read back is one more than what was written.<br>For odd values, the value read back is same as the value that was written.<br>Note that this<br>5-bit field (desc_size) is unrelated to the code for size of descriptors in a descriptor region.<br>It is just a place holder for a<br>5-bit value that is maintained across the push and pop operations for every descriptor managed by the queue manager. |

### 16.4.7.659 QUEUE_0_STATUS_A Register (offset = 3000h) [reset = 0h]

QUEUE_0_STATUS_A is shown in Figure 16-935 and described in Table 16-949.

**Figure 16-935. QUEUE_0_STATUS_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-949. QUEUE_0_STATUS_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>Queue Manager Queue N Status Registers A |

### 16.4.7.660 QUEUE_0_STATUS_B Register (offset = 3004h) [reset = 0h]

QUEUE_0_STATUS_B is shown in Figure 16-936 and described in Table 16-950.

**Figure 16-936. QUEUE_0_STATUS_B Register**

| 31–28 | 27–0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-950. QUEUE_0_STATUS_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. Queue_Manager_Queue_n_Status_B Registers B |

### 16.4.7.661 QUEUE_0_STATUS_C Register (offset = 3008h) [reset = 0h]

QUEUE_0_STATUS_C is shown in Figure 16-937 and described in Table 16-951.

**Figure 16-937. QUEUE_0_STATUS_C Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | PACKET_SIZE |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-951. QUEUE_0_STATUS_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | PACKET_SIZE | R-0 | 0 | This field indicates packet size of the head element of a queue. Queue_Manager_Queue_N_Status_C Registers C |

### 16.4.7.662 QUEUE_1_STATUS_A Register (offset = 3010h) [reset = 0h]

QUEUE_1_STATUS_A is shown in Figure 16-938 and described in Table 16-952.

**Figure 16-938. QUEUE_1_STATUS_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-952. QUEUE_1_STATUS_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>Queue Manager Queue N Status Registers A |

### 16.4.7.663 QUEUE_1_STATUS_B Register (offset = 3014h) [reset = 0h]

QUEUE_1_STATUS_B is shown in Figure 16-939 and described in Table 16-953.

**Figure 16-939. QUEUE_1_STATUS_B Register**

| 31–28 | 27–0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-953. QUEUE_1_STATUS_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. Queue_Manager_Queue_n_Status_B Registers B |

### 16.4.7.664 QUEUE_1_STATUS_C Register (offset = 3018h) [reset = 0h]

QUEUE_1_STATUS_C is shown in Figure 16-940 and described in Table 16-954.

**Figure 16-940. QUEUE_1_STATUS_C Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | PACKET_SIZE |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-954. QUEUE_1_STATUS_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | PACKET_SIZE | R-0 | 0 | This field indicates packet size of the head element of a queue. Queue_Manager_Queue_N_Status_C Registers C |

### 16.4.7.665 QUEUE_2_STATUS_A Register (offset = 3020h) [reset = 0h]

QUEUE_2_STATUS_A is shown in Figure 16-941 and described in Table 16-955.

**Figure 16-941. QUEUE_2_STATUS_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-955. QUEUE_2_STATUS_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>Queue Manager Queue N Status Registers A |

### 16.4.7.666 QUEUE_2_STATUS_B Register (offset = 3024h) [reset = 0h]

QUEUE_2_STATUS_B is shown in Figure 16-942 and described in Table 16-956.

**Figure 16-942. QUEUE_2_STATUS_B Register**

| 31–28 | 27–0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-956. QUEUE_2_STATUS_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue.<br>Queue_Manager_Queue_n_Status_B Registers B |

### 16.4.7.667 QUEUE_2_STATUS_C Register (offset = 3028h) [reset = 0h]

QUEUE_2_STATUS_C is shown in Figure 16-943 and described in Table 16-957.

**Figure 16-943. QUEUE_2_STATUS_C Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | PACKET_SIZE |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-957. QUEUE_2_STATUS_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | PACKET_SIZE | R-0 | 0 | This field indicates packet size of the head element of a queue.<br>Queue_Manager_Queue_N_Status_C Registers C |

### 16.4.7.668 QUEUE_3_STATUS_A Register (offset = 3030h) [reset = 0h]

QUEUE_3_STATUS_A is shown in Figure 16-944 and described in Table 16-958.

**Figure 16-944. QUEUE_3_STATUS_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-958. QUEUE_3_STATUS_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>Queue Manager Queue N Status Registers A |

### 16.4.7.669 QUEUE_3_STATUS_B Register (offset = 3034h) [reset = 0h]

QUEUE_3_STATUS_B is shown in Figure 16-945 and described in Table 16-959.

**Figure 16-945. QUEUE_3_STATUS_B Register**

| 31-28 | 27-0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-959. QUEUE_3_STATUS_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. Queue_Manager_Queue_n_Status_B Registers B |

### 16.4.7.670 QUEUE_3_STATUS_C Register (offset = 3038h) [reset = 0h]

QUEUE_3_STATUS_C is shown in Figure 16-946 and described in Table 16-960.

**Figure 16-946. QUEUE_3_STATUS_C Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | PACKET_SIZE |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-960. QUEUE_3_STATUS_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | PACKET_SIZE | R-0 | 0 | This field indicates packet size of the head element of a queue. Queue_Manager_Queue_N_Status_C Registers C |

### 16.4.7.671 QUEUE_4_STATUS_A Register (offset = 3040h) [reset = 0h]

QUEUE_4_STATUS_A is shown in Figure 16-947 and described in Table 16-961.

**Figure 16-947. QUEUE_4_STATUS_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-961. QUEUE_4_STATUS_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>Queue Manager Queue N Status Registers A |

### 16.4.7.672 QUEUE_4_STATUS_B Register (offset = 3044h) [reset = 0h]

QUEUE_4_STATUS_B is shown in Figure 16-948 and described in Table 16-962.

**Figure 16-948. QUEUE_4_STATUS_B Register**

| 31-28 | 27-0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-962. QUEUE_4_STATUS_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. Queue_Manager_Queue_n_Status_B Registers B |

### 16.4.7.673 QUEUE_4_STATUS_C Register (offset = 3048h) [reset = 0h]

QUEUE_4_STATUS_C is shown in Figure 16-949 and described in Table 16-963.

**Figure 16-949. QUEUE_4_STATUS_C Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | PACKET_SIZE |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-963. QUEUE_4_STATUS_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | PACKET_SIZE | R-0 | 0 | This field indicates packet size of the head element of a queue. Queue_Manager_Queue_N_Status_C Registers C |

### 16.4.7.674 QUEUE_5_STATUS_A Register (offset = 3050h) [reset = 0h]

QUEUE_5_STATUS_A is shown in Figure 16-950 and described in Table 16-964.

**Figure 16-950. QUEUE_5_STATUS_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-964. QUEUE_5_STATUS_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>Queue Manager Queue N Status Registers A |

### 16.4.7.675 QUEUE_5_STATUS_B Register (offset = 3054h) [reset = 0h]

QUEUE_5_STATUS_B is shown in Figure 16-951 and described in Table 16-965.

**Figure 16-951. QUEUE_5_STATUS_B Register**

| 31-28 | 27-0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-965. QUEUE_5_STATUS_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. Queue_Manager_Queue_n_Status_B Registers B |

### 16.4.7.676 QUEUE_5_STATUS_C Register (offset = 3058h) [reset = 0h]

QUEUE_5_STATUS_C is shown in Figure 16-952 and described in Table 16-966.

**Figure 16-952. QUEUE_5_STATUS_C Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | PACKET_SIZE |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-966. QUEUE_5_STATUS_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | PACKET_SIZE | R-0 | 0 | This field indicates packet size of the head element of a queue. Queue_Manager_Queue_N_Status_C Registers C |

### 16.4.7.677 QUEUE_6_STATUS_A Register (offset = 3060h) [reset = 0h]

QUEUE_6_STATUS_A is shown in Figure 16-953 and described in Table 16-967.

**Figure 16-953. QUEUE_6_STATUS_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-967. QUEUE_6_STATUS_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>Queue Manager Queue N Status Registers A |

### 16.4.7.678 QUEUE_6_STATUS_B Register (offset = 3064h) [reset = 0h]

QUEUE_6_STATUS_B is shown in Figure 16-954 and described in Table 16-968.

**Figure 16-954. QUEUE_6_STATUS_B Register**

| 31–28 | 27–0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-968. QUEUE_6_STATUS_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. Queue_Manager_Queue_n_Status_B Registers B |

### 16.4.7.679 QUEUE_6_STATUS_C Register (offset = 3068h) [reset = 0h]

QUEUE_6_STATUS_C is shown in Figure 16-955 and described in Table 16-969.

**Figure 16-955. QUEUE_6_STATUS_C Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | PACKET_SIZE |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-969. QUEUE_6_STATUS_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | PACKET_SIZE | R-0 | 0 | This field indicates packet size of the head element of a queue. Queue_Manager_Queue_N_Status_C Registers C |

### 16.4.7.680 QUEUE_7_STATUS_A Register (offset = 3070h) [reset = 0h]

QUEUE_7_STATUS_A is shown in Figure 16-956 and described in Table 16-970.

**Figure 16-956. QUEUE_7_STATUS_A Register**

| 31-14 | 13-0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-970. QUEUE_7_STATUS_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>Queue Manager Queue N Status Registers A |

### 16.4.7.681 QUEUE_7_STATUS_B Register (offset = 3074h) [reset = 0h]

QUEUE_7_STATUS_B is shown in Figure 16-957 and described in Table 16-971.

**Figure 16-957. QUEUE_7_STATUS_B Register**

| 31–28 | 27–0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-971. QUEUE_7_STATUS_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. Queue_Manager_Queue_n_Status_B Registers B |

### 16.4.7.682 QUEUE_7_STATUS_C Register (offset = 3078h) [reset = 0h]

QUEUE_7_STATUS_C is shown in Figure 16-958 and described in Table 16-972.

**Figure 16-958. QUEUE_7_STATUS_C Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | PACKET_SIZE |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-972. QUEUE_7_STATUS_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | PACKET_SIZE | R-0 | 0 | This field indicates packet size of the head element of a queue. Queue_Manager_Queue_N_Status_C Registers C |

### 16.4.7.683 QUEUE_8_STATUS_A Register (offset = 3080h) [reset = 0h]

QUEUE_8_STATUS_A is shown in Figure 16-959 and described in Table 16-973.

**Figure 16-959. QUEUE_8_STATUS_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-973. QUEUE_8_STATUS_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>Queue Manager Queue N Status Registers A |

### 16.4.7.684 QUEUE_8_STATUS_B Register (offset = 3084h) [reset = 0h]

QUEUE_8_STATUS_B is shown in Figure 16-960 and described in Table 16-974.

**Figure 16-960. QUEUE_8_STATUS_B Register**

| 31–28 | 27–0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-974. QUEUE_8_STATUS_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. Queue_Manager_Queue_n_Status_B Registers B |

### 16.4.7.685 QUEUE_8_STATUS_C Register (offset = 3088h) [reset = 0h]

QUEUE_8_STATUS_C is shown in Figure 16-961 and described in Table 16-975.

**Figure 16-961. QUEUE_8_STATUS_C Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | PACKET_SIZE |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-975. QUEUE_8_STATUS_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | PACKET_SIZE | R-0 | 0 | This field indicates packet size of the head element of a queue.<br>Queue_Manager_Queue_N_Status_C Registers C |

### 16.4.7.686 QUEUE_9_STATUS_A Register (offset = 3090h) [reset = 0h]

QUEUE_9_STATUS_A is shown in Figure 16-962 and described in Table 16-976.

**Figure 16-962. QUEUE_9_STATUS_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-976. QUEUE_9_STATUS_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>Queue Manager Queue N Status Registers A |

### 16.4.7.687 QUEUE_9_STATUS_B Register (offset = 3094h) [reset = 0h]

QUEUE_9_STATUS_B is shown in Figure 16-963 and described in Table 16-977.

**Figure 16-963. QUEUE_9_STATUS_B Register**

| 31–28 | 27–0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-977. QUEUE_9_STATUS_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue.<br>Queue_Manager_Queue_n_Status_B Registers B |

### 16.4.7.688 QUEUE_9_STATUS_C Register (offset = 3098h) [reset = 0h]

QUEUE_9_STATUS_C is shown in Figure 16-964 and described in Table 16-978.

**Figure 16-964. QUEUE_9_STATUS_C Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | PACKET_SIZE |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-978. QUEUE_9_STATUS_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | PACKET_SIZE | R-0 | 0 | This field indicates packet size of the head element of a queue. Queue_Manager_Queue_N_Status_C Registers C |

### 16.4.7.689 QUEUE_10_STATUS_A Register (offset = 30A0h) [reset = 0h]

QUEUE_10_STATUS_A is shown in Figure 16-965 and described in Table 16-979.

**Figure 16-965. QUEUE_10_STATUS_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-979. QUEUE_10_STATUS_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>Queue Manager Queue N Status Registers A |

### 16.4.7.690 QUEUE_10_STATUS_B Register (offset = 30A4h) [reset = 0h]

QUEUE_10_STATUS_B is shown in Figure 16-966 and described in Table 16-980.

**Figure 16-966. QUEUE_10_STATUS_B Register**

| 31–28 | 27–0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-980. QUEUE_10_STATUS_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. Queue_Manager_Queue_n_Status_B Registers B |

### 16.4.7.691 QUEUE_10_STATUS_C Register (offset = 30A8h) [reset = 0h]

QUEUE_10_STATUS_C is shown in Figure 16-967 and described in Table 16-981.

**Figure 16-967. QUEUE_10_STATUS_C Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | PACKET_SIZE |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-981. QUEUE_10_STATUS_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | PACKET_SIZE | R-0 | 0 | This field indicates packet size of the head element of a queue. Queue_Manager_Queue_N_Status_C Registers C |

### 16.4.7.692 QUEUE_11_STATUS_A Register (offset = 30B0h) [reset = 0h]

QUEUE_11_STATUS_A is shown in Figure 16-968 and described in Table 16-982.

**Figure 16-968. QUEUE_11_STATUS_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-982. QUEUE_11_STATUS_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>Queue Manager Queue N Status Registers A |

### 16.4.7.693 QUEUE_11_STATUS_B Register (offset = 30B4h) [reset = 0h]

QUEUE_11_STATUS_B is shown in Figure 16-969 and described in Table 16-983.

**Figure 16-969. QUEUE_11_STATUS_B Register**

| 31-28 | 27-0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-983. QUEUE_11_STATUS_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. Queue_Manager_Queue_n_Status_B Registers B |

### 16.4.7.694 QUEUE_11_STATUS_C Register (offset = 30B8h) [reset = 0h]

QUEUE_11_STATUS_C is shown in Figure 16-970 and described in Table 16-984.

**Figure 16-970. QUEUE_11_STATUS_C Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | PACKET_SIZE |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-984. QUEUE_11_STATUS_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | PACKET_SIZE | R-0 | 0 | This field indicates packet size of the head element of a queue. Queue_Manager_Queue_N_Status_C Registers C |

### 16.4.7.695 QUEUE_12_STATUS_A Register (offset = 30C0h) [reset = 0h]

QUEUE_12_STATUS_A is shown in Figure 16-971 and described in Table 16-985.

**Figure 16-971. QUEUE_12_STATUS_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-985. QUEUE_12_STATUS_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>Queue Manager Queue N Status Registers A |

### 16.4.7.696 QUEUE_12_STATUS_B Register (offset = 30C4h) [reset = 0h]

QUEUE_12_STATUS_B is shown in Figure 16-972 and described in Table 16-986.

**Figure 16-972. QUEUE_12_STATUS_B Register**

| 31–28 | 27–0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-986. QUEUE_12_STATUS_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. Queue_Manager_Queue_n_Status_B Registers B |

### 16.4.7.697 QUEUE_12_STATUS_C Register (offset = 30C8h) [reset = 0h]

QUEUE_12_STATUS_C is shown in Figure 16-973 and described in Table 16-987.

**Figure 16-973. QUEUE_12_STATUS_C Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | PACKET_SIZE |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-987. QUEUE_12_STATUS_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | PACKET_SIZE | R-0 | 0 | This field indicates packet size of the head element of a queue. Queue_Manager_Queue_N_Status_C Registers C |

### 16.4.7.698 QUEUE_13_STATUS_A Register (offset = 30D0h) [reset = 0h]

QUEUE_13_STATUS_A is shown in Figure 16-974 and described in Table 16-988.

**Figure 16-974. QUEUE_13_STATUS_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-988. QUEUE_13_STATUS_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>Queue Manager Queue N Status Registers A |

### 16.4.7.699 QUEUE_13_STATUS_B Register (offset = 30D4h) [reset = 0h]

QUEUE_13_STATUS_B is shown in Figure 16-975 and described in Table 16-989.

**Figure 16-975. QUEUE_13_STATUS_B Register**

| 31–28 | 27–0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-989. QUEUE_13_STATUS_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. Queue_Manager_Queue_n_Status_B Registers B |

### 16.4.7.700 QUEUE_13_STATUS_C Register (offset = 30D8h) [reset = 0h]

QUEUE_13_STATUS_C is shown in Figure 16-976 and described in Table 16-990.

**Figure 16-976. QUEUE_13_STATUS_C Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | PACKET_SIZE |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-990. QUEUE_13_STATUS_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | PACKET_SIZE | R-0 | 0 | This field indicates packet size of the head element of a queue. Queue_Manager_Queue_N_Status_C Registers C |

### 16.4.7.701 QUEUE_14_STATUS_A Register (offset = 30E0h) [reset = 0h]

QUEUE_14_STATUS_A is shown in Figure 16-977 and described in Table 16-991.

**Figure 16-977. QUEUE_14_STATUS_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-991. QUEUE_14_STATUS_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>Queue Manager Queue N Status Registers A |

### 16.4.7.702 QUEUE_14_STATUS_B Register (offset = 30E4h) [reset = 0h]

QUEUE_14_STATUS_B is shown in Figure 16-978 and described in Table 16-992.

**Figure 16-978. QUEUE_14_STATUS_B Register**

| 31–28 | 27–0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-992. QUEUE_14_STATUS_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue.<br>Queue_Manager_Queue_n_Status_B Registers B |

### 16.4.7.703 QUEUE_14_STATUS_C Register (offset = 30E8h) [reset = 0h]

QUEUE_14_STATUS_C is shown in Figure 16-979 and described in Table 16-993.

**Figure 16-979. QUEUE_14_STATUS_C Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | PACKET_SIZE |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-993. QUEUE_14_STATUS_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | PACKET_SIZE | R-0 | 0 | This field indicates packet size of the head element of a queue.<br>Queue_Manager_Queue_N_Status_C Registers C |

### 16.4.7.704 QUEUE_15_STATUS_A Register (offset = 30F0h) [reset = 0h]

QUEUE_15_STATUS_A is shown in Figure 16-980 and described in Table 16-994.

**Figure 16-980. QUEUE_15_STATUS_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-994. QUEUE_15_STATUS_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>Queue Manager Queue N Status Registers A |

### 16.4.7.705 QUEUE_15_STATUS_B Register (offset = 30F4h) [reset = 0h]

QUEUE_15_STATUS_B is shown in Figure 16-981 and described in Table 16-995.

**Figure 16-981. QUEUE_15_STATUS_B Register**

| 31-28 | 27-0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-995. QUEUE_15_STATUS_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. Queue_Manager_Queue_n_Status_B Registers B |

### 16.4.7.706 QUEUE_15_STATUS_C Register (offset = 30F8h) [reset = 0h]

QUEUE_15_STATUS_C is shown in Figure 16-982 and described in Table 16-996.

**Figure 16-982. QUEUE_15_STATUS_C Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | PACKET_SIZE |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-996. QUEUE_15_STATUS_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | PACKET_SIZE | R-0 | 0 | This field indicates packet size of the head element of a queue. Queue_Manager_Queue_N_Status_C Registers C |

### 16.4.7.707 QUEUE_16_STATUS_A Register (offset = 3100h) [reset = 0h]

QUEUE_16_STATUS_A is shown in Figure 16-983 and described in Table 16-997.

**Figure 16-983. QUEUE_16_STATUS_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-997. QUEUE_16_STATUS_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>Queue Manager Queue N Status Registers A |

### 16.4.7.708 QUEUE_16_STATUS_B Register (offset = 3104h) [reset = 0h]

QUEUE_16_STATUS_B is shown in Figure 16-984 and described in Table 16-998.

**Figure 16-984. QUEUE_16_STATUS_B Register**

| 31–28 | 27–0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-998. QUEUE_16_STATUS_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. Queue_Manager_Queue_n_Status_B Registers B |

### 16.4.7.709 QUEUE_16_STATUS_C Register (offset = 3108h) [reset = 0h]

QUEUE_16_STATUS_C is shown in Figure 16-985 and described in Table 16-999.

**Figure 16-985. QUEUE_16_STATUS_C Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | PACKET_SIZE |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-999. QUEUE_16_STATUS_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | PACKET_SIZE | R-0 | 0 | This field indicates packet size of the head element of a queue. Queue_Manager_Queue_N_Status_C Registers C |

### 16.4.7.710 QUEUE_17_STATUS_A Register (offset = 3110h) [reset = 0h]

QUEUE_17_STATUS_A is shown in Figure 16-986 and described in Table 16-1000.

**Figure 16-986. QUEUE_17_STATUS_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-1000. QUEUE_17_STATUS_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>Queue Manager Queue N Status Registers A |

### 16.4.7.711 QUEUE_17_STATUS_B Register (offset = 3114h) [reset = 0h]

QUEUE_17_STATUS_B is shown in Figure 16-987 and described in Table 16-1001.

**Figure 16-987. QUEUE_17_STATUS_B Register**

| 31–28 | 27–0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-1001. QUEUE_17_STATUS_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. Queue_Manager_Queue_n_Status_B Registers B |

### 16.4.7.712 QUEUE_17_STATUS_C Register (offset = 3118h) [reset = 0h]

QUEUE_17_STATUS_C is shown in Figure 16-988 and described in Table 16-1002.

**Figure 16-988. QUEUE_17_STATUS_C Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | PACKET_SIZE |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-1002. QUEUE_17_STATUS_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | PACKET_SIZE | R-0 | 0 | This field indicates packet size of the head element of a queue. Queue_Manager_Queue_N_Status_C Registers C |

### 16.4.7.713 QUEUE_18_STATUS_A Register (offset = 3120h) [reset = 0h]

QUEUE_18_STATUS_A is shown in Figure 16-989 and described in Table 16-1003.

**Figure 16-989. QUEUE_18_STATUS_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-1003. QUEUE_18_STATUS_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>Queue Manager Queue N Status Registers A |

### 16.4.7.714 QUEUE_18_STATUS_B Register (offset = 3124h) [reset = 0h]

QUEUE_18_STATUS_B is shown in Figure 16-990 and described in Table 16-1004.

**Figure 16-990. QUEUE_18_STATUS_B Register**

| 31–28 | 27–0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-1004. QUEUE_18_STATUS_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. Queue_Manager_Queue_n_Status_B Registers B |

### 16.4.7.715 QUEUE_18_STATUS_C Register (offset = 3128h) [reset = 0h]

QUEUE_18_STATUS_C is shown in Figure 16-991 and described in Table 16-1005.

**Figure 16-991. QUEUE_18_STATUS_C Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | PACKET_SIZE |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-1005. QUEUE_18_STATUS_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | PACKET_SIZE | R-0 | 0 | This field indicates packet size of the head element of a queue. Queue_Manager_Queue_N_Status_C Registers C |

### 16.4.7.716 QUEUE_19_STATUS_A Register (offset = 3130h) [reset = 0h]

QUEUE_19_STATUS_A is shown in Figure 16-992 and described in Table 16-1006.

**Figure 16-992. QUEUE_19_STATUS_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-1006. QUEUE_19_STATUS_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>Queue Manager Queue N Status Registers A |

### 16.4.7.717 QUEUE_19_STATUS_B Register (offset = 3134h) [reset = 0h]

QUEUE_19_STATUS_B is shown in Figure 16-993 and described in Table 16-1007.

**Figure 16-993. QUEUE_19_STATUS_B Register**

| 31-28 | 27-0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-1007. QUEUE_19_STATUS_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. Queue_Manager_Queue_n_Status_B Registers B |

### 16.4.7.718 QUEUE_19_STATUS_C Register (offset = 3138h) [reset = 0h]

QUEUE_19_STATUS_C is shown in Figure 16-994 and described in Table 16-1008.

**Figure 16-994. QUEUE_19_STATUS_C Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | PACKET_SIZE |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-1008. QUEUE_19_STATUS_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | PACKET_SIZE | R-0 | 0 | This field indicates packet size of the head element of a queue. Queue_Manager_Queue_N_Status_C Registers C |

### 16.4.7.719 QUEUE_20_STATUS_A Register (offset = 3140h) [reset = 0h]

QUEUE_20_STATUS_A is shown in Figure 16-995 and described in Table 16-1009.

**Figure 16-995. QUEUE_20_STATUS_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-1009. QUEUE_20_STATUS_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>Queue Manager Queue N Status Registers A |

### 16.4.7.720 QUEUE_20_STATUS_B Register (offset = 3144h) [reset = 0h]

QUEUE_20_STATUS_B is shown in Figure 16-996 and described in Table 16-1010.

**Figure 16-996. QUEUE_20_STATUS_B Register**

| 31-28 | 27-0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-1010. QUEUE_20_STATUS_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. Queue_Manager_Queue_n_Status_B Registers B |

### 16.4.7.721 QUEUE_20_STATUS_C Register (offset = 3148h) [reset = 0h]

QUEUE_20_STATUS_C is shown in Figure 16-997 and described in Table 16-1011.

**Figure 16-997. QUEUE_20_STATUS_C Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | PACKET_SIZE |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-1011. QUEUE_20_STATUS_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | PACKET_SIZE | R-0 | 0 | This field indicates packet size of the head element of a queue. Queue_Manager_Queue_N_Status_C Registers C |

### 16.4.7.722 QUEUE_21_STATUS_A Register (offset = 3150h) [reset = 0h]

QUEUE_21_STATUS_A is shown in Figure 16-998 and described in Table 16-1012.

**Figure 16-998. QUEUE_21_STATUS_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-1012. QUEUE_21_STATUS_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>Queue Manager Queue N Status Registers A |

### 16.4.7.723 QUEUE_21_STATUS_B Register (offset = 3154h) [reset = 0h]

QUEUE_21_STATUS_B is shown in Figure 16-999 and described in Table 16-1013.

**Figure 16-999. QUEUE_21_STATUS_B Register**

| 31–28 | 27–0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-1013. QUEUE_21_STATUS_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. Queue_Manager_Queue_n_Status_B Registers B |

### 16.4.7.724 QUEUE_21_STATUS_C Register (offset = 3158h) [reset = 0h]

QUEUE_21_STATUS_C is shown in Figure 16-1000 and described in Table 16-1014.

**Figure 16-1000. QUEUE_21_STATUS_C Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | PACKET_SIZE |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-1014. QUEUE_21_STATUS_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | PACKET_SIZE | R-0 | 0 | This field indicates packet size of the head element of a queue. Queue_Manager_Queue_N_Status_C Registers C |

### 16.4.7.725 QUEUE_22_STATUS_A Register (offset = 3160h) [reset = 0h]

QUEUE_22_STATUS_A is shown in Figure 16-1001 and described in Table 16-1015.

**Figure 16-1001. QUEUE_22_STATUS_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-1015. QUEUE_22_STATUS_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>Queue Manager Queue N Status Registers A |

### 16.4.7.726 QUEUE_22_STATUS_B Register (offset = 3164h) [reset = 0h]

QUEUE_22_STATUS_B is shown in Figure 16-1002 and described in Table 16-1016.

**Figure 16-1002. QUEUE_22_STATUS_B Register**

| 31–28 | 27–0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-1016. QUEUE_22_STATUS_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. Queue_Manager_Queue_n_Status_B Registers B |

### 16.4.7.727 QUEUE_22_STATUS_C Register (offset = 3168h) [reset = 0h]

QUEUE_22_STATUS_C is shown in Figure 16-1003 and described in Table 16-1017.

**Figure 16-1003. QUEUE_22_STATUS_C Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | PACKET_SIZE |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-1017. QUEUE_22_STATUS_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | PACKET_SIZE | R-0 | 0 | This field indicates packet size of the head element of a queue.<br>Queue_Manager_Queue_N_Status_C Registers C |

### 16.4.7.728 QUEUE_23_STATUS_A Register (offset = 3170h) [reset = 0h]

QUEUE_23_STATUS_A is shown in Figure 16-1004 and described in Table 16-1018.

**Figure 16-1004. QUEUE_23_STATUS_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-1018. QUEUE_23_STATUS_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>Queue Manager Queue N Status Registers A |

### 16.4.7.729 QUEUE_23_STATUS_B Register (offset = 3174h) [reset = 0h]

QUEUE_23_STATUS_B is shown in Figure 16-1005 and described in Table 16-1019.

**Figure 16-1005. QUEUE_23_STATUS_B Register**

| 31–28 | 27–0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-1019. QUEUE_23_STATUS_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. Queue_Manager_Queue_n_Status_B Registers B |

### 16.4.7.730 QUEUE_23_STATUS_C Register (offset = 3178h) [reset = 0h]

QUEUE_23_STATUS_C is shown in Figure 16-1006 and described in Table 16-1020.

**Figure 16-1006. QUEUE_23_STATUS_C Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | PACKET_SIZE |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-1020. QUEUE_23_STATUS_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | PACKET_SIZE | R-0 | 0 | This field indicates packet size of the head element of a queue. Queue_Manager_Queue_N_Status_C Registers C |

### 16.4.7.731 QUEUE_24_STATUS_A Register (offset = 3180h) [reset = 0h]

QUEUE_24_STATUS_A is shown in Figure 16-1007 and described in Table 16-1021.

**Figure 16-1007. QUEUE_24_STATUS_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-1021. QUEUE_24_STATUS_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>Queue Manager Queue N Status Registers A |

### 16.4.7.732 QUEUE_24_STATUS_B Register (offset = 3184h) [reset = 0h]

QUEUE_24_STATUS_B is shown in Figure 16-1008 and described in Table 16-1022.

**Figure 16-1008. QUEUE_24_STATUS_B Register**

| 31–28 | 27–0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-1022. QUEUE_24_STATUS_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. Queue_Manager_Queue_n_Status_B Registers B |

### 16.4.7.733 QUEUE_24_STATUS_C Register (offset = 3188h) [reset = 0h]

QUEUE_24_STATUS_C is shown in Figure 16-1009 and described in Table 16-1023.

**Figure 16-1009. QUEUE_24_STATUS_C Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | PACKET_SIZE |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-1023. QUEUE_24_STATUS_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | PACKET_SIZE | R-0 | 0 | This field indicates packet size of the head element of a queue.<br>Queue_Manager_Queue_N_Status_C Registers C |

### 16.4.7.734 QUEUE_25_STATUS_A Register (offset = 3190h) [reset = 0h]

QUEUE_25_STATUS_A is shown in Figure 16-1010 and described in Table 16-1024.

**Figure 16-1010. QUEUE_25_STATUS_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-1024. QUEUE_25_STATUS_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>Queue Manager Queue N Status Registers A |

### 16.4.7.735 QUEUE_25_STATUS_B Register (offset = 3194h) [reset = 0h]

QUEUE_25_STATUS_B is shown in Figure 16-1011 and described in Table 16-1025.

**Figure 16-1011. QUEUE_25_STATUS_B Register**

| 31–28 | 27–0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-1025. QUEUE_25_STATUS_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. Queue_Manager_Queue_n_Status_B Registers B |

### 16.4.7.736 QUEUE_25_STATUS_C Register (offset = 3198h) [reset = 0h]

QUEUE_25_STATUS_C is shown in Figure 16-1012 and described in Table 16-1026.

**Figure 16-1012. QUEUE_25_STATUS_C Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | PACKET_SIZE |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-1026. QUEUE_25_STATUS_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | PACKET_SIZE | R-0 | 0 | This field indicates packet size of the head element of a queue. Queue_Manager_Queue_N_Status_C Registers C |

### 16.4.7.737 QUEUE_26_STATUS_A Register (offset = 31A0h) [reset = 0h]

QUEUE_26_STATUS_A is shown in Figure 16-1013 and described in Table 16-1027.

**Figure 16-1013. QUEUE_26_STATUS_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-1027. QUEUE_26_STATUS_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>Queue Manager Queue N Status Registers A |

### 16.4.7.738 QUEUE_26_STATUS_B Register (offset = 31A4h) [reset = 0h]

QUEUE_26_STATUS_B is shown in Figure 16-1014 and described in Table 16-1028.

**Figure 16-1014. QUEUE_26_STATUS_B Register**

| 31–28 | 27–0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-1028. QUEUE_26_STATUS_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. Queue_Manager_Queue_n_Status_B Registers B |

### 16.4.7.739 QUEUE_26_STATUS_C Register (offset = 31A8h) [reset = 0h]

QUEUE_26_STATUS_C is shown in Figure 16-1015 and described in Table 16-1029.

**Figure 16-1015. QUEUE_26_STATUS_C Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | PACKET_SIZE |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-1029. QUEUE_26_STATUS_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | PACKET_SIZE | R-0 | 0 | This field indicates packet size of the head element of a queue. Queue_Manager_Queue_N_Status_C Registers C |

#### 16.4.7.740 QUEUE_27_STATUS_A Register (offset = 31B0h) [reset = 0h]

QUEUE_27_STATUS_A is shown in Figure 16-1016 and described in Table 16-1030.

**Figure 16-1016. QUEUE_27_STATUS_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-1030. QUEUE_27_STATUS_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>Queue Manager Queue N Status Registers A |

### 16.4.7.741 QUEUE_27_STATUS_B Register (offset = 31B4h) [reset = 0h]

QUEUE_27_STATUS_B is shown in Figure 16-1017 and described in Table 16-1031.

**Figure 16-1017. QUEUE_27_STATUS_B Register**

| 31-28 | 27-0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-1031. QUEUE_27_STATUS_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. Queue_Manager_Queue_n_Status_B Registers B |

### 16.4.7.742 QUEUE_27_STATUS_C Register (offset = 31B8h) [reset = 0h]

QUEUE_27_STATUS_C is shown in Figure 16-1018 and described in Table 16-1032.

**Figure 16-1018. QUEUE_27_STATUS_C Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | PACKET_SIZE |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-1032. QUEUE_27_STATUS_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | PACKET_SIZE | R-0 | 0 | This field indicates packet size of the head element of a queue.<br>Queue_Manager_Queue_N_Status_C Registers C |

### 16.4.7.743 QUEUE_28_STATUS_A Register (offset = 31C0h) [reset = 0h]

QUEUE_28_STATUS_A is shown in Figure 16-1019 and described in Table 16-1033.

**Figure 16-1019. QUEUE_28_STATUS_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-1033. QUEUE_28_STATUS_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>Queue Manager Queue N Status Registers A |

### 16.4.7.744 QUEUE_28_STATUS_B Register (offset = 31C4h) [reset = 0h]

QUEUE_28_STATUS_B is shown in Figure 16-1020 and described in Table 16-1034.

**Figure 16-1020. QUEUE_28_STATUS_B Register**

| 31–28 | 27–0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-1034. QUEUE_28_STATUS_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. Queue_Manager_Queue_n_Status_B Registers B |

### 16.4.7.745 QUEUE_28_STATUS_C Register (offset = 31C8h) [reset = 0h]

QUEUE_28_STATUS_C is shown in Figure 16-1021 and described in Table 16-1035.

**Figure 16-1021. QUEUE_28_STATUS_C Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | PACKET_SIZE |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-1035. QUEUE_28_STATUS_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | PACKET_SIZE | R-0 | 0 | This field indicates packet size of the head element of a queue. Queue_Manager_Queue_N_Status_C Registers C |

### 16.4.7.746 QUEUE_29_STATUS_A Register (offset = 31D0h) [reset = 0h]

QUEUE_29_STATUS_A is shown in Figure 16-1022 and described in Table 16-1036.

**Figure 16-1022. QUEUE_29_STATUS_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-1036. QUEUE_29_STATUS_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>Queue Manager Queue N Status Registers A |

### 16.4.7.747 QUEUE_29_STATUS_B Register (offset = 31D4h) [reset = 0h]

QUEUE_29_STATUS_B is shown in Figure 16-1023 and described in Table 16-1037.

**Figure 16-1023. QUEUE_29_STATUS_B Register**

| 31-28 | 27-0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-1037. QUEUE_29_STATUS_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. Queue_Manager_Queue_n_Status_B Registers B |

### 16.4.7.748 QUEUE_29_STATUS_C Register (offset = 31D8h) [reset = 0h]

QUEUE_29_STATUS_C is shown in Figure 16-1024 and described in Table 16-1038.

**Figure 16-1024. QUEUE_29_STATUS_C Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | PACKET_SIZE |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-1038. QUEUE_29_STATUS_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | PACKET_SIZE | R-0 | 0 | This field indicates packet size of the head element of a queue. Queue_Manager_Queue_N_Status_C Registers C |

### 16.4.7.749 QUEUE_30_STATUS_A Register (offset = 31E0h) [reset = 0h]

QUEUE_30_STATUS_A is shown in Figure 16-1025 and described in Table 16-1039.

**Figure 16-1025. QUEUE_30_STATUS_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-1039. QUEUE_30_STATUS_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>Queue Manager Queue N Status Registers A |

### 16.4.7.750 QUEUE_30_STATUS_B Register (offset = 31E4h) [reset = 0h]

QUEUE_30_STATUS_B is shown in Figure 16-1026 and described in Table 16-1040.

**Figure 16-1026. QUEUE_30_STATUS_B Register**

| 31–28 | 27–0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-1040. QUEUE_30_STATUS_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. Queue_Manager_Queue_n_Status_B Registers B |

### 16.4.7.751 QUEUE_30_STATUS_C Register (offset = 31E8h) [reset = 0h]

QUEUE_30_STATUS_C is shown in Figure 16-1027 and described in Table 16-1041.

**Figure 16-1027. QUEUE_30_STATUS_C Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | PACKET_SIZE |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-1041. QUEUE_30_STATUS_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | PACKET_SIZE | R-0 | 0 | This field indicates packet size of the head element of a queue. Queue_Manager_Queue_N_Status_C Registers C |

### 16.4.7.752 QUEUE_31_STATUS_A Register (offset = 31F0h) [reset = 0h]

QUEUE_31_STATUS_A is shown in Figure 16-1028 and described in Table 16-1042.

**Figure 16-1028. QUEUE_31_STATUS_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-1042. QUEUE_31_STATUS_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>Queue Manager Queue N Status Registers A |

### 16.4.7.753 QUEUE_31_STATUS_B Register (offset = 31F4h) [reset = 0h]

QUEUE_31_STATUS_B is shown in Figure 16-1029 and described in Table 16-1043.

**Figure 16-1029. QUEUE_31_STATUS_B Register**

| 31–28 | 27–0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-1043. QUEUE_31_STATUS_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. Queue_Manager_Queue_n_Status_B Registers B |

### 16.4.7.754 QUEUE_31_STATUS_C Register (offset = 31F8h) [reset = 0h]

QUEUE_31_STATUS_C is shown in Figure 16-1030 and described in Table 16-1044.

**Figure 16-1030. QUEUE_31_STATUS_C Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | PACKET_SIZE |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-1044. QUEUE_31_STATUS_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | PACKET_SIZE | R-0 | 0 | This field indicates packet size of the head element of a queue. Queue_Manager_Queue_N_Status_C Registers C |

### 16.4.7.755 QUEUE_32_STATUS_A Register (offset = 3200h) [reset = 0h]

QUEUE_32_STATUS_A is shown in Figure 16-1031 and described in Table 16-1045.

**Figure 16-1031. QUEUE_32_STATUS_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-1045. QUEUE_32_STATUS_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>Queue Manager Queue N Status Registers A |

### 16.4.7.756 QUEUE_32_STATUS_B Register (offset = 3204h) [reset = 0h]

QUEUE_32_STATUS_B is shown in Figure 16-1032 and described in Table 16-1046.

**Figure 16-1032. QUEUE_32_STATUS_B Register**

| 31–28 | 27–0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-1046. QUEUE_32_STATUS_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. Queue_Manager_Queue_n_Status_B Registers B |

### 16.4.7.757 QUEUE_32_STATUS_C Register (offset = 3208h) [reset = 0h]

QUEUE_32_STATUS_C is shown in Figure 16-1033 and described in Table 16-1047.

**Figure 16-1033. QUEUE_32_STATUS_C Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | PACKET_SIZE |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-1047. QUEUE_32_STATUS_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | PACKET_SIZE | R-0 | 0 | This field indicates packet size of the head element of a queue. Queue_Manager_Queue_N_Status_C Registers C |

### 16.4.7.758 QUEUE_33_STATUS_A Register (offset = 3210h) [reset = 0h]

QUEUE_33_STATUS_A is shown in Figure 16-1034 and described in Table 16-1048.

**Figure 16-1034. QUEUE_33_STATUS_A Register**

| 31–14 | 13–0 |
| --- | --- |
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-1048. QUEUE_33_STATUS_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
| --- | --- | --- | --- | --- |
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>Queue Manager Queue N Status Registers A |

### 16.4.7.759 QUEUE_33_STATUS_B Register (offset = 3214h) [reset = 0h]

QUEUE_33_STATUS_B is shown in Figure 16-1035 and described in Table 16-1049.

**Figure 16-1035. QUEUE_33_STATUS_B Register**

| 31-28 | 27-0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-1049. QUEUE_33_STATUS_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. Queue_Manager_Queue_n_Status_B Registers B |

### 16.4.7.760 QUEUE_33_STATUS_C Register (offset = 3218h) [reset = 0h]

QUEUE_33_STATUS_C is shown in Figure 16-1036 and described in Table 16-1050.

**Figure 16-1036. QUEUE_33_STATUS_C Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | PACKET_SIZE |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-1050. QUEUE_33_STATUS_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | PACKET_SIZE | R-0 | 0 | This field indicates packet size of the head element of a queue. Queue_Manager_Queue_N_Status_C Registers C |

### 16.4.7.761 QUEUE_34_STATUS_A Register (offset = 3220h) [reset = 0h]

QUEUE_34_STATUS_A is shown in Figure 16-1037 and described in Table 16-1051.

**Figure 16-1037. QUEUE_34_STATUS_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-1051. QUEUE_34_STATUS_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>Queue Manager Queue N Status Registers A |

### 16.4.7.762 QUEUE_34_STATUS_B Register (offset = 3224h) [reset = 0h]

QUEUE_34_STATUS_B is shown in Figure 16-1038 and described in Table 16-1052.

**Figure 16-1038. QUEUE_34_STATUS_B Register**

| 31–28 | 27–0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-1052. QUEUE_34_STATUS_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. Queue_Manager_Queue_n_Status_B Registers B |

### 16.4.7.763 QUEUE_34_STATUS_C Register (offset = 3228h) [reset = 0h]

QUEUE_34_STATUS_C is shown in Figure 16-1039 and described in Table 16-1053.

**Figure 16-1039. QUEUE_34_STATUS_C Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | PACKET_SIZE |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-1053. QUEUE_34_STATUS_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | PACKET_SIZE | R-0 | 0 | This field indicates packet size of the head element of a queue. Queue_Manager_Queue_N_Status_C Registers C |

### 16.4.7.764 QUEUE_35_STATUS_A Register (offset = 3230h) [reset = 0h]

QUEUE_35_STATUS_A is shown in Figure 16-1040 and described in Table 16-1054.

**Figure 16-1040. QUEUE_35_STATUS_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-1054. QUEUE_35_STATUS_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>Queue Manager Queue N Status Registers A |

### 16.4.7.765 QUEUE_35_STATUS_B Register (offset = 3234h) [reset = 0h]

QUEUE_35_STATUS_B is shown in Figure 16-1041 and described in Table 16-1055.

**Figure 16-1041. QUEUE_35_STATUS_B Register**

| 31–28 | 27–0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-1055. QUEUE_35_STATUS_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. Queue_Manager_Queue_n_Status_B Registers B |

### 16.4.7.766 QUEUE_35_STATUS_C Register (offset = 3238h) [reset = 0h]

QUEUE_35_STATUS_C is shown in Figure 16-1042 and described in Table 16-1056.

**Figure 16-1042. QUEUE_35_STATUS_C Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | PACKET_SIZE |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-1056. QUEUE_35_STATUS_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | PACKET_SIZE | R-0 | 0 | This field indicates packet size of the head element of a queue. Queue_Manager_Queue_N_Status_C Registers C |

### 16.4.7.767 QUEUE_36_STATUS_A Register (offset = 3240h) [reset = 0h]

QUEUE_36_STATUS_A is shown in Figure 16-1043 and described in Table 16-1057.

**Figure 16-1043. QUEUE_36_STATUS_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-1057. QUEUE_36_STATUS_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>Queue Manager Queue N Status Registers A |

### 16.4.7.768 QUEUE_36_STATUS_B Register (offset = 3244h) [reset = 0h]

QUEUE_36_STATUS_B is shown in Figure 16-1044 and described in Table 16-1058.

**Figure 16-1044. QUEUE_36_STATUS_B Register**

| 31–28 | 27–0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-1058. QUEUE_36_STATUS_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. Queue_Manager_Queue_n_Status_B Registers B |

### 16.4.7.769 QUEUE_36_STATUS_C Register (offset = 3248h) [reset = 0h]

QUEUE_36_STATUS_C is shown in Figure 16-1045 and described in Table 16-1059.

**Figure 16-1045. QUEUE_36_STATUS_C Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | PACKET_SIZE |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-1059. QUEUE_36_STATUS_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | PACKET_SIZE | R-0 | 0 | This field indicates packet size of the head element of a queue.<br>Queue_Manager_Queue_N_Status_C Registers C |

### 16.4.7.770 QUEUE_37_STATUS_A Register (offset = 3250h) [reset = 0h]

QUEUE_37_STATUS_A is shown in Figure 16-1046 and described in Table 16-1060.

**Figure 16-1046. QUEUE_37_STATUS_A Register**

| 31-14 | 13-0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-1060. QUEUE_37_STATUS_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>Queue Manager Queue N Status Registers A |

### 16.4.7.771 QUEUE_37_STATUS_B Register (offset = 3254h) [reset = 0h]

QUEUE_37_STATUS_B is shown in Figure 16-1047 and described in Table 16-1061.

**Figure 16-1047. QUEUE_37_STATUS_B Register**

| 31–28 | 27–0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-1061. QUEUE_37_STATUS_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. Queue_Manager_Queue_n_Status_B Registers B |

### 16.4.7.772 QUEUE_37_STATUS_C Register (offset = 3258h) [reset = 0h]

QUEUE_37_STATUS_C is shown in Figure 16-1048 and described in Table 16-1062.

**Figure 16-1048. QUEUE_37_STATUS_C Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | PACKET_SIZE |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-1062. QUEUE_37_STATUS_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | PACKET_SIZE | R-0 | 0 | This field indicates packet size of the head element of a queue. Queue_Manager_Queue_N_Status_C Registers C |

### 16.4.7.773 QUEUE_38_STATUS_A Register (offset = 3260h) [reset = 0h]

QUEUE_38_STATUS_A is shown in Figure 16-1049 and described in Table 16-1063.

**Figure 16-1049. QUEUE_38_STATUS_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-1063. QUEUE_38_STATUS_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>Queue Manager Queue N Status Registers A |

### 16.4.7.774 QUEUE_38_STATUS_B Register (offset = 3264h) [reset = 0h]

QUEUE_38_STATUS_B is shown in Figure 16-1050 and described in Table 16-1064.

**Figure 16-1050. QUEUE_38_STATUS_B Register**

| 31–28 | 27–0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-1064. QUEUE_38_STATUS_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. Queue_Manager_Queue_n_Status_B Registers B |

### 16.4.7.775 QUEUE_38_STATUS_C Register (offset = 3268h) [reset = 0h]

QUEUE_38_STATUS_C is shown in Figure 16-1051 and described in Table 16-1065.

**Figure 16-1051. QUEUE_38_STATUS_C Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | PACKET_SIZE |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-1065. QUEUE_38_STATUS_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | PACKET_SIZE | R-0 | 0 | This field indicates packet size of the head element of a queue.<br>Queue_Manager_Queue_N_Status_C Registers C |

### 16.4.7.776 QUEUE_39_STATUS_A Register (offset = 3270h) [reset = 0h]

QUEUE_39_STATUS_A is shown in Figure 16-1052 and described in Table 16-1066.

**Figure 16-1052. QUEUE_39_STATUS_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-1066. QUEUE_39_STATUS_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>Queue Manager Queue N Status Registers A |

### 16.4.7.777 QUEUE_39_STATUS_B Register (offset = 3274h) [reset = 0h]

QUEUE_39_STATUS_B is shown in Figure 16-1053 and described in Table 16-1067.

**Figure 16-1053. QUEUE_39_STATUS_B Register**

| 31–28 | 27–0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-1067. QUEUE_39_STATUS_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. Queue_Manager_Queue_n_Status_B Registers B |

### 16.4.7.778 QUEUE_39_STATUS_C Register (offset = 3278h) [reset = 0h]

QUEUE_39_STATUS_C is shown in Figure 16-1054 and described in Table 16-1068.

**Figure 16-1054. QUEUE_39_STATUS_C Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | PACKET_SIZE |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-1068. QUEUE_39_STATUS_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | PACKET_SIZE | R-0 | 0 | This field indicates packet size of the head element of a queue. Queue_Manager_Queue_N_Status_C Registers C |

### 16.4.7.779 QUEUE_40_STATUS_A Register (offset = 3280h) [reset = 0h]

QUEUE_40_STATUS_A is shown in Figure 16-1055 and described in Table 16-1069.

**Figure 16-1055. QUEUE_40_STATUS_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-1069. QUEUE_40_STATUS_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>Queue Manager Queue N Status Registers A |

### 16.4.7.780 QUEUE_40_STATUS_B Register (offset = 3284h) [reset = 0h]

QUEUE_40_STATUS_B is shown in Figure 16-1056 and described in Table 16-1070.

**Figure 16-1056. QUEUE_40_STATUS_B Register**

| 31–28 | 27–0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-1070. QUEUE_40_STATUS_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. Queue_Manager_Queue_n_Status_B Registers B |

### 16.4.7.781 QUEUE_40_STATUS_C Register (offset = 3288h) [reset = 0h]

QUEUE_40_STATUS_C is shown in Figure 16-1057 and described in Table 16-1071.

**Figure 16-1057. QUEUE_40_STATUS_C Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | PACKET_SIZE |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-1071. QUEUE_40_STATUS_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | PACKET_SIZE | R-0 | 0 | This field indicates packet size of the head element of a queue. Queue_Manager_Queue_N_Status_C Registers C |

### 16.4.7.782 QUEUE_41_STATUS_A Register (offset = 3290h) [reset = 0h]

QUEUE_41_STATUS_A is shown in Figure 16-1058 and described in Table 16-1072.

**Figure 16-1058. QUEUE_41_STATUS_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-1072. QUEUE_41_STATUS_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>Queue Manager Queue N Status Registers A |

### 16.4.7.783 QUEUE_41_STATUS_B Register (offset = 3294h) [reset = 0h]

QUEUE_41_STATUS_B is shown in Figure 16-1059 and described in Table 16-1073.

**Figure 16-1059. QUEUE_41_STATUS_B Register**

| 31–28 | 27–0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-1073. QUEUE_41_STATUS_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. Queue_Manager_Queue_n_Status_B Registers B |

### 16.4.7.784 QUEUE_41_STATUS_C Register (offset = 3298h) [reset = 0h]

QUEUE_41_STATUS_C is shown in Figure 16-1060 and described in Table 16-1074.

**Figure 16-1060. QUEUE_41_STATUS_C Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | PACKET_SIZE |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-1074. QUEUE_41_STATUS_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | PACKET_SIZE | R-0 | 0 | This field indicates packet size of the head element of a queue. Queue_Manager_Queue_N_Status_C Registers C |

### 16.4.7.785 QUEUE_42_STATUS_A Register (offset = 32A0h) [reset = 0h]

QUEUE_42_STATUS_A is shown in Figure 16-1061 and described in Table 16-1075.

**Figure 16-1061. QUEUE_42_STATUS_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-1075. QUEUE_42_STATUS_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>Queue Manager Queue N Status Registers A |

### 16.4.7.786 QUEUE_42_STATUS_B Register (offset = 32A4h) [reset = 0h]

QUEUE_42_STATUS_B is shown in Figure 16-1062 and described in Table 16-1076.

**Figure 16-1062. QUEUE_42_STATUS_B Register**

| 31–28 | 27–0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-1076. QUEUE_42_STATUS_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. Queue_Manager_Queue_n_Status_B Registers B |

### 16.4.7.787 QUEUE_42_STATUS_C Register (offset = 32A8h) [reset = 0h]

QUEUE_42_STATUS_C is shown in Figure 16-1063 and described in Table 16-1077.

**Figure 16-1063. QUEUE_42_STATUS_C Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | PACKET_SIZE |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-1077. QUEUE_42_STATUS_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | PACKET_SIZE | R-0 | 0 | This field indicates packet size of the head element of a queue. Queue_Manager_Queue_N_Status_C Registers C |

### 16.4.7.788 QUEUE_43_STATUS_A Register (offset = 32B0h) [reset = 0h]

QUEUE_43_STATUS_A is shown in Figure 16-1064 and described in Table 16-1078.

**Figure 16-1064. QUEUE_43_STATUS_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-1078. QUEUE_43_STATUS_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>Queue Manager Queue N Status Registers A |

### 16.4.7.789 QUEUE_43_STATUS_B Register (offset = 32B4h) [reset = 0h]

QUEUE_43_STATUS_B is shown in Figure 16-1065 and described in Table 16-1079.

**Figure 16-1065. QUEUE_43_STATUS_B Register**

| 31-28 | 27-0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-1079. QUEUE_43_STATUS_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. Queue_Manager_Queue_n_Status_B Registers B |

### 16.4.7.790 QUEUE_43_STATUS_C Register (offset = 32B8h) [reset = 0h]

QUEUE_43_STATUS_C is shown in Figure 16-1066 and described in Table 16-1080.

**Figure 16-1066. QUEUE_43_STATUS_C Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | PACKET_SIZE |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-1080. QUEUE_43_STATUS_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | PACKET_SIZE | R-0 | 0 | This field indicates packet size of the head element of a queue. Queue_Manager_Queue_N_Status_C Registers C |

### 16.4.7.791 QUEUE_44_STATUS_A Register (offset = 32C0h) [reset = 0h]

QUEUE_44_STATUS_A is shown in Figure 16-1067 and described in Table 16-1081.

**Figure 16-1067. QUEUE_44_STATUS_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-1081. QUEUE_44_STATUS_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>Queue Manager Queue N Status Registers A |

### 16.4.7.792 QUEUE_44_STATUS_B Register (offset = 32C4h) [reset = 0h]

QUEUE_44_STATUS_B is shown in Figure 16-1068 and described in Table 16-1082.

**Figure 16-1068. QUEUE_44_STATUS_B Register**

| 31–28 | 27–0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-1082. QUEUE_44_STATUS_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue.<br>Queue_Manager_Queue_n_Status_B Registers B |

### 16.4.7.793 QUEUE_44_STATUS_C Register (offset = 32C8h) [reset = 0h]

QUEUE_44_STATUS_C is shown in Figure 16-1069 and described in Table 16-1083.

**Figure 16-1069. QUEUE_44_STATUS_C Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | PACKET_SIZE |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-1083. QUEUE_44_STATUS_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | PACKET_SIZE | R-0 | 0 | This field indicates packet size of the head element of a queue. Queue_Manager_Queue_N_Status_C Registers C |

### 16.4.7.794 QUEUE_45_STATUS_A Register (offset = 32D0h) [reset = 0h]

QUEUE_45_STATUS_A is shown in Figure 16-1070 and described in Table 16-1084.

**Figure 16-1070. QUEUE_45_STATUS_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-1084. QUEUE_45_STATUS_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>Queue Manager Queue N Status Registers A |

### 16.4.7.795 QUEUE_45_STATUS_B Register (offset = 32D4h) [reset = 0h]

QUEUE_45_STATUS_B is shown in Figure 16-1071 and described in Table 16-1085.

**Figure 16-1071. QUEUE_45_STATUS_B Register**

| 31-28 | 27-0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-1085. QUEUE_45_STATUS_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. Queue_Manager_Queue_n_Status_B Registers B |

### 16.4.7.796 QUEUE_45_STATUS_C Register (offset = 32D8h) [reset = 0h]

QUEUE_45_STATUS_C is shown in Figure 16-1072 and described in Table 16-1086.

**Figure 16-1072. QUEUE_45_STATUS_C Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | PACKET_SIZE |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-1086. QUEUE_45_STATUS_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | PACKET_SIZE | R-0 | 0 | This field indicates packet size of the head element of a queue. Queue_Manager_Queue_N_Status_C Registers C |

### 16.4.7.797 QUEUE_46_STATUS_A Register (offset = 32E0h) [reset = 0h]

QUEUE_46_STATUS_A is shown in Figure 16-1073 and described in Table 16-1087.

**Figure 16-1073. QUEUE_46_STATUS_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-1087. QUEUE_46_STATUS_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>Queue Manager Queue N Status Registers A |

### 16.4.7.798 QUEUE_46_STATUS_B Register (offset = 32E4h) [reset = 0h]

QUEUE_46_STATUS_B is shown in Figure 16-1074 and described in Table 16-1088.

**Figure 16-1074. QUEUE_46_STATUS_B Register**

| 31-28 | 27-0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-1088. QUEUE_46_STATUS_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. Queue_Manager_Queue_n_Status_B Registers B |

### 16.4.7.799 QUEUE_46_STATUS_C Register (offset = 32E8h) [reset = 0h]

QUEUE_46_STATUS_C is shown in Figure 16-1075 and described in Table 16-1089.

**Figure 16-1075. QUEUE_46_STATUS_C Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | PACKET_SIZE |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-1089. QUEUE_46_STATUS_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | PACKET_SIZE | R-0 | 0 | This field indicates packet size of the head element of a queue. Queue_Manager_Queue_N_Status_C Registers C |

### 16.4.7.800 QUEUE_47_STATUS_A Register (offset = 32F0h) [reset = 0h]

QUEUE_47_STATUS_A is shown in Figure 16-1076 and described in Table 16-1090.

**Figure 16-1076. QUEUE_47_STATUS_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-1090. QUEUE_47_STATUS_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>Queue Manager Queue N Status Registers A |

### 16.4.7.801 QUEUE_47_STATUS_B Register (offset = 32F4h) [reset = 0h]

QUEUE_47_STATUS_B is shown in Figure 16-1077 and described in Table 16-1091.

**Figure 16-1077. QUEUE_47_STATUS_B Register**

| 31–28 | 27–0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-1091. QUEUE_47_STATUS_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. Queue_Manager_Queue_n_Status_B Registers B |

### 16.4.7.802 QUEUE_47_STATUS_C Register (offset = 32F8h) [reset = 0h]

QUEUE_47_STATUS_C is shown in Figure 16-1078 and described in Table 16-1092.

**Figure 16-1078. QUEUE_47_STATUS_C Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | PACKET_SIZE |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-1092. QUEUE_47_STATUS_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | PACKET_SIZE | R-0 | 0 | This field indicates packet size of the head element of a queue. Queue_Manager_Queue_N_Status_C Registers C |

### 16.4.7.803 QUEUE_48_STATUS_A Register (offset = 3300h) [reset = 0h]

QUEUE_48_STATUS_A is shown in Figure 16-1079 and described in Table 16-1093.

**Figure 16-1079. QUEUE_48_STATUS_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-1093. QUEUE_48_STATUS_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>Queue Manager Queue N Status Registers A |

### 16.4.7.804 QUEUE_48_STATUS_B Register (offset = 3304h) [reset = 0h]

QUEUE_48_STATUS_B is shown in Figure 16-1080 and described in Table 16-1094.

**Figure 16-1080. QUEUE_48_STATUS_B Register**

| 31–28 | 27–0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-1094. QUEUE_48_STATUS_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue.<br>Queue_Manager_Queue_n_Status_B Registers B |

### 16.4.7.805 QUEUE_48_STATUS_C Register (offset = 3308h) [reset = 0h]

QUEUE_48_STATUS_C is shown in Figure 16-1081 and described in Table 16-1095.

**Figure 16-1081. QUEUE_48_STATUS_C Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | PACKET_SIZE |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-1095. QUEUE_48_STATUS_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | PACKET_SIZE | R-0 | 0 | This field indicates packet size of the head element of a queue. Queue_Manager_Queue_N_Status_C Registers C |

### 16.4.7.806 QUEUE_49_STATUS_A Register (offset = 3310h) [reset = 0h]

QUEUE_49_STATUS_A is shown in Figure 16-1082 and described in Table 16-1096.

**Figure 16-1082. QUEUE_49_STATUS_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-1096. QUEUE_49_STATUS_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>Queue Manager Queue N Status Registers A |

### 16.4.7.807 QUEUE_49_STATUS_B Register (offset = 3314h) [reset = 0h]

QUEUE_49_STATUS_B is shown in Figure 16-1083 and described in Table 16-1097.

**Figure 16-1083. QUEUE_49_STATUS_B Register**

| 31–28 | 27–0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-1097. QUEUE_49_STATUS_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. Queue_Manager_Queue_n_Status_B Registers B |

### 16.4.7.808 QUEUE_49_STATUS_C Register (offset = 3318h) [reset = 0h]

QUEUE_49_STATUS_C is shown in Figure 16-1084 and described in Table 16-1098.

**Figure 16-1084. QUEUE_49_STATUS_C Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | PACKET_SIZE |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-1098. QUEUE_49_STATUS_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | PACKET_SIZE | R-0 | 0 | This field indicates packet size of the head element of a queue. Queue_Manager_Queue_N_Status_C Registers C |

### 16.4.7.809 QUEUE_50_STATUS_A Register (offset = 3320h) [reset = 0h]

QUEUE_50_STATUS_A is shown in Figure 16-1085 and described in Table 16-1099.

**Figure 16-1085. QUEUE_50_STATUS_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-1099. QUEUE_50_STATUS_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>Queue Manager Queue N Status Registers A |

### 16.4.7.810 QUEUE_50_STATUS_B Register (offset = 3324h) [reset = 0h]

QUEUE_50_STATUS_B is shown in Figure 16-1086 and described in Table 16-1100.

**Figure 16-1086. QUEUE_50_STATUS_B Register**

| 31–28 | 27–0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-1100. QUEUE_50_STATUS_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. Queue_Manager_Queue_n_Status_B Registers B |

### 16.4.7.811 QUEUE_50_STATUS_C Register (offset = 3328h) [reset = 0h]

QUEUE_50_STATUS_C is shown in Figure 16-1087 and described in Table 16-1101.

**Figure 16-1087. QUEUE_50_STATUS_C Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | PACKET_SIZE |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-1101. QUEUE_50_STATUS_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | PACKET_SIZE | R-0 | 0 | This field indicates packet size of the head element of a queue. Queue_Manager_Queue_N_Status_C Registers C |

### 16.4.7.812 QUEUE_51_STATUS_A Register (offset = 3330h) [reset = 0h]

QUEUE_51_STATUS_A is shown in Figure 16-1088 and described in Table 16-1102.

**Figure 16-1088. QUEUE_51_STATUS_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-1102. QUEUE_51_STATUS_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>Queue Manager Queue N Status Registers A |

### 16.4.7.813 QUEUE_51_STATUS_B Register (offset = 3334h) [reset = 0h]

QUEUE_51_STATUS_B is shown in Figure 16-1089 and described in Table 16-1103.

**Figure 16-1089. QUEUE_51_STATUS_B Register**

| 31-28 | 27-0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-1103. QUEUE_51_STATUS_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue.<br>Queue_Manager_Queue_n_Status_B Registers B |

### 16.4.7.814 QUEUE_51_STATUS_C Register (offset = 3338h) [reset = 0h]

QUEUE_51_STATUS_C is shown in Figure 16-1090 and described in Table 16-1104.

**Figure 16-1090. QUEUE_51_STATUS_C Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | PACKET_SIZE |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-1104. QUEUE_51_STATUS_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | PACKET_SIZE | R-0 | 0 | This field indicates packet size of the head element of a queue. Queue_Manager_Queue_N_Status_C Registers C |

### 16.4.7.815 QUEUE_52_STATUS_A Register (offset = 3340h) [reset = 0h]

QUEUE_52_STATUS_A is shown in Figure 16-1091 and described in Table 16-1105.

**Figure 16-1091. QUEUE_52_STATUS_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-1105. QUEUE_52_STATUS_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>Queue Manager Queue N Status Registers A |

### 16.4.7.816 QUEUE_52_STATUS_B Register (offset = 3344h) [reset = 0h]

QUEUE_52_STATUS_B is shown in Figure 16-1092 and described in Table 16-1106.

**Figure 16-1092. QUEUE_52_STATUS_B Register**

| 31–28 | 27–0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-1106. QUEUE_52_STATUS_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. Queue_Manager_Queue_n_Status_B Registers B |

### 16.4.7.817 QUEUE_52_STATUS_C Register (offset = 3348h) [reset = 0h]

QUEUE_52_STATUS_C is shown in Figure 16-1093 and described in Table 16-1107.

**Figure 16-1093. QUEUE_52_STATUS_C Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | PACKET_SIZE |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-1107. QUEUE_52_STATUS_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | PACKET_SIZE | R-0 | 0 | This field indicates packet size of the head element of a queue. Queue_Manager_Queue_N_Status_C Registers C |

### 16.4.7.818 QUEUE_53_STATUS_A Register (offset = 3350h) [reset = 0h]

QUEUE_53_STATUS_A is shown in Figure 16-1094 and described in Table 16-1108.

**Figure 16-1094. QUEUE_53_STATUS_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-1108. QUEUE_53_STATUS_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>Queue Manager Queue N Status Registers A |

### 16.4.7.819 QUEUE_53_STATUS_B Register (offset = 3354h) [reset = 0h]

QUEUE_53_STATUS_B is shown in Figure 16-1095 and described in Table 16-1109.

**Figure 16-1095. QUEUE_53_STATUS_B Register**

| 31–28 | 27–0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-1109. QUEUE_53_STATUS_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. Queue_Manager_Queue_n_Status_B Registers B |

### 16.4.7.820 QUEUE_53_STATUS_C Register (offset = 3358h) [reset = 0h]

QUEUE_53_STATUS_C is shown in Figure 16-1096 and described in Table 16-1110.

**Figure 16-1096. QUEUE_53_STATUS_C Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | PACKET_SIZE |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-1110. QUEUE_53_STATUS_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | PACKET_SIZE | R-0 | 0 | This field indicates packet size of the head element of a queue. Queue_Manager_Queue_N_Status_C Registers C |

### 16.4.7.821 QUEUE_54_STATUS_A Register (offset = 3360h) [reset = 0h]

QUEUE_54_STATUS_A is shown in Figure 16-1097 and described in Table 16-1111.

**Figure 16-1097. QUEUE_54_STATUS_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-1111. QUEUE_54_STATUS_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>Queue Manager Queue N Status Registers A |

### 16.4.7.822 QUEUE_54_STATUS_B Register (offset = 3364h) [reset = 0h]

QUEUE_54_STATUS_B is shown in Figure 16-1098 and described in Table 16-1112.

**Figure 16-1098. QUEUE_54_STATUS_B Register**

| 31-28 | 27-0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-1112. QUEUE_54_STATUS_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. Queue_Manager_Queue_n_Status_B Registers B |

### 16.4.7.823 QUEUE_54_STATUS_C Register (offset = 3368h) [reset = 0h]

QUEUE_54_STATUS_C is shown in Figure 16-1099 and described in Table 16-1113.

**Figure 16-1099. QUEUE_54_STATUS_C Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | PACKET_SIZE |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-1113. QUEUE_54_STATUS_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | PACKET_SIZE | R-0 | 0 | This field indicates packet size of the head element of a queue. Queue_Manager_Queue_N_Status_C Registers C |

### 16.4.7.824 QUEUE_55_STATUS_A Register (offset = 3370h) [reset = 0h]

QUEUE_55_STATUS_A is shown in Figure 16-1100 and described in Table 16-1114.

**Figure 16-1100. QUEUE_55_STATUS_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-1114. QUEUE_55_STATUS_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>Queue Manager Queue N Status Registers A |

### 16.4.7.825 QUEUE_55_STATUS_B Register (offset = 3374h) [reset = 0h]

QUEUE_55_STATUS_B is shown in Figure 16-1101 and described in Table 16-1115.

**Figure 16-1101. QUEUE_55_STATUS_B Register**

| 31–28 | 27–0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-1115. QUEUE_55_STATUS_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. Queue_Manager_Queue_n_Status_B Registers B |

### 16.4.7.826 QUEUE_55_STATUS_C Register (offset = 3378h) [reset = 0h]

QUEUE_55_STATUS_C is shown in Figure 16-1102 and described in Table 16-1116.

**Figure 16-1102. QUEUE_55_STATUS_C Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | PACKET_SIZE |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-1116. QUEUE_55_STATUS_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | PACKET_SIZE | R-0 | 0 | This field indicates packet size of the head element of a queue. Queue_Manager_Queue_N_Status_C Registers C |

### 16.4.7.827 QUEUE_56_STATUS_A Register (offset = 3380h) [reset = 0h]

QUEUE_56_STATUS_A is shown in Figure 16-1103 and described in Table 16-1117.

**Figure 16-1103. QUEUE_56_STATUS_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-1117. QUEUE_56_STATUS_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>Queue Manager Queue N Status Registers A |

### 16.4.7.828 QUEUE_56_STATUS_B Register (offset = 3384h) [reset = 0h]

QUEUE_56_STATUS_B is shown in Figure 16-1104 and described in Table 16-1118.

**Figure 16-1104. QUEUE_56_STATUS_B Register**

| 31–28 | 27–0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-1118. QUEUE_56_STATUS_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. Queue_Manager_Queue_n_Status_B Registers B |

### 16.4.7.829 QUEUE_56_STATUS_C Register (offset = 3388h) [reset = 0h]

QUEUE_56_STATUS_C is shown in Figure 16-1105 and described in Table 16-1119.

**Figure 16-1105. QUEUE_56_STATUS_C Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | PACKET_SIZE |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-1119. QUEUE_56_STATUS_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | PACKET_SIZE | R-0 | 0 | This field indicates packet size of the head element of a queue.<br>Queue_Manager_Queue_N_Status_C Registers C |

### 16.4.7.830 QUEUE_57_STATUS_A Register (offset = 3390h) [reset = 0h]

QUEUE_57_STATUS_A is shown in Figure 16-1106 and described in Table 16-1120.

**Figure 16-1106. QUEUE_57_STATUS_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-1120. QUEUE_57_STATUS_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>Queue Manager Queue N Status Registers A |

### 16.4.7.831 QUEUE_57_STATUS_B Register (offset = 3394h) [reset = 0h]

QUEUE_57_STATUS_B is shown in Figure 16-1107 and described in Table 16-1121.

**Figure 16-1107. QUEUE_57_STATUS_B Register**

| 31–28 | 27–0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-1121. QUEUE_57_STATUS_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. Queue_Manager_Queue_n_Status_B Registers B |

### 16.4.7.832 QUEUE_57_STATUS_C Register (offset = 3398h) [reset = 0h]

QUEUE_57_STATUS_C is shown in Figure 16-1108 and described in Table 16-1122.

**Figure 16-1108. QUEUE_57_STATUS_C Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | PACKET_SIZE |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-1122. QUEUE_57_STATUS_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | PACKET_SIZE | R-0 | 0 | This field indicates packet size of the head element of a queue. Queue_Manager_Queue_N_Status_C Registers C |

### 16.4.7.833 QUEUE_58_STATUS_A Register (offset = 33A0h) [reset = 0h]

QUEUE_58_STATUS_A is shown in Figure 16-1109 and described in Table 16-1123.

**Figure 16-1109. QUEUE_58_STATUS_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-1123. QUEUE_58_STATUS_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>Queue Manager Queue N Status Registers A |

### 16.4.7.834 QUEUE_58_STATUS_B Register (offset = 33A4h) [reset = 0h]

QUEUE_58_STATUS_B is shown in Figure 16-1110 and described in Table 16-1124.

**Figure 16-1110. QUEUE_58_STATUS_B Register**

| 31-28 | 27-0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-1124. QUEUE_58_STATUS_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue.<br>Queue_Manager_Queue_n_Status_B Registers B |

### 16.4.7.835 QUEUE_58_STATUS_C Register (offset = 33A8h) [reset = 0h]

QUEUE_58_STATUS_C is shown in Figure 16-1111 and described in Table 16-1125.

**Figure 16-1111. QUEUE_58_STATUS_C Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | PACKET_SIZE |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-1125. QUEUE_58_STATUS_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | PACKET_SIZE | R-0 | 0 | This field indicates packet size of the head element of a queue. Queue_Manager_Queue_N_Status_C Registers C |

### 16.4.7.836 QUEUE_59_STATUS_A Register (offset = 33B0h) [reset = 0h]

QUEUE_59_STATUS_A is shown in Figure 16-1112 and described in Table 16-1126.

**Figure 16-1112. QUEUE_59_STATUS_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-1126. QUEUE_59_STATUS_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>Queue Manager Queue N Status Registers A |

### 16.4.7.837 QUEUE_59_STATUS_B Register (offset = 33B4h) [reset = 0h]

QUEUE_59_STATUS_B is shown in Figure 16-1113 and described in Table 16-1127.

**Figure 16-1113. QUEUE_59_STATUS_B Register**

| 31–28 | 27–0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-1127. QUEUE_59_STATUS_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. Queue_Manager_Queue_n_Status_B Registers B |

### 16.4.7.838 QUEUE_59_STATUS_C Register (offset = 33B8h) [reset = 0h]

QUEUE_59_STATUS_C is shown in Figure 16-1114 and described in Table 16-1128.

**Figure 16-1114. QUEUE_59_STATUS_C Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | PACKET_SIZE |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-1128. QUEUE_59_STATUS_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | PACKET_SIZE | R-0 | 0 | This field indicates packet size of the head element of a queue. Queue_Manager_Queue_N_Status_C Registers C |

### 16.4.7.839 QUEUE_60_STATUS_A Register (offset = 33C0h) [reset = 0h]

QUEUE_60_STATUS_A is shown in Figure 16-1115 and described in Table 16-1129.

**Figure 16-1115. QUEUE_60_STATUS_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-1129. QUEUE_60_STATUS_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>Queue Manager Queue N Status Registers A |

### 16.4.7.840 QUEUE_60_STATUS_B Register (offset = 33C4h) [reset = 0h]

QUEUE_60_STATUS_B is shown in Figure 16-1116 and described in Table 16-1130.

**Figure 16-1116. QUEUE_60_STATUS_B Register**

| 31–28 | 27–0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-1130. QUEUE_60_STATUS_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. Queue_Manager_Queue_n_Status_B Registers B |

### 16.4.7.841 QUEUE_60_STATUS_C Register (offset = 33C8h) [reset = 0h]

QUEUE_60_STATUS_C is shown in Figure 16-1117 and described in Table 16-1131.

**Figure 16-1117. QUEUE_60_STATUS_C Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | PACKET_SIZE |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-1131. QUEUE_60_STATUS_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | PACKET_SIZE | R-0 | 0 | This field indicates packet size of the head element of a queue. Queue_Manager_Queue_N_Status_C Registers C |

### 16.4.7.842 QUEUE_61_STATUS_A Register (offset = 33D0h) [reset = 0h]

QUEUE_61_STATUS_A is shown in Figure 16-1118 and described in Table 16-1132.

**Figure 16-1118. QUEUE_61_STATUS_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-1132. QUEUE_61_STATUS_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>Queue Manager Queue N Status Registers A |

### 16.4.7.843 QUEUE_61_STATUS_B Register (offset = 33D4h) [reset = 0h]

QUEUE_61_STATUS_B is shown in Figure 16-1119 and described in Table 16-1133.

**Figure 16-1119. QUEUE_61_STATUS_B Register**

| 31-28 | 27-0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-1133. QUEUE_61_STATUS_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. Queue_Manager_Queue_n_Status_B Registers B |

### 16.4.7.844 QUEUE_61_STATUS_C Register (offset = 33D8h) [reset = 0h]

QUEUE_61_STATUS_C is shown in Figure 16-1120 and described in Table 16-1134.

**Figure 16-1120. QUEUE_61_STATUS_C Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | PACKET_SIZE |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-1134. QUEUE_61_STATUS_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | PACKET_SIZE | R-0 | 0 | This field indicates packet size of the head element of a queue. Queue_Manager_Queue_N_Status_C Registers C |

### 16.4.7.845 QUEUE_62_STATUS_A Register (offset = 33E0h) [reset = 0h]

QUEUE_62_STATUS_A is shown in Figure 16-1121 and described in Table 16-1135.

**Figure 16-1121. QUEUE_62_STATUS_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-1135. QUEUE_62_STATUS_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>Queue Manager Queue N Status Registers A |

### 16.4.7.846 QUEUE_62_STATUS_B Register (offset = 33E4h) [reset = 0h]

QUEUE_62_STATUS_B is shown in Figure 16-1122 and described in Table 16-1136.

**Figure 16-1122. QUEUE_62_STATUS_B Register**

| 31-28 | 27-0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-1136. QUEUE_62_STATUS_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. Queue_Manager_Queue_n_Status_B Registers B |

### 16.4.7.847 QUEUE_62_STATUS_C Register (offset = 33E8h) [reset = 0h]

QUEUE_62_STATUS_C is shown in Figure 16-1123 and described in Table 16-1137.

**Figure 16-1123. QUEUE_62_STATUS_C Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | PACKET_SIZE |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-1137. QUEUE_62_STATUS_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | PACKET_SIZE | R-0 | 0 | This field indicates packet size of the head element of a queue. Queue_Manager_Queue_N_Status_C Registers C |

### 16.4.7.848 QUEUE_63_STATUS_A Register (offset = 33F0h) [reset = 0h]

QUEUE_63_STATUS_A is shown in Figure 16-1124 and described in Table 16-1138.

**Figure 16-1124. QUEUE_63_STATUS_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-1138. QUEUE_63_STATUS_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>Queue Manager Queue N Status Registers A |

### 16.4.7.849 QUEUE_63_STATUS_B Register (offset = 33F4h) [reset = 0h]

QUEUE_63_STATUS_B is shown in Figure 16-1125 and described in Table 16-1139.

**Figure 16-1125. QUEUE_63_STATUS_B Register**

| 31-28 | 27-0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-1139. QUEUE_63_STATUS_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. Queue_Manager_Queue_n_Status_B Registers B |

### 16.4.7.850 QUEUE_63_STATUS_C Register (offset = 33F8h) [reset = 0h]

QUEUE_63_STATUS_C is shown in Figure 16-1126 and described in Table 16-1140.

**Figure 16-1126. QUEUE_63_STATUS_C Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | PACKET_SIZE |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-1140. QUEUE_63_STATUS_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | PACKET_SIZE | R-0 | 0 | This field indicates packet size of the head element of a queue. Queue_Manager_Queue_N_Status_C Registers C |

### 16.4.7.851 QUEUE_64_STATUS_A Register (offset = 3400h) [reset = 0h]

QUEUE_64_STATUS_A is shown in Figure 16-1127 and described in Table 16-1141.

**Figure 16-1127. QUEUE_64_STATUS_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-1141. QUEUE_64_STATUS_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>Queue Manager Queue N Status Registers A |

### 16.4.7.852 QUEUE_64_STATUS_B Register (offset = 3404h) [reset = 0h]

QUEUE_64_STATUS_B is shown in Figure 16-1128 and described in Table 16-1142.

**Figure 16-1128. QUEUE_64_STATUS_B Register**

| 31–28 | 27–0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-1142. QUEUE_64_STATUS_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. Queue_Manager_Queue_n_Status_B Registers B |

### 16.4.7.853 QUEUE_64_STATUS_C Register (offset = 3408h) [reset = 0h]

QUEUE_64_STATUS_C is shown in Figure 16-1129 and described in Table 16-1143.

**Figure 16-1129. QUEUE_64_STATUS_C Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | PACKET_SIZE |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-1143. QUEUE_64_STATUS_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | PACKET_SIZE | R-0 | 0 | This field indicates packet size of the head element of a queue.<br>Queue_Manager_Queue_N_Status_C Registers C |

### 16.4.7.854 QUEUE_65_STATUS_A Register (offset = 3410h) [reset = 0h]

QUEUE_65_STATUS_A is shown in Figure 16-1130 and described in Table 16-1144.

**Figure 16-1130. QUEUE_65_STATUS_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-1144. QUEUE_65_STATUS_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>Queue Manager Queue N Status Registers A |

### 16.4.7.855 QUEUE_65_STATUS_B Register (offset = 3414h) [reset = 0h]

QUEUE_65_STATUS_B is shown in Figure 16-1131 and described in Table 16-1145.

**Figure 16-1131. QUEUE_65_STATUS_B Register**

| 31–28 | 27–0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-1145. QUEUE_65_STATUS_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. Queue_Manager_Queue_n_Status_B Registers B |

### 16.4.7.856 QUEUE_65_STATUS_C Register (offset = 3418h) [reset = 0h]

QUEUE_65_STATUS_C is shown in Figure 16-1132 and described in Table 16-1146.

**Figure 16-1132. QUEUE_65_STATUS_C Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | PACKET_SIZE |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-1146. QUEUE_65_STATUS_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | PACKET_SIZE | R-0 | 0 | This field indicates packet size of the head element of a queue. Queue_Manager_Queue_N_Status_C Registers C |

### 16.4.7.857 QUEUE_66_STATUS_A Register (offset = 3420h) [reset = 0h]

QUEUE_66_STATUS_A is shown in Figure 16-1133 and described in Table 16-1147.

**Figure 16-1133. QUEUE_66_STATUS_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-1147. QUEUE_66_STATUS_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>Queue Manager Queue N Status Registers A |

### 16.4.7.858 QUEUE_66_STATUS_B Register (offset = 3424h) [reset = 0h]

QUEUE_66_STATUS_B is shown in Figure 16-1134 and described in Table 16-1148.

**Figure 16-1134. QUEUE_66_STATUS_B Register**

| 31–28 | 27–0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-1148. QUEUE_66_STATUS_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. Queue_Manager_Queue_n_Status_B Registers B |

### 16.4.7.859 QUEUE_66_STATUS_C Register (offset = 3428h) [reset = 0h]

QUEUE_66_STATUS_C is shown in Figure 16-1135 and described in Table 16-1149.

**Figure 16-1135. QUEUE_66_STATUS_C Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | PACKET_SIZE |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-1149. QUEUE_66_STATUS_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | PACKET_SIZE | R-0 | 0 | This field indicates packet size of the head element of a queue. Queue_Manager_Queue_N_Status_C Registers C |

### 16.4.7.860 QUEUE_67_STATUS_A Register (offset = 3430h) [reset = 0h]

QUEUE_67_STATUS_A is shown in Figure 16-1136 and described in Table 16-1150.

**Figure 16-1136. QUEUE_67_STATUS_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-1150. QUEUE_67_STATUS_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>Queue Manager Queue N Status Registers A |

### 16.4.7.861 QUEUE_67_STATUS_B Register (offset = 3434h) [reset = 0h]

QUEUE_67_STATUS_B is shown in Figure 16-1137 and described in Table 16-1151.

**Figure 16-1137. QUEUE_67_STATUS_B Register**

| 31–28 | 27–0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-1151. QUEUE_67_STATUS_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. Queue_Manager_Queue_n_Status_B Registers B |

### 16.4.7.862 QUEUE_67_STATUS_C Register (offset = 3438h) [reset = 0h]

QUEUE_67_STATUS_C is shown in Figure 16-1138 and described in Table 16-1152.

**Figure 16-1138. QUEUE_67_STATUS_C Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | PACKET_SIZE |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-1152. QUEUE_67_STATUS_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | PACKET_SIZE | R-0 | 0 | This field indicates packet size of the head element of a queue. Queue_Manager_Queue_N_Status_C Registers C |

### 16.4.7.863 QUEUE_68_STATUS_A Register (offset = 3440h) [reset = 0h]

QUEUE_68_STATUS_A is shown in Figure 16-1139 and described in Table 16-1153.

**Figure 16-1139. QUEUE_68_STATUS_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-1153. QUEUE_68_STATUS_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>Queue Manager Queue N Status Registers A |

### 16.4.7.864 QUEUE_68_STATUS_B Register (offset = 3444h) [reset = 0h]

QUEUE_68_STATUS_B is shown in Figure 16-1140 and described in Table 16-1154.

**Figure 16-1140. QUEUE_68_STATUS_B Register**

| 31–28 | 27–0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-1154. QUEUE_68_STATUS_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. Queue_Manager_Queue_n_Status_B Registers B |

### 16.4.7.865 QUEUE_68_STATUS_C Register (offset = 3448h) [reset = 0h]

QUEUE_68_STATUS_C is shown in Figure 16-1141 and described in Table 16-1155.

**Figure 16-1141. QUEUE_68_STATUS_C Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | PACKET_SIZE |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-1155. QUEUE_68_STATUS_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | PACKET_SIZE | R-0 | 0 | This field indicates packet size of the head element of a queue. Queue_Manager_Queue_N_Status_C Registers C |

### 16.4.7.866 QUEUE_69_STATUS_A Register (offset = 3450h) [reset = 0h]

QUEUE_69_STATUS_A is shown in Figure 16-1142 and described in Table 16-1156.

**Figure 16-1142. QUEUE_69_STATUS_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-1156. QUEUE_69_STATUS_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>Queue Manager Queue N Status Registers A |

### 16.4.7.867 QUEUE_69_STATUS_B Register (offset = 3454h) [reset = 0h]

QUEUE_69_STATUS_B is shown in Figure 16-1143 and described in Table 16-1157.

**Figure 16-1143. QUEUE_69_STATUS_B Register**

| 31-28 | 27-0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-1157. QUEUE_69_STATUS_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. Queue_Manager_Queue_n_Status_B Registers B |

### 16.4.7.868 QUEUE_69_STATUS_C Register (offset = 3458h) [reset = 0h]

QUEUE_69_STATUS_C is shown in Figure 16-1144 and described in Table 16-1158.

**Figure 16-1144. QUEUE_69_STATUS_C Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | PACKET_SIZE |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-1158. QUEUE_69_STATUS_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | PACKET_SIZE | R-0 | 0 | This field indicates packet size of the head element of a queue. Queue_Manager_Queue_N_Status_C Registers C |

### 16.4.7.869 QUEUE_70_STATUS_A Register (offset = 3460h) [reset = 0h]

QUEUE_70_STATUS_A is shown in Figure 16-1145 and described in Table 16-1159.

**Figure 16-1145. QUEUE_70_STATUS_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-1159. QUEUE_70_STATUS_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>Queue Manager Queue N Status Registers A |

### 16.4.7.870 QUEUE_70_STATUS_B Register (offset = 3464h) [reset = 0h]

QUEUE_70_STATUS_B is shown in Figure 16-1146 and described in Table 16-1160.

**Figure 16-1146. QUEUE_70_STATUS_B Register**

| 31–28 | 27–0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-1160. QUEUE_70_STATUS_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. Queue_Manager_Queue_n_Status_B Registers B |

### 16.4.7.871 QUEUE_70_STATUS_C Register (offset = 3468h) [reset = 0h]

QUEUE_70_STATUS_C is shown in Figure 16-1147 and described in Table 16-1161.

**Figure 16-1147. QUEUE_70_STATUS_C Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | PACKET_SIZE |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-1161. QUEUE_70_STATUS_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | PACKET_SIZE | R-0 | 0 | This field indicates packet size of the head element of a queue. Queue_Manager_Queue_N_Status_C Registers C |

### 16.4.7.872 QUEUE_71_STATUS_A Register (offset = 3470h) [reset = 0h]

QUEUE_71_STATUS_A is shown in Figure 16-1148 and described in Table 16-1162.

**Figure 16-1148. QUEUE_71_STATUS_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-1162. QUEUE_71_STATUS_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>Queue Manager Queue N Status Registers A |

### 16.4.7.873 QUEUE_71_STATUS_B Register (offset = 3474h) [reset = 0h]

QUEUE_71_STATUS_B is shown in Figure 16-1149 and described in Table 16-1163.

**Figure 16-1149. QUEUE_71_STATUS_B Register**

| 31–28 | 27–0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-1163. QUEUE_71_STATUS_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. Queue_Manager_Queue_n_Status_B Registers B |

### 16.4.7.874 QUEUE_71_STATUS_C Register (offset = 3478h) [reset = 0h]

QUEUE_71_STATUS_C is shown in Figure 16-1150 and described in Table 16-1164.

**Figure 16-1150. QUEUE_71_STATUS_C Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | PACKET_SIZE |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-1164. QUEUE_71_STATUS_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | PACKET_SIZE | R-0 | 0 | This field indicates packet size of the head element of a queue. Queue_Manager_Queue_N_Status_C Registers C |

### 16.4.7.875 QUEUE_72_STATUS_A Register (offset = 3480h) [reset = 0h]

QUEUE_72_STATUS_A is shown in Figure 16-1151 and described in Table 16-1165.

**Figure 16-1151. QUEUE_72_STATUS_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-1165. QUEUE_72_STATUS_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>Queue Manager Queue N Status Registers A |

### 16.4.7.876 QUEUE_72_STATUS_B Register (offset = 3484h) [reset = 0h]

QUEUE_72_STATUS_B is shown in Figure 16-1152 and described in Table 16-1166.

**Figure 16-1152. QUEUE_72_STATUS_B Register**

| 31-28 | 27-0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-1166. QUEUE_72_STATUS_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. Queue_Manager_Queue_n_Status_B Registers B |

### 16.4.7.877 QUEUE_72_STATUS_C Register (offset = 3488h) [reset = 0h]

QUEUE_72_STATUS_C is shown in Figure 16-1153 and described in Table 16-1167.

**Figure 16-1153. QUEUE_72_STATUS_C Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | PACKET_SIZE |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-1167. QUEUE_72_STATUS_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | PACKET_SIZE | R-0 | 0 | This field indicates packet size of the head element of a queue. Queue_Manager_Queue_N_Status_C Registers C |

### 16.4.7.878 QUEUE_73_STATUS_A Register (offset = 3490h) [reset = 0h]

QUEUE_73_STATUS_A is shown in Figure 16-1154 and described in Table 16-1168.

**Figure 16-1154. QUEUE_73_STATUS_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-1168. QUEUE_73_STATUS_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>Queue Manager Queue N Status Registers A |

### 16.4.7.879 QUEUE_73_STATUS_B Register (offset = 3494h) [reset = 0h]

QUEUE_73_STATUS_B is shown in Figure 16-1155 and described in Table 16-1169.

**Figure 16-1155. QUEUE_73_STATUS_B Register**

| 31–28 | 27–0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-1169. QUEUE_73_STATUS_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. Queue_Manager_Queue_n_Status_B Registers B |

### 16.4.7.880 QUEUE_73_STATUS_C Register (offset = 3498h) [reset = 0h]

QUEUE_73_STATUS_C is shown in Figure 16-1156 and described in Table 16-1170.

**Figure 16-1156. QUEUE_73_STATUS_C Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | PACKET_SIZE |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-1170. QUEUE_73_STATUS_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | PACKET_SIZE | R-0 | 0 | This field indicates packet size of the head element of a queue. Queue_Manager_Queue_N_Status_C Registers C |

### 16.4.7.881 QUEUE_74_STATUS_A Register (offset = 34A0h) [reset = 0h]

QUEUE_74_STATUS_A is shown in Figure 16-1157 and described in Table 16-1171.

**Figure 16-1157. QUEUE_74_STATUS_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-1171. QUEUE_74_STATUS_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>Queue Manager Queue N Status Registers A |

### 16.4.7.882 QUEUE_74_STATUS_B Register (offset = 34A4h) [reset = 0h]

QUEUE_74_STATUS_B is shown in Figure 16-1158 and described in Table 16-1172.

**Figure 16-1158. QUEUE_74_STATUS_B Register**

| 31–28 | 27–0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-1172. QUEUE_74_STATUS_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. Queue_Manager_Queue_n_Status_B Registers B |

### 16.4.7.883 QUEUE_74_STATUS_C Register (offset = 34A8h) [reset = 0h]

QUEUE_74_STATUS_C is shown in Figure 16-1159 and described in Table 16-1173.

**Figure 16-1159. QUEUE_74_STATUS_C Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | PACKET_SIZE |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-1173. QUEUE_74_STATUS_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | PACKET_SIZE | R-0 | 0 | This field indicates packet size of the head element of a queue. Queue_Manager_Queue_N_Status_C Registers C |

### 16.4.7.884 QUEUE_75_STATUS_A Register (offset = 34B0h) [reset = 0h]

QUEUE_75_STATUS_A is shown in Figure 16-1160 and described in Table 16-1174.

**Figure 16-1160. QUEUE_75_STATUS_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-1174. QUEUE_75_STATUS_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>Queue Manager Queue N Status Registers A |

### 16.4.7.885 QUEUE_75_STATUS_B Register (offset = 34B4h) [reset = 0h]

QUEUE_75_STATUS_B is shown in Figure 16-1161 and described in Table 16-1175.

**Figure 16-1161. QUEUE_75_STATUS_B Register**

| 31-28 | 27-0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-1175. QUEUE_75_STATUS_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue.<br>Queue_Manager_Queue_n_Status_B Registers B |

### 16.4.7.886 QUEUE_75_STATUS_C Register (offset = 34B8h) [reset = 0h]

QUEUE_75_STATUS_C is shown in Figure 16-1162 and described in Table 16-1176.

**Figure 16-1162. QUEUE_75_STATUS_C Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | PACKET_SIZE |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-1176. QUEUE_75_STATUS_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | PACKET_SIZE | R-0 | 0 | This field indicates packet size of the head element of a queue. Queue_Manager_Queue_N_Status_C Registers C |

### 16.4.7.887 QUEUE_76_STATUS_A Register (offset = 34C0h) [reset = 0h]

QUEUE_76_STATUS_A is shown in Figure 16-1163 and described in Table 16-1177.

**Figure 16-1163. QUEUE_76_STATUS_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-1177. QUEUE_76_STATUS_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>Queue Manager Queue N Status Registers A |

### 16.4.7.888 QUEUE_76_STATUS_B Register (offset = 34C4h) [reset = 0h]

QUEUE_76_STATUS_B is shown in Figure 16-1164 and described in Table 16-1178.

**Figure 16-1164. QUEUE_76_STATUS_B Register**

| 31-28 | 27-0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-1178. QUEUE_76_STATUS_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. Queue_Manager_Queue_n_Status_B Registers B |

### 16.4.7.889 QUEUE_76_STATUS_C Register (offset = 34C8h) [reset = 0h]

QUEUE_76_STATUS_C is shown in Figure 16-1165 and described in Table 16-1179.

**Figure 16-1165. QUEUE_76_STATUS_C Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | PACKET_SIZE |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-1179. QUEUE_76_STATUS_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | PACKET_SIZE | R-0 | 0 | This field indicates packet size of the head element of a queue. Queue_Manager_Queue_N_Status_C Registers C |

### 16.4.7.890 QUEUE_77_STATUS_A Register (offset = 34D0h) [reset = 0h]

QUEUE_77_STATUS_A is shown in Figure 16-1166 and described in Table 16-1180.

**Figure 16-1166. QUEUE_77_STATUS_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-1180. QUEUE_77_STATUS_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>Queue Manager Queue N Status Registers A |

### 16.4.7.891 QUEUE_77_STATUS_B Register (offset = 34D4h) [reset = 0h]

QUEUE_77_STATUS_B is shown in Figure 16-1167 and described in Table 16-1181.

**Figure 16-1167. QUEUE_77_STATUS_B Register**

| 31-28 | 27-0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-1181. QUEUE_77_STATUS_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue.<br>Queue_Manager_Queue_n_Status_B Registers B |

### 16.4.7.892 QUEUE_77_STATUS_C Register (offset = 34D8h) [reset = 0h]

QUEUE_77_STATUS_C is shown in Figure 16-1168 and described in Table 16-1182.

**Figure 16-1168. QUEUE_77_STATUS_C Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | PACKET_SIZE |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-1182. QUEUE_77_STATUS_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | PACKET_SIZE | R-0 | 0 | This field indicates packet size of the head element of a queue. Queue_Manager_Queue_N_Status_C Registers C |

### 16.4.7.893 QUEUE_78_STATUS_A Register (offset = 34E0h) [reset = 0h]

QUEUE_78_STATUS_A is shown in Figure 16-1169 and described in Table 16-1183.

**Figure 16-1169. QUEUE_78_STATUS_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-1183. QUEUE_78_STATUS_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>Queue Manager Queue N Status Registers A |

### 16.4.7.894 QUEUE_78_STATUS_B Register (offset = 34E4h) [reset = 0h]

QUEUE_78_STATUS_B is shown in Figure 16-1170 and described in Table 16-1184.

**Figure 16-1170. QUEUE_78_STATUS_B Register**

| 31–28 | 27–0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-1184. QUEUE_78_STATUS_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. Queue_Manager_Queue_n_Status_B Registers B |

### 16.4.7.895 QUEUE_78_STATUS_C Register (offset = 34E8h) [reset = 0h]

QUEUE_78_STATUS_C is shown in Figure 16-1171 and described in Table 16-1185.

**Figure 16-1171. QUEUE_78_STATUS_C Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | PACKET_SIZE |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-1185. QUEUE_78_STATUS_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | PACKET_SIZE | R-0 | 0 | This field indicates packet size of the head element of a queue. Queue_Manager_Queue_N_Status_C Registers C |

### 16.4.7.896 QUEUE_79_STATUS_A Register (offset = 34F0h) [reset = 0h]

QUEUE_79_STATUS_A is shown in Figure 16-1172 and described in Table 16-1186.

**Figure 16-1172. QUEUE_79_STATUS_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-1186. QUEUE_79_STATUS_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>Queue Manager Queue N Status Registers A |

### 16.4.7.897 QUEUE_79_STATUS_B Register (offset = 34F4h) [reset = 0h]

QUEUE_79_STATUS_B is shown in Figure 16-1173 and described in Table 16-1187.

**Figure 16-1173. QUEUE_79_STATUS_B Register**

| 31–28 | 27–0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-1187. QUEUE_79_STATUS_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue.<br>Queue_Manager_Queue_n_Status_B Registers B |

### 16.4.7.898 QUEUE_79_STATUS_C Register (offset = 34F8h) [reset = 0h]

QUEUE_79_STATUS_C is shown in Figure 16-1174 and described in Table 16-1188.

**Figure 16-1174. QUEUE_79_STATUS_C Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | PACKET_SIZE |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-1188. QUEUE_79_STATUS_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | PACKET_SIZE | R-0 | 0 | This field indicates packet size of the head element of a queue. Queue_Manager_Queue_N_Status_C Registers C |

### 16.4.7.899 QUEUE_80_STATUS_A Register (offset = 3500h) [reset = 0h]

QUEUE_80_STATUS_A is shown in Figure 16-1175 and described in Table 16-1189.

**Figure 16-1175. QUEUE_80_STATUS_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-1189. QUEUE_80_STATUS_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>Queue Manager Queue N Status Registers A |

### 16.4.7.900 QUEUE_80_STATUS_B Register (offset = 3504h) [reset = 0h]

QUEUE_80_STATUS_B is shown in Figure 16-1176 and described in Table 16-1190.

**Figure 16-1176. QUEUE_80_STATUS_B Register**

| 31–28 | 27–0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-1190. QUEUE_80_STATUS_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. Queue_Manager_Queue_n_Status_B Registers B |

### 16.4.7.901 QUEUE_80_STATUS_C Register (offset = 3508h) [reset = 0h]

QUEUE_80_STATUS_C is shown in Figure 16-1177 and described in Table 16-1191.

**Figure 16-1177. QUEUE_80_STATUS_C Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | PACKET_SIZE |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-1191. QUEUE_80_STATUS_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | PACKET_SIZE | R-0 | 0 | This field indicates packet size of the head element of a queue. Queue_Manager_Queue_N_Status_C Registers C |

### 16.4.7.902 QUEUE_81_STATUS_A Register (offset = 3510h) [reset = 0h]

QUEUE_81_STATUS_A is shown in Figure 16-1178 and described in Table 16-1192.

**Figure 16-1178. QUEUE_81_STATUS_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-1192. QUEUE_81_STATUS_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>Queue Manager Queue N Status Registers A |

### 16.4.7.903 QUEUE_81_STATUS_B Register (offset = 3514h) [reset = 0h]

QUEUE_81_STATUS_B is shown in Figure 16-1179 and described in Table 16-1193.

**Figure 16-1179. QUEUE_81_STATUS_B Register**

| 31–28 | 27–0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-1193. QUEUE_81_STATUS_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. Queue_Manager_Queue_n_Status_B Registers B |

### 16.4.7.904 QUEUE_81_STATUS_C Register (offset = 3518h) [reset = 0h]

QUEUE_81_STATUS_C is shown in Figure 16-1180 and described in Table 16-1194.

**Figure 16-1180. QUEUE_81_STATUS_C Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | PACKET_SIZE |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-1194. QUEUE_81_STATUS_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | PACKET_SIZE | R-0 | 0 | This field indicates packet size of the head element of a queue. Queue_Manager_Queue_N_Status_C Registers C |

### 16.4.7.905 QUEUE_82_STATUS_A Register (offset = 3520h) [reset = 0h]

QUEUE_82_STATUS_A is shown in Figure 16-1181 and described in Table 16-1195.

**Figure 16-1181. QUEUE_82_STATUS_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-1195. QUEUE_82_STATUS_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>Queue Manager Queue N Status Registers A |

### 16.4.7.906 QUEUE_82_STATUS_B Register (offset = 3524h) [reset = 0h]

QUEUE_82_STATUS_B is shown in Figure 16-1182 and described in Table 16-1196.

**Figure 16-1182. QUEUE_82_STATUS_B Register**

| 31–28 | 27–0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-1196. QUEUE_82_STATUS_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. Queue_Manager_Queue_n_Status_B Registers B |

### 16.4.7.907 QUEUE_82_STATUS_C Register (offset = 3528h) [reset = 0h]

QUEUE_82_STATUS_C is shown in Figure 16-1183 and described in Table 16-1197.

**Figure 16-1183. QUEUE_82_STATUS_C Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | PACKET_SIZE |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-1197. QUEUE_82_STATUS_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | PACKET_SIZE | R-0 | 0 | This field indicates packet size of the head element of a queue. Queue_Manager_Queue_N_Status_C Registers C |

### 16.4.7.908 QUEUE_83_STATUS_A Register (offset = 3530h) [reset = 0h]

QUEUE_83_STATUS_A is shown in Figure 16-1184 and described in Table 16-1198.

**Figure 16-1184. QUEUE_83_STATUS_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-1198. QUEUE_83_STATUS_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>Queue Manager Queue N Status Registers A |

### 16.4.7.909 QUEUE_83_STATUS_B Register (offset = 3534h) [reset = 0h]

QUEUE_83_STATUS_B is shown in Figure 16-1185 and described in Table 16-1199.

**Figure 16-1185. QUEUE_83_STATUS_B Register**

| 31-28 | 27-0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-1199. QUEUE_83_STATUS_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. Queue_Manager_Queue_n_Status_B Registers B |

### 16.4.7.910 QUEUE_83_STATUS_C Register (offset = 3538h) [reset = 0h]

QUEUE_83_STATUS_C is shown in Figure 16-1186 and described in Table 16-1200.

**Figure 16-1186. QUEUE_83_STATUS_C Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | PACKET_SIZE |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-1200. QUEUE_83_STATUS_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | PACKET_SIZE | R-0 | 0 | This field indicates packet size of the head element of a queue. Queue_Manager_Queue_N_Status_C Registers C |

### 16.4.7.911 QUEUE_84_STATUS_A Register (offset = 3540h) [reset = 0h]

QUEUE_84_STATUS_A is shown in Figure 16-1187 and described in Table 16-1201.

**Figure 16-1187. QUEUE_84_STATUS_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-1201. QUEUE_84_STATUS_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>Queue Manager Queue N Status Registers A |

### 16.4.7.912 QUEUE_84_STATUS_B Register (offset = 3544h) [reset = 0h]

QUEUE_84_STATUS_B is shown in Figure 16-1188 and described in Table 16-1202.

**Figure 16-1188. QUEUE_84_STATUS_B Register**

| 31-28 | 27-0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-1202. QUEUE_84_STATUS_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. Queue_Manager_Queue_n_Status_B Registers B |

### 16.4.7.913 QUEUE_84_STATUS_C Register (offset = 3548h) [reset = 0h]

QUEUE_84_STATUS_C is shown in Figure 16-1189 and described in Table 16-1203.

**Figure 16-1189. QUEUE_84_STATUS_C Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | PACKET_SIZE |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-1203. QUEUE_84_STATUS_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | PACKET_SIZE | R-0 | 0 | This field indicates packet size of the head element of a queue. Queue_Manager_Queue_N_Status_C Registers C |

### 16.4.7.914 QUEUE_85_STATUS_A Register (offset = 3550h) [reset = 0h]

QUEUE_85_STATUS_A is shown in Figure 16-1190 and described in Table 16-1204.

**Figure 16-1190. QUEUE_85_STATUS_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-1204. QUEUE_85_STATUS_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>Queue Manager Queue N Status Registers A |

### 16.4.7.915 QUEUE_85_STATUS_B Register (offset = 3554h) [reset = 0h]

QUEUE_85_STATUS_B is shown in Figure 16-1191 and described in Table 16-1205.

**Figure 16-1191. QUEUE_85_STATUS_B Register**

| 31-28 | 27-0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-1205. QUEUE_85_STATUS_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. Queue_Manager_Queue_n_Status_B Registers B |

### 16.4.7.916 QUEUE_85_STATUS_C Register (offset = 3558h) [reset = 0h]

QUEUE_85_STATUS_C is shown in Figure 16-1192 and described in Table 16-1206.

**Figure 16-1192. QUEUE_85_STATUS_C Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | PACKET_SIZE |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-1206. QUEUE_85_STATUS_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | PACKET_SIZE | R-0 | 0 | This field indicates packet size of the head element of a queue. Queue_Manager_Queue_N_Status_C Registers C |

### 16.4.7.917 QUEUE_86_STATUS_A Register (offset = 3560h) [reset = 0h]

QUEUE_86_STATUS_A is shown in Figure 16-1193 and described in Table 16-1207.

**Figure 16-1193. QUEUE_86_STATUS_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-1207. QUEUE_86_STATUS_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>Queue Manager Queue N Status Registers A |

### 16.4.7.918 QUEUE_86_STATUS_B Register (offset = 3564h) [reset = 0h]

QUEUE_86_STATUS_B is shown in Figure 16-1194 and described in Table 16-1208.

**Figure 16-1194. QUEUE_86_STATUS_B Register**

| 31–28 | 27–0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-1208. QUEUE_86_STATUS_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. Queue_Manager_Queue_n_Status_B Registers B |

### 16.4.7.919 QUEUE_86_STATUS_C Register (offset = 3568h) [reset = 0h]

QUEUE_86_STATUS_C is shown in Figure 16-1195 and described in Table 16-1209.

**Figure 16-1195. QUEUE_86_STATUS_C Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | PACKET_SIZE |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-1209. QUEUE_86_STATUS_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | PACKET_SIZE | R-0 | 0 | This field indicates packet size of the head element of a queue. Queue_Manager_Queue_N_Status_C Registers C |

### 16.4.7.920 QUEUE_87_STATUS_A Register (offset = 3570h) [reset = 0h]

QUEUE_87_STATUS_A is shown in Figure 16-1196 and described in Table 16-1210.

**Figure 16-1196. QUEUE_87_STATUS_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-1210. QUEUE_87_STATUS_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>Queue Manager Queue N Status Registers A |

### 16.4.7.921 QUEUE_87_STATUS_B Register (offset = 3574h) [reset = 0h]

QUEUE_87_STATUS_B is shown in Figure 16-1197 and described in Table 16-1211.

**Figure 16-1197. QUEUE_87_STATUS_B Register**

| 31-28 | 27-0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-1211. QUEUE_87_STATUS_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. Queue_Manager_Queue_n_Status_B Registers B |

### 16.4.7.922 QUEUE_87_STATUS_C Register (offset = 3578h) [reset = 0h]

QUEUE_87_STATUS_C is shown in Figure 16-1198 and described in Table 16-1212.

**Figure 16-1198. QUEUE_87_STATUS_C Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | PACKET_SIZE |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-1212. QUEUE_87_STATUS_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | PACKET_SIZE | R-0 | 0 | This field indicates packet size of the head element of a queue. Queue_Manager_Queue_N_Status_C Registers C |

### 16.4.7.923 QUEUE_88_STATUS_A Register (offset = 3580h) [reset = 0h]

QUEUE_88_STATUS_A is shown in Figure 16-1199 and described in Table 16-1213.

**Figure 16-1199. QUEUE_88_STATUS_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-1213. QUEUE_88_STATUS_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>Queue Manager Queue N Status Registers A |

### 16.4.7.924 QUEUE_88_STATUS_B Register (offset = 3584h) [reset = 0h]

QUEUE_88_STATUS_B is shown in Figure 16-1200 and described in Table 16-1214.

**Figure 16-1200. QUEUE_88_STATUS_B Register**

| 31–28 | 27–0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-1214. QUEUE_88_STATUS_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. Queue_Manager_Queue_n_Status_B Registers B |

### 16.4.7.925 QUEUE_88_STATUS_C Register (offset = 3588h) [reset = 0h]

QUEUE_88_STATUS_C is shown in Figure 16-1201 and described in Table 16-1215.

**Figure 16-1201. QUEUE_88_STATUS_C Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | PACKET_SIZE |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-1215. QUEUE_88_STATUS_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | PACKET_SIZE | R-0 | 0 | This field indicates packet size of the head element of a queue. Queue_Manager_Queue_N_Status_C Registers C |

### 16.4.7.926 QUEUE_89_STATUS_A Register (offset = 3590h) [reset = 0h]

QUEUE_89_STATUS_A is shown in Figure 16-1202 and described in Table 16-1216.

**Figure 16-1202. QUEUE_89_STATUS_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-1216. QUEUE_89_STATUS_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>Queue Manager Queue N Status Registers A |

### 16.4.7.927 QUEUE_89_STATUS_B Register (offset = 3594h) [reset = 0h]

QUEUE_89_STATUS_B is shown in Figure 16-1203 and described in Table 16-1217.

**Figure 16-1203. QUEUE_89_STATUS_B Register**

| 31-28 | 27-0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-1217. QUEUE_89_STATUS_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. Queue_Manager_Queue_n_Status_B Registers B |

### 16.4.7.928 QUEUE_89_STATUS_C Register (offset = 3598h) [reset = 0h]

QUEUE_89_STATUS_C is shown in Figure 16-1204 and described in Table 16-1218.

**Figure 16-1204. QUEUE_89_STATUS_C Register**

| 31–14 | 13–0 |
| --- | --- |
| Reserved | PACKET_SIZE |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-1218. QUEUE_89_STATUS_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
| --- | --- | --- | --- | --- |
| 13-0 | PACKET_SIZE | R-0 | 0 | This field indicates packet size of the head element of a queue.<br>Queue_Manager_Queue_N_Status_C Registers C |

### 16.4.7.929 QUEUE_90_STATUS_A Register (offset = 35A0h) [reset = 0h]

QUEUE_90_STATUS_A is shown in Figure 16-1205 and described in Table 16-1219.

**Figure 16-1205. QUEUE_90_STATUS_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-1219. QUEUE_90_STATUS_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>Queue Manager Queue N Status Registers A |

### 16.4.7.930 QUEUE_90_STATUS_B Register (offset = 35A4h) [reset = 0h]

QUEUE_90_STATUS_B is shown in Figure 16-1206 and described in Table 16-1220.

**Figure 16-1206. QUEUE_90_STATUS_B Register**

| 31–28 | 27–0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-1220. QUEUE_90_STATUS_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. Queue_Manager_Queue_n_Status_B Registers B |

### 16.4.7.931 QUEUE_90_STATUS_C Register (offset = 35A8h) [reset = 0h]

QUEUE_90_STATUS_C is shown in Figure 16-1207 and described in Table 16-1221.

**Figure 16-1207. QUEUE_90_STATUS_C Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | PACKET_SIZE |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-1221. QUEUE_90_STATUS_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | PACKET_SIZE | R-0 | 0 | This field indicates packet size of the head element of a queue. Queue_Manager_Queue_N_Status_C Registers C |

### 16.4.7.932 QUEUE_91_STATUS_A Register (offset = 35B0h) [reset = 0h]

QUEUE_91_STATUS_A is shown in Figure 16-1208 and described in Table 16-1222.

**Figure 16-1208. QUEUE_91_STATUS_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-1222. QUEUE_91_STATUS_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>Queue Manager Queue N Status Registers A |

### 16.4.7.933 QUEUE_91_STATUS_B Register (offset = 35B4h) [reset = 0h]

QUEUE_91_STATUS_B is shown in Figure 16-1209 and described in Table 16-1223.

**Figure 16-1209. QUEUE_91_STATUS_B Register**

| 31–28 | 27–0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-1223. QUEUE_91_STATUS_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue.<br>Queue_Manager_Queue_n_Status_B Registers B |

### 16.4.7.934 QUEUE_91_STATUS_C Register (offset = 35B8h) [reset = 0h]

QUEUE_91_STATUS_C is shown in Figure 16-1210 and described in Table 16-1224.

**Figure 16-1210. QUEUE_91_STATUS_C Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | PACKET_SIZE |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-1224. QUEUE_91_STATUS_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | PACKET_SIZE | R-0 | 0 | This field indicates packet size of the head element of a queue. Queue_Manager_Queue_N_Status_C Registers C |

### 16.4.7.935 QUEUE_92_STATUS_A Register (offset = 35C0h) [reset = 0h]

QUEUE_92_STATUS_A is shown in Figure 16-1211 and described in Table 16-1225.

**Figure 16-1211. QUEUE_92_STATUS_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-1225. QUEUE_92_STATUS_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>Queue Manager Queue N Status Registers A |

### 16.4.7.936 QUEUE_92_STATUS_B Register (offset = 35C4h) [reset = 0h]

QUEUE_92_STATUS_B is shown in Figure 16-1212 and described in Table 16-1226.

**Figure 16-1212. QUEUE_92_STATUS_B Register**

| 31–28 | 27–0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-1226. QUEUE_92_STATUS_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. Queue_Manager_Queue_n_Status_B Registers B |

### 16.4.7.937 QUEUE_92_STATUS_C Register (offset = 35C8h) [reset = 0h]

QUEUE_92_STATUS_C is shown in Figure 16-1213 and described in Table 16-1227.

**Figure 16-1213. QUEUE_92_STATUS_C Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | PACKET_SIZE |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-1227. QUEUE_92_STATUS_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | PACKET_SIZE | R-0 | 0 | This field indicates packet size of the head element of a queue. Queue_Manager_Queue_N_Status_C Registers C |

### 16.4.7.938 QUEUE_93_STATUS_A Register (offset = 35D0h) [reset = 0h]

QUEUE_93_STATUS_A is shown in Figure 16-1214 and described in Table 16-1228.

**Figure 16-1214. QUEUE_93_STATUS_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-1228. QUEUE_93_STATUS_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>Queue Manager Queue N Status Registers A |

### 16.4.7.939 QUEUE_93_STATUS_B Register (offset = 35D4h) [reset = 0h]

QUEUE_93_STATUS_B is shown in Figure 16-1215 and described in Table 16-1229.

**Figure 16-1215. QUEUE_93_STATUS_B Register**

| 31-28 | 27-0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-1229. QUEUE_93_STATUS_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. Queue_Manager_Queue_n_Status_B Registers B |

### 16.4.7.940 QUEUE_93_STATUS_C Register (offset = 35D8h) [reset = 0h]

QUEUE_93_STATUS_C is shown in Figure 16-1216 and described in Table 16-1230.

**Figure 16-1216. QUEUE_93_STATUS_C Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | PACKET_SIZE |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-1230. QUEUE_93_STATUS_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | PACKET_SIZE | R-0 | 0 | This field indicates packet size of the head element of a queue. Queue_Manager_Queue_N_Status_C Registers C |

### 16.4.7.941 QUEUE_94_STATUS_A Register (offset = 35E0h) [reset = 0h]

QUEUE_94_STATUS_A is shown in Figure 16-1217 and described in Table 16-1231.

**Figure 16-1217. QUEUE_94_STATUS_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-1231. QUEUE_94_STATUS_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>Queue Manager Queue N Status Registers A |

### 16.4.7.942 QUEUE_94_STATUS_B Register (offset = 35E4h) [reset = 0h]

QUEUE_94_STATUS_B is shown in Figure 16-1218 and described in Table 16-1232.

**Figure 16-1218. QUEUE_94_STATUS_B Register**

| 31–28 | 27–0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-1232. QUEUE_94_STATUS_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. Queue_Manager_Queue_n_Status_B Registers B |

### 16.4.7.943 QUEUE_94_STATUS_C Register (offset = 35E8h) [reset = 0h]

QUEUE_94_STATUS_C is shown in Figure 16-1219 and described in Table 16-1233.

**Figure 16-1219. QUEUE_94_STATUS_C Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | PACKET_SIZE |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-1233. QUEUE_94_STATUS_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | PACKET_SIZE | R-0 | 0 | This field indicates packet size of the head element of a queue. Queue_Manager_Queue_N_Status_C Registers C |

### 16.4.7.944 QUEUE_95_STATUS_A Register (offset = 35F0h) [reset = 0h]

QUEUE_95_STATUS_A is shown in Figure 16-1220 and described in Table 16-1234.

**Figure 16-1220. QUEUE_95_STATUS_A Register**

| 31–14 | 13–0 |
| --- | --- |
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-1234. QUEUE_95_STATUS_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
| --- | --- | --- | --- | --- |
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>Queue Manager Queue N Status Registers A |

### 16.4.7.945 QUEUE_95_STATUS_B Register (offset = 35F4h) [reset = 0h]

QUEUE_95_STATUS_B is shown in Figure 16-1221 and described in Table 16-1235.

**Figure 16-1221. QUEUE_95_STATUS_B Register**

| 31–28 | 27–0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-1235. QUEUE_95_STATUS_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. Queue_Manager_Queue_n_Status_B Registers B |

### 16.4.7.946 QUEUE_95_STATUS_C Register (offset = 35F8h) [reset = 0h]

QUEUE_95_STATUS_C is shown in Figure 16-1222 and described in Table 16-1236.

**Figure 16-1222. QUEUE_95_STATUS_C Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | PACKET_SIZE |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-1236. QUEUE_95_STATUS_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | PACKET_SIZE | R-0 | 0 | This field indicates packet size of the head element of a queue. Queue_Manager_Queue_N_Status_C Registers C |

### 16.4.7.947 QUEUE_96_STATUS_A Register (offset = 3600h) [reset = 0h]

QUEUE_96_STATUS_A is shown in Figure 16-1223 and described in Table 16-1237.

**Figure 16-1223. QUEUE_96_STATUS_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-1237. QUEUE_96_STATUS_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>Queue Manager Queue N Status Registers A |

### 16.4.7.948 QUEUE_96_STATUS_B Register (offset = 3604h) [reset = 0h]

QUEUE_96_STATUS_B is shown in Figure 16-1224 and described in Table 16-1238.

**Figure 16-1224. QUEUE_96_STATUS_B Register**

| 31–28 | 27–0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-1238. QUEUE_96_STATUS_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. Queue_Manager_Queue_n_Status_B Registers B |

### 16.4.7.949 QUEUE_96_STATUS_C Register (offset = 3608h) [reset = 0h]

QUEUE_96_STATUS_C is shown in Figure 16-1225 and described in Table 16-1239.

**Figure 16-1225. QUEUE_96_STATUS_C Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | PACKET_SIZE |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-1239. QUEUE_96_STATUS_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | PACKET_SIZE | R-0 | 0 | This field indicates packet size of the head element of a queue. Queue_Manager_Queue_N_Status_C Registers C |

### 16.4.7.950 QUEUE_97_STATUS_A Register (offset = 3610h) [reset = 0h]

QUEUE_97_STATUS_A is shown in Figure 16-1226 and described in Table 16-1240.

**Figure 16-1226. QUEUE_97_STATUS_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-1240. QUEUE_97_STATUS_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>Queue Manager Queue N Status Registers A |

### 16.4.7.951 QUEUE_97_STATUS_B Register (offset = 3614h) [reset = 0h]

QUEUE_97_STATUS_B is shown in Figure 16-1227 and described in Table 16-1241.

**Figure 16-1227. QUEUE_97_STATUS_B Register**

| 31-28 | 27-0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-1241. QUEUE_97_STATUS_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. Queue_Manager_Queue_n_Status_B Registers B |

### 16.4.7.952 QUEUE_97_STATUS_C Register (offset = 3618h) [reset = 0h]

QUEUE_97_STATUS_C is shown in Figure 16-1228 and described in Table 16-1242.

**Figure 16-1228. QUEUE_97_STATUS_C Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | PACKET_SIZE |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-1242. QUEUE_97_STATUS_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | PACKET_SIZE | R-0 | 0 | This field indicates packet size of the head element of a queue. Queue_Manager_Queue_N_Status_C Registers C |

### 16.4.7.953 QUEUE_98_STATUS_A Register (offset = 3620h) [reset = 0h]

QUEUE_98_STATUS_A is shown in Figure 16-1229 and described in Table 16-1243.

**Figure 16-1229. QUEUE_98_STATUS_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-1243. QUEUE_98_STATUS_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>Queue Manager Queue N Status Registers A |

### 16.4.7.954 QUEUE_98_STATUS_B Register (offset = 3624h) [reset = 0h]

QUEUE_98_STATUS_B is shown in Figure 16-1230 and described in Table 16-1244.

**Figure 16-1230. QUEUE_98_STATUS_B Register**

| 31–28 | 27–0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-1244. QUEUE_98_STATUS_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. Queue_Manager_Queue_n_Status_B Registers B |

### 16.4.7.955 QUEUE_98_STATUS_C Register (offset = 3628h) [reset = 0h]

QUEUE_98_STATUS_C is shown in Figure 16-1231 and described in Table 16-1245.

**Figure 16-1231. QUEUE_98_STATUS_C Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | PACKET_SIZE |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-1245. QUEUE_98_STATUS_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | PACKET_SIZE | R-0 | 0 | This field indicates packet size of the head element of a queue. Queue_Manager_Queue_N_Status_C Registers C |

### 16.4.7.956 QUEUE_99_STATUS_A Register (offset = 3630h) [reset = 0h]

QUEUE_99_STATUS_A is shown in Figure 16-1232 and described in Table 16-1246.

**Figure 16-1232. QUEUE_99_STATUS_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-1246. QUEUE_99_STATUS_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>Queue Manager Queue N Status Registers A |

### 16.4.7.957 QUEUE_99_STATUS_B Register (offset = 3634h) [reset = 0h]

QUEUE_99_STATUS_B is shown in Figure 16-1233 and described in Table 16-1247.

**Figure 16-1233. QUEUE_99_STATUS_B Register**

| 31–28 | 27–0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-1247. QUEUE_99_STATUS_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. Queue_Manager_Queue_n_Status_B Registers B |

### 16.4.7.958 QUEUE_99_STATUS_C Register (offset = 3638h) [reset = 0h]

QUEUE_99_STATUS_C is shown in Figure 16-1234 and described in Table 16-1248.

**Figure 16-1234. QUEUE_99_STATUS_C Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | PACKET_SIZE |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-1248. QUEUE_99_STATUS_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | PACKET_SIZE | R-0 | 0 | This field indicates packet size of the head element of a queue. Queue_Manager_Queue_N_Status_C Registers C |

### 16.4.7.959 QUEUE_100_STATUS_A Register (offset = 3640h) [reset = 0h]

QUEUE_100_STATUS_A is shown in Figure 16-1235 and described in Table 16-1249.

**Figure 16-1235. QUEUE_100_STATUS_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-1249. QUEUE_100_STATUS_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>Queue Manager Queue N Status Registers A |

### 16.4.7.960 QUEUE_100_STATUS_B Register (offset = 3644h) [reset = 0h]

QUEUE_100_STATUS_B is shown in Figure 16-1236 and described in Table 16-1250.

**Figure 16-1236. QUEUE_100_STATUS_B Register**

| 31–28 | 27–0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-1250. QUEUE_100_STATUS_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue.<br>Queue_Manager_Queue_n_Status_B Registers B |

### 16.4.7.961 QUEUE_100_STATUS_C Register (offset = 3648h) [reset = 0h]

QUEUE_100_STATUS_C is shown in Figure 16-1237 and described in Table 16-1251.

**Figure 16-1237. QUEUE_100_STATUS_C Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | PACKET_SIZE |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-1251. QUEUE_100_STATUS_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | PACKET_SIZE | R-0 | 0 | This field indicates packet size of the head element of a queue.<br>Queue_Manager_Queue_N_Status_C Registers C |

### 16.4.7.962 QUEUE_101_STATUS_A Register (offset = 3650h) [reset = 0h]

QUEUE_101_STATUS_A is shown in Figure 16-1238 and described in Table 16-1252.

**Figure 16-1238. QUEUE_101_STATUS_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-1252. QUEUE_101_STATUS_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>Queue Manager Queue N Status Registers A |

### 16.4.7.963 QUEUE_101_STATUS_B Register (offset = 3654h) [reset = 0h]

QUEUE_101_STATUS_B is shown in Figure 16-1239 and described in Table 16-1253.

**Figure 16-1239. QUEUE_101_STATUS_B Register**

| 31–28 | 27–0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-1253. QUEUE_101_STATUS_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. Queue_Manager_Queue_n_Status_B Registers B |

### 16.4.7.964 QUEUE_101_STATUS_C Register (offset = 3658h) [reset = 0h]

QUEUE_101_STATUS_C is shown in Figure 16-1240 and described in Table 16-1254.

**Figure 16-1240. QUEUE_101_STATUS_C Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | PACKET_SIZE |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-1254. QUEUE_101_STATUS_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | PACKET_SIZE | R-0 | 0 | This field indicates packet size of the head element of a queue. Queue_Manager_Queue_N_Status_C Registers C |

### 16.4.7.965 QUEUE_102_STATUS_A Register (offset = 3660h) [reset = 0h]

QUEUE_102_STATUS_A is shown in Figure 16-1241 and described in Table 16-1255.

**Figure 16-1241. QUEUE_102_STATUS_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-1255. QUEUE_102_STATUS_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>Queue Manager Queue N Status Registers A |

### 16.4.7.966 QUEUE_102_STATUS_B Register (offset = 3664h) [reset = 0h]

QUEUE_102_STATUS_B is shown in Figure 16-1242 and described in Table 16-1256.

**Figure 16-1242. QUEUE_102_STATUS_B Register**

| 31–28 | 27–0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-1256. QUEUE_102_STATUS_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. Queue_Manager_Queue_n_Status_B Registers B |

### 16.4.7.967 QUEUE_102_STATUS_C Register (offset = 3668h) [reset = 0h]

QUEUE_102_STATUS_C is shown in Figure 16-1243 and described in Table 16-1257.

**Figure 16-1243. QUEUE_102_STATUS_C Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | PACKET_SIZE |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-1257. QUEUE_102_STATUS_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | PACKET_SIZE | R-0 | 0 | This field indicates packet size of the head element of a queue. Queue_Manager_Queue_N_Status_C Registers C |

### 16.4.7.968 QUEUE_103_STATUS_A Register (offset = 3670h) [reset = 0h]

QUEUE_103_STATUS_A is shown in Figure 16-1244 and described in Table 16-1258.

**Figure 16-1244. QUEUE_103_STATUS_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-1258. QUEUE_103_STATUS_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>Queue Manager Queue N Status Registers A |

### 16.4.7.969 QUEUE_103_STATUS_B Register (offset = 3674h) [reset = 0h]

QUEUE_103_STATUS_B is shown in Figure 16-1245 and described in Table 16-1259.

**Figure 16-1245. QUEUE_103_STATUS_B Register**

| 31-28 | 27-0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-1259. QUEUE_103_STATUS_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. Queue_Manager_Queue_n_Status_B Registers B |

### 16.4.7.970 QUEUE_103_STATUS_C Register (offset = 3678h) [reset = 0h]

QUEUE_103_STATUS_C is shown in Figure 16-1246 and described in Table 16-1260.

**Figure 16-1246. QUEUE_103_STATUS_C Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | PACKET_SIZE |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-1260. QUEUE_103_STATUS_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | PACKET_SIZE | R-0 | 0 | This field indicates packet size of the head element of a queue. Queue_Manager_Queue_N_Status_C Registers C |

### 16.4.7.971 QUEUE_104_STATUS_A Register (offset = 3680h) [reset = 0h]

QUEUE_104_STATUS_A is shown in Figure 16-1247 and described in Table 16-1261.

**Figure 16-1247. QUEUE_104_STATUS_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-1261. QUEUE_104_STATUS_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>Queue Manager Queue N Status Registers A |

### 16.4.7.972 QUEUE_104_STATUS_B Register (offset = 3684h) [reset = 0h]

QUEUE_104_STATUS_B is shown in Figure 16-1248 and described in Table 16-1262.

**Figure 16-1248. QUEUE_104_STATUS_B Register**

| 31–28 | 27–0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-1262. QUEUE_104_STATUS_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. Queue_Manager_Queue_n_Status_B Registers B |

### 16.4.7.973 QUEUE_104_STATUS_C Register (offset = 3688h) [reset = 0h]

QUEUE_104_STATUS_C is shown in Figure 16-1249 and described in Table 16-1263.

**Figure 16-1249. QUEUE_104_STATUS_C Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | PACKET_SIZE |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-1263. QUEUE_104_STATUS_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | PACKET_SIZE | R-0 | 0 | This field indicates packet size of the head element of a queue. Queue_Manager_Queue_N_Status_C Registers C |

### 16.4.7.974 QUEUE_105_STATUS_A Register (offset = 3690h) [reset = 0h]

QUEUE_105_STATUS_A is shown in Figure 16-1250 and described in Table 16-1264.

**Figure 16-1250. QUEUE_105_STATUS_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-1264. QUEUE_105_STATUS_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>Queue Manager Queue N Status Registers A |

### 16.4.7.975 QUEUE_105_STATUS_B Register (offset = 3694h) [reset = 0h]

QUEUE_105_STATUS_B is shown in Figure 16-1251 and described in Table 16-1265.

**Figure 16-1251. QUEUE_105_STATUS_B Register**

| 31–28 | 27–0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-1265. QUEUE_105_STATUS_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. Queue_Manager_Queue_n_Status_B Registers B |

### 16.4.7.976 QUEUE_105_STATUS_C Register (offset = 3698h) [reset = 0h]

QUEUE_105_STATUS_C is shown in Figure 16-1252 and described in Table 16-1266.

**Figure 16-1252. QUEUE_105_STATUS_C Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | PACKET_SIZE |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-1266. QUEUE_105_STATUS_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | PACKET_SIZE | R-0 | 0 | This field indicates packet size of the head element of a queue. Queue_Manager_Queue_N_Status_C Registers C |

### 16.4.7.977 QUEUE_106_STATUS_A Register (offset = 36A0h) [reset = 0h]

QUEUE_106_STATUS_A is shown in Figure 16-1253 and described in Table 16-1267.

**Figure 16-1253. QUEUE_106_STATUS_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-1267. QUEUE_106_STATUS_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>Queue Manager Queue N Status Registers A |

### 16.4.7.978 QUEUE_106_STATUS_B Register (offset = 36A4h) [reset = 0h]

QUEUE_106_STATUS_B is shown in Figure 16-1254 and described in Table 16-1268.

**Figure 16-1254. QUEUE_106_STATUS_B Register**

| 31-28 | 27-0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-1268. QUEUE_106_STATUS_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. Queue_Manager_Queue_n_Status_B Registers B |

### 16.4.7.979 QUEUE_106_STATUS_C Register (offset = 36A8h) [reset = 0h]

QUEUE_106_STATUS_C is shown in Figure 16-1255 and described in Table 16-1269.

**Figure 16-1255. QUEUE_106_STATUS_C Register**

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 | 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 | 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 | 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| Reserved | | | | | | | | | | | | | | | | | | PACKET_SIZE | | | | | | | | | | | | | |
| | | | | | | | | | | | | | | | | | | R-0 | | | | | | | | | | | | | |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-1269. QUEUE_106_STATUS_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | PACKET_SIZE | R-0 | 0 | This field indicates packet size of the head element of a queue.<br>Queue_Manager_Queue_N_Status_C Registers C |

### 16.4.7.980 QUEUE_107_STATUS_A Register (offset = 36B0h) [reset = 0h]

QUEUE_107_STATUS_A is shown in Figure 16-1256 and described in Table 16-1270.

**Figure 16-1256. QUEUE_107_STATUS_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-1270. QUEUE_107_STATUS_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>Queue Manager Queue N Status Registers A |

### 16.4.7.981 QUEUE_107_STATUS_B Register (offset = 36B4h) [reset = 0h]

QUEUE_107_STATUS_B is shown in Figure 16-1257 and described in Table 16-1271.

**Figure 16-1257. QUEUE_107_STATUS_B Register**

| 31–28 | 27–0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-1271. QUEUE_107_STATUS_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. Queue_Manager_Queue_n_Status_B Registers B |

### 16.4.7.982 QUEUE_107_STATUS_C Register (offset = 36B8h) [reset = 0h]

QUEUE_107_STATUS_C is shown in Figure 16-1258 and described in Table 16-1272.

**Figure 16-1258. QUEUE_107_STATUS_C Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | PACKET_SIZE |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-1272. QUEUE_107_STATUS_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | PACKET_SIZE | R-0 | 0 | This field indicates packet size of the head element of a queue. Queue_Manager_Queue_N_Status_C Registers C |

### 16.4.7.983 QUEUE_108_STATUS_A Register (offset = 36C0h) [reset = 0h]

QUEUE_108_STATUS_A is shown in Figure 16-1259 and described in Table 16-1273.

**Figure 16-1259. QUEUE_108_STATUS_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-1273. QUEUE_108_STATUS_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>Queue Manager Queue N Status Registers A |

### 16.4.7.984 QUEUE_108_STATUS_B Register (offset = 36C4h) [reset = 0h]

QUEUE_108_STATUS_B is shown in Figure 16-1260 and described in Table 16-1274.

**Figure 16-1260. QUEUE_108_STATUS_B Register**

| 31–28 | 27–0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-1274. QUEUE_108_STATUS_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. Queue_Manager_Queue_n_Status_B Registers B |

### 16.4.7.985 QUEUE_108_STATUS_C Register (offset = 36C8h) [reset = 0h]

QUEUE_108_STATUS_C is shown in Figure 16-1261 and described in Table 16-1275.

**Figure 16-1261. QUEUE_108_STATUS_C Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | PACKET_SIZE |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-1275. QUEUE_108_STATUS_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | PACKET_SIZE | R-0 | 0 | This field indicates packet size of the head element of a queue. Queue_Manager_Queue_N_Status_C Registers C |

### 16.4.7.986 QUEUE_109_STATUS_A Register (offset = 36D0h) [reset = 0h]

QUEUE_109_STATUS_A is shown in Figure 16-1262 and described in Table 16-1276.

**Figure 16-1262. QUEUE_109_STATUS_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-1276. QUEUE_109_STATUS_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>Queue Manager Queue N Status Registers A |

### 16.4.7.987 QUEUE_109_STATUS_B Register (offset = 36D4h) [reset = 0h]

QUEUE_109_STATUS_B is shown in Figure 16-1263 and described in Table 16-1277.

**Figure 16-1263. QUEUE_109_STATUS_B Register**

| 31–28 | 27–0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-1277. QUEUE_109_STATUS_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. Queue_Manager_Queue_n_Status_B Registers B |

### 16.4.7.988 QUEUE_109_STATUS_C Register (offset = 36D8h) [reset = 0h]

QUEUE_109_STATUS_C is shown in Figure 16-1264 and described in Table 16-1278.

**Figure 16-1264. QUEUE_109_STATUS_C Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | PACKET_SIZE |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-1278. QUEUE_109_STATUS_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | PACKET_SIZE | R-0 | 0 | This field indicates packet size of the head element of a queue. Queue_Manager_Queue_N_Status_C Registers C |

### 16.4.7.989 QUEUE_110_STATUS_A Register (offset = 36E0h) [reset = 0h]

QUEUE_110_STATUS_A is shown in Figure 16-1265 and described in Table 16-1279.

**Figure 16-1265. QUEUE_110_STATUS_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-1279. QUEUE_110_STATUS_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>Queue Manager Queue N Status Registers A |

### 16.4.7.990 QUEUE_110_STATUS_B Register (offset = 36E4h) [reset = 0h]

QUEUE_110_STATUS_B is shown in Figure 16-1266 and described in Table 16-1280.

**Figure 16-1266. QUEUE_110_STATUS_B Register**

| 31–28 | 27–0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-1280. QUEUE_110_STATUS_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. Queue_Manager_Queue_n_Status_B Registers B |

### 16.4.7.991 QUEUE_110_STATUS_C Register (offset = 36E8h) [reset = 0h]

QUEUE_110_STATUS_C is shown in Figure 16-1267 and described in Table 16-1281.

**Figure 16-1267. QUEUE_110_STATUS_C Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | PACKET_SIZE |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-1281. QUEUE_110_STATUS_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | PACKET_SIZE | R-0 | 0 | This field indicates packet size of the head element of a queue. Queue_Manager_Queue_N_Status_C Registers C |

### 16.4.7.992 QUEUE_111_STATUS_A Register (offset = 36F0h) [reset = 0h]

QUEUE_111_STATUS_A is shown in Figure 16-1268 and described in Table 16-1282.

**Figure 16-1268. QUEUE_111_STATUS_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-1282. QUEUE_111_STATUS_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>Queue Manager Queue N Status Registers A |

### 16.4.7.993 QUEUE_111_STATUS_B Register (offset = 36F4h) [reset = 0h]

QUEUE_111_STATUS_B is shown in Figure 16-1269 and described in Table 16-1283.

**Figure 16-1269. QUEUE_111_STATUS_B Register**

| 31–28 | 27–0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-1283. QUEUE_111_STATUS_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. Queue_Manager_Queue_n_Status_B Registers B |

### 16.4.7.994 QUEUE_111_STATUS_C Register (offset = 36F8h) [reset = 0h]

QUEUE_111_STATUS_C is shown in Figure 16-1270 and described in Table 16-1284.

**Figure 16-1270. QUEUE_111_STATUS_C Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | PACKET_SIZE |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-1284. QUEUE_111_STATUS_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | PACKET_SIZE | R-0 | 0 | This field indicates packet size of the head element of a queue. Queue_Manager_Queue_N_Status_C Registers C |

### 16.4.7.995 QUEUE_112_STATUS_A Register (offset = 3700h) [reset = 0h]

QUEUE_112_STATUS_A is shown in Figure 16-1271 and described in Table 16-1285.

**Figure 16-1271. QUEUE_112_STATUS_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-1285. QUEUE_112_STATUS_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>Queue Manager Queue N Status Registers A |

### 16.4.7.996 QUEUE_112_STATUS_B Register (offset = 3704h) [reset = 0h]

QUEUE_112_STATUS_B is shown in Figure 16-1272 and described in Table 16-1286.

**Figure 16-1272. QUEUE_112_STATUS_B Register**

| 31–28 | 27–0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-1286. QUEUE_112_STATUS_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. Queue_Manager_Queue_n_Status_B Registers B |

### 16.4.7.997 QUEUE_112_STATUS_C Register (offset = 3708h) [reset = 0h]

QUEUE_112_STATUS_C is shown in Figure 16-1273 and described in Table 16-1287.

**Figure 16-1273. QUEUE_112_STATUS_C Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | PACKET_SIZE |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-1287. QUEUE_112_STATUS_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | PACKET_SIZE | R-0 | 0 | This field indicates packet size of the head element of a queue. Queue_Manager_Queue_N_Status_C Registers C |

### 16.4.7.998 QUEUE_113_STATUS_A Register (offset = 3710h) [reset = 0h]

QUEUE_113_STATUS_A is shown in Figure 16-1274 and described in Table 16-1288.

**Figure 16-1274. QUEUE_113_STATUS_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-1288. QUEUE_113_STATUS_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>Queue Manager Queue N Status Registers A |

### 16.4.7.999 QUEUE_113_STATUS_B Register (offset = 3714h) [reset = 0h]

QUEUE_113_STATUS_B is shown in Figure 16-1275 and described in Table 16-1289.

**Figure 16-1275. QUEUE_113_STATUS_B Register**

| 31–28 | 27–0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-1289. QUEUE_113_STATUS_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. Queue_Manager_Queue_n_Status_B Registers B |

### 16.4.7.1000 QUEUE_113_STATUS_C Register (offset = 3718h) [reset = 0h]

QUEUE_113_STATUS_C is shown in Figure 16-1276 and described in Table 16-1290.

**Figure 16-1276. QUEUE_113_STATUS_C Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | PACKET_SIZE |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-1290. QUEUE_113_STATUS_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | PACKET_SIZE | R-0 | 0 | This field indicates packet size of the head element of a queue. Queue_Manager_Queue_N_Status_C Registers C |

### 16.4.7.1001 QUEUE_114_STATUS_A Register (offset = 3720h) [reset = 0h]

QUEUE_114_STATUS_A is shown in Figure 16-1277 and described in Table 16-1291.

**Figure 16-1277. QUEUE_114_STATUS_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-1291. QUEUE_114_STATUS_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>Queue Manager Queue N Status Registers A |

### 16.4.7.1002 QUEUE_114_STATUS_B Register (offset = 3724h) [reset = 0h]

QUEUE_114_STATUS_B is shown in Figure 16-1278 and described in Table 16-1292.

**Figure 16-1278. QUEUE_114_STATUS_B Register**

| 31–28 | 27–0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-1292. QUEUE_114_STATUS_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. Queue_Manager_Queue_n_Status_B Registers B |

### 16.4.7.1003 QUEUE_114_STATUS_C Register (offset = 3728h) [reset = 0h]

QUEUE_114_STATUS_C is shown in Figure 16-1279 and described in Table 16-1293.

**Figure 16-1279. QUEUE_114_STATUS_C Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | PACKET_SIZE |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-1293. QUEUE_114_STATUS_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | PACKET_SIZE | R-0 | 0 | This field indicates packet size of the head element of a queue. Queue_Manager_Queue_N_Status_C Registers C |

### 16.4.7.1004 QUEUE_115_STATUS_A Register (offset = 3730h) [reset = 0h]

QUEUE_115_STATUS_A is shown in Figure 16-1280 and described in Table 16-1294.

**Figure 16-1280. QUEUE_115_STATUS_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-1294. QUEUE_115_STATUS_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>Queue Manager Queue N Status Registers A |

### 16.4.7.1005 QUEUE_115_STATUS_B Register (offset = 3734h) [reset = 0h]

QUEUE_115_STATUS_B is shown in Figure 16-1281 and described in Table 16-1295.

**Figure 16-1281. QUEUE_115_STATUS_B Register**

| 31–28 | 27–0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-1295. QUEUE_115_STATUS_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. Queue_Manager_Queue_n_Status_B Registers B |

### 16.4.7.1006 QUEUE_115_STATUS_C Register (offset = 3738h) [reset = 0h]

QUEUE_115_STATUS_C is shown in Figure 16-1282 and described in Table 16-1296.

**Figure 16-1282. QUEUE_115_STATUS_C Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | PACKET_SIZE |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-1296. QUEUE_115_STATUS_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | PACKET_SIZE | R-0 | 0 | This field indicates packet size of the head element of a queue. Queue_Manager_Queue_N_Status_C Registers C |

### 16.4.7.1007 QUEUE_116_STATUS_A Register (offset = 3740h) [reset = 0h]

QUEUE_116_STATUS_A is shown in Figure 16-1283 and described in Table 16-1297.

**Figure 16-1283. QUEUE_116_STATUS_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-1297. QUEUE_116_STATUS_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>Queue Manager Queue N Status Registers A |

### 16.4.7.1008 QUEUE_116_STATUS_B Register (offset = 3744h) [reset = 0h]

QUEUE_116_STATUS_B is shown in Figure 16-1284 and described in Table 16-1298.

**Figure 16-1284. QUEUE_116_STATUS_B Register**

| 31-28 | 27-0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-1298. QUEUE_116_STATUS_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. Queue_Manager_Queue_n_Status_B Registers B |

### 16.4.7.1009 QUEUE_116_STATUS_C Register (offset = 3748h) [reset = 0h]

QUEUE_116_STATUS_C is shown in Figure 16-1285 and described in Table 16-1299.

**Figure 16-1285. QUEUE_116_STATUS_C Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | PACKET_SIZE |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-1299. QUEUE_116_STATUS_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | PACKET_SIZE | R-0 | 0 | This field indicates packet size of the head element of a queue. Queue_Manager_Queue_N_Status_C Registers C |

### 16.4.7.1010 QUEUE_117_STATUS_A Register (offset = 3750h) [reset = 0h]

QUEUE_117_STATUS_A is shown in Figure 16-1286 and described in Table 16-1300.

**Figure 16-1286. QUEUE_117_STATUS_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-1300. QUEUE_117_STATUS_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>Queue Manager Queue N Status Registers A |

### 16.4.7.1011 QUEUE_117_STATUS_B Register (offset = 3754h) [reset = 0h]

QUEUE_117_STATUS_B is shown in Figure 16-1287 and described in Table 16-1301.

**Figure 16-1287. QUEUE_117_STATUS_B Register**

| 31–28 | 27–0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-1301. QUEUE_117_STATUS_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. Queue_Manager_Queue_n_Status_B Registers B |

### 16.4.7.1012 QUEUE_117_STATUS_C Register (offset = 3758h) [reset = 0h]

QUEUE_117_STATUS_C is shown in Figure 16-1288 and described in Table 16-1302.

**Figure 16-1288. QUEUE_117_STATUS_C Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | PACKET_SIZE |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-1302. QUEUE_117_STATUS_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | PACKET_SIZE | R-0 | 0 | This field indicates packet size of the head element of a queue. Queue_Manager_Queue_N_Status_C Registers C |

### 16.4.7.1013 QUEUE_118_STATUS_A Register (offset = 3760h) [reset = 0h]

QUEUE_118_STATUS_A is shown in Figure 16-1289 and described in Table 16-1303.

**Figure 16-1289. QUEUE_118_STATUS_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-1303. QUEUE_118_STATUS_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>Queue Manager Queue N Status Registers A |

### 16.4.7.1014 QUEUE_118_STATUS_B Register (offset = 3764h) [reset = 0h]

QUEUE_118_STATUS_B is shown in Figure 16-1290 and described in Table 16-1304.

**Figure 16-1290. QUEUE_118_STATUS_B Register**

| 31–28 | 27–0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-1304. QUEUE_118_STATUS_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. Queue_Manager_Queue_n_Status_B Registers B |

### 16.4.7.1015 QUEUE_118_STATUS_C Register (offset = 3768h) [reset = 0h]

QUEUE_118_STATUS_C is shown in Figure 16-1291 and described in Table 16-1305.

**Figure 16-1291. QUEUE_118_STATUS_C Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | PACKET_SIZE |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-1305. QUEUE_118_STATUS_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | PACKET_SIZE | R-0 | 0 | This field indicates packet size of the head element of a queue. Queue_Manager_Queue_N_Status_C Registers C |

### 16.4.7.1016 QUEUE_119_STATUS_A Register (offset = 3770h) [reset = 0h]

QUEUE_119_STATUS_A is shown in Figure 16-1292 and described in Table 16-1306.

**Figure 16-1292. QUEUE_119_STATUS_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-1306. QUEUE_119_STATUS_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>Queue Manager Queue N Status Registers A |

### 16.4.7.1017 QUEUE_119_STATUS_B Register (offset = 3774h) [reset = 0h]

QUEUE_119_STATUS_B is shown in Figure 16-1293 and described in Table 16-1307.

**Figure 16-1293. QUEUE_119_STATUS_B Register**

| 31–28 | 27–0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-1307. QUEUE_119_STATUS_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue.<br>Queue_Manager_Queue_n_Status_B Registers B |

### 16.4.7.1018 QUEUE_119_STATUS_C Register (offset = 3778h) [reset = 0h]

QUEUE_119_STATUS_C is shown in Figure 16-1294 and described in Table 16-1308.

**Figure 16-1294. QUEUE_119_STATUS_C Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | PACKET_SIZE |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-1308. QUEUE_119_STATUS_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | PACKET_SIZE | R-0 | 0 | This field indicates packet size of the head element of a queue. Queue_Manager_Queue_N_Status_C Registers C |

### 16.4.7.1019 QUEUE_120_STATUS_A Register (offset = 3780h) [reset = 0h]

QUEUE_120_STATUS_A is shown in Figure 16-1295 and described in Table 16-1309.

**Figure 16-1295. QUEUE_120_STATUS_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-1309. QUEUE_120_STATUS_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>Queue Manager Queue N Status Registers A |

### 16.4.7.1020 QUEUE_120_STATUS_B Register (offset = 3784h) [reset = 0h]

QUEUE_120_STATUS_B is shown in Figure 16-1296 and described in Table 16-1310.

**Figure 16-1296. QUEUE_120_STATUS_B Register**

| 31–28 | 27–0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-1310. QUEUE_120_STATUS_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. Queue_Manager_Queue_n_Status_B Registers B |

### 16.4.7.1021 QUEUE_120_STATUS_C Register (offset = 3788h) [reset = 0h]

QUEUE_120_STATUS_C is shown in Figure 16-1297 and described in Table 16-1311.

**Figure 16-1297. QUEUE_120_STATUS_C Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | PACKET_SIZE |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-1311. QUEUE_120_STATUS_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | PACKET_SIZE | R-0 | 0 | This field indicates packet size of the head element of a queue. Queue_Manager_Queue_N_Status_C Registers C |

### 16.4.7.1022 QUEUE_121_STATUS_A Register (offset = 3790h) [reset = 0h]

QUEUE_121_STATUS_A is shown in Figure 16-1298 and described in Table 16-1312.

**Figure 16-1298. QUEUE_121_STATUS_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-1312. QUEUE_121_STATUS_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>Queue Manager Queue N Status Registers A |

### 16.4.7.1023 QUEUE_121_STATUS_B Register (offset = 3794h) [reset = 0h]

QUEUE_121_STATUS_B is shown in Figure 16-1299 and described in Table 16-1313.

**Figure 16-1299. QUEUE_121_STATUS_B Register**

| 31–28 | 27–0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-1313. QUEUE_121_STATUS_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. Queue_Manager_Queue_n_Status_B Registers B |

### 16.4.7.1024 QUEUE_121_STATUS_C Register (offset = 3798h) [reset = 0h]

QUEUE_121_STATUS_C is shown in Figure 16-1300 and described in Table 16-1314.

**Figure 16-1300. QUEUE_121_STATUS_C Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | PACKET_SIZE |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-1314. QUEUE_121_STATUS_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | PACKET_SIZE | R-0 | 0 | This field indicates packet size of the head element of a queue. Queue_Manager_Queue_N_Status_C Registers C |

### 16.4.7.1025 QUEUE_122_STATUS_A Register (offset = 37A0h) [reset = 0h]

QUEUE_122_STATUS_A is shown in Figure 16-1301 and described in Table 16-1315.

**Figure 16-1301. QUEUE_122_STATUS_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-1315. QUEUE_122_STATUS_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>Queue Manager Queue N Status Registers A |

### 16.4.7.1026 QUEUE_122_STATUS_B Register (offset = 37A4h) [reset = 0h]

QUEUE_122_STATUS_B is shown in Figure 16-1302 and described in Table 16-1316.

**Figure 16-1302. QUEUE_122_STATUS_B Register**

| 31–28 | 27–0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-1316. QUEUE_122_STATUS_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. Queue_Manager_Queue_n_Status_B Registers B |

### 16.4.7.1027 QUEUE_122_STATUS_C Register (offset = 37A8h) [reset = 0h]

QUEUE_122_STATUS_C is shown in Figure 16-1303 and described in Table 16-1317.

**Figure 16-1303. QUEUE_122_STATUS_C Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | PACKET_SIZE |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-1317. QUEUE_122_STATUS_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | PACKET_SIZE | R-0 | 0 | This field indicates packet size of the head element of a queue. Queue_Manager_Queue_N_Status_C Registers C |

### 16.4.7.1028 QUEUE_123_STATUS_A Register (offset = 37B0h) [reset = 0h]

QUEUE_123_STATUS_A is shown in Figure 16-1304 and described in Table 16-1318.

**Figure 16-1304. QUEUE_123_STATUS_A Register**

| 31-14 | 13-0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-1318. QUEUE_123_STATUS_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>Queue Manager Queue N Status Registers A |

### 16.4.7.1029 QUEUE_123_STATUS_B Register (offset = 37B4h) [reset = 0h]

QUEUE_123_STATUS_B is shown in Figure 16-1305 and described in Table 16-1319.

**Figure 16-1305. QUEUE_123_STATUS_B Register**

| 31–28 | 27–0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-1319. QUEUE_123_STATUS_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. Queue_Manager_Queue_n_Status_B Registers B |

### 16.4.7.1030 QUEUE_123_STATUS_C Register (offset = 37B8h) [reset = 0h]

QUEUE_123_STATUS_C is shown in Figure 16-1306 and described in Table 16-1320.

**Figure 16-1306. QUEUE_123_STATUS_C Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | PACKET_SIZE |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-1320. QUEUE_123_STATUS_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | PACKET_SIZE | R-0 | 0 | This field indicates packet size of the head element of a queue. Queue_Manager_Queue_N_Status_C Registers C |

### 16.4.7.1031 QUEUE_124_STATUS_A Register (offset = 37C0h) [reset = 0h]

QUEUE_124_STATUS_A is shown in Figure 16-1307 and described in Table 16-1321.

**Figure 16-1307. QUEUE_124_STATUS_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-1321. QUEUE_124_STATUS_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>Queue Manager Queue N Status Registers A |

### 16.4.7.1032 QUEUE_124_STATUS_B Register (offset = 37C4h) [reset = 0h]

QUEUE_124_STATUS_B is shown in Figure 16-1308 and described in Table 16-1322.

**Figure 16-1308. QUEUE_124_STATUS_B Register**

| 31–28 | 27–0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-1322. QUEUE_124_STATUS_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. Queue_Manager_Queue_n_Status_B Registers B |

### 16.4.7.1033 QUEUE_124_STATUS_C Register (offset = 37C8h) [reset = 0h]

QUEUE_124_STATUS_C is shown in Figure 16-1309 and described in Table 16-1323.

**Figure 16-1309. QUEUE_124_STATUS_C Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | PACKET_SIZE |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-1323. QUEUE_124_STATUS_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | PACKET_SIZE | R-0 | 0 | This field indicates packet size of the head element of a queue. Queue_Manager_Queue_N_Status_C Registers C |

### 16.4.7.1034 QUEUE_125_STATUS_A Register (offset = 37D0h) [reset = 0h]

QUEUE_125_STATUS_A is shown in Figure 16-1310 and described in Table 16-1324.

**Figure 16-1310. QUEUE_125_STATUS_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-1324. QUEUE_125_STATUS_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>Queue Manager Queue N Status Registers A |

### 16.4.7.1035 QUEUE_125_STATUS_B Register (offset = 37D4h) [reset = 0h]

QUEUE_125_STATUS_B is shown in Figure 16-1311 and described in Table 16-1325.

**Figure 16-1311. QUEUE_125_STATUS_B Register**

| 31–28 | 27–0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-1325. QUEUE_125_STATUS_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. Queue_Manager_Queue_n_Status_B Registers B |

### 16.4.7.1036 QUEUE_125_STATUS_C Register (offset = 37D8h) [reset = 0h]

QUEUE_125_STATUS_C is shown in Figure 16-1312 and described in Table 16-1326.

**Figure 16-1312. QUEUE_125_STATUS_C Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | PACKET_SIZE |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-1326. QUEUE_125_STATUS_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | PACKET_SIZE | R-0 | 0 | This field indicates packet size of the head element of a queue. Queue_Manager_Queue_N_Status_C Registers C |

### 16.4.7.1037 QUEUE_126_STATUS_A Register (offset = 37E0h) [reset = 0h]

QUEUE_126_STATUS_A is shown in Figure 16-1313 and described in Table 16-1327.

**Figure 16-1313. QUEUE_126_STATUS_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-1327. QUEUE_126_STATUS_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>Queue Manager Queue N Status Registers A |

### 16.4.7.1038 QUEUE_126_STATUS_B Register (offset = 37E4h) [reset = 0h]

QUEUE_126_STATUS_B is shown in Figure 16-1314 and described in Table 16-1328.

**Figure 16-1314. QUEUE_126_STATUS_B Register**

| 31–28 | 27–0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-1328. QUEUE_126_STATUS_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. Queue_Manager_Queue_n_Status_B Registers B |

### 16.4.7.1039 QUEUE_126_STATUS_C Register (offset = 37E8h) [reset = 0h]

QUEUE_126_STATUS_C is shown in Figure 16-1315 and described in Table 16-1329.

**Figure 16-1315. QUEUE_126_STATUS_C Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | PACKET_SIZE |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-1329. QUEUE_126_STATUS_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | PACKET_SIZE | R-0 | 0 | This field indicates packet size of the head element of a queue. Queue_Manager_Queue_N_Status_C Registers C |

### 16.4.7.1040 QUEUE_127_STATUS_A Register (offset = 37F0h) [reset = 0h]

QUEUE_127_STATUS_A is shown in Figure 16-1316 and described in Table 16-1330.

**Figure 16-1316. QUEUE_127_STATUS_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-1330. QUEUE_127_STATUS_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>Queue Manager Queue N Status Registers A |

### 16.4.7.1041 QUEUE_127_STATUS_B Register (offset = 37F4h) [reset = 0h]

QUEUE_127_STATUS_B is shown in Figure 16-1317 and described in Table 16-1331.

**Figure 16-1317. QUEUE_127_STATUS_B Register**

| 31–28 | 27–0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-1331. QUEUE_127_STATUS_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue.<br>Queue_Manager_Queue_n_Status_B Registers B |

### 16.4.7.1042 QUEUE_127_STATUS_C Register (offset = 37F8h) [reset = 0h]

QUEUE_127_STATUS_C is shown in Figure 16-1318 and described in Table 16-1332.

**Figure 16-1318. QUEUE_127_STATUS_C Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | PACKET_SIZE |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-1332. QUEUE_127_STATUS_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | PACKET_SIZE | R-0 | 0 | This field indicates packet size of the head element of a queue. Queue_Manager_Queue_N_Status_C Registers C |

### 16.4.7.1043 QUEUE_128_STATUS_A Register (offset = 3800h) [reset = 0h]

QUEUE_128_STATUS_A is shown in Figure 16-1319 and described in Table 16-1333.

**Figure 16-1319. QUEUE_128_STATUS_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-1333. QUEUE_128_STATUS_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>Queue Manager Queue N Status Registers A |

### 16.4.7.1044 QUEUE_128_STATUS_B Register (offset = 3804h) [reset = 0h]

QUEUE_128_STATUS_B is shown in Figure 16-1320 and described in Table 16-1334.

**Figure 16-1320. QUEUE_128_STATUS_B Register**

| 31-28 | 27-0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-1334. QUEUE_128_STATUS_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. Queue_Manager_Queue_n_Status_B Registers B |

### 16.4.7.1045 QUEUE_128_STATUS_C Register (offset = 3808h) [reset = 0h]

QUEUE_128_STATUS_C is shown in Figure 16-1321 and described in Table 16-1335.

**Figure 16-1321. QUEUE_128_STATUS_C Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | PACKET_SIZE |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-1335. QUEUE_128_STATUS_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | PACKET_SIZE | R-0 | 0 | This field indicates packet size of the head element of a queue. Queue_Manager_Queue_N_Status_C Registers C |

### 16.4.7.1046 QUEUE_129_STATUS_A Register (offset = 3810h) [reset = 0h]

QUEUE_129_STATUS_A is shown in Figure 16-1322 and described in Table 16-1336.

**Figure 16-1322. QUEUE_129_STATUS_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-1336. QUEUE_129_STATUS_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>Queue Manager Queue N Status Registers A |

### 16.4.7.1047 QUEUE_129_STATUS_B Register (offset = 3814h) [reset = 0h]

QUEUE_129_STATUS_B is shown in Figure 16-1323 and described in Table 16-1337.

**Figure 16-1323. QUEUE_129_STATUS_B Register**

| 31–28 | 27–0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-1337. QUEUE_129_STATUS_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue.<br>Queue_Manager_Queue_n_Status_B Registers B |

### 16.4.7.1048 QUEUE_129_STATUS_C Register (offset = 3818h) [reset = 0h]

QUEUE_129_STATUS_C is shown in Figure 16-1324 and described in Table 16-1338.

**Figure 16-1324. QUEUE_129_STATUS_C Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | PACKET_SIZE |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-1338. QUEUE_129_STATUS_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | PACKET_SIZE | R-0 | 0 | This field indicates packet size of the head element of a queue. Queue_Manager_Queue_N_Status_C Registers C |

### 16.4.7.1049 QUEUE_130_STATUS_A Register (offset = 3820h) [reset = 0h]

QUEUE_130_STATUS_A is shown in Figure 16-1325 and described in Table 16-1339.

**Figure 16-1325. QUEUE_130_STATUS_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-1339. QUEUE_130_STATUS_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>Queue Manager Queue N Status Registers A |

### 16.4.7.1050 QUEUE_130_STATUS_B Register (offset = 3824h) [reset = 0h]

QUEUE_130_STATUS_B is shown in Figure 16-1326 and described in Table 16-1340.

**Figure 16-1326. QUEUE_130_STATUS_B Register**

| 31-28 | 27-0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-1340. QUEUE_130_STATUS_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. Queue_Manager_Queue_n_Status_B Registers B |

### 16.4.7.1051 QUEUE_130_STATUS_C Register (offset = 3828h) [reset = 0h]

QUEUE_130_STATUS_C is shown in Figure 16-1327 and described in Table 16-1341.

**Figure 16-1327. QUEUE_130_STATUS_C Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | PACKET_SIZE |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-1341. QUEUE_130_STATUS_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | PACKET_SIZE | R-0 | 0 | This field indicates packet size of the head element of a queue. Queue_Manager_Queue_N_Status_C Registers C |

### 16.4.7.1052 QUEUE_131_STATUS_A Register (offset = 3830h) [reset = 0h]

QUEUE_131_STATUS_A is shown in Figure 16-1328 and described in Table 16-1342.

**Figure 16-1328. QUEUE_131_STATUS_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-1342. QUEUE_131_STATUS_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>Queue Manager Queue N Status Registers A |

### 16.4.7.1053 QUEUE_131_STATUS_B Register (offset = 3834h) [reset = 0h]

QUEUE_131_STATUS_B is shown in Figure 16-1329 and described in Table 16-1343.

**Figure 16-1329. QUEUE_131_STATUS_B Register**

| 31–28 | 27–0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-1343. QUEUE_131_STATUS_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue.<br>Queue_Manager_Queue_n_Status_B Registers B |

### 16.4.7.1054 QUEUE_131_STATUS_C Register (offset = 3838h) [reset = 0h]

QUEUE_131_STATUS_C is shown in Figure 16-1330 and described in Table 16-1344.

**Figure 16-1330. QUEUE_131_STATUS_C Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | PACKET_SIZE |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-1344. QUEUE_131_STATUS_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | PACKET_SIZE | R-0 | 0 | This field indicates packet size of the head element of a queue. Queue_Manager_Queue_N_Status_C Registers C |

### 16.4.7.1055 QUEUE_132_STATUS_A Register (offset = 3840h) [reset = 0h]

QUEUE_132_STATUS_A is shown in Figure 16-1331 and described in Table 16-1345.

**Figure 16-1331. QUEUE_132_STATUS_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-1345. QUEUE_132_STATUS_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>Queue Manager Queue N Status Registers A |

### 16.4.7.1056 QUEUE_132_STATUS_B Register (offset = 3844h) [reset = 0h]

QUEUE_132_STATUS_B is shown in Figure 16-1332 and described in Table 16-1346.

**Figure 16-1332. QUEUE_132_STATUS_B Register**

| 31–28 | 27–0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-1346. QUEUE_132_STATUS_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue.<br>Queue_Manager_Queue_n_Status_B Registers B |

### 16.4.7.1057 QUEUE_132_STATUS_C Register (offset = 3848h) [reset = 0h]

QUEUE_132_STATUS_C is shown in Figure 16-1333 and described in Table 16-1347.

**Figure 16-1333. QUEUE_132_STATUS_C Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | PACKET_SIZE |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-1347. QUEUE_132_STATUS_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | PACKET_SIZE | R-0 | 0 | This field indicates packet size of the head element of a queue. Queue_Manager_Queue_N_Status_C Registers C |

### 16.4.7.1058 QUEUE_133_STATUS_A Register (offset = 3850h) [reset = 0h]

QUEUE_133_STATUS_A is shown in Figure 16-1334 and described in Table 16-1348.

**Figure 16-1334. QUEUE_133_STATUS_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-1348. QUEUE_133_STATUS_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>Queue Manager Queue N Status Registers A |

### 16.4.7.1059 QUEUE_133_STATUS_B Register (offset = 3854h) [reset = 0h]

QUEUE_133_STATUS_B is shown in Figure 16-1335 and described in Table 16-1349.

**Figure 16-1335. QUEUE_133_STATUS_B Register**

| 31-28 | 27-0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-1349. QUEUE_133_STATUS_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. Queue_Manager_Queue_n_Status_B Registers B |

### 16.4.7.1060 QUEUE_133_STATUS_C Register (offset = 3858h) [reset = 0h]

QUEUE_133_STATUS_C is shown in Figure 16-1336 and described in Table 16-1350.

**Figure 16-1336. QUEUE_133_STATUS_C Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | PACKET_SIZE |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-1350. QUEUE_133_STATUS_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | PACKET_SIZE | R-0 | 0 | This field indicates packet size of the head element of a queue. Queue_Manager_Queue_N_Status_C Registers C |

### 16.4.7.1061 QUEUE_134_STATUS_A Register (offset = 3860h) [reset = 0h]

QUEUE_134_STATUS_A is shown in Figure 16-1337 and described in Table 16-1351.

**Figure 16-1337. QUEUE_134_STATUS_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-1351. QUEUE_134_STATUS_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>Queue Manager Queue N Status Registers A |

### 16.4.7.1062 QUEUE_134_STATUS_B Register (offset = 3864h) [reset = 0h]

QUEUE_134_STATUS_B is shown in Figure 16-1338 and described in Table 16-1352.

**Figure 16-1338. QUEUE_134_STATUS_B Register**

| 31–28 | 27–0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-1352. QUEUE_134_STATUS_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. Queue_Manager_Queue_n_Status_B Registers B |

### 16.4.7.1063 QUEUE_134_STATUS_C Register (offset = 3868h) [reset = 0h]

QUEUE_134_STATUS_C is shown in Figure 16-1339 and described in Table 16-1353.

**Figure 16-1339. QUEUE_134_STATUS_C Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | PACKET_SIZE |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-1353. QUEUE_134_STATUS_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | PACKET_SIZE | R-0 | 0 | This field indicates packet size of the head element of a queue. Queue_Manager_Queue_N_Status_C Registers C |

### 16.4.7.1064 QUEUE_135_STATUS_A Register (offset = 3870h) [reset = 0h]

QUEUE_135_STATUS_A is shown in Figure 16-1340 and described in Table 16-1354.

**Figure 16-1340. QUEUE_135_STATUS_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-1354. QUEUE_135_STATUS_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>Queue Manager Queue N Status Registers A |

### 16.4.7.1065 QUEUE_135_STATUS_B Register (offset = 3874h) [reset = 0h]

QUEUE_135_STATUS_B is shown in Figure 16-1341 and described in Table 16-1355.

**Figure 16-1341. QUEUE_135_STATUS_B Register**

| 31–28 | 27–0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-1355. QUEUE_135_STATUS_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. Queue_Manager_Queue_n_Status_B Registers B |

### 16.4.7.1066 QUEUE_135_STATUS_C Register (offset = 3878h) [reset = 0h]

QUEUE_135_STATUS_C is shown in Figure 16-1342 and described in Table 16-1356.

**Figure 16-1342. QUEUE_135_STATUS_C Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | PACKET_SIZE |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-1356. QUEUE_135_STATUS_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | PACKET_SIZE | R-0 | 0 | This field indicates packet size of the head element of a queue. Queue_Manager_Queue_N_Status_C Registers C |

### 16.4.7.1067 QUEUE_136_STATUS_A Register (offset = 3880h) [reset = 0h]

QUEUE_136_STATUS_A is shown in Figure 16-1343 and described in Table 16-1357.

**Figure 16-1343. QUEUE_136_STATUS_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-1357. QUEUE_136_STATUS_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>Queue Manager Queue N Status Registers A |

### 16.4.7.1068 QUEUE_136_STATUS_B Register (offset = 3884h) [reset = 0h]

QUEUE_136_STATUS_B is shown in Figure 16-1344 and described in Table 16-1358.

**Figure 16-1344. QUEUE_136_STATUS_B Register**

| 31–28 | 27–0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-1358. QUEUE_136_STATUS_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. Queue_Manager_Queue_n_Status_B Registers B |

### 16.4.7.1069 QUEUE_136_STATUS_C Register (offset = 3888h) [reset = 0h]

QUEUE_136_STATUS_C is shown in Figure 16-1345 and described in Table 16-1359.

**Figure 16-1345. QUEUE_136_STATUS_C Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | PACKET_SIZE |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-1359. QUEUE_136_STATUS_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | PACKET_SIZE | R-0 | 0 | This field indicates packet size of the head element of a queue. Queue_Manager_Queue_N_Status_C Registers C |

### 16.4.7.1070 QUEUE_137_STATUS_A Register (offset = 3890h) [reset = 0h]

QUEUE_137_STATUS_A is shown in Figure 16-1346 and described in Table 16-1360.

**Figure 16-1346. QUEUE_137_STATUS_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-1360. QUEUE_137_STATUS_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>Queue Manager Queue N Status Registers A |

### 16.4.7.1071 QUEUE_137_STATUS_B Register (offset = 3894h) [reset = 0h]

QUEUE_137_STATUS_B is shown in Figure 16-1347 and described in Table 16-1361.

**Figure 16-1347. QUEUE_137_STATUS_B Register**

| 31-28 | 27-0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-1361. QUEUE_137_STATUS_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. Queue_Manager_Queue_n_Status_B Registers B |

### 16.4.7.1072 QUEUE_137_STATUS_C Register (offset = 3898h) [reset = 0h]

QUEUE_137_STATUS_C is shown in Figure 16-1348 and described in Table 16-1362.

**Figure 16-1348. QUEUE_137_STATUS_C Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | PACKET_SIZE |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-1362. QUEUE_137_STATUS_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | PACKET_SIZE | R-0 | 0 | This field indicates packet size of the head element of a queue. Queue_Manager_Queue_N_Status_C Registers C |

### 16.4.7.1073 QUEUE_138_STATUS_A Register (offset = 38A0h) [reset = 0h]

QUEUE_138_STATUS_A is shown in Figure 16-1349 and described in Table 16-1363.

**Figure 16-1349. QUEUE_138_STATUS_A Register**

| 31-14 | 13-0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-1363. QUEUE_138_STATUS_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>Queue Manager Queue N Status Registers A |

### 16.4.7.1074 QUEUE_138_STATUS_B Register (offset = 38A4h) [reset = 0h]

QUEUE_138_STATUS_B is shown in Figure 16-1350 and described in Table 16-1364.

**Figure 16-1350. QUEUE_138_STATUS_B Register**

| 31–28 | 27–0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-1364. QUEUE_138_STATUS_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. Queue_Manager_Queue_n_Status_B Registers B |

### 16.4.7.1075 QUEUE_138_STATUS_C Register (offset = 38A8h) [reset = 0h]

QUEUE_138_STATUS_C is shown in Figure 16-1351 and described in Table 16-1365.

**Figure 16-1351. QUEUE_138_STATUS_C Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | PACKET_SIZE |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-1365. QUEUE_138_STATUS_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | PACKET_SIZE | R-0 | 0 | This field indicates packet size of the head element of a queue. Queue_Manager_Queue_N_Status_C Registers C |

### 16.4.7.1076 QUEUE_139_STATUS_A Register (offset = 38B0h) [reset = 0h]

QUEUE_139_STATUS_A is shown in Figure 16-1352 and described in Table 16-1366.

**Figure 16-1352. QUEUE_139_STATUS_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-1366. QUEUE_139_STATUS_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>Queue Manager Queue N Status Registers A |

### 16.4.7.1077 QUEUE_139_STATUS_B Register (offset = 38B4h) [reset = 0h]

QUEUE_139_STATUS_B is shown in Figure 16-1353 and described in Table 16-1367.

**Figure 16-1353. QUEUE_139_STATUS_B Register**

| 31–28 | 27–0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-1367. QUEUE_139_STATUS_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue.<br>Queue_Manager_Queue_n_Status_B Registers B |

### 16.4.7.1078 QUEUE_139_STATUS_C Register (offset = 38B8h) [reset = 0h]

QUEUE_139_STATUS_C is shown in Figure 16-1354 and described in Table 16-1368.

**Figure 16-1354. QUEUE_139_STATUS_C Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | PACKET_SIZE |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-1368. QUEUE_139_STATUS_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | PACKET_SIZE | R-0 | 0 | This field indicates packet size of the head element of a queue. Queue_Manager_Queue_N_Status_C Registers C |

### 16.4.7.1079 QUEUE_140_STATUS_A Register (offset = 38C0h) [reset = 0h]

QUEUE_140_STATUS_A is shown in Figure 16-1355 and described in Table 16-1369.

**Figure 16-1355. QUEUE_140_STATUS_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-1369. QUEUE_140_STATUS_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>Queue Manager Queue N Status Registers A |

### 16.4.7.1080 QUEUE_140_STATUS_B Register (offset = 38C4h) [reset = 0h]

QUEUE_140_STATUS_B is shown in Figure 16-1356 and described in Table 16-1370.

**Figure 16-1356. QUEUE_140_STATUS_B Register**

| 31–28 | 27–0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-1370. QUEUE_140_STATUS_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. Queue_Manager_Queue_n_Status_B Registers B |

### 16.4.7.1081 QUEUE_140_STATUS_C Register (offset = 38C8h) [reset = 0h]

QUEUE_140_STATUS_C is shown in Figure 16-1357 and described in Table 16-1371.

**Figure 16-1357. QUEUE_140_STATUS_C Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | PACKET_SIZE |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-1371. QUEUE_140_STATUS_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | PACKET_SIZE | R-0 | 0 | This field indicates packet size of the head element of a queue. Queue_Manager_Queue_N_Status_C Registers C |

### 16.4.7.1082 QUEUE_141_STATUS_A Register (offset = 38D0h) [reset = 0h]

QUEUE_141_STATUS_A is shown in Figure 16-1358 and described in Table 16-1372.

**Figure 16-1358. QUEUE_141_STATUS_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-1372. QUEUE_141_STATUS_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>Queue Manager Queue N Status Registers A |

### 16.4.7.1083 QUEUE_141_STATUS_B Register (offset = 38D4h) [reset = 0h]

QUEUE_141_STATUS_B is shown in Figure 16-1359 and described in Table 16-1373.

**Figure 16-1359. QUEUE_141_STATUS_B Register**

| 31–28 | 27–0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-1373. QUEUE_141_STATUS_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. Queue_Manager_Queue_n_Status_B Registers B |

### 16.4.7.1084 QUEUE_141_STATUS_C Register (offset = 38D8h) [reset = 0h]

QUEUE_141_STATUS_C is shown in Figure 16-1360 and described in Table 16-1374.

**Figure 16-1360. QUEUE_141_STATUS_C Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | PACKET_SIZE |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-1374. QUEUE_141_STATUS_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | PACKET_SIZE | R-0 | 0 | This field indicates packet size of the head element of a queue. Queue_Manager_Queue_N_Status_C Registers C |

### 16.4.7.1085 QUEUE_142_STATUS_A Register (offset = 38E0h) [reset = 0h]

QUEUE_142_STATUS_A is shown in Figure 16-1361 and described in Table 16-1375.

**Figure 16-1361. QUEUE_142_STATUS_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-1375. QUEUE_142_STATUS_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>Queue Manager Queue N Status Registers A |

### 16.4.7.1086 QUEUE_142_STATUS_B Register (offset = 38E4h) [reset = 0h]

QUEUE_142_STATUS_B is shown in Figure 16-1362 and described in Table 16-1376.

**Figure 16-1362. QUEUE_142_STATUS_B Register**

| 31–28 | 27–0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-1376. QUEUE_142_STATUS_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. Queue_Manager_Queue_n_Status_B Registers B |

### 16.4.7.1087 QUEUE_142_STATUS_C Register (offset = 38E8h) [reset = 0h]

QUEUE_142_STATUS_C is shown in Figure 16-1363 and described in Table 16-1377.

**Figure 16-1363. QUEUE_142_STATUS_C Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | PACKET_SIZE |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-1377. QUEUE_142_STATUS_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | PACKET_SIZE | R-0 | 0 | This field indicates packet size of the head element of a queue. Queue_Manager_Queue_N_Status_C Registers C |

### 16.4.7.1088 QUEUE_143_STATUS_A Register (offset = 38F0h) [reset = 0h]

QUEUE_143_STATUS_A is shown in Figure 16-1364 and described in Table 16-1378.

**Figure 16-1364. QUEUE_143_STATUS_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-1378. QUEUE_143_STATUS_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>Queue Manager Queue N Status Registers A |

### 16.4.7.1089 QUEUE_143_STATUS_B Register (offset = 38F4h) [reset = 0h]

QUEUE_143_STATUS_B is shown in Figure 16-1365 and described in Table 16-1379.

**Figure 16-1365. QUEUE_143_STATUS_B Register**

| 31–28 | 27–0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-1379. QUEUE_143_STATUS_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. Queue_Manager_Queue_n_Status_B Registers B |

### 16.4.7.1090 QUEUE_143_STATUS_C Register (offset = 38F8h) [reset = 0h]

QUEUE_143_STATUS_C is shown in Figure 16-1366 and described in Table 16-1380.

**Figure 16-1366. QUEUE_143_STATUS_C Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | PACKET_SIZE |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-1380. QUEUE_143_STATUS_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | PACKET_SIZE | R-0 | 0 | This field indicates packet size of the head element of a queue. Queue_Manager_Queue_N_Status_C Registers C |

### 16.4.7.1091 QUEUE_144_STATUS_A Register (offset = 3900h) [reset = 0h]

QUEUE_144_STATUS_A is shown in Figure 16-1367 and described in Table 16-1381.

**Figure 16-1367. QUEUE_144_STATUS_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-1381. QUEUE_144_STATUS_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>Queue Manager Queue N Status Registers A |

### 16.4.7.1092 QUEUE_144_STATUS_B Register (offset = 3904h) [reset = 0h]

QUEUE_144_STATUS_B is shown in Figure 16-1368 and described in Table 16-1382.

**Figure 16-1368. QUEUE_144_STATUS_B Register**

| 31–28 | 27–0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-1382. QUEUE_144_STATUS_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. Queue_Manager_Queue_n_Status_B Registers B |

### 16.4.7.1093 QUEUE_144_STATUS_C Register (offset = 3908h) [reset = 0h]

QUEUE_144_STATUS_C is shown in Figure 16-1369 and described in Table 16-1383.

**Figure 16-1369. QUEUE_144_STATUS_C Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | PACKET_SIZE |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-1383. QUEUE_144_STATUS_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | PACKET_SIZE | R-0 | 0 | This field indicates packet size of the head element of a queue. Queue_Manager_Queue_N_Status_C Registers C |

### 16.4.7.1094 QUEUE_145_STATUS_A Register (offset = 3910h) [reset = 0h]

QUEUE_145_STATUS_A is shown in Figure 16-1370 and described in Table 16-1384.

**Figure 16-1370. QUEUE_145_STATUS_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-1384. QUEUE_145_STATUS_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>Queue Manager Queue N Status Registers A |

### 16.4.7.1095 QUEUE_145_STATUS_B Register (offset = 3914h) [reset = 0h]

QUEUE_145_STATUS_B is shown in Figure 16-1371 and described in Table 16-1385.

**Figure 16-1371. QUEUE_145_STATUS_B Register**

| 31–28 | 27–0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-1385. QUEUE_145_STATUS_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. Queue_Manager_Queue_n_Status_B Registers B |

### 16.4.7.1096 QUEUE_145_STATUS_C Register (offset = 3918h) [reset = 0h]

QUEUE_145_STATUS_C is shown in Figure 16-1372 and described in Table 16-1386.

**Figure 16-1372. QUEUE_145_STATUS_C Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | PACKET_SIZE |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-1386. QUEUE_145_STATUS_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | PACKET_SIZE | R-0 | 0 | This field indicates packet size of the head element of a queue. Queue_Manager_Queue_N_Status_C Registers C |

### 16.4.7.1097 QUEUE_146_STATUS_A Register (offset = 3920h) [reset = 0h]

QUEUE_146_STATUS_A is shown in Figure 16-1373 and described in Table 16-1387.

**Figure 16-1373. QUEUE_146_STATUS_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-1387. QUEUE_146_STATUS_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>Queue Manager Queue N Status Registers A |

### 16.4.7.1098 QUEUE_146_STATUS_B Register (offset = 3924h) [reset = 0h]

QUEUE_146_STATUS_B is shown in Figure 16-1374 and described in Table 16-1388.

**Figure 16-1374. QUEUE_146_STATUS_B Register**

| 31–28 | 27–0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-1388. QUEUE_146_STATUS_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. Queue_Manager_Queue_n_Status_B Registers B |

### 16.4.7.1099 QUEUE_146_STATUS_C Register (offset = 3928h) [reset = 0h]

QUEUE_146_STATUS_C is shown in Figure 16-1375 and described in Table 16-1389.

**Figure 16-1375. QUEUE_146_STATUS_C Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | PACKET_SIZE |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-1389. QUEUE_146_STATUS_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | PACKET_SIZE | R-0 | 0 | This field indicates packet size of the head element of a queue. Queue_Manager_Queue_N_Status_C Registers C |

### 16.4.7.1100 QUEUE_147_STATUS_A Register (offset = 3930h) [reset = 0h]

QUEUE_147_STATUS_A is shown in Figure 16-1376 and described in Table 16-1390.

**Figure 16-1376. QUEUE_147_STATUS_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-1390. QUEUE_147_STATUS_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>Queue Manager Queue N Status Registers A |

### 16.4.7.1101 QUEUE_147_STATUS_B Register (offset = 3934h) [reset = 0h]

QUEUE_147_STATUS_B is shown in Figure 16-1377 and described in Table 16-1391.

**Figure 16-1377. QUEUE_147_STATUS_B Register**

| 31–28 | 27–0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-1391. QUEUE_147_STATUS_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. Queue_Manager_Queue_n_Status_B Registers B |

### 16.4.7.1102 QUEUE_147_STATUS_C Register (offset = 3938h) [reset = 0h]

QUEUE_147_STATUS_C is shown in Figure 16-1378 and described in Table 16-1392.

**Figure 16-1378. QUEUE_147_STATUS_C Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | PACKET_SIZE |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-1392. QUEUE_147_STATUS_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | PACKET_SIZE | R-0 | 0 | This field indicates packet size of the head element of a queue. Queue_Manager_Queue_N_Status_C Registers C |

### 16.4.7.1103 QUEUE_148_STATUS_A Register (offset = 3940h) [reset = 0h]

QUEUE_148_STATUS_A is shown in Figure 16-1379 and described in Table 16-1393.

**Figure 16-1379. QUEUE_148_STATUS_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-1393. QUEUE_148_STATUS_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>Queue Manager Queue N Status Registers A |

### 16.4.7.1104 QUEUE_148_STATUS_B Register (offset = 3944h) [reset = 0h]

QUEUE_148_STATUS_B is shown in Figure 16-1380 and described in Table 16-1394.

**Figure 16-1380. QUEUE_148_STATUS_B Register**

| 31–28 | 27–0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-1394. QUEUE_148_STATUS_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. Queue_Manager_Queue_n_Status_B Registers B |

### 16.4.7.1105 QUEUE_148_STATUS_C Register (offset = 3948h) [reset = 0h]

QUEUE_148_STATUS_C is shown in Figure 16-1381 and described in Table 16-1395.

**Figure 16-1381. QUEUE_148_STATUS_C Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | PACKET_SIZE |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-1395. QUEUE_148_STATUS_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | PACKET_SIZE | R-0 | 0 | This field indicates packet size of the head element of a queue. Queue_Manager_Queue_N_Status_C Registers C |

### 16.4.7.1106 QUEUE_149_STATUS_A Register (offset = 3950h) [reset = 0h]

QUEUE_149_STATUS_A is shown in Figure 16-1382 and described in Table 16-1396.

**Figure 16-1382. QUEUE_149_STATUS_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-1396. QUEUE_149_STATUS_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>Queue Manager Queue N Status Registers A |

### 16.4.7.1107 QUEUE_149_STATUS_B Register (offset = 3954h) [reset = 0h]

QUEUE_149_STATUS_B is shown in Figure 16-1383 and described in Table 16-1397.

**Figure 16-1383. QUEUE_149_STATUS_B Register**

| 31–28 | 27–0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-1397. QUEUE_149_STATUS_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. Queue_Manager_Queue_n_Status_B Registers B |

### 16.4.7.1108 QUEUE_149_STATUS_C Register (offset = 3958h) [reset = 0h]

QUEUE_149_STATUS_C is shown in Figure 16-1384 and described in Table 16-1398.

**Figure 16-1384. QUEUE_149_STATUS_C Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | PACKET_SIZE |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-1398. QUEUE_149_STATUS_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | PACKET_SIZE | R-0 | 0 | This field indicates packet size of the head element of a queue. Queue_Manager_Queue_N_Status_C Registers C |

### 16.4.7.1109 QUEUE_150_STATUS_A Register (offset = 3960h) [reset = 0h]

QUEUE_150_STATUS_A is shown in Figure 16-1385 and described in Table 16-1399.

**Figure 16-1385. QUEUE_150_STATUS_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-1399. QUEUE_150_STATUS_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>Queue Manager Queue N Status Registers A |

### 16.4.7.1110 QUEUE_150_STATUS_B Register (offset = 3964h) [reset = 0h]

QUEUE_150_STATUS_B is shown in Figure 16-1386 and described in Table 16-1400.

**Figure 16-1386. QUEUE_150_STATUS_B Register**

| 31–28 | 27–0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-1400. QUEUE_150_STATUS_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. Queue_Manager_Queue_n_Status_B Registers B |

### 16.4.7.1111 QUEUE_150_STATUS_C Register (offset = 3968h) [reset = 0h]

QUEUE_150_STATUS_C is shown in Figure 16-1387 and described in Table 16-1401.

**Figure 16-1387. QUEUE_150_STATUS_C Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | PACKET_SIZE |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-1401. QUEUE_150_STATUS_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | PACKET_SIZE | R-0 | 0 | This field indicates packet size of the head element of a queue. Queue_Manager_Queue_N_Status_C Registers C |

### 16.4.7.1112 QUEUE_151_STATUS_A Register (offset = 3970h) [reset = 0h]

QUEUE_151_STATUS_A is shown in Figure 16-1388 and described in Table 16-1402.

**Figure 16-1388. QUEUE_151_STATUS_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-1402. QUEUE_151_STATUS_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>Queue Manager Queue N Status Registers A |

### 16.4.7.1113 QUEUE_151_STATUS_B Register (offset = 3974h) [reset = 0h]

QUEUE_151_STATUS_B is shown in Figure 16-1389 and described in Table 16-1403.

**Figure 16-1389. QUEUE_151_STATUS_B Register**

| 31–28 | 27–0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-1403. QUEUE_151_STATUS_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. Queue_Manager_Queue_n_Status_B Registers B |

### 16.4.7.1114 QUEUE_151_STATUS_C Register (offset = 3978h) [reset = 0h]

QUEUE_151_STATUS_C is shown in Figure 16-1390 and described in Table 16-1404.

**Figure 16-1390. QUEUE_151_STATUS_C Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | PACKET_SIZE |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-1404. QUEUE_151_STATUS_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | PACKET_SIZE | R-0 | 0 | This field indicates packet size of the head element of a queue. Queue_Manager_Queue_N_Status_C Registers C |

### 16.4.7.1115 QUEUE_152_STATUS_A Register (offset = 3980h) [reset = 0h]

QUEUE_152_STATUS_A is shown in Figure 16-1391 and described in Table 16-1405.

**Figure 16-1391. QUEUE_152_STATUS_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-1405. QUEUE_152_STATUS_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>Queue Manager Queue N Status Registers A |

### 16.4.7.1116 QUEUE_152_STATUS_B Register (offset = 3984h) [reset = 0h]

QUEUE_152_STATUS_B is shown in Figure 16-1392 and described in Table 16-1406.

**Figure 16-1392. QUEUE_152_STATUS_B Register**

| 31-28 | 27-0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-1406. QUEUE_152_STATUS_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. Queue_Manager_Queue_n_Status_B Registers B |

### 16.4.7.1117 QUEUE_152_STATUS_C Register (offset = 3988h) [reset = 0h]

QUEUE_152_STATUS_C is shown in Figure 16-1393 and described in Table 16-1407.

**Figure 16-1393. QUEUE_152_STATUS_C Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | PACKET_SIZE |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-1407. QUEUE_152_STATUS_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | PACKET_SIZE | R-0 | 0 | This field indicates packet size of the head element of a queue. Queue_Manager_Queue_N_Status_C Registers C |

### 16.4.7.1118 QUEUE_153_STATUS_A Register (offset = 3990h) [reset = 0h]

QUEUE_153_STATUS_A is shown in Figure 16-1394 and described in Table 16-1408.

**Figure 16-1394. QUEUE_153_STATUS_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-1408. QUEUE_153_STATUS_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>Queue Manager Queue N Status Registers A |

### 16.4.7.1119 QUEUE_153_STATUS_B Register (offset = 3994h) [reset = 0h]

QUEUE_153_STATUS_B is shown in Figure 16-1395 and described in Table 16-1409.

**Figure 16-1395. QUEUE_153_STATUS_B Register**

| 31–28 | 27–0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-1409. QUEUE_153_STATUS_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. Queue_Manager_Queue_n_Status_B Registers B |

### 16.4.7.1120 QUEUE_153_STATUS_C Register (offset = 3998h) [reset = 0h]

QUEUE_153_STATUS_C is shown in Figure 16-1396 and described in Table 16-1410.

**Figure 16-1396. QUEUE_153_STATUS_C Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | PACKET_SIZE |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-1410. QUEUE_153_STATUS_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | PACKET_SIZE | R-0 | 0 | This field indicates packet size of the head element of a queue. Queue_Manager_Queue_N_Status_C Registers C |

### 16.4.7.1121 QUEUE_154_STATUS_A Register (offset = 39A0h) [reset = 0h]

QUEUE_154_STATUS_A is shown in Figure 16-1397 and described in Table 16-1411.

**Figure 16-1397. QUEUE_154_STATUS_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-1411. QUEUE_154_STATUS_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>Queue Manager Queue N Status Registers A |

### 16.4.7.1122 QUEUE_154_STATUS_B Register (offset = 39A4h) [reset = 0h]

QUEUE_154_STATUS_B is shown in Figure 16-1398 and described in Table 16-1412.

**Figure 16-1398. QUEUE_154_STATUS_B Register**

| 31–28 | 27–0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-1412. QUEUE_154_STATUS_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue.<br>Queue_Manager_Queue_n_Status_B Registers B |

### 16.4.7.1123 QUEUE_154_STATUS_C Register (offset = 39A8h) [reset = 0h]

QUEUE_154_STATUS_C is shown in Figure 16-1399 and described in Table 16-1413.

**Figure 16-1399. QUEUE_154_STATUS_C Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | PACKET_SIZE |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-1413. QUEUE_154_STATUS_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | PACKET_SIZE | R-0 | 0 | This field indicates packet size of the head element of a queue.<br>Queue_Manager_Queue_N_Status_C Registers C |

### 16.4.7.1124 QUEUE_155_STATUS_A Register (offset = 39B0h) [reset = 0h]

QUEUE_155_STATUS_A is shown in Figure 16-1400 and described in Table 16-1414.

**Figure 16-1400. QUEUE_155_STATUS_A Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | QUEUE_ENTRY_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -*n* = value after reset

**Table 16-1414. QUEUE_155_STATUS_A Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | QUEUE_ENTRY_COUNT | R-0 | 0 | This field indicates how many packets are currently queued on the queue.<br>Queue Manager Queue N Status Registers A |

### 16.4.7.1125 QUEUE_155_STATUS_B Register (offset = 39B4h) [reset = 0h]

QUEUE_155_STATUS_B is shown in Figure 16-1401 and described in Table 16-1415.

**Figure 16-1401. QUEUE_155_STATUS_B Register**

| 31–28 | 27–0 |
|---|---|
| Reserved | QUEUE_BYTE_COUNT |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-1415. QUEUE_155_STATUS_B Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 27-0 | QUEUE_BYTE_COUNT | R-0 | 0 | This field indicates how many bytes total are contained in all of the packets which are currently queued on this queue. Queue_Manager_Queue_n_Status_B Registers B |

#### 16.4.7.1126 QUEUE_155_STATUS_C Register (offset = 39B8h) [reset = 0h]

QUEUE_155_STATUS_C is shown in Figure 16-1402 and described in Table 16-1416.

**Figure 16-1402. QUEUE_155_STATUS_C Register**

| 31–14 | 13–0 |
|---|---|
| Reserved | PACKET_SIZE |
| | R-0 |

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; *-n* = value after reset

**Table 16-1416. QUEUE_155_STATUS_C Register Field Descriptions**

| Bit | Field | Type | Reset | Description |
|---|---|---|---|---|
| 13-0 | PACKET_SIZE | R-0 | 0 | This field indicates packet size of the head element of a queue. Queue_Manager_Queue_N_Status_C Registers C |